The PyGMT Documentation

Release 0.20.0.dev184+g832c69a1f

The PyGMT Developers

Oct 22, 2026

GETTING STARTED

1 Installing **1**
- 1.1 Quickstart . . . 1
- 1.2 Which Python? . . . 2
- 1.3 Which GMT? . . . 2
- 1.4 Dependencies . . . 2
- 1.5 Installing GMT and other dependencies . . . 2
- 1.6 Installing PyGMT . . . 3
- 1.7 Testing your install . . . 4
- 1.8 Common installation issues . . . 5

2 Intro to PyGMT **7**
- 2.1 1. Making your first figure . . . 7
- 2.2 2. Create a contour map . . . 13
- 2.3 3. Figure elements . . . 18
- 2.4 4. PyGMT I/O: Table inputs . . . 20

3 Overview **25**
- 3.1 Why PyGMT . . . 25
- 3.2 Background . . . 25
- 3.3 Project goals . . . 25
- 3.4 Community . . . 26
- 3.5 Related projects . . . 26
- 3.6 Funding . . . 26
- 3.7 License . . . 26

4 PyGMT Ecosystem **27**
- 4.1 PyGMT dependencies . . . 27
- 4.2 Packages depending on PyGMT . . . 28

5 Presentations **31**

6 Citing PyGMT **37**

7 Tutorials **39**
- 7.1 Basics . . . 39
- 7.2 Advanced . . . 85

8 Gallery **179**
- 8.1 Maps and map elements . . . 179
- 8.2 Lines and vectors . . . 188
- 8.3 Symbols and markers . . . 220

8.4 Images, contours, and fields . . . 240
8.5 3-D Plots . . . 258
8.6 Seismology and geodesy . . . 264
8.7 Base maps . . . 267
8.8 Histograms . . . 271
8.9 Plot embellishments . . . 279

9 Projections **301**
9.1 Azimuthal Projections . . . 301
9.2 Conic Projections . . . 309
9.3 Cylindric Projections . . . 313
9.4 Miscellaneous Projections . . . 325
9.5 Non-geographic Projections . . . 334

10 External Resources **343**
10.1 Tutorials . . . 343
10.2 Examples from Publications and Posters . . . 352

11 API Reference **357**
11.1 Plotting . . . 357
11.2 Data Processing . . . 473
11.3 GMT Defaults . . . 566
11.4 Metadata . . . 569
11.5 xarray Integration . . . 572
11.6 Class-style Parameters . . . 604
11.7 Enums . . . 612
11.8 Miscellaneous . . . 612
11.9 Datasets . . . 614
11.10 Exceptions . . . 646
11.11 GMT C API . . . 649

12 Technical Reference **673**
12.1 Common Parameters . . . 673
12.2 GMT Map Projections . . . 673
12.3 Dimension and Distance Units . . . 674
12.4 Supported Fonts . . . 675
12.5 Text Formatting . . . 676
12.6 Supported Encodings and Non-ASCII Characters . . . 677
12.7 Justification Codes . . . 681
12.8 Environment Variables . . . 682
12.9 Supported Array Dtypes . . . 683

13 Minimum Supported Versions **687**

14 Changelog **689**
14.1 Release v0.19.0 (2026/07/07) . . . 689
14.2 Release v0.18.0 (2026/01/12) . . . 691
14.3 Release v0.17.0 (2025/10/03) . . . 693
14.4 Release v0.16.0 (2025/06/18) . . . 695
14.5 Release v0.15.0 (2025/03/31) . . . 696
14.6 Release v0.14.2 (2025/02/15) . . . 697
14.7 Release v0.14.1 (2025/02/01) . . . 698
14.8 Release v0.14.0 (2024/12/31) . . . 698
14.9 Release v0.13.0 (2024/09/05) . . . 701
14.10 Release v0.12.0 (2024/05/01) . . . 703

14.11 Release v0.11.0 (2024/02/01) . . . 705
14.12 Release v0.10.0 (2023/09/02) . . . 708
14.13 Release v0.9.0 (2023/03/31) . . . 710
14.14 Release v0.8.0 (2022/12/30) . . . 712
14.15 Release v0.7.0 (2022/07/01) . . . 714
14.16 Release v0.6.1 (2022/04/11) . . . 716
14.17 Release v0.6.0 (2022/03/14) . . . 716
14.18 Release v0.5.0 (2021/10/29) . . . 718
14.19 Release v0.4.1 (2021/08/07) . . . 721
14.20 Release v0.4.0 (2021/06/20) . . . 722
14.21 Release v0.3.1 (2021/03/14) . . . 724
14.22 Release v0.3.0 (2021/02/15) . . . 726
14.23 Release v0.2.1 (2020/11/14) . . . 728
14.24 Release v0.2.0 (2020/09/12) . . . 730
14.25 Release v0.1.2 (2020/07/07) . . . 731
14.26 Release v0.1.1 (2020/05/22) . . . 732
14.27 Release v0.1.0 (2020/05/03) . . . 733

15 Contributors Guide **737**
15.1 Ways to Contribute . . . 737
15.2 Providing Feedback . . . 737
15.3 General Guidelines . . . 738
15.4 Setting up your Environment . . . 740
15.5 Contributing Documentation . . . 741
15.6 Contributing Code . . . 744

16 Maintainers Guide **749**
16.1 Onboarding/Offboarding Access Checklist . . . 749
16.2 Branches . . . 750
16.3 Managing GitHub Issues . . . 750
16.4 Reviewing and Merging Pull Requests . . . 750
16.5 Continuous Integration . . . 750
16.6 Continuous Documentation . . . 751
16.7 Continuous Benchmarking . . . 751
16.8 Dependencies Policy . . . 751
16.9 Backwards Compatibility and Deprecation Policy . . . 751
16.10 Making a Release . . . 752

17 PyGMT Team **755**
17.1 Founders . . . 755
17.2 Active Maintainers . . . 755
17.3 Distinguished Contributors . . . 755

CHAPTER

ONE

INSTALLING

Note

The *Quickstart* section just below provides the fastest way to install PyGMT, whereas the following sections afterward provide more detailed, step-by-step instructions for users who may have different setups or want to install the latest development version.

1.1 Quickstart

The fastest way to install PyGMT is with the conda package manager, which takes care of setting up a virtual environment, as well as the installation of GMT and all the dependencies PyGMT depends on:

```
conda create --name pygmt --channel conda-forge pygmt
```

To activate the virtual environment, you can do:

```
conda activate pygmt
```

After this, check that everything works by running the following in a Python interpreter (e.g., in a Jupyter notebook):

```
import pygmt
pygmt.show_versions()
```

```
PyGMT information:
  version: 0.20.0.dev184+g832c69a1f
System information:
  python: 3.15.0 | packaged by conda-forge | (main, Oct 12 2026, 17:36:43) [GCC␣
↪15.3.0]
  executable: /home/runner/micromamba/envs/pygmt/bin/python
  machine: Linux-7.0.0-1017-azure-x86_64-with-glibc2.39
Dependency information:
  numpy: 2.5.4
  pandas: 3.0.6
  xarray: 2026.9.0
  packaging: 26.3
  contextily: 1.7.1
  geopandas: 1.2.0
  IPython: 9.17.1
  pyarrow: 25.0.0
  rioxarray: 0.23.0
```

(continues on next page)

(continued from previous page)

```
  gdal: 3.13.3
  ghostscript: 10.08.0
GMT library information:
  version: 6.7.0
  padding: 2
  share dir: /home/runner/micromamba/envs/pygmt/share/gmt
  plugin dir: /home/runner/micromamba/envs/pygmt/lib/gmt/plugins
  library path: /home/runner/micromamba/envs/pygmt/lib/libgmt.so
  cores: 4
  grid layout: rows
  image layout:
  binary version: 6.7.0
```

You are now ready to make your first figure! Start by looking at our *Intro*, *Tutorials*, and *Gallery*. Good luck!

1.2 Which Python?

PyGMT is tested to run on Python >=3.12.

We recommend using the Miniforge Python distribution to ensure you have all dependencies installed. Installing Miniforge does not require administrative rights to your computer and doesn't interfere with any other Python installations on your system.

1.3 Which GMT?

PyGMT requires Generic Mapping Tools (GMT) >=6.5.0 since there are many changes being made to GMT itself in response to the development of PyGMT.

Compiled conda packages of GMT for Linux, macOS and Windows are provided through conda-forge. Advanced users can also build GMT from source instead.

We recommend following the instructions further on to install GMT.

1.4 Dependencies

PyGMT depends on GMT and the following Python libraries:

- **Required**: NumPy, pandas, xarray, packaging
- **Optional**: IPython, GeoPandas, contextily, rioxarray, PyArrow

For detailed version requirements, see *Minimum Supported Versions*.

1.5 Installing GMT and other dependencies

Before installing PyGMT, we must install GMT itself along with the other dependencies. The easiest way to do this is via the `conda` package manager. We recommend working in an isolated virtual environment to avoid issues with conflicting versions of dependencies.

First, we must configure conda to get packages from the conda-forge channel:

```
conda config --prepend channels conda-forge
```

Now we can create a new virtual environment with Python and all our dependencies installed (we'll call it `pygmt` but feel free to change it to whatever you want):

```
conda create --name pygmt python=3.15 numpy pandas xarray packaging gmt
```

Activate the environment by running the following (**do not forget this step!**):

```
conda activate pygmt
```

From now on, all commands will take place inside the virtual environment called `pygmt` and won't affect your default `base` installation.

Tip

You can also enable more PyGMT functionalities by installing PyGMT's optional dependencies in the environment.

```
conda install contextily geopandas ipython pyarrow-core rioxarray
```

1.6 Installing PyGMT

Now that you have GMT installed and your virtual environment activated, you can install PyGMT using any of the following methods.

1.6.1 Using conda (recommended)

This installs the latest stable release of PyGMT from conda-forge:

```
conda install pygmt
```

This upgrades the installed PyGMT version to be the latest stable release:

```
conda update pygmt
```

1.6.2 Using pip

This installs the latest stable release from PyPI:

```
python -m pip install pygmt
```

Tip

You can also run `python -m pip install pygmt[all]` to install PyGMT with all of its optional dependencies.

Alternatively, you can install the latest development version from TestPyPI:

```
python -m pip install --pre --extra-index-url https://test.pypi.org/simple/ pygmt
```

To upgrade the installed stable release or development version to be the latest one, just add `--upgrade` to the corresponding command above.

Any of the above methods (conda/pip) should allow you to use the PyGMT package from Python.

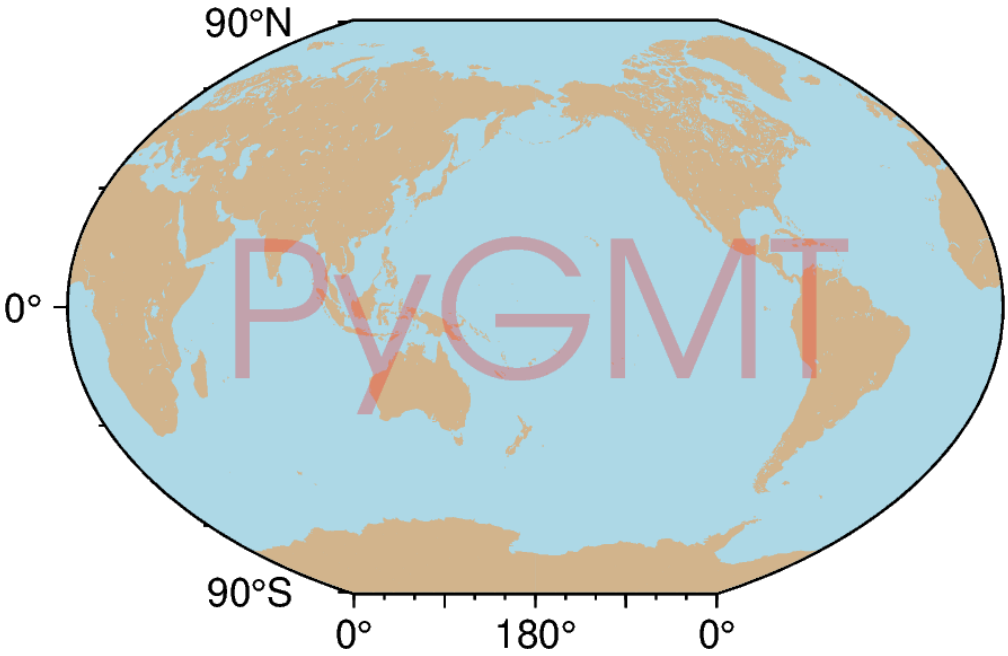
1.7 Testing your install

To ensure that PyGMT and its dependencies are installed correctly, run the following in your Python interpreter:

```
import pygmt
pygmt.show_versions()
```

```
PyGMT information:
  version: 0.20.0.dev184+g832c69a1f
System information:
  python: 3.15.0 | packaged by conda-forge | (main, Oct 12 2026, 17:36:43) [GCC␣
↪15.3.0]
  executable: /home/runner/micromamba/envs/pygmt/bin/python
  machine: Linux-7.0.0-1017-azure-x86_64-with-glibc2.39
Dependency information:
  numpy: 2.5.4
  pandas: 3.0.6
  xarray: 2026.9.0
  packaging: 26.3
  contextily: 1.7.1
  geopandas: 1.2.0
  IPython: 9.17.1
  pyarrow: 25.0.0
  rioxarray: 0.23.0
  gdal: 3.13.3
  ghostscript: 10.08.0
GMT library information:
  version: 6.7.0
  padding: 2
  share dir: /home/runner/micromamba/envs/pygmt/share/gmt
  plugin dir: /home/runner/micromamba/envs/pygmt/lib/gmt/plugins
  library path: /home/runner/micromamba/envs/pygmt/lib/libgmt.so
  cores: 4
  grid layout: rows
  image layout:
  binary version: 6.7.0
```

```
fig = pygmt.Figure()
fig.basemap(projection="R7c", region=[0, 360, -90, 90], frame=True)
fig.coast(land="tan", water="lightblue")
fig.text(position="MC", text="PyGMT", font="40p,AvantGarde-Book,red", transparency=75)
fig.show()
```



You should see a global map with land and water masses colored in tan and lightblue respectively. On top, there should be the semi-transparent text "PyGMT". If the semi-transparency does not show up, there is probably an incompatibility between your GMT and Ghostscript versions. For details, please run `pygmt.show_versions()` and see *Not working transparency*.

1.8 Common installation issues

If you have any issues with the installation, please check out the following common problems and solutions.

1.8.1 "Error loading GMT shared library at ..."

Sometimes, PyGMT will be unable to find the correct version of the GMT shared library (`libgmt`). This can happen if you have multiple versions of GMT installed.

You can tell PyGMT exactly where to look for `libgmt` by setting the environment variable *GMT_LIBRARY_PATH* to the directory where `libgmt.so`, `libgmt.dylib` or `gmt.dll` can be found on Linux, macOS or Windows, respectively.

For Linux/macOS, add the following line to your shell configuration file (usually `~/.bashrc` for Bash on Linux and `~/.zshrc` for Zsh on macOS):

```
export GMT_LIBRARY_PATH=$HOME/miniforge3/envs/pygmt/lib
```

For Windows, add the environment variable *GMT_LIBRARY_PATH* following these instructions and set its value to a path like:

```
C:\Users\USERNAME\Miniforge3\envs\pygmt\Library\bin\
```

1.8.2 `ModuleNotFoundError` in Jupyter notebook environment

If you can successfully import PyGMT in a Python interpreter or IPython, but get a `ModuleNotFoundError` when importing PyGMT in Jupyter, you may need to activate your `pygmt` virtual environment (using `conda activate pygmt`) and install a `pygmt` kernel following the commands below:

```
python -m ipykernel install --user --name pygmt  # install virtual environment␣
↪properly
jupyter kernelspec list --json
```

After that, you need to restart Jupyter, open your notebook, select the `pygmt` kernel and then import pygmt.

1.8.3 Not working transparency

It is known that some combinations of GMT and Ghostscript versions cause issues, especially regarding transparency. If the transparency doesn't work in your figures, please check your GMT and Ghostscript versions (you can run `pygmt.show_versions()`). We recommend:

- Ghostscript 9.53-9.56 for GMT 6.4.0 (or below)
- Ghostscript 10.03-10.07 for GMT 6.5.0-6.7.0

CHAPTER

TWO

INTRO TO PYGMT

Welcome to PyGMT! The tutorials in this intro are designed to teach basic concepts to create maps in PyGMT.

About this intro

It is assumed that PyGMT has been successfully *installed* on your system. To test this, run `import pygmt` in a Python IDE or Jupyter notebook.

This intro will progressively cover PyGMT data manipulation and plotting concepts, and later tutorials will use concepts explained in previous ones. It will not cover all PyGMT functions and methods.

2.1 1. Making your first figure

This tutorial covers the basics of creating a figure using PyGMT - a Python wrapper for the Generic Mapping Tools (GMT). It will only use the *pygmt.Figure.coast* method for plotting. Later tutorials will address other PyGMT methods.

2.1.1 Loading the library

The first step is to import `pygmt`. All methods and figure generation are accessible from the `pygmt` top level package.

```
import pygmt
from pygmt.params import Frame
```

2.1.2 Creating a figure

All figure generation in PyGMT is handled by the *pygmt.Figure* class. Start a new figure by creating an instance of this class:

```
fig = pygmt.Figure()
```

To add elements to the figure instance or object (`fig` in this example) different methods can be called on it. This example will use the *pygmt.Figure.coast* method, which can be used to create a map without any other methods or external data. The *pygmt.Figure.coast* method plots the coastlines, borders, and bodies of water using a database that is included in GMT.

First, a region for the figure must be selected. This example will plot some of the coast of Maine in the northeastern US. A Python list can be passed to the `region` parameter with the minimum and maximum X-values (longitude) and the minimum and maximum Y-values (latitude). For this example, the minimum (bottom left) coordinates are (N43.75, W69) and the maximum (top right) coordinates are (N44.75, W68). Negative values can be passed for latitudes in the southern hemisphere or longitudes in the western hemisphere.

In addition to the region, an argument needs to be passed to *pygmt.Figure.coast* to tell it what to plot. In this example, *pygmt.Figure.coast* will be told to plot the shorelines by passing the Boolean value `True` to the `shorelines` parameter. The `shorelines` parameter has other options for finer control, but setting it to `True` uses the default values.

```
fig.coast(region=[-69, -68, 43.75, 44.75], shorelines=True)
```

To see the figure, call *pygmt.Figure.show*.

```
fig.show()
```

2.1.3 Color the land and water

This figure plots all of the coastlines in the given region, but it does not indicate where the land and water are. Color values can be passed to `land` and `water` to set the colors on the map.

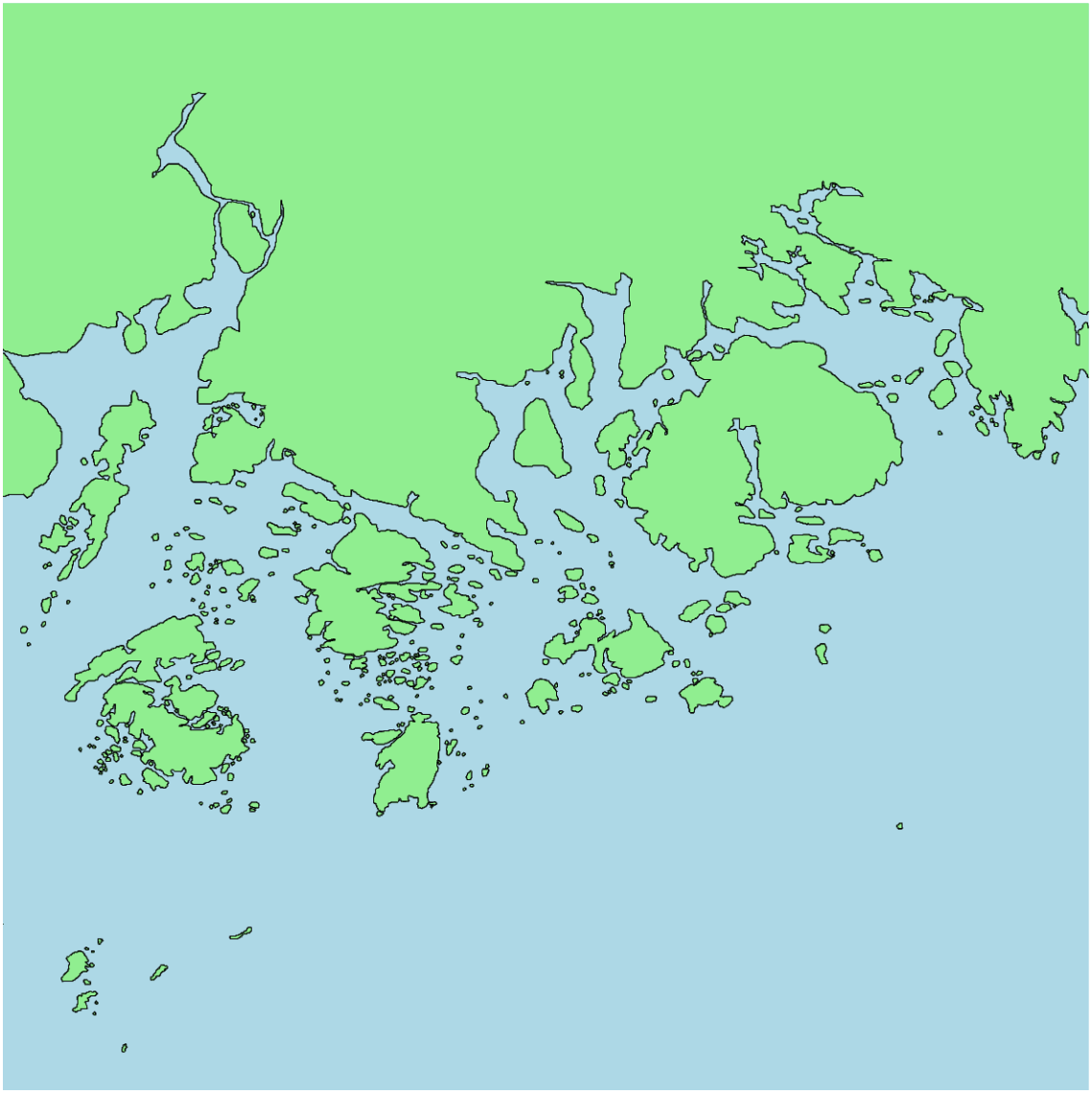
When plotting colors in PyGMT, there are multiple color codes, that can be used. This includes standard GMT color names (like `"skyblue"`), R/G/B levels (like `"0/0/255"`), a hex value (like `"#333333"`), or a gray level (like `"gray50"`). For this example, GMT color names are used.

```
fig = pygmt.Figure()
fig.coast(
    region=[-69, -68, 43.75, 44.75],
```

(continues on next page)

(continued from previous page)

```
    shorelines=True,
    land="lightgreen",
    water="lightblue",
)
fig.show()
```

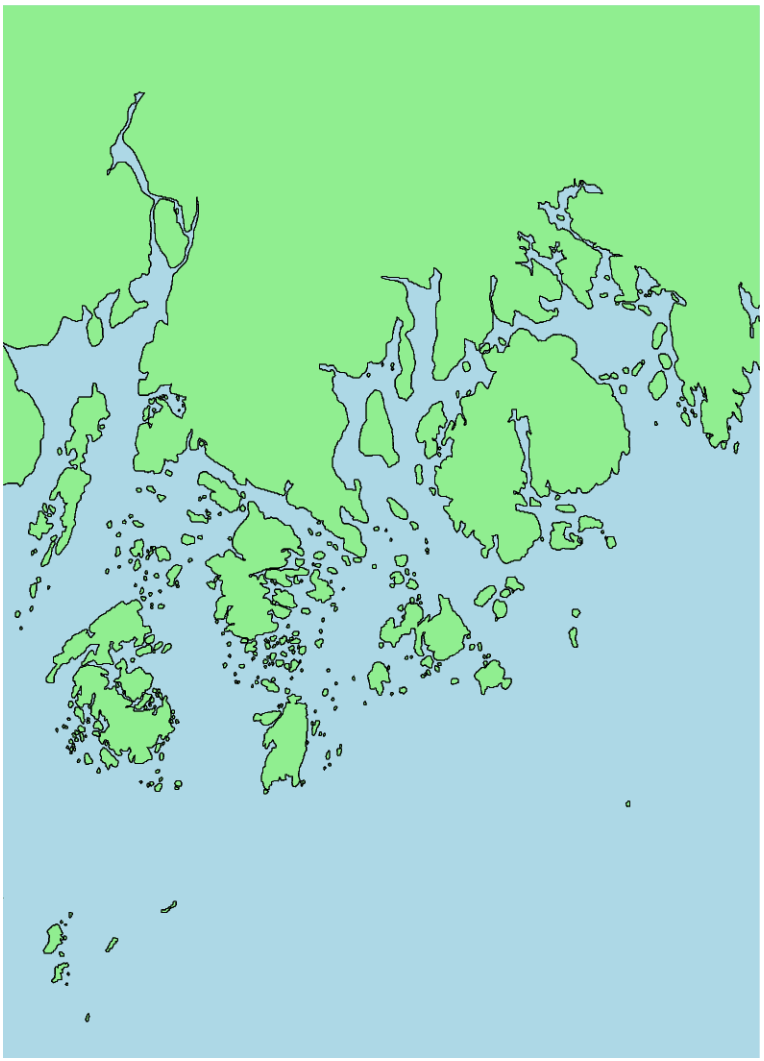


2.1.4 Set the projection

This figure now has its colors set. By default the projection and size of the map is set to `"Q15c+du"`. Here, `"Q"` defines a cylindrical equidistant map projection, `"15c+du"` means setting the maximum (upper) map dimension to 15 cm. However, both of these values can be customized according to the requirements using the `projection` parameter.

The appropriate projection varies for the type of map. The available projections are explained in the *projection* gallery. For this example, the Mercator projection is set using `"M"`. The width of the map will be 10 centimeters, as set by `"10c"`. The map size can also be set in inches using "i" (e.g. a 5-inch wide Mercator projection would use `"M5i"`).

```
fig = pygmt.Figure()
fig.coast(
    region=[-69, -68, 43.75, 44.75],
    shorelines=True,
    land="lightgreen",
    water="lightblue",
    projection="M10c",
)
fig.show()
```



2.1.5 Add a frame

While the map's colors, projection, and size have been set, the region that is being displayed is not apparent. A frame can be added to annotate the latitude and longitude of the region.

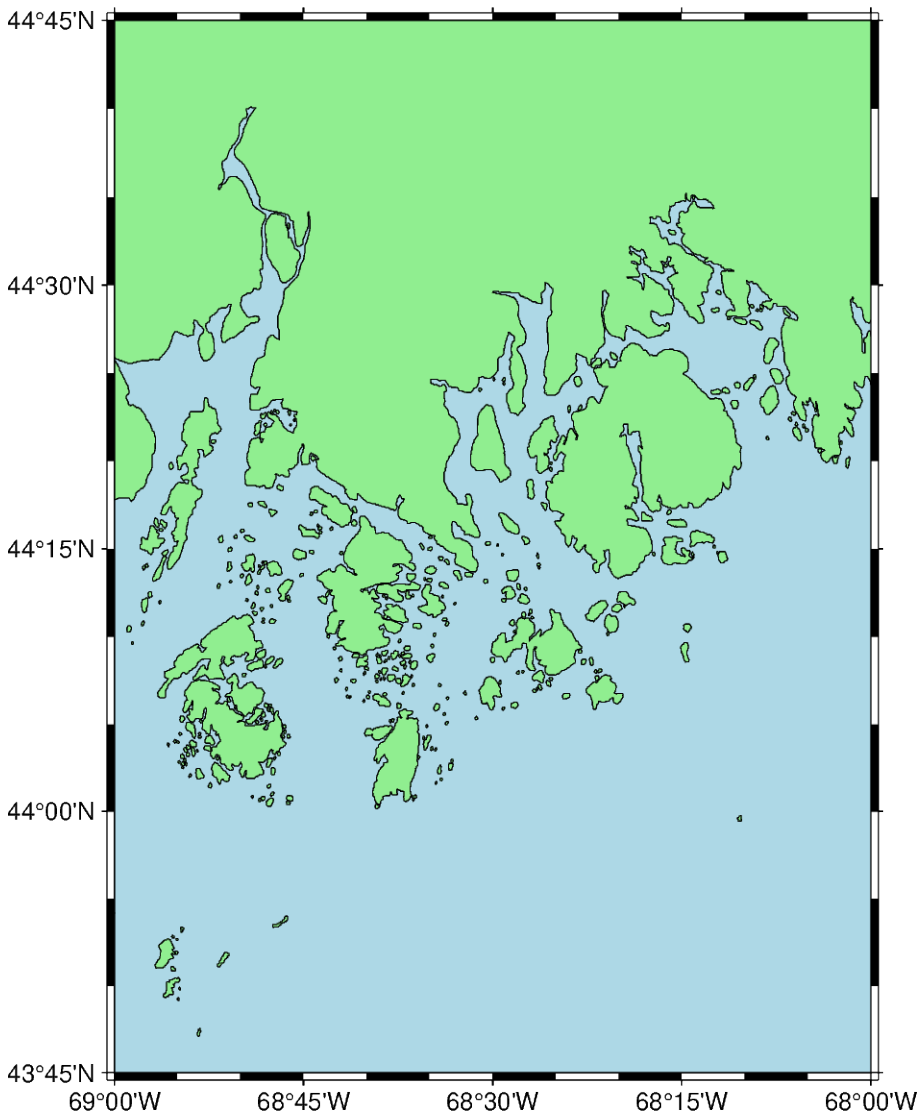
The `frame` parameter is used to add a frame to the map. For now, it will be set to `True` to annotate the axes automatically.

```
fig = pygmt.Figure()
fig.coast(
```

(continues on next page)

(continued from previous page)

```
    region=[-69, -68, 43.75, 44.75],
    shorelines=True,
    land="lightgreen",
    water="lightblue",
    projection="M10c",
    frame=True,
)
fig.show()
```



2.1.6 Add a title

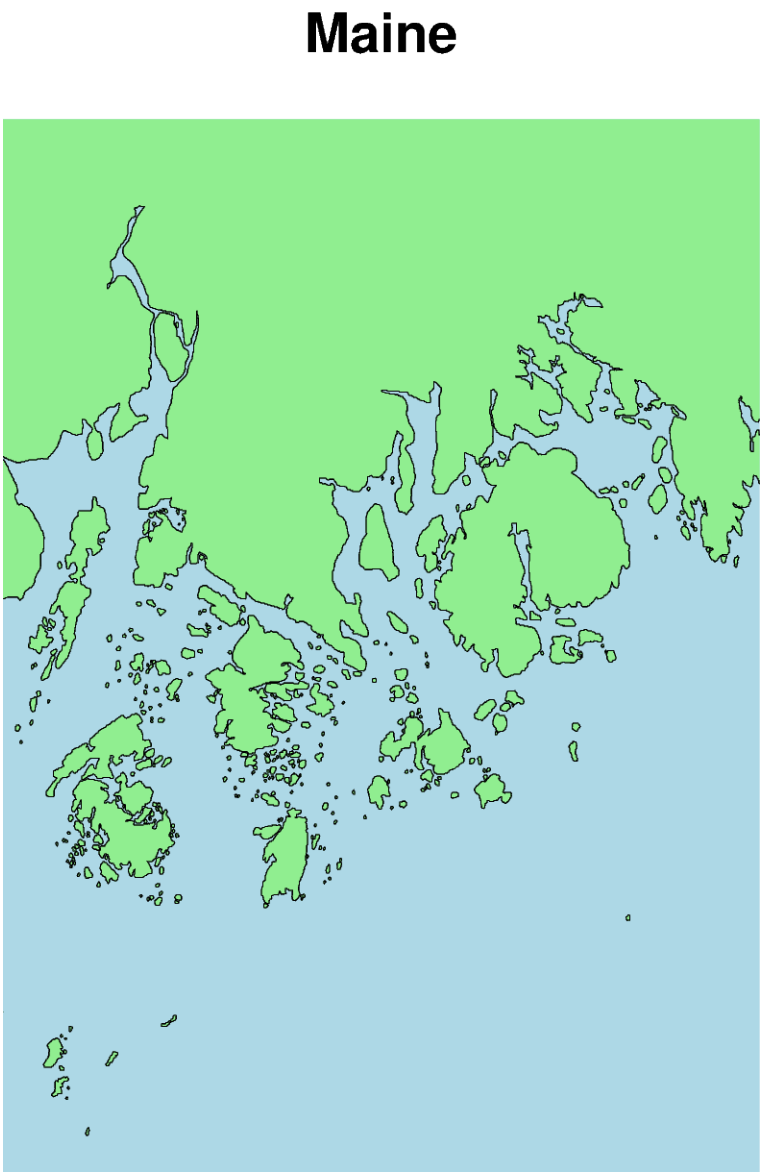
The `frame` parameter can also be used to add a title to the map. Here, *`pygmt.params.Frame`* sets the map title.

```
fig = pygmt.Figure()
fig.coast(
    region=[-69, -68, 43.75, 44.75],
    shorelines=True,
```

(continues on next page)

(continued from previous page)

```
    land="lightgreen",
    water="lightblue",
    projection="M10c",
    frame=Frame(title="Maine"),
)
fig.show()
```



2.1.7 Additional exercises

This is the end of the first tutorial. Here are some additional exercises for the concepts that were discussed:

1. Make a map of Germany using its ISO country code ("DE"). Pass the ISO code as a Python string to the `region` parameter.
2. Change the color of the landmass to "khaki" and the water to "azure".
3. Change the color of the lakes (using the `lakes` parameter) to "red".

4. Create a global map. Set the region to "d" to center the map at the Prime Meridian or "g" to center the map at the International Date Line. When the region is set without using a list full of integers or floating numbers, the argument needs to be passed as a Python string. Create a map with a width of 15 centimeters using the Mollweide ("W") projection.

Total running time of the script: (0 minutes 0.891 seconds)

2.2 2. Create a contour map

This tutorial page covers the basics of creating a figure of the Earth relief, using a remote dataset hosted by GMT, using the method *pygmt.datasets.load_earth_relief*. It will use the *pygmt.Figure.grdimage*, *pygmt.Figure.grdcontour*, *pygmt.Figure.colorbar*, and *pygmt.Figure.coast* methods for plotting.

```
import pygmt
from pygmt.params import Axis
```

2.2.1 Loading the Earth relief dataset

The first step is to use *pygmt.datasets.load_earth_relief*. The `resolution` parameter sets the resolution of the remote grid file, which will affect the resolution of the plot made later in the tutorial. The `registration` parameter determines the grid registration.

This grid region covers the islands of Guam and Rota in the western Pacific Ocean.

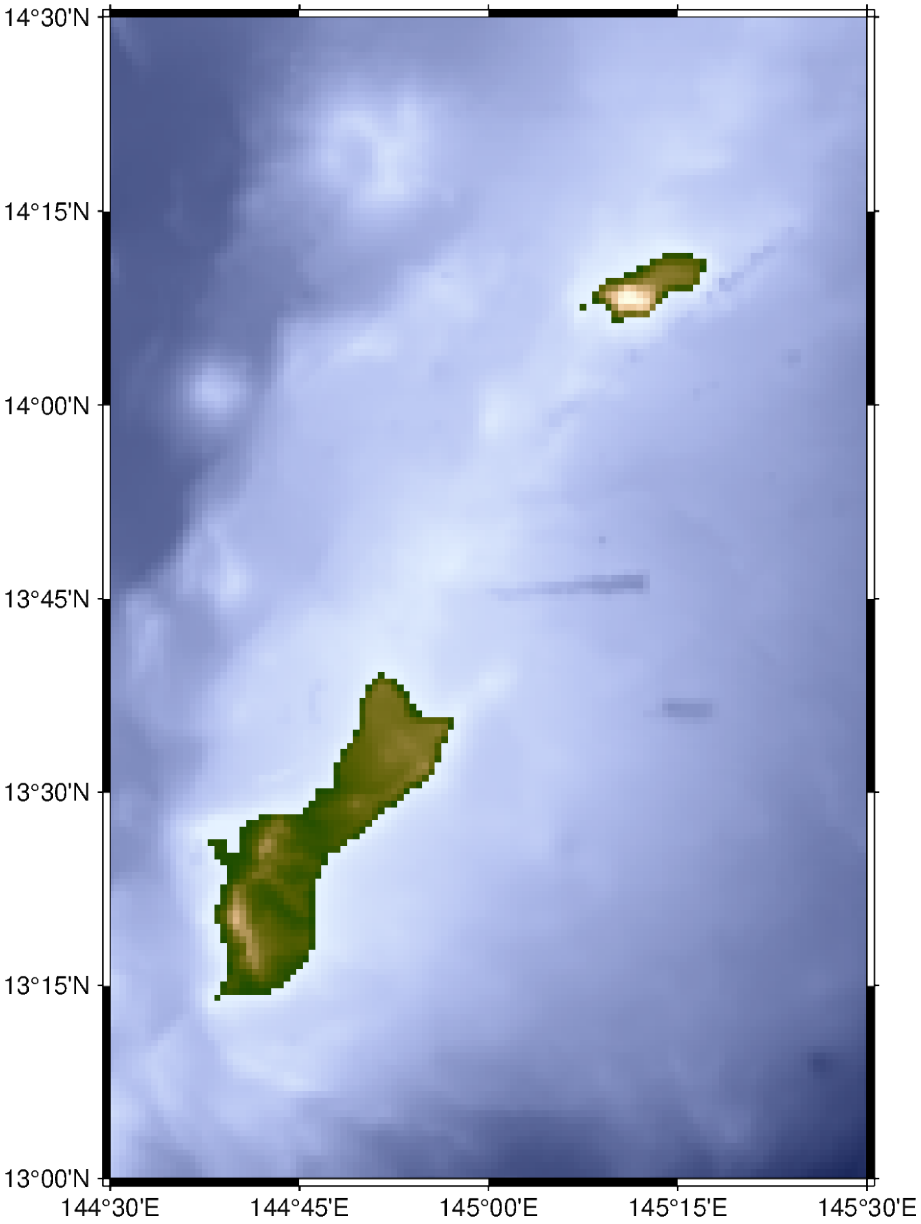
```
grid = pygmt.datasets.load_earth_relief(
    resolution="30s", region=[144.5, 145.5, 13, 14.5], registration="gridline"
)
```

2.2.2 Plotting Earth relief

To plot Earth relief data, the method *pygmt.Figure.grdimage* can be used to plot a color-coded figure to display the topography and bathymetry in the grid file. The `grid` parameter accepts the input grid, which in this case is the remote file downloaded in the previous step. If the `region` parameter is not set, the region boundaries of the input grid are used.

The `cmap` parameter sets the color palette table (CPT) used for portraying the Earth relief. The *pygmt.Figure.grdimage* method uses the input grid to relate the Earth relief values to a specific color within the CPT. In this case, the CPT "oleron" is used; a full list of CPTs can be found at https://docs.generic-mapping-tools.org/6.7/reference/cpts.html.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid, frame=Axis(annot=True), projection="M10c", cmap="SCM/oleron")
fig.show()
```



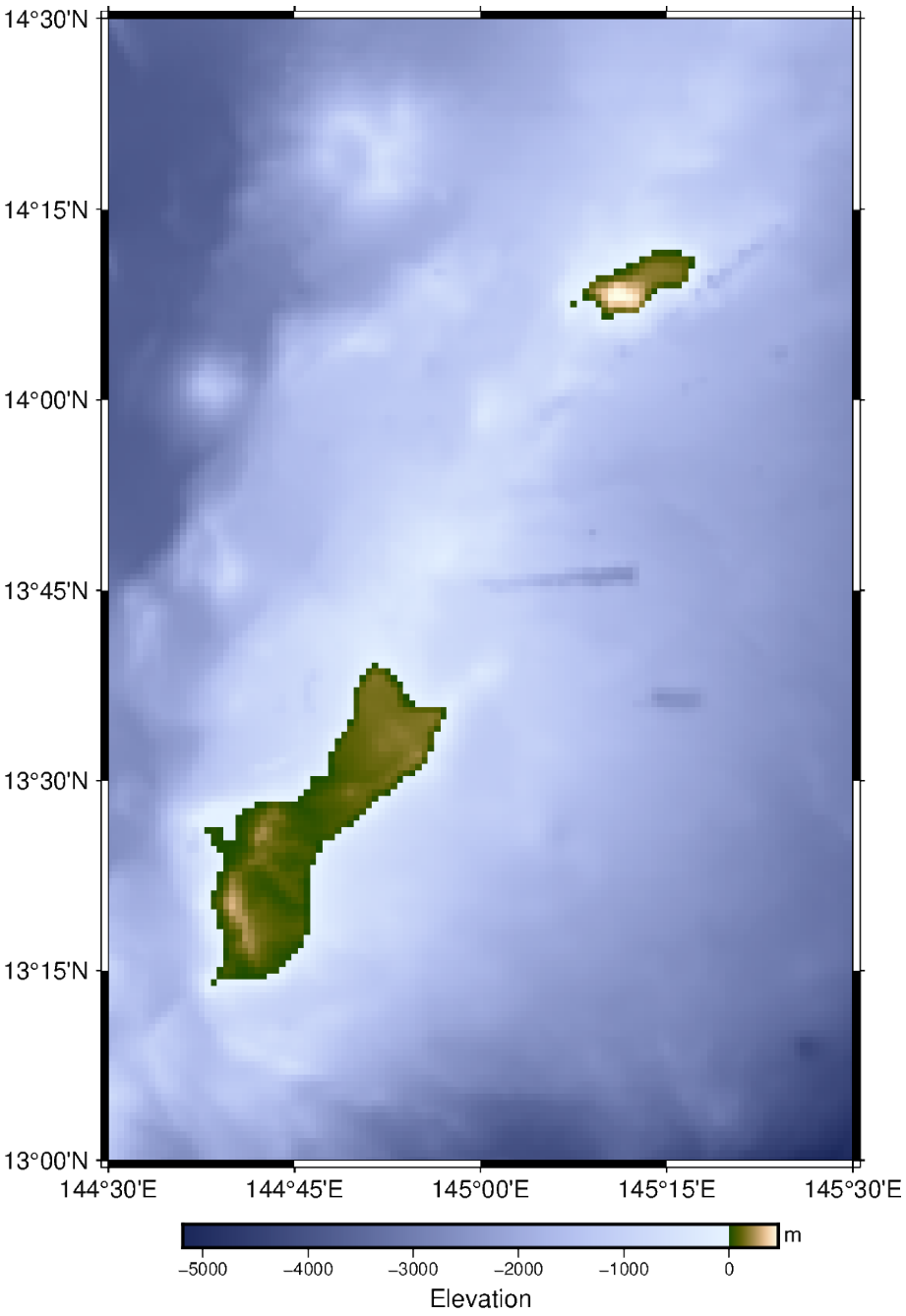
2.2.3 Adding a colorbar

To show how the plotted colors relate to the Earth relief, a colorbar can be added using the *pygmt.Figure.colorbar* method.

To control the annotation and labels on the colorbar, use the `annot` parameter to set the annotation interval (in this case every 1,000 meters), the `label` and `unit` parameters to set the labels along the long and short dimensions of the colorbar, respectively.

By default, the CPT for the colorbar is the same as the one set in *pygmt.Figure.grdimage*.

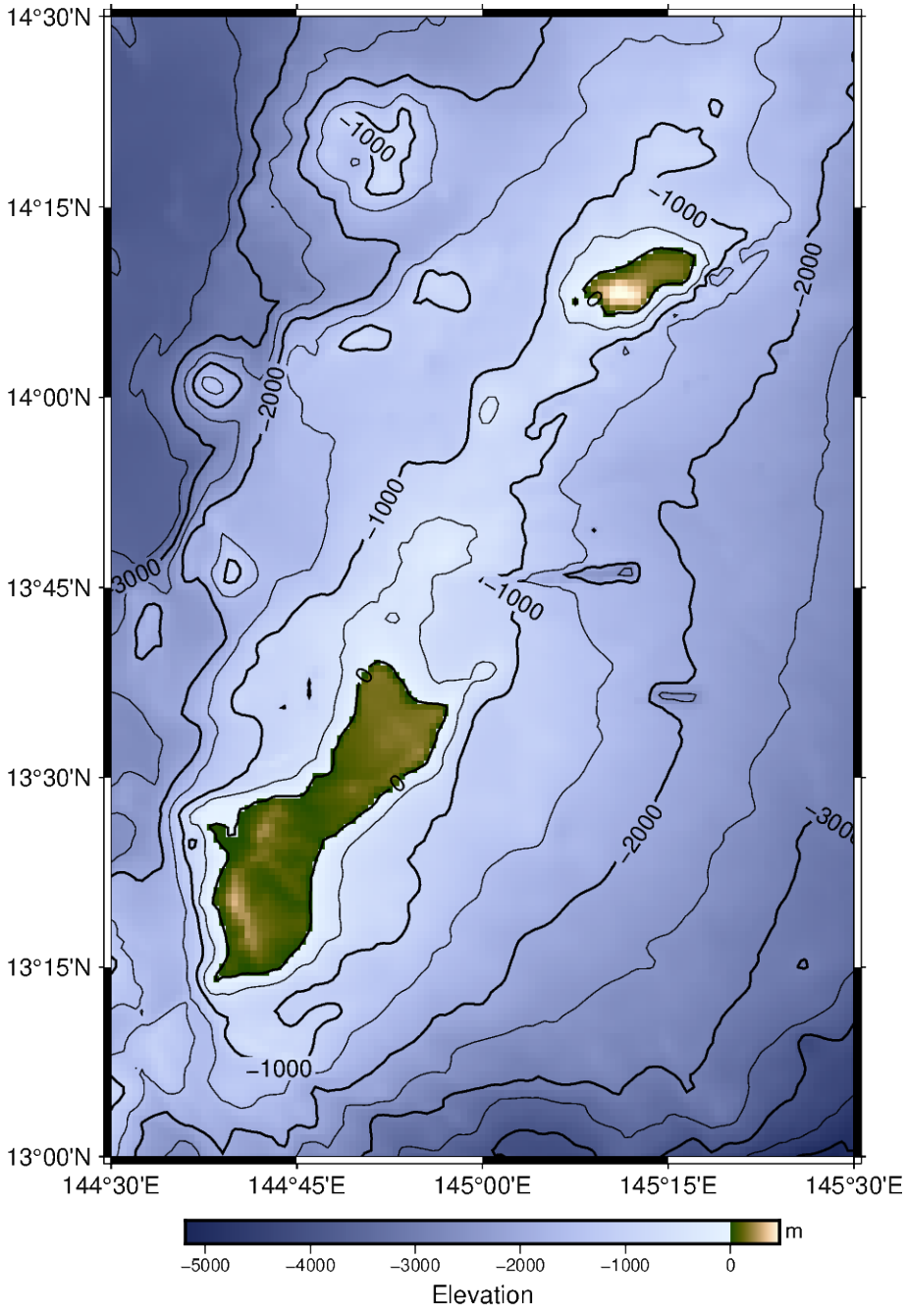
```
fig = pygmt.Figure()
fig.grdimage(grid=grid, frame=Axis(annot=True), projection="M10c", cmap="SCM/oleron")
fig.colorbar(annot=1000, label="Elevation", unit="m")
fig.show()
```



2.2.4 Adding contour lines

To add contour lines to the color-coded figure, the *pygmt.Figure.grdcontour* method is used. The `frame` and `projection` are already set using *pygmt.Figure.grdimage* and are not needed again. However, the same input for `grid` (in this case, the variable named "grid") must be input again. The `levels` parameter sets the spacing between adjacent contour lines (in this case, 500 meters). The `annotation` parameter annotates the contour lines corresponding to the given interval (in this case, 1,000 meters) with the related values, here elevation or bathymetry. By default, these contour lines are drawn thicker. Optionally, the appearance (thickness, color, style) of the annotated and the not-annotated contour lines can be adjusted (separately) by specifying the desired `pen`.

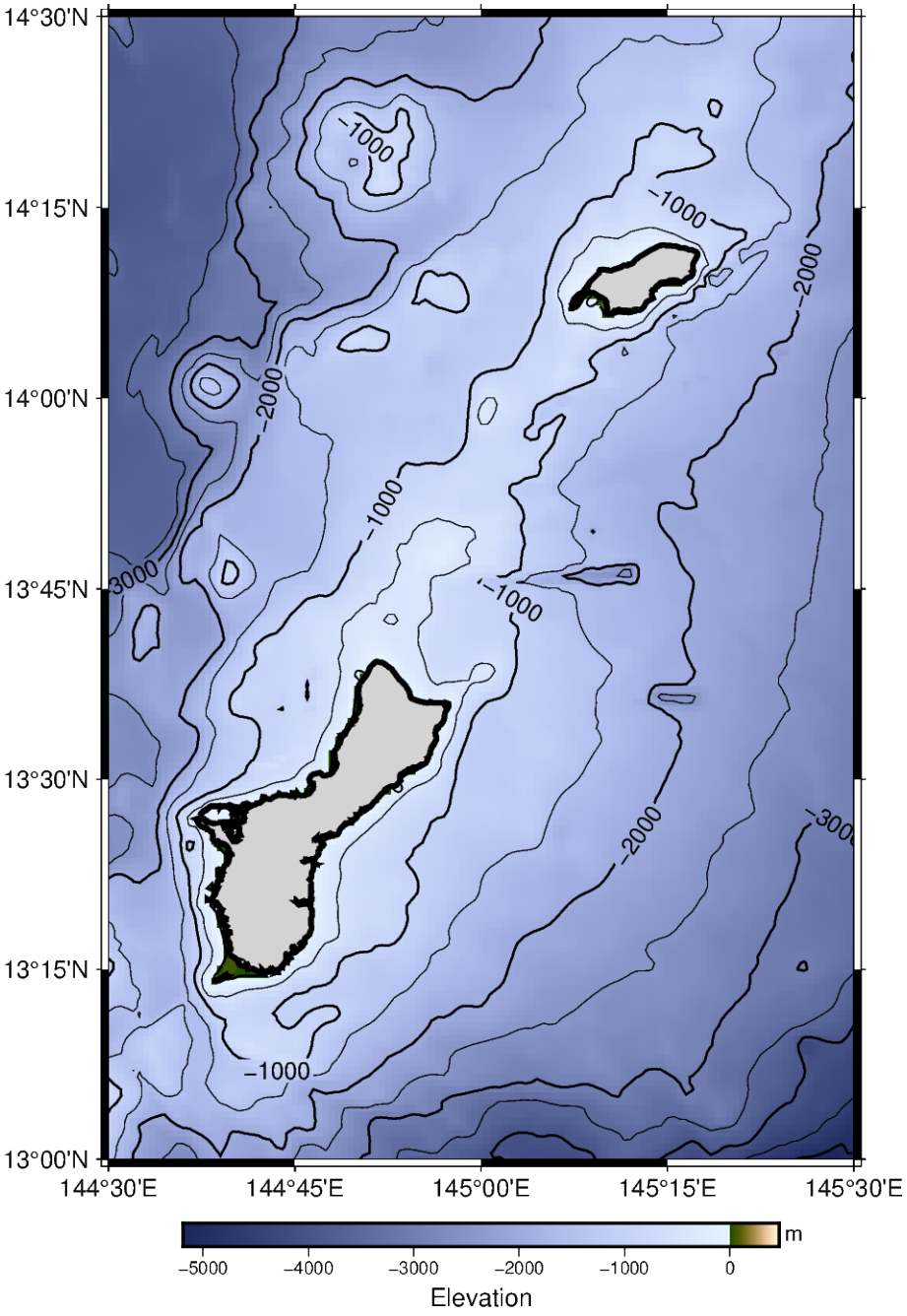
```
fig = pygmt.Figure()
fig.grdimage(grid=grid, frame=Axis(annot=True), projection="M10c", cmap="SCM/oleron")
fig.grdcontour(grid=grid, levels=500, annotation=1000)
fig.colorbar(annot=1000, label="Elevation", unit="m")
fig.show()
```



2.2.5 Color in land

To make it clear where the islands are located, the *pygmt.Figure.coast* method can be used to color in the landmasses. The `land` is colored in as "lightgray", and the `shorelines` parameter draws a border around the islands.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid, frame=Axis(annot=True), projection="M10c", cmap="SCM/oleron")
fig.grdcontour(grid=grid, levels=500, annotation=1000)
fig.coast(shorelines="2p", land="lightgray")
fig.colorbar(annot=1000, label="Elevation", unit="m")
fig.show()
```



2.2.6 Additional exercises

This is the end of the second tutorial. Here are some additional exercises for the concepts that were discussed:

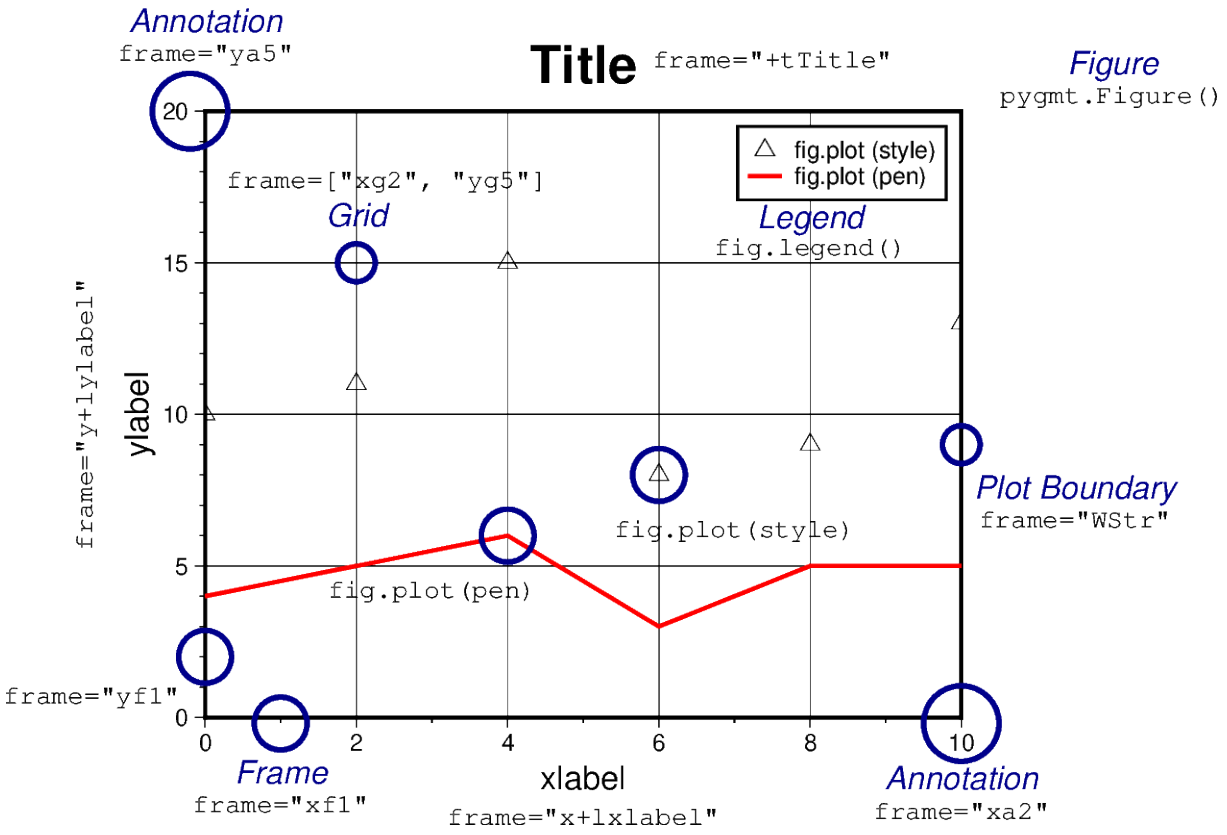
1. Change the resolution of the grid file to either `"01m"` (1 arc-minute, a lower resolution) or `"15s"` (15 arc-seconds, a higher resolution). Note that higher resolution grids will have larger file sizes. Available resolutions can be found at *pygmt.datasets.load_earth_relief*.
2. Create a contour map of the area around Mt. Rainier. A suggestion for the `region` would be `[-122, -121, 46.5, 47.5]`. Adjust the *pygmt.Figure.grdcontour* and *pygmt.Figure.colorbar* settings as needed to make the figure look good.
3. Create a contour map of São Miguel Island in the Azores; a suggested `region` is `[-26, -25, 37.5, 38]`. Instead of coloring in `land`, set `water` to "lightblue" to only display Earth relief information for the land.
4. Try other CPTs, such as "SCM/fes" or "geo".

Total running time of the script: (0 minutes 0.897 seconds)

2.3 3. Figure elements

The figure below shows the naming of figure elements in PyGMT.

- *pygmt.Figure*: having a number of plotting methods. Every new figure must start with the creation of a *pygmt.Figure* instance
- `frame`: setting plot boundaries (a combination of the single letters of **WSNE**, **wsne**, or **lbtr**), adding annotations, ticks, gridlines (**afg**), axis labels (**+l**), and title (**+t**), e.g., in *pygmt.Figure.basemap*. Detailed examples can be found at *frame and axes attributes*
- *pygmt.Figure.plot*: plotting lines or symbols based on `pen` or `style` parameters, respectively
- *pygmt.Figure.text*: plotting text strings whereby the `font` parameter adjusts fontsize, fontstyle, and color
- *pygmt.Figure.legend*: showing the naming of lines or symbols while the `label` is given in *pygmt.Figure.plot*
- *pygmt.Figure.show*: previewing the content added to the current figure instance




```
import pygmt

fig = pygmt.Figure()

x = range(0, 11, 2)
y_1 = [10, 11, 15, 8, 9, 13]
y_2 = [4, 5, 6, 3, 5, 5]

fig.basemap(
    region=[0, 10, 0, 20],
    projection="X10c/8c",
    frame=["WStr+tTitle", "xa2f1g2+lxlabel", "ya5f1g5+lylabel"],
)
fig.plot(x=x, y=y_1, style="t0.3c", label="fig.plot (style)")
fig.plot(x=x, y=y_2, pen="1.5p,red", label="fig.plot (pen)")

mainexplain = {"font": "12p,2,darkblue", "justify": "TC", "no_clip": True}
minorexplain = {"font": "10p,8", "justify": "TC", "no_clip": True}
# ============ Figure
fig.text(x=12, y=22, text="Figure", **mainexplain)
fig.text(x=12, y=20.8, text="pygmt.Figure()", **minorexplain)
# ============ Title
fig.text(x=7.5, y=22, text='frame="+tTitle"', **minorexplain)
# ============ xlabel
fig.text(x=5, y=-3, text='frame="x+lxlabel"', **minorexplain)
# ============ ylabel
fig.text(x=-1.7, y=10, text='frame="y+lylabel"', angle=90, **minorexplain)
```

(continues on next page)

(continued from previous page)

```
# ============ x-majorticks
fig.plot(x=10, y=-0.2, style="c1c", pen="2p,darkblue", no_clip=True)
fig.text(x=10, y=-1.6, text="Annotation", **mainexplain)
fig.text(x=10, y=-2.8, text='frame="xa2"', **minorexplain)
# ============ y-majorticks
fig.plot(x=-0.2, y=20, style="c1c", pen="2p,darkblue", no_clip=True)
fig.text(x=0, y=23.4, text="Annotation", **mainexplain)
fig.text(x=0, y=22.2, text='frame="ya5"', **minorexplain)
# ============ x-minorticks
fig.plot(x=1, y=-0.2, style="c0.7c", pen="2p,darkblue", no_clip=True)
fig.text(x=1, y=-1.4, text="Frame", **mainexplain)
fig.text(x=1, y=-2.6, text='frame="xf1"', **minorexplain)
# ============ y-minorticks
fig.plot(x=0, y=2, style="c0.7c", pen="2p,darkblue", no_clip=True)
fig.text(x=-1.5, y=1, text='frame="yf1"', **minorexplain)
# ============ Grid
fig.plot(x=2, y=15, style="c0.5c", pen="2p,darkblue")
fig.text(x=2, y=17, text="Grid", **mainexplain)
fig.text(x=2.4, y=18, text='frame=["xg2", "yg5"]', **minorexplain)
# ============ Plot Boundaries
fig.plot(x=10, y=9, style="c0.5c", pen="2p,darkblue", no_clip=True)
fig.text(x=11.5, y=8, text="Plot Boundary", **mainexplain)
fig.text(x=11.5, y=6.8, text='frame="WStr"', **minorexplain)
# ============ fig.plot (style)
fig.plot(x=6, y=8, style="c0.7c", pen="2p,darkblue")
fig.text(x=7, y=6.5, text="fig.plot(style)", **minorexplain)
# ============ fig.plot (pen)
fig.plot(x=4, y=6, style="c0.7c", pen="2p,darkblue")
fig.text(x=3, y=4.5, text="fig.plot(pen)", **minorexplain)
# ============ Legend
fig.legend()
fig.text(x=8, y=16.9, text="Legend", **mainexplain)
fig.text(x=8, y=15.8, text="fig.legend()", **minorexplain)

fig.show()
```

Total running time of the script: (0 minutes 0.428 seconds)

2.4 4. PyGMT I/O: Table inputs

Generally, PyGMT accepts two different types of data inputs: tables and grids.

- A table is a 2-D array with rows and columns. Each column represents a different variable (e.g., x, y and z) and each row represents a different record.
- A grid is a 2-D array of data that is regularly spaced in the x- and y-directions (or longitude and latitude).

In this tutorial, we'll focus on working with table inputs, and cover grid inputs in a separate tutorial.

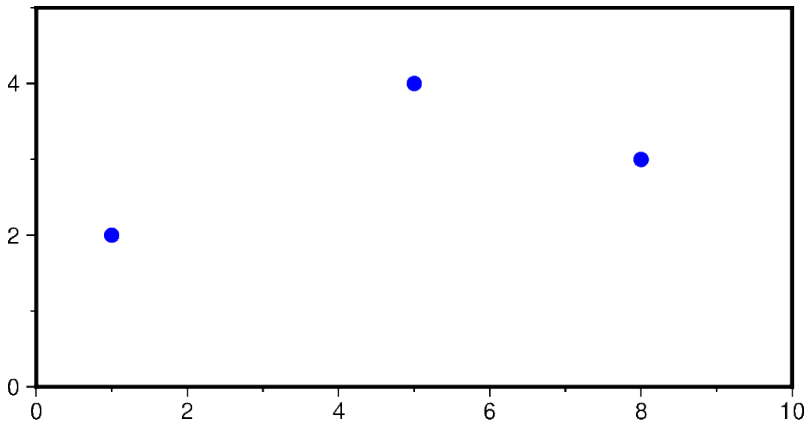
PyGMT supports a variety of table input types that allow you to work with data in a format that suits your needs. In this tutorial, we'll explore the different table input types available in PyGMT and provide examples for each. By understanding the different table input types, you can choose the one that best fits your data and analysis needs, and work more efficiently with PyGMT.

```
from pathlib import Path

import geopandas
import numpy as np
import pandas as pd
import pygmt
```

2.4.1 ASCII table file

Most PyGMT functions/methods that accept table input data have a `data` parameter. The easiest way to provide table input data to PyGMT is by specifying the file name of an ASCII table (e.g., `data="input_data.dat"`). This is useful when your data is stored in a separate text file.

```
# Create an example file with 3 rows and 2 columns
data = np.array([[1.0, 2.0], [5.0, 4.0], [8.0, 3.0]])
np.savetxt("input_data.dat", data, fmt="%f")

# Pass the file name to the data parameter
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 5], projection="X10c/5c", frame=True)
fig.plot(data="input_data.dat", style="p0.2c", fill="blue")
fig.show()

# Now let's delete the example file
Path("input_data.dat").unlink()
```



Besides a plain string to a table file, the following variants are also accepted:

- A `pathlib.Path` object.
- A full URL. PyGMT will download the file to the current directory first.
- A file name prefixed with `@` (e.g., `data="@input_data.dat"`), which is a special syntax in GMT to indicate that the file is a remote file hosted on the GMT data server.

Additionally, PyGMT also supports a list of file names, `pathlib.Path` objects, URLs, or remote files, to provide more flexibility in specifying input files.

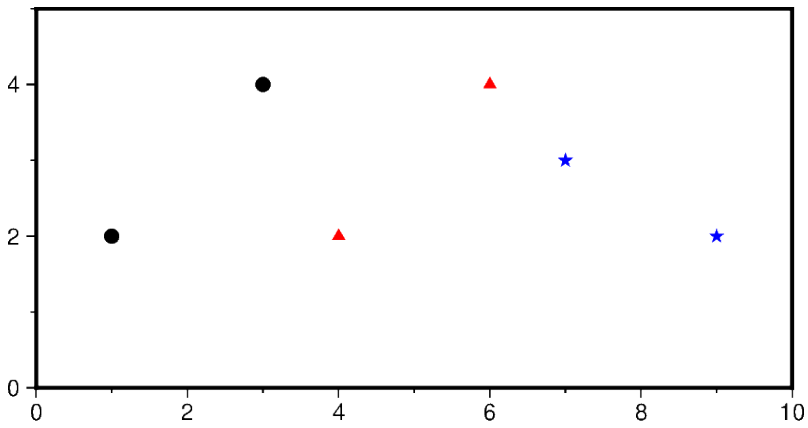
2.4.2 2-D array: *list*, *numpy.ndarray*, and *pandas.DataFrame*

The `data` parameter also accepts a 2-D array, e.g.,

- A 2-D `list` (i.e., a list of lists)
- A `numpy.ndarray` object with a dimension of 2
- A `pandas.DataFrame` object

This is useful when you want to plot data that is already in memory.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 5], projection="X10c/5c", frame=True)

# Pass a 2-D list to the 'data' parameter
fig.plot(data=[[1.0, 2.0], [3.0, 4.0]], style="c0.2c", fill="black")

# Pass a 2-D numpy array to the 'data' parameter
fig.plot(data=np.array([[4.0, 2.0], [6.0, 4.0]]), style="t0.2c", fill="red")

# Pass a pandas.DataFrame to the 'data' parameter
df = pd.DataFrame(np.array([[7.0, 3.0], [9.0, 2.0]]), columns=["x", "y"])
fig.plot(data=df, style="a0.2c", fill="blue")

fig.show()
```



2.4.3 `geopandas.GeoDataFrame`

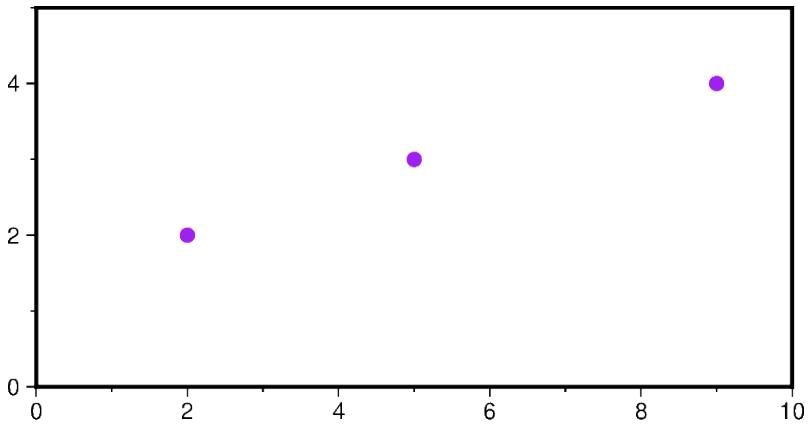
If you're working with geospatial data, you can read your data as a `geopandas.GeoDataFrame` object and pass it to the `data` parameter. This is useful if your data is stored in a geospatial data format (e.g., GeoJSON, etc.) that GMT and PyGMT do not support natively.

```
# Example GeoDataFrame
gdf = geopandas.GeoDataFrame(
    {
        "geometry": geopandas.points_from_xy([2, 5, 9], [2, 3, 4]),
        "value": [10, 20, 30],
    }
)

# Use the GeoDataFrame to specify the 'data' parameter
```

(continues on next page)

(continued from previous page)

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 5], projection="X10c/5c", frame=True)
fig.plot(data=gdf, style="c0.2c", fill="purple")
fig.show()
```

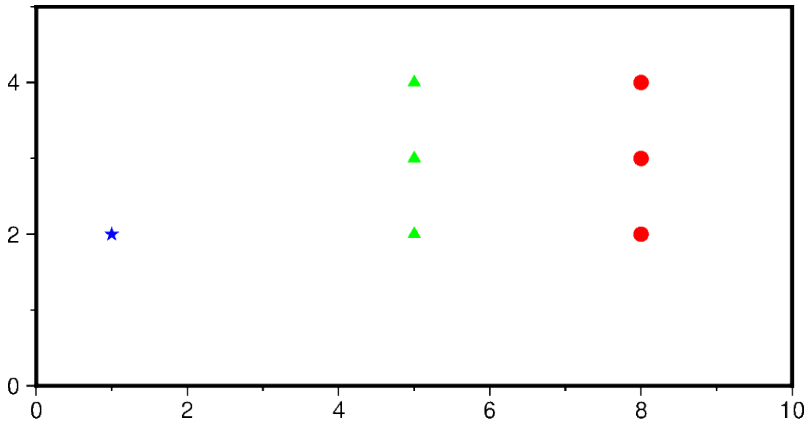



```
/
↪home/runner/micromamba/envs/pygmt/lib/python3.15/site-packages/pyogrio/geopandas.py:948:␣
↪UserWarning: 'crs' was not provided.  The output dataset will not have projection␣
↪information defined and may not be usable in other systems.
  write(
```

2.4.4 Scalar values or 1-D arrays

In addition to the `data` parameter, some PyGMT functions/methods also provide individual parameters (e.g., `x` and `y` for data coordinates) which allow you to specify the data. These parameters accept individual scalar values or 1-D arrays (lists or 1-D numpy arrays).

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 5], projection="X10c/5c", frame=True)

# Pass scalar values to plot a single data point
fig.plot(x=1.0, y=2.0, style="a0.2c", fill="blue")

# Pass 1-D lists to plot multiple data points
fig.plot(x=[5.0, 5.0, 5.0], y=[2.0, 3.0, 4.0], style="t0.2c", fill="green")

# Pass 1-D numpy arrays to plot multiple data points
fig.plot(
    x=np.array([8.0, 8.0, 8.0]), y=np.array([2.0, 3.0, 4.0]), style="c0.2c", fill="red
↪"
)

fig.show()
```



2.4.5 Conclusion

In PyGMT, you have the flexibility to provide data in various table input types, including file names, 2-D arrays (2-D `list`, `numpy.ndarray`, `pandas.DataFrames`), scalar values or a series of 1-D arrays, and `geopandas.GeoDataFrame`. Choose the input type that best suits your data source and analysis requirements.

Total running time of the script: (0 minutes 0.950 seconds)

CHAPTER

THREE

OVERVIEW

3.1 Why PyGMT

A beautiful map is worth a thousand words. To experience the power of PyGMT, try it online with Binder or Google Colab without installing anything on your computer.

For a quick introduction, check out our 3-minute overview. Afterward, try the *Intro to PyGMT* tutorial on your own, explore the *Gallery* for inspiration, follow the *Tutorials* to learn more, explore available map *Projections*, and finally check out some *external PyGMT examples*.

3.2 Background

PyGMT is a Python wrapper for the Generic Mapping Tools (GMT), a command-line program widely used across the Earth, Ocean, and Planetary sciences and beyond. It provides capabilities for processing spatial data (gridding, filtering, masking, FFTs, etc) and making high quality plots and maps.

PyGMT is different from Python libraries like Bokeh and Matplotlib, which have a larger focus on interactivity and allowing different backends. GMT uses the PostScript format to generate high quality (static) vector graphics for publications, posters, talks, etc. It is memory efficient and very fast. The PostScript figures can be converted to other formats like PDF, PNG, and JPG for use on the web and elsewhere. In fact, PyGMT users will usually not have any contact with the original PostScript files and get only the more convenient formats like PDF and PNG.

For more information about the design and development of PyGMT, see the PyGMT paper published in *Geochemistry, Geophysics, Geosystems*.

3.3 Project goals

From the beginning, the project has aimed to achieve the following goals:

- Make GMT more accessible to new users.
- Build a Pythonic API for GMT.
- Interface with the GMT C API directly using ctypes (no system calls).
- Support rich display in the Jupyter notebook.
- Integrate with the scientific Python ecosystem: `numpy.ndarray` or `pandas.DataFrame` for data tables, `xarray.DataArray` for grids, and `geopandas.GeoDataFrame` for geographical data.

3.4 Community

The PyGMT project was started in 2017 by Leonardo Uieda and Paul Wessel (the co-creator and main developer of GMT) at the University of Hawai'i at Mānoa. Currently, the project is community-driven and maintained by an *international team*, with contributions from multiple contributors.

We welcome any feedback, ideas, and contributions! Let us know by submitting issues on GitHub or by posting on our Discourse forum.

3.5 Related projects

Other official GMT wrappers include:

- GMT.jl: A Julia wrapper for GMT.
- gmtmex: A Matlab/Octave wrapper for GMT.

3.6 Funding

The development of PyGMT has been supported by NSF grants OCE-1558403 and EAR-1948602.

3.7 License

PyGMT is free software: you can redistribute it and/or modify it under the terms of the BSD 3-clause License. A copy of this license is provided in LICENSE.txt.

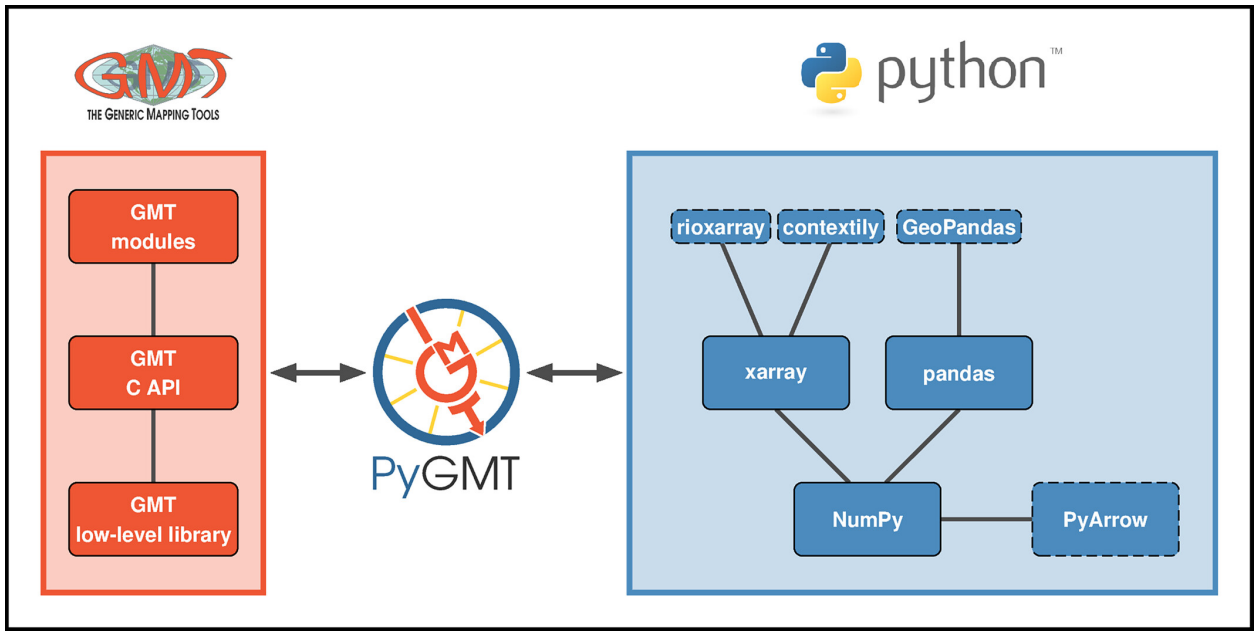
CHAPTER

FOUR

PYGMT ECOSYSTEM

The PyGMT ecosystem consists of the packages that PyGMT depends on and the packages that depend on PyGMT. Besides GMT itself, PyGMT integrates well with the scientific Python ecosystem, with NumPy for its fundamental array data structure, pandas for tabular data I/O and xarray for raster grids/images/cubes I/O. In addition to these core dependencies, it also relies on several optional packages to provide additional functionality for users. In turn, a growing number of packages build on PyGMT for geospatial data processing, analysis, and visualization.

4.1 PyGMT dependencies



The PyGMT ecosystem. This figure was originally published in the PyGMT paper in G-Cubed. The full publication is released under CC BY-NC 4.0. No modifications were made.

An asterisk (*) after the package name indicates the package is a required dependency of PyGMT.

NumPy*
: The fundamental package for scientific computing in Python, providing a multidimensional array object and an assortment of routines for fast operations on arrays.

pandas*
: A Python package providing fast, flexible, and expressive data structures designed to make working with tabular data easy and intuitive.

xarray*
: A Python package that introduces labels in the form of dimensions, coordinates, and attributes on top of raw NumPy-like arrays, which allows for more intuitive, more concise, and less error-prone user experience.

IPython
: A rich toolkit for using Python interactively, including a powerful interactive Python shell and a Jupyter kernel to work with Python code in Jupyter notebooks and other interactive frontends. PyGMT relies on it to provide a rich interactive experience in Jupyter notebooks.

GeoPandas
: A Python package that extends the datatypes used by pandas to allow spatial operations on geometric types. PyGMT doesn't directly rely on it, but supports its two main data structures, `geopandas.GeoDataFrame` and `geopandas.GeoSeries`, in data processing and plotting functions/methods.

contextily
: A small Python package to retrieve tile maps from the internet. These tiles can be added as background of a map or saved to disk into geospatial raster files. In PyGMT, *`pygmt.datasets.load_tile_map`* and *`pygmt.Figure.tilemap`* rely on it.

rioxarray
: A geospatial xarray extension powered by rasterio, enabling seamless reading, writing, and manipulation of multi-dimensional arrays with geospatial attributes such as coordinate reference systems (CRS) and spatial extent (bounds). PyGMT relies on it in in several aspects:

  1. To save multi-band rasters to temporary files in GeoTIFF format, to support processing and plotting 3-D `xarray.DataArray` images.
  2. To write CRS information to the `xarray.DataArray` objects.
  3. To reproject raster tiles to the target CRS in *`pygmt.datasets.load_tile_map`*.

  Note

  We're working towards avoiding temporary files when processing/plotting multi-band rasters in PR #3468.

PyArrow
: The Python bindings for Apache Arrow, a development platform for in-memory analytics that specifies a standardized language-independent columnar memory format for flat and hierarchical data, organized for efficient analytic operations on modern hardware.

  Note

  If you have PyArrow installed, PyGMT does have some initial support for `pandas.Series` and `pandas.DataFrame` objects with Apache Arrow-backed arrays. Specifically, only uint/int/float, date32/date64 and string types are supported for now. Support for Duration types and GeoArrow geometry types is still a work in progress. For more details, see issue #2800.

4.2 Packages depending on PyGMT

Various packages rely on PyGMT for geospatial data processing, analysis, and visualization. Below is an incomplete list (in no particular order) of these tools.

Note

If your package relies on PyGMT, please let us know or *add it by yourself*.

This page was adapted from GeoPandas's Ecosystem page.

CHAPTER

FIVE

PRESENTATIONS

These are international presentations, including both talks and posters, on the development of PyGMT (previously "GMT/Python"):

AGU 2024

PyGMT - Accessing and Integrating GMT with Python and the Scientific Python Ecosystem

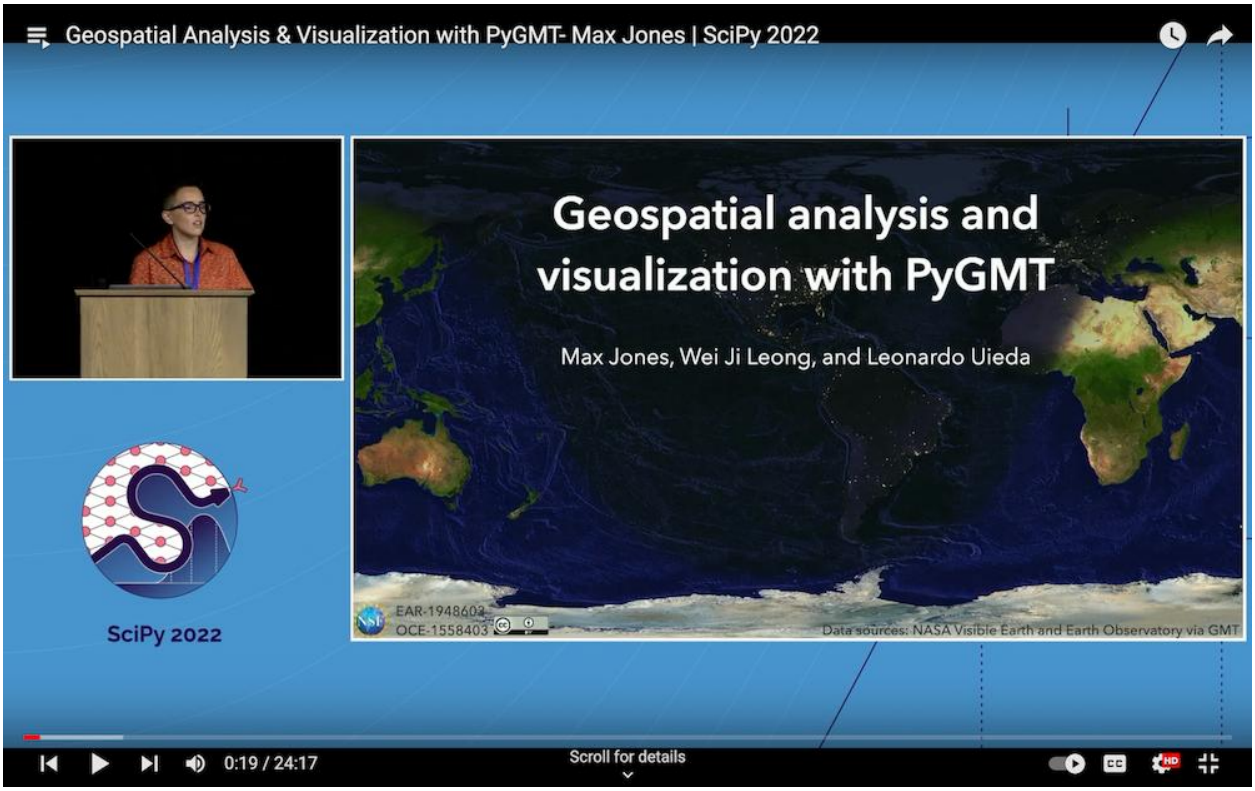
Yvonne Fröhlich, Dongdong Tian, Wei Ji Leong, Max Jones, Michael Grund

doi: 10.6084/m9.figshare.28049495

Slides Code

SciPy 2022

Geospatial Analysis & Visualization with PyGMT



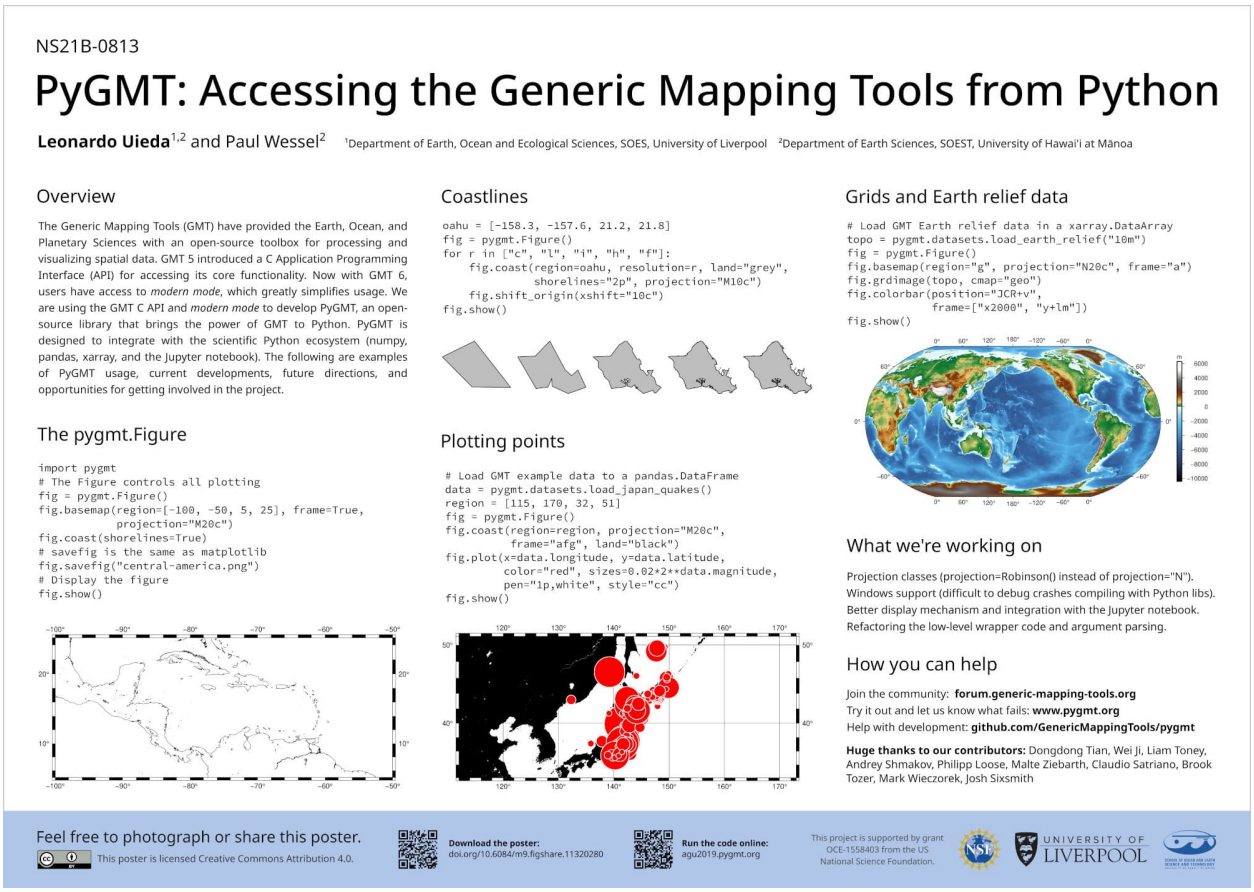
Max Jones, Wei Ji Leong, Leonardo Uieda

doi: 10.6084/m9.figshare.20483793

Slides Recording Code

AGU 2019

PyGMT: Accessing the Generic Mapping Tools from Python



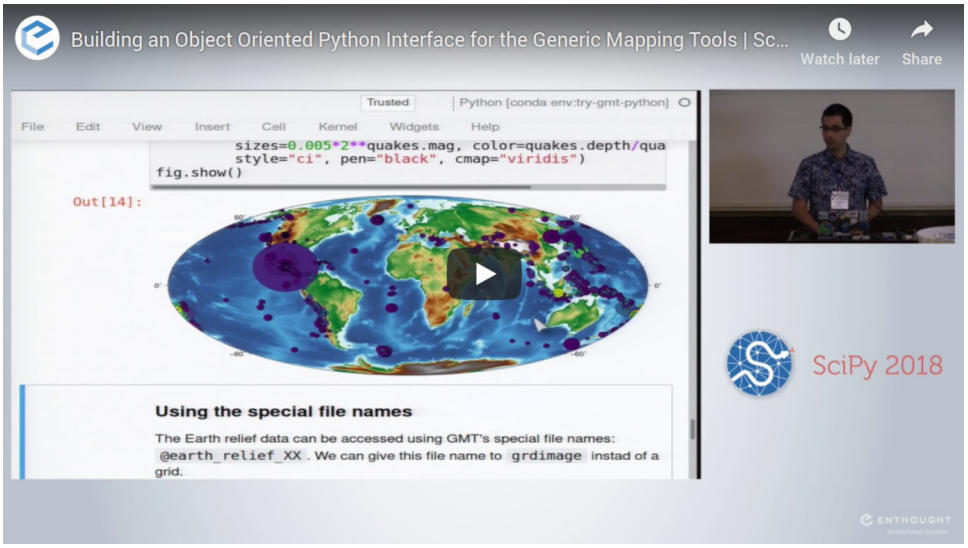
Leonardo Uieda, Paul Wessel

doi: 10.6084/m9.figshare.11320280

Poster

SciPy 2018

Building an object-oriented Python interface for the Generic Mapping Tools



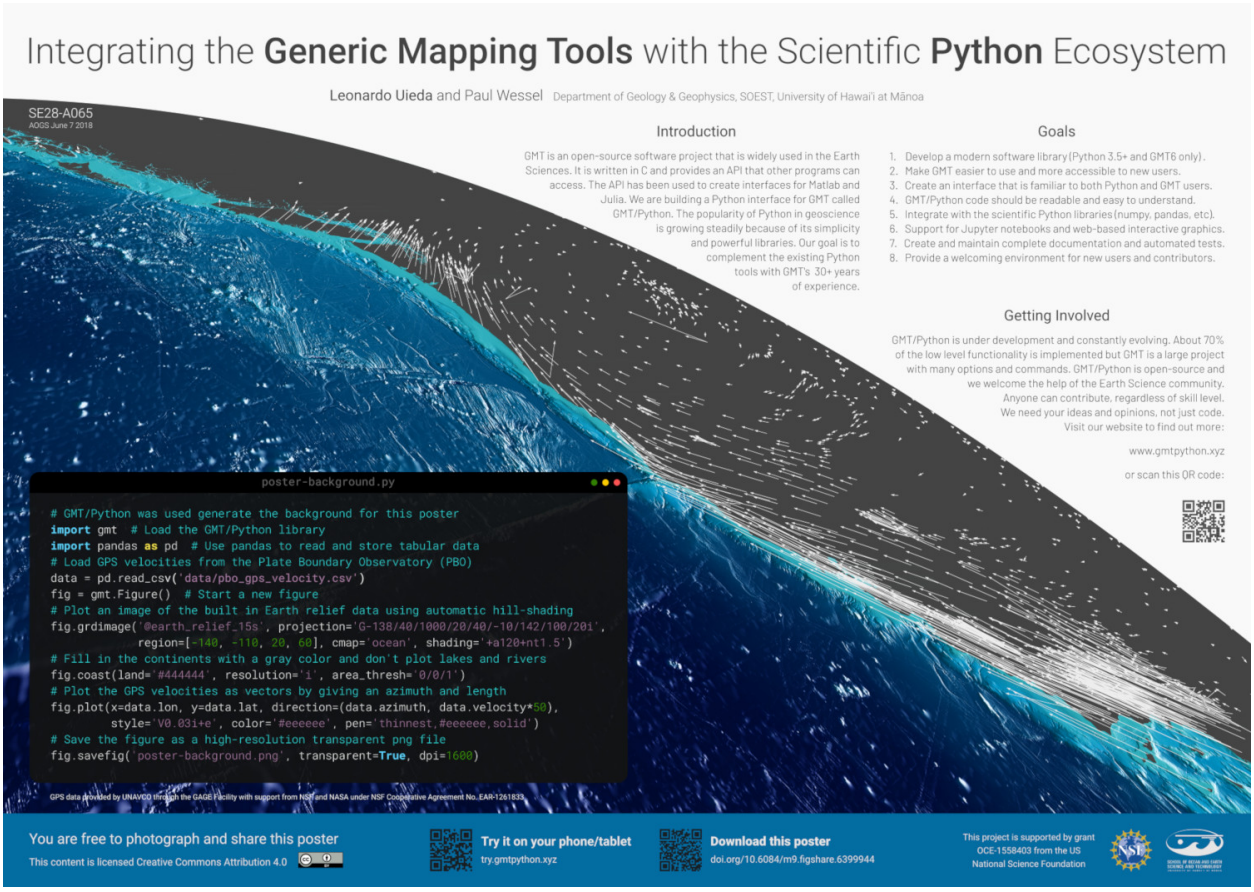
Leonardo Uieda, Paul Wessel

doi: 10.6084/m9.figshare.6814052

Slides Recording Code

AOGS Annual Meeting 2018

Integrating the Generic Mapping Tools with the Scientific Python Ecosystem



Leonardo Uieda, Paul Wessel

doi: 10.6084/m9.figshare.6399944

Poster

SciPy 2017

Bringing the Generic Mapping Tools to Python

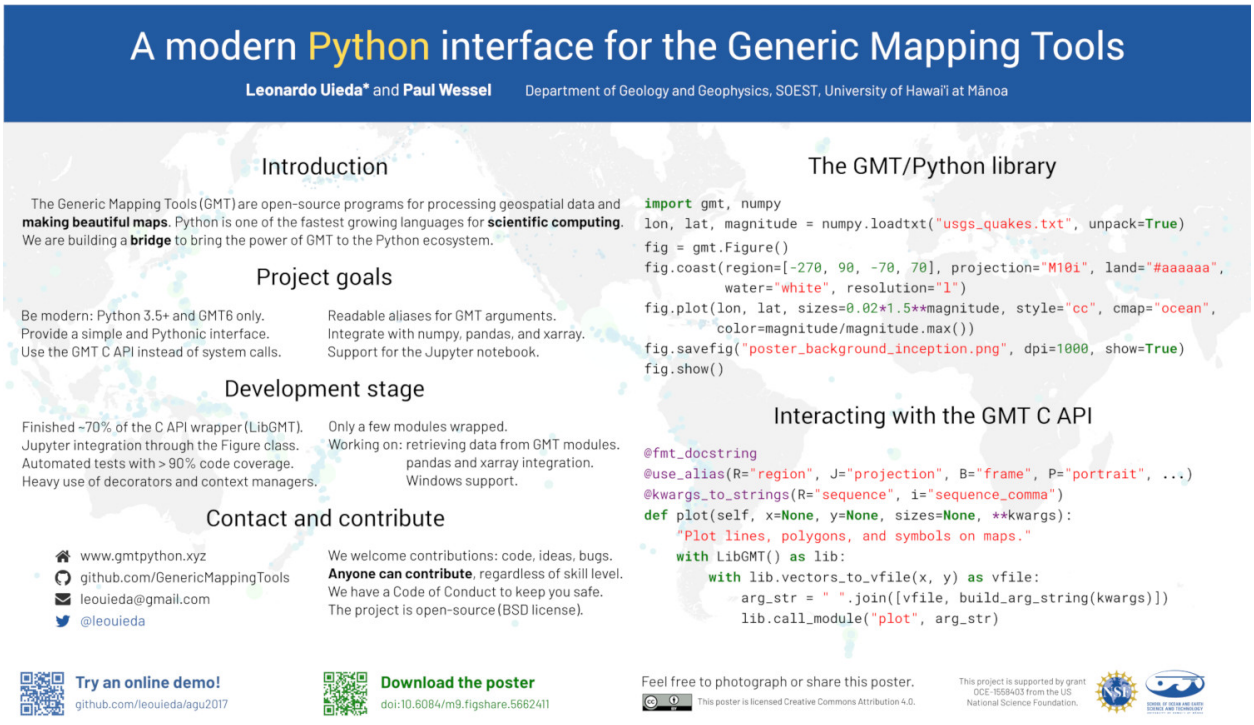
Leonardo Uieda, Paul Wessel

doi: 10.6084/m9.figshare.7635833

Slides Recording Code

AGU 2017

A modern Python interface for the Generic Mapping Tools



Leonardo Uieda, Paul Wessel

doi: 10.6084/m9.figshare.5662411

Poster

CHAPTER

SIX

CITING PYGMT

If you used PyGMT in your research please consider citing PyGMT. You can either cite the PyGMT paper or the Zenodo archive of the PyGMT software.

It is also strongly recommended to cite the GMT 6 paper (which PyGMT wraps around). Note that some modules like `dimfilter`, `surface`, and `x2sys` also have their dedicated citations. Further information for all these can be found at https://www.generic-mapping-tools.org/cite.

The BibTeX entry for the PyGMT paper is:

```
@article{pygmt_2026_2026GC013105,
  author       = {Tian, Dongdong and
                  Fröhlich, Yvonne and
                  Leong, Wei Ji and
                  Grund, Michael and
                  Schlitzer, William and
                  Jones, Max and
                  Uieda, Leonardo and
                  Luis, Joaquim M. F.},
  title        = {{PyGMT: Bridging Python and the Generic Mapping Tools for␣
↪Geospatial Visualization and Analysis}},
  journal      = {Geochemistry, Geophysics, Geosystems},
  year         = 2026,
  volume       = {27},
  number       = {7},
  pages        = {e2026GC013105},
  doi          = {10.1029/2026GC013105},
  url          = {https://doi.org/10.1029/2026GC013105},
}
```

The BibTeX entry for the latest Zenodo archive, i.e., release 0.19.0, of the PyGMT software is:

```
@software{pygmt_2026_19398871,
  author       = {Tian, Dongdong and
                  Leong, Wei Ji and
                  Fröhlich, Yvonne and
                  Grund, Michael and
                  Schlitzer, William and
                  Jones, Max and
                  Toney, Liam and
                  Yao, Jiayuan and
                  Tong, Jing-Hui and
                  Magen, Yohai and
```

(continues on next page)

(continued from previous page)

```
                  Materna, Kathryn and
                  Belem, Andre and
                  Newton, Tyler and
                  Anant, Abhishek and
                  Ziebarth, Malte and
                  Quinn, Jamie and
                  He, Xingchen and
                  Uieda, Leonardo and
                  Wessel, Paul},
  title        = {{PyGMT: A Python interface for the Generic Mapping Tools}},
  month        = jul,
  year         = 2026,
  publisher    = {Zenodo},
  version      = {0.19.0},
  doi          = {10.5281/zenodo.19398871},
  url          = {https://doi.org/10.5281/zenodo.19398871}
}
```

To cite a specific version of PyGMT, go to the Zenodo archive, select the specific version, and use the "Export to BibTeX" function there.

CHAPTER

SEVEN

TUTORIALS

These examples teach us how to complete various tasks using PyGMT!

7.1 Basics

7.1.1 Coastlines and borders

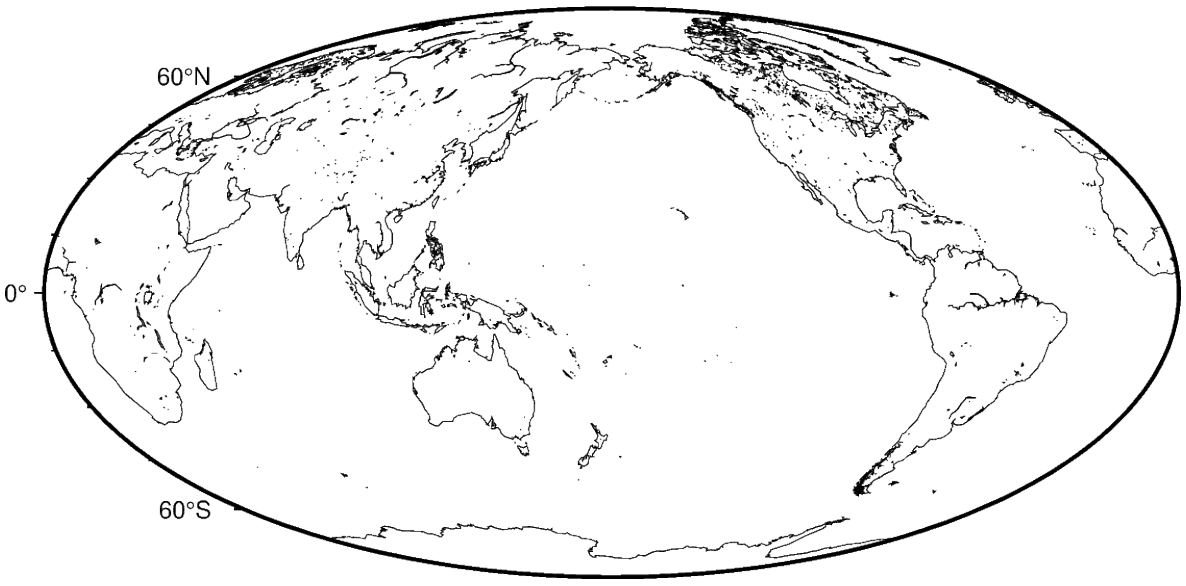
Plotting coastlines and borders is handled by *pygmt.Figure.coast*.

```
import pygmt
```

Shorelines

Use the `shorelines` parameter to plot only the shorelines:

```
fig = pygmt.Figure()
fig.basemap(region="g", projection="W15c", frame=True)
fig.coast(shorelines=True)
fig.show()
```



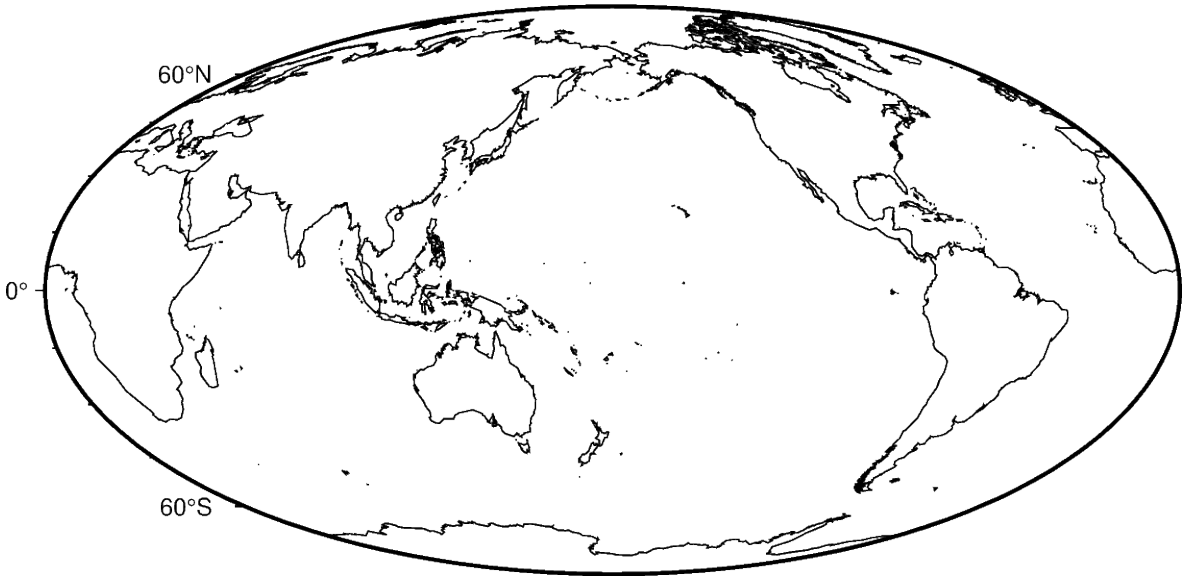
The shorelines are divided in 4 levels:

1. coastline

2. lakeshore
3. island-in-lake shore
4. lake-in-island-in-lake shore

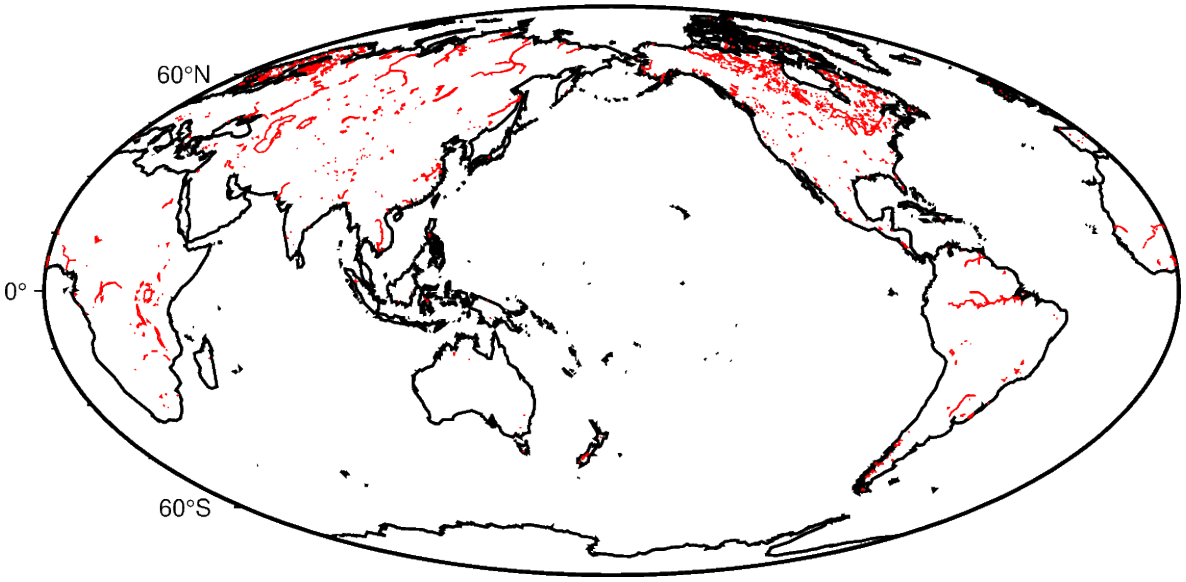
You can specify which level you want to plot by passing the level number and a GMT pen configuration. For example, to plot just the coastlines with 0.5p thickness and black lines:

```
fig = pygmt.Figure()
fig.basemap(region="g", projection="W15c", frame=True)
fig.coast(shorelines="1/0.5p,black")
fig.show()
```



You can specify multiple levels (with their own pens) by passing a list to `shorelines`:

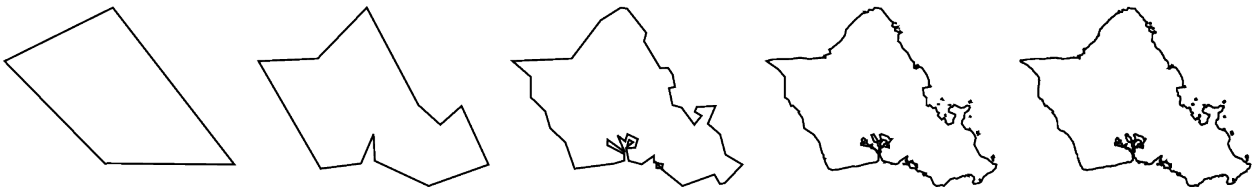
```
fig = pygmt.Figure()
fig.basemap(region="g", projection="W15c", frame=True)
fig.coast(shorelines=["1/1p,black", "2/0.5p,red"])
fig.show()
```



Resolutions

The coastline database comes with 5 resolutions: `"full"`, `"high"`, `"intermediate"`, `"low"`, and `"crude"`. The resolution drops by 80% between levels. The `resolution` parameter defaults to `"auto"` to automatically select the best resolution given the chosen map scale.

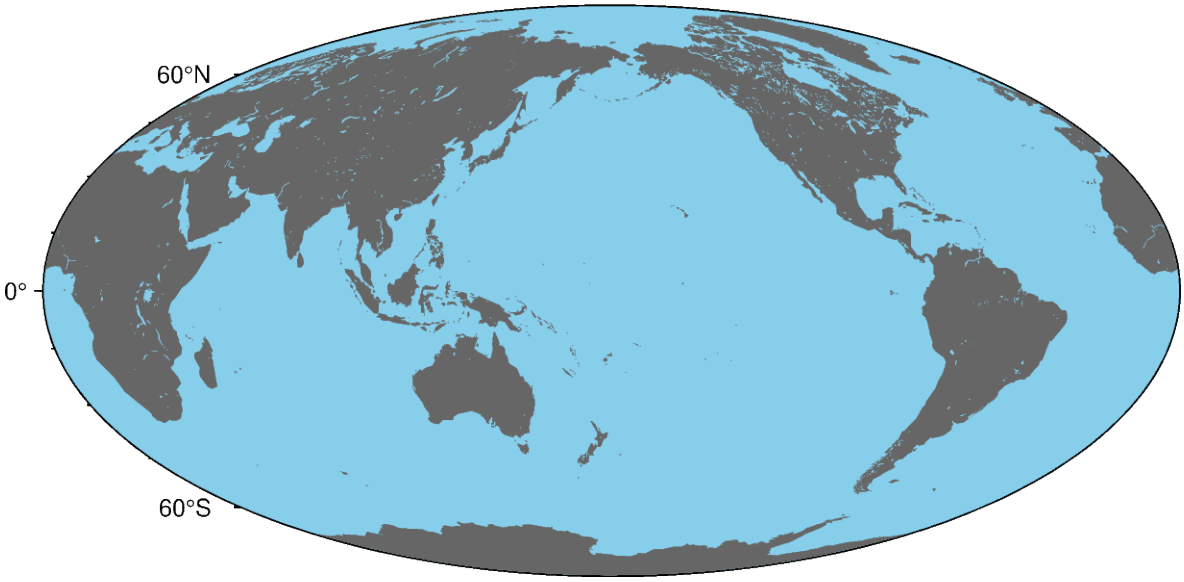
```
oahu = [-158.3, -157.6, 21.2, 21.8]
fig = pygmt.Figure()
for res in ["crude", "low", "intermediate", "high", "full"]:
    fig.coast(resolution=res, shorelines="1p", region=oahu, projection="M5c")
    fig.shift_origin(xshift="5c")
fig.show()
```



Land and water

Use the `land` and `water` parameters to specify a fill color for land and water bodies. The colors can be given by name or hex codes (like the ones used in HTML and CSS):

```
fig = pygmt.Figure()
fig.basemap(region="g", projection="W15c", frame=True)
fig.coast(land="#666666", water="skyblue")
fig.show()
```



Total running time of the script: (0 minutes 1.146 seconds)

7.1.2 Frames, ticks, titles, and labels

Setting frame, ticks, titles, and labels is handled by the `frame` parameter that many plotting methods of the *pygmt.Figure* class accept. This tutorial focuses on setting the `frame` parameter using the *pygmt.params.Axis* and *pygmt.params.Frame* classes.

```
import pygmt
from pygmt.params import Axis, Frame
```

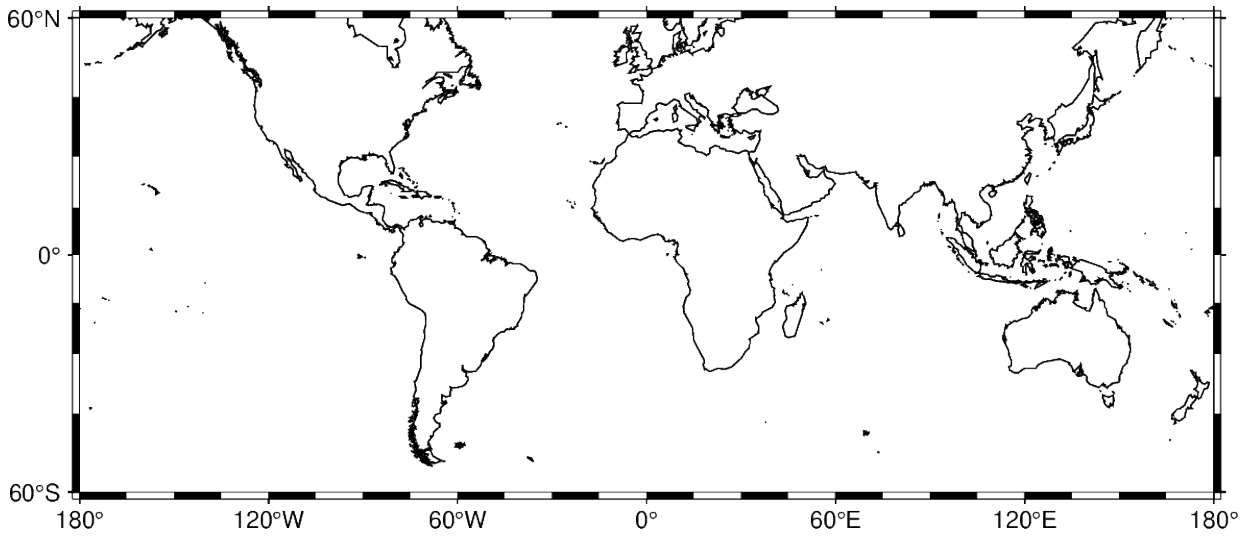
Plot frame

By default, PyGMT does not add a frame to your plot. For example, we can plot the coastlines of the world with a Mercator projection:

```
fig = pygmt.Figure()
fig.coast(shorelines="1/0.5p", region=[-180, 180, -60, 60], projection="M15c")
fig.show()
```

To add the default GMT frame style to the plot, use `frame=True` in *pygmt.Figure.basemap* or another plotting method that accepts a `frame` parameter:

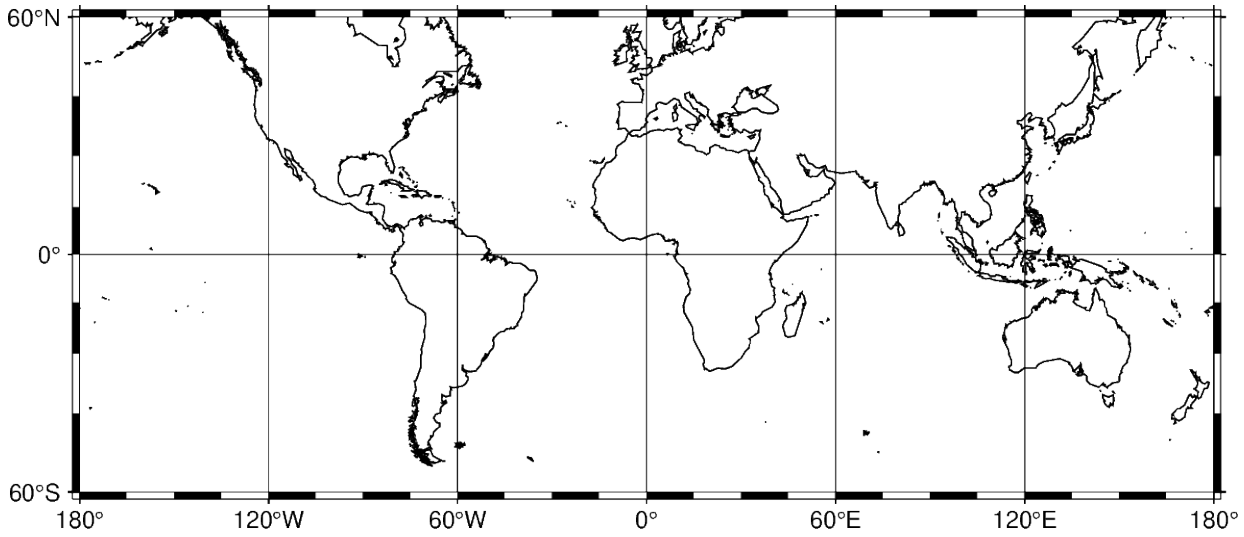
```
fig = pygmt.Figure()
fig.coast(shorelines="1/0.5p", region=[-180, 180, -60, 60], projection="M15c")
fig.basemap(frame=True)
fig.show()
```



Annotations, tick marks and grid lines

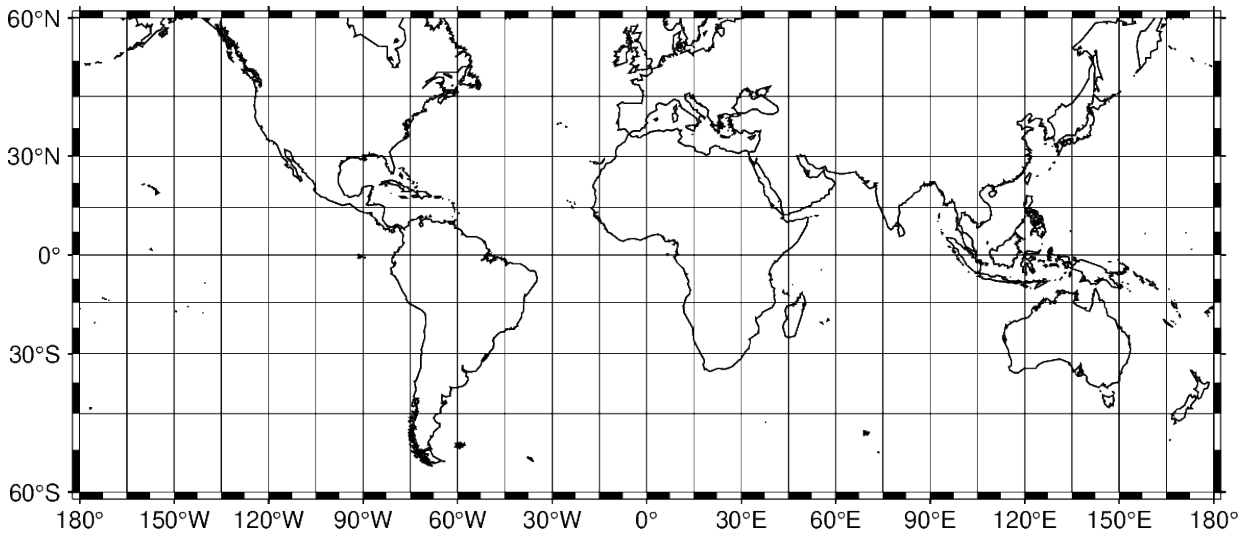
The default frame style includes tick labels (called annotations in GMT) and tick marks at intervals determined by GMT, but doesn't include grid lines. To control the presence and intervals of annotations, tick marks, and grid lines, you can use the *pygmt.params.Axis* class. The `annot`, `tick`, and `grid` attributes of *pygmt.params.Axis* control the presence and intervals of annotations, tick marks, and grid lines, respectively. Setting them to `True` adds them at intervals determined by GMT:

```
fig = pygmt.Figure()
fig.coast(shorelines="1/0.5p", region=[-180, 180, -60, 60], projection="M15c")
fig.basemap(frame=Axis(annot=True, tick=True, grid=True))
fig.show()
```



To set specific intervals, pass values to `annot`, `tick`, and `grid`. In the example below, the annotation, tick, and gridline intervals are set to 30, 7.5, and 15 degrees, respectively.

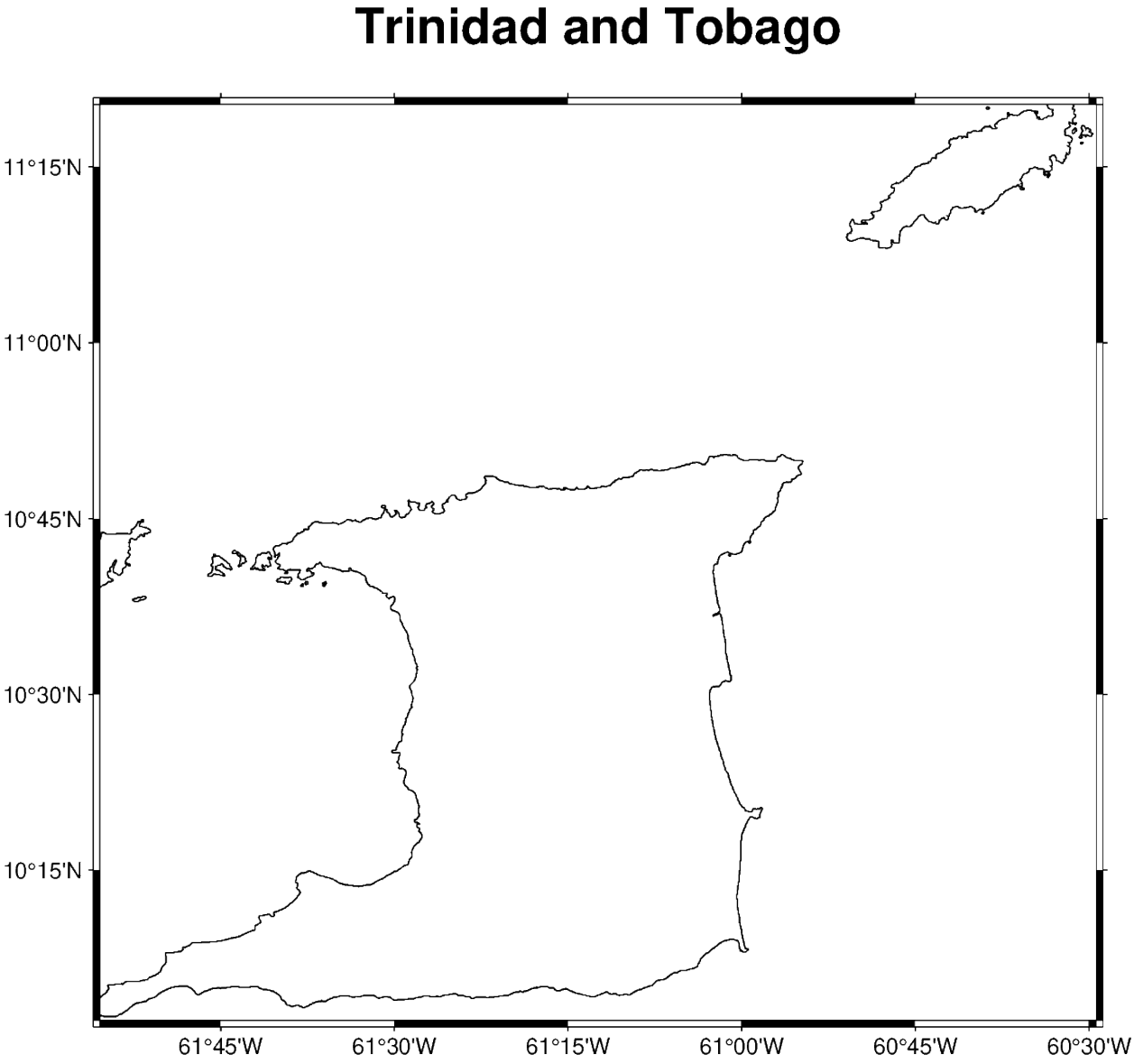
```
fig = pygmt.Figure()
fig.coast(shorelines="1/0.5p", region=[-180, 180, -60, 60], projection="M15c")
fig.basemap(frame=Axis(annot=30, tick=7.5, grid=15))
fig.show()
```



Title

The *pygmt.params.Frame* class lets us configure frame-wide settings such as titles. Combine it with an `Axis` object to keep automatic annotations.

```
fig = pygmt.Figure()
# region="TT" specifies Trinidad and Tobago using the ISO country code
fig.coast(shorelines="1/0.5p", region="TT", projection="M15c")
fig.basemap(frame=Frame(title="Trinidad and Tobago", axis=Axis(annot=True)))
fig.show()
```



Axis labels

Axis labels, in GMT simply called labels, can be set through the `xaxis` and `yaxis` parameters of `Frame`. The map boundaries (or plot axes) are named as West/west/left (**W**, **w**, **l**), South/south/bottom (**S**, **s**, **b**), North/north/top (**N**, **n**, **t**), and East/east/right (**E**, **e**, **r**) sides of a figure. Uppercase letters (**W**, **S**, **N**, **E**) draw axes with annotations and tick marks, lowercase letters (**w**, **s**, **n**, **e**) draw axes with tick marks only, and **l**, **b**, **t**, **r** draw plain axis lines without ticks or annotations. A frame like `Frame(axes="WS")` draws annotated west and south axes only.

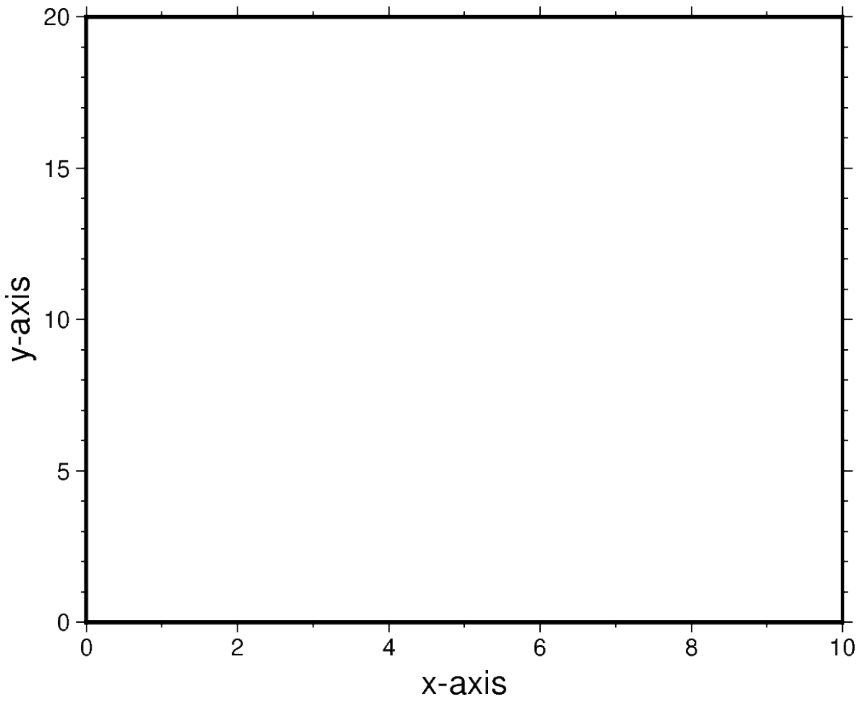
The example below uses a Cartesian projection, as GMT does not allow labels to be set for geographic maps.

```
fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 20],
    projection="X10c/8c",
    # Plot axis with tick marks, annotations, and labels on the West and South axes.
    # Plot axis with tick marks on the north and east axes
    frame=Frame(
```

(continues on next page)

(continued from previous page)

```
        axes="WSne",
        xaxis=Axis(annot=True, tick=True, label="x-axis"),
        yaxis=Axis(annot=True, tick=True, label="y-axis"),
    ),
)
fig.show()
```



Total running time of the script: (0 minutes 1.742 seconds)

7.1.3 Plotting data points

GMT shines when it comes to plotting data on a map. We can use some sample data that is packaged with GMT to try this out. PyGMT provides access to these datasets through the `pygmt.datasets` package. If you don't have the data files already, they are automatically downloaded and saved to a cache directory the first time you use them (usually `~/.gmt/cache`).

```
import io

import pygmt
from pygmt.params import Box, Position
```

For example, let's load the sample dataset of tsunami generating earthquakes around Japan using *`pygmt.datasets.load_sample_data`*. The data are loaded as a `pandas.DataFrame`.

```
data = pygmt.datasets.load_sample_data(name="japan_quakes")
data.head()
```

Set the region for the plot to be slightly larger than the data bounds.

```
region = [
    data.longitude.min() - 1,
```

(continues on next page)

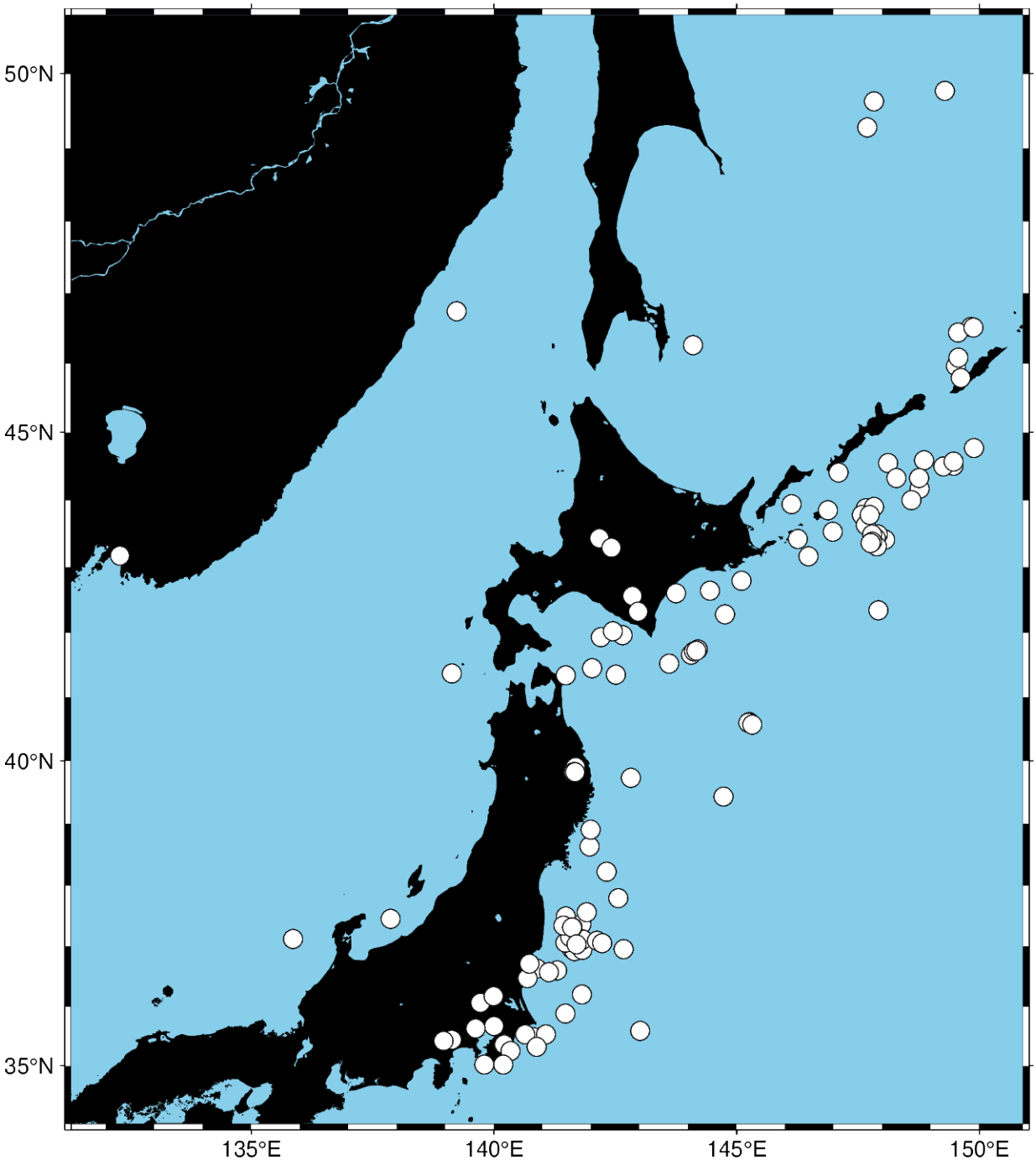
(continued from previous page)

```
    data.longitude.max() + 1,
    data.latitude.min() - 1,
    data.latitude.max() + 1,
]
region
```

```
[np.float64(131.29), np.float64(150.89), np.float64(34.02), np.float64(50.77)]
```

We'll use the *pygmt.Figure.plot* method to plot circles on the earthquake epicenters.

```
fig = pygmt.Figure()
fig.basemap(region=region, projection="M15c", frame=True)
fig.coast(land="black", water="skyblue")
fig.plot(x=data.longitude, y=data.latitude, style="c0.3c", fill="white", pen="black")
fig.show()
```

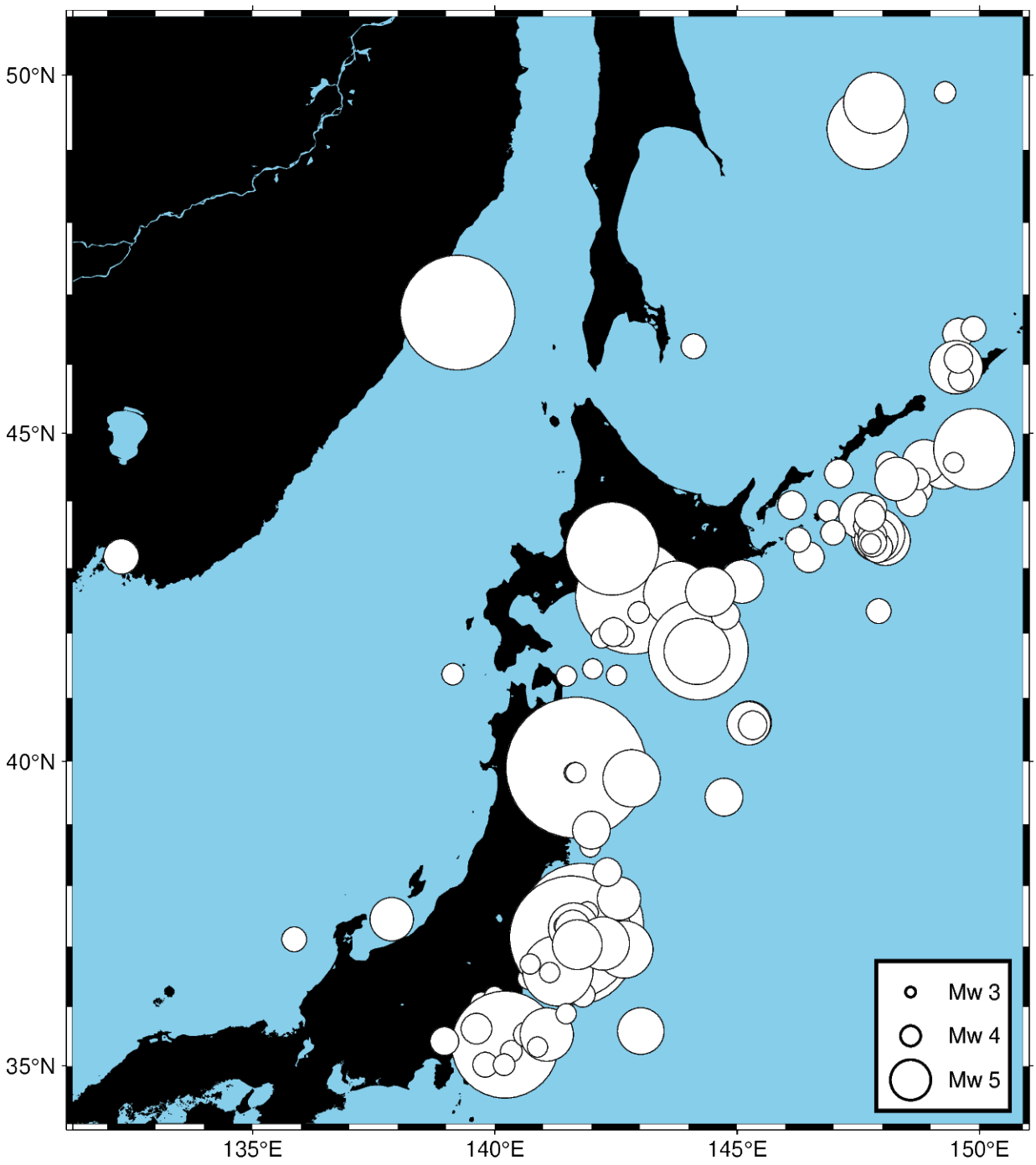


We used the style `c0.3c` which means "circles with a diameter of 0.3 centimeters". The `pen` parameter controls the outline of the symbols and the `fill` parameter controls the fill.

We can map the size of the circles to the earthquake magnitude by passing an array to the `size` parameter. Because the magnitude is on a logarithmic scale, it helps to show the differences by scaling the values using a power law.

A legend for the size of the circles can not be added automatically. But users can create an `io.StringIO` object, which can be passed to the `spec` parameter of *`pygmt.Figure.legend`*. For details on creating legends, see the tutorial *multiple-column legend*.

```
fig = pygmt.Figure()
fig.basemap(region=region, projection="M15c", frame=True)
fig.coast(land="black", water="skyblue")
fig.plot(
    x=data.longitude,
    y=data.latitude,
    size=0.02 * (2**data.magnitude),
    style="cc",
    fill="white",
    pen="black",
)
legend = io.StringIO(
    "\n".join(f"S 0.4 c {0.02 * 2**m:.2f} - 1p 1 Mw {m}" for m in [3, 4, 5])
)
fig.legend(
    spec=legend,
    position=Position("BR", offset=0.2),
    line_spacing=2,
    box=Box(fill="white", pen="black"),
)
fig.show()
```



Notice that we didn't include the size in the `style` parameter this time, just the symbol `c` (circles) and the unit `c` (centimeters). So in this case, the size will be interpreted as being in centimeters.

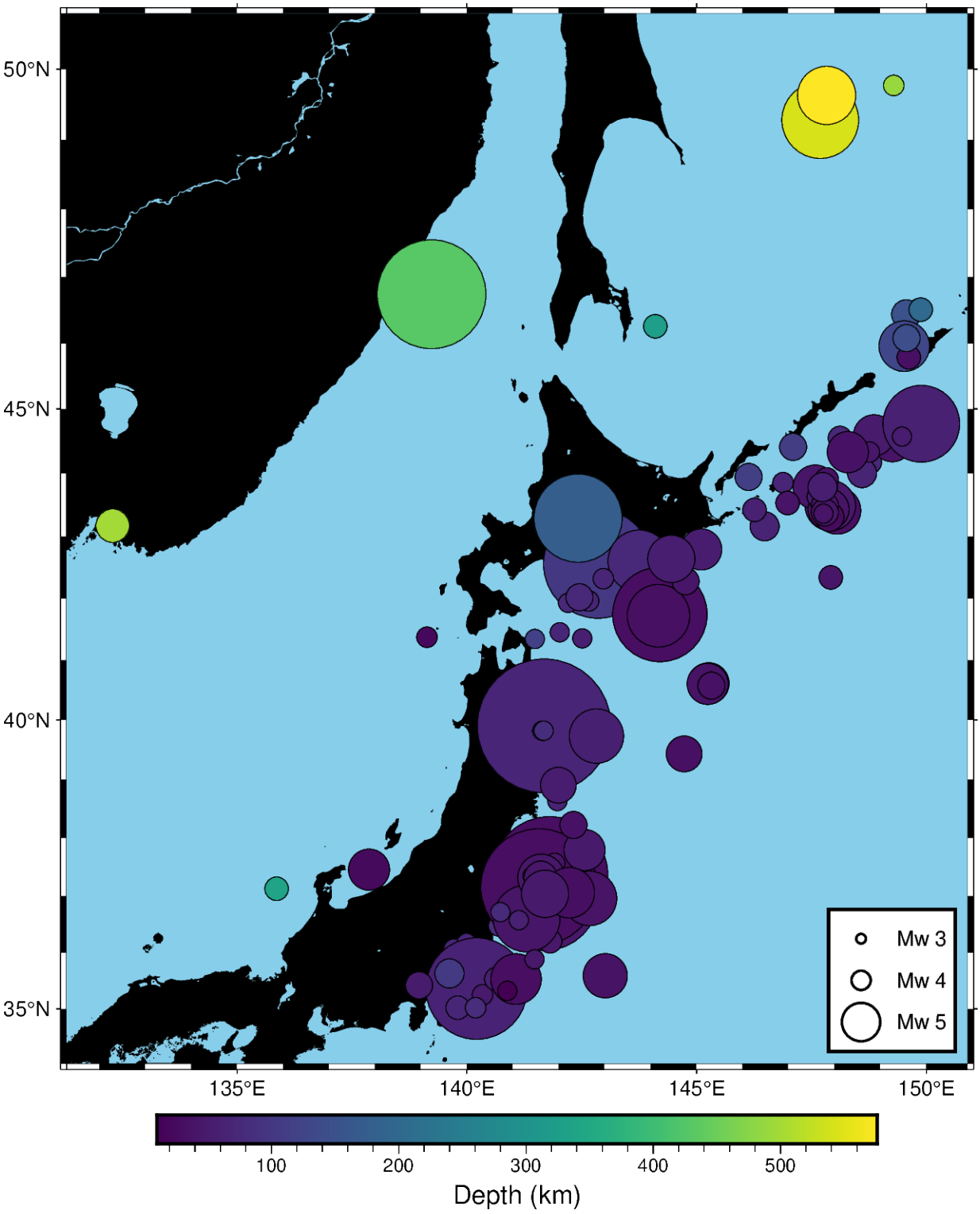
We can also map the colors of the markers to the depths by passing an array to the `fill` parameter and providing a colormap name (`cmap`). We can even use the new matplotlib colormap "viridis". Here, we first create a continuous colormap ranging from the minimum depth to the maximum depth of the earthquakes using *pygmt.makecpt*, then set `cmap=True` in *pygmt.Figure.plot* to use the colormap. At the end of the plot, we also plot a colorbar showing the colormap used in the plot.

```
fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

```
fig.basemap(region=region, projection="M15c", frame=True)
fig.coast(land="black", water="skyblue")
pygmt.makecpt(
    cmap="matplotlib/viridis", series=[data.depth_km.min(), data.depth_km.max()]
)
fig.plot(
    x=data.longitude,
    y=data.latitude,
    size=0.02 * 2**data.magnitude,
    fill=data.depth_km,
    cmap=True,
    style="cc",
    pen="black",
)
fig.colorbar(annot=True, tick=True, label="Depth (km)")
fig.legend(
    spec=legend,
    position=Position("BR", offset=0.2),
    line_spacing=2,
    box=Box(fill="white", pen="black"),
)
fig.show()
```



Total running time of the script: (0 minutes 1.683 seconds)

7.1.4 Plotting lines

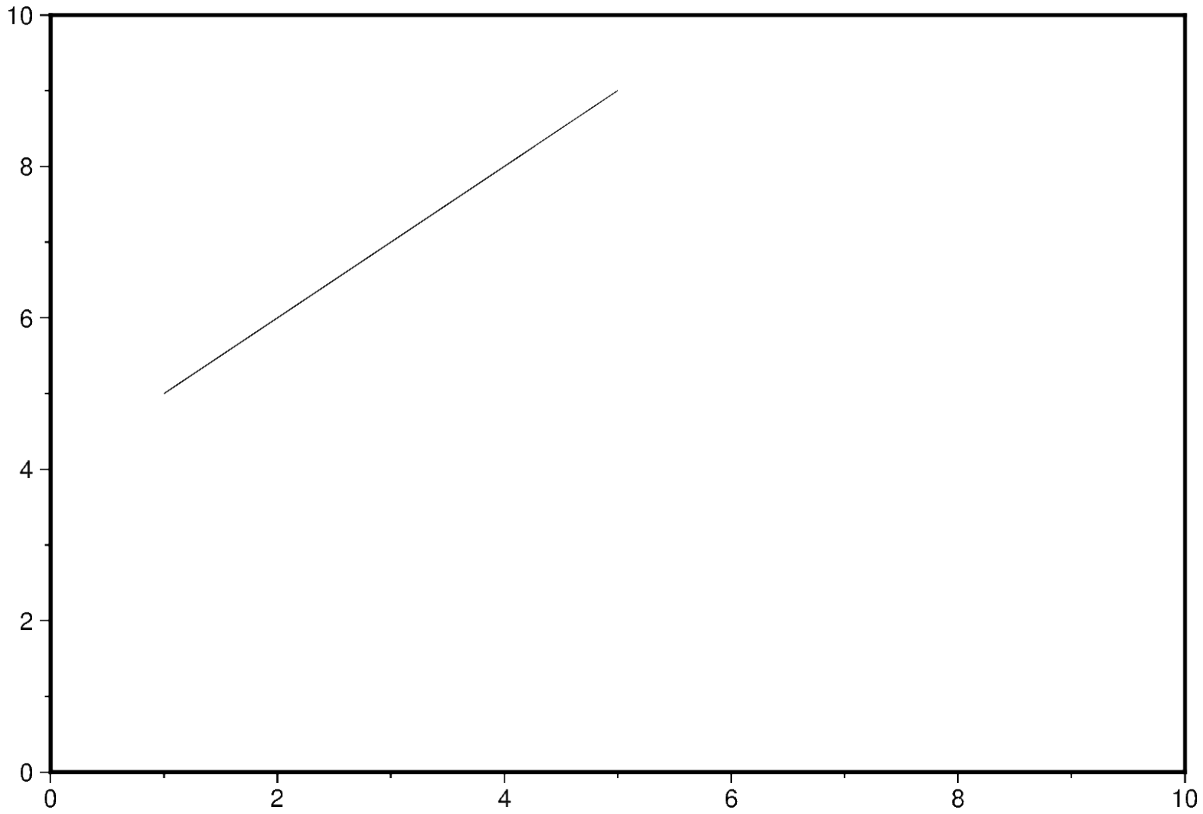
Plotting lines is handled by the *pygmt.Figure.plot* method.

```
import pygmt
```

Plot lines

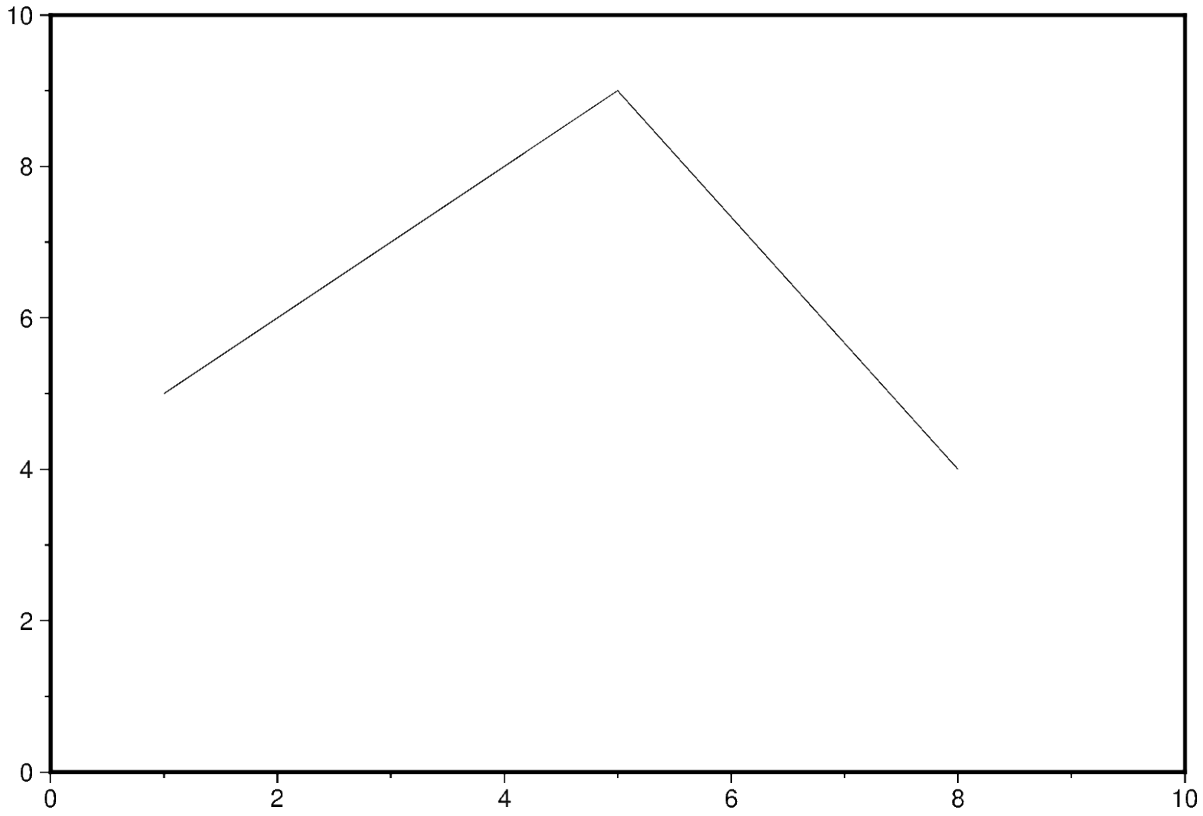
Create a Cartesian figure using the *pygmt.Figure.basemap* method. Pass lists containing two values to the `x` and `y` parameters of the *pygmt.Figure.plot* method. By default, a 0.25-point thick, black, solid line is drawn between these two data points.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

fig.plot(x=[1, 5], y=[5, 9])

fig.show()
```



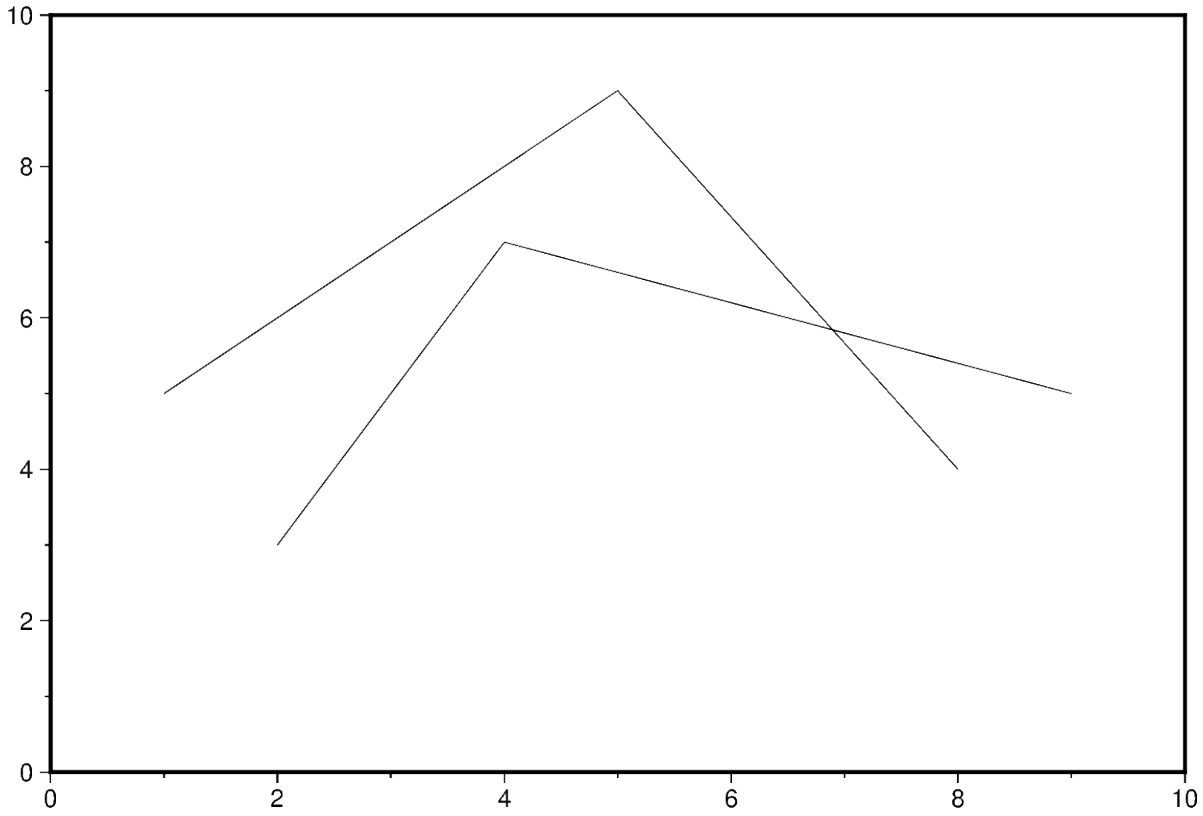
Additional line segments can be added by including more data points in the lists passed to `x` and `y`.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

fig.plot(x=[1, 5, 8], y=[5, 9, 4])

fig.show()
```



To plot multiple lines, *pygmt.Figure.plot* needs to be used for each line separately.

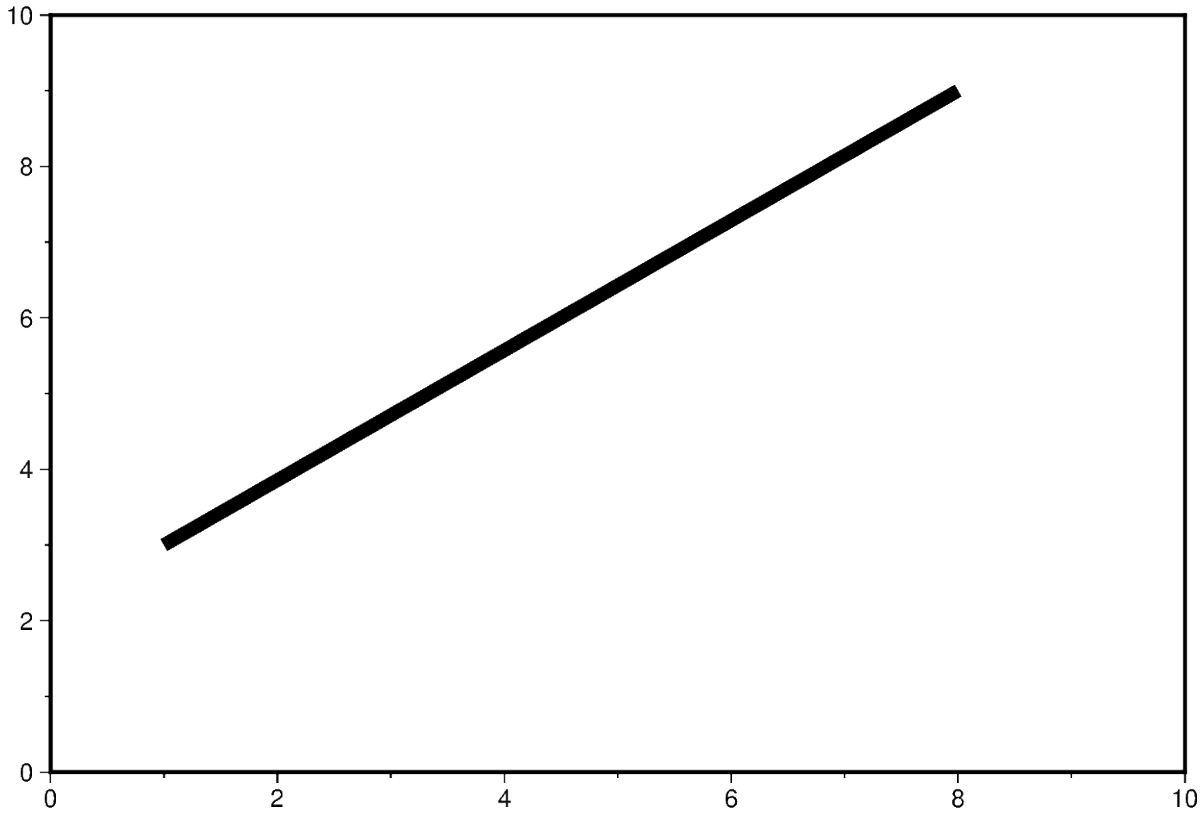
```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

fig.plot(x=[1, 5, 8], y=[5, 9, 4])
fig.plot(x=[2, 4, 9], y=[3, 7, 5])

fig.show()
```



Change line attributes

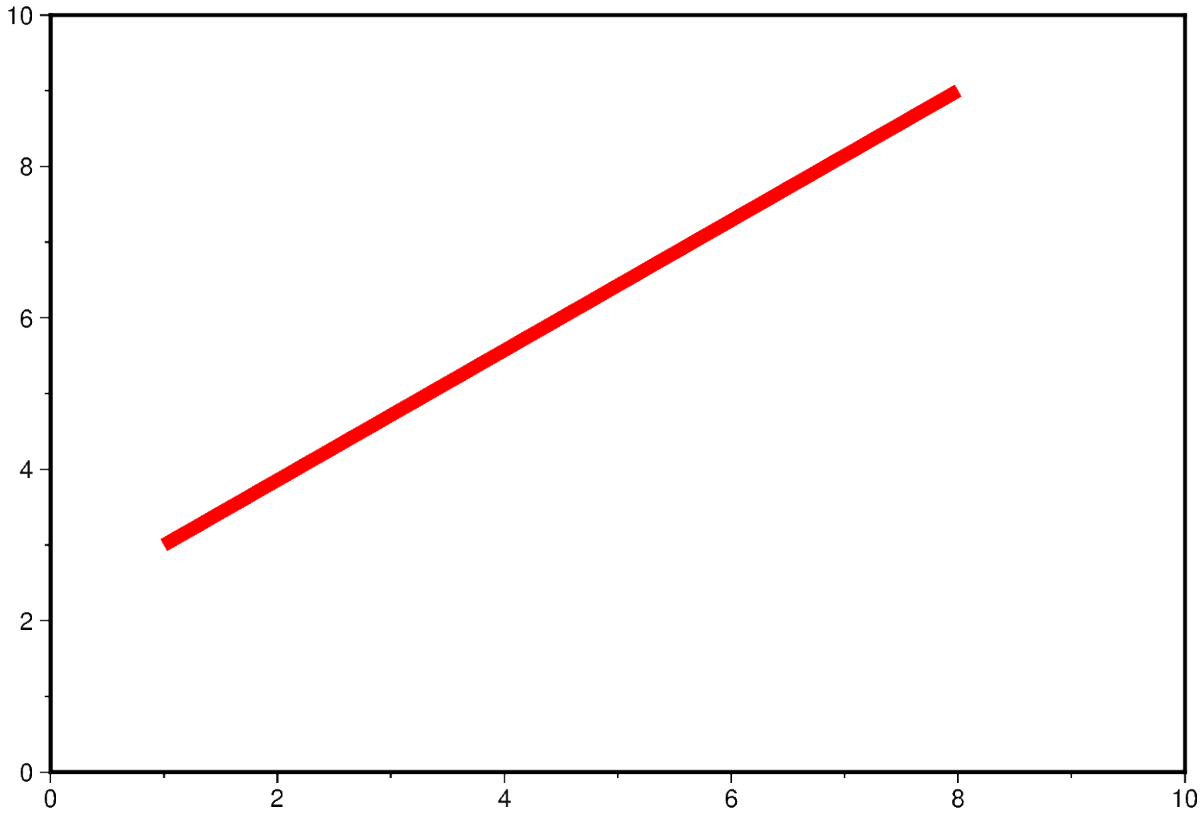
The line attributes can be set by the `pen` parameter which takes a string argument with the optional values *width*,*color*,*style*.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

# Set the pen width to "5p" (5 points), and use the default color "black" and the
# default style "solid"
fig.plot(x=[1, 8], y=[3, 9], pen="5p")

fig.show()
```



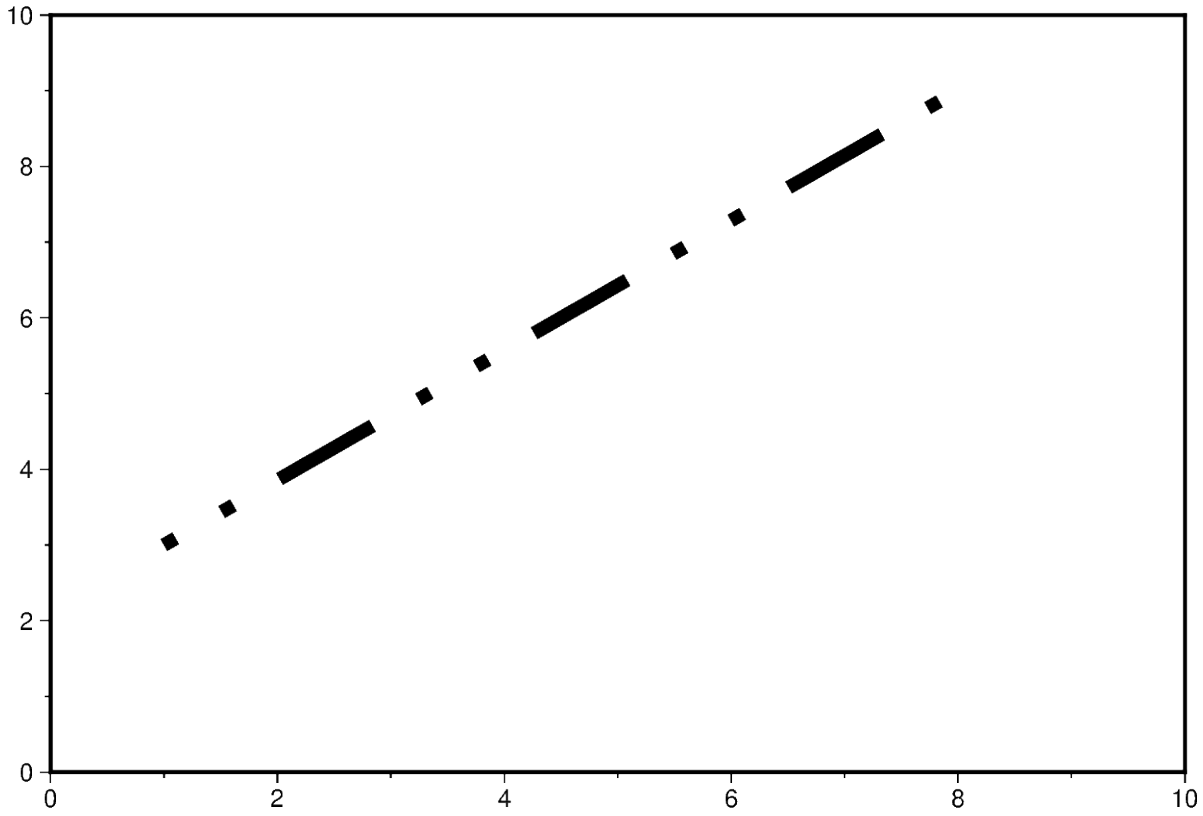
The line color can be set and is added after the line width to the `pen` parameter.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

# Set the line color to "red", use the default style "solid"
fig.plot(x=[1, 8], y=[3, 9], pen="5p,red")

fig.show()
```



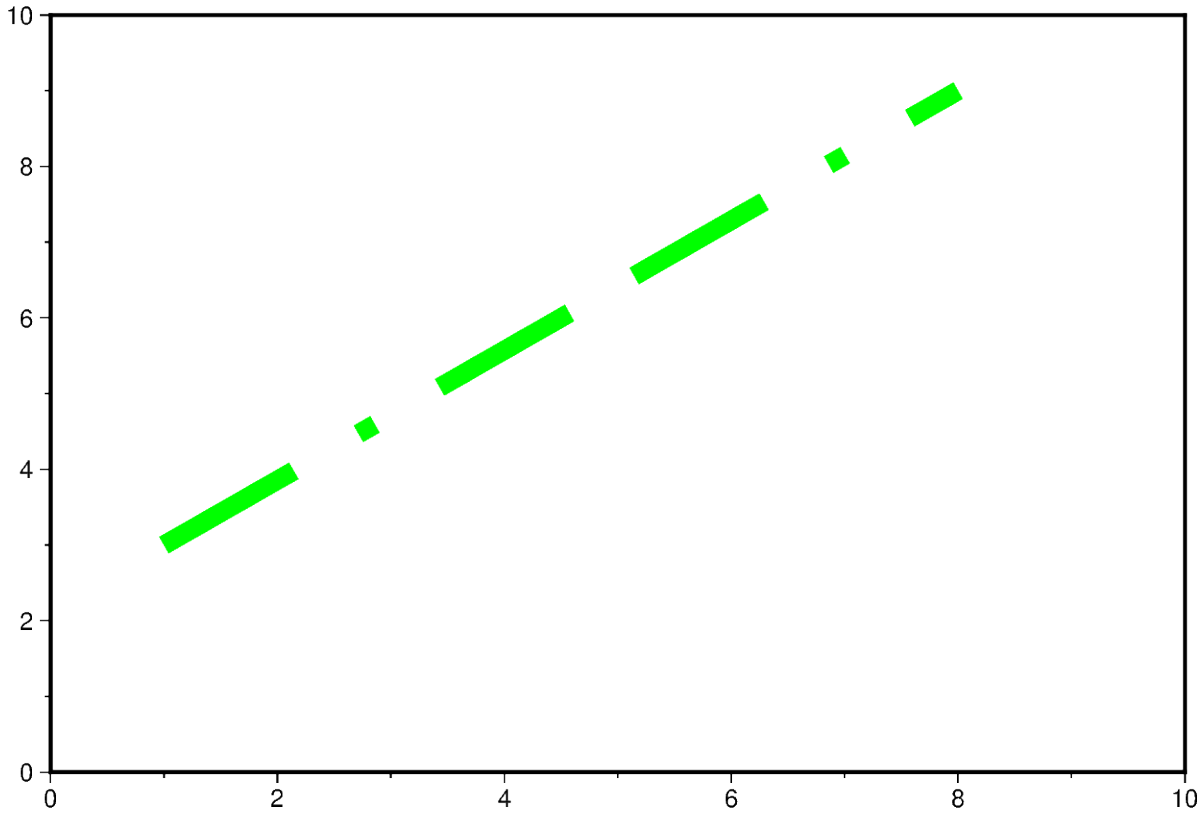
The line style can be set and is added after the line width or color to the `pen` parameter.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

# Set the line style to "..-" (dot dot dash), use the default color "black"
fig.plot(x=[1, 8], y=[3, 9], pen="5p,..-")

fig.show()
```



The line width, color, and style can all be set in the same `pen` parameter. For a gallery example showing other `pen` settings, see *Line styles*.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 10], projection="X15c/10c", frame=True)

# Draw a 7-point thick, green line with style "-.-" (dash dot dash)
fig.plot(x=[1, 8], y=[3, 9], pen="7p,green,-.-")

fig.show()
```



Total running time of the script: (0 minutes 1.478 seconds)

7.1.5 Plotting polygons

Plotting polygons is handled by the *pygmt.Figure.plot* method.

This tutorial focuses on input data given as NumPy arrays. Besides NumPy arrays, array-like objects are supported. Here, a polygon is a closed shape defined by a series of data points with x- and y-coordinates, connected by line segments, with the start and end points being identical. For plotting a `geopandas.GeoDataFrame` object with polygon geometries, e.g., to create a choropleth map, see the gallery example *Choropleth map*.

```
import numpy as np
import pygmt
```

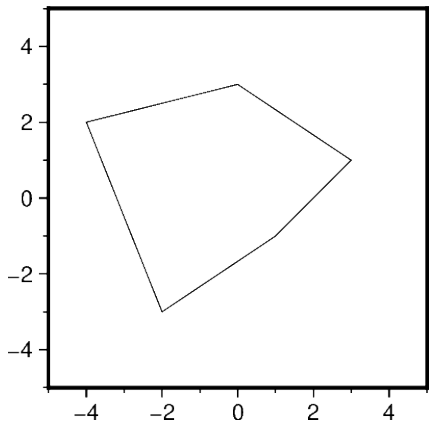
Plot polygons

Set up sample data points as NumPy arrays for the x- and y-values.

```
x = np.array([-2, 1, 3, 0, -4, -2])
y = np.array([-3, -1, 1, 3, 2, -3])
```

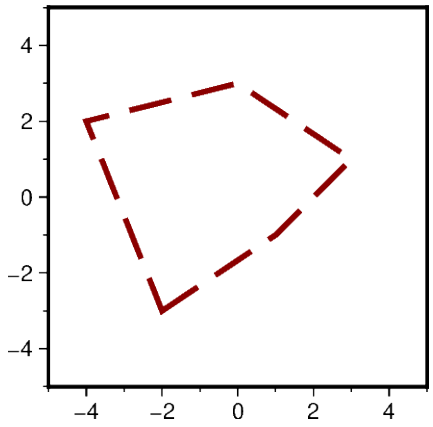
Create a Cartesian plot via the *pygmt.Figure.basemap* method. Pass arrays to the `x` and `y` parameters of the *pygmt.Figure.plot* method. Without further adjustments, lines are drawn between the data points. By default, the lines are 0.25-point thick, black, and solid. In this example, the data points are chosen to make the lines form a polygon.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.plot(x=x, y=y)
fig.show()
```



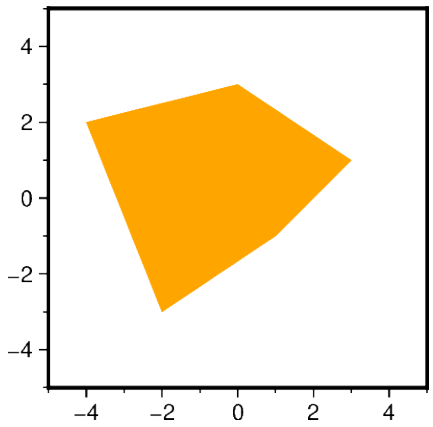
The `pen` parameter can be used to adjust the lines or outline of the polygon. The argument passed to `pen` is one string with the comma-separated optional values *width*,*color*,*style*.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
# Use a 2-point thick, darkred, dashed outline
fig.plot(x=x, y=y, pen="2p,darkred,dashed")
fig.show()
```



Use the `fill` parameter to fill the polygon with a color or *`pattern`*. Note, that there are no lines drawn between the data points by default if `fill` is used. Use the `pen` parameter to add an outline around the polygon.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
# Fill the polygon with color "orange"
fig.plot(x=x, y=y, fill="orange")
fig.show()
```



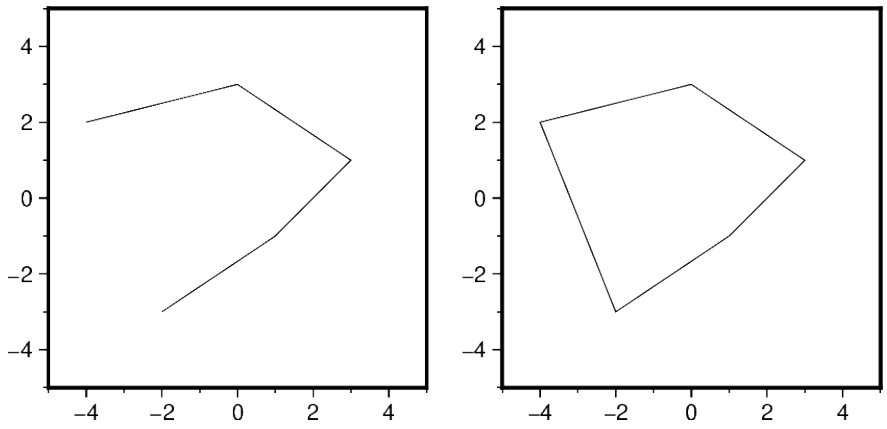
Close polygons

Set up sample data points as NumPy arrays for the x- and y-values. Now, the data points do not form a polygon.

```
x = np.array([-2, 1, 3, 0, -4])
y = np.array([-3, -1, 1, 3, 2])
```

The `close` parameter can be used to force the polygon to be closed.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.plot(x=x, y=y, pen=True)

fig.shift_origin(xshift="w+1c")

fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.plot(x=x, y=y, pen=True, close=True)
fig.show()
```



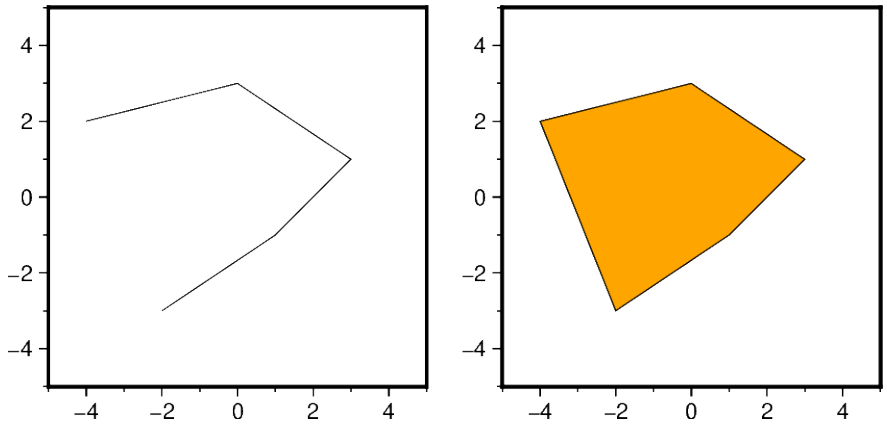
When using the `fill` parameter, the polygon is automatically closed.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.plot(x=x, y=y, pen=True)

fig.shift_origin(xshift="w+1c")
```

(continues on next page)

(continued from previous page)

```
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.plot(x=x, y=y, pen=True, fill="orange")
fig.show()
```



Total running time of the script: (0 minutes 0.424 seconds)

7.1.6 Plotting single-parameter symbols

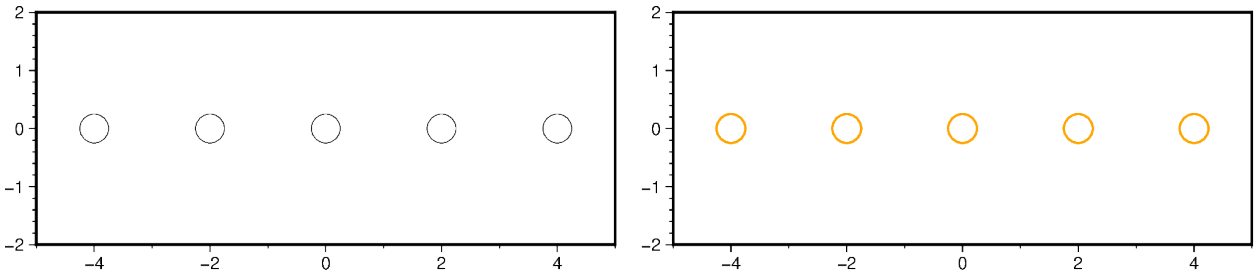
The *pygmt.Figure.plot* method can plot symbols via the `style`, `size`, and `symbol` parameters. This tutorial focuses on single-parameter symbols; for an overview of the available single-parameter symbols, see the Gallery example *Single-parameter symbols*. The available multi-parameter symbols are explained in the Gallery example *Multi-parameter symbols*.

```
import numpy as np
import pygmt

# Set up five sample data points as NumPy arrays for the x and y values
x = np.array([-4, -2, 0, 2, 4])
y = np.array([0, 0, 0, 0, 0])
```

Plot single-parameter symbols

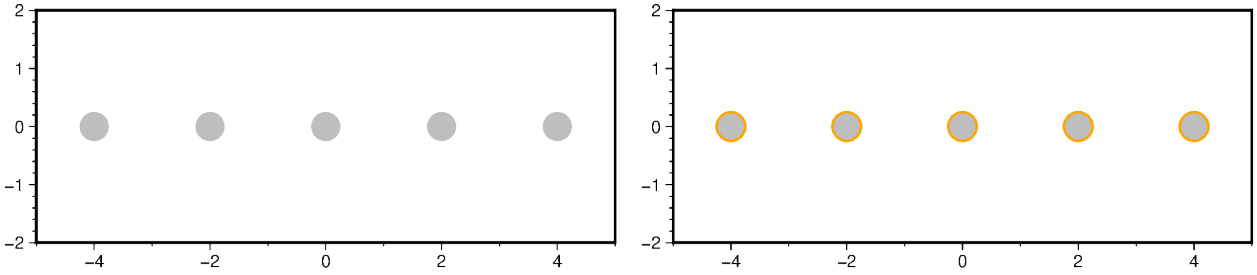
Use the `style` parameter of the *pygmt.Figure.plot* method to plot all data points with the same symbol and size. By default, the symbol is drawn unfilled with an 0.25-points, thick, solid outline. Use the `pen` parameter to adjust the outline by providing a string argument in the form *width*,*color*,*style*; for details see the Gallery example *Line styles*.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

# Plot circles (first "c") with a diameter of 0.5 centimeters (second "c")
fig.plot(x=x, y=y, style="c0.5c")

fig.shift_origin(xshift="w+1c")
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

# Adjust the outline via the pen parameter
fig.plot(x=x, y=y, style="c0.5c", pen="1p,orange")

fig.show()
```



Use the `fill` the parameter to add a fill color or pattern. Note, that no outline is drawn by default when `fill` is used. For the available patterns and their usage see *pygmt.params.Pattern* and the Gallery example *Bit and hachure patterns*.

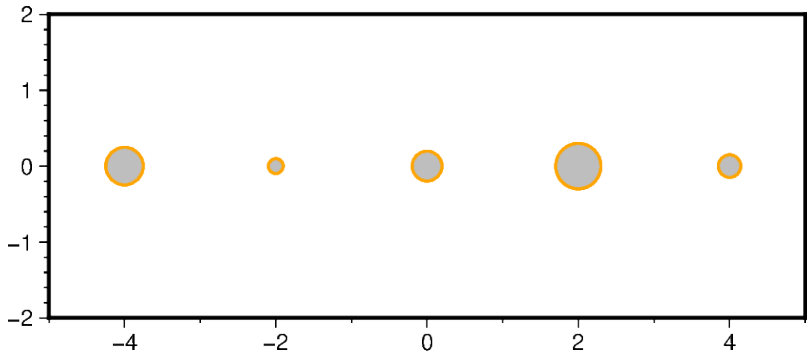
```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

# Add a color via the fill parameter
fig.plot(x=x, y=y, style="c0.5c", fill="gray")

fig.shift_origin(xshift="w+1c")
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

# Add an outline via the pen parameter
fig.plot(x=x, y=y, style="c0.5c", fill="gray", pen="1p,orange")

fig.show()
```



Use individual sizes

Use the `size` parameter to plot the data points with individual sizes. Provide the different sizes as a NumPy array or array-like object of integers or floats.

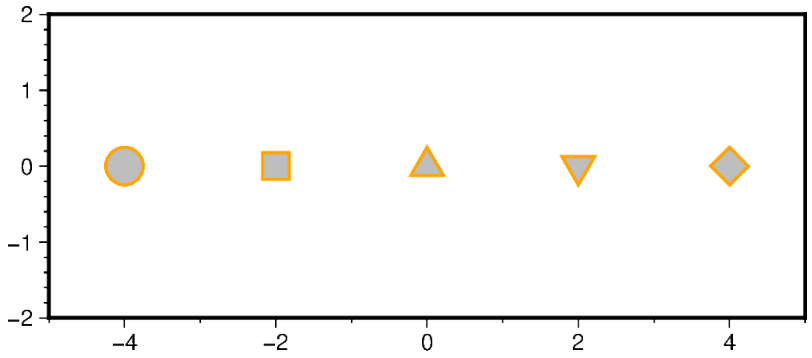
```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

fig.plot(
    x=x,
    y=y,
    # Plot circles (first "c") with a diameter in centimeters (second "c")
    style="cc",
    size=[0.5, 0.2, 0.4, 0.6, 0.3],  # Use individual sizes
    fill="gray",
    pen="1p,orange",
)

fig.show()
```



Use individual symbols

Use the `symbol` parameter to plot the data points with individual symbols. Provide the different symbols as a list of strings. Here, only single-parameter symbols can be used.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)

fig.plot(
    x=x,
    y=y,
    # Use a constant size of 0.5 centimeters
    style="0.5c",
    # Plot a circle, a square, a triangle, an inverse triangle, a diamond
    symbol=["c", "s", "t", "i", "d"],
    fill="gray",
    pen="1p,orange",
)

fig.show()
```



Use individual symbols and sizes

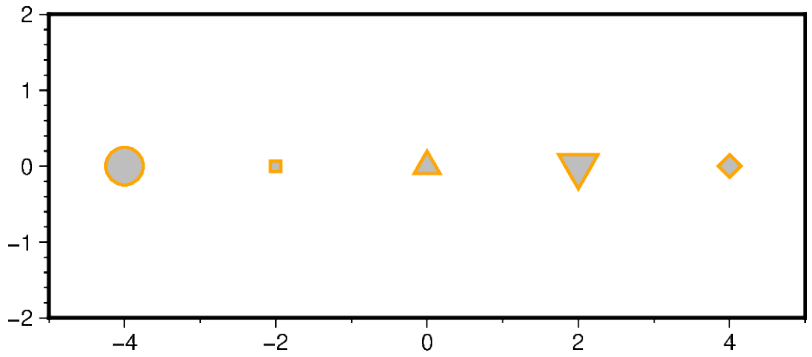
Use the `symbol` and `size` parameters together to plot the data points with individual symbols and sizes. The arguments passed to `symbol` and `size` must have the same length. The unit for `size` is now set via the GMT default parameter `PROJ_LENGTH_UNIT` which can by adjusted using *`pygmt.config`* [Default is centimeters].

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -2, 2], projection="X10c/4c", frame=True)
```

(continues on next page)

(continued from previous page)

```
fig.plot(
    x=x,
    y=y,
    symbol=["c", "s", "t", "i", "d"],
    size=[0.5, 0.2, 0.4, 0.6, 0.3],
    fill="gray",
    pen="1p,orange",
)

fig.show()
```



Total running time of the script: (0 minutes 0.479 seconds)

7.1.7 Plotting text

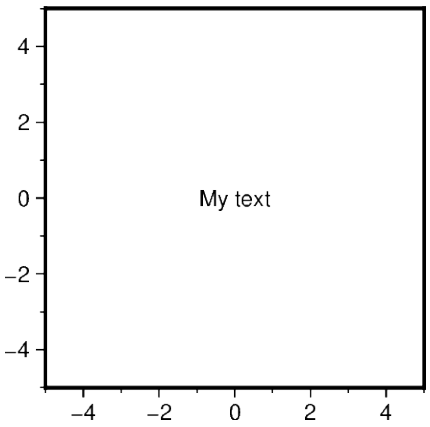
It is often useful to add text annotations to a plot. This is handled by the *pygmt.Figure.text* method of the *pygmt.Figure* class.

```
from pathlib import Path

import pygmt
from pygmt.params import Axis, Frame
```

Adding a single text label

To add a single text label to a plot, use the `text` and `x` and `y` parameters to specify the text and position.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)
fig.text(x=0, y=0, text="My text")
fig.show()
```



Adjusting the text label

There are several optional parameters to adjust the text label:

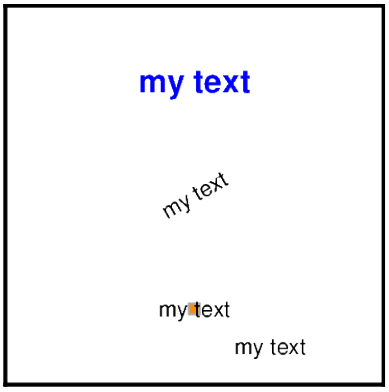
- `font`: Sets the size, family/weight, and color of the font for the text. A list of all recognized fonts can be found at *Supported Fonts*. For details of how to use non-default fonts, refer to PostScript Fonts Used by GMT.
- `angle`: Specifies the rotation of the text. It is measured counter-clockwise from the horizontal in degrees.
- `justify`: Defines the anchor point of the bounding box for the text. It is specified by a *2-character justification code*.
- `offset`: Shifts the text relatively to the reference point.

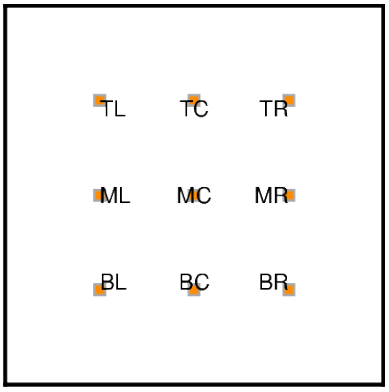
```
fig = pygmt.Figure()

# --------------------------------------------------------------------------
# Left: "font", "angle", and "offset" parameters
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rtlb"))

# Change font size, family/weight, color of the text
fig.text(x=0, y=3, text="my text", font="12p,Helvetica-Bold,blue")

# Rotate the text by 30 degrees counter-clockwise from the horizontal
fig.text(x=0, y=0, text="my text", angle=30)

# Plot marker and text label for reference
fig.plot(x=0, y=-3, style="s0.2c", fill="darkorange", pen="0.7p,darkgray")
fig.text(x=0, y=-3, text="my text")
# Shift the text label relatively to the position given via the x and y parameters
# by 1 centimeter to the right (positive x-direction) and 0.5 centimeters down
# (negative y-direction)
fig.text(x=0, y=-3, text="my text", offset=(1, -0.5))

fig.shift_origin(xshift="w+0.5c")

# --------------------------------------------------------------------------
# Right: "justify" parameter
fig.basemap(region=[-1, 1, -1, 1], projection="X5c", frame=Frame(axes="rtlb"))

# Plot markers for reference
```

(continues on next page)

(continued from previous page)

```
fig.plot(
    x=[-0.5, 0, 0.5, -0.5, 0, 0.5, -0.5, 0, 0.5],
    y=[0.5, 0.5, 0.5, 0, 0, 0, -0.5, -0.5, -0.5],
    style="s0.2c",
    fill="darkorange",
    pen="0.7p,darkgray",
)

# Plot text labels at the x and y positions of the markers while varying the anchor
# point via the justify parameter
fig.text(x=-0.5, y=0.5, text="TL", justify="TL")  # Top Left
fig.text(x=0, y=0.5, text="TC", justify="TC")  # Top Center
fig.text(x=0.5, y=0.5, text="TR", justify="TR")  # Top Right
fig.text(x=-0.5, y=0, text="ML", justify="ML")  # Middle Left
fig.text(x=0, y=0, text="MC", justify="MC")  # Middle Center
fig.text(x=0.5, y=0, text="MR", justify="MR")  # Middle Right
fig.text(x=-0.5, y=-0.5, text="BL", justify="BL")  # Bottom Left
fig.text(x=0, y=-0.5, text="BC", justify="BC")  # Bottom Center
fig.text(x=0.5, y=-0.5, text="BR", justify="BR")  # Bottom Right

fig.show()
```





Adding a text box

There are different optional parameters to add and customize a text box:

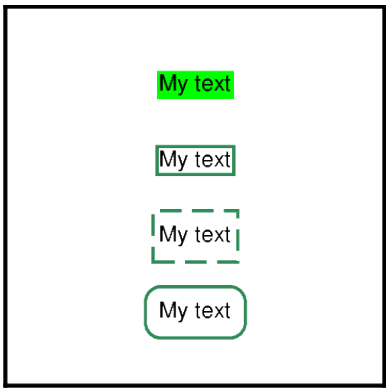
- `fill`: Fills the text box with a color.
- `pen`: Outlines the text box.
- `clearance`: Adds margins in x-and y-directions between the text and the outline of the text box. Can be used to get a text box with rounded edges.

```
fig = pygmt.Figure()

fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rtlb"))

# Add a box with a fill in green color
fig.text(x=0, y=3, text="My text", fill="green")

# Add box with a seagreen, 1-point thick, solid outline
```

(continues on next page)

(continued from previous page)

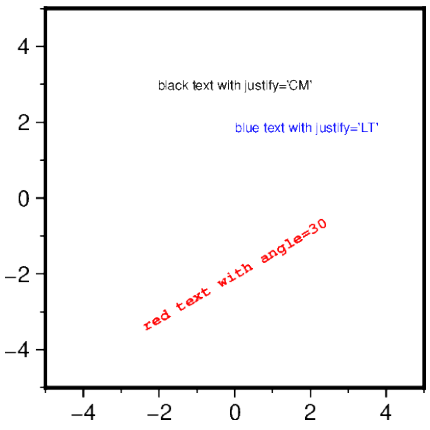
```
fig.text(x=0, y=1, text="My text", pen="1p,seagreen,solid")

# Add margins between the text and the outline of the text box of 0.1
# centimeters in x-direction and 0.2 centimeters in y-direction
fig.text(x=0, y=-1, text="My text", pen="1p,seagreen,dashed", clearance="0.1c/0.2c")

# Get rounded edges by passing "+tO" to the "clearance" parameter
fig.text(x=0, y=-3, text="My text", pen="1p,seagreen,solid", clearance="0.2c/0.2c+tO")

fig.show()
```



Adding multiple text labels with individual configurations

To add multiple text labels with individual `font`, `angle`, and `justify`, one can provide lists with the corresponding arguments.

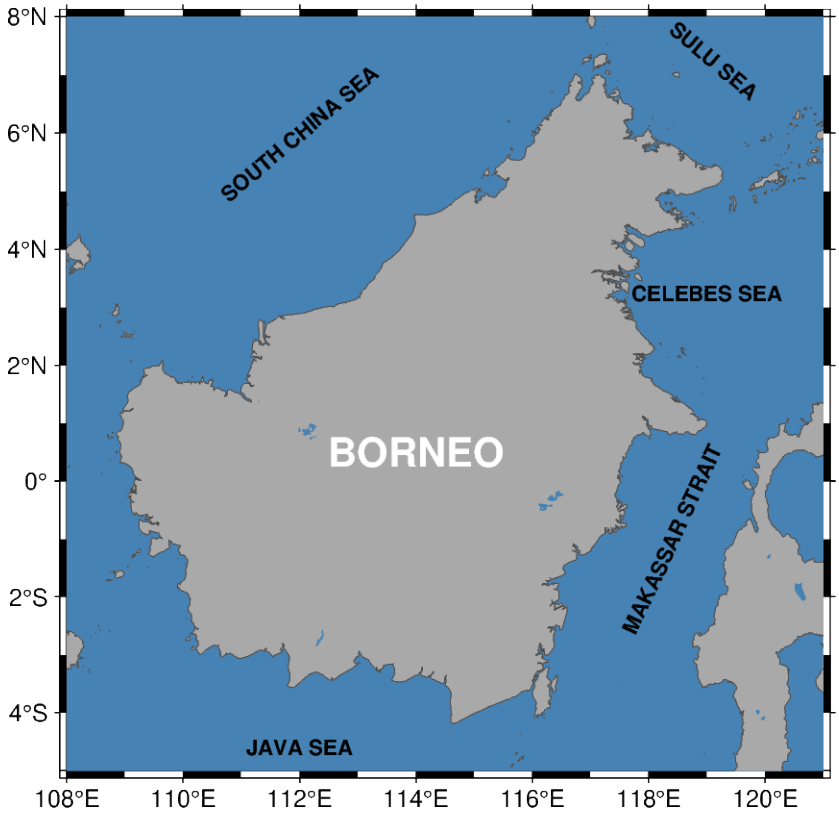
```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)

fig.text(
    x=[0, 0, 0],
    y=[3, 2, -2],
    font=["5p,Helvetica,black", "5p,Helvetica,blue", "6p,Courier-Bold,red"],
    angle=[0, 0, 30],
    justify=["CM", "LT", "CM"],
    text=[
        "black text with justify='CM'",
        "blue text with justify='LT'",
        "red text with angle=30",
    ],
)

fig.show()
```



Using an external input file

It is also possible to add text labels via an external input file containing `x`, `y`, and `text` columns. Additionally, columns to set the `angle`, `font`, and `justify` parameters can be provided. Here, we give a complete example.

```
fig = pygmt.Figure()
fig.basemap(region=[108, 121, -5, 8], projection="M10c", frame=Axis(annot=2, tick=1))
fig.coast(land="darkgray", water="steelblue", shorelines="1/0.1p,gray30")

# Create space-delimited file with region / sea names:
# - longitude (x) and latitude (y) coordinates are in the first two columns
# - angle, font, and justify must be present in this order in the next three columns
# - the text to be printed is given in the last column
with Path.open("examples.txt", "w") as f:
    f.write("114.00  0.50   0 15p,Helvetica-Bold,white CM BORNEO\n")
    f.write("119.00  3.25   0  8p,Helvetica-Bold,black CM CELEBES SEA\n")
    f.write("112.00 -4.60   0  8p,Helvetica-Bold,black CM JAVA SEA\n")
    f.write("112.00  6.00  40  8p,Helvetica-Bold,black CM SOUTH CHINA SEA\n")
    f.write("119.12  7.25 -40  8p,Helvetica-Bold,black CM SULU SEA\n")
    f.write("118.40 -1.00  65  8p,Helvetica-Bold,black CM MAKASSAR STRAIT\n")

# Setting the angle, font, and justify parameters to True indicates that those columns
# are present in the text file
fig.text(textfiles="examples.txt", angle=True, font=True, justify=True)

# Cleanups
Path("examples.txt").unlink()

fig.show()
```



Using the position parameter

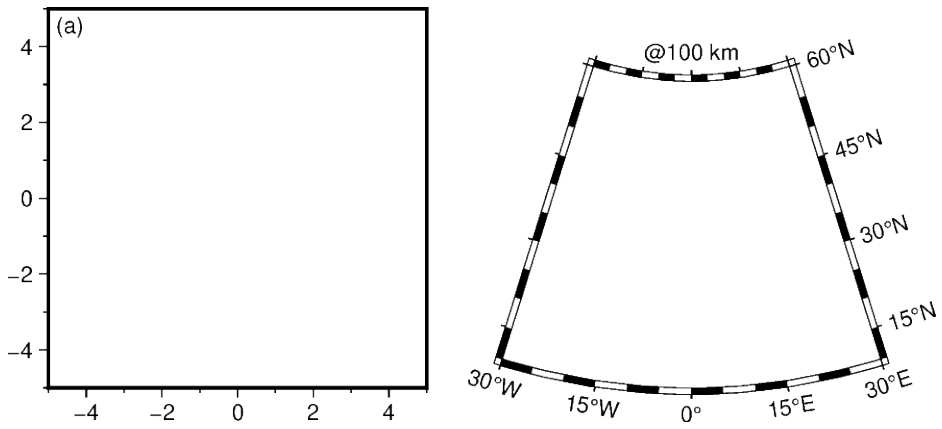
Instead of using the `x` and `y` parameters, the `position` parameter can be specified to set the reference point for the text on the plot. As for the `justify` parameter, the `position` parameter is specified by a *2-character justification code*. This can be helpful to add a tag to a subplot or text labels out of the plot frame, e.g., for depth slices.

```
fig = pygmt.Figure()

# --------------------------------------------------------------------------------
# Left: Add a tag to a subplot at the Top Left corner
fig.basemap(
    region=[-5, 5, -5, 5],
    projection="X5c",
    frame=Frame(axes="WStr", axis=Axis(annot=True, tick=True)),
)
fig.text(text="(a)", position="TL", justify="TL", offset=(0.1, -0.1))

fig.shift_origin(xshift="w+1c")

# --------------------------------------------------------------------------------
# Right: Add a text label outside of the plot frame
fig.basemap(
    region=[-30, 30, 10, 60],
    projection="L0/35/23/47/5c",
    frame=Frame(axes="wSnE", axis=Axis(annot=True, tick=True)),
)
```

(continues on next page)

(continued from previous page)

```
fig.text(
    text="@@100 km",  # "@@" gives "@" in GMT or PyGMT
    position="TC",  # Top Center
    justify="MC",  # Middle Center
    offset=(0, 0.2),
    no_clip=True,  # Allow plotting outside of the plot frame
)

fig.show()
```



Advanced configuration

For crafting more advanced styles, including using special symbols and other character sets, be sure to check out the GMT documentation at https://docs.generic-mapping-tools.org/6.7/text.html and also the Technical References at https://docs.generic-mapping-tools.org/6.7/reference/features.html#placement-of-text. Good luck!

Total running time of the script: (0 minutes 0.862 seconds)

7.1.8 Setting the region

Many of the plotting methods take the `region` parameter, which sets the area that will be shown in the figure. This tutorial covers the different types of inputs that it can accept.

```
import pygmt
from pygmt.params import Axis
```

Coordinates

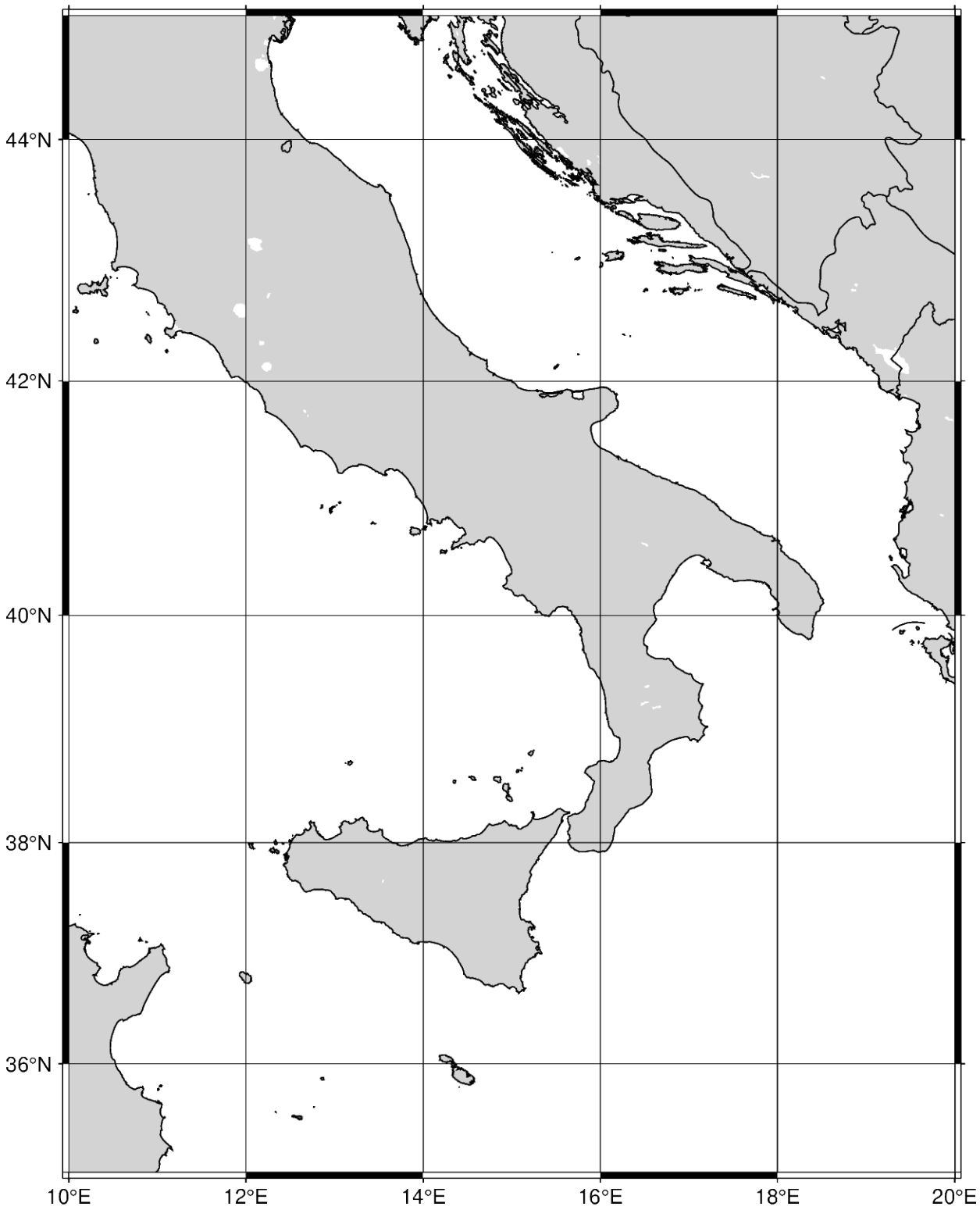
A string of coordinates can be passed to `region`, in the form of *xmin*/*xmax*/*ymin*/*ymax*.

```
fig = pygmt.Figure()
fig.coast(
    # Set the x-range from 10E to 20E and the y-range to 35N to 45N
    region="10/20/35/45",
    # Set projection to Mercator, and the map width to 15 centimeters
    projection="M15c",
    # Set the color of the land to light gray
    land="lightgray",
    # Set the color of the water to white
```

(continues on next page)

(continued from previous page)

```
    water="white",
    # Display the national borders and set the pen thickness to 0.5p
    borders="1/0.5p",
    # Display the shorelines and set the pen thickness to 0.5p
    shorelines="1/0.5p",
    # Set the frame to display annotations and gridlines
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



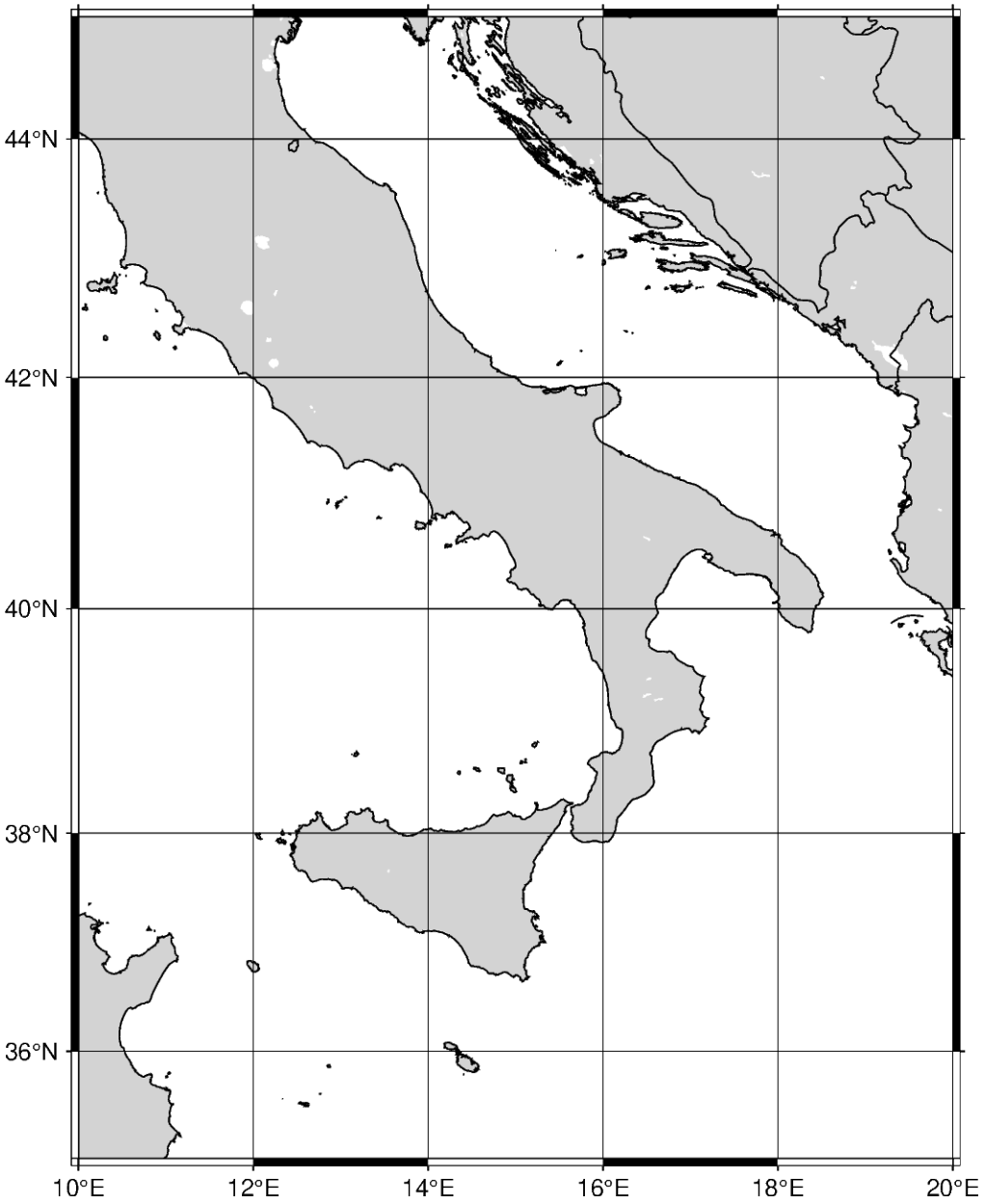
The coordinates can be passed to `region` as a list, in the form of [*xmin*, *xmax*, *ymin*, *ymax*].

```
fig = pygmt.Figure()
fig.coast(
    # Set the x-range from 10E to 20E and the y-range to 35N to 45N
```

(continues on next page)

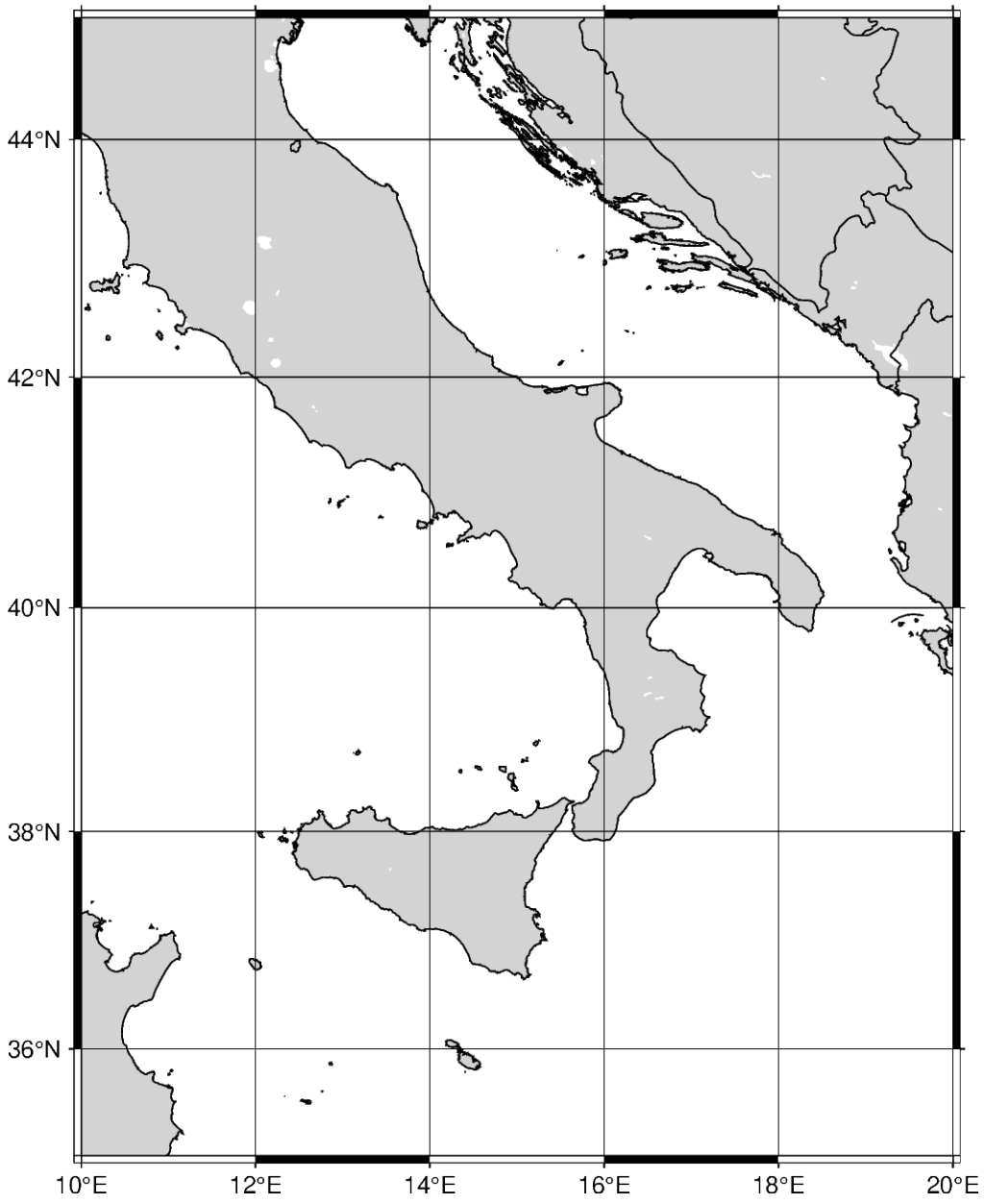
(continued from previous page)

```
    region=[10, 20, 35, 45],
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



Instead of passing axes minima and maxima, the coordinates can be passed for the bottom-left and top-right corners. The string format takes the coordinates for the bottom-left and top-right coordinates. To specify corner coordinates, append **+r** at the end of the `region` string.

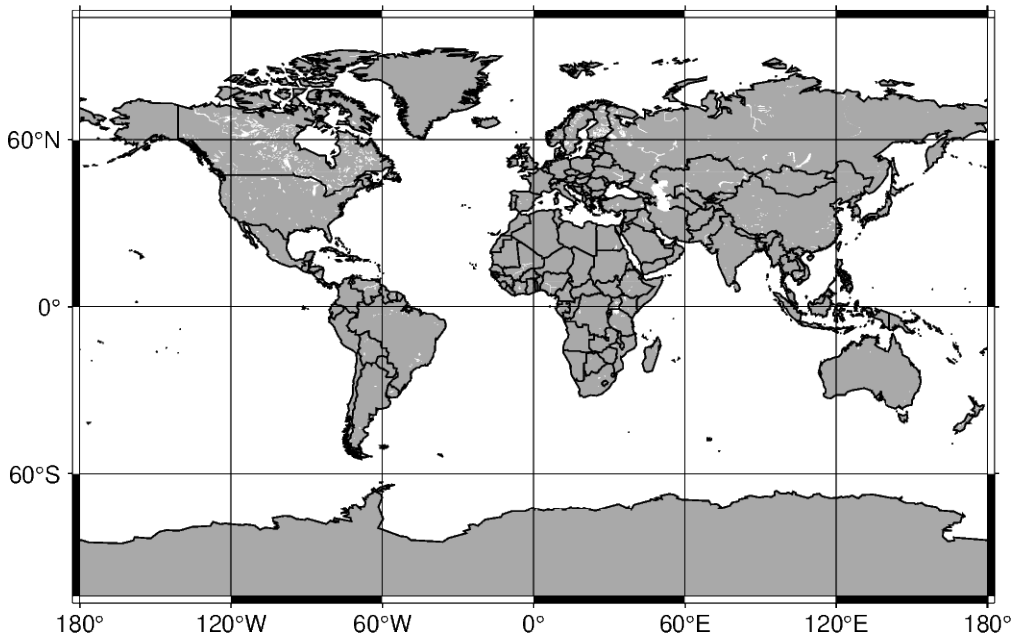
```
fig = pygmt.Figure()
fig.coast(
    # Set the bottom-left corner as 10E, 35N and the top-right corner as
    # 20E, 45N
    region="10/35/20/45+r",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



Global regions

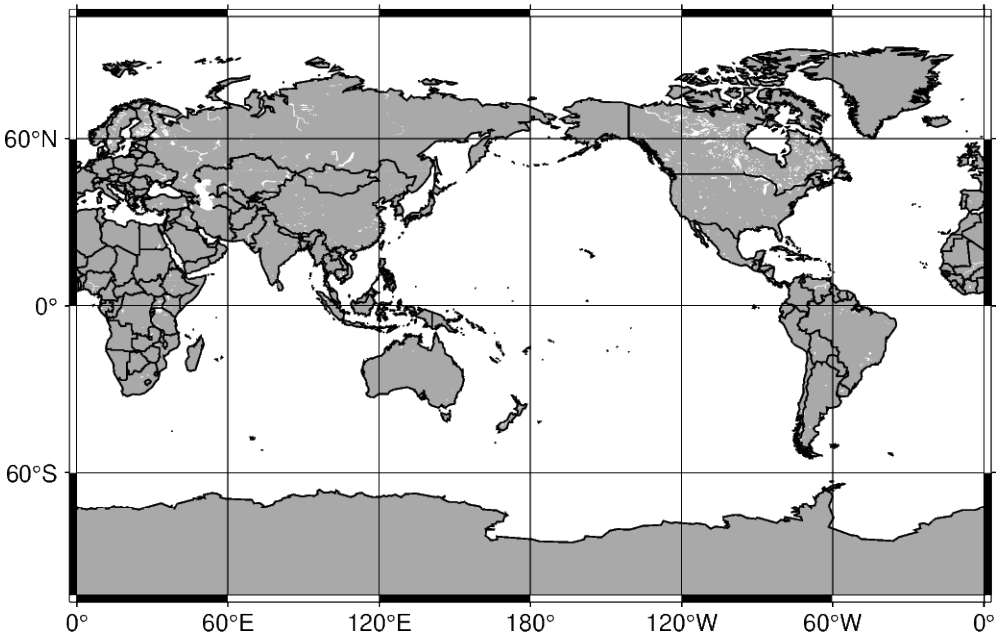
In addition to passing coordinates, the argument **d** can be passed to set the region to the entire globe. The range is 180W to 180E (-180, 180) and 90S to 90N (-90 to 90). With no parameters set for the projection, the map defaults to be centered at the mid-point of both x- and y-axes. Using **d**, the map is centered at (0, 0), or the intersection of the equator and prime meridian.

```
fig = pygmt.Figure()
fig.coast(
    region="d",
    projection="Cyl_stere/12c",
    land="darkgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



The argument **g** can be passed, which encompasses the entire globe. The range is 0E to 360E (0, 360) and 90S to 90N (-90 to 90). With no parameters set for the projection, the map is centered at (180, 0), or the intersection of the equator and International Date Line.

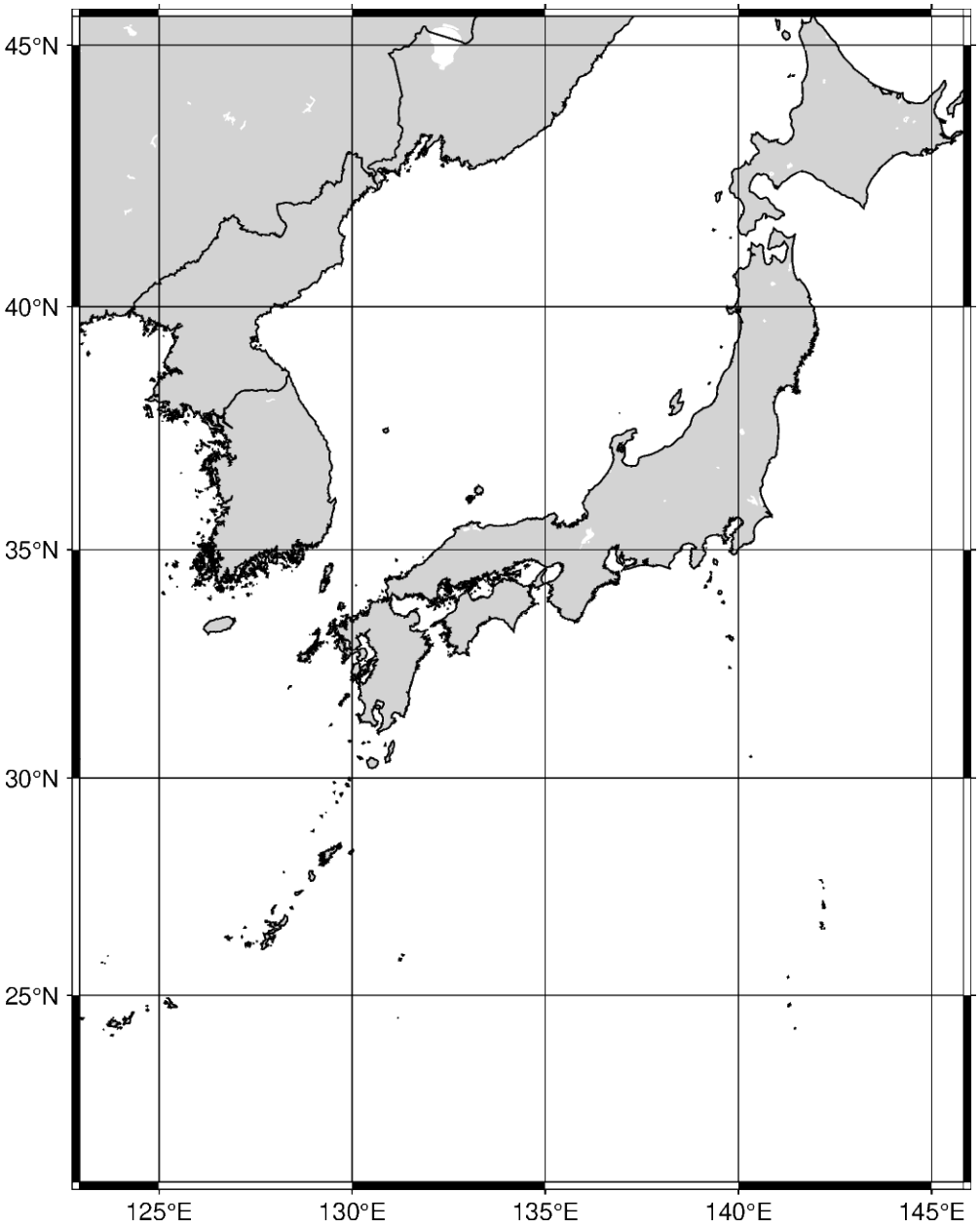
```
fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="Cyl_stere/12c",
    land="darkgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



ISO code

The `region` can be set to include a specific area specified by the two-character ISO 3166-1 alpha-2 convention (for further information: https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2).

```
fig = pygmt.Figure()
fig.coast(
    # Set the map region to encompass Japan with the ISO code "JP"
    region="JP",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



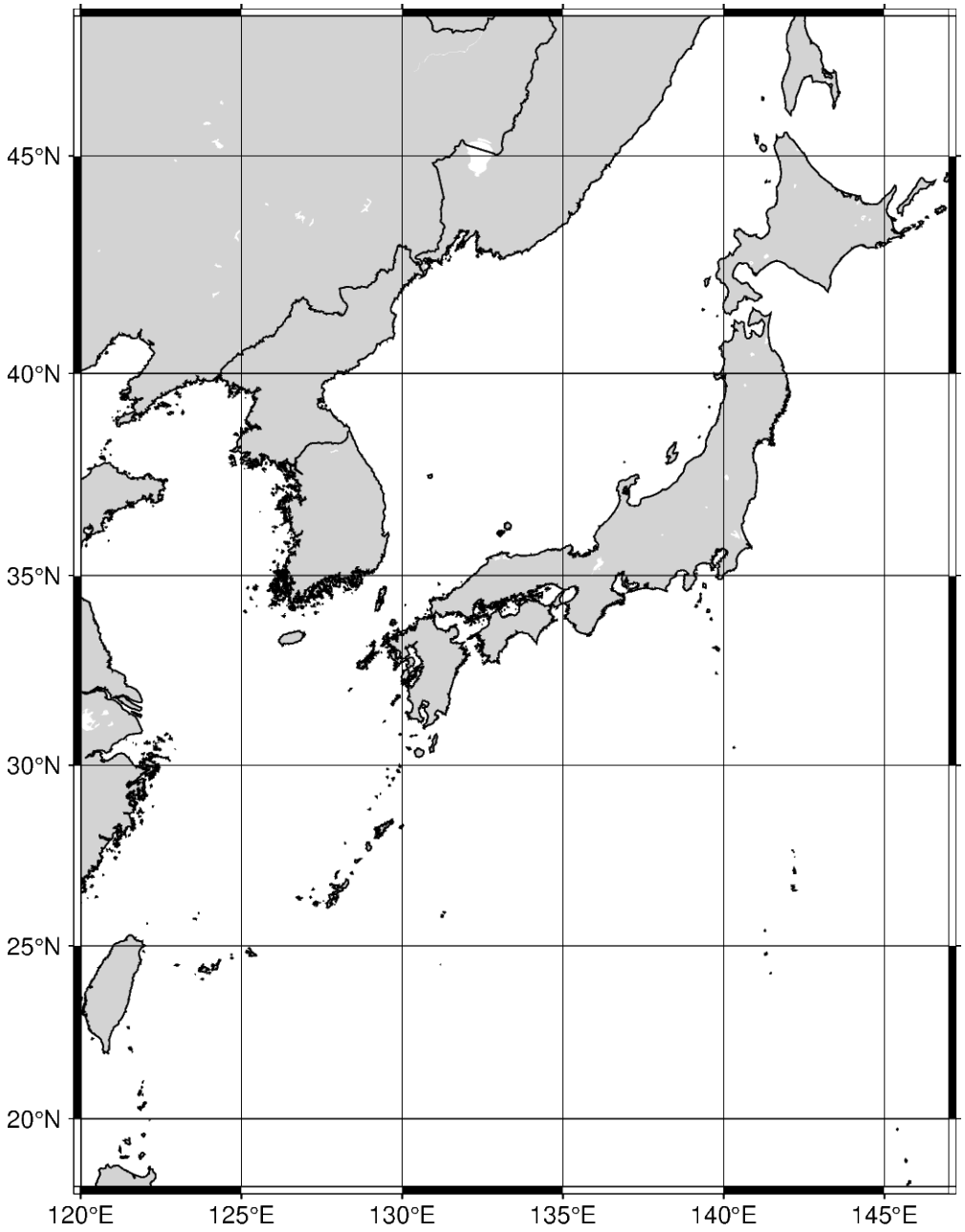
The area encompassed by the ISO code can be expanded by appending **+r***increment* to the ISO code. The *increment* unit is in degrees, and if only one value is added it expands the range of the region in all directions. Using **+r** expands the final region boundaries to be multiples of *increment* .

```
fig = pygmt.Figure()
fig.coast(
    # Expand the region boundaries to be multiples of 3 degrees in all
    # directions
    region="JP+r3",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
```

(continues on next page)

(continued from previous page)

```
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



Instead of expanding the range of the plot uniformly in all directions, two values can be passed to expand differently on each axis. The format is *xinc*/*yinc*.

```
fig = pygmt.Figure()
fig.coast(
    # Expand the region boundaries to be multiples of 3 degrees on the x-axis
    # and 5 degrees on the y-axis.
    region="JP+r3/5",
```

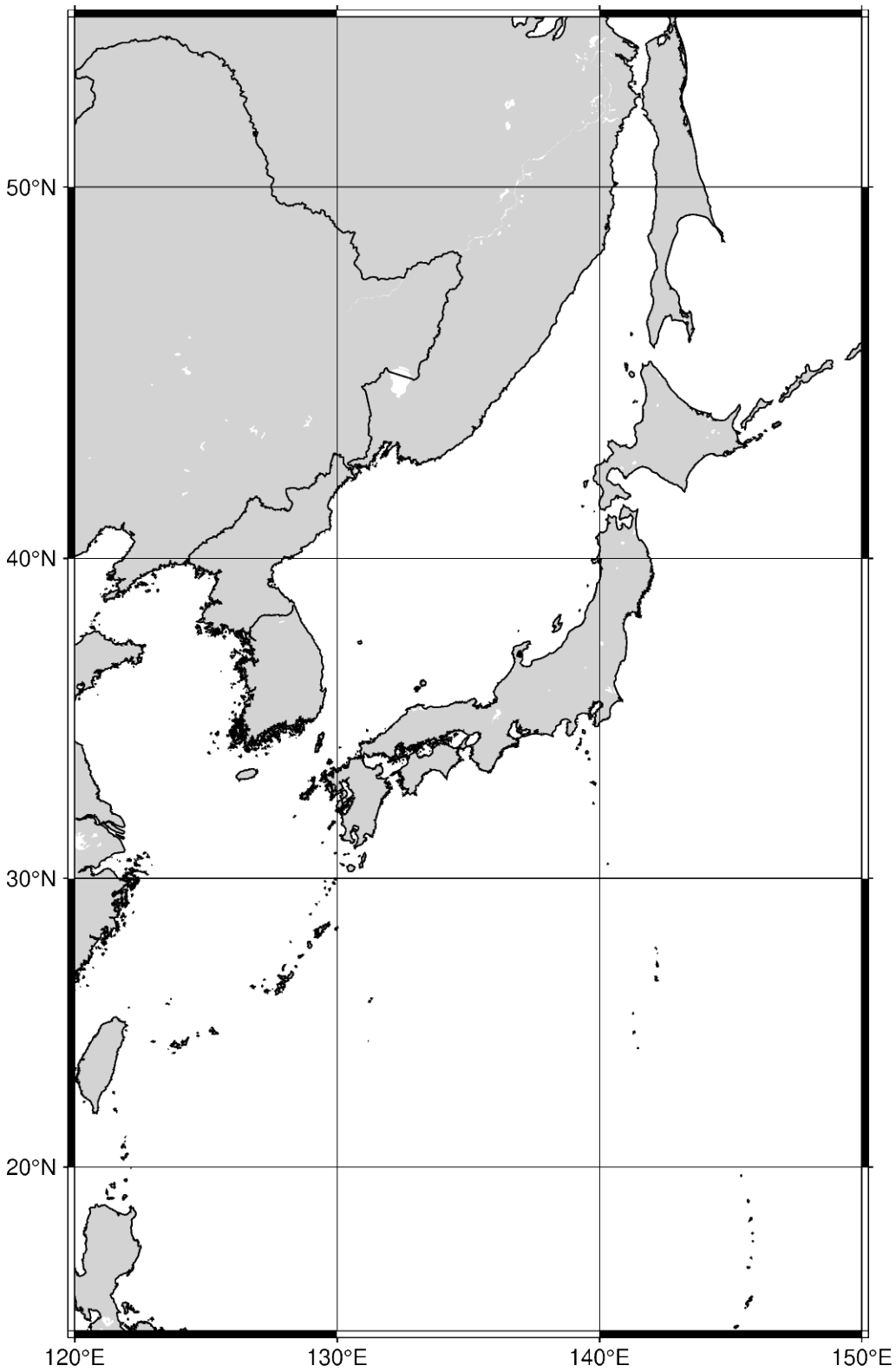
(continues on next page)

(continued from previous page)

```
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```

Instead of expanding the range of the plot uniformly in all directions, four values can be passed to expand differently in each direction. The format is *winc*/*einc*/*sinc*/*ninc*, which expands on the west, east, south, and north axes.

```
fig = pygmt.Figure()
fig.coast(
    # Expand the region boundaries to be multiples of 3 degrees to the west, 5
    # degrees to the east, 7 degrees to the south, and 9 degrees to the north.
    region="JP+r3/5/7/9",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



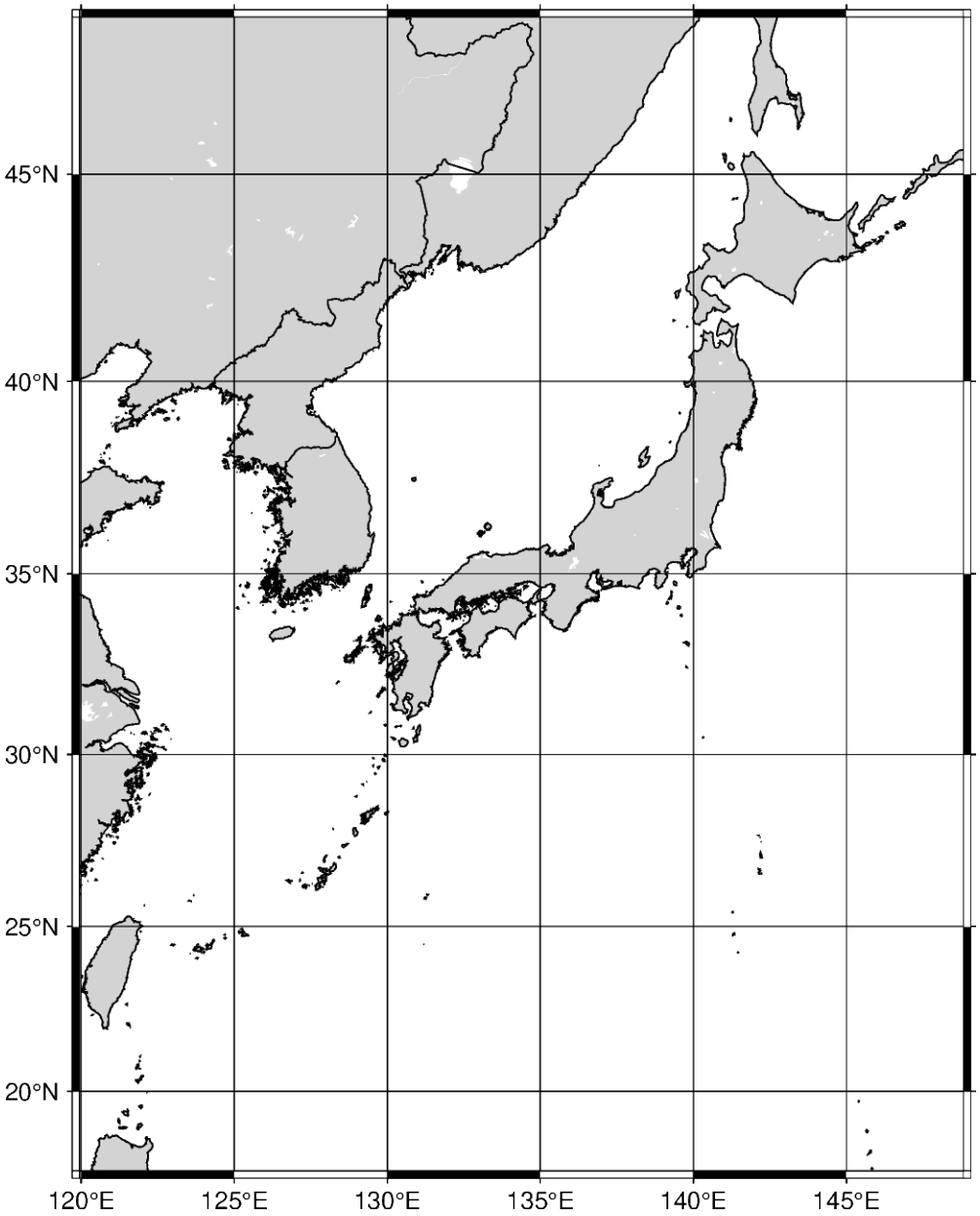
The `region` increment can be appended with **+R**, which adds the increment without rounding.

```
fig = pygmt.Figure()
fig.coast(
```

(continues on next page)

(continued from previous page)

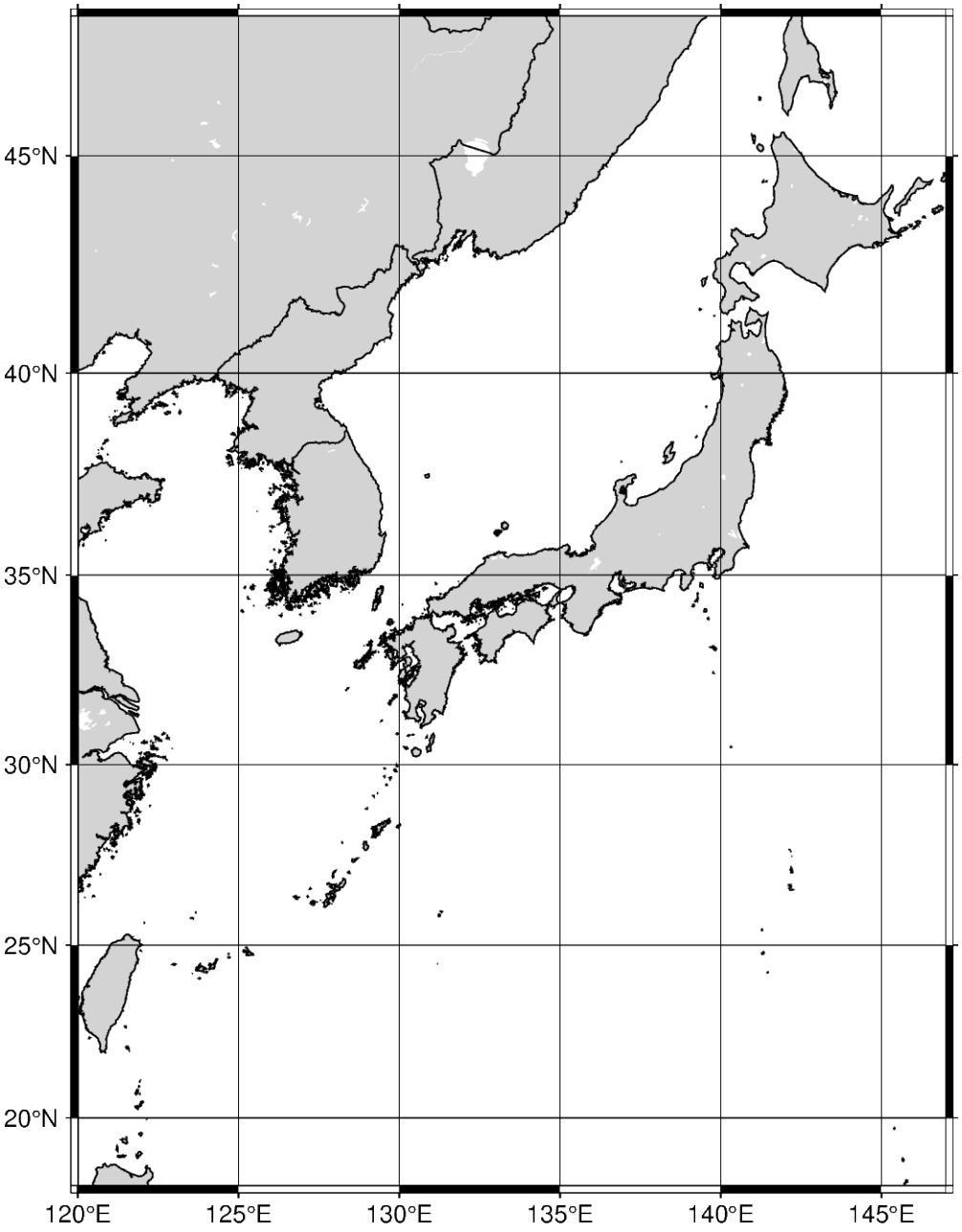
```
    # Expand the region setting outside the range of Japan by 3 degrees in all
    # directions, without rounding to the nearest increment.
    region="JP+R3",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



The `region` increment can be appended with **+e**, which is like **+r** and expands the final region boundaries to be multiples

of *increment*. However, it ensures that the bounding box extends by at least 0.25 times the increment.

```
fig = pygmt.Figure()
fig.coast(
    # Expand the region boundaries to be multiples of 3 degrees in all
    # directions
    region="JP+e3",
    projection="M12c",
    land="lightgray",
    water="white",
    borders="1/0.5p",
    shorelines="1/0.5p",
    frame=Axis(annot=True, grid=True),
)
fig.show()
```



Total running time of the script: (0 minutes 2.721 seconds)

7.2 Advanced

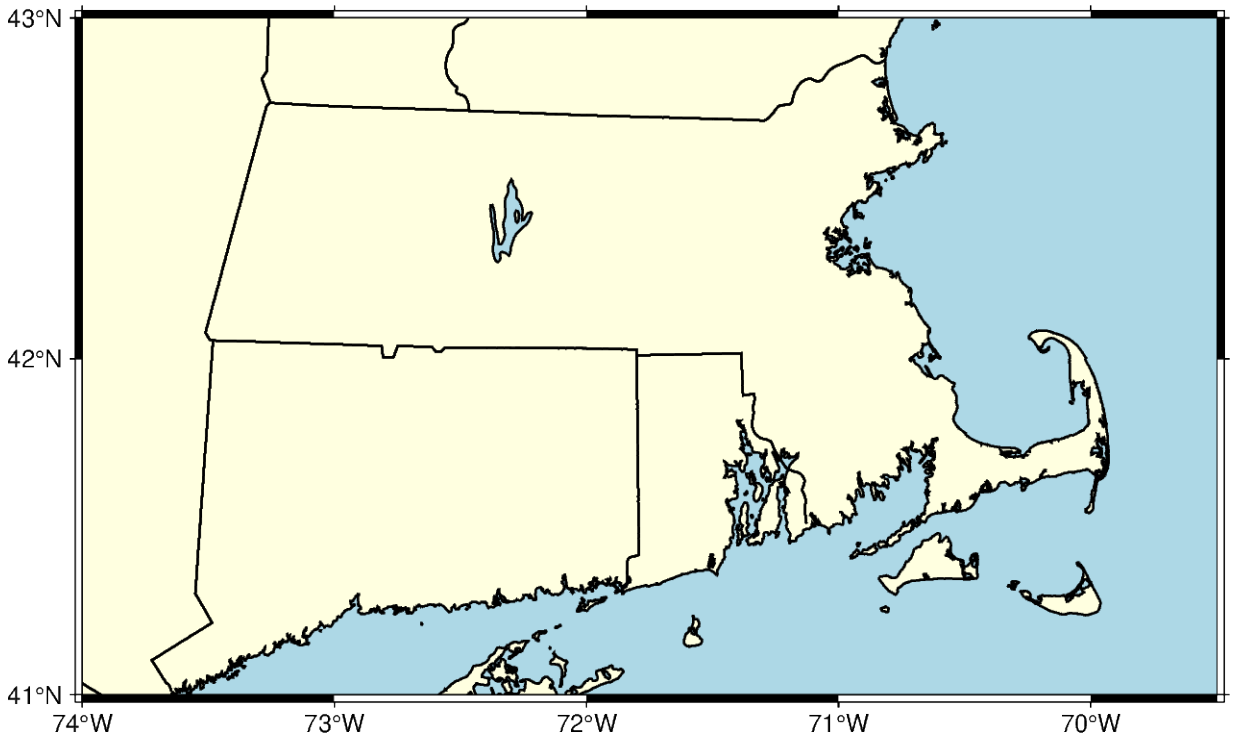
7.2.1 Adding an inset to the figure

To plot an inset figure inside another larger figure, we can use the *pygmt.Figure.inset* method. After a large figure has been created, call `inset` using a `with` statement, and new plot elements will be added to the inset figure instead of the larger figure.

```
import pygmt
from pygmt.params import Axis, Box, Position
```

Prior to creating an inset figure, a larger figure must first be plotted. In the example below, *pygmt.Figure.coast* is used to create a map of the US state of Massachusetts.

```
fig = pygmt.Figure()
fig.coast(
    region=[-74, -69.5, 41, 43],  # Set bounding box of the large figure
    borders="2/thin",  # Plot state boundaries with thin lines
    shorelines="thin",  # Plot coastline with thin lines
    projection="M15c",  # Set Mercator projection and size of 15 centimeter
    land="lightyellow",  # Color land areas light yellow
    water="lightblue",  # Color water areas light blue
    frame=Axis(annot=True),  # Set frame with annotations
)
fig.show()
```



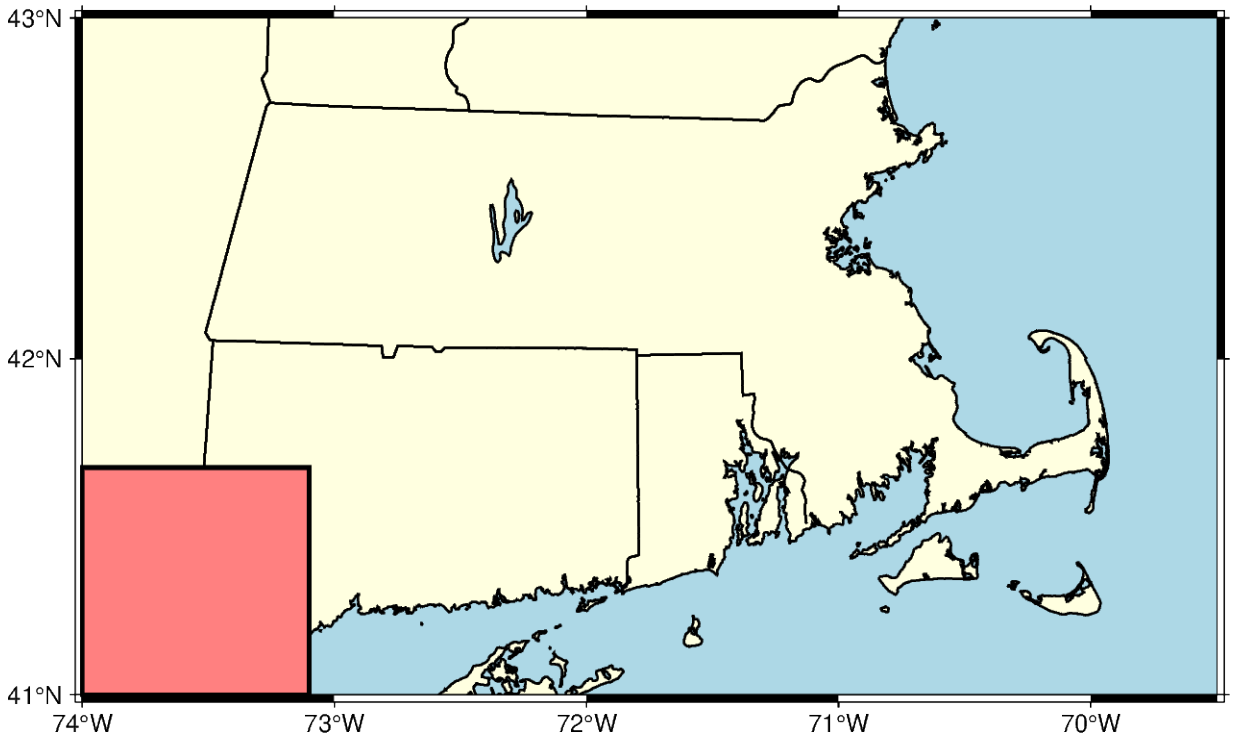
The *pygmt.Figure.inset* method uses a context manager, and is called using a `with` statement. The `position` parameter, including the inset width, is required to plot the inset. In the example below, the inset is placed at the Bottom Left (`BL`) inside the plot. The `box` parameter can set the fill and border of the inset.

```
fig = pygmt.Figure()
fig.coast(
    region=[-74, -69.5, 41, 43],
    borders="2/thin",
    shorelines="thin",
    projection="M15c",
    land="lightyellow",
    water="lightblue",
    frame=Axis(annot=True),
)
with fig.inset(position=Position("BL"), width=3, box=Box(pen="black", fill="lightred
```

(continues on next page)

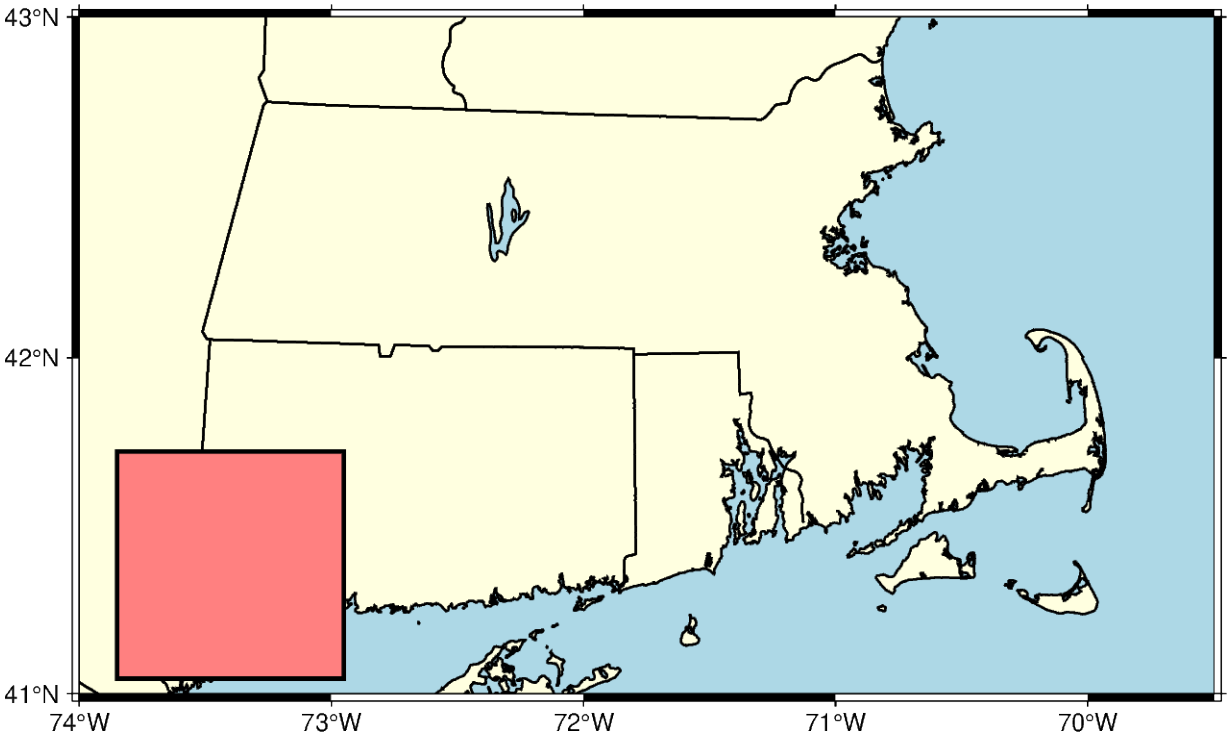
(continued from previous page)

```
↪")):
    # pass is used to exit the with statement as no plotting methods are called
    pass
fig.show()
```



When placed at the Bottom Left corner inside the plot, the default location is in contact with the nearby axis or axes. The offsets along the x- and y-axes can be set with the `offset` parameter of the `Position` class. If only one offset is passed, it is applied to both axes. Each offset can have its own unit. In the example below, the inset is shifted 0.5 centimeters on the x-axis and 0.2 centimeters on the y-axis.

```
fig = pygmt.Figure()
fig.coast(
    region=[-74, -69.5, 41, 43],
    borders="2/thin",
    shorelines="thin",
    projection="M15c",
    land="lightyellow",
    water="lightblue",
    frame=Axis(annot=True),
)
with fig.inset(
    position=Position("BL", offset=(0.5, 0.2)),
    width=3,
    box=Box(pen="black", fill="lightred"),
):
    pass
fig.show()
```



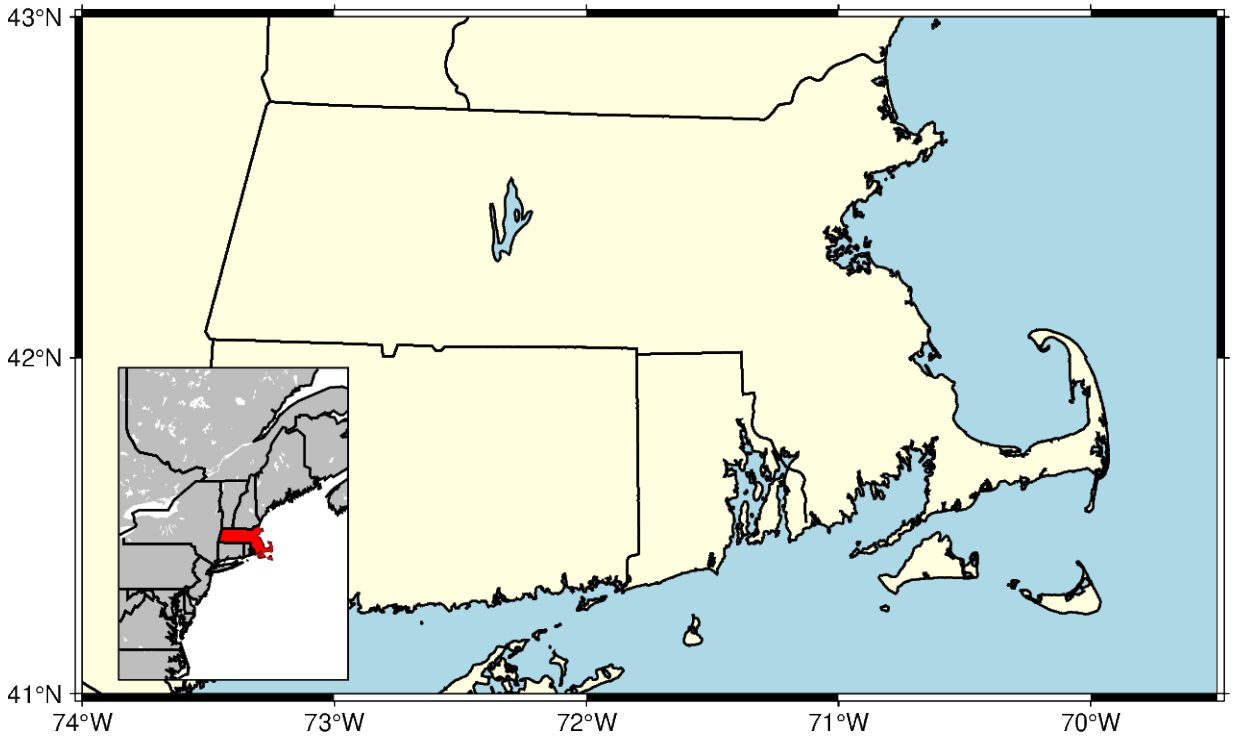
Standard plotting methods can be called from within the `inset` context manager. The example below uses *pygmt.Figure.coast* to plot a zoomed out map that selectively paints the state of Massachusetts to show its location relative to other states.

```
fig = pygmt.Figure()
fig.coast(
    region=[-74, -69.5, 41, 43],
    borders="2/thin",
    shorelines="thin",
    projection="M15c",
    land="lightyellow",
    water="lightblue",
    frame=Axis(annot=True),
)
# This does not include an inset fill as it is covered by the inset figure. Inset
# width/height are determined by the ``region`` and ``projection`` parameters.
with fig.inset(
    position=Position("BL", offset=(0.5, 0.2)),
    box=Box(pen="black"),
    region=[-80, -65, 35, 50],
    projection="M3c",
):
    # Use a plotting method to create a figure inside the inset.
    fig.coast(
        land="gray",
        borders=[1, 2],
        shorelines="1/thin",
        water="white",
        dcw="US.MA+gred",  # Use dcw to selectively highlight an area
    )
```

(continues on next page)

(continued from previous page)

```
fig.show()
```



Total running time of the script: (0 minutes 0.805 seconds)

7.2.2 Cartesian histograms

Cartesian histograms can be generated using the *pygmt.Figure.histogram* method. In this tutorial, different histogram related aspects are addressed:

- Using vertical and horizontal bars
- Using stair-steps
- Showing counts and frequency percent
- Adding annotations to the bars
- Showing cumulative values
- Using color and pattern as fill for the bars
- Using overlaid, stacked, and grouped bars

Import the required packages

```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame, Pattern
```

Generate random data from a normal distribution:

```
rng = np.random.default_rng(seed=100)
```

(continues on next page)

(continued from previous page)

```
# Mean of distribution
mean = 100
# Standard deviation of distribution
stddev = 20

# Create two data sets
data01 = rng.normal(loc=mean, scale=stddev, size=42)
data02 = rng.normal(loc=mean, scale=stddev * 2, size=42)
```

Vertical and horizontal bars

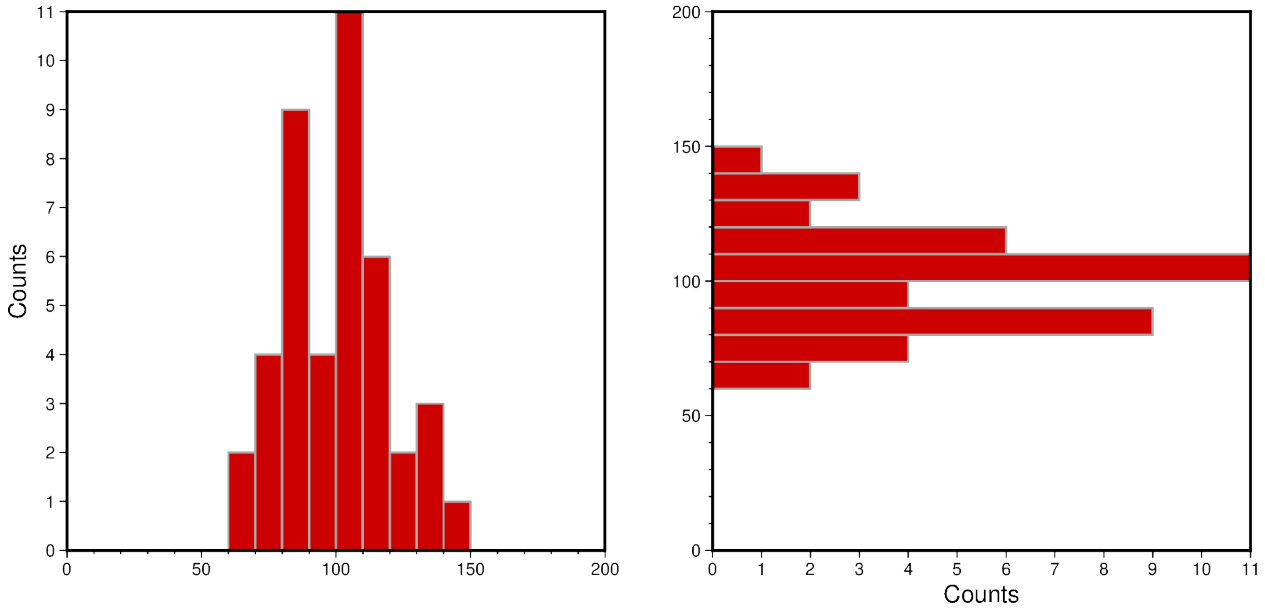
To define the width of the bins, the `series` parameter has to be specified. The bars can be filled via the `fill` parameter with either a color or a pattern (see later in this tutorial). Use the `pen` parameter to adjust width, color, and style of the outlines. By default, a histogram with vertical bars is created. Horizontal bars can be achieved via `horizontal=True`.

```
fig = pygmt.Figure()

# Create histogram for data01 with vertical bars
fig.histogram(
    # Define the plot range as a list of xmin, xmax, ymin, ymax
    # Let ymin and ymax determined automatically by setting both to the same value
    region=[0, 200, 0, 0],
    projection="X10c",  # Cartesian projection with a width of 10 centimeters
    # Add frame, annotations ("a"), ticks ("f"), and y-axis label ("+l") "Counts"; the
    # numbers give the steps of annotations and ticks
    frame=Frame(
        axes="WStr",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data01,
    # Set the bin width via the "series" parameter
    series=10,
    # Fill the bars with color "red3"
    fill="red3",
    # Draw a 1-point thick, solid outline in "darkgray" around the bars
    pen="1p,darkgray,solid",
    # Choose counts via the "histtype" parameter
    histtype="count",
)

# Shift plot origin by the previous plot's width ("w") plus 2 centimeters to the right
fig.shift_origin(xshift="w+2c")

# Create histogram for data01 with horizontal bars
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WStr",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
```

(continues on next page)

(continued from previous page)

```
    ),
    data=data01,
    series=10,
    fill="red3",
    pen="1p,darkgray,solid",
    histtype="count",
    # Use horizontal bars. Note that the x- and y-axes are flipped, with the x-axis
    # plotted vertically and the y-axis plotted horizontally.
    horizontal=True,
)

fig.show()
```



Stair-steps

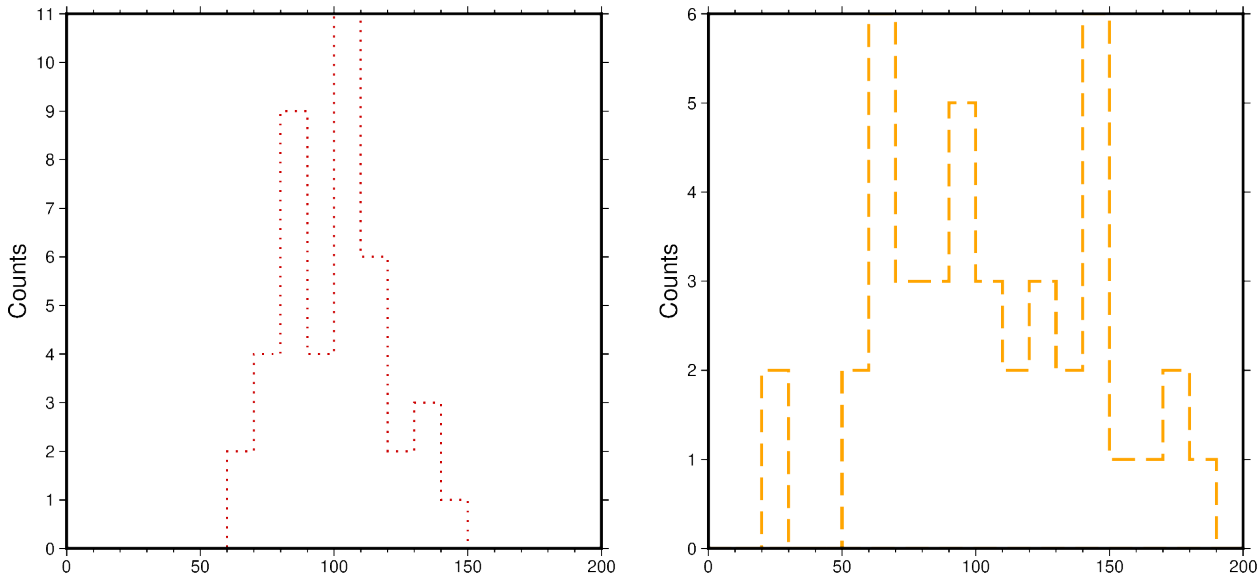
A stair-step diagram can be created by setting `stairs=True`. Then only the outer outlines of the bars are drawn, and no internal bars are visible.

```
fig = pygmt.Figure()

# Create histogram for data01
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data01,
    series=10,
    # Draw a 1-point thick, dotted outline in "red3"
    pen="1p,red3,dotted",
```

(continues on next page)

(continued from previous page)

```
    histtype="count",
    # Draw stair-steps in stead of bars
    stairs=True,
)

fig.shift_origin(xshift="w+2c")

# Create histogram for data02
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data02,
    series=10,
    # Draw a 1.5-point thick, dashed outline in "orange"
    pen="1.5p,orange,dashed",
    histtype="count",
    stairs=True,
)

fig.show()
```



Counts and frequency percent

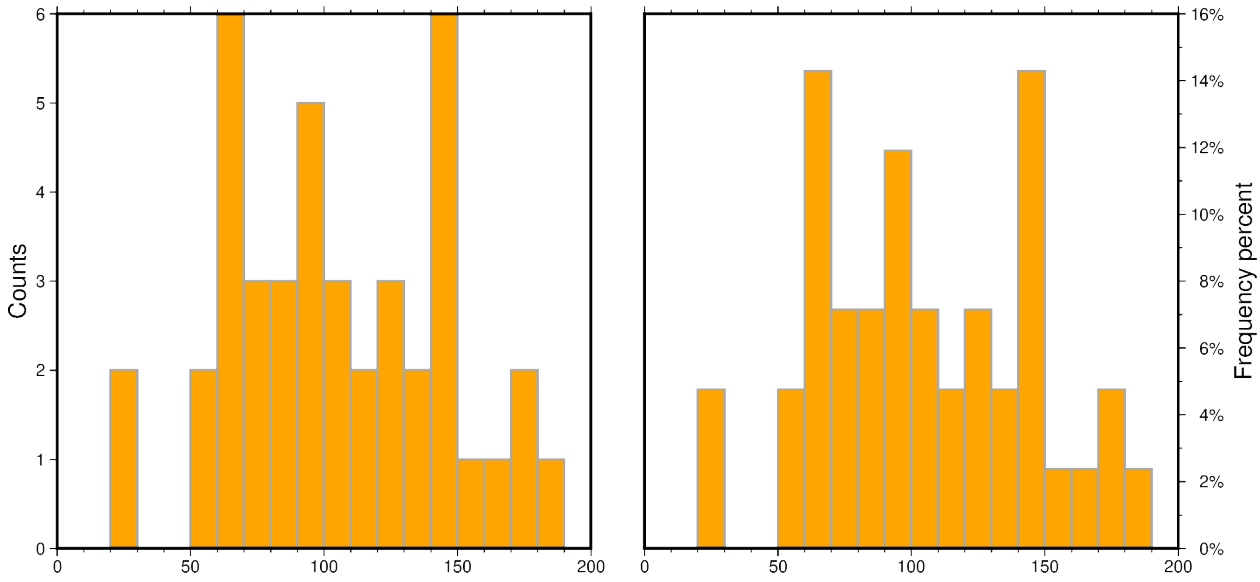
By default, a histogram showing the counts in each bin is created (`histtype="count"`). To show the percentage of values in each bin, set the `histtype` parameter to `"percent"`. For further options please have a look at the documentation of *pygmt.Figure.histogram*.

```
fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

```
# Create histogram for data02 showing counts
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WSnr",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data02,
    series=10,
    fill="orange",
    pen="1p,darkgray,solid",
    # Choose counts via the "histtype" parameter
    histtype="count",
)

fig.shift_origin(xshift="w+1c")

# Create histogram for data02 showing frequency percent
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    # Add suffix % (+u)
    frame=Frame(
        axes="lSnE",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=2, tick=1, unit="%", label="Frequency percent"),
    ),
    data=data02,
    series=10,
    fill="orange",
    pen="1p,darkgray,solid",
    # Choose percentages via the "histtype" parameter
    histtype="percent",
)

fig.show()
```



Cumulative values

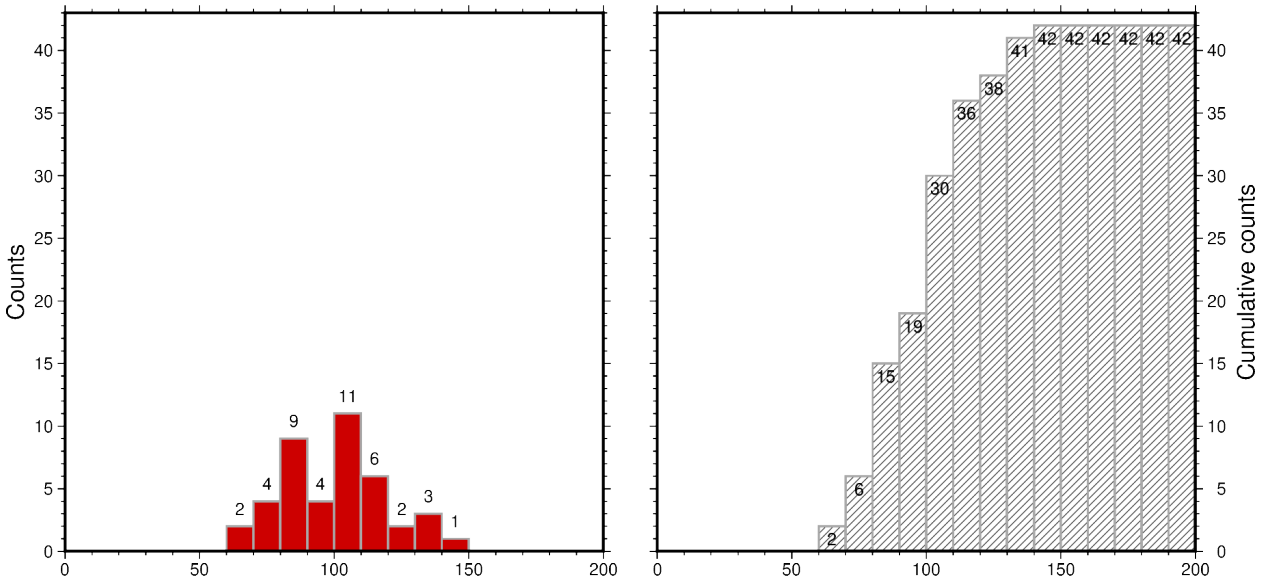
To create a histogram showing the cumulative values set `cumulative=True`. Here, the bars of the cumulative histogram are filled with a *pygmt.params.Pattern* via the `fill` parameter. Annotate each bar with the counts it represents using the `annot` parameter.

```
fig = pygmt.Figure()

# Create histogram for data01 showing the counts per bin
fig.histogram(
    region=[0, 200, 0, len(data01) + 1],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=5, tick=1, label="Counts"),
    ),
    data=data01,
    series=10,
    fill="red3",
    pen="1p,darkgray,solid",
    histtype="count",
    annot=True,
)

fig.shift_origin(xshift="w+1c")

# Create histogram for data01 showing the cumulative counts
fig.histogram(
    region=[0, 200, 0, len(data01) + 1],
    projection="X10c",
    frame=Frame(
        axes="wSnE",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=5, tick=1, label="Cumulative counts"),
```

(continues on next page)

(continued from previous page)

```
    ),
    data=data01,
    series=10,
    # Fill bars with GMT pattern 8, with white background and black foreground.
    fill=Pattern(8, bgcolor="white", fgcolor="black"),
    pen="1p,darkgray,solid",
    histtype="count",
    cumulative=True,  # Show cumulative counts
    annot=True,
    annot_offset="-10p",  # Offset the annotations by 10 points in negative␣
↪y-direction
)

fig.show()
```



Overlaid bars

Overlaid or overlapping bars can be achieved by plotting two or several histograms, each for one data set, on top of each other. The legend entry can be specified via the `label` parameter.

Limitations of histograms with overlaid bars are:

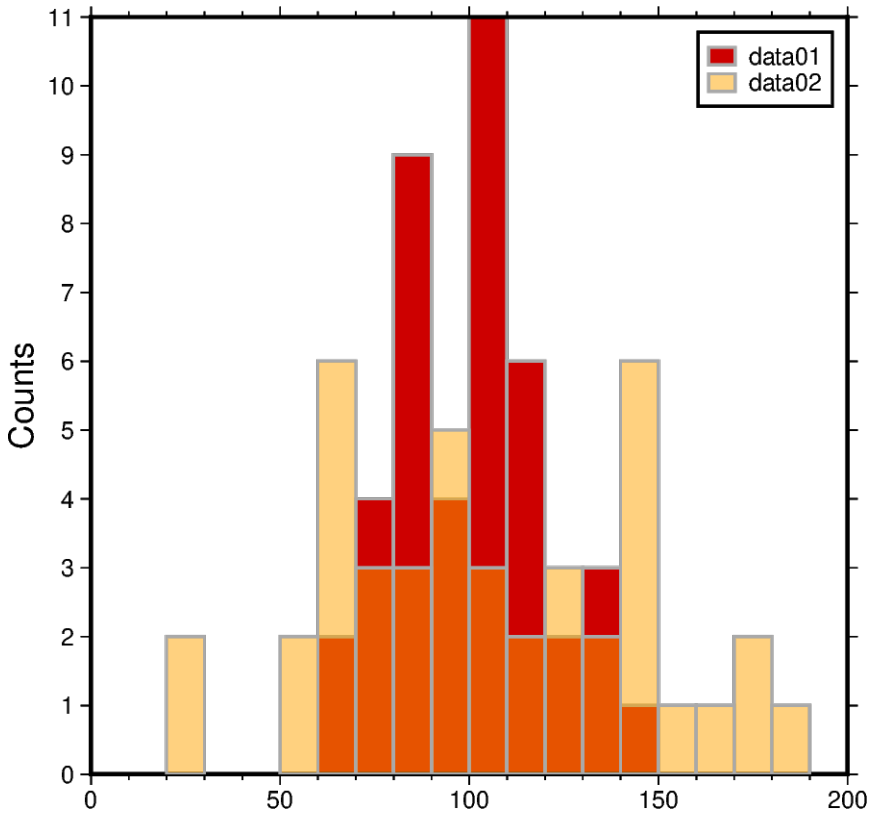
- Mixing of colors or/and patterns
- Visually more colors or/and patterns than data sets
- Visually a "third histogram" (or more in case of more than two data sets)

```
fig = pygmt.Figure()

# Create histogram for data01
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
```

(continues on next page)

(continued from previous page)

```
        axes="WSne",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data01,
    series=10,
    fill="red3",
    pen="1p,darkgray,solid",
    histtype="count",
    # Set legend entry
    label="data01",
)

# Create histogram for data02
# It is plotted on top of the histogram for data01
fig.histogram(
    data=data02,
    series=10,
    # Fill bars with color "orange", use a transparency of 50% ("@50")
    fill="orange@50",
    pen="1p,darkgray,solid",
    histtype="count",
    label="data02",
)

# Add legend
fig.legend()

fig.show()
```



Stacked bars

Histograms with stacked bars are not directly supported by PyGMT. Thus, before plotting, combined data sets have to be created from the single data sets. Then, stacked bars can be achieved similar to overlaid bars via plotting two or several histograms on top of each other.

Limitations of histograms with stacked bars are:

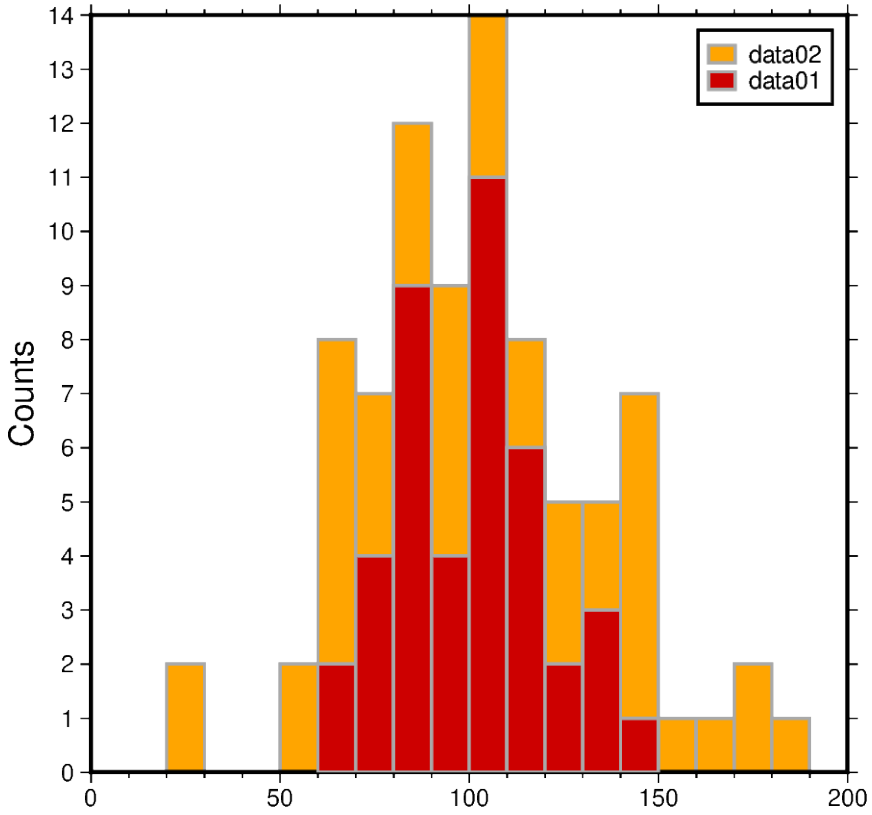
- No common baseline
- Partly not directly clear whether overlaid or stacked bars

```
# Combine the two data sets to one data set
data_merge = np.concatenate((data01, data02), axis=None)

fig = pygmt.Figure()

# Create histogram for data02 by using the combined data set
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data_merge,
    series=10,
    fill="orange",
```

(continues on next page)

(continued from previous page)

```
    pen="1p,darkgray,solid",
    histtype="count",
    # The combined data set appears in the final histogram visually as data set data02
    label="data02",
)

# Create histogram for data01
# It is plotted on top of the histogram for data02
fig.histogram(
    data=data01,
    series=10,
    fill="red3",
    pen="1p,darkgray,solid",
    histtype="count",
    label="data01",
)

# Add legend
fig.legend()

fig.show()
```



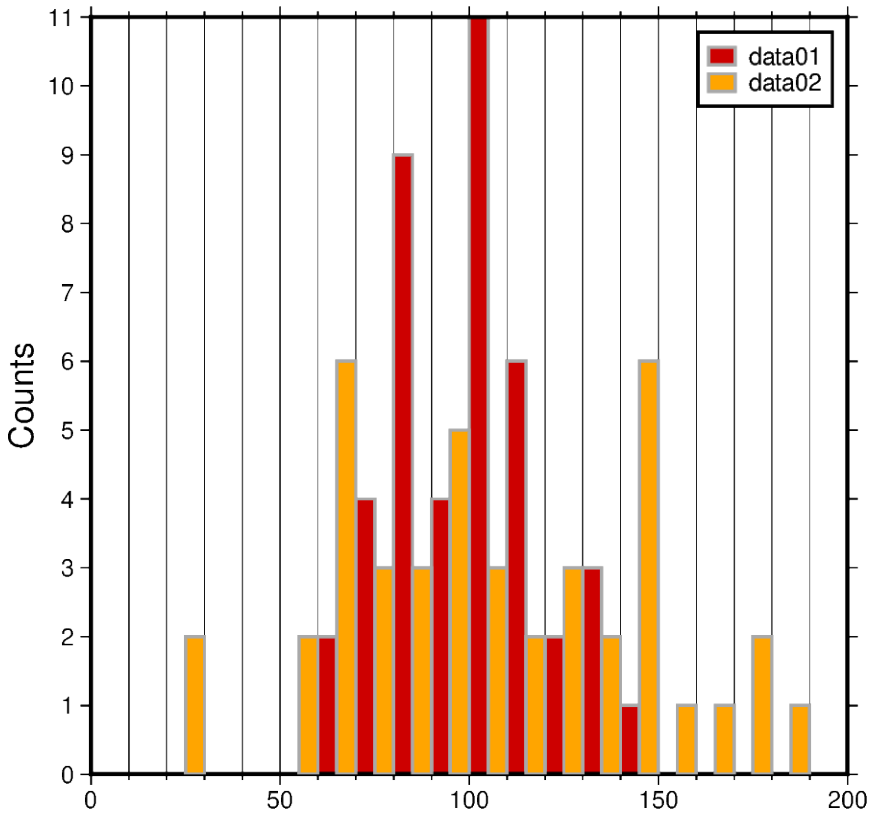
Grouped bars

By setting the `bar_width` parameter in respect to the values passed to the `series` parameter histograms with grouped bars can be created.

Limitations of histograms with grouped bars are:

- Careful setting width and position of the bars in respect to the bin width
- Difficult to see the variations of the single data sets

```
# Width used for binning the data
binwidth = 10

fig = pygmt.Figure()

# Create histogram for data01
fig.histogram(
    region=[0, 200, 0, 0],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=10, grid=10),
        yaxis=Axis(annot=1, tick=1, label="Counts"),
    ),
    data=data01,
    series=binwidth,
    fill="red3",
    pen="1p,darkgray,solid",
    histtype="count",
    # Calculate the bar width in respect to the bin width, here for two data sets half
    # of the bin width
    bar_width=binwidth / 2,
    # Offset the bars to align each bar with the left limit of the corresponding bin
    bar_offset=-binwidth / 4,
    label="data01",
)

# Create histogram for data02
fig.histogram(
    data=data02,
    series=binwidth,
    fill="orange",
    pen="1p,darkgray,solid",
    histtype="count",
    bar_width=binwidth / 2,
    bar_offset=binwidth / 4,
    label="data02",
)

# Add legend
fig.legend()

fig.show()
```



Total running time of the script: (0 minutes 1.000 seconds)

7.2.3 Configuring PyGMT defaults

Default GMT parameters can be set globally or locally using *pygmt.config*.

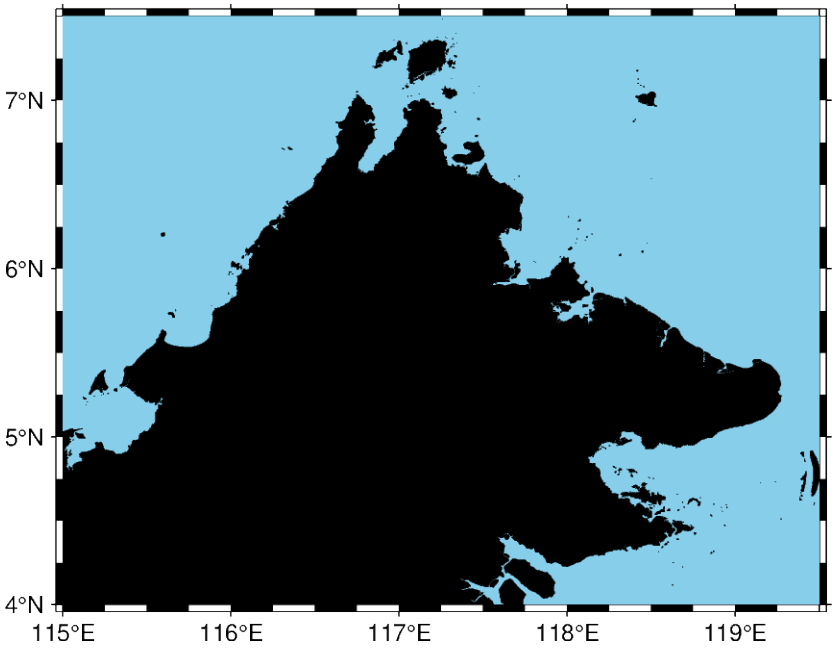
```
import pygmt
```

Configuring default GMT parameters

Users can override default parameters either temporarily (locally) or permanently (globally) using *pygmt.config*. The full list of default parameters that can be changed can be found at https://docs.generic-mapping-tools.org/6.7/gmt.conf.html.

We demonstrate the usage of *pygmt.config* by configuring a map plot.

```
# Start with a basic figure with the default style
fig = pygmt.Figure()
fig.basemap(region=[115, 119.5, 4, 7.5], projection="M10c", frame=True)
fig.coast(land="black", water="skyblue")

fig.show()
```

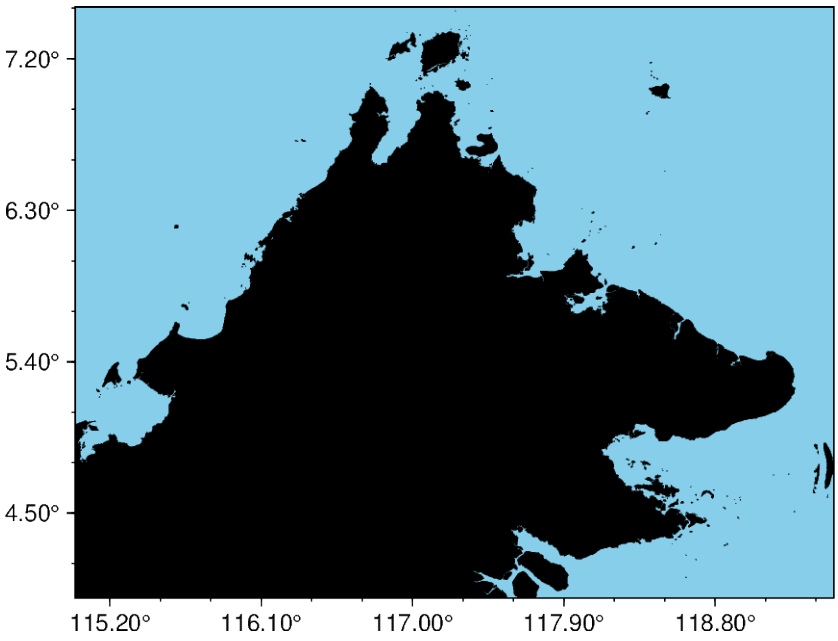


Globally overriding defaults

The `MAP_FRAME_TYPE` parameter specifies the style of map frame to use, of which there are 5 options: `fancy` (default, see above), `fancy+`, `plain`, `graph` (which does not apply to geographic maps) and `inside`.

The `FORMAT_GEO_MAP` parameter controls the format of geographic tick annotations. The default uses degrees and minutes. Here we specify the ticks to be a decimal number of degrees.

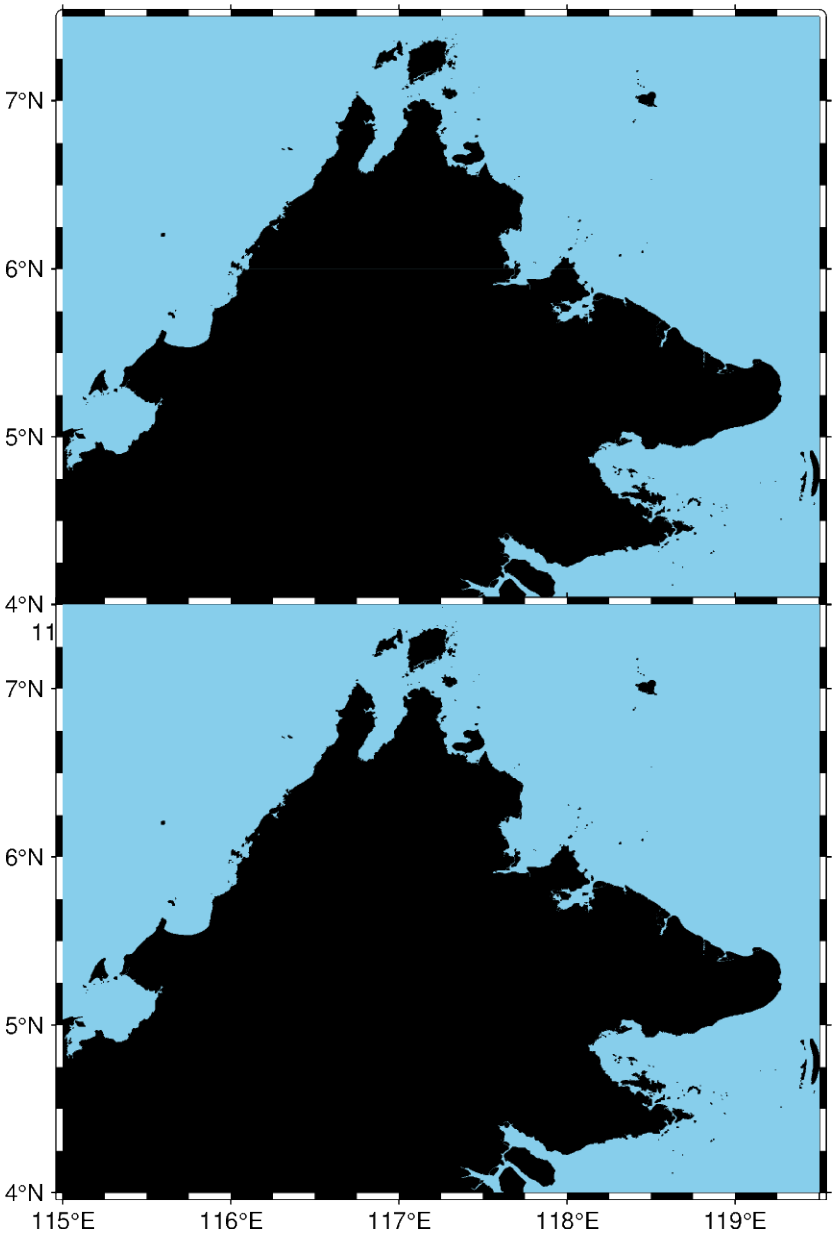
```
fig = pygmt.Figure()

# Configuration for the 'current figure'
pygmt.config(MAP_FRAME_TYPE="plain")
pygmt.config(FORMAT_GEO_MAP="ddd.xx")

fig.basemap(region=[115, 119.5, 4, 7.5], projection="M10c", frame=True)
fig.coast(land="black", water="skyblue")

fig.show()
```



Locally overriding defaults

It is also possible to temporarily override the default parameters, which is very useful for limiting the scope of changes to a particular plot. *pygmt.config* is implemented as a context manager, which handles the setup and teardown of a GMT session. Python users are likely familiar with the `with open(...) as file:` snippet, which returns a `file` context manager. In this way, it can be used to override a parameter for a single command, or a sequence of commands. An application of *pygmt.config* as a context manager is shown below:

```
fig = pygmt.Figure()

# This will have a fancy+ frame
with pygmt.config(MAP_FRAME_TYPE="fancy+"):
    fig.basemap(region=[115, 119.5, 4, 7.5], projection="M10c", frame=True)
fig.coast(land="black", water="skyblue")

# Shift plot origin down by the height of the previous plot to draw another map
fig.shift_origin(yshift="-h")

# This plot retains the default "fancy" frame
fig.basemap(region=[115, 119.5, 4, 7.5], projection="M10c", frame=True)
fig.coast(land="black", water="skyblue")

fig.show()
```



Total running time of the script: (0 minutes 0.429 seconds)

7.2.4 Creating a 3-D perspective image

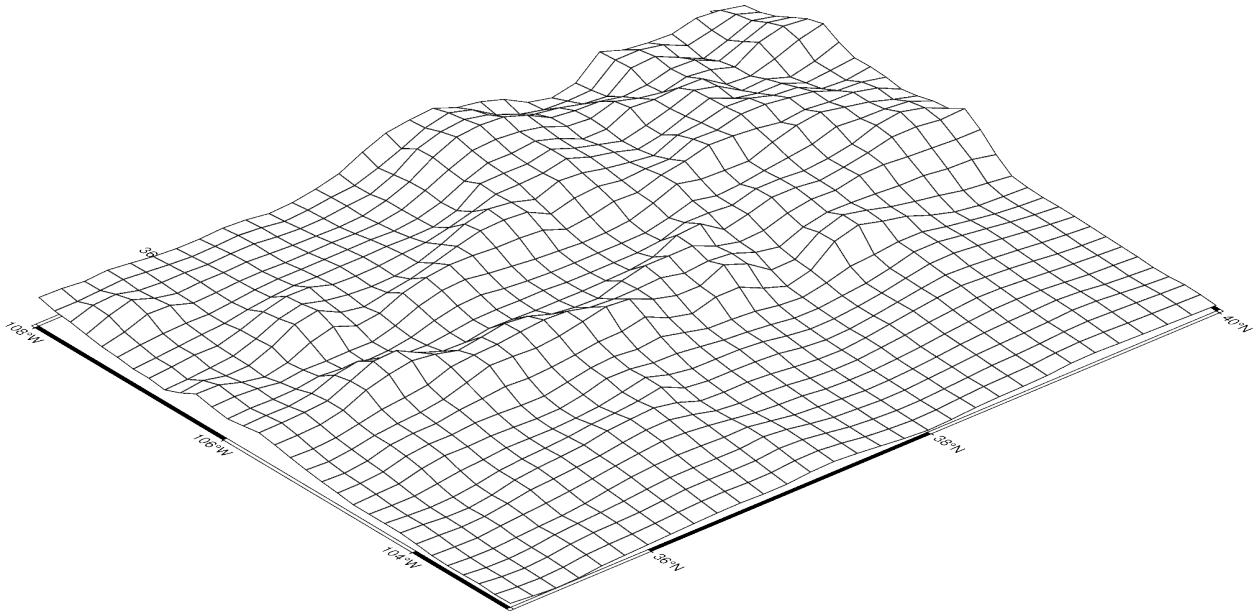
Create 3-D perspective image or surface mesh from a grid using *pygmt.Figure.grdview*.

```
import pygmt
from pygmt.params import Axis, Frame

# Load sample earth relief data
grid = pygmt.datasets.load_earth_relief(resolution="10m", region=[-108, -103, 35, 40])
```

The *pygmt.Figure.grdview* method takes the `grid` input. The `perspective` parameter changes the azimuth and elevation of the viewpoint; the default is [180, 90], which is looking directly down on the figure and north is "up". The `zsize` parameter sets how tall the three-dimensional portion appears.

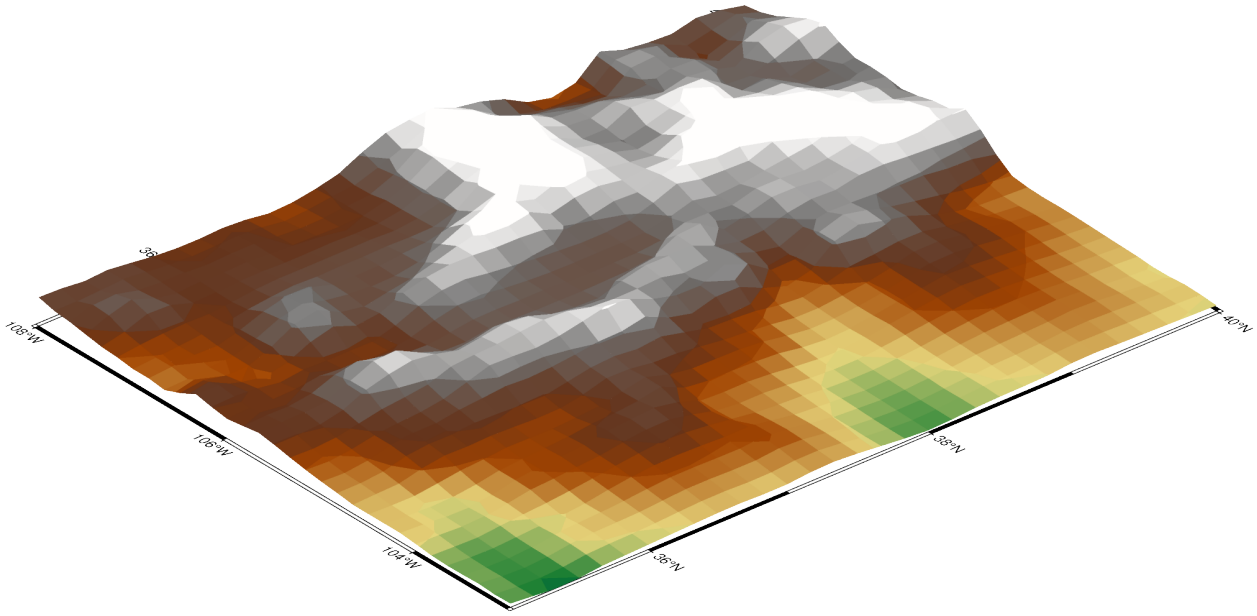
The default grid surface type is *mesh plot*.

```
fig = pygmt.Figure()
fig.grdview(
    grid=grid,
    # Sets the view azimuth as 130 degrees, and the view elevation as 30
    # degrees
    perspective=[130, 30],
    # Sets the x- and y-axis labels, and annotates the west, south, and east axes
    frame=Frame(axes="WSnE", xaxis=Axis(annot=True), yaxis=Axis(annot=True)),
    # Sets a Mercator projection on a 15-centimeter figure
    projection="M15c",
    # Sets the height of the three-dimensional relief at 1.5 centimeters
    zsize="1.5c",
)
fig.show()
```



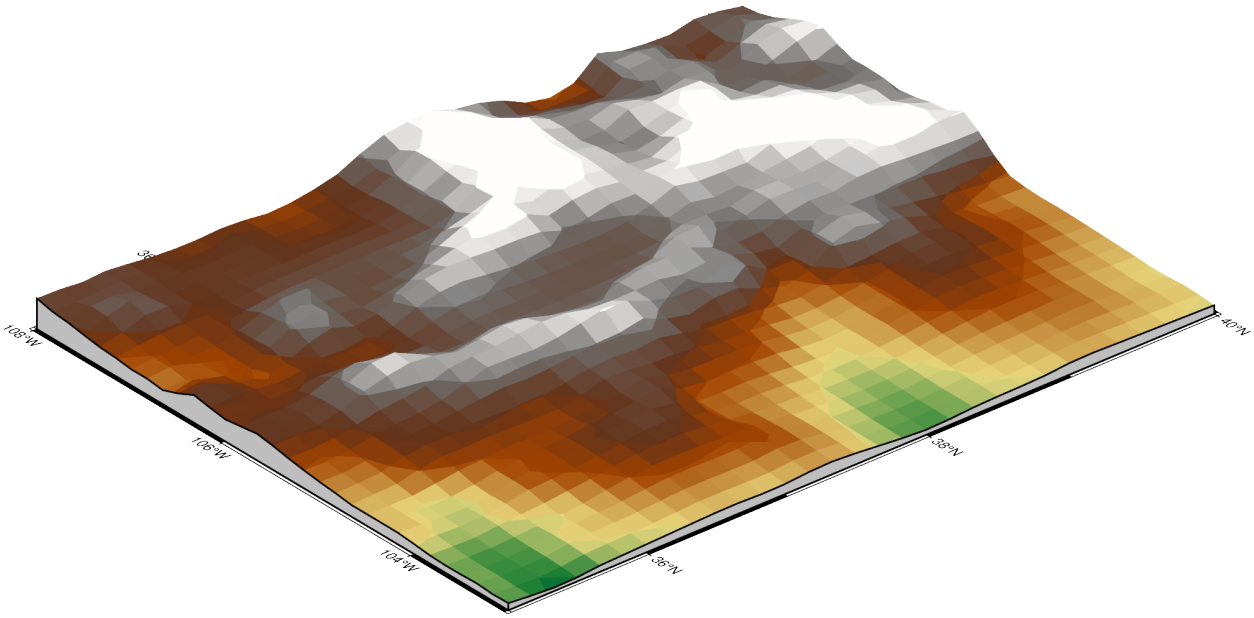
The grid surface type can be set with the `surftype` parameter. The default CPT is *turbo* and can be customized with the `cmap` parameter.

```
fig = pygmt.Figure()
fig.grdview(
    grid=grid,
    perspective=[130, 30],
    frame=Frame(axes="WSnE", xaxis=Axis(annot=True), yaxis=Axis(annot=True,␣
↪tick=True)),
    projection="M15c",
    zsize="1.5c",
    surftype="surface",
    cmap="gmt/geo",  # Set the CPT to "geo"
)
fig.show()
```



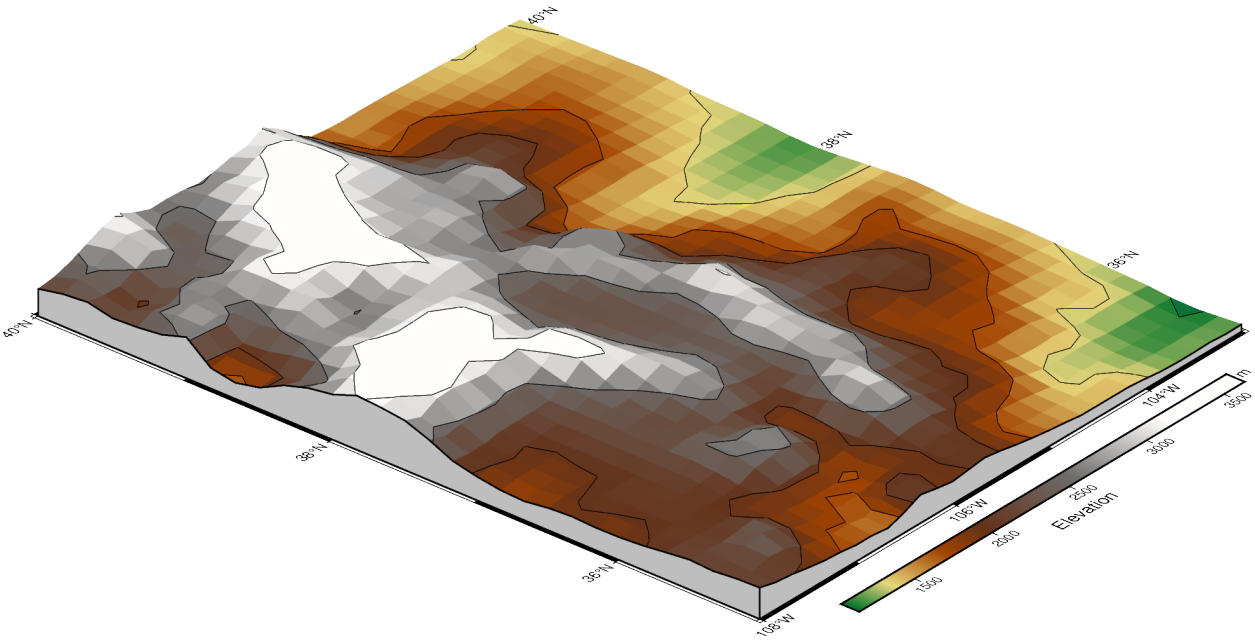
The `plane` parameter sets the elevation and color of a plane that provides a fill below the surface relief.

```
fig = pygmt.Figure()
fig.grdview(
    grid=grid,
    perspective=[130, 30],
    frame=Frame(axes="WSnE", xaxis=Axis(annot=True), yaxis=Axis(annot=True,␣
↪tick=True)),
    projection="M15c",
    zsize="1.5c",
    surftype="surface",
    cmap="gmt/geo",
    plane=1000,  # Set the plane elevation to 1,000 meters
    facade_fill="gray",  # Color the facade in "gray"
)
fig.show()
```



The `perspective` azimuth can be changed to set the direction that is "up" in the figure. The `contour_pen` parameter sets the pen used to draw contour lines on the surface. *`pygmt.Figure.colorbar`* can be used to add a colorbar to the figure. The `cmap` parameter does not need to be passed again. To keep the colorbar's alignment similar to the figure, use `True` as argument for the `perspective` parameter.

```
fig = pygmt.Figure()
fig.grdview(
    grid=grid,
    # Set the azimuth to -130 (230) degrees and the elevation to 30 degrees
    perspective=[-130, 30],
    frame=Frame(
        axes="WSnE",
        xaxis=Axis(annot=True, tick=True),
        yaxis=Axis(annot=True, tick=True),
    ),
    projection="M15c",
    zsize="1.5c",
    surftype="surface",
    cmap="gmt/geo",
    plane=1000,
    facade_fill="gray",
    # Set the contour pen thickness to 0.1 points
    contour_pen="0.1p",
)
fig.colorbar(perspective=True, annot=500, label="Elevation", unit="m")
fig.show()
```



Total running time of the script: (0 minutes 0.789 seconds)

7.2.5 Creating a map with contour lines

Plotting a contour map is handled by *pygmt.Figure.grdcontour*.

```
import pygmt

# Load sample earth relief data
grid = pygmt.datasets.load_earth_relief(resolution="05m", region=[-92.5, -82.5, -3,␣
↪7])
```

Create contour plot

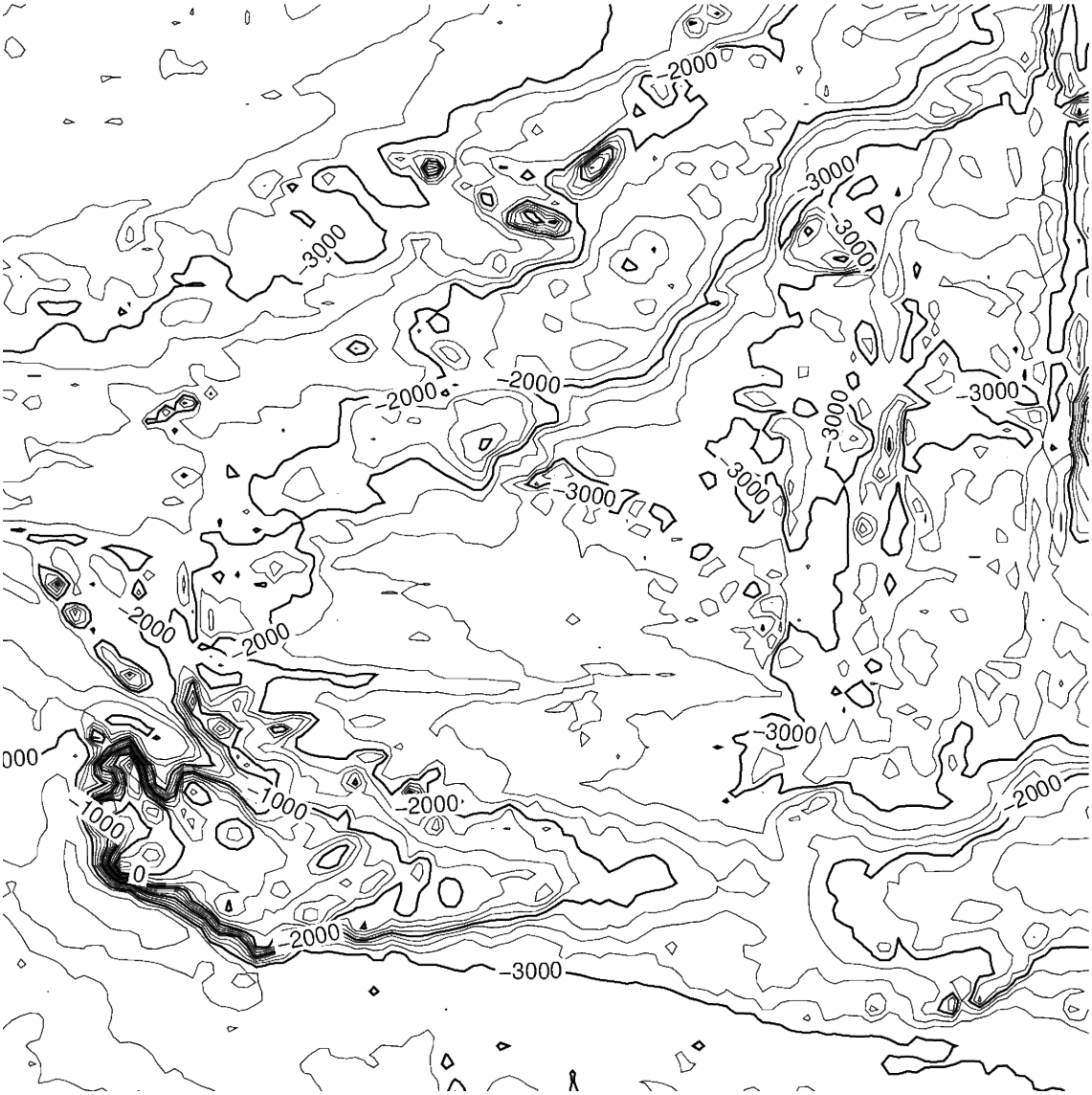
The *pygmt.Figure.grdcontour* method takes the grid input. It plots annotated contour lines, which are thicker and have the elevation/depth written on them, and unannotated contour lines. In the example below, the default contour line intervals are 500 meters, with an annotated contour line every 1,000 meters. By default, it plots the map with the equidistant cylindrical projection and with no frame.

```
fig = pygmt.Figure()
fig.grdcontour(grid=grid)
fig.show()
```

Contour line settings

Use the `annotation` and `levels` parameters to adjust contour line intervals. In the example below, there are contour intervals every 250 meters and annotated contour lines every 1,000 meters.

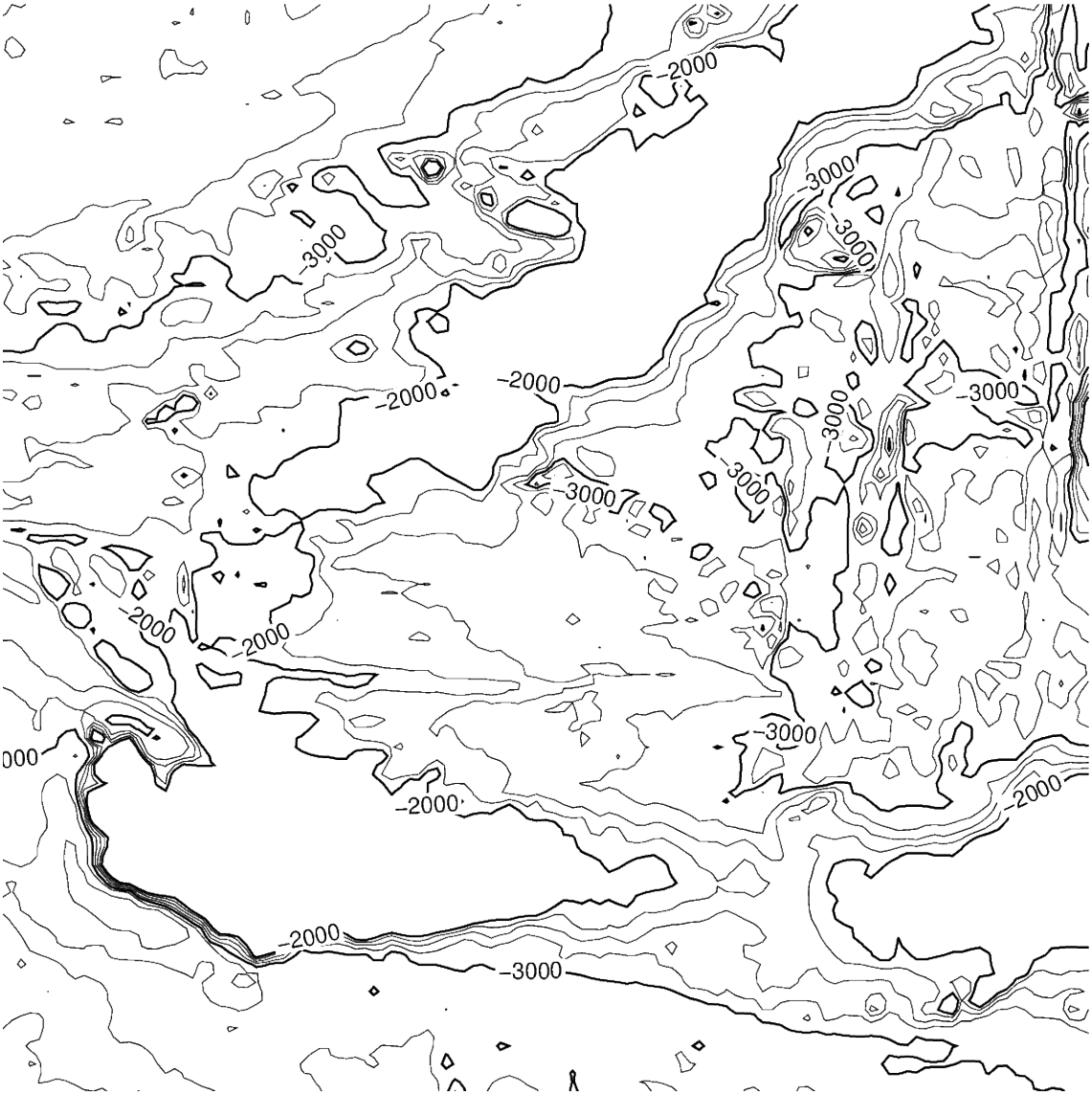
```
fig = pygmt.Figure()
fig.grdcontour(grid=grid, annotation=1000, levels=250)
fig.show()
```



Contour limits

The `limit` parameter sets the minimum and maximum values for the contour lines. The parameter takes the low and high values, and is either a list (as below) or a string `limit="-4000/-2000"`.

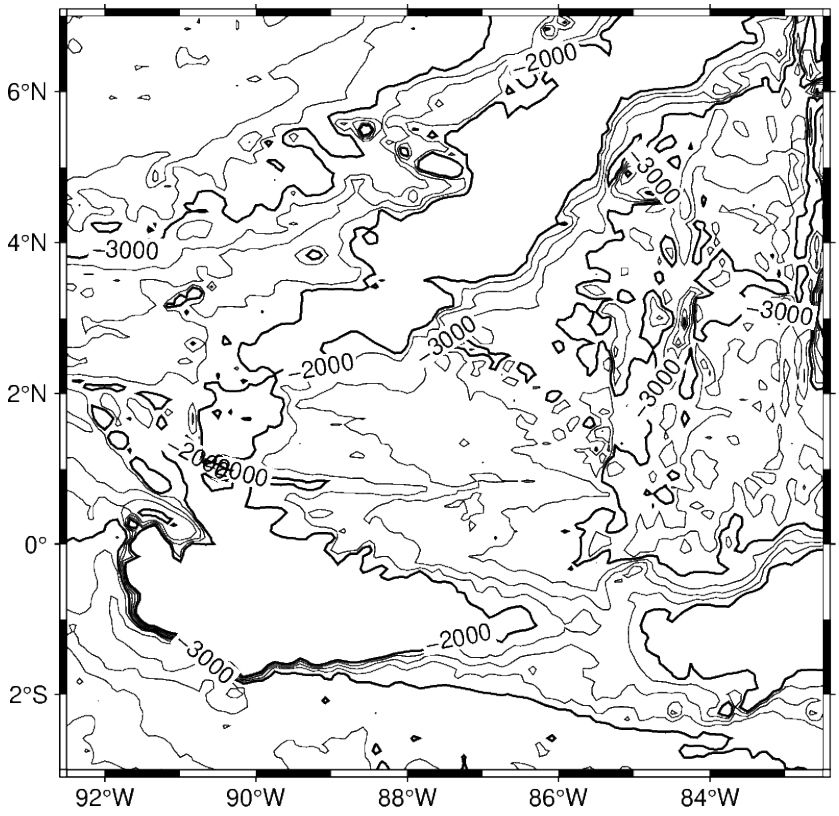
```
fig = pygmt.Figure()
fig.grdcontour(grid=grid, annotation=1000, levels=250, limit=[-4000, -2000])
fig.show()
```



Map settings

The *pygmt.Figure.grdcontour* method accepts additional parameters, including setting the projection and frame.

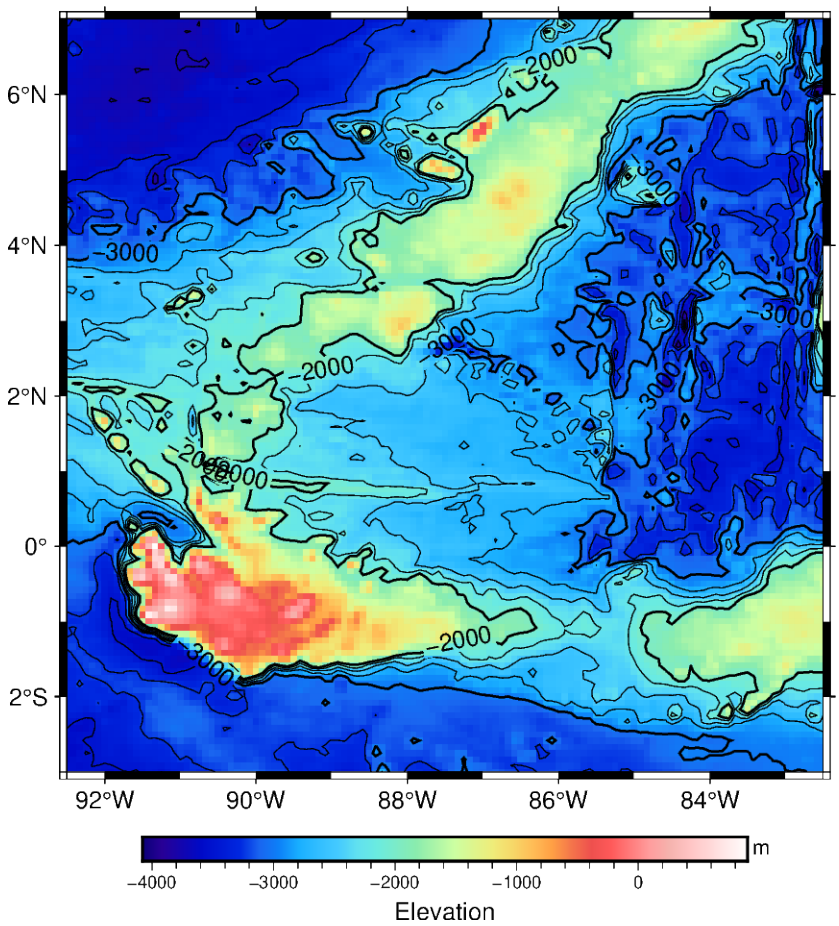
```
fig = pygmt.Figure()
fig.grdcontour(
    grid=grid,
    annotation=1000,
    levels=250,
    limit=[-4000, -2000],
    projection="M10c",
    frame=True,
)
fig.show()
```



Adding a colormap

The *pygmt.Figure.grdimage* method can be used to add a colormap to the contour map. It must be called prior to *pygmt.Figure.grdcontour* to keep the contour lines visible on the final map. If the `projection` parameter is specified in the *pygmt.Figure.grdimage* method, it does not need to be repeated in the *pygmt.Figure.grdcontour* method. Finally, a colorbar is added using the *pygmt.Figure.colorbar* method.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid, cmap="gmt/haxby", projection="M10c", frame=True)
fig.grdcontour(grid=grid, annotation=1000, levels=250, limit=[-4000, -2000])
fig.colorbar(label="Elevation", unit="m")
fig.show()
```



Total running time of the script: (0 minutes 1.182 seconds)

7.2.6 Creating legends

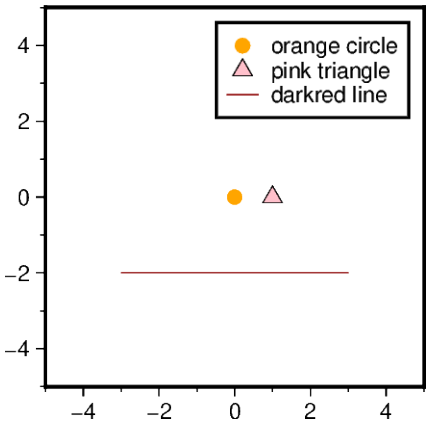
The *pygmt.Figure.legend* method creates legends, whereby auto-legends as well as manually created legends are supported.

```
import io

import pygmt
from pygmt.params import Box, Frame, Position
```

Create an auto-legend

An auto-legend can be created for the methods *pygmt.Figure.plot* and *pygmt.Figure.plot3d*, *pygmt.Figure.hlines* and *pygmt.Figure.vlines* as well as *pygmt.Figure.histogram*. Therefore the `label` parameter has to be specified to state the desired text for the legend entry (white spaces are supported). Here, we use *pygmt.Figure.plot*, exemplary. By default, the legend is placed in the Upper Right corner and surrounded by a box with a white fill and a 1-point thick, black, solid outline. The order of the legend entries (top to bottom) is determined by the plotting order. Optionally, to adjust the legend, append different modifiers to the string passed to `label`. For a list of available modifiers see https://docs.generic-mapping-tools.org/6.7/gmt.html#l-full. To create a *multiple-column legend* **+N** is used with the desired number of columns.

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=True)

# Plot two data points and one line
fig.plot(x=0, y=0, style="c0.2c", fill="orange", label="orange circle")
fig.plot(x=1, y=0, style="t0.3c", fill="pink", pen="black", label="pink triangle")
fig.plot(x=[-3, 3], y=[-2, -2], pen="darkred", label="darkred line")

# Add a legend based on the explanation text given via the "label" parameter.
fig.legend()

fig.show()
```



Adjust position and line spacing

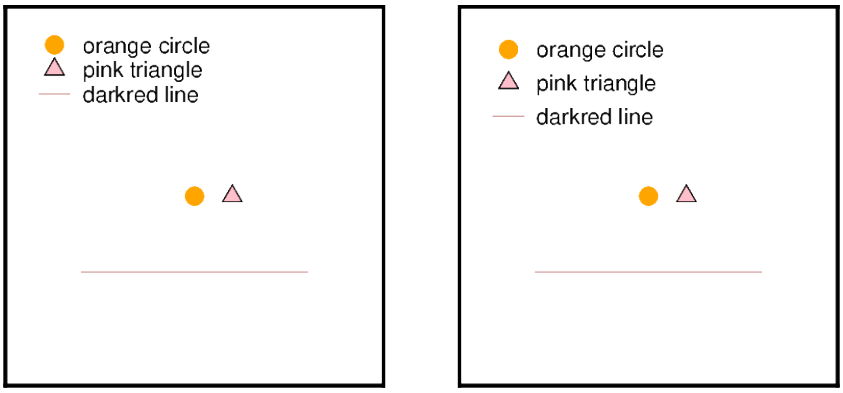
Use the `position` parameter to adjust the position of the legend. Note, no box is drawn by default if `position` is used

```
fig = pygmt.Figure()
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rltb"))

fig.plot(x=0, y=0, style="c0.25c", fill="orange", label="orange circle")
fig.plot(x=1, y=0, style="t0.3c", fill="pink", pen="black", label="pink triangle")
fig.plot(x=[-3, 3], y=[-2, -2], pen="darkred", label="darkred line")

# Set the reference point to the Top Left corner inside the plot and use offsets of␣
↪0.3
# and 0.2 centimeters in the x- and y-directions, respectively.
fig.legend(position=Position("TL", offset=(0.3, 0.2)))

fig.shift_origin(xshift="w+1c")
fig.basemap(region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rltb"))

fig.plot(x=0, y=0, style="c0.25c", fill="orange", label="orange circle")
fig.plot(x=1, y=0, style="t0.3c", fill="pink", pen="black", label="pink triangle")
fig.plot(x=[-3, 3], y=[-2, -2], pen="darkred", label="darkred line")

# Use a line spacing factor of 1.5
fig.legend(position=Position("TL", offset=(0.3, 0.2)), line_spacing=1.5)
```

(continues on next page)

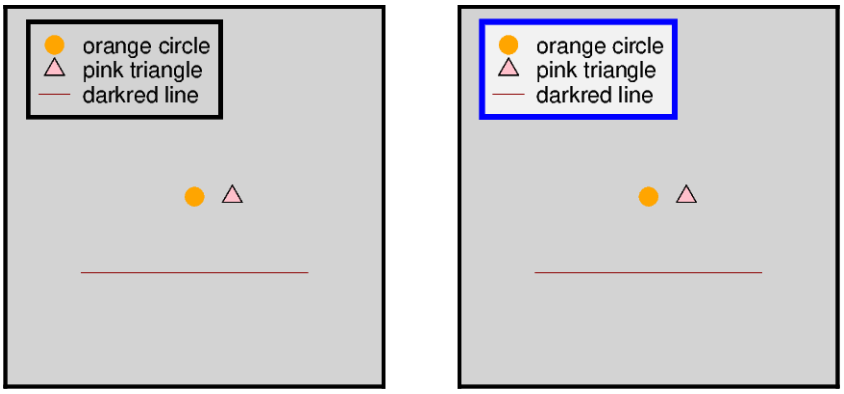
(continued from previous page)

```
fig.show()
```



Add a box

```
# Use the ``box`` parameter for adjusting the box around the legend.

fig = pygmt.Figure()
fig.basemap(
    region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rltb", fill="lightgray
↪")
)

fig.plot(x=0, y=0, style="c0.25c", fill="orange", label="orange circle")
fig.plot(x=1, y=0, style="t0.3c", fill="pink", pen="black", label="pink triangle")
fig.plot(x=[-3, 3], y=[-2, -2], pen="darkred", label="darkred line")

fig.legend(position=Position("TL", offset=(0.3, 0.2)), box=True)

fig.shift_origin(xshift="w+1c")
fig.basemap(
    region=[-5, 5, -5, 5], projection="X5c", frame=Frame(axes="rltb", fill="lightgray
↪")
)

fig.plot(x=0, y=0, style="c0.25c", fill="orange", label="orange circle")
fig.plot(x=1, y=0, style="t0.3c", fill="pink", pen="black", label="pink triangle")
fig.plot(x=[-3, 3], y=[-2, -2], pen="darkred", label="darkred line")

# Add a box with a 2-point thick blue, solid outline and a white fill with a
# transparency of 30 percent ("@30").
fig.legend(
    position=Position("TL", offset=(0.3, 0.2)), box=Box(pen="2p,blue", fill="white@30
↪")
)

fig.show()
```



Create a manual legend

For more complicated legends, users need to prepare a legend specification with instructions for the layout of the legend entries. In PyGMT, the legend specification can be either an ASCII file or an `io.StringIO` object. Both are passed to the `spec` parameter of *`pygmt.Figure.legend`*. Multiple legend codes are available to create complicated legends. In the example below we show an subset; a full overview can be found at https://docs.generic-mapping-tools.org/6.7/legend.html#legend-codes. It's also supported to include length scales (for geographic projections), faults, and images as well as to add specific lines.

The following example is orientated on the related GMT example at https://docs.generic-mapping-tools.org/6.7/legend.html#examples, but modified to use an `io.StringIO` object.

We start with setting up the `io.StringIO` object.

```
spec_io = io.StringIO(
    """
G -0.1c
H 24p,Times-Roman My Map Legend
D 0.2c 1p
N 2
V 0 1p
S 0.1c c 0.20c p300/12 0.25p 0.3c This circle is hachured
S 0.1c e 0.20c yellow 0.25p 0.3c This ellipse is yellow
S 0.1c w 0.20c green 0.25p 0.3c This wedge is green
S 0.1c f 0.25c blue 0.25p 0.3c This is a fault
S 0.1c - 0.15c - 0.25p,- 0.3c A contour
S 0.1c v 0.25c magenta 0.5p 0.3c This is a vector
S 0.1c i 0.20c cyan 0.25p 0.3c This inverse triangle is cyan
D 0.2c 1p
V 0 1p
N 1
G 0.1c
M 5 5 600+u+f
G 0.1c
I @SOEST_block4.png 3i CT
G 0.05c
L 9p,Times-Roman R Smith et al., @%5%J. Geophys. Res., 99@%%, 2000
G 0.1c
T Let us just try some simple text that can go on a few lines.
T There is no easy way to predetermine how many lines may be required
T so we may have to adjust the height to get the right size box.
```

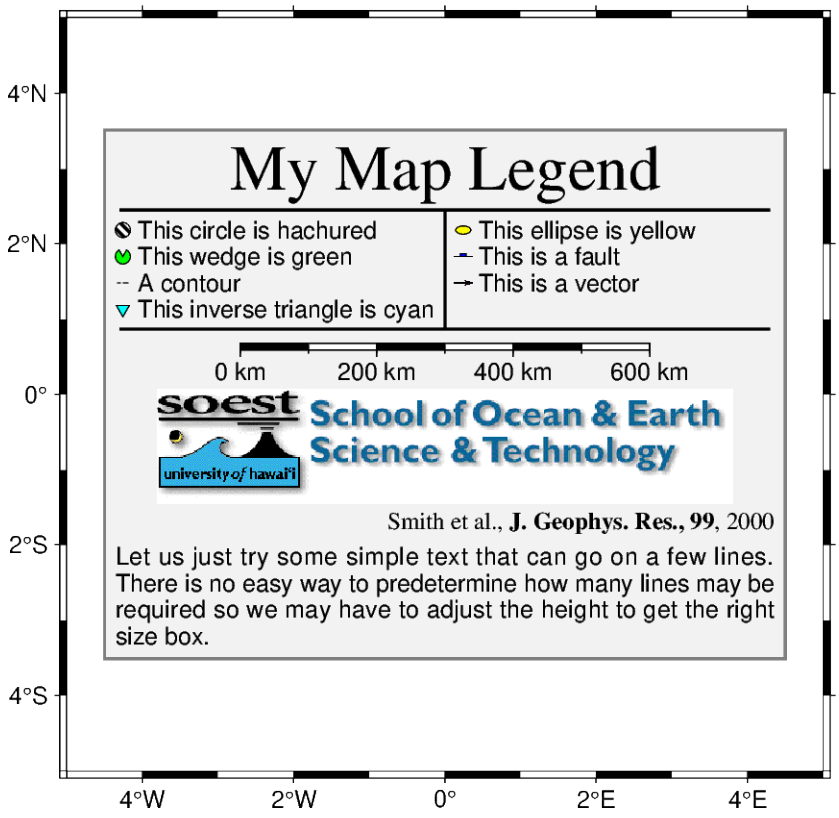
(continues on next page)

(continued from previous page)

```
"""
)
```

Now, we can add a legend based on this `io.StringIO` object. For multi-columns legends, the width must be specified.

```
fig = pygmt.Figure()
# Note, that we are now using a Mercator projection
fig.basemap(region=[-5, 5, -5, 5], projection="M10c", frame=True)
# Pass the io.StringIO object to the "spec" parameter
fig.legend(
    spec=spec_io, position="MC", width="9c", box=Box(pen="1p,gray50", fill="gray95")
)
fig.show()
```



Total running time of the script: (0 minutes 1.402 seconds)

7.2.7 Draping a dataset on top of a topographic surface

It can be visually appealing to "drape" a dataset over a topographic surface. This can be accomplished using the `drape_grid` parameter of *`pygmt.Figure.grdview`*.

This tutorial consists of two examples:

1. Draping a grid
2. Draping an image

```
# Load the required packages
import pygmt
import rasterio
import xarray as xr
```

1. Drapping a grid

In the first example, the seafloor crustal age is plotted with color-coding on top of the topographic map of an area of the Mid-Atlantic Ridge.

```
# Define the study area in degrees East or North
region_2d = [-50, 0, 36, 70]  # [lon_min, lon_max, lat_min, lat_max]

# Download elevation and crustal age grids for the study region with a resolution of␣
↪10
# arc-minutes and load them into xarray.DataArrays
grd_relief = pygmt.datasets.load_earth_relief(resolution="10m", region=region_2d)
grd_age = pygmt.datasets.load_earth_age(resolution="10m", region=region_2d)

# Determine the 3-D region from the minimum and maximum values of the relief grid
region_3d = [*region_2d, grd_relief.min().to_numpy(), grd_relief.max().to_numpy()]
```

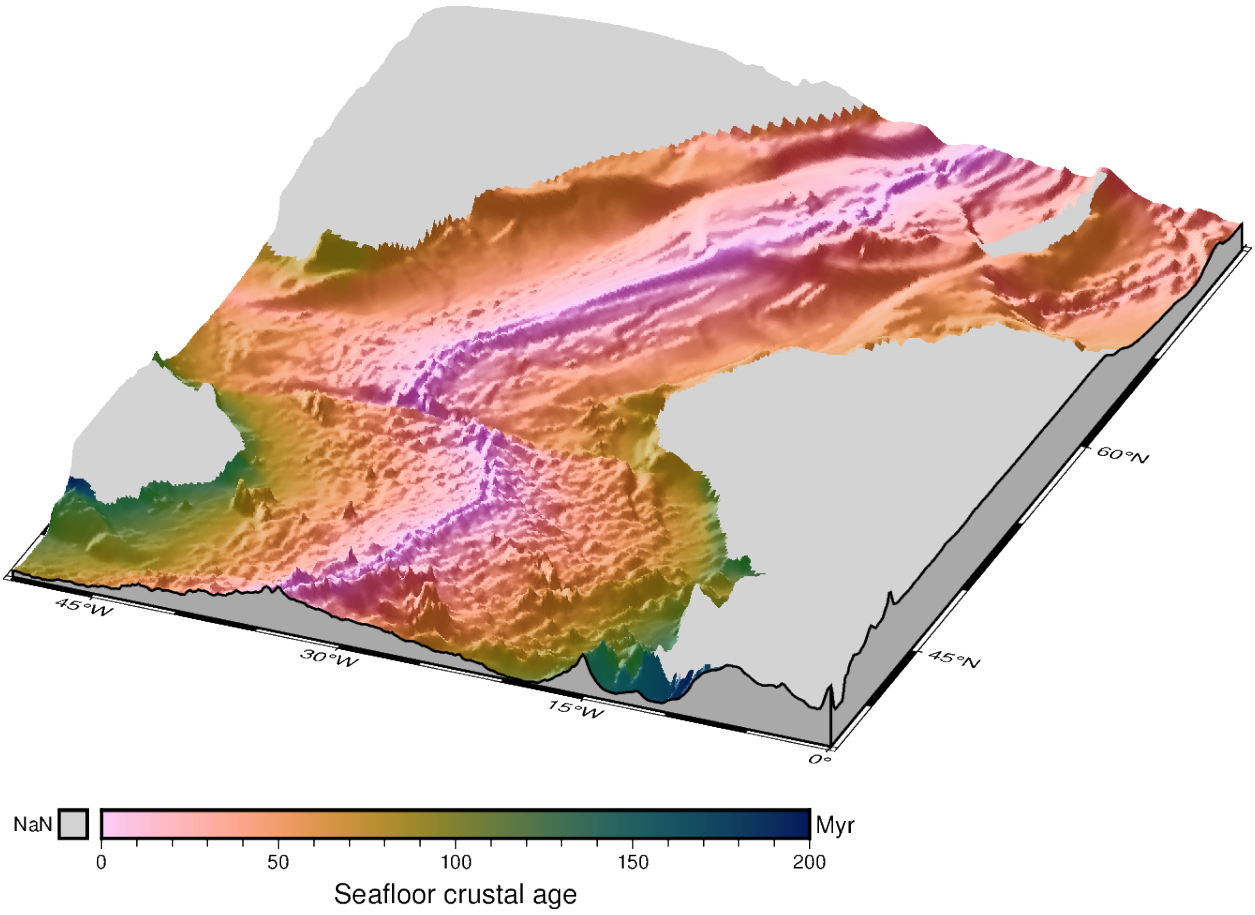
The topographic surface is created based on the grid passed to the `grid` parameter of *pygmt.Figure.grdview*; here we use a grid of the Earth relief. To add a color-coding based on *another* grid we have to pass a second grid to the `drape_grid` parameter; here we use a grid of the crustal age. In this case the colormap specified via the `cmap` parameter applies to the grid passed to `drape_grid`, not to `grid`. The azimuth and elevation of the 3-D plot are set via the `perspective` parameter.

```
fig = pygmt.Figure()

# Set up colormap for the crustal age
pygmt.config(COLOR_NAN="lightgray")
pygmt.makecpt(cmap="SCM/batlow", series=[0, 200, 1], reverse=True, overrule_bg=True)

fig.grdview(
    projection="M12c",  # Mercator projection with a width of 12 cm
    region=region_3d,
    grid=grd_relief,  # Use elevation grid for z values
    drape_grid=grd_age,  # Use crustal age grid for color-coding
    cmap=True,  # Use colormap created for the crustal age
    surftype="image",  # Create an image plot
    # Use an illumination from the azimuthal directions 0° (north) and 270°
    # (west) with a normalization via a cumulative Laplace distribution for
    # the shading
    shading="+a0/270+ne0.6",
    perspective=[157.5, 30],  # Azimuth and elevation for the 3-D plot
    zsize="1.5c",
    facade_fill="darkgray",
    frame=True,
)

# Add colorbar for the crustal age
fig.colorbar(label="Seafloor crustal age", unit="Myr", nan=True)
```

(continues on next page)

(continued from previous page)

```
fig.show()
```



2. Draping an image

In the second example, the flag of the European Union (EU) is plotted on top of a topographic map of northwest Europe. This example is modified from GMT example 32. We have to consider the dimension of the image we want to drap. The image we will download in this example has 1000 x 667 pixels, i.e. an aspect ratio of 3 x 2.

```
# Define the study area in degrees East or North, with an extend of 6 degrees for
# the longitude and 4 degrees for the latitude
region_2d = [3, 9, 50, 54]  # [lon_min, lon_max, lat_min, lat_max]

# Download elevation grid for the study region with a resolution of 30 arc-seconds and
# pixel registration and load it into an xarray.DataArray
grd_relief = pygmt.datasets.load_earth_relief(resolution="30s", region=region_2d)

# Determine the 3-D region from the minimum and maximum values of the relief grid
region_3d = [*region_2d, grd_relief.min().to_numpy(), grd_relief.max().to_numpy()]

# Load a PNG image of the EU flag using rasterio into an xarray.DataArray.
# The original image is available on Wikimedia Commons at
# https://commons.wikimedia.org/wiki/File:Flag_of_Europe.svg
```

(continues on next page)

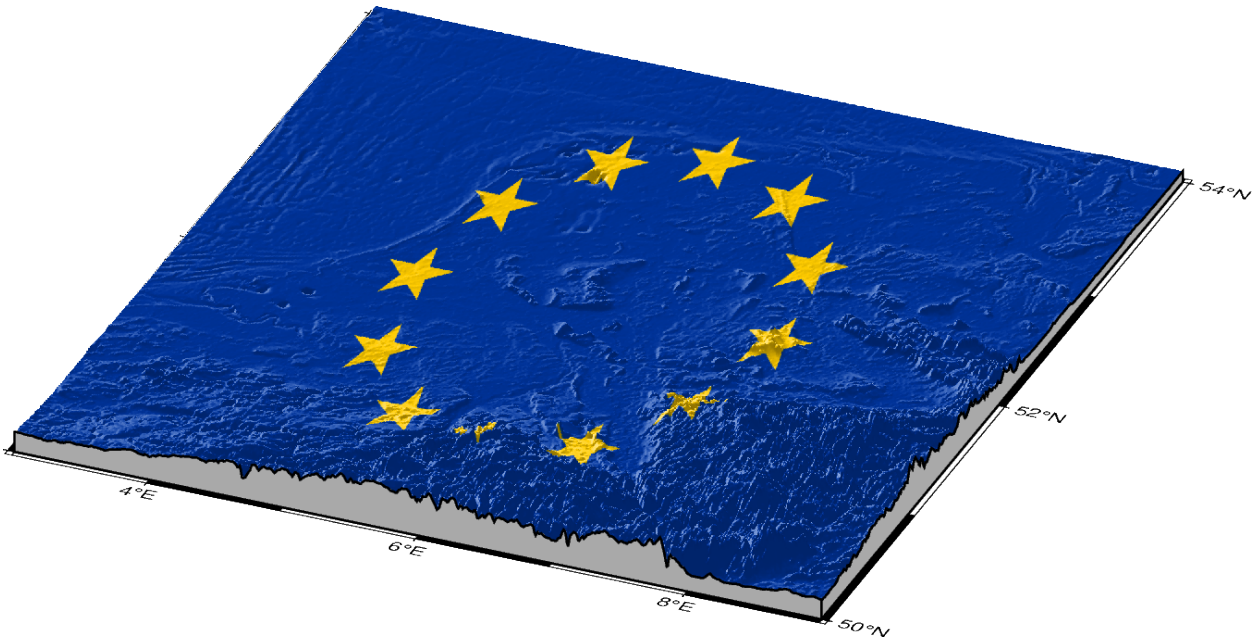
(continued from previous page)

```
# but we use a cached version on the GMT data server.
url_to_image = (

↪"https://github.com/GenericMappingTools/gmtserver-admin/raw/master/cache/euflag.png"
)
with rasterio.open(url_to_image) as dataset:
    data = dataset.read()
    drape_grid = xr.DataArray(data, dims=("band", "y", "x"))
```

```
grdblend [NOTICE]: Remote data courtesy of GMT data server␣
↪https://opentopography.s3.sdsc.edu/gmtdata␣
↪[https://opentopography.s3.sdsc.edu/gmtdata]
grdblend [NOTICE]: SRTM15 Earth Relief v2.7 at 30x30 arc seconds reduced by Gaussian␣
↪Cartesian filtering (2.6 km fullwidth) [Tozer et al., 2019].
grdblend [NOTICE]:   -> Download 15x15 degree grid tile (earth_relief_30s_g): N45E000
/
↪home/runner/micromamba/envs/pygmt/lib/python3.15/site-packages/rasterio/__init__.py:367:␣
↪NotGeoreferencedWarning: Dataset has no geotransform, gcps, or rpcs. The identity␣
↪matrix will be returned.
  dataset = DatasetReader(path, driver=driver, sharing=sharing,␣
↪thread_safe=thread_safe, **kwargs)
```

Again we create a 3-D plot with *pygmt.Figure.grdview* and pass an Earth relief grid to the `grid` parameter to create the topographic surface. But now we pass the PNG image which was loaded into an `xarray.DataArray` to the `drape_grid` parameter.

```
fig = pygmt.Figure()

# Set up a colormap with two colors for the EU flag: blue (0/51/153) for the␣
↪background
# (value 0 in the netCDF file -> lower half of 0-255 range) and yellow (255/204/0) for
# the stars (value 255 -> upper half)
pygmt.makecpt(cmap="0/51/153,255/204/0", series=[0, 256, 128])

fig.grdview(
    projection="M12c",  # Mercator projection with a width of 12 cm
    region=region_3d,
    grid=grd_relief,  # Use elevation grid for z values
    drape_grid=drape_grid,  # Drape image grid for the EU flag on top
    cmap=True,  # Use colormap defined for the EU flag
    surftype="image",  # Create an image plot
    # Use an illumination from the azimuthal directions 0° (north) and 270° (west)␣
↪with
    # a normalization via a cumulative Laplace distribution for the shading
    shading="+a0/270+ne0.6",
    perspective=(157.5, 30),  # Define azimuth, elevation for the 3-D plot
    zsize="1c",
    facade_fill="darkgray",
    frame=True,
)

fig.show()
```




```
/
↪home/runner/micromamba/envs/pygmt/lib/python3.15/site-packages/rasterio/__init__.py:377:␣
↪NotGeoreferencedWarning: The given matrix is equal to Affine.identity or its␣
↪flipped counterpart. GDAL may ignore this matrix and save no geotransform without␣
↪raising an error. This behavior is somewhat driver-specific.
  dataset = writer(
```

Total running time of the script: (0 minutes 9.010 seconds)

7.2.8 Interactive data visualization using `Panel`

> **Note**
>
> Please run the following code examples in a notebook environment otherwise the interactive parts of this tutorial will not work. You can use the button "Download Jupyter notebook" at the bottom of this page to download this script as a Jupyter notebook.

The library Panel can be used to create interactive dashboards by connecting user-defined widgets to plots. `Panel` can be used as an extension to Jupyter notebook/lab.

This tutorial is split into three parts:

- Make a static map
- Make an interactive map
- Add a grid for Earth relief

Import the required packages

```
import numpy as np
import panel as pn
import pygmt
from pygmt.params import Axis
```

(continues on next page)

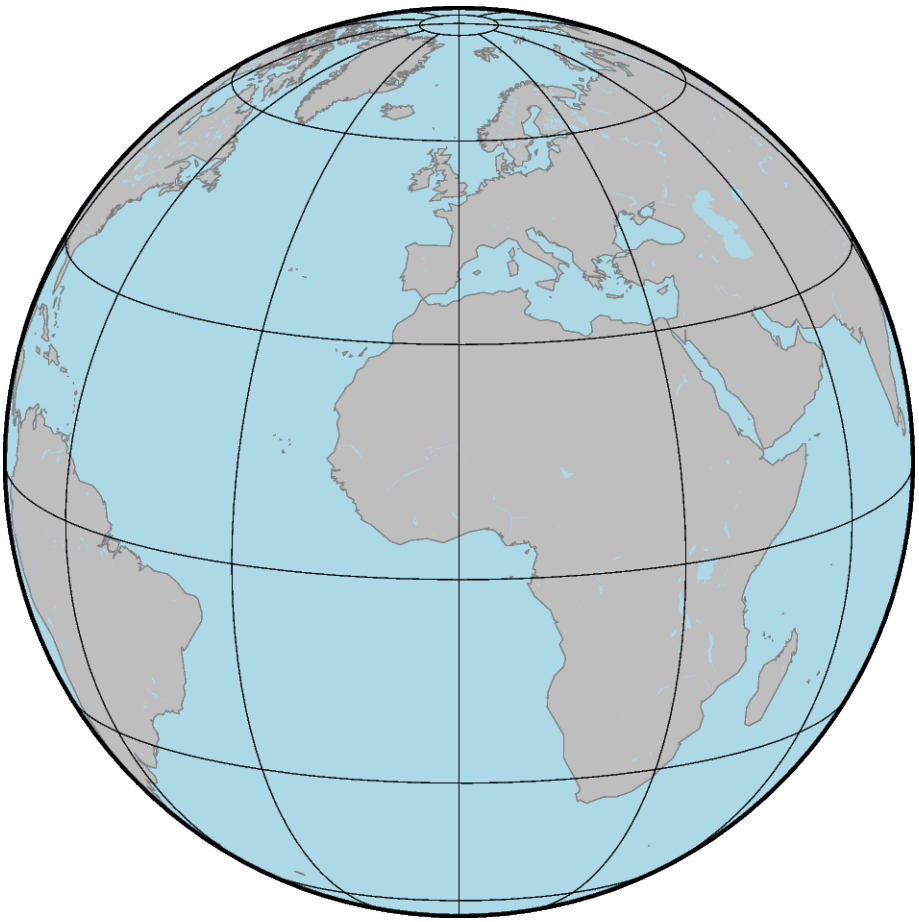
(continued from previous page)

```
pn.extension()
```

Make a static map

The Orthographic projection can be used to show the Earth as a globe. Land and water masses are filled with colors via the `land` and `water` parameters of *pygmt.Figure.coast*, respectively. Coastlines are added using the `shorelines` parameter.

```
# Create a new instance or object of the pygmt.Figure() class
fig = pygmt.Figure()
fig.coast(
    # Orthographic projection (G) with projection center at 0° East and
    # 15° North and a width of 12 centimeters
    projection="G0/15/12c",
    region="g",  # global
    frame=Axis(grid=30),  # Add gridlines in steps of 30 degrees on top
    land="gray",  # Color land masses in "gray"
    water="lightblue",  # Color water masses in "lightblue"
    # Add coastlines with a 0.25-point thick pen in "gray50"
    shorelines="1/0.25p,gray50",
)
fig.show()
```



Make an interactive map

To generate a rotation of the Earth around the vertical axis, the central longitude of the Orthographic projection is varied iteratively in steps of 10 degrees. The library `Panel` is used to create an interactive dashboard with a slider (works only in a notebook environment, e.g., Jupyter notebook).

```
# Create a slider
slider_lon = pn.widgets.DiscreteSlider(
    name="Central longitude",  # Give name for quantity shown at the slider
    options=list(np.arange(0, 361, 10)),  # Range corresponding to longitude
    value=0,  # Set start value
)


@pn.depends(central_lon=slider_lon)
def view(central_lon):
    """
    Define a function for plotting the single slices.
    """
    # Create a new instance or object of the pygmt.Figure() class
    fig = pygmt.Figure()
    fig.coast(
        # Vary the central longitude used for the Orthographic projection
        projection=f"G{central_lon}/15/12c",
        region="g",
        frame=Axis(grid=30),
        land="gray",
        water="lightblue",
        shorelines="1/0.25p,gray50",
    )
    return fig


# Make an interactive dashboard
pn.Column(slider_lon, view)
```

```
Column
    [0] DiscreteSlider(formatter='%d', label='Central longitude', name='Central␣
↪longitude', options=[np.int64(0), ...], value=0)
    [1] ParamFunction(function, _pane=PNG, defer_load=False)
```

Add a grid for Earth relief

Instead of using colors as fill for the land and water masses a grid can be displayed. Here, the Earth relief is shown by color-coding the elevation.

```
# Download a grid for Earth relief with a resolution of 10 arc-minutes
grd_relief = pygmt.datasets.load_earth_relief(resolution="10m")

# Create a slider
slider_lon = pn.widgets.DiscreteSlider(
    name="Central longitude",
    options=list(np.arange(0, 361, 10)),
    value=0,
```

(continues on next page)

(continued from previous page)

```
)


@pn.depends(central_lon=slider_lon)
def view(central_lon):
    """
    Define a function for plotting the single slices.
    """
    # Create a new instance or object of the pygmt.Figure() class
    fig = pygmt.Figure()
    # Set up a colormap for the elevation in meters
    pygmt.makecpt(
        cmap="SCM/oleron",
        # minimum, maximum, step
        series=[int(grd_relief.data.min()) - 1, int(grd_relief.data.max()) + 1, 100],
    )
    # Plot the grid for the elevation
    fig.grdimage(
        projection=f"G{central_lon}/15/12c",
        region="g",
        grid=grd_relief,  # Use grid downloaded above
        cmap=True,  # Use colormap defined above
        frame=Axis(grid=30),
    )
    # Add a horizontal colorbar for the elevation.
    fig.colorbar(annot=2000, tick=1000, label="Elevation", unit="m")
    return fig


# Make an interactive dashboard
pn.Column(slider_lon, view)
```

```
Column
    [0] DiscreteSlider(formatter='%d', label='Central longitude', name='Central␣
↪longitude', options=[np.int64(0), ...], value=0)
    [1] ParamFunction(function, _pane=PNG, defer_load=False)
```

Total running time of the script: (0 minutes 1.296 seconds)

7.2.9 Making subplots

When you're preparing a figure for a paper, there will often be times when you'll need to put many individual plots into one large figure, and tag them 'abcd'. These individual plots are called subplots.

There are two main ways to create subplots in GMT:

- Use *pygmt.Figure.shift_origin* to manually move each individual plot to the right position.
- Use *pygmt.Figure.subplot* to define the layout of the subplots.

The first method is easier to use and should handle simple cases involving a couple of subplots. For more advanced subplot layouts, however, we recommend the use of *pygmt.Figure.subplot* which offers finer grained control, and this is what the tutorial below will cover.

```
import pygmt
from pygmt.params import Axis, Frame
```

Let's start by initializing a *pygmt.Figure* instance.

```
fig = pygmt.Figure()
```

Define subplot layout

The *pygmt.Figure.subplot* method is used to set up the layout, size, and other attributes of the figure. It divides the whole canvas into regular grid areas with *n* rows and *m* columns. Each grid area can contain an individual subplot. For example:

```
 with fig.subplot(
     nrows=2,
     ncols=3,
     figsize=("15c", "6c"),
     frame=Frame(axes="lrtb"),
):
     ...
```

will define our figure to have a 2 row and 3 column grid layout. `figsize=("15c", "6c")` defines the overall size of the figure to be 15 cm wide by 6 cm high. Using `frame=Frame(axes="lrtb")` allows us to customize the plot frame for all subplots instead of setting them individually. The figure layout will look like the following:

```
with fig.subplot(nrows=2, ncols=3, figsize=("15c", "6c"), frame=Frame(axes="lrtb")):
    for i in range(2):  # row number starting from 0
        for j in range(3):  # column number starting from 0
            index = i * 3 + j  # index number starting from 0
            with fig.set_panel(panel=index):  # sets the current panel
                fig.text(
                    position="MC",
                    text=f"index: {index}; row: {i}, col: {j}",
                    region=[0, 1, 0, 1],
                )
fig.show()
```

The *pygmt.Figure.set_panel* method activates a specified subplot, and all subsequent plotting methods will take place in that subplot panel. This is similar to matplotlib's `plt.sca` method. In order to specify a subplot, you will need

to provide the identifier for that subplot via the `panel` parameter. Pass in either the *index* number, or a tuple/list like (*row*, *col*) to `panel`.

Note

The row and column numbering starts from 0. So for a subplot layout with N rows and M columns, row numbers will go from 0 to N-1, and column numbers will go from 0 to M-1.

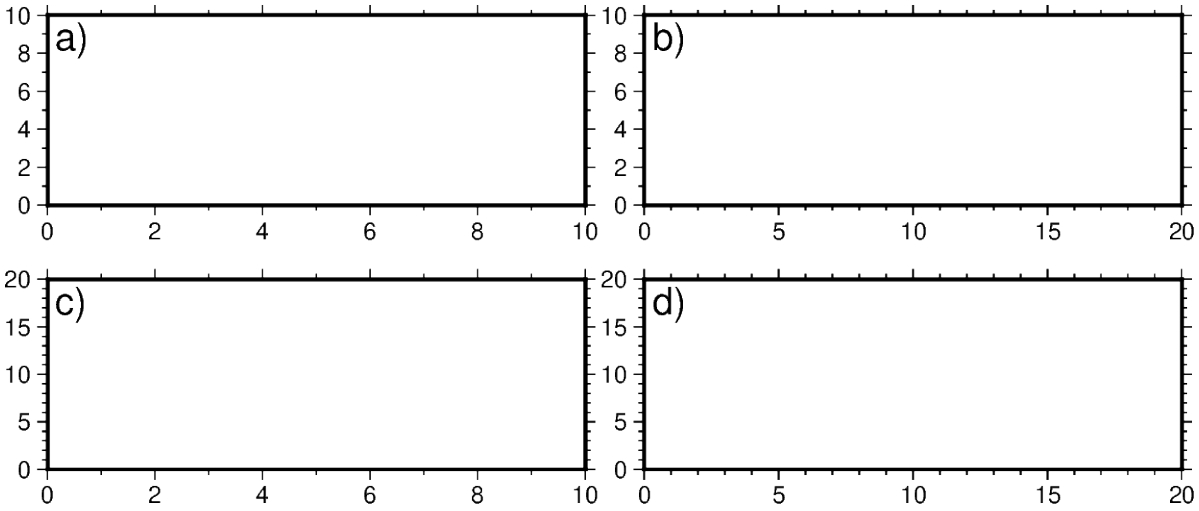
For example, to activate the subplot on the top right corner (index: 2) at *row*=0 and *col*=2, so that all subsequent plotting commands happen there, you can use the following command:

```
with fig.set_panel(panel=(0, 2)):
    ...
```

Making your first subplot

Next, let's use what we learned above to make a 2 row by 2 column subplot figure. We'll also pick up on some new parameters to configure our subplot.

```
fig = pygmt.Figure()
with fig.subplot(
    nrows=2,
    ncols=2,
    figsize=("15c", "6c"),
    tag=True,
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
    margins=["0.1c", "0.2c"],
    title="My Subplot Heading",
):
    fig.basemap(region=[0, 10, 0, 10], projection="X?", panel=[0, 0])
    fig.basemap(region=[0, 20, 0, 10], projection="X?", panel=[0, 1])
    fig.basemap(region=[0, 10, 0, 20], projection="X?", panel=[1, 0])
    fig.basemap(region=[0, 20, 0, 20], projection="X?", panel=[1, 1])
fig.show()
```



In this example, we define a 2-row, 2-column (2x2) subplot layout using *pygmt.Figure.subplot*. The overall figure dimensions is set to be 15 cm wide and 6 cm high (`figsize=("15c", "6c")`). In addition, we use some optional parameters to fine-tune some details of the figure creation:

- `tag=True`: Each subplot is automatically tagged 'abcd'.
- `margins=["0.1c", "0.2c"]`: Adjusts the space between adjacent subplots. In this case, it is set as 0.1 cm in the x-direction and 0.2 cm in the y-direction.
- `title="My Subplot Heading"`: Adds a title on top of the whole figure.

Notice that each subplot was set to use a linear projection `"X?"`. Usually, we need to specify the width and height of the plot frame, but it is also possible to use a question mark `"?"` to let GMT decide automatically on what is the most appropriate width/height for each subplot's frame.

Tip

In the above example, we used the following commands to activate the four subplots explicitly one after another:

```
fig.basemap(..., panel=[0, 0])
fig.basemap(..., panel=[0, 1])
fig.basemap(..., panel=[1, 0])
fig.basemap(..., panel=[1, 1])
```

In fact, we can just use `fig.basemap(..., panel=True)` without specifying any subplot index number, and GMT will automatically activate the next subplot panel.

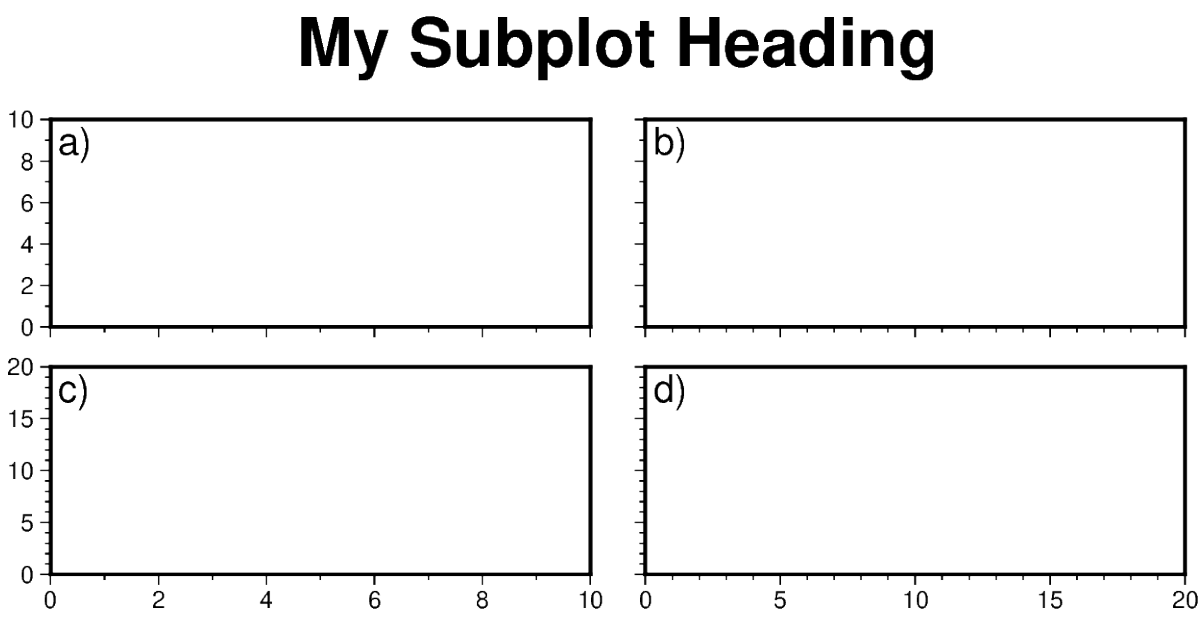
Note

All plotting methods (e.g. *pygmt.Figure.coast*, *pygmt.Figure.text*, etc) are able to use `panel` parameter when in subplot mode. Once a panel is activated using `panel` or *pygmt.Figure.set_panel*, subsequent plotting commands that don't set a `panel` will have their elements added to the same panel as before.

Shared x- and y-axes

In the example above with the four subplots, the two subplots for each row have the same y-axis range, and the two subplots for each column have the same x-axis range. You can use the `sharex`/`sharey` parameters to set a common x- and/or y-axis between subplots.

```
fig = pygmt.Figure()
with fig.subplot(
    nrows=2,
    ncols=2,
    figsize=("15c", "6c"),  # width of 15 cm, height of 6 cm
    tag=True,
    margins=["0.3c", "0.2c"],  # horizontal 0.3 cm and vertical 0.2 cm margins
    title="My Subplot Heading",
    sharex="b",  # shared x-axis on the bottom side
    sharey="l",  # shared y-axis on the left side
    frame=Frame(axes="WSrt"),
):
    fig.basemap(region=[0, 10, 0, 10], projection="X?", panel=True)
    fig.basemap(region=[0, 20, 0, 10], projection="X?", panel=True)
    fig.basemap(region=[0, 10, 0, 20], projection="X?", panel=True)
    fig.basemap(region=[0, 20, 0, 20], projection="X?", panel=True)
fig.show()
```



`sharex="b"` indicates that subplots in a column will share the x-axis, and only the **b**ottom axis is displayed. `sharey="l"` indicates that subplots within a row will share the y-axis, and only the **l**eft axis is displayed.

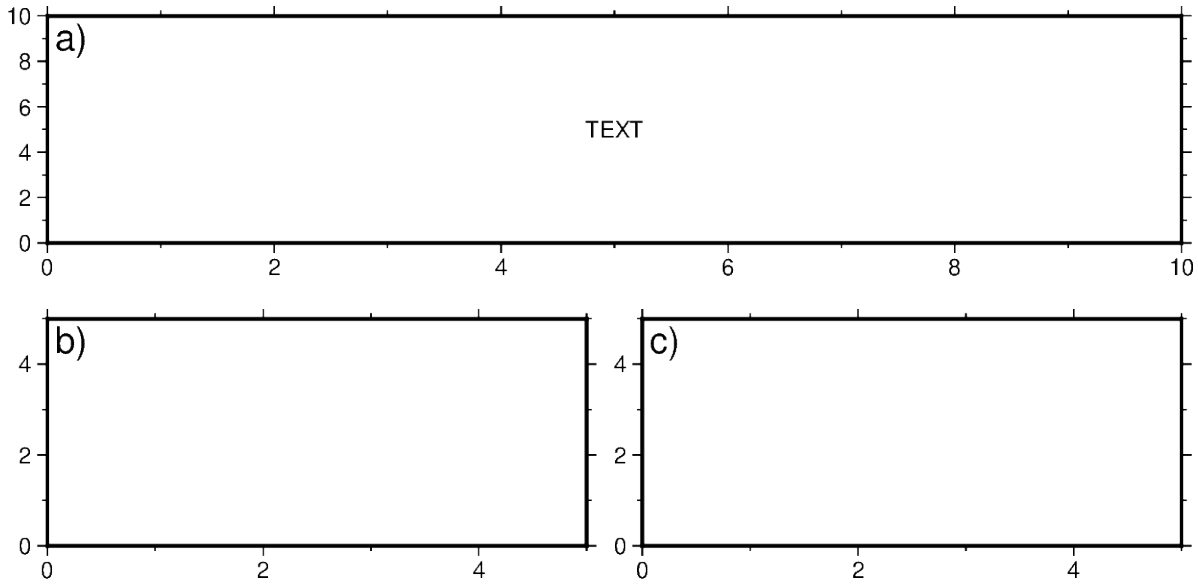
Of course, instead of using the `sharex`/`sharey` parameters, you can also set a different `frame` for each subplot to control the axis properties individually for each subplot.

Advanced subplot layouts

Nested subplots are currently not supported. If you want to create more complex subplot layouts, some manual adjustments are needed.

The following example draws three subplots in a 2-row, 2-column layout, with the first subplot occupying the first row.

```
fig = pygmt.Figure()
# Bottom row, two subplots
with fig.subplot(nrows=1, ncols=2, figsize=("15c", "3c"), tag="b)"):
    fig.basemap(
        region=[0, 5, 0, 5],
        projection="X?",
        frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
        panel=[0, 0],
    )
    fig.basemap(
        region=[0, 5, 0, 5],
        projection="X?",
        frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
        panel=[0, 1],
    )
# Move plot origin by 1 cm above the height of the entire figure
fig.shift_origin(yshift="h+1c")
# Top row, one subplot
with fig.subplot(nrows=1, ncols=1, figsize=("15c", "3c"), tag="a)"):
    fig.basemap(
        region=[0, 10, 0, 10],
        projection="X?",
        frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
        panel=[0, 0],
    )
    fig.text(text="TEXT", x=5, y=5)

fig.show()
```



We start by drawing the bottom two subplots, setting `tag="b)"` so that the subplots are tagged 'b)' and 'c)'. Next, we

use *pygmt.Figure.shift_origin* to move the plot origin 1 cm above the **h**eight of the entire figure that is currently plotted (i.e. the bottom row subplots). A single subplot is then plotted on the top row. You may need to adjust the `yshift` parameter to make your plot look nice. This top row uses `tag="a)"`, and we also plotted some text inside. Note that `projection="X?"` was used to let GMT automatically determine the size of the subplot according to the size of the subplot area.

You can also manually override the `tag` for each subplot using for example, `fig.set_panel(..., tag="b) Panel 2")` which would allow you to manually tag a single subplot as you wish. This can be useful for adding a more descriptive subtitle to individual subplots.

Total running time of the script: (0 minutes 0.529 seconds)

7.2.10 Performing grid histogram equalization

The *pygmt.grdhisteq.equalize_grid* method creates a grid using statistics based on a cumulative distribution function.

```
import pygmt
from pygmt.params import Axis, Frame, Position
```

Load sample data

Load the sample Earth relief data for a region around Yosemite Valley and use *pygmt.grd2xyz* to create a `pandas.Series` with the z-values.

```
grid = pygmt.datasets.load_earth_relief(
    resolution="03s", region=[-119.825, -119.4, 37.6, 37.825]
)
grid_dist = pygmt.grd2xyz(grid=grid, output_type="pandas")["z"]
```

Plot the original digital elevation model and data distribution

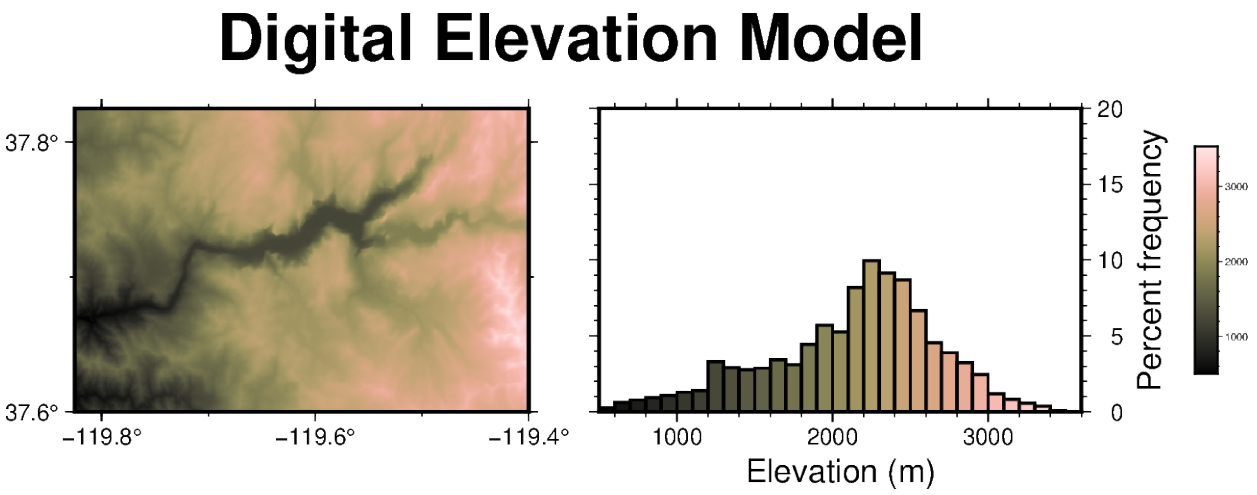
For comparison, we will create a map of the original digital elevation model and a histogram showing the distribution of elevation data values.

```
# Create an instance of the Figure class
fig = pygmt.Figure()
# Define figure configuration
pygmt.config(FORMAT_GEO_MAP="ddd.x", MAP_FRAME_TYPE="plain")
# Define the colormap for the figure
pygmt.makecpt(series=[500, 3540], cmap="SCM/turku")
# Setup subplots with two panels
with fig.subplot(
    nrows=1, ncols=2, figsize=("13.5c", "4c"), title="Digital Elevation Model"
):
    # Plot the original digital elevation model in the first panel
    with fig.set_panel(panel=0):
        fig.grdimage(grid=grid, projection="M?", frame=Frame(axes="WSne"), cmap=True)
    # Plot a histogram showing the z-value distribution in the original digital
    # elevation model
    with fig.set_panel(panel=1):
        fig.histogram(
            data=grid_dist,
            projection="X?",
```

(continues on next page)

(continued from previous page)

```
            region=[500, 3600, 0, 20],
            series=[500, 3600, 100],
            frame=Frame(
                axes="wnSE",
                xaxis=Axis(annot=True, tick=True, label="Elevation (m)"),
                yaxis=Axis(annot=True, tick=True, label="Percent frequency"),
            ),
            cmap=True,
            histtype="percent",
            pen="1p,black",
        )
        fig.colorbar(
            position=Position("MR", cstype="outside", offset=(1.5, 0)),
            length=3,
            width=0.3,
            frame=True,
        )
fig.show()
```



Equalize grid based on a linear distribution

The *pygmt.grdhisteq.equalize_grid* method creates a new grid with the z-values representing the position of the original z-values in a given cumulative distribution. By default, it computes the position in a linear distribution. Here, we equalize the grid into nine divisions based on a linear distribution and produce a `pandas.Series` with the z-values for the new grid.

```
divisions = 9
linear = pygmt.grdhisteq.equalize_grid(grid=grid, divisions=divisions)
linear_dist = pygmt.grd2xyz(grid=linear, output_type="pandas")["z"]
```

Calculate the bins used for data transformation

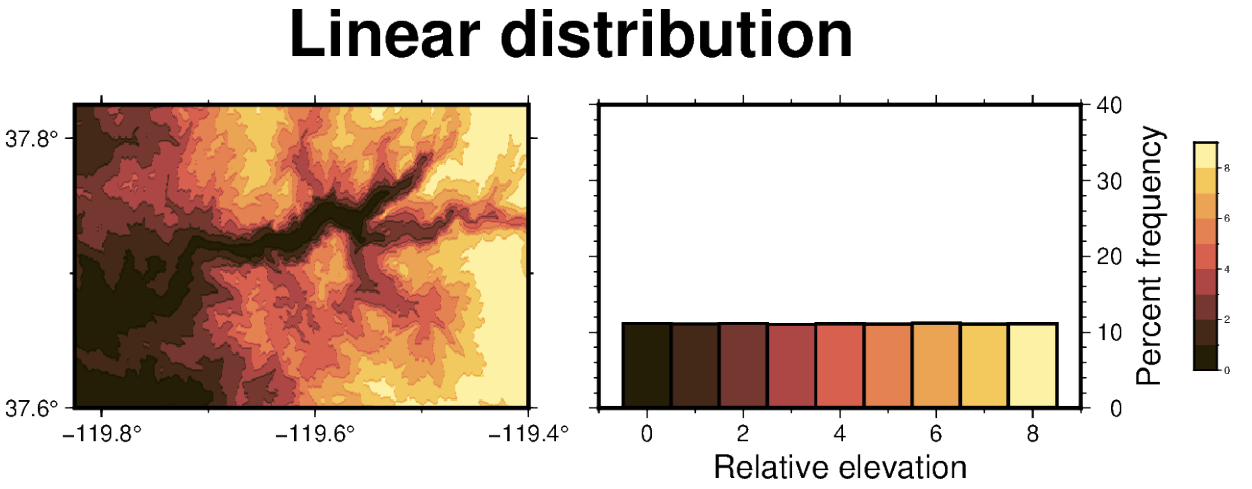
The *pygmt.grdhisteq.compute_bins* method reports statistics about the grid equalization. Here, we report the bins that would linearly divide the original data into 9 divisions with equal area. In our new grid produced by *pygmt.grdhisteq.equalize_grid*, all the grid cells with values between `start` and `stop` of `bin_id=0` are assigned the value 0, all grid cells with values between `start` and `stop` of `bin_id=1` are assigned the value 1, and so on.

```
pygmt.grdhisteq.compute_bins(grid=grid, divisions=divisions)
```

Plot the equally distributed data

Here we create a map showing the grid that has been transformed to have a linear distribution with nine divisions and a histogram of the data values.

```
# Create an instance of the Figure class
fig = pygmt.Figure()
# Define figure configuration
pygmt.config(FORMAT_GEO_MAP="ddd.x", MAP_FRAME_TYPE="plain")
# Define the colormap for the figure
pygmt.makecpt(series=[0, divisions, 1], cmap="SCM/lajolla")
# Setup subplots with two panels
with fig.subplot(
    nrows=1, ncols=2, figsize=("13.5c", "4c"), title="Linear distribution"
):
    # Plot the grid with a linear distribution in the first panel
    with fig.set_panel(panel=0):
        fig.grdimage(grid=linear, projection="M?", frame=Frame(axes="WSne"),␣
↪cmap=True)
    # Plot a histogram showing the linear z-value distribution
    with fig.set_panel(panel=1):
        fig.histogram(
            data=linear_dist,
            projection="X?",
            region=[-1, divisions, 0, 40],
            series=[0, divisions, 1],
            frame=Frame(
                axes="wnSE",
                xaxis=Axis(annot=True, tick=True, label="Relative elevation"),
                yaxis=Axis(annot=True, tick=True, label="Percent frequency"),
            ),
            cmap=True,
            histtype="percent",
            pen="1p,black",
            center=True,
        )
        fig.colorbar(
            position=Position("MR", cstype="outside", offset=(1.5, 0)),
            length=3,
            width=0.3,
            frame=True,
        )
fig.show()
```



Transform grid based on a normal distribution

The `gaussian` parameter of *pygmt.grdhisteq.equalize_grid* can be used to transform the z-values relative to their position in a normal distribution rather than a linear distribution. In this case, the output data are continuous rather than discrete.

```
normal = pygmt.grdhisteq.equalize_grid(grid=grid, gaussian=True)
normal_dist = pygmt.grd2xyz(grid=normal, output_type="pandas")["z"]
```

Plot the normally distributed data

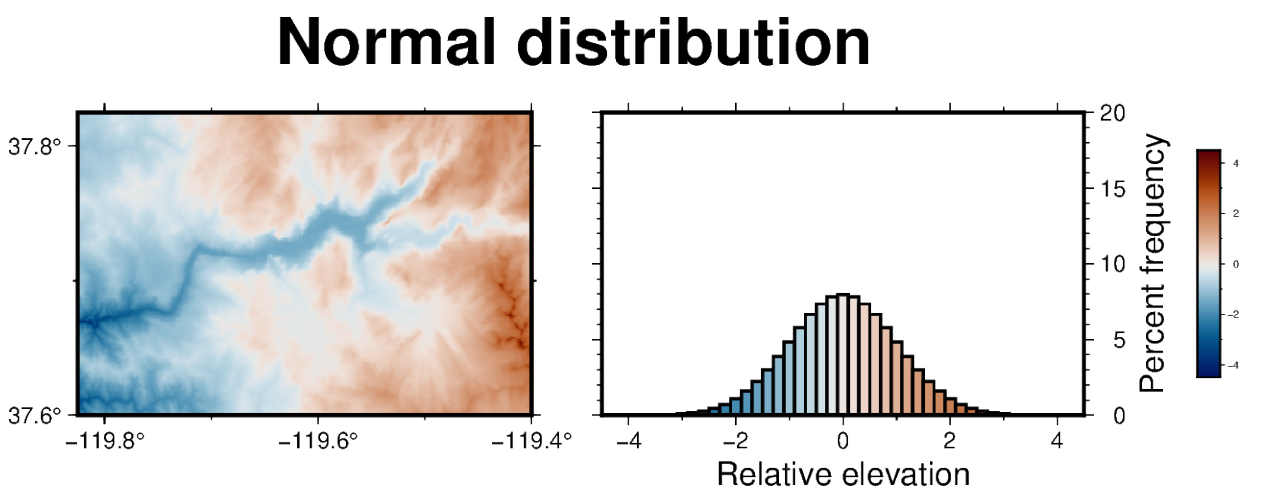
Here we create a map showing the grid that has been transformed to have a normal distribution and a histogram of the data values.

```
# Create an instance of the Figure class
fig = pygmt.Figure()
# Define figure configuration
pygmt.config(FORMAT_GEO_MAP="ddd.x", MAP_FRAME_TYPE="plain")
# Define the colormap for the figure
pygmt.makecpt(series=[-4.5, 4.5], cmap="SCM/vik")
# Setup subplots with two panels
with fig.subplot(
    nrows=1, ncols=2, figsize=("13.5c", "4c"), title="Normal distribution"
):
    # Plot the grid with a normal distribution in the first panel
    with fig.set_panel(panel=0):
        fig.grdimage(grid=normal, projection="M?", frame=Frame(axes="WSne"),␣
↪cmap=True)
    # Plot a histogram showing the normal z-value distribution
    with fig.set_panel(panel=1):
        fig.histogram(
            data=normal_dist,
            projection="X?",
            region=[-4.5, 4.5, 0, 20],
            series=[-4.5, 4.5, 0.2],
            frame=Frame(
                axes="wnSE",
                xaxis=Axis(annot=True, tick=True, label="Relative elevation"),
```

(continues on next page)

(continued from previous page)

```
                yaxis=Axis(annot=True, tick=True, label="Percent frequency"),
            ),
            cmap=True,
            histtype="percent",
            pen="1p,black",
        )
        fig.colorbar(
            position=Position("MR", cstype="outside", offset=(1.5, 0)),
            length=3,
            width=0.3,
            frame=True,
        )
fig.show()
```



Equalize grid based on a quadratic distribution

The `quadratic` parameter of *`pygmt.grdhisteq.equalize_grid`* can be used to transform the z-values relative to their position in a quadratic distribution rather than a linear distribution. Here, we equalize the grid into nine divisions based on a quadratic distribution and produce a `pandas.Series` with the z-values for the new grid.

```
quadratic = pygmt.grdhisteq.equalize_grid(
    grid=grid, quadratic=True, divisions=divisions
)
quadratic_dist = pygmt.grd2xyz(grid=quadratic, output_type="pandas")["z"]
```

Calculate the bins used for data transformation

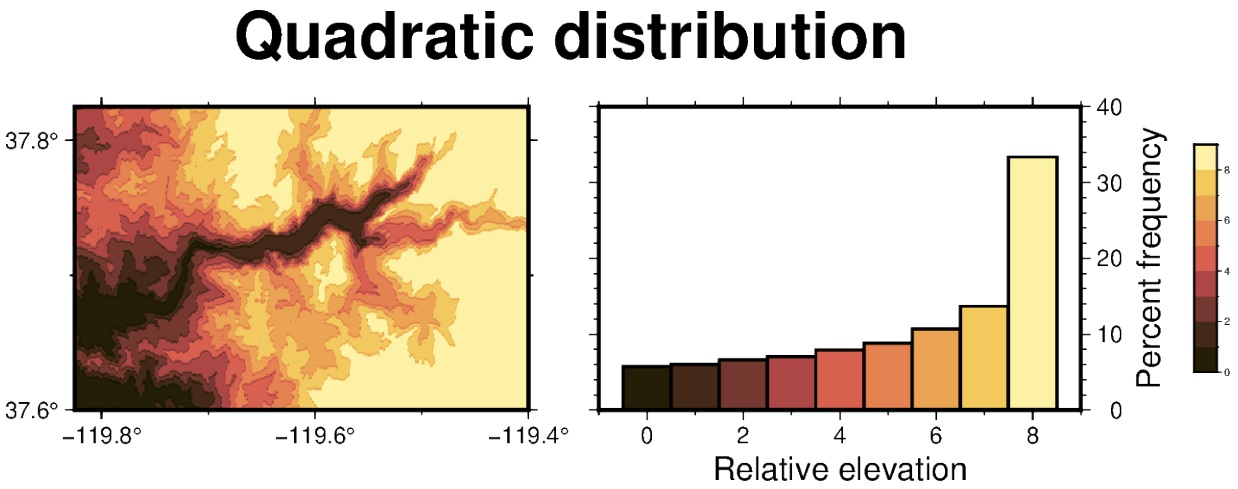
We can also use the `quadratic` parameter of *`pygmt.grdhisteq.compute_bins`* to report the bins used for dividing the grid into 9 divisions based on their position in a quadratic distribution.

```
pygmt.grdhisteq.compute_bins(grid=grid, divisions=divisions, quadratic=True)
```

Plot the quadratic distribution of data

Here we create a map showing the grid that has been transformed to have a quadratic distribution and a histogram of the data values.

```
# Create an instance of the Figure class
fig = pygmt.Figure()
# Define figure configuration
pygmt.config(FORMAT_GEO_MAP="ddd.x", MAP_FRAME_TYPE="plain")
# Define the colormap for the figure
pygmt.makecpt(series=[0, divisions, 1], cmap="SCM/lajolla")
# Setup subplots with two panels
with fig.subplot(
    nrows=1, ncols=2, figsize=("13.5c", "4c"), title="Quadratic distribution"
):
    # Plot the grid with a quadratic distribution in the first panel
    with fig.set_panel(panel=0):
        fig.grdimage(
            grid=quadratic, projection="M?", frame=Frame(axes="WSne"), cmap=True
        )
    # Plot a histogram showing the quadratic z-value distribution
    with fig.set_panel(panel=1):
        fig.histogram(
            data=quadratic_dist,
            projection="X?",
            region=[-1, divisions, 0, 40],
            series=[0, divisions, 1],
            frame=Frame(
                axes="wnSE",
                xaxis=Axis(annot=True, tick=True, label="Relative elevation"),
                yaxis=Axis(annot=True, tick=True, label="Percent frequency"),
            ),
            cmap=True,
            histtype="percent",
            pen="1p,black",
            center=True,
        )
        fig.colorbar(
            position=Position("MR", cstype="outside", offset=(1.5, 0)),
            length=3,
            width=0.3,
            frame=True,
        )
fig.show()
```



Total running time of the script: (0 minutes 1.146 seconds)

7.2.11 Plotting Earth relief

PyGMT provides the *pygmt.datasets.load_earth_relief* function to download the Earth relief data from the GMT remote server and load as an `xarray.DataArray` object. The data can then be plotted using the *pygmt.Figure.grdimage* method.

```
import pygmt
from pygmt.params import Axis
```

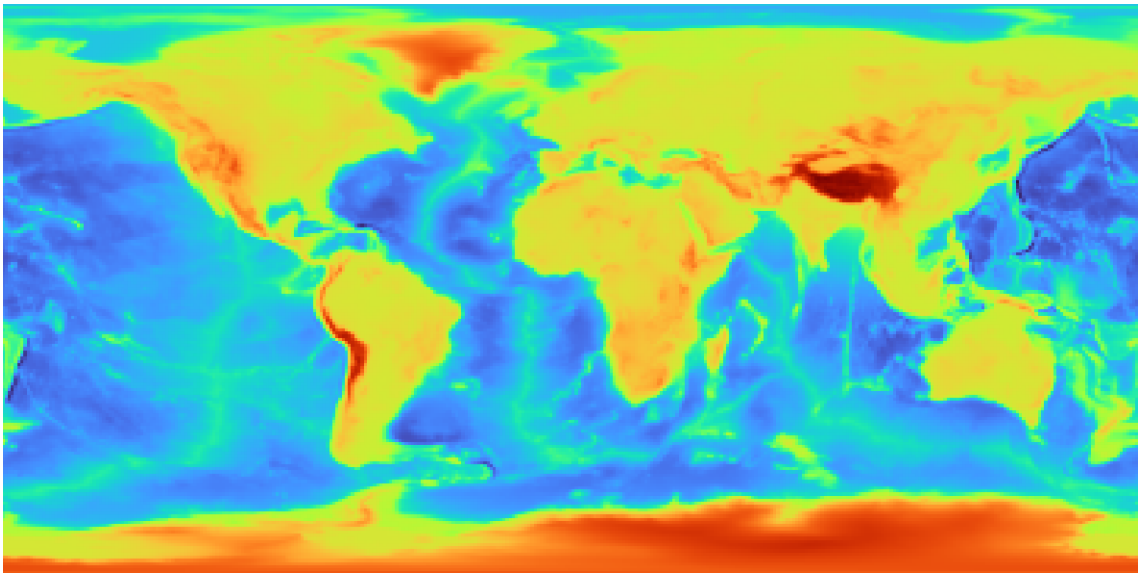
Load sample Earth relief data for the entire globe at a resolution of 1 arc-degree. Refer to *pygmt.datasets.load_earth_relief* for the other available resolutions.

```
grid = pygmt.datasets.load_earth_relief(resolution="01d")
```

Create a plot

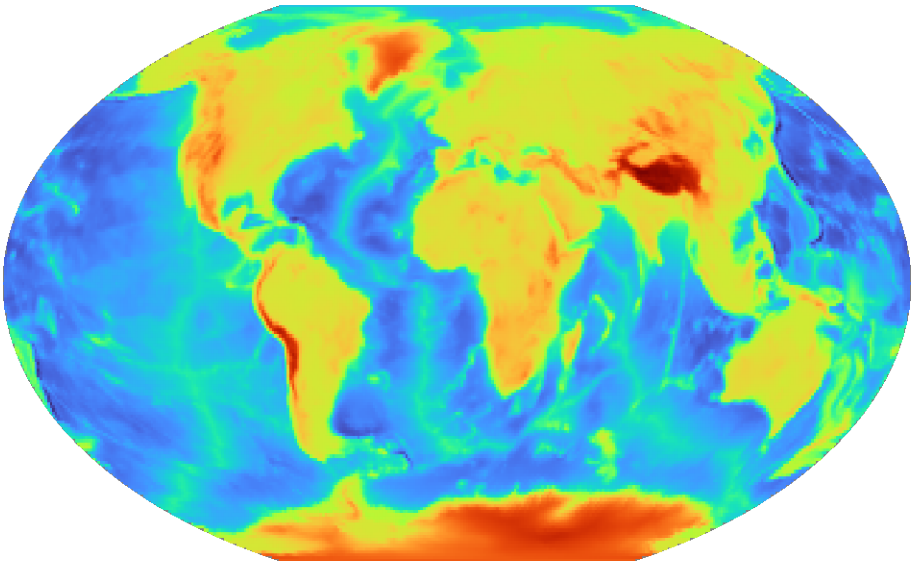
The *pygmt.Figure.grdimage* method takes the `grid` input to create a figure. It creates and applies a color palette to the figure based upon the z-values of the data. By default, it plots the map with the *turbo* CPT, an equidistant cylindrical projection, and with no frame.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid)
fig.show()
```



pygmt.Figure.grdimage can take the optional parameter `projection` for the map. In the example below, `projection` is set to `"R12c"` for a 12-centimeters-wide figure with a Winkel Tripel projection. For a list of available projections, see *GMT Map Projections*.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid, projection="R12c")
fig.show()
```



Set a colormap

pygmt.Figure.grdimage takes the `cmap` parameter to set the CPT of the figure. Examples of common CPTs for Earth relief are shown below. A full list of CPTs can be found at https://docs.generic-mapping-tools.org/6.7/reference/cpts.html.

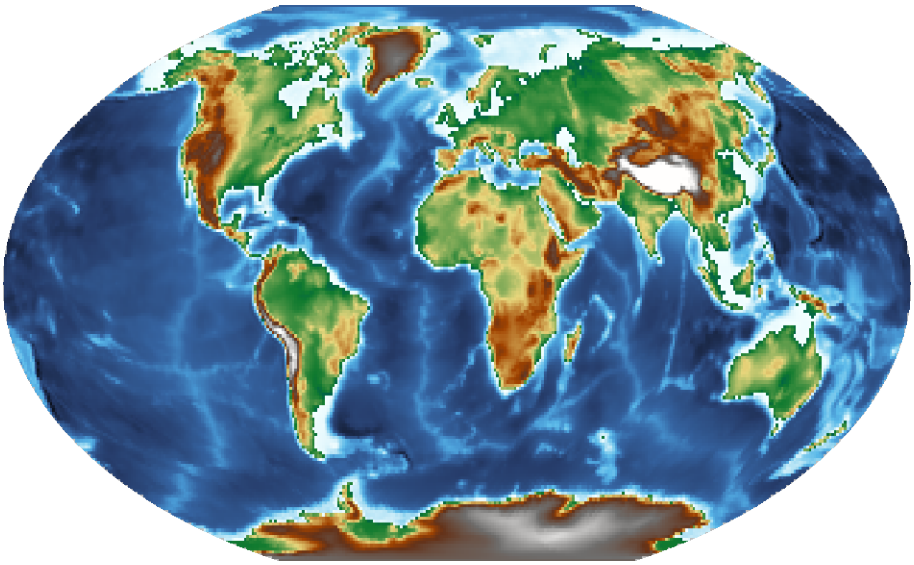
Using the *geo* CPT:

```
fig = pygmt.Figure()
```

(continues on next page)

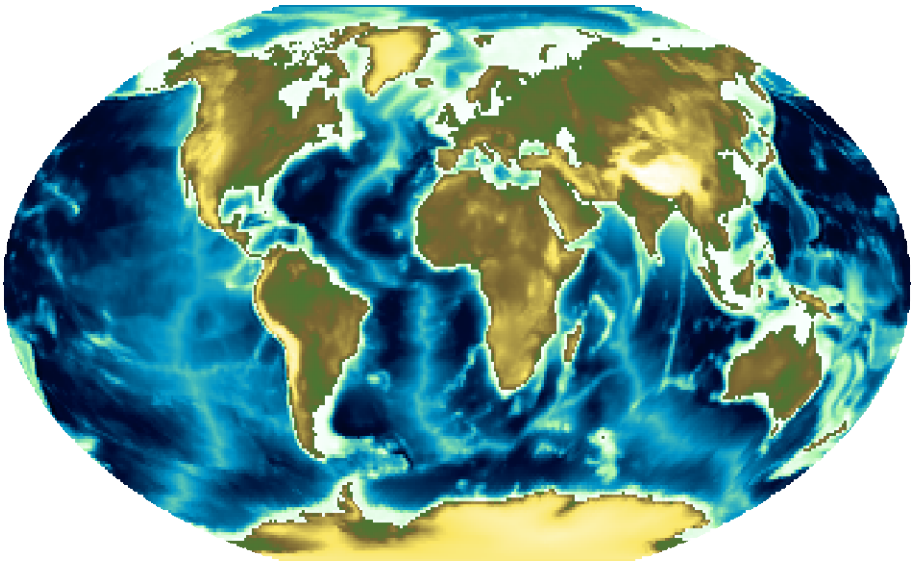
(continued from previous page)

```
fig.grdimage(grid=grid, projection="R12c", cmap="gmt/geo")
fig.show()
```



Using the *relief* CPT:

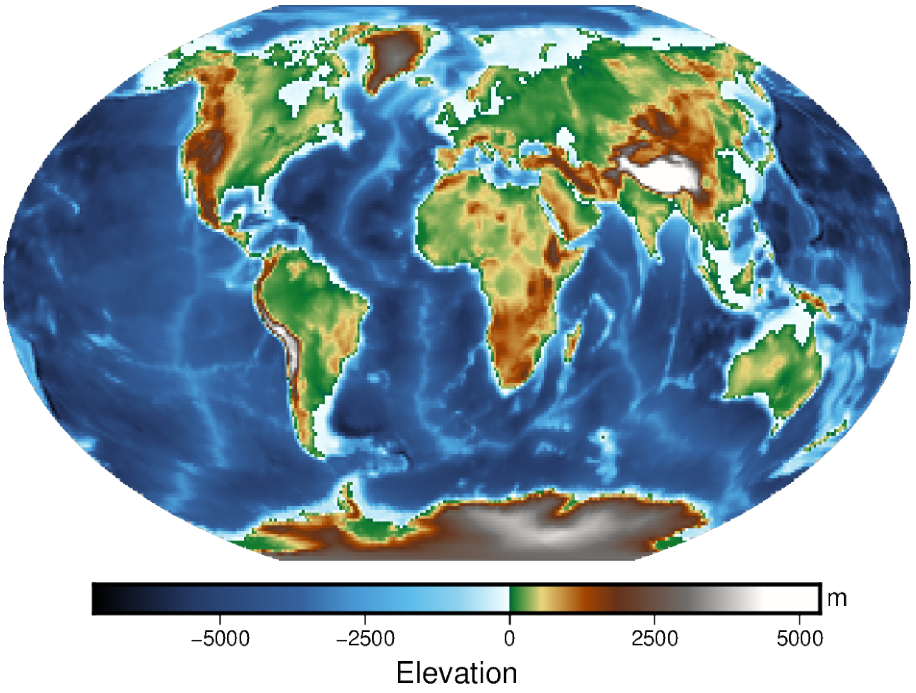
```
fig = pygmt.Figure()
fig.grdimage(grid=grid, projection="R12c", cmap="gmt/relief")
fig.show()
```



Add a colorbar

The *pygmt.Figure.colorbar* method displays the CPT and the associated z-values of the figure, and by default uses the same CPT set by the `cmap` parameter for *pygmt.Figure.grdimage*. The `annot` parameter sets the annotation interval, the `label` parameter sets the x-axis label, and the `unit` parameter sets the y-axis label.

```
fig = pygmt.Figure()
fig.grdimage(grid=grid, projection="R12c", cmap="gmt/geo")
fig.colorbar(annot=2500, label="Elevation", unit="m")
fig.show()
```

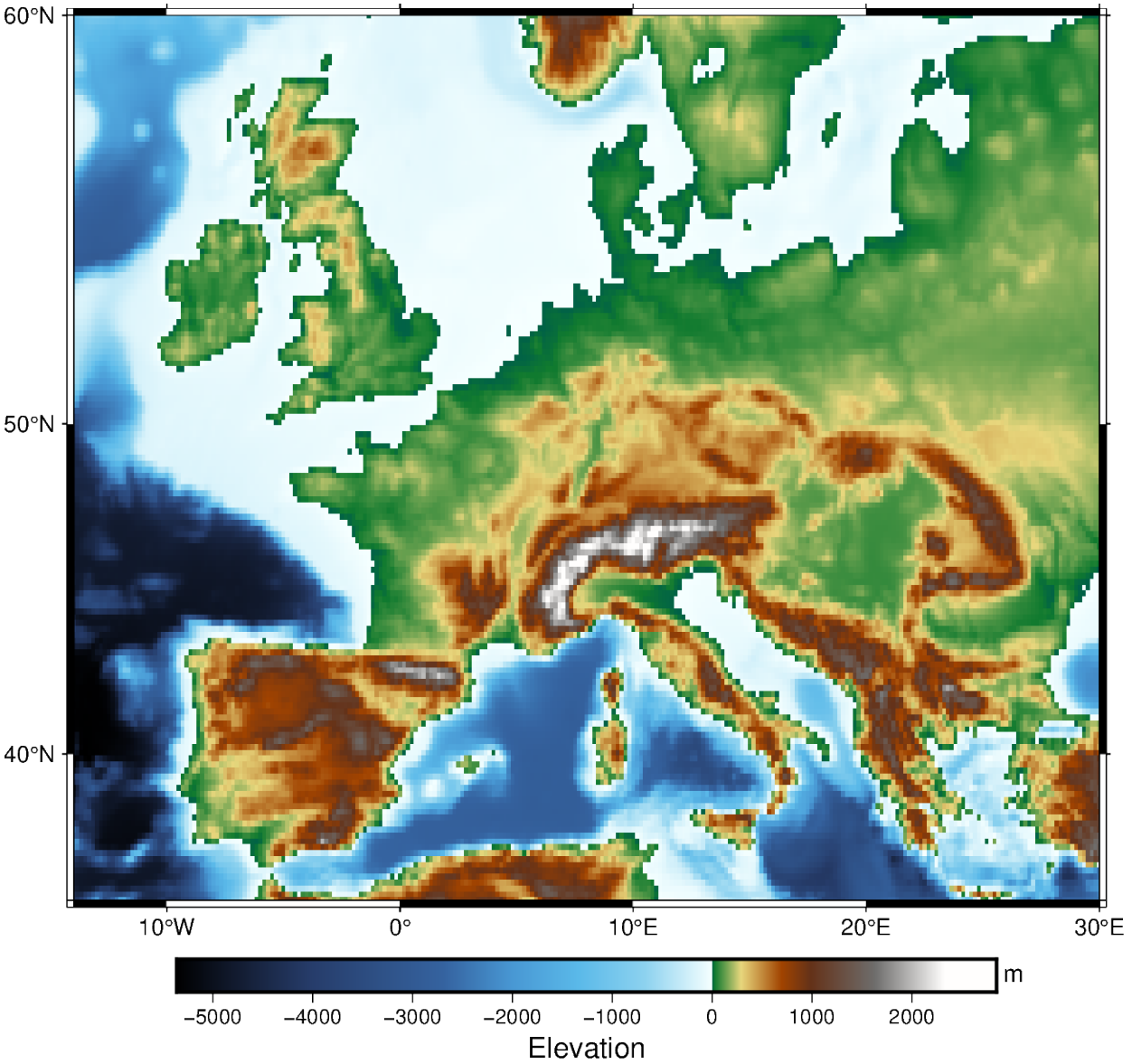


Create a region map

In addition to providing global data, the `region` parameter of *`pygmt.datasets.load_earth_relief`* can be used to provide data for a specific area. The `region` parameter is required for resolutions at 5 arc-minutes or higher, and accepts a list in the form of [*xmin*, *xmax*, *ymin*, *ymax*].

The example below uses data with a 10 arc-minutes resolution, and plots it on a 15-centimeters-wide figure with a Mercator projection and a CPT set to *geo*. `frame=Axis(annot=True)` is used to add a frame with annotations to the map.

```
grid = pygmt.datasets.load_earth_relief(resolution="10m", region=[-14, 30, 35, 60])
fig = pygmt.Figure()
fig.grdimage(grid=grid, projection="M15c", frame=Axis(annot=True), cmap="gmt/geo")
fig.colorbar(annot=1000, label="Elevation", unit="m")
fig.show()
```



Total running time of the script: (0 minutes 1.024 seconds)

7.2.12 Plotting datetime charts

PyGMT accepts a variety of datetime objects to plot data and create charts. Aside from the built-in Python `datetime` module, PyGMT supports inputs containing ISO formatted strings as well as objects generated with `numpy`, `pandas`, and `xarray`. These data types can be used to plot specific points as well as get passed into the `region` parameter to create a range of the data on an axis.

The following examples will demonstrate how to create plots using these different datetime objects.

```
import datetime as dt

import numpy as np
import pandas as pd
import pygmt
import xarray as xr
```

(continues on next page)

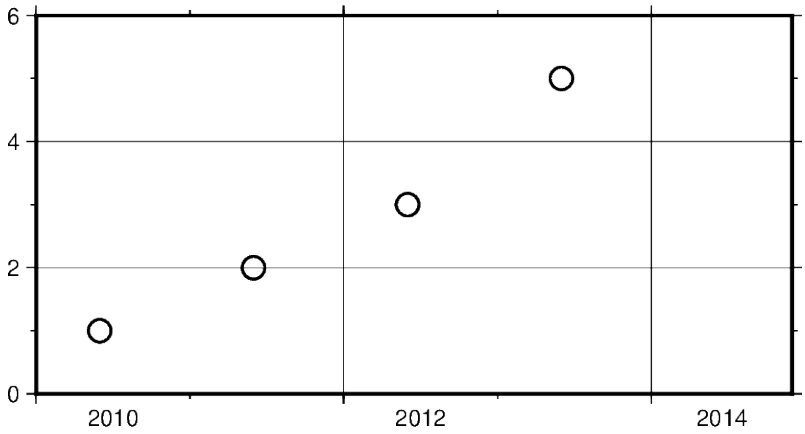
(continued from previous page)

```
from pygmt.params import Axis, Frame
```

Using Python's `datetime`

In this example, Python's built-in `datetime` module is used to create data points stored in the list `x`. Additionally, dates are passed into the `region` parameter in the format `[x_start, x_end, y_start, y_end]`, where the date range is plotted on the x-axis. An additional notable parameter is `style`, where it's specified that data points are plotted as circles with a diameter of 0.3 centimeters.

```
x = [
    dt.date(2010, 6, 1),
    dt.date(2011, 6, 1),
    dt.date(2012, 6, 1),
    dt.date(2013, 6, 1),
]
y = [1, 2, 3, 5]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/5c",
    region=[dt.date(2010, 1, 1), dt.date(2014, 12, 1), 0, 6],
    frame=Frame(axes="WSen", axis=Axis(annot=True, tick=True, grid=True)),
    x=x,
    y=y,
    style="c0.3c",
    pen="1p",
)
fig.show()
```



In addition to specifying the date, `datetime` supports the time at which the data points were recorded. Using `datetime`. `datetime` the `region` parameter as well as data points can be created with both date and time information.

Some notable differences to the previous example include:

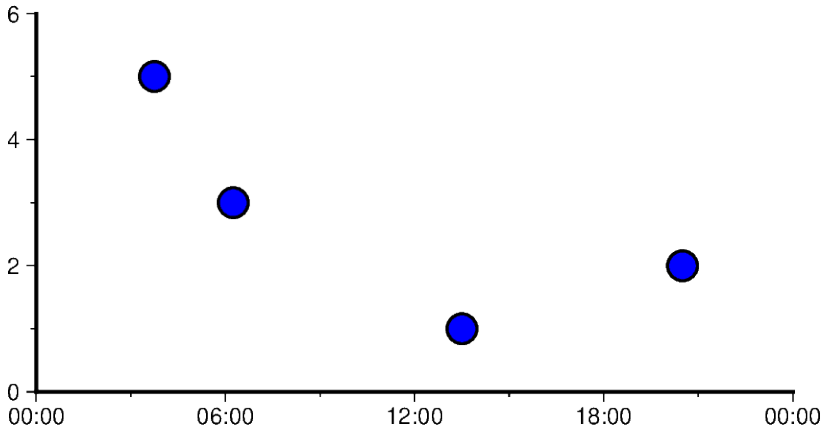
- Modifying `frame` to only include West (left) and South (bottom) borders, and removing grid lines
- Using circles to plot data points defined by `c` in the argument passed through the `style` parameter

```
x = [
    dt.datetime(2021, 1, 1, 3, 45, 1),
```

(continues on next page)

(continued from previous page)

```
    dt.datetime(2021, 1, 1, 6, 15, 1),
    dt.datetime(2021, 1, 1, 13, 30, 1),
    dt.datetime(2021, 1, 1, 20, 30, 1),
]
y = [5, 3, 1, 2]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/5c",
    region=[
        dt.datetime(2021, 1, 1, 0, 0, 0),
        dt.datetime(2021, 1, 2, 0, 0, 0),
        0,
        6,
    ],
    frame=Frame(axes="WS", axis=Axis(annot=True, tick=True)),
    x=x,
    y=y,
    style="c0.4c",
    pen="1p",
    fill="blue",
)
fig.show()
```



Using ISO Format

In addition to Python's `datetime` module, PyGMT also supports passing dates in ISO format. Basic ISO strings are formatted as `YYYY-MM-DD` with each `-` delineated section marking the four-digit year value, two-digit month value, and two-digit day value, respectively.

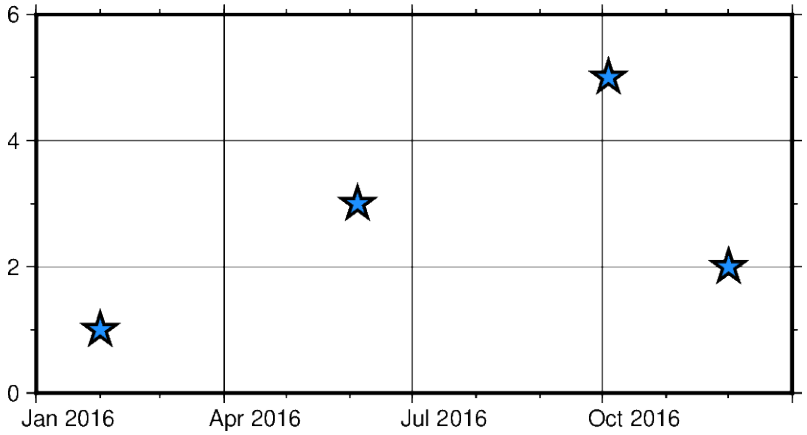
For including the time into an ISO string, the `T` character is used, as it can be seen in the following example. This character is immediately followed by a string formatted as `hh:mm:ss` where each `:` delineated section marking the two-digit hour value, two-digit minute value, and two-digit second value, respectively. The figure in the following example is plotted over a horizontal range of one year from 2016-01-01 to 2017-01-01.

```
x = ["2016-02-01", "2016-06-04T14", "2016-10-04T00:00:15", "2016-12-01T05:00:15"]
y = [1, 3, 5, 2]

fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

```
fig.plot(
    projection="X10c/5c",
    region=["2016-01-01", "2017-01-01", 0, 6],
    frame=Frame(axes="WSen", axis=Axis(annot=True, tick=True, grid=True)),
    x=x,
    y=y,
    style="a0.45c",
    pen="1p",
    fill="dodgerblue",
)
fig.show()
```



Note

PyGMT doesn't recognize non-ISO datetime strings like "Jun 05, 2018". If your data contain non-ISO datetime strings, you can convert them to a recognized format using `pandas.to_datetime` and then pass it to PyGMT.

Mixing and matching Python `datetime` and ISO dates

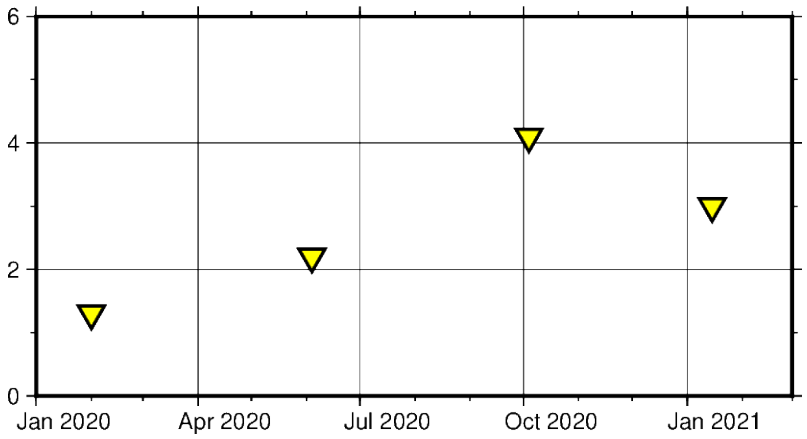
The following example provides context on how both `datetime` and ISO date data can be plotted using PyGMT. This can be helpful when dates and times are coming from different sources, meaning conversions do not need to take place between ISO and datetime in order to create valid plots.

```
x = ["2020-02-01", "2020-06-04", "2020-10-04", dt.datetime(2021, 1, 15)]
y = [1.3, 2.2, 4.1, 3]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/5c",
    region=[dt.datetime(2020, 1, 1), dt.datetime(2021, 3, 1), 0, 6],
    frame=Frame(axes="WSen", axis=Axis(annot=True, tick=True, grid=True)),
    x=x,
    y=y,
    style="i0.4c",
    pen="1p",
    fill="yellow",
)
```

(continues on next page)

(continued from previous page)

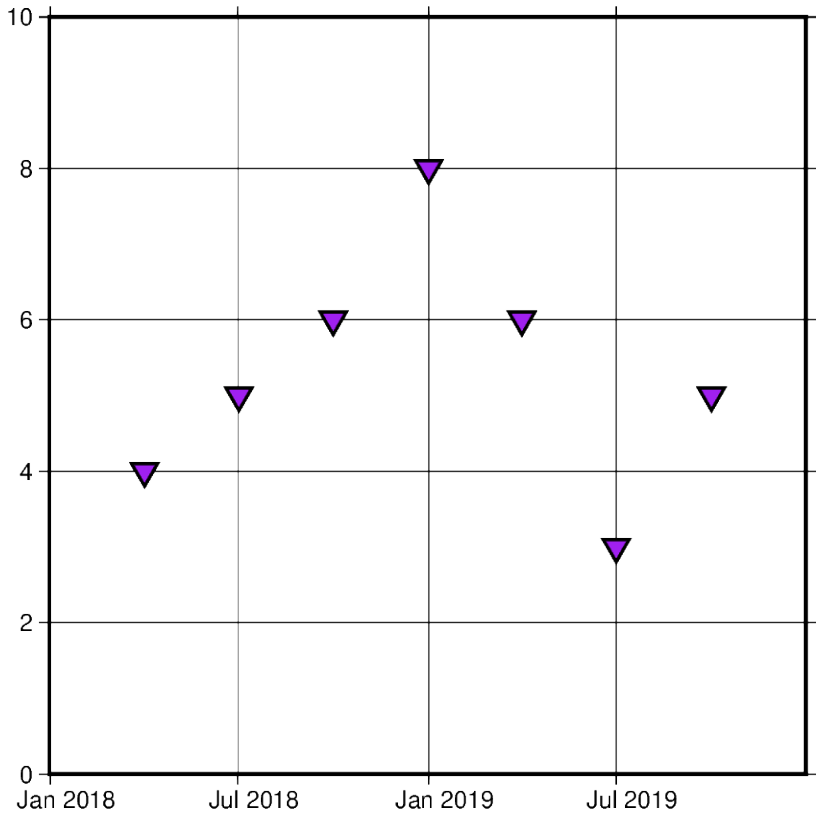
```
fig.show()
```



Using `pandas.date_range`

In the following example, `pandas.date_range` produces a list of `pandas.DatetimeIndex` objects, which is used to pass date data to the PyGMT figure. Specifically `x` contains 7 different `pandas.DatetimeIndex` objects, with the number being manipulated by the `periods` parameter. Each period begins at the start of a business quarter as denoted by BQS when passed to the `freq` parameter. The initial date is the first argument that is passed to `pandas.date_range` and it marks the first data point in the list `x` that will be plotted.

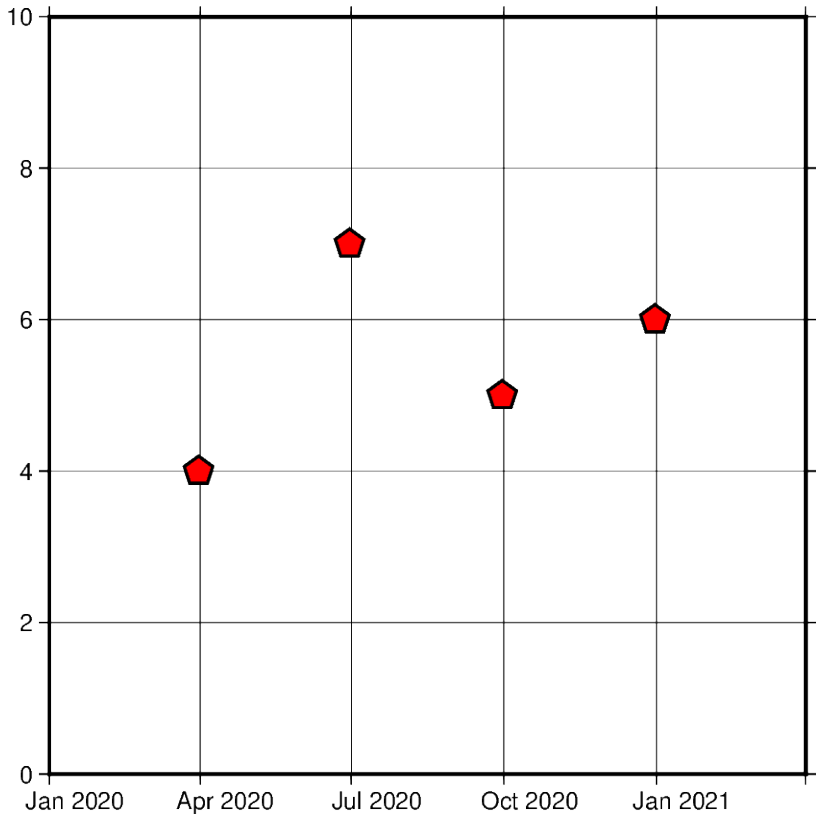
```
x = pd.date_range("2018-03-01", periods=7, freq="BQS")
y = [4, 5, 6, 8, 6, 3, 5]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/10c",
    region=[dt.datetime(2017, 12, 31), dt.datetime(2019, 12, 31), 0, 10],
    frame=Frame(axes="WSen", axis=Axis(annot=True, grid=True)),
    x=x,
    y=y,
    style="i0.4c",
    pen="1p",
    fill="purple",
)
fig.show()
```



Using `xarray.DataArray`

In this example, instead of using a list of `pandas.DatetimeIndex` objects, `x` is initialized as an `xarray.DataArray` object. This object provides a wrapper around numpy ndarrays. It also allows the data to have labeled dimensions while supporting operations that use various pieces of metadata. The following code uses `pandas.date_range` to fill the DataArray with data, but this is not essential for the creation of a valid DataArray.

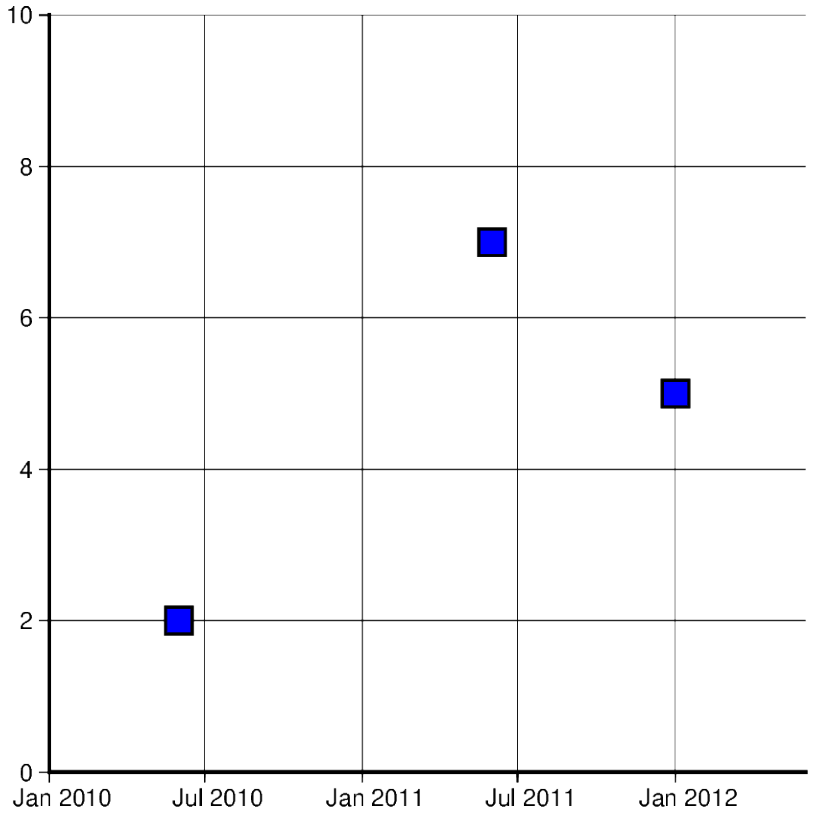
```
x = xr.DataArray(data=pd.date_range(start="2020-01-01", periods=4, freq="QE"))
y = [4, 7, 5, 6]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/10c",
    region=[dt.datetime(2020, 1, 1), dt.datetime(2021, 4, 1), 0, 10],
    frame=Frame(axes="WSen", axis=Axis(annot=True, grid=True)),
    x=x,
    y=y,
    style="n0.4c",
    pen="1p",
    fill="red",
)
fig.show()
```



Using `numpy.datetime64`

In this example, instead of using `pd.date_range`, `x` is initialized as an `np.array` object. Similar to `xarray.DataArray` this wraps the dataset before passing it as an argument. However, `np.array` objects use less memory and allow developers to specify data types.

```
x = np.array(
    ["2010-06-01", "2011-06-01T12", "2012-01-01T12:34:56"], dtype=np.datetime64
)
y = [2, 7, 5]

fig = pygmt.Figure()
fig.plot(
    projection="X10c/10c",
    region=[dt.datetime(2010, 1, 1), dt.datetime(2012, 6, 1), 0, 10],
    frame=Frame(axes="WS", axis=Axis(annot=True, grid=True)),
    x=x,
    y=y,
    style="s0.5c",
    pen="1p",
    fill="blue",
)
fig.show()
```



Generating an automatic region

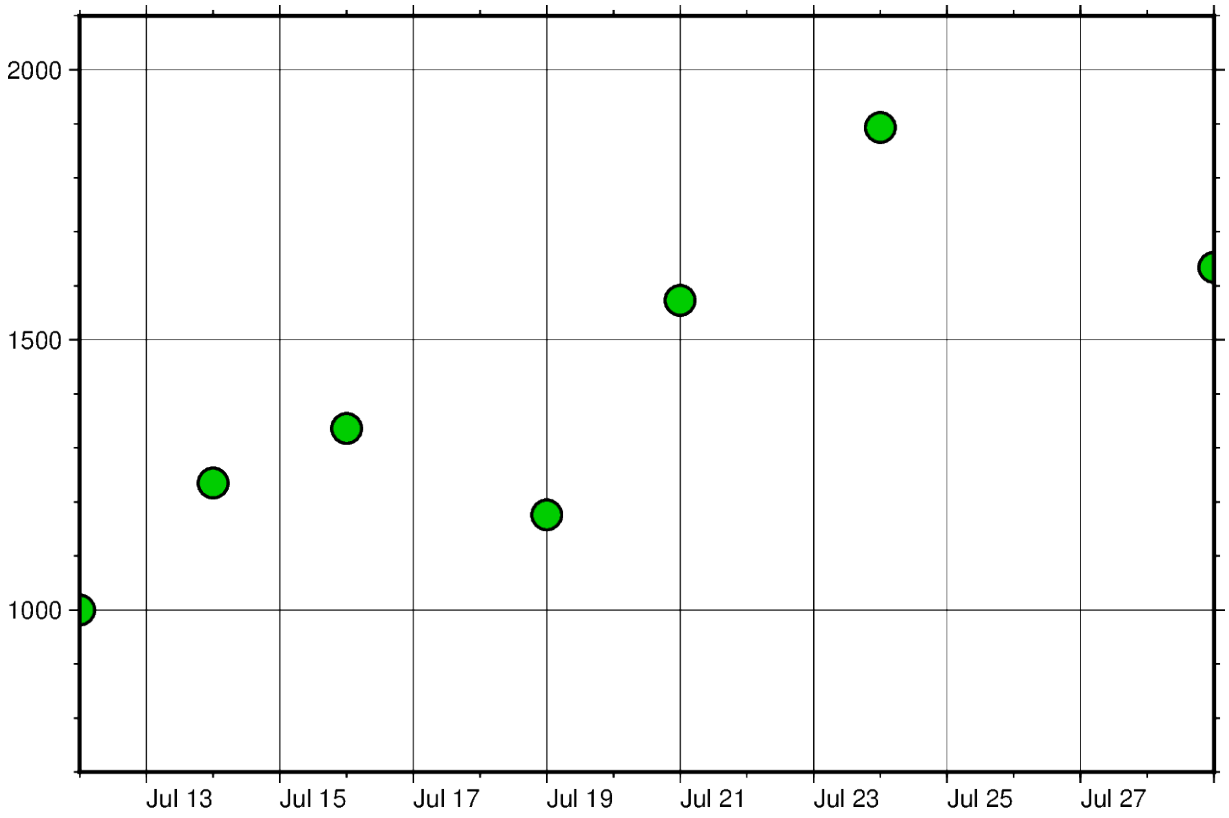
Another way of creating charts involving datetime data can be done by automatically generating the region of the plot. This can be done by passing the DataFrame to *pygmt.info*, which will find the maximum and minimum values for each column and create a list that could be passed as region. Additionally, the `spacing` parameter can be used to increase the range past the maximum and minimum data points.

```
data = [
    ["20200712", 1000],
    ["20200714", 1235],
    ["20200716", 1336],
    ["20200719", 1176],
    ["20200721", 1573],
    ["20200724", 1893],
    ["20200729", 1634],
]
df = pd.DataFrame(data, columns=["Date", "Score"])
df.Date = pd.to_datetime(df["Date"], format="%Y%m%d")
region = pygmt.info(
    data=df[["Date", "Score"]], per_column=True, spacing=(700, 700), coltypes="T"
)

fig = pygmt.Figure()
fig.plot(
    region=region,
    projection="X15c/10c",
    frame=Frame(axes="WSen", axis=Axis(annot=True, tick=True, grid=True)),
```

(continues on next page)

(continued from previous page)

```
    x=df.Date,
    y=df.Score,
    style="c0.4c",
    pen="1p",
    fill="green3",
)
fig.show()
```



Setting Primary and Secondary Time Axes

This example focuses on annotating the axes and setting the interval in which the annotations should appear. All of these modifications are passed to the `frame` parameter via a *pygmt.params.Frame* object.

Setting `axes="WS"` in *pygmt.params.Frame* means that only the Western/Left (**W**) and Southern/Bottom (**S**) borders of the plot are annotated. For more information on this, please refer to the *Frames, ticks, titles, and labels tutorial*.

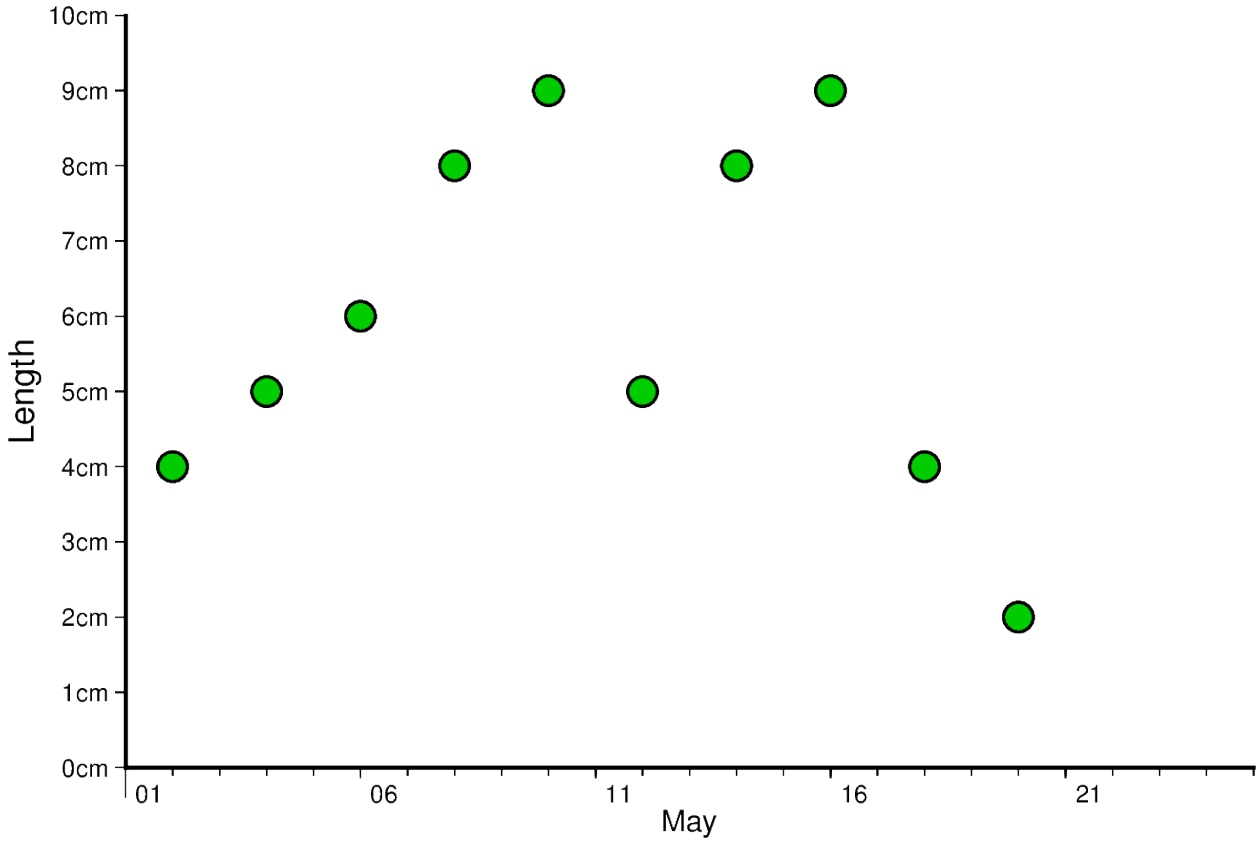
Setting `xaxis2=Axis(annot="1O", tick="1D")` modifies the secondary x-axis annotation interval to one month (**1O**) (capital letter O, not zero), and sets the minor tick spacing interval to 1 day (**1D**). To use the month name instead of its number set FORMAT_DATE_MAP to **o**. More information on configuring date formats can be found at FORMAT_DATE_MAP, FORMAT_DATE_IN, and FORMAT_DATE_OUT.

```
x = pd.date_range("2013-05-02", periods=10, freq="2D")
y = [4, 5, 6, 8, 9, 5, 8, 9, 4, 2]

fig = pygmt.Figure()
with pygmt.config(FORMAT_DATE_MAP="o"):
    fig.plot(
```

(continues on next page)

(continued from previous page)

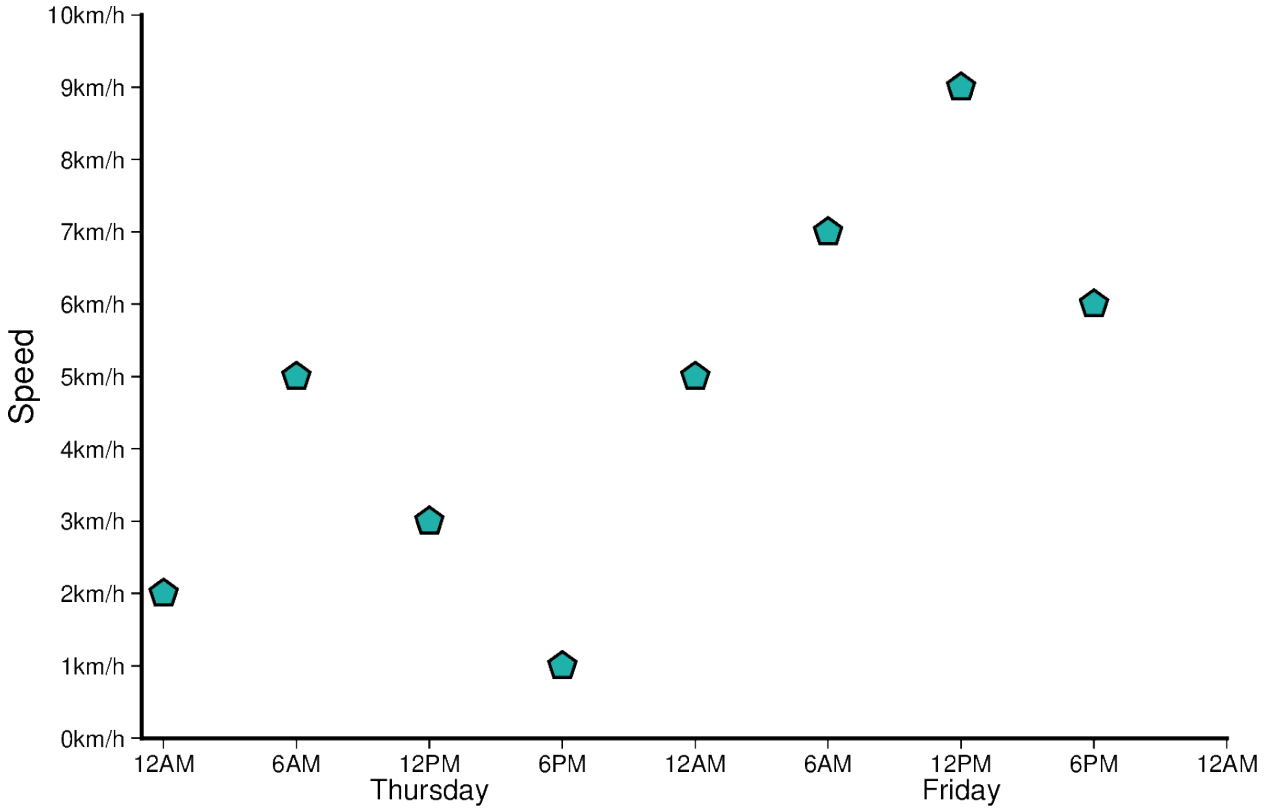
```
        projection="X15c/10c",
        region=[dt.datetime(2013, 5, 1), dt.datetime(2013, 5, 25), 0, 10],
        frame=Frame(
            axes="WS",
            xaxis=Axis(annot="5d"),
            yaxis=Axis(annot=1, unit="cm"),
            xaxis2=Axis(annot="1O", tick="1D"),
            yaxis2=Axis(label="Length"),
        ),
        x=x,
        y=y,
        style="c0.4c",
        pen="1p",
        fill="green3",
    )
fig.show()
```



The same concept shown above can be applied to smaller as well as larger intervals. In this example, data are plotted for different times throughout two days. The primary x-axis annotations are modified to repeat every 6 hours, and the secondary x-axis annotations repeat every day and show the day of the week.

Another notable mention in this example is setting FORMAT_CLOCK_MAP to **-hhAM** which specifies the format used for time. In this case, leading zeros are removed using (**-**), and only hours are displayed. Additionally, an AM/PM system is used instead of a 24-hour system. More information on configuring time formats can be found at FORMAT_CLOCK_MAP, FORMAT_CLOCK_IN, and FORMAT_CLOCK_OUT.

```
x = pd.date_range("2021-04-15", periods=8, freq="6h")
y = [2, 5, 3, 1, 5, 7, 9, 6]

fig = pygmt.Figure()
with pygmt.config(FORMAT_CLOCK_MAP="-hhAM"):
    fig.plot(
        projection="X15c/10c",
        region=[
            dt.datetime(2021, 4, 14, 23, 0, 0),
            dt.datetime(2021, 4, 17),
            0,
            10,
        ],
        frame=Frame(
            axes="WS",
            xaxis=Axis(annot="6H"),
            yaxis=Axis(annot=1, unit="km/h"),
            xaxis2=Axis(annot="1K"),
            yaxis2=Axis(label="Speed"),
        ),
        x=x,
        y=y,
        style="n0.4c",
        pen="1p",
        fill="lightseagreen",
    )
fig.show()
```



Total running time of the script: (0 minutes 1.100 seconds)

7.2.13 Plotting features on a 3-D surface

In addition to draping a dataset (grid or image) on top of a topographic surface, you may want to add additional features like coastlines, symbols, and text annotations. This tutorial shows how to use *pygmt.Figure.coast*, *pygmt.Figure.plot3d*, and *pygmt.Figure.text* to add these features on a 3-D surface created by *pygmt.Figure.grdview*.

This tutorial builds a 3-D map with additional features in four steps:

1. Creating a 3-D surface
2. Adding coastlines on a 3-D surface
3. Adding symbols on a 3-D surface
4. Adding text annotations on a 3-D surface

```
import pandas as pd
import pygmt
from pygmt.params import Axis, Frame
```

1. Creating a 3-D surface

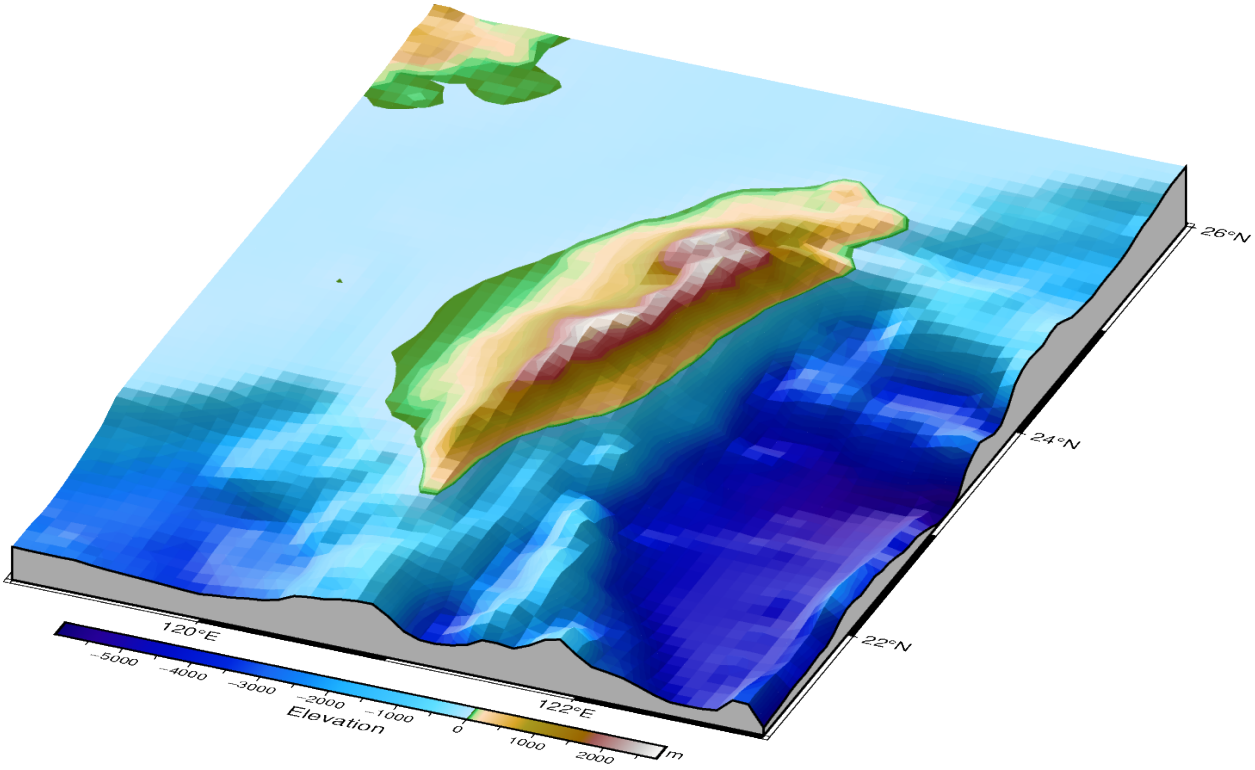
In the first step, we create a 3-D topographic map using *pygmt.Figure.grdview*. We use a region around Taiwan to demonstrate adding features on a 3-D surface.

```
# Define the study area in degrees East or North
region_2d = [119, 123, 21, 26]  # [lon_min, lon_max, lat_min, lat_max]

# Download elevation grid for the study region with a resolution of 5 arc-minutes.
grd_relief = pygmt.datasets.load_earth_relief(resolution="05m", region=region_2d)

# Determine the 3-D region from the minimum and maximum values of the relief grid
region_3d = [*region_2d, grd_relief.min().to_numpy(), grd_relief.max().to_numpy()]

fig = pygmt.Figure()

# Set up a colormap for topography and bathymetry that matches the grid range
pygmt.makecpt(
    cmap="gmt/etopo1",
    series=[grd_relief.min().to_numpy(), grd_relief.max().to_numpy()],
)

# Create a 3-D surface
fig.grdview(
    projection="M12c",  # Mercator projection with a width of 12 cm
    region=region_3d,
    grid=grd_relief,
    cmap=True,
    surftype="surface",
    shading="+a0/270+ne0.6",
    perspective=[157.5, 30],  # Azimuth and elevation for the 3-D plot
    zsize="1.5c",
    facade_fill="darkgray",
    frame=Frame(axes="wSnE", axis=Axis(annot=True, tick=True)),
```

(continues on next page)

(continued from previous page)

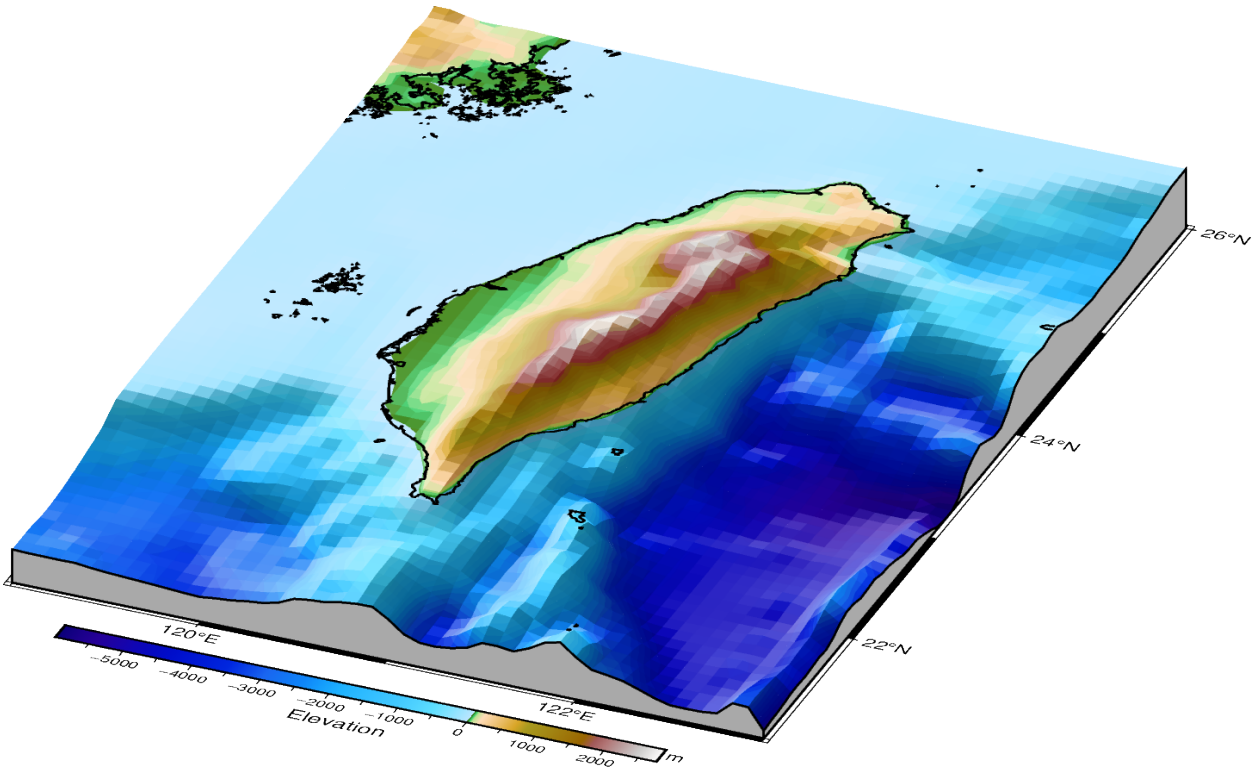
```
)

# Add a colorbar
fig.colorbar(perspective=True, annot=1000, tick=500, label="Elevation", unit="m")

fig.show()
```



2. Adding coastlines on a 3-D surface

Next, we add coastlines using *pygmt.Figure.coast* with a matching `perspective` setting. Here we set the z-level to 0 so coastlines are drawn at sea level.

```
# Add coastlines on top of the 3-D surface
# Use an explicit perspective to match grdview (azimuth=157.5, elevation=30)
# and set the z-level to 0 so the coastlines are drawn at sea level.
fig.coast(perspective=[157.5, 30, 0], resolution="high", shorelines="1/1p,black")

fig.show()
```



3. Adding symbols on a 3-D surface

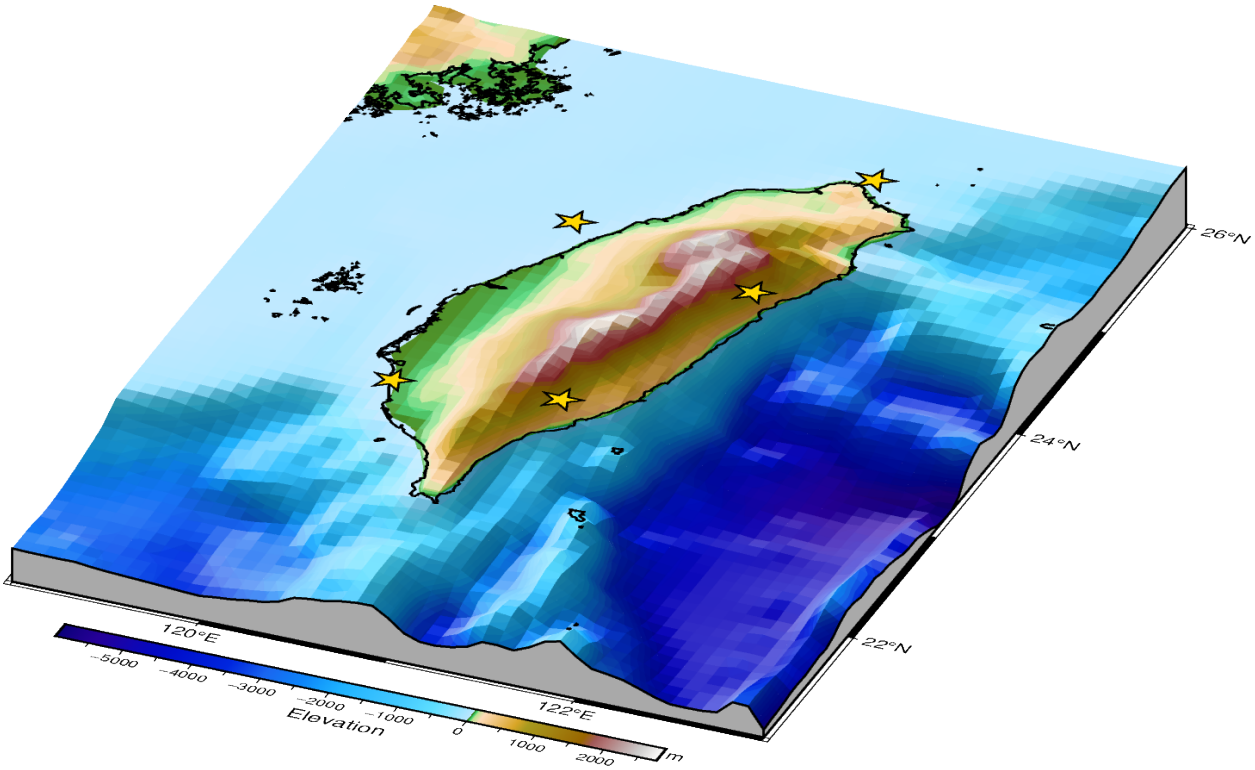
In the third step, we add star symbols on top of the same 3-D map. To plot symbols on a 3-D surface, use *pygmt.Figure.plot3d*. The z-coordinate should be set to a value at or above the maximum elevation to ensure the symbols are visible. Note that 3-D rendering in GMT/PyGMT uses a painter's algorithm (depth sorting) rather than true 3-D occlusion. From some viewpoints, symbols that should be hidden behind terrain may still appear visible.

```
# Sample point data: five coastal cities around Taiwan
cities = pd.DataFrame(
    {
        "longitude": [121.74, 121.61, 121.14, 120.30, 120.53],
        "latitude": [25.13, 23.99, 22.76, 22.63, 24.27],
        "name": ["Keelung", "Hualien", "Taitung", "Kaohsiung", "Taichung Port"],
    }
)

# Use one common z-level so all stars are drawn at the same height above the surface.
cities["z"] = grd_relief.max().to_numpy()

# Add five identical star symbols on top of the 3-D surface
fig.plot3d(
    x=cities.longitude,
    y=cities.latitude,
    z=cities.z,
    style="a0.65c",
    fill="gold",
    pen="0.8p,black",
    perspective=True,
)
```

(continues on next page)

(continued from previous page)

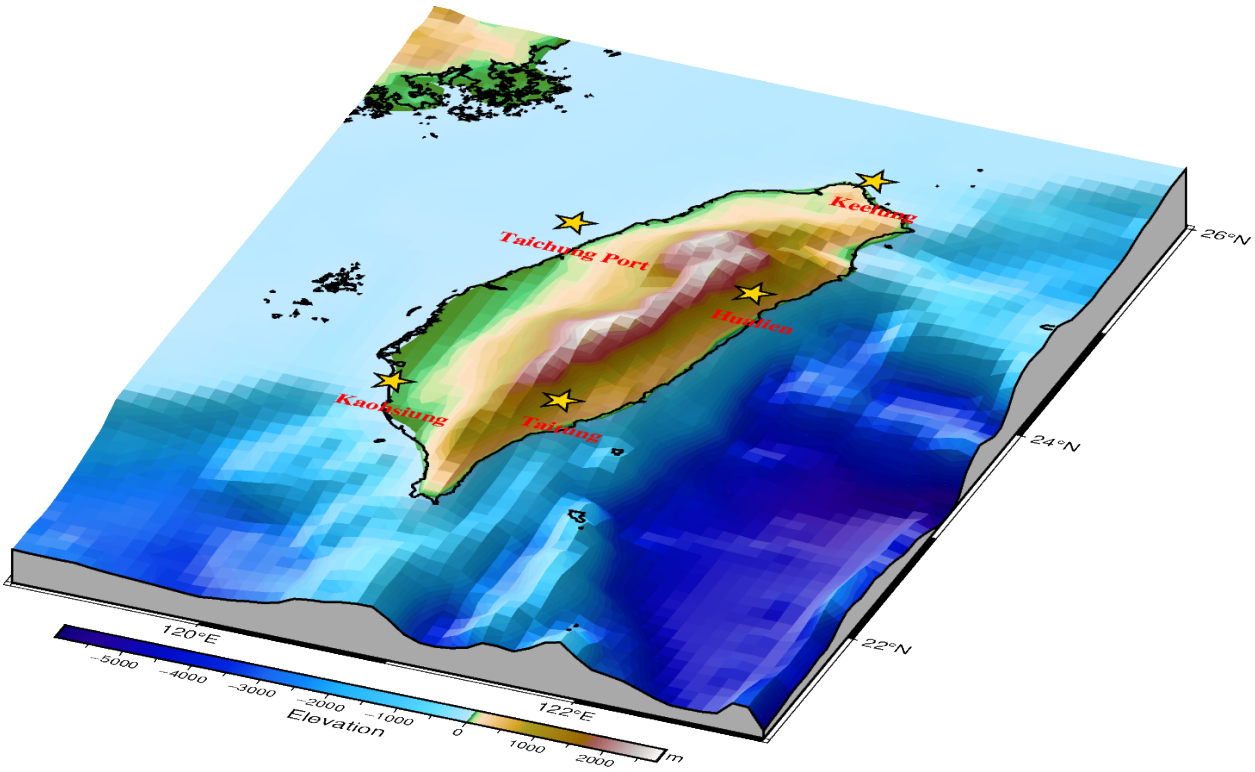
```
fig.show()
```



4. Adding text annotations on a 3-D surface

In the final step, we add text labels to the same 3-D map. To add text annotations on a 3-D surface, use *pygmt.Figure.text* with `perspective=True`. Note that the current implementation of `text` does not support a `z` parameter for controlling the vertical position of text labels. The text will be placed at the base of the 3-D plot (z=0).

```
# Add text labels for cities
# Note: text is placed at z=0 (base level) since z parameter is not yet supported
fig.text(
    x=cities.longitude,
    y=cities.latitude,
    text=cities.name,
    perspective=True,
    font="11p,Times-Bold,red",
)

fig.show()
```



Total running time of the script: (0 minutes 1.117 seconds)

7.2.14 Plotting focal mechanisms

Focal mechanisms can be plotted as beachballs with the *pygmt.Figure.meca* method.

The focal mechanism data or parameters can be provided as various input types: ASCII file, `numpy.array`, dictionary, or `pandas.Dataframe`. Different conventions to define the focal mechanism are supported: Aki and Richards (`"aki"`), global CMT (`"gcmt"`), moment tensor (`"mt"`), partial focal mechanism (`"partial"`), and, principal axis (`"principal_axis"`). Please refer to the table in the documentation of *pygmt.Figure.meca* regarding how to set up the input data in respect to the chosen input type and convention (i.e., the expected column order, keys, or column names). In this tutorial we focus on how to adjust the display of the beachballs.

```
import pandas as pd
import pygmt
from pygmt.params import Pattern

# Set up arguments for basemap
region = [-5, 5, -5, 5]
projection = "X10c/4c"
frame = ["af", "+ggray90"]
```

Setting up the focal mechanism data

We store focal mechanism parameters for two single events in dictionaries using the moment tensor and Aki and Richards conventions:

```
# moment tensor convention
mt_single = {
```

(continues on next page)

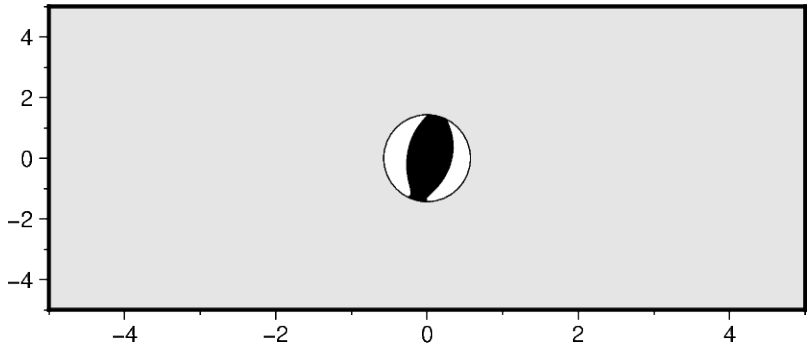
(continued from previous page)

```
    "mrr": 4.71,
    "mtt": 0.0381,
    "mff": -4.74,
    "mrt": 0.399,
    "mrf": -0.805,
    "mtf": -1.23,
    "exponent": 24,
}
# Aki and Richards convention
aki_single = {"strike": 318, "dip": 89, "rake": -179, "magnitude": 7.75}
```

Plotting a single beachball

Required parameters are `spec` and `scale` as well as `longitude`, `latitude` (event location), and depth (if these values are not included in the argument passed to `spec`). Additionally, the `convention` parameter is required if `spec` is an 1-D or 2-D numpy array; for the input types dictionary and `pandas.Dataframe`, the focal mechanism convention is automatically determined from dictionary keys or `pandas.DataFrame` column names. The `scale` parameter controls the radius of the beachball. By default, the value defines the size for a magnitude of 5 (i.e., a scalar seismic moment of $M_0 = 4.0 \times 10^{23}$ dyn cm) and the beachball size is proportional to the magnitude. Append `"+l"` to force the radius to be proportional to the seismic moment.

```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

fig.meca(spec=mt_single, scale="1c", longitude=0, latitude=0, depth=0)

fig.show()
```



Plotting the components of a seismic moment tensor

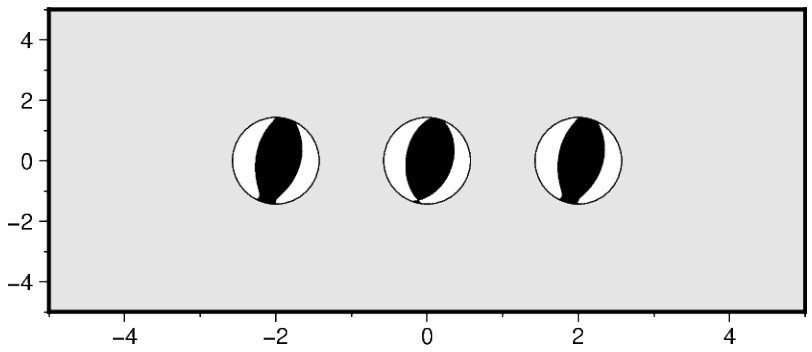
A moment tensor can be decomposed into isotropic and deviatoric parts. The deviatoric part can be further decomposed into multiple parts (e.g., a double couple (DC) and a compensated linear vector dipole (CLVD)). Use the `component` parameter to specify the component you want to plot.

```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

for component, longitude in zip(["full", "dc", "deviatoric"], [-2, 0, 2],␣
↪strict=True):
    fig.meca(
        spec=mt_single,
```

(continues on next page)

(continued from previous page)

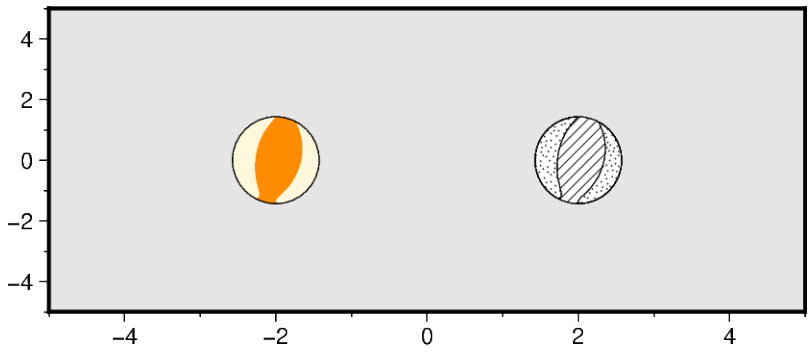
```
        scale="1c",
        longitude=longitude,
        latitude=0,
        depth=0,
        component=component,
    )

fig.show()
```



Filling the quadrants

Use the parameters `compression_fill` and `extension_fill` to fill the quadrants with different colors or *patterns*.

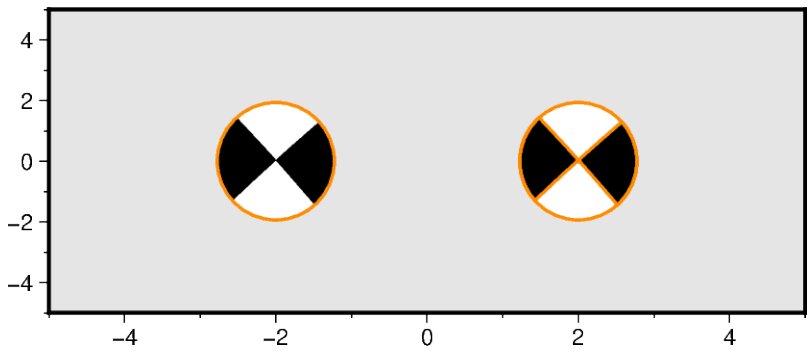
```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

fig.meca(
    spec=mt_single,
    scale="1c",
    longitude=-2,
    latitude=0,
    depth=0,
    compression_fill="darkorange",
    extension_fill="cornsilk",
)

fig.meca(
    spec=mt_single,
    scale="1c",
    longitude=2,
    latitude=0,
    depth=0,
    compression_fill=Pattern(8),
    extension_fill=Pattern(31),
    outline=True,
)

fig.show()
```



Adjusting the outlines

Use the parameters `pen` and `outline` for adjusting the circumference of the beachball or all lines (i.e, circumference and both nodal planes).

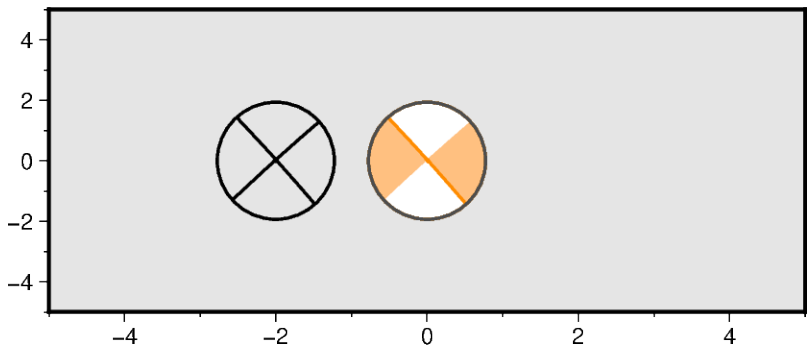
```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

fig.meca(
    spec=aki_single,
    scale="1c",
    longitude=-2,
    latitude=0,
    depth=0,
    # Use a 1-point thick, darkorange and solid line
    pen="1p,darkorange",
)

fig.meca(
    spec=aki_single,
    scale="1c",
    longitude=2,
    latitude=0,
    depth=0,
    outline="1p,darkorange",
)

fig.show()
```



Highlighting the nodal planes

Use the parameter `nodal` to highlight specific nodal planes. `"0"` refers to both, `"1"` to the first, and `"2"` to the second nodal plane(s). Only the circumference and the specified nodal plane(s) are plotted, i.e. the quadrants remain unfilled (transparent). We can make use of the stacking concept of (Py)GMT, and use `nodal` in combination with the `outline`, `compression_fill` / `extension_fill` and `pen` parameters.

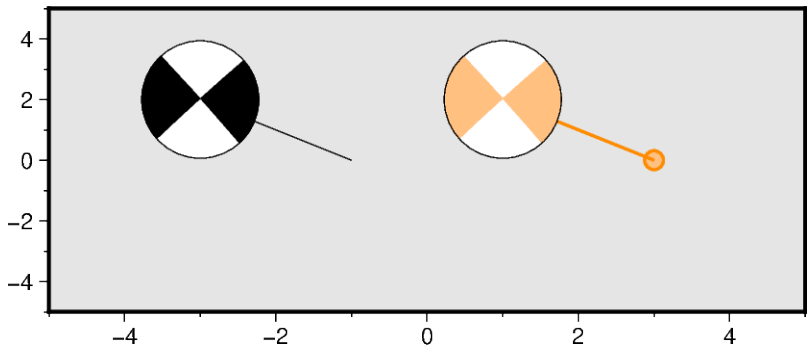
```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

fig.meca(
    spec=aki_single,
    scale="1c",
    longitude=-2,
    latitude=0,
    depth=0,
    nodal="0/1p,black",
)

# Plot the same beachball three times with different settings:
# (i) Fill the compressive quadrants
# (ii) Plot the first nodal plane and the circumference in darkorange
# (iii) Plot the circumference in black on top; use "-" to not fill the quadrants
for kwargs in [
    {"compression_fill": "lightorange"},
    {"nodal": "1/1p,darkorange"},
    {"compression_fill": "-", "extension_fill": "-", "pen": "1p,gray30"},
]:
    fig.meca(
        spec=aki_single,
        scale="1c",
        longitude=0,
        latitude=0,
        depth=0,
        **kwargs,
    )
fig.show()
```



Adding offset from event location

Specify the optional parameters `plot_longitude` and `plot_latitude`. If `spec` is an ASCII file with columns for `plot_longitude` and `plot_latitude`, the `offset` parameter has to be set to `True`. Besides just drawing a line between the beachball and the event location, a small circle can be plotted at the event location by appending **+s** and the desired circle diameter. The connecting line as well as the outline of the circle are plotted with the setting of pen, or can be adjusted separately. The fill of the small circle corresponds to the fill of the compressive quadrants.

```
fig = pygmt.Figure()
fig.basemap(region=region, projection=projection, frame=frame)

fig.meca(
    spec=aki_single,
    scale="1c",
    longitude=-1,
    latitude=0,
    depth=0,
    plot_longitude=-3,
    plot_latitude=2,
)

fig.meca(
    spec=aki_single,
    scale="1c",
    longitude=3,
    latitude=0,
    depth=0,
    plot_longitude=1,
    plot_latitude=2,
    offset="+p1p,darkorange+s0.25c",
    compression_fill="lightorange",
)

fig.show()
```



Plotting multiple beachballs

Now we want to plot multiple beachballs with one call of *pygmt.Figure.meca*. We use data of four earthquakes taken from USGS. For each focal mechanism parameter a list with a length corresponding to the number of events has to be given.

```
# Set up a pandas.DataFrame with multiple focal mechanism parameters.
aki_multiple = pd.DataFrame(
```

(continues on next page)

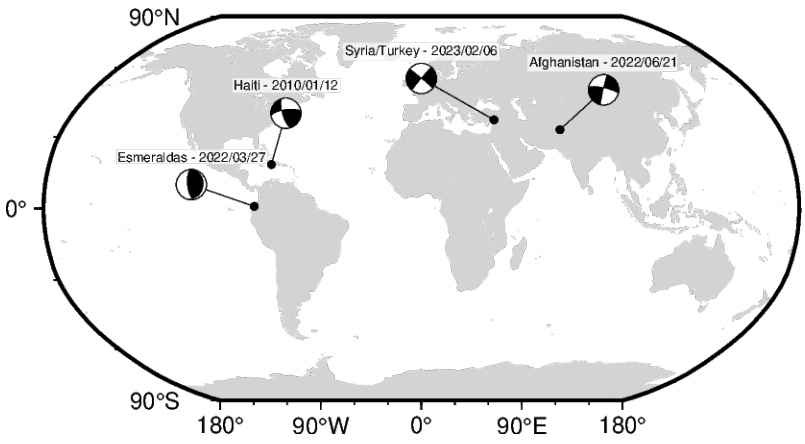
(continued from previous page)

```
    {
        "strike": [255, 173, 295, 318],
        "dip": [70, 68, 79, 89],
        "rake": [20, 83, -177, -179],
        "magnitude": [7.0, 5.8, 6.0, 7.8],
        "longitude": [-72.53, -79.61, 69.46, 37.01],
        "latitude": [18.44, 0.90, 33.02, 37.23],
        "depth": [13, 19, 4, 10],
        "plot_longitude": [-70, -110, 100, 0],
        "plot_latitude": [40, 10, 50, 55],
        "event_name": [
            "Haiti - 2010/01/12",
            "Esmeraldas - 2022/03/27",
            "Afghanistan - 2022/06/21",
            "Syria/Turkey - 2023/02/06",
        ],
    }
)
```

Adding a label

Use the optional parameter `event_name` to add a label near the beachball, e.g., event name or event date and time. Change the font of the label text by appending **+f** and the desired font (size,name,color) to the argument passed to the `scale` parameter. Additionally, the location of the label relative to the beachball [Default is `"TC"`, i.e., Top Center] can be changed by appending **+j** and an offset can be applied by appending **+o** with values for *dx*/*dy*. Add a colored [Default is white] box behind the label via the parameter `label_box`. Force a fixed size of the beachball by appending **+m** to the argument passed to the `scale` parameter.

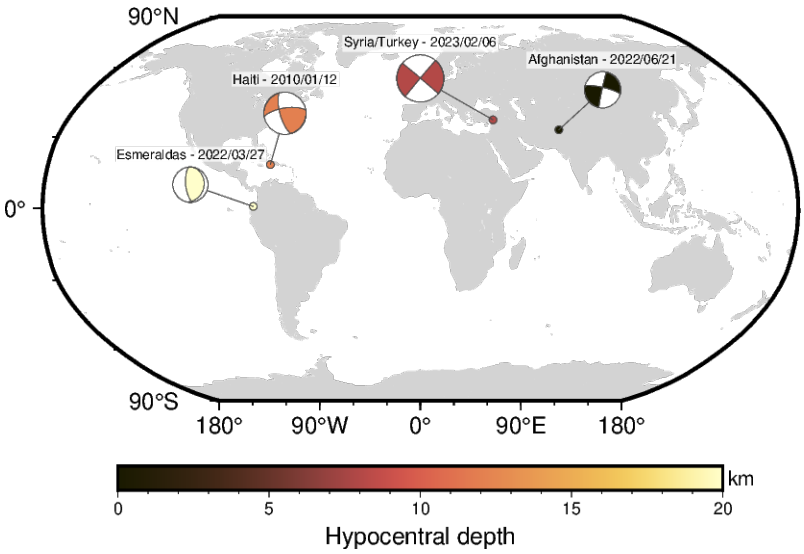
```
fig = pygmt.Figure()
fig.coast(region="d", projection="N10c", land="lightgray", frame=True)

fig.meca(spec=aki_multiple, scale="0.4c+m+f5p", label_box="white@30", offset="+s0.1c")

fig.show()
```



Using size-coding and color-coding

The beachball can be sized and colored by the quantities given as `magnitude` and `depth`, e.g., by moment magnitude or hypocentral depth, respectively. Use the parameter `cmap` to pass the desired colormap. Now, the fills of the small circles indicating the event locations are given by the colormap.

```
fig = pygmt.Figure()
fig.coast(region="d", projection="N10c", land="lightgray", frame=True)

# Set up colormap and colorbar for hypocentral depth
pygmt.makecpt(cmap="SCM/lajolla", series=[0, 20])
fig.colorbar(label="Hypocentral depth", unit="km")

fig.meca(
    spec=aki_multiple,
    scale="0.4c+f5p",
    offset="0.2p,gray30+s0.1c",
    label_box="white@30",
    cmap=True,
    outline="0.2p,gray30",
)

fig.show()
```



Total running time of the script: (0 minutes 0.973 seconds)

7.2.15 Plotting vectors

Plotting vectors is handled by *pygmt.Figure.plot*.

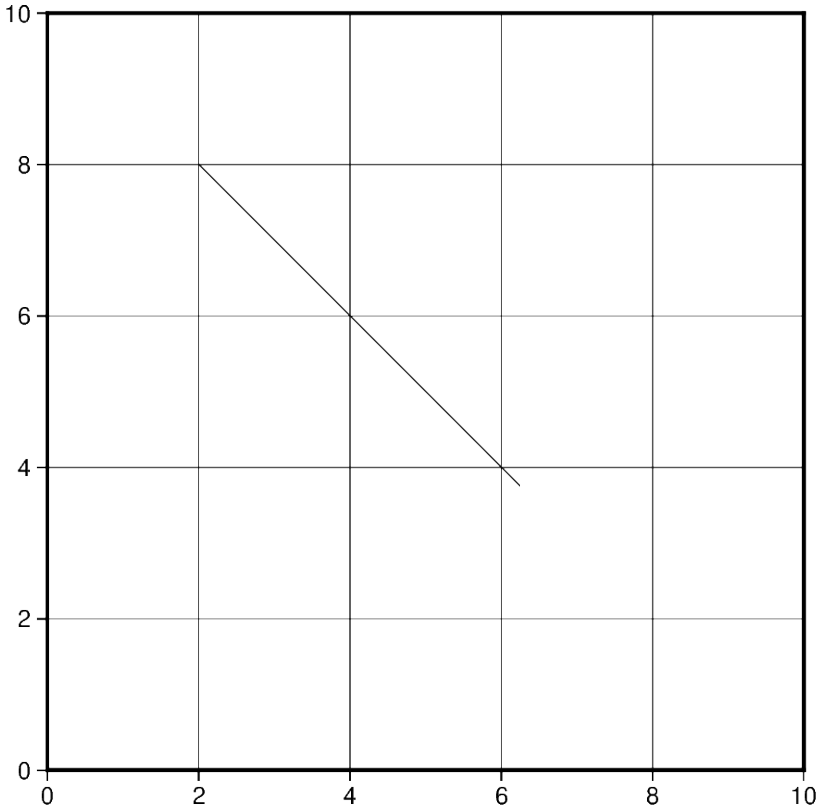
```
import numpy as np
import pygmt
from pygmt.params import Axis
```

Plot Cartesian Vectors

Create a simple Cartesian vector using a start point through `x`, `y`, and `direction` parameters. On the shown figure, the plot is projected on a 10cm X 10cm region, which is specified by the `projection` parameter. The direction is specified by a list of two 1-D arrays structured as `[[angle_in_degrees], [length]]`. The angle is measured in degrees and moves counter-clockwise from the horizontal. The length of the vector uses centimeters by default but could be changed using *`pygmt.config`* (Check the next examples for unit changes).

Notice that the `v` in the `style` parameter stands for vector; it distinguishes it from regular lines and allows for different customization. `0c` is used to specify the size of the arrow head which explains why there is no arrow on either side of the vector.

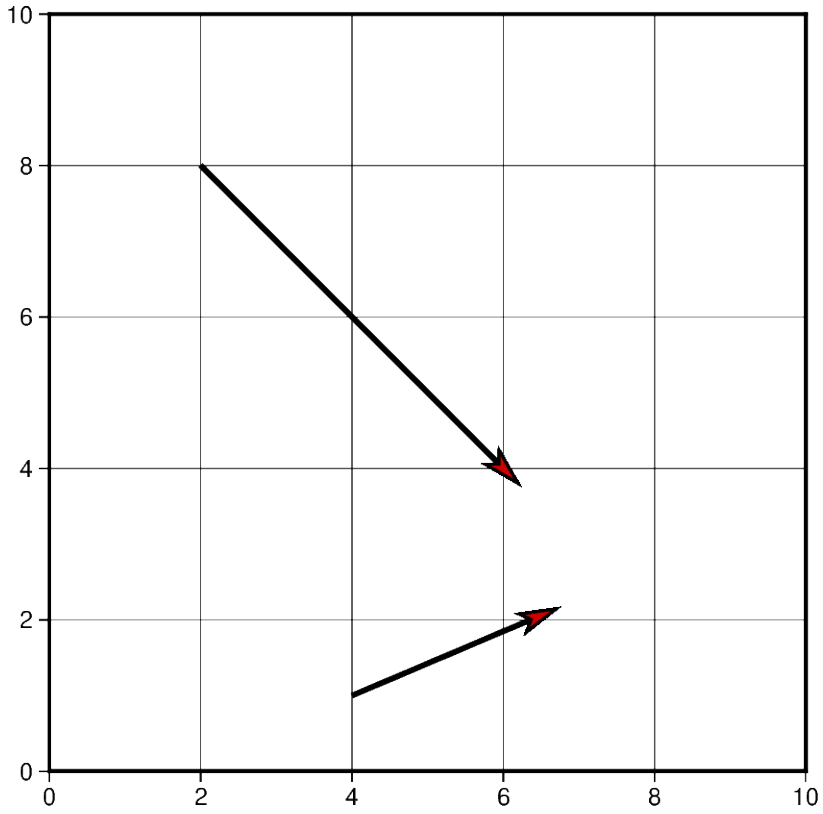
```
fig = pygmt.Figure()
fig.plot(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Axis(annot=True, grid=True),
    x=2,
    y=8,
    style="v0c",
    direction=[[-45], [6]],
)
fig.show()
```



In this example, we apply the same concept shown previously to plot multiple vectors. Notice that instead of passing int/float to `x` and `y`, a list of all x- and y-coordinates will be passed. Similarly, the length of direction list will increase accordingly.

Additionally, we change the style of the vector to include a red arrow head at the end (**+e**) of the vector and increase the thickness (`pen="2p"`) of the vector stem. A list of different styling attributes can be found in *Vector heads and tails*.

```
fig = pygmt.Figure()
fig.plot(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Axis(annot=True, grid=True),
    x=[2, 4],
    y=[8, 1],
    style="v0.6c+e",
    direction=[[-45, 23], [6, 3]],
    pen="2p",
    fill="red3",
)
fig.show()
```



The default unit of vector length is centimeters, however, this can be changed to inches or points. Note that, in PyGMT, one point is defined as 1/72 inch.

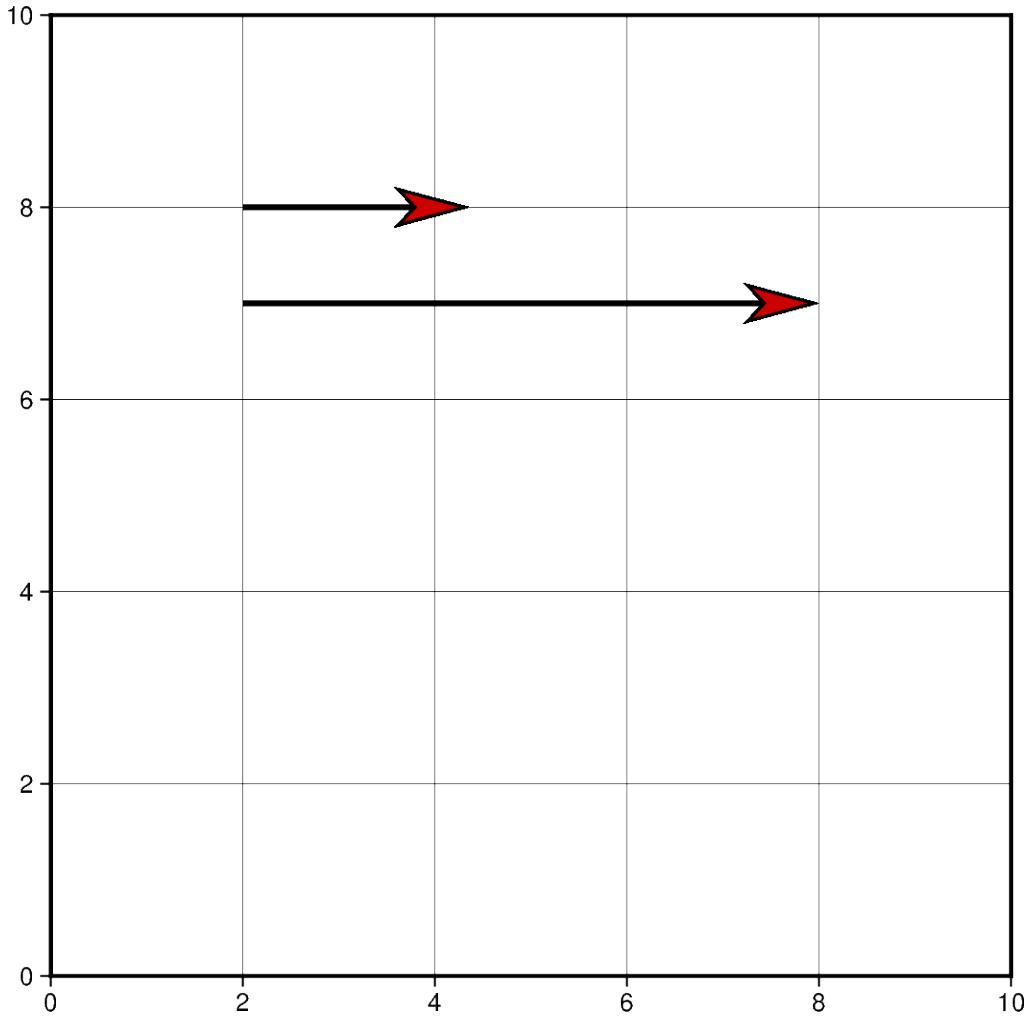
In this example, the graphed region is 5in X 5in, but the length of the first vector is still graphed in centimeters. Using `pygmt.config(PROJ_LENGTH_UNIT="i")`, the default unit can be changed to inches in the second plotted vector.

```
fig = pygmt.Figure()
# Vector 1 with default unit as cm
fig.plot(
    region=[0, 10, 0, 10],
    projection="X5i/5i",
    frame=Axis(annot=True, grid=True),
    x=2,
    y=8,
```

(continues on next page)

(continued from previous page)

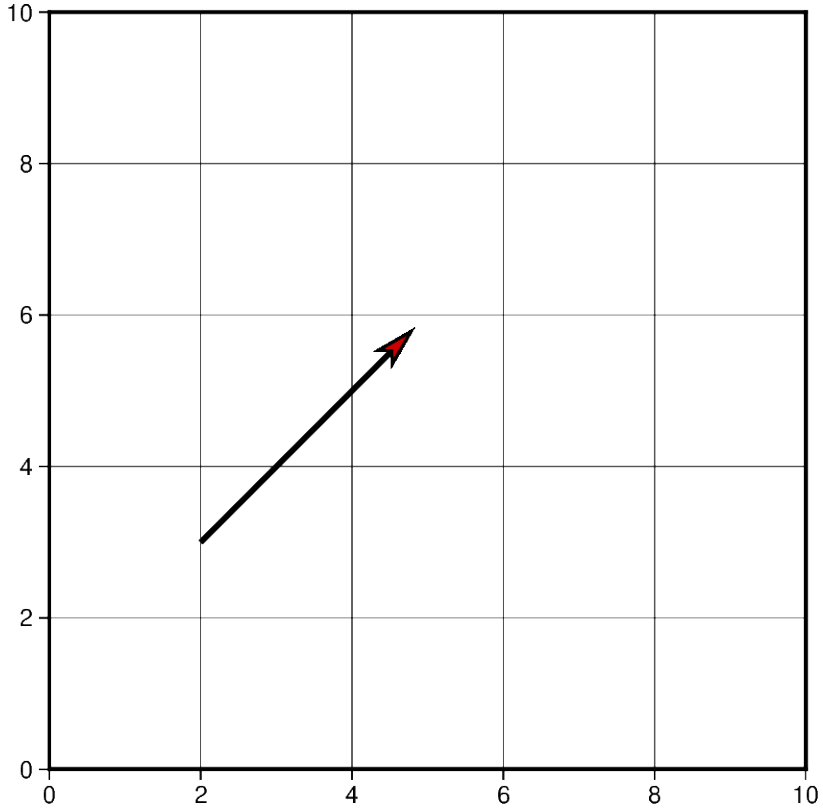
```
    style="v1c+e",
    direction=[[0], [3]],
    pen="2p",
    fill="red3",
)
# Vector 2 after changing default unit to inches
with pygmt.config(PROJ_LENGTH_UNIT="i"):
    fig.plot(
        x=2,
        y=7,
        direction=[[0], [3]],
        style="v1c+e",
        pen="2p",
        fill="red3",
    )
fig.show()
```



Vectors can also be plotted by including all the information about a vector in a single list. However, this requires creating a 2-D list or numpy array containing all vectors. Each vector list contains the information structured as: `[x_start, y_start, direction_degrees, length]`.

If this approach is chosen, the `data` parameter must be used instead of `x`, `y`, and `direction`.

```
# Create a list of lists that include each vector information
vectors = [[2, 3, 45, 4]]

fig = pygmt.Figure()
fig.plot(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Axis(annot=True, grid=True),
    data=vectors,
    style="v0.6c+e",
    pen="2p",
    fill="red3",
)
fig.show()
```



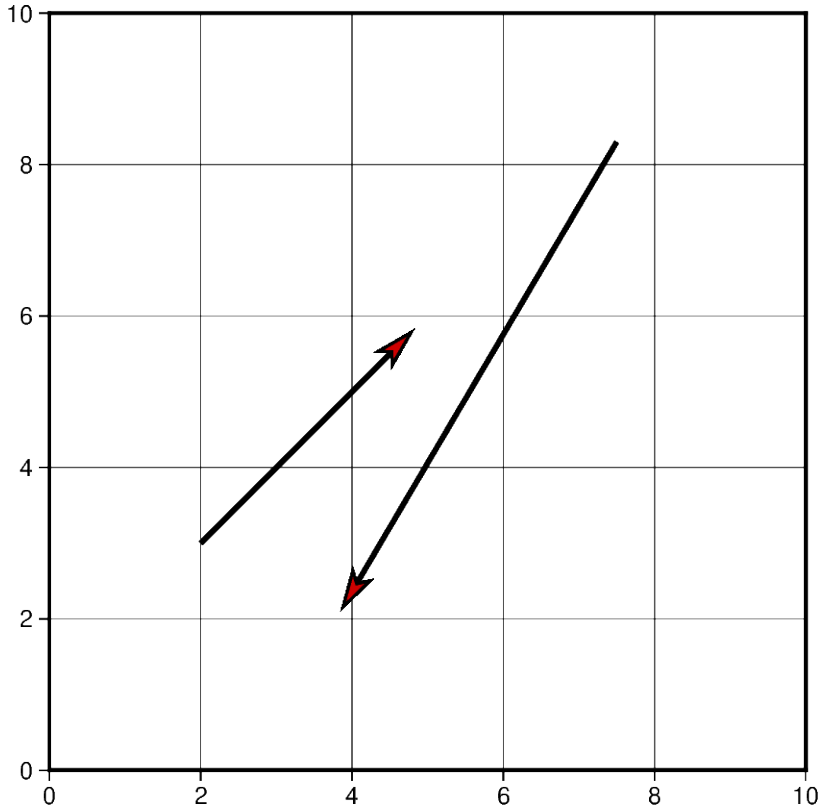
Using the functionality mentioned in the previous example, multiple vectors can be plotted at the same time. Another vector could be simply added to the 2-D list or numpy array object and passed using `data` parameter.

```
# Vector specifications structured as:
# [x_start, y_start, direction_degrees, length]
vector_1 = [2, 3, 45, 4]
vector_2 = [7.5, 8.3, -120.5, 7.2]
# Create a list of lists that include each vector information
vectors = [vector_1, vector_2]
# Vectors structure: [[2, 3, 45, 4], [7.5, 8.3, -120.5, 7.2]]
```

(continues on next page)

(continued from previous page)

```
fig = pygmt.Figure()
fig.plot(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Axis(annot=True, grid=True),
    data=vectors,
    style="v0.6c+e",
    pen="2p",
    fill="red3",
)
fig.show()
```



In this example, Cartesian vectors are plotted over a Mercator projection of the continental US. The x-values represent the longitude and y-values represent the latitude where the vector starts.

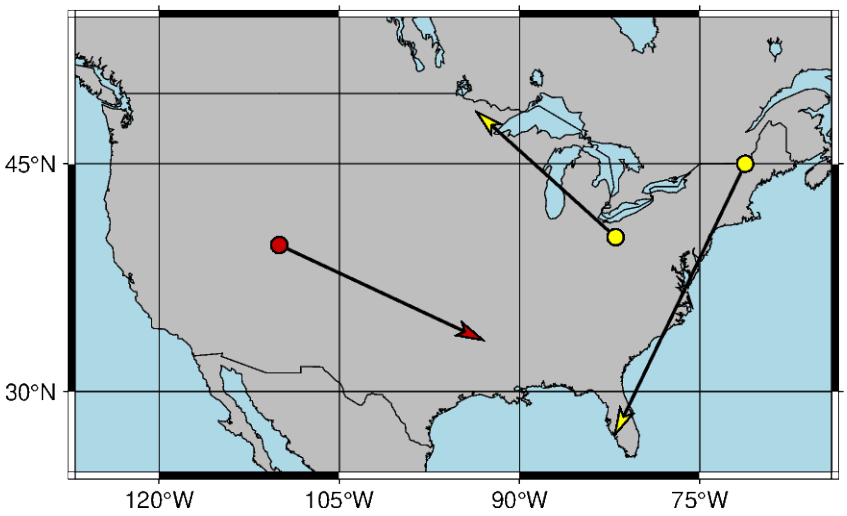
This example also shows some of the styles a vector supports. The beginning point of the vector (**+b**) should take the shape of a circle (**c**). Similarly, the end point of the vector (**+e**) should have an arrow shape (**a**) (to draw a plain arrow, use **A** instead). Lastly, the **+a** specifies the angle of the vector head apex (30 degrees in this example).

```
# Create a plot with coast, Mercator projection (M) over the continental US
fig = pygmt.Figure()
fig.coast(
    region=[-127, -64, 24, 53],
    projection="M10c",
    frame=Axis(annot=True, grid=True),
    borders=1,
    shorelines="0.25p,black",
```

(continues on next page)

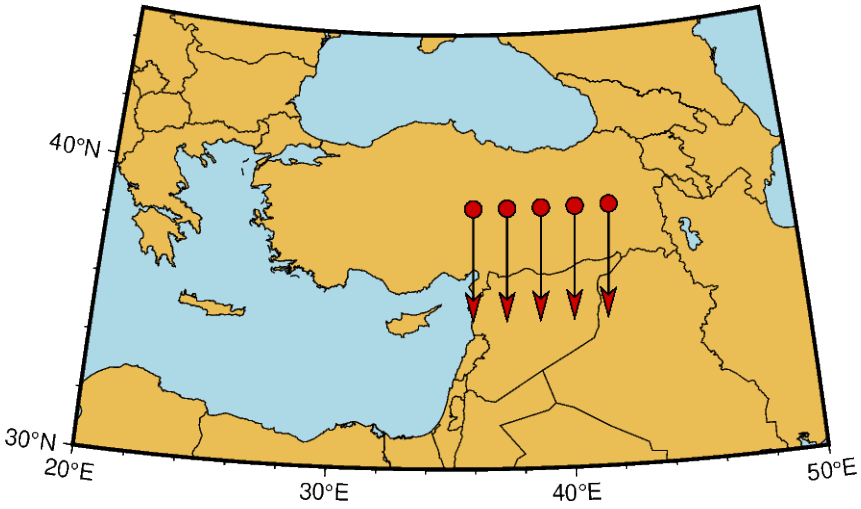
(continued from previous page)

```
    area_thresh=4000,
    land="grey",
    water="lightblue",
)

# Plot a vector using the x, y, direction parameters
style = "v0.4c+bc+ea+a30"
fig.plot(
    x=-110,
    y=40,
    style=style,
    direction=[[-25], [3]],
    pen="1p",
    fill="red3",
)

# vector specifications structured as:
# [x_start, y_start, direction_degrees, length]
vector_2 = [-82, 40.5, 138, 2.5]
vector_3 = [-71.2, 45, -115.7, 4]
# Create a list of lists that include each vector information
vectors = [vector_2, vector_3]

# Plot vectors using the data parameter.
fig.plot(
    data=vectors,
    style=style,
    pen="1p",
    fill="yellow",
)
fig.show()
```



Another example of plotting Cartesian vectors over a coast plot. This time a Transverse Mercator projection is used. Additionally, `numpy.linspace` is used to create 5 vectors with equal stops.

```
x = np.linspace(36, 42, 5)  # x-values = [36.  37.5 39.  40.5 42.]
y = np.linspace(39, 39, 5)  # y-values = [39. 39. 39. 39.]
direction = np.linspace(-90, -90, 5)  # direction values = [-90. -90. -90. -90.]
length = np.linspace(1.5, 1.5, 5)  # length values = [1.5 1.5 1.5 1.5]

# Create a plot with coast,
# Transverse Mercator projection (T) over Turkey and Syria
fig = pygmt.Figure()
fig.coast(
    region=[20, 50, 30, 45],
    projection="T35/10c",
    frame=True,
    borders=1,
    shorelines="0.25p,black",
    area_thresh=4000,
    land="lightbrown",
    water="lightblue",
)

fig.plot(
    x=x,
    y=y,
    style="v0.4c+ea+bc",
    direction=[direction, length],
    pen="0.6p",
    fill="red3",
)

fig.show()
```

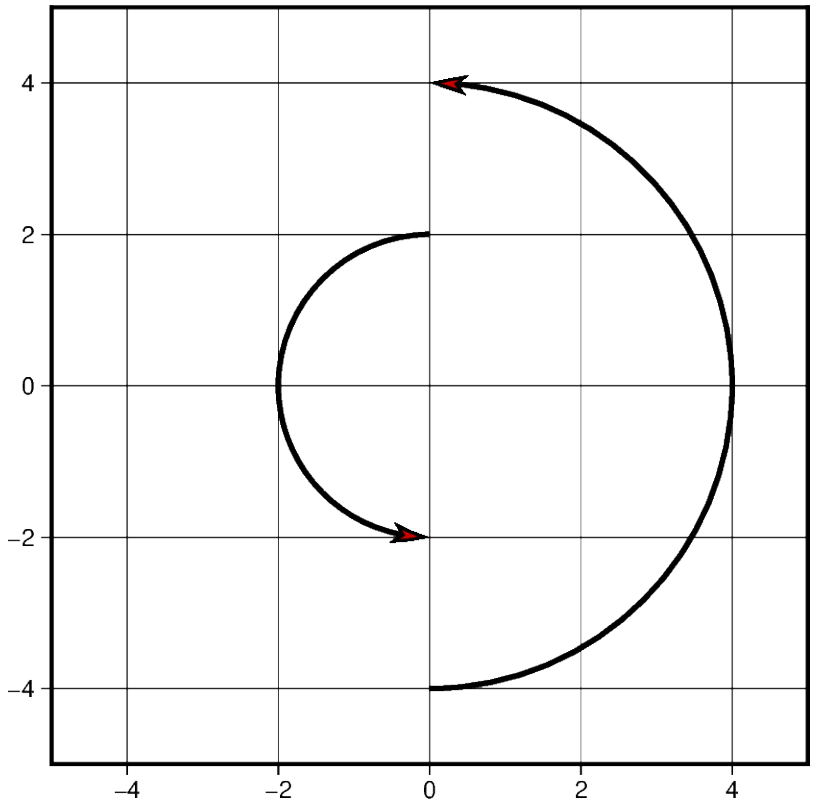


Plot Circular Vectors

When plotting circular vectors, all of the information for a single vector is to be stored in a list. Each circular vector list is structured as: `[x_start, y_start, radius, degree_start, degree_stop]`. The first two values in the vector list represent the origin of the circle that will be plotted. The next value is the radius which is represented on the plot in cm.

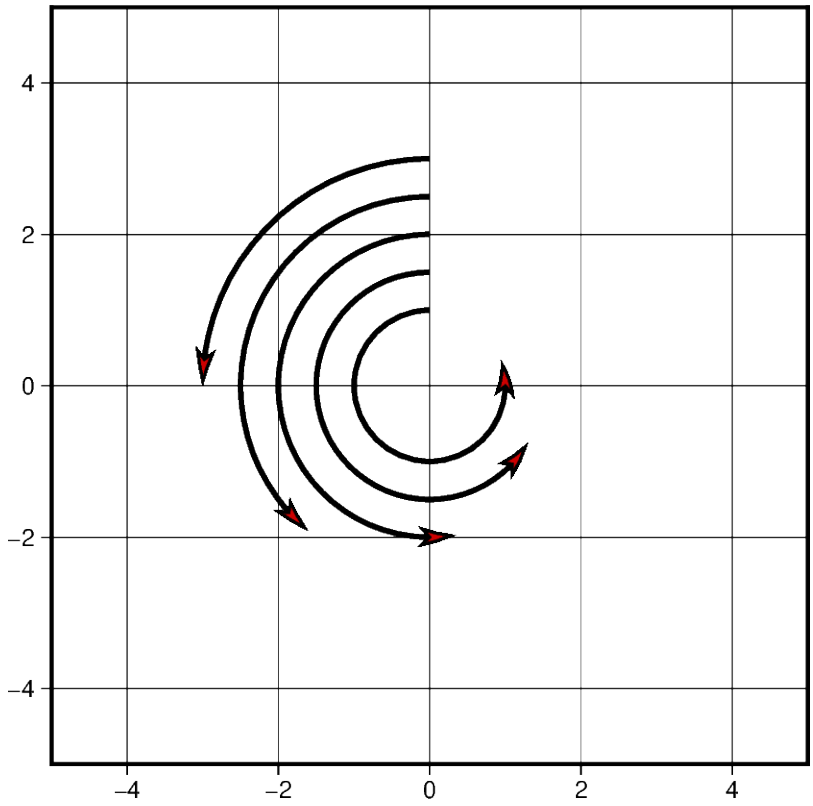
The last two values in the vector list represent the degree at which the plot will start and stop. These values are measured counter-clockwise from the horizontal axis. In this example, the result shown is the left half of a circle as the plot starts at 90 degrees and goes until 270. Notice that the `m` in the `style` parameter stands for circular vectors.

```
fig = pygmt.Figure()

circular_vector_1 = [0, 0, 2, 90, 270]
data = [circular_vector_1]
fig.plot(
    region=[-5, 5, -5, 5],
    projection="X10c",
    frame=Axis(annot=True, grid=True),
    data=data,
    style="m0.5c+ea",
    pen="2p",
    fill="red3",
)

# Another example using np.array()
circular_vector_2 = [0, 0, 4, -90, 90]
data = np.array([circular_vector_2])

fig.plot(
    data=data,
    style="m0.5c+ea",
    pen="2p",
    fill="red3",
)
fig.show()
```



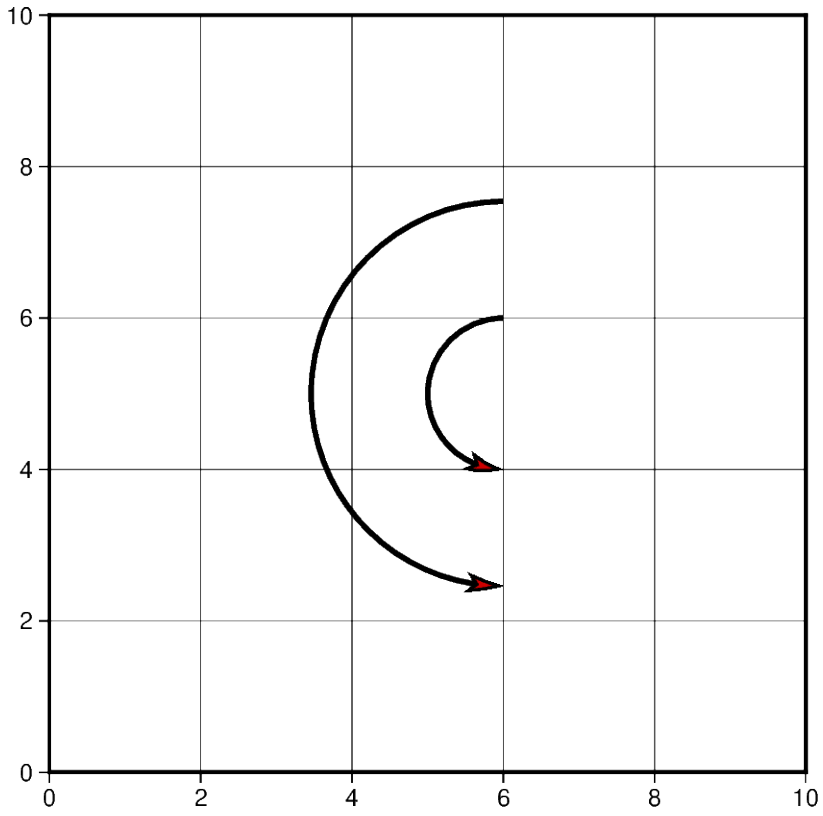
When plotting multiple circular vectors, a two dimensional array or numpy array object should be passed as the `data` parameter. In this example, `numpy.column_stack` is used to generate this two dimensional array. Other numpy objects are used to generate linear values for the `radius` parameter and random values for the `degree_stop` parameter discussed in the previous example. This is the reason in which each vector has a different appearance on the projection.

```
vector_num = 5
radius = 3 - (0.5 * np.arange(0, vector_num))
startdir = np.full(vector_num, 90)
stopdir = 180 + (50 * np.arange(0, vector_num))
data = np.column_stack(
    [np.full(vector_num, 0), np.full(vector_num, 0), radius, startdir, stopdir]
)

fig = pygmt.Figure()
fig.plot(
    region=[-5, 5, -5, 5],
    projection="X10c",
    frame=Axis(annot=True, grid=True),
    data=data,
    style="m0.5c+ea",
    pen="2p",
    fill="red3",
)
fig.show()
```



Much like when plotting Cartesian vectors, the default unit used is centimeters. When this is changed to inches, the size of the plot appears larger when the projection units do not change. Below is an example of two circular vectors. One is plotted using the default unit, and the second is plotted using inches. Despite using the same list to plot the vectors, a different measurement unit causes one to be larger than the other.

```
circular_vector = [6, 5, 1, 90, 270]

fig = pygmt.Figure()
fig.plot(
    region=[0, 10, 0, 10],
    projection="X10c",
    frame=Axis(annot=True, grid=True),
    data=[circular_vector],
    style="m0.5c+ea",
    pen="2p",
    fill="red3",
)

with pygmt.config(PROJ_LENGTH_UNIT="i"):
    fig.plot(
        data=[circular_vector],
        style="m0.5c+ea",
        pen="2p",
        fill="red3",
    )
fig.show()
```



Plot Geographic Vectors

On this map, `point_1` and `point_2` are coordinate pairs used to set the start and end points of the geographic vector. The geographical vector is going from Idaho to Chicago. To style geographic vectors, use = at the beginning of the `style` parameter. Other styling features such as vector stem thickness and head color can be passed into the `pen` and `fill` parameters.

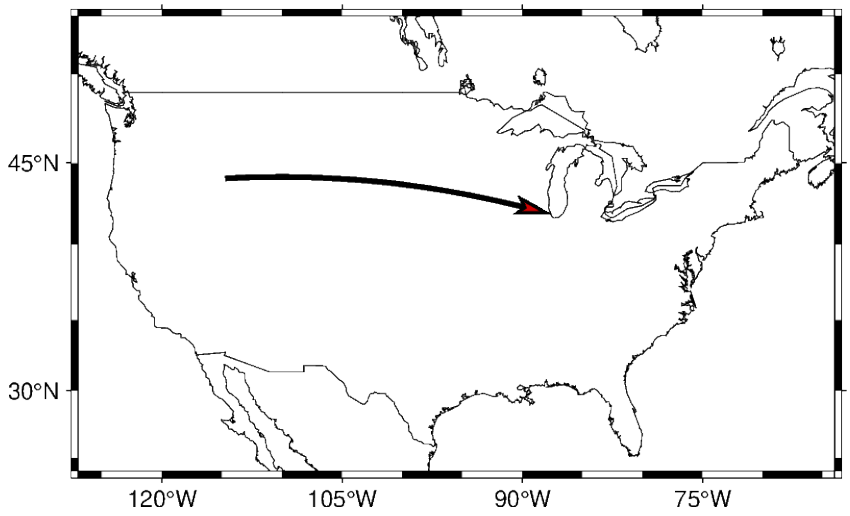
Note that the **+s** is added to use a start point and an end point to represent the vector instead of input angle and length.

```
point_1 = [-114.7420, 44.0682]
point_2 = [-87.6298, 41.8781]
data = np.array([point_1 + point_2])

fig = pygmt.Figure()
fig.coast(
    region=[-127, -64, 24, 53],
    projection="M10c",
    frame=True,
    borders=1,
    shorelines="0.25p,black",
    area_thresh=4000,
)
fig.plot(
    data=data,
    style="=0.5c+ea+s",
    pen="2p",
    fill="red3",
)
```

(continues on next page)

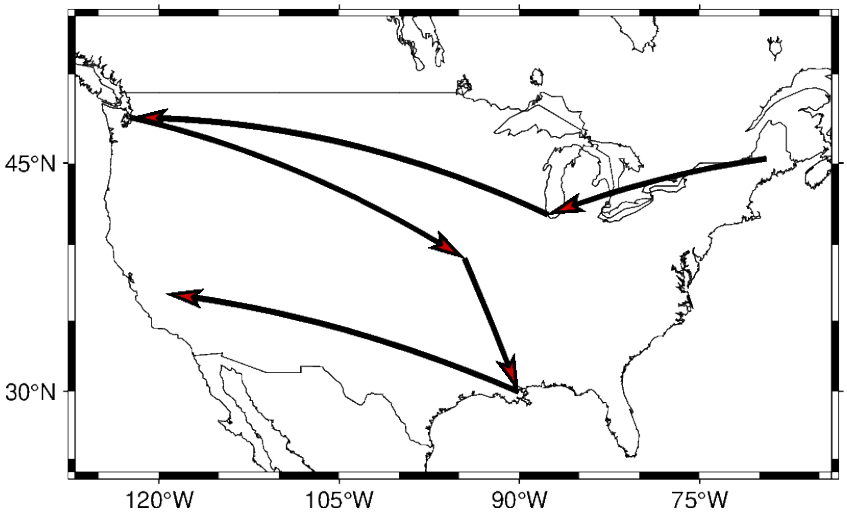
(continued from previous page)

```
fig.show()
```



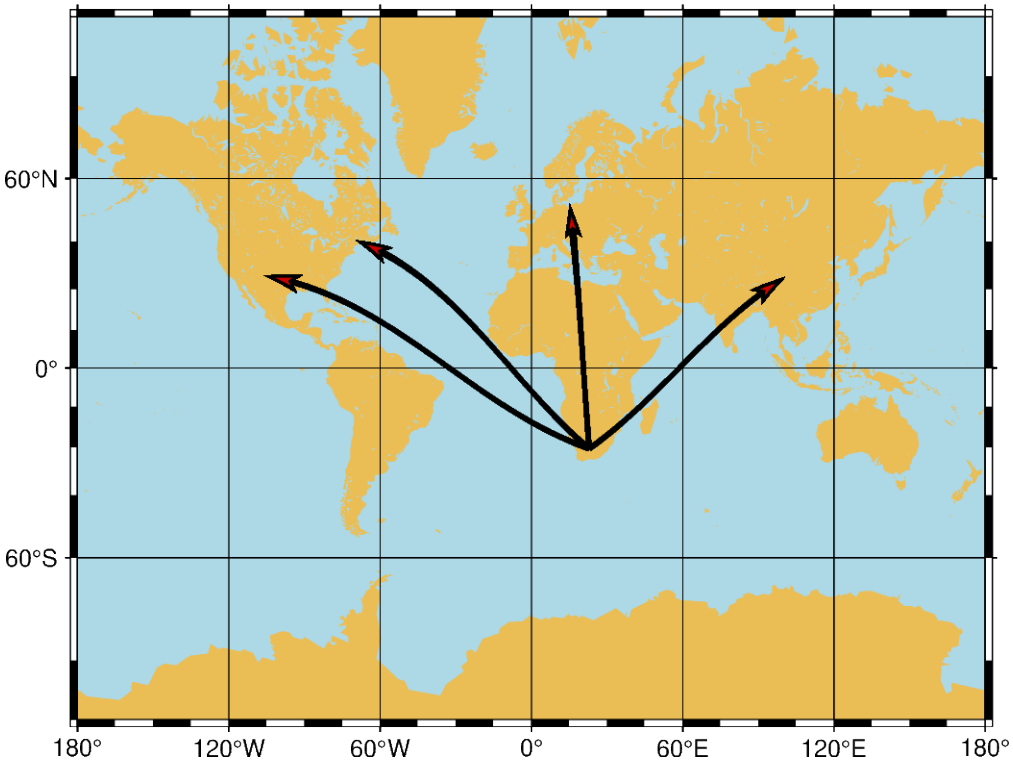
Using the same technique shown in the previous example, multiple vectors can be plotted in a chain where the end point of one is the start point of another. This can be done by adding the coordinate lists together to create this structure: `[[start_latitude, start_longitude, end_latitude, end_longitude]]`. Each list within the 2-D list contains the start and end information for each vector.

```
# Coordinate pairs for all the locations used
ME = [-69.4455, 45.2538]
CHI = [-87.6298, 41.8781]
SEA = [-122.3321, 47.6062]
NO = [-90.0715, 29.9511]
KC = [-94.5786, 39.0997]
CA = [-119.4179, 36.7783]

# Add array to piece together the vectors
data = [ME + CHI, CHI + SEA, SEA + KC, KC + NO, NO + CA]

fig = pygmt.Figure()
fig.coast(
    region=[-127, -64, 24, 53],
    projection="M10c",
    frame=True,
    borders=1,
    shorelines="0.25p,black",
    area_thresh=4000,
)
fig.plot(
    data=data,
    style="=0.5c+ea+s",
    pen="2p",
    fill="red3",
)
fig.show()
```



This example plots vectors over a Mercator projection. The start points are located at SA which is South Africa and going to four different locations.

```
SA = [22.9375, -30.5595]
EUR = [15.2551, 54.5260]
ME = [-69.4455, 45.2538]
AS = [100.6197, 34.0479]
NM = [-105.8701, 34.5199]
data = np.array([SA + EUR, SA + ME, SA + AS, SA + NM])

fig = pygmt.Figure()
fig.coast(
    region=[-180, 180, -80, 80],
    projection="M0/0/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="lightbrown",
    water="lightblue",
)
fig.plot(
    data=data,
    style="=0.5c+ea+s",
    pen="2p",
    fill="red3",
)
fig.show()
```



Total running time of the script: (0 minutes 2.096 seconds)

7.2.16 Typesetting non-ASCII text

In addition to ASCII printable characters, sometimes you may also want to typeset non-ASCII characters on the plot, such as Greek letters, mathematical symbols, or special characters.

Due to the limitations of the underlying PostScript language, PyGMT doesn't support all characters in the Unicode standard. Instead, PyGMT supports a limited set of characters in the "Adobe Symbol", "Adobe ZapfDingbats", "Adobe ISOLatin1+", and "ISO-8859-*x*" (*x* can be 1-11, 13-16) encodings. Refer to *Supported Encodings and Non-ASCII Characters* for the complete list of supported characters.

In PyGMT, the supported (ASCII and non-ASCII) characters can be directly used in the `text` parameter of the *`pygmt.Figure.text`* method for typesetting text strings. They can also be used in the arguments of other plotting functions (e.g., in the `frame` parameter to set the labels or title).

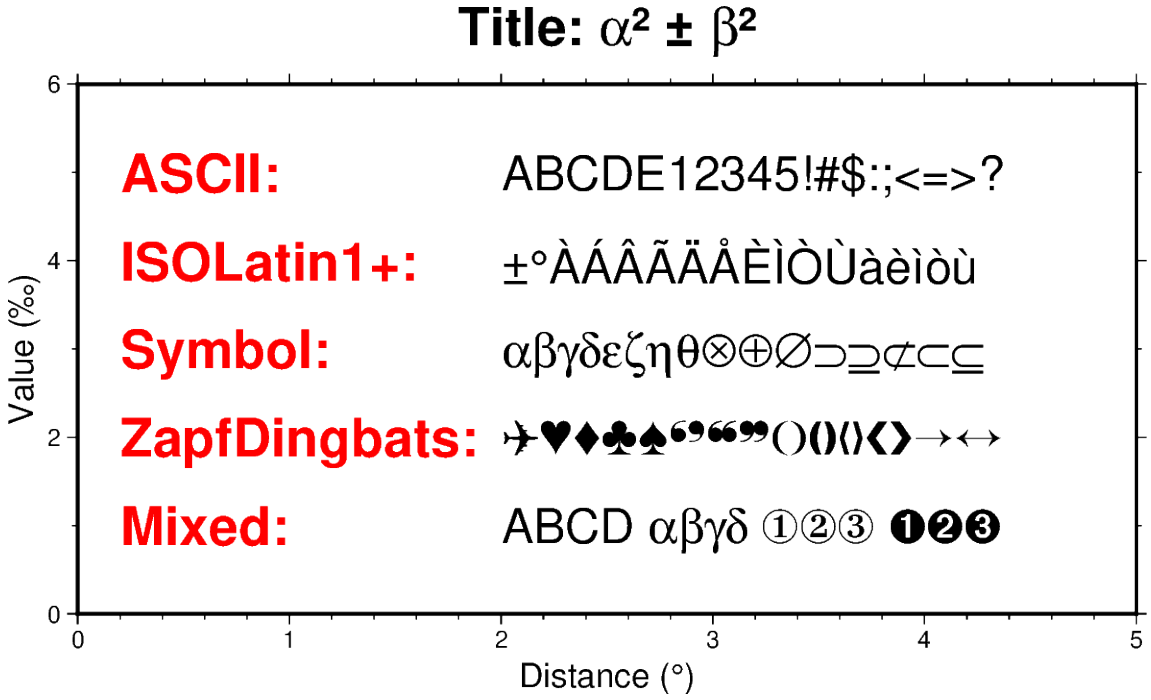
In this example, we demonstrate how to typeset non-ASCII characters in PyGMT.

```
import pygmt
from pygmt.params import Axis, Frame

fig = pygmt.Figure()
fig.basemap(
    region=[0, 5, 0, 6],
    projection="X14c/7c",
    frame=Frame(
        axes="WSen",
        title="Title: α² ± β²",
        xaxis=Axis(annot=True, tick=True, label="Distance (°)"),
        yaxis=Axis(annot=True, tick=True, label="Value (‰)"),
    ),
```

(continues on next page)

(continued from previous page)

```
)

fig.text(
    x=[0.2] * 5,
    y=[5, 4, 3, 2, 1],
    text=["ASCII:", "ISOLatin1+:", "Symbol:", "ZapfDingbats:", "Mixed:"],
    font="20p,Helvetica-Bold,red",
    justify="LM",
)
fig.text(
    x=[2] * 5,
    y=[5, 4, 3, 2, 1],
    text=[
        "ABCDE12345!#$:;<=>?",  # ASCII only
        "±°ÀÁÂÃÄÅÈÌÒÙàèìòù",  # Non-ASCII characters from Adobe ISOLatin1+
        "αβγδεζηθ⊗⊕Ø⊃⊇⊄⊂⊆",  # Non-ASCII characters from Adobe Symbol
        "✈♥♦♣♠❝❞❨❩❪❫❬❭❮❯→↔",  # Non-ASCII characters from Adobe ZapfDingbats
        "ABCD αβγδ ①②③ ❶❷❸",  # Mix characters from ISOLatin1+, Symbol and␣
↪ZapfDingbats
    ],
    font="18p,Helvetica",
    justify="LM",
)

fig.show()
```



Here are some important tips when using non-ASCII characters:

- **Similar-looking characters**: Be cautious when using characters that appear visually similar but are distinct. For example, Ω (OHM SIGN) and Ω (GREEK CAPITAL LETTER OMEGA) may look alike, but PyGMT only supports the latter. Using the incorrect character can lead to unexpected results. To avoid this, it's recommended to

copy and paste characters from the *Supported Encodings and Non-ASCII Characters* documentation.

- **Mix characters from different encodings**: As shown in the example above, you can mix characters from different encodings in the same text string. However, due to the limitations of the underlying PostScript language, you cannot mix characters from the "Adobe ISOLatin1+" and "ISO-8859-*x*" encodings in the same text string. For example, you cannot mix characters from "Adobe ISOLatin1+" and "ISO-8859-2". If you need to use characters from different encodings, you can use them in different PyGMT function/method calls.
- **Non-ASCII characters in text files**: Non-ASCII characters are not supported if you have them in a text file and pass it to `pygmt.Figure.text`. In this case, you may want to load the text file into `pandas.DataFrame` and then pass it to the `text` parameter.

Total running time of the script: (0 minutes 0.145 seconds)

CHAPTER

EIGHT

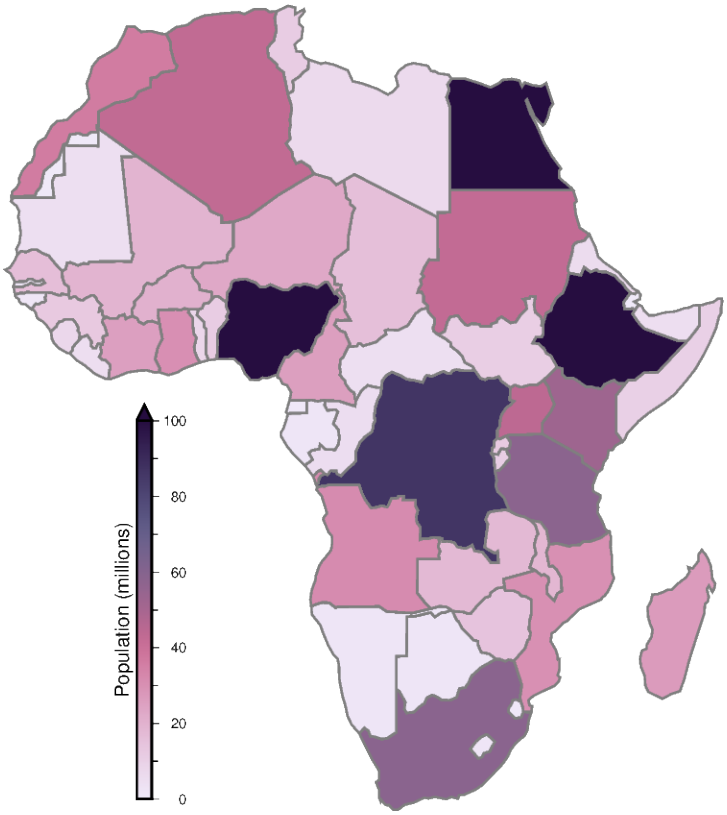
GALLERY

This gallery contains examples of what PyGMT can do. Click on any example to see the code used to generate it.

8.1 Maps and map elements

8.1.1 Choropleth map

The *pygmt.Figure.choropleth* method allows us to plot geographical data such as polygons which are stored in a `geopandas.GeoDataFrame` object or an OGR_GMT file. Use `geopandas.read_file` to load data from any supported OGR formats such as a shapefile (.shp), GeoJSON (.geojson), geopackage (.gpkg), etc. You can also use a full URL pointing to your desired data source. Then, pass the `geopandas.GeoDataFrame` as an argument to the `data` parameter of *pygmt.Figure.choropleth*, and style the geometry using the `pen` parameter. To fill the polygons based on a corresponding column you need to specify the column name to the `column` parameter.

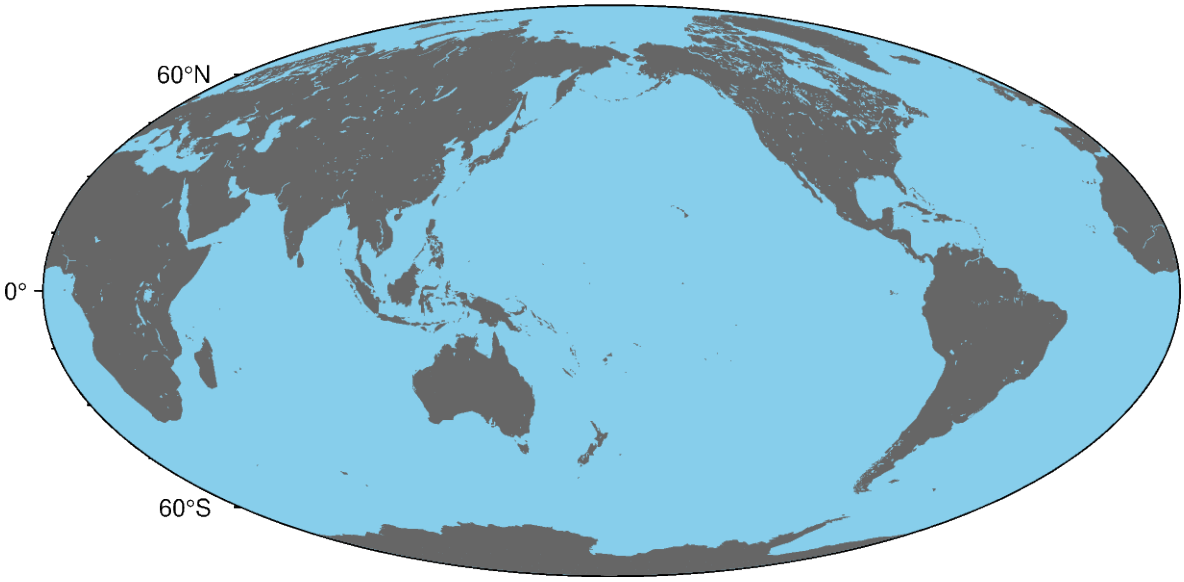


```
import geopandas
import pygmt
from pygmt.params.position import Position

provider = "https://naciscdn.org/naturalearth"
world = geopandas.read_file(f"{provider}/110m/cultural/ne_110m_admin_0_countries.zip")

# The dataset contains different attributes, here we focus on the population within
# the different countries (column "POP_EST") for the continent "Africa".
world["POP_EST"] *= 1e-6
africa = world[world["CONTINENT"] == "Africa"].copy()

fig = pygmt.Figure()
fig.basemap(region=[-19.5, 53, -37.5, 38], projection="M10c", frame="none")

# First, we define the colormap to fill the polygons based on the "POP_EST" column.
pygmt.makecpt(cmap="SCM/acton", series=(0, 100), reverse=True)

# Next, we plot the polygons and fill them using the defined colormap. The target␣
↪column
# is defined by the aspatial parameter.
fig.choropleth(data=africa, column="POP_EST", pen="0.8p,gray50")

# Add colorbar legend.
fig.colorbar(
    label="Population (millions)",
    position=Position("ML", offset=(2, -2.5)),
    length=5,
    fg_triangle=True,
    triangle_height=0.2,
    move_text="label",
)
fig.show()
```

Total running time of the script: (0 minutes 0.417 seconds)

8.1.2 Color land and water

The `land` and `water` parameters of *pygmt.Figure.coast* specify a color to fill in the land and water masses, respectively. There are many color codes in GMT, including standard GMT color names (like `"skyblue"`), R/G/B levels (like `"0/0/255"`), a hex value (like `"#333333"`), and a gray level (like `"gray50"`).

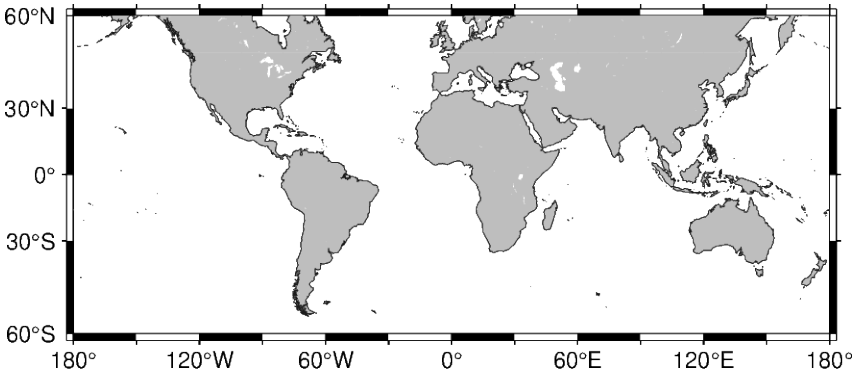



```
import pygmt

fig = pygmt.Figure()
# Make a global Mollweide map with automatic ticks
fig.basemap(region="g", projection="W15c", frame=True)
# Plot the land as light gray, and the water as sky blue
fig.coast(land="#666666", water="skyblue")
fig.show()
```

Total running time of the script: (0 minutes 0.178 seconds)

8.1.3 EPSG codes

Besides one of the *31 projections supported by GMT*, users can pass an EPSG (European Petroleum Survey Group) code to the `projection` parameter. A commonly used EPSG code is `EPSG:3857`, that refers to the Web Mercator projection WGS84. More information on the EPSG dataset can be found at https://epsg.org and https://spatialreference.org/. Please note, that not all EPSG codes are supported by GMT / PyGMT.




```
import pygmt

fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

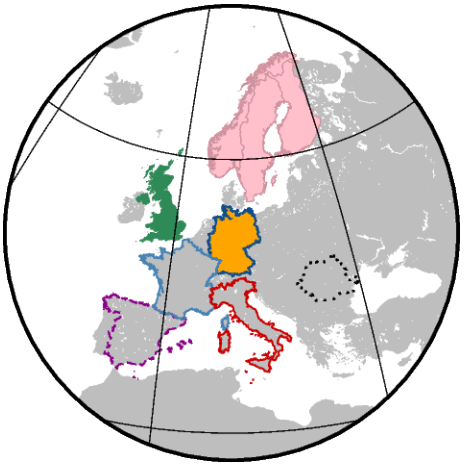
```
# Pass the desired EPSG code and the width of the map (separated by a slash) to the
# projection parameter
fig.basemap(region=[-180, 180, -60, 60], projection="EPSG:3857/10c", frame=30)
fig.coast(land="gray", shorelines="1/0.1p,gray10")

fig.show()
```

Total running time of the script: (0 minutes 0.247 seconds)

8.1.4 Highlight country, continent and state polygons

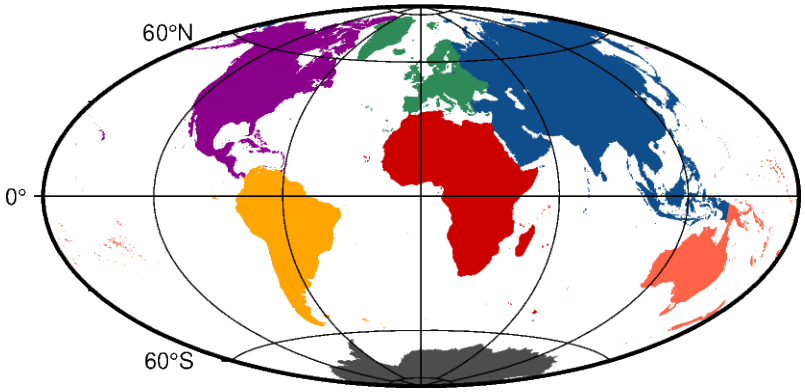
The *pygmt.Figure.coast* method can highlight country polygons via the `dcw` parameter. It accepts the country code (following the 2-letter ISO 3166-1 alpha-2 convention) or full country name and can draw its borders and add a color to its landmass. It's also possible to define multiple countries at once by separating the individual names with commas.

```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()

fig.coast(
    region=[-12, 32, 34, 72],
    # Lambert Azimuthal Equal Area lon0/lat0/horizon/width
    projection="A10/52/25/6c",
    land="gray",
    water="white",
    frame=Axis(annot=True, tick=True, grid=True),
    dcw=[
        # Great Britain (country code) with seagreen land
        "GB+gseagreen",
        # Italy with a red border
        "IT+p0.5p,red3",
        # Spain with a magenta dashed border
        "ES+p0.5p,magenta4,-",
        # Romania with a black dotted border
        "RO+p0.75p,black,.",
        # Germany with orange land and a blue border
        "DE+gorange+p0.5p,dodgerblue4",
        # France (full country name) with a steelblue border
        "France+p0.5p,steelblue",
        # Norway, Sweden and Finland (multiple countries) with pink
        # land and pink3 borders
        "Norway,Sweden,Finland+gpink+p0.2p,pink3",
    ],
)

fig.show()
```



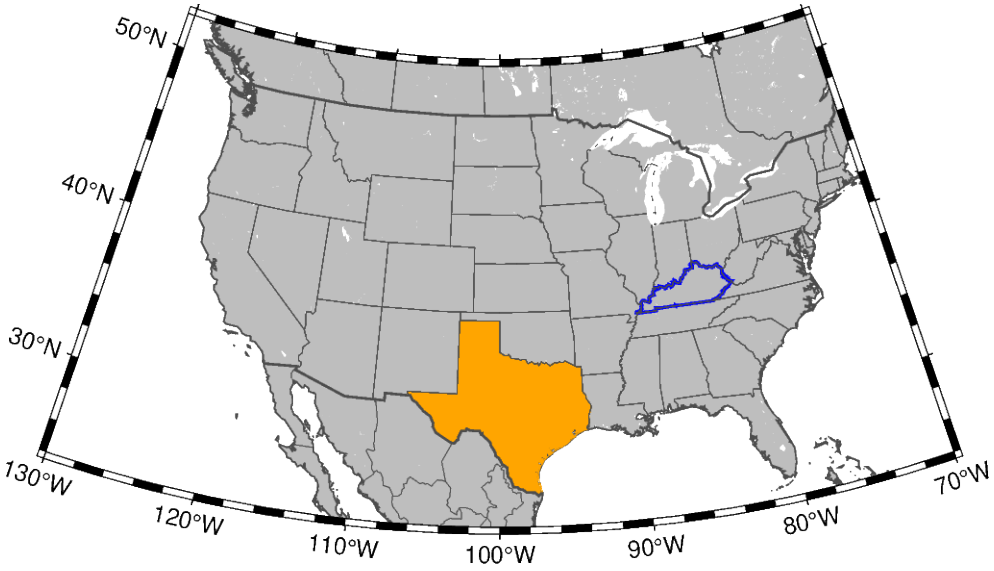
Entire continents can also be highlighted by adding `"="` in front of the continent code to differentiate it from a country code.

```
fig = pygmt.Figure()

fig.coast(
    region="d",
    projection="H10c",
    land="gray",
    water="white",
    frame=Axis(annot=True, tick=True, grid=True),
    dcw=[
        # Europe
        "=EU+gseagreen",
        # Africa
        "=AF+gred3",
        # North America
        "=NA+gmagenta4",
        # South America
        "=SA+gorange",
        # Asia
        "=AS+gdodgerblue4",
        # Oceania
        "=OC+gtomato",
        # Antarctica
        "=AN+ggray30",
    ],
)

fig.show()
```



If available, states/territories of a country can be highlighted, too.

```
fig = pygmt.Figure()

fig.coast(
    region=[-130, -70, 24, 52],
    projection="L-100/35/33/45/12c",
    land="gray",
    shorelines="1/0.5p,gray30",
    borders=["1/0.8p,gray30", "2/0.2p,gray30"],
    frame=True,
    dcw=[
        # Texas with orange fill
        "US.TX+gorange",
        # Kentucky with blue outline
        "US.KY+p1p,blue",
    ],
)

fig.show()
```



Total running time of the script: (0 minutes 5.621 seconds)

8.1.5 Political boundaries

The `borders` parameter of *`pygmt.Figure.coast`* specifies levels of political boundaries to plot and the pen used to draw them. Choose from the list of boundaries below:

- **1** = National boundaries
- **2** = State boundaries within the Americas
- **3** = Marine boundaries
- **a** = All boundaries (1-3)

For example, to draw national boundaries with a line thickness of 1 point and black line color use `borders="1/1p,black"`. You can draw multiple boundaries by passing in a list to `borders`.

```
import pygmt
from pygmt.params import Axis
```

(continues on next page)

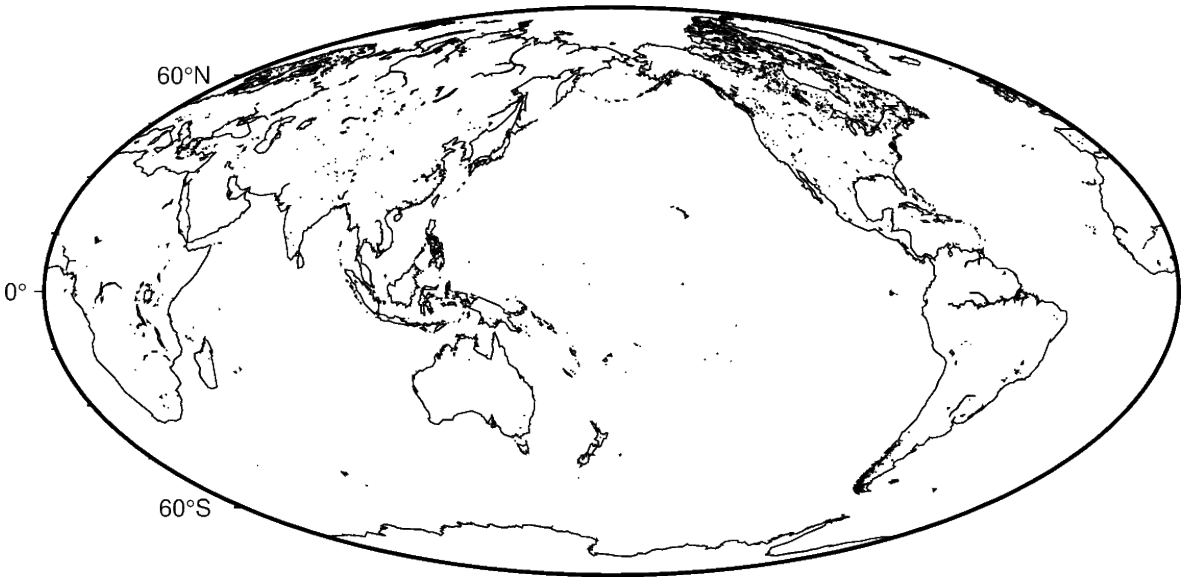
(continued from previous page)

```
fig = pygmt.Figure()
# Make a Sinusoidal projection map of the Americas with automatic annotations,
# ticks and gridlines
fig.basemap(
    region=[-150, -30, -60, 60],
    projection="I-90/15c",
    frame=Axis(annot=True, tick=True, grid=True),
)
# Plot each level of the boundaries dataset with a different color.
fig.coast(borders=["1/0.5p,black", "2/0.5p,red", "3/0.5p,blue"], land="gray")
fig.show()
```

Total running time of the script: (0 minutes 0.269 seconds)

8.1.6 Shorelines

Use *pygmt.Figure.coast* to display shorelines as black lines.



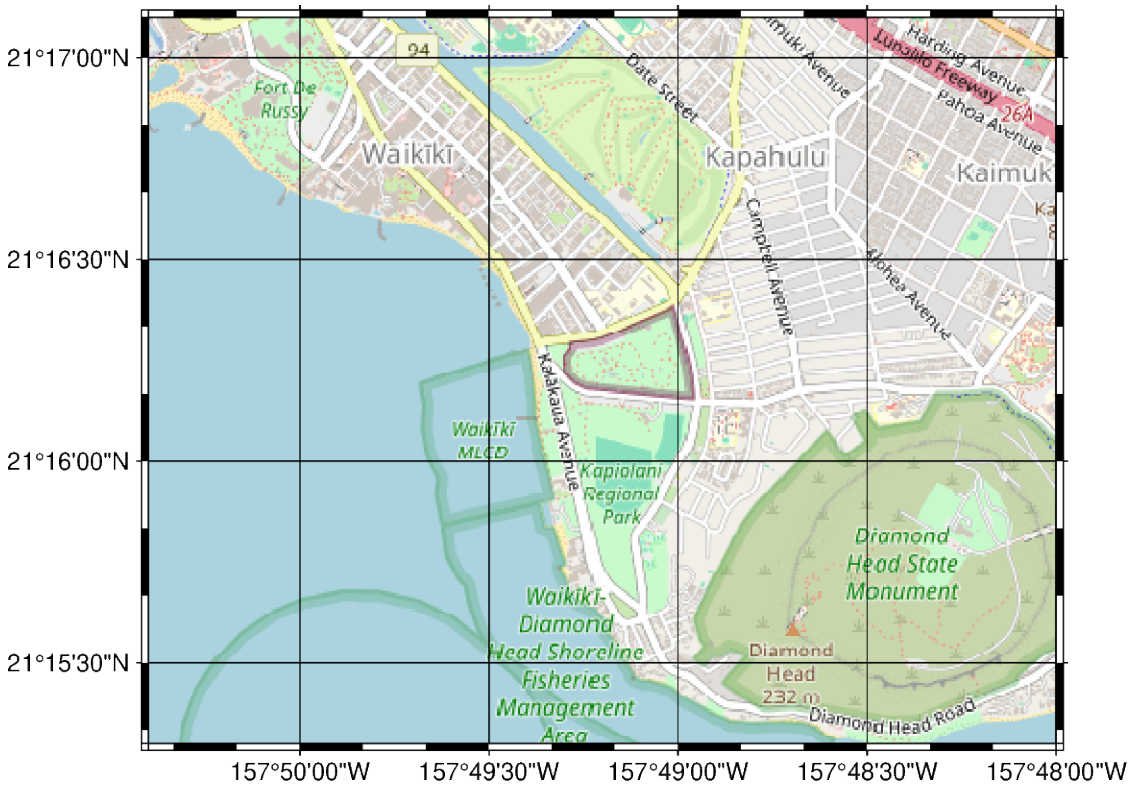

```
import pygmt

fig = pygmt.Figure()
# Make a global Mollweide map with automatic ticks
fig.basemap(region="g", projection="W15c", frame=True)
# Display the shorelines as black lines with 0.5-point thickness
fig.coast(shorelines="0.5p,black")
fig.show()
```

Total running time of the script: (0 minutes 0.175 seconds)

8.1.7 Tile maps

The *pygmt.Figure.tilemap* method allows to plot tiles from a tile server or local file as a basemap or overlay.

```
import contextily
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.tilemap(
    region=[-157.84, -157.8, 21.255, 21.285],
    projection="M12c",
    # Set level of details (0-22).
    # Higher levels mean a zoom level closer to the Earth's surface with more tiles
    # covering a smaller geographic area and thus more details and vice versa.
    # Please note, not all zoom levels are always available.
    zoom=14,
    # Use tiles from OpenStreetMap tile server
    source="https://tile.openstreetmap.org/{z}/{x}/{y}.png",
    frame=Axis(annot=True, tick=True, grid=True),
)

fig.show()
```



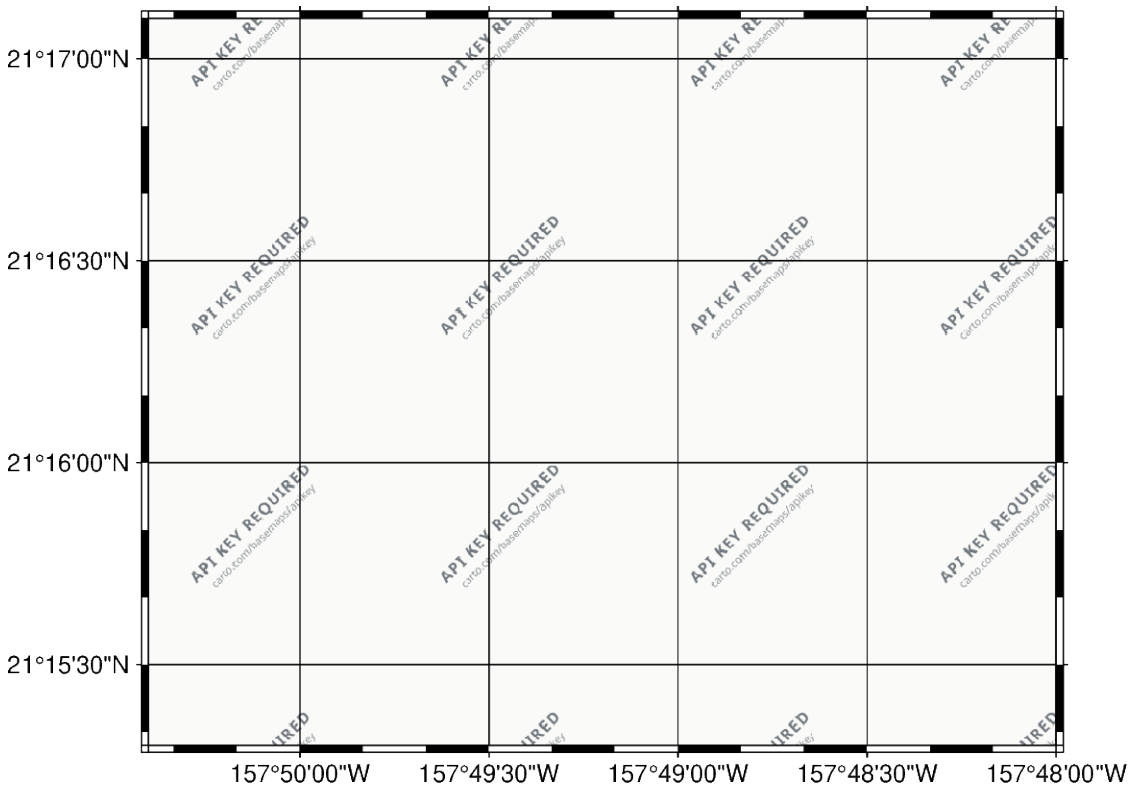
It's also possible to use tiles provided via the contextily library. See Contextily providers for a list of possible tilemap options.

```
fig = pygmt.Figure()
fig.tilemap(
    region=[-157.84, -157.8, 21.255, 21.285],
    projection="M12c",
    # Use the CartoDB Positron option from contextily
```

(continues on next page)

(continued from previous page)

```
    source=contextily.providers.CartoDB.Positron,
    frame=Axis(annot=True, tick=True, grid=True),
)

fig.show()
```



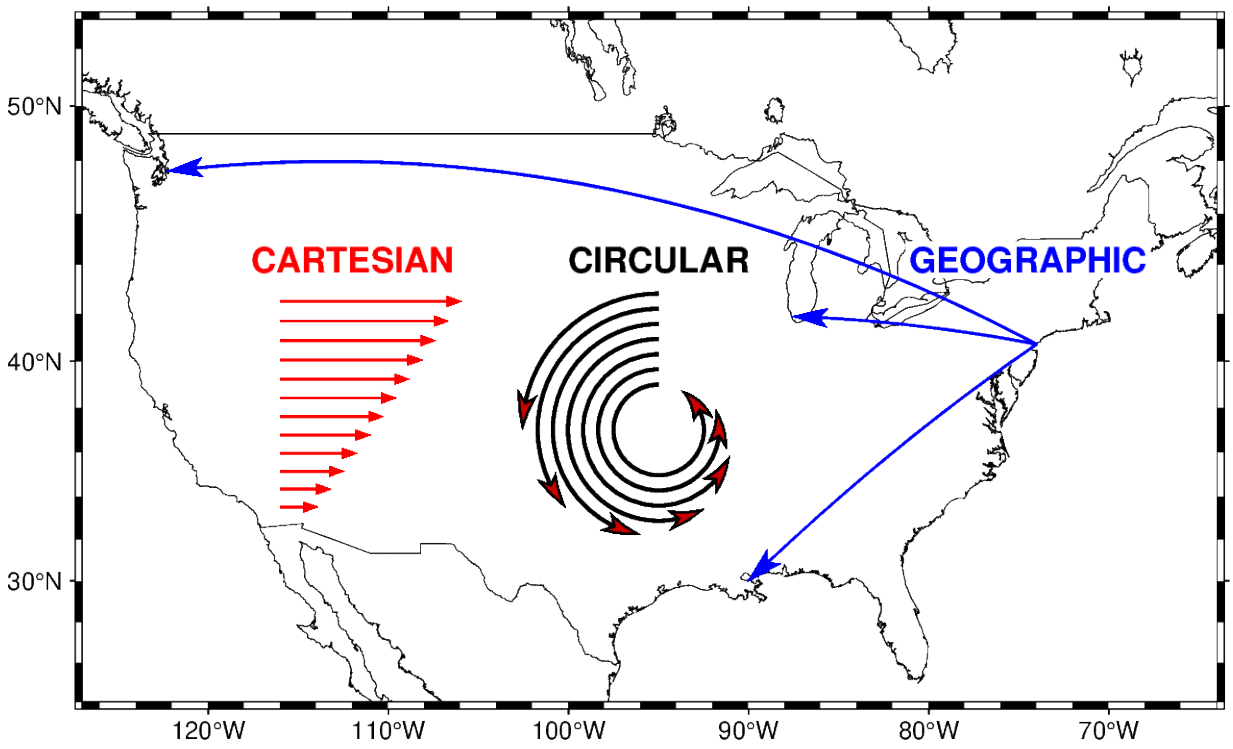

```
/
↪home/runner/micromamba/envs/pygmt/lib/python3.15/site-packages/contextily/tile.py:298:␣
↪UserWarning: CartoDB tiles now require an API key. Please provide one to continue␣
↪using the tiles. You can request the key at https://carto.com/basemaps/apikey/.
  tile_urls = [provider.build_url(x=tile.x, y=tile.y, z=tile.z) for tile in tiles]
```

Total running time of the script: (0 minutes 1.541 seconds)

8.2 Lines and vectors

8.2.1 Cartesian, circular, and geographic vectors

The *pygmt.Figure.plot* method can plot Cartesian, circular, and geographic vectors. The `style` parameter controls vector attributes. See also the *Vector attributes example*.




```
import numpy as np
import pygmt

# Create a plot with coast, Mercator projection (M) over the continental US
fig = pygmt.Figure()
fig.coast(
    region=[-127, -64, 24, 53],
    projection="M15c",
    frame=True,
    borders=1,
    area_thresh=4000,
    shorelines="0.25p,black",
)


# Left: plot 12 Cartesian vectors with different lengths
x = np.linspace(-116, -116, 12)  # x-vector coordinates
y = np.linspace(33.5, 42.5, 12)  # y-vector coordinates
direction = np.zeros(x.shape)  # direction of vectors
length = np.linspace(0.5, 2.4, 12)  # length of vectors
# Cartesian vectors (v) with red fill and pen (+g, +p), vector head at the end (+e),␣
↪and
# 40 degree angle (+a) with no indentation for the vector head (+h)
style = "v0.2c+e+a40+gred+h0+p1p,red"
fig.plot(x=x, y=y, style=style, pen="1p,red", direction=[direction, length])
fig.text(text="CARTESIAN", x=-112, y=44.2, font="13p,Helvetica-Bold,red", fill="white
↪")
```

(continues on next page)

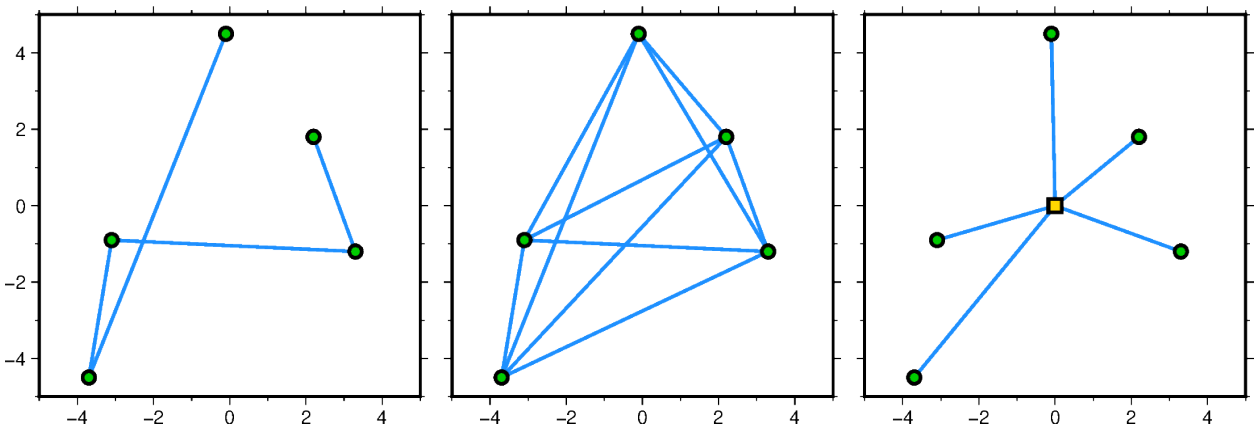
(continued from previous page)

```
# Middle: plot 7 math angle arcs with different radii
num = 7
x = np.full(num, -95)  # x-coordinates of the center
y = np.full(num, 37)  # y-coordinates of the center
radius = 1.8 - 0.2 * np.arange(0, num)  # radius
startdir = np.full(num, 90)  # start direction in degrees
stopdir = 180 + 40 * np.arange(0, num)  # stop direction in degrees
# data for circular vectors
data = np.column_stack([x, y, radius, startdir, stopdir])
arcstyle = "m0.5c+ea"  # Circular vector (m) with an arrow at the end
fig.plot(data=data, style=arcstyle, fill="red3", pen="1.5p,black")
fig.text(text="CIRCULAR", x=-95, y=44.2, font="13p,Helvetica-Bold,black", fill="white
↪")


# Right: plot geographic vectors using endpoints
NYC = [-74.0060, 40.7128]
CHI = [-87.6298, 41.8781]
SEA = [-122.3321, 47.6062]
NO = [-90.0715, 29.9511]
# '=' means geographic vectors. With the modifier '+s', the input data should contain
# coordinates of start and end points
style = "=0.5c+s+e+a30+gblue+h0.5+p1p,blue"
data = np.array([NYC + CHI, NYC + SEA, NYC + NO])
fig.plot(data=data, style=style, pen="1.0p,blue")
fig.text(
    text="GEOGRAPHIC", x=-74.5, y=44.2, font="13p,Helvetica-Bold,blue", fill="white"
)
fig.show()
```

Total running time of the script: (0 minutes 0.265 seconds)

8.2.2 Connection lines

The `connection` parameter of the *pygmt.Figure.plot* method allows to plot connection lines between a set of data points. Width, color, and style of the lines can be adjusted via the `pen` parameter. The data points must be plotted separately using the `style` parameter, with adjustments for the symbol fill and outline via the `fill` and `pen` parameters, respectively.



```
import pygmt
from pygmt.params import Axis, Frame

# Set up same sample data
x = [2.2, 3.3, -3.1, -3.7, -0.1]
y = [1.8, -1.2, -0.9, -4.5, 4.5]

# Create new Figure instance
fig = pygmt.Figure()

# ------------------------------------------------------------------------
# Left: record order
fig.basemap(
    region=[-5, 5, -5, 5],
    projection="X6c",
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
)

# Connect data points based on the record order [Default connection=None]
fig.plot(x=x, y=y, pen="1.5p,dodgerblue")
# Plot data points
fig.plot(x=x, y=y, style="c0.2c", fill="green3", pen="1.5p")

fig.shift_origin(xshift="w+0.5c")

# ------------------------------------------------------------------------
# Middle: network
fig.basemap(
    region=[-5, 5, -5, 5],
    projection="X6c",
    frame=Frame(axes="wSne", axis=Axis(annot=True, tick=True)),
)

# Connect data points as network
fig.plot(x=x, y=y, pen="1.5p,dodgerblue", connection="n")
# Plot data points
fig.plot(x=x, y=y, style="c0.2c", fill="green3", pen="1.5p")

fig.shift_origin(xshift="w+0.5c")

# ------------------------------------------------------------------------
# Right: reference point
fig.basemap(
    region=[-5, 5, -5, 5],
    projection="X6c",
    frame=Frame(axes="wSne", axis=Axis(annot=True, tick=True)),
)

# Connect data points with the reference point (0,0)
fig.plot(x=x, y=y, pen="1.5p,dodgerblue", connection="p0/0")
# Plot data points
fig.plot(x=x, y=y, style="c0.2c", fill="green3", pen="1.5p")
# Plot reference point
```

(continues on next page)

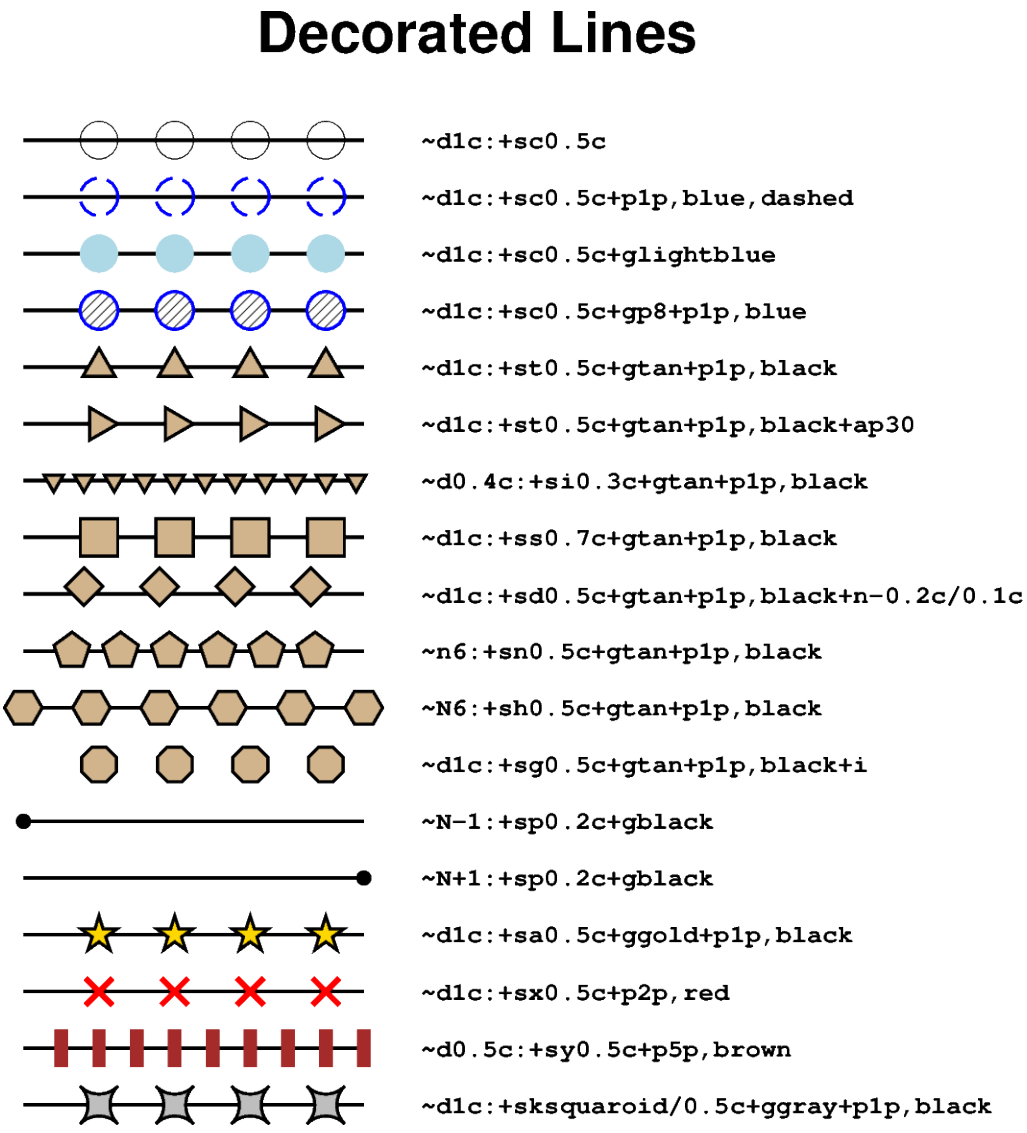
(continued from previous page)

```
fig.plot(x=0, y=0, style="s0.3c", fill="gold", pen="1.5p")

fig.show()
```

Total running time of the script: (0 minutes 0.131 seconds)

8.2.3 Decorated lines

To draw a so-called *decorated line*, i.e., symbols along a line or curve, use the `style` parameter of the *`pygmt.Figure.plot`* method with the argument `"~"` and the desired modifiers. A colon (`":"`) is used to separate the algorithm settings from the symbol information. This example shows how to adjust the symbols. Beside the built-in symbols also custom symbols can be used. For modifying the main decorated line via the `pen` parameter, see the *Line styles example*. For details on the input data see the upstream GMT documentation at https://docs.generic-mapping-tools.org/6.7/plot.html#s. Furthermore, there are so-called *line fronts*, which are often used to plot fault lines, subduction zones, or weather fronts; for details see the *Line fronts example*.



```
import numpy as np
import pygmt
from pygmt.params import Frame

# Generate a two-point line for plotting
x = np.array([1, 4])
y = np.array([24, 24])

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 24], projection="X15c", frame=Frame(title="Decorated Lines")
)

# Plot different decorated lines
for decoline in [
    # Line with circles ("c") of 0.5 centimeters radius in distance of 1 centimeter
    "~d1c:+sc0.5c",
    # Adjust thickness, color, and style of the outline via "+p"
    # Here, we plot a 1-point thick, blue, dashed outline
    "~d1c:+sc0.5c+p1p,blue,dashed",
    # Add a fill color using "+g" with the desired color
    "~d1c:+sc0.5c+glightblue",
    # To use a pattern as fill append "p" and give the pattern number
    "~d1c:+sc0.5c+gp8+p1p,blue",
    # Line with triangles ("t")
    "~d1c:+st0.5c+gtan+p1p,black",
    # Rotate counter-clockwise from line-parallel ("+ap") by 30 degrees
    "~d1c:+st0.5c+gtan+p1p,black+ap30",
    # Line with inverse triangles with a size of 0.3 centimeters in a
    # distance of 0.4 centimeters
    "~d0.4c:+si0.3c+gtan+p1p,black",
    # Line with squares ("s") with a size of 0.7 centimeters in a distance of
    # 1 centimeter
    "~d1c:+ss0.7c+gtan+p1p,black",
    # Shift symbols using "+n" in x- and y-directions relative to the main decorated
    # line
    "~d1c:+sd0.5c+gtan+p1p,black+n-0.2c/0.1c",
    # Give the number of equally spaced symbols by using "n" instead of "d"
    "~n6:+sn0.5c+gtan+p1p,black",
    # Use uppercase "N" to have symbols at the start and end of the line
    "~N6:+sh0.5c+gtan+p1p,black",
    # Suppress the main decorated line by appending "+i"
    "~d1c:+sg0.5c+gtan+p1p,black+i",
    # To only plot a symbol at the start of the line use "N-1"
    "~N-1:+sp0.2c+gblack",
    # To only plot a symbol at the end of the line use "N+1"
    "~N+1:+sp0.2c+gblack",
    # Line with stars ("a")
    "~d1c:+sa0.5c+ggold+p1p,black",
    # Line with crosses ("x")
    "~d1c:+sx0.5c+p2p,red",
    # Line with (vertical) lines or bars ("y")
    "~d0.5c:+sy0.5c+p5p,brown",
```

(continues on next page)

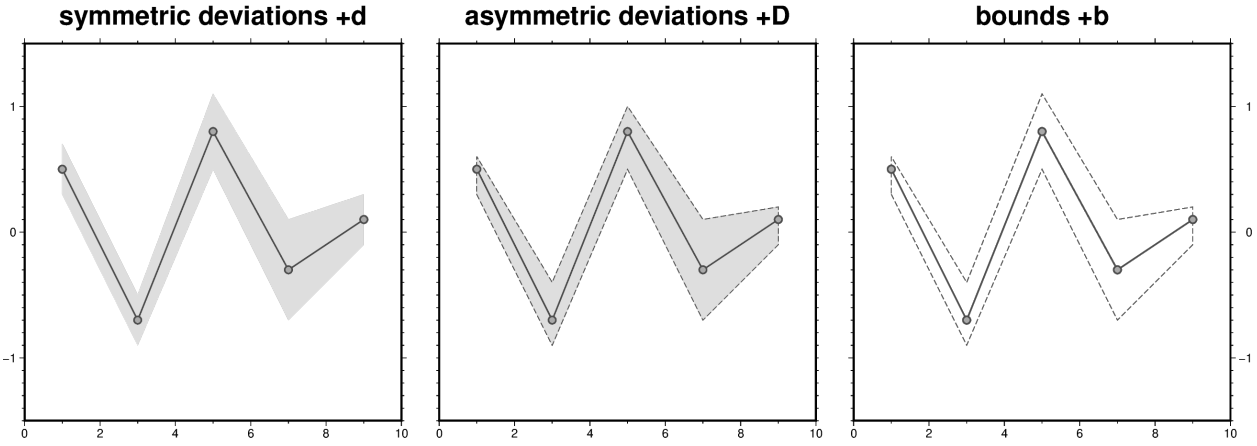
(continued from previous page)

```
    # Use custom symbol ("k") "squaroid" with a size of 0.5 centimeters
    "~d1c:+sksquaroid/0.5c+ggray+p1p,black",
]:
    y = y - 1.2  # Move current line down
    fig.plot(x=x, y=y, style=decoline, pen="1.25p,black")
    fig.text(
        x=x[-1],
        y=y[-1],
        text=decoline,
        font="Courier-Bold",
        justify="ML",
        offset=(0.75, 0),
    )

fig.show()
```

Total running time of the script: (0 minutes 0.267 seconds)

8.2.4 Envelope

The `close` parameter of the *pygmt.Figure.plot* method can be used to build a symmetrical or an asymmetrical envelope. The user can give either the deviations or the bounds in y-direction. For the first case append `"+d"` or `"+D"` and for the latter case `"+b"`.




```
import pandas as pd
import pygmt
from pygmt.params import Axis, Frame

# Define a pandas.DataFrame with columns for x and y as well as the lower and upper
# deviations
df_devi = pd.DataFrame(
    data={
        "x": [1, 3, 5, 7, 9],
        "y": [0.5, -0.7, 0.8, -0.3, 0.1],
        "y_deviation_low": [0.2, 0.2, 0.3, 0.4, 0.2],
        "y_deviation_upp": [0.1, 0.3, 0.2, 0.4, 0.1],
    }
)
```

(continues on next page)

(continued from previous page)

```
# Define the same pandas.DataFrame but with lower and upper bounds
df_bound = pd.DataFrame(
    data={
        "x": [1, 3, 5, 7, 9],
        "y": [0.5, -0.7, 0.8, -0.3, 0.1],
        "y_bound_low": [0.3, -0.9, 0.5, -0.7, -0.1],
        "y_bound_upp": [0.6, -0.4, 1.1, 0.1, 0.2],
    }
)


fig = pygmt.Figure()

# --------------------------------------------------------------------------
# Left
fig.basemap(
    region=[0, 10, -1.5, 1.5],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        title="symmetric deviations +d",
        xaxis=Axis(annot=2, tick=1),
        yaxis=Axis(annot=1, tick=0.1),
    ),
)

# Plot a symmetrical envelope based on the deviations ("+d")
fig.plot(
    data=df_devi,
    close="+d",
    # Fill the envelope in gray color with a transparency of 50 %
    fill="gray@50",
    pen="1p,gray30",
)

# Plot the data points on top
fig.plot(data=df_devi, style="c0.2c", pen="1p,gray30", fill="darkgray")

# Shift plot origin by the previous plot's width ("w") plus 1 centimeter in␣
↪x-direction
fig.shift_origin(xshift="w+1c")

# --------------------------------------------------------------------------
# Middle
fig.basemap(
    region=[0, 10, -1.5, 1.5],
    projection="X10c",
    frame=Frame(
        axes="WSne",
        title="asymmetric deviations +D",
        xaxis=Axis(annot=2, tick=1),
```

(continues on next page)

(continued from previous page)

```
        yaxis=Axis(tick=0.1),
    ),
)

# Plot an asymmetrical envelope based on the deviations ("+D")
fig.plot(
    data=df_devi,
    fill="gray@50",
    # Add an outline around the envelope. Here, a dashed pen ("+p") with 0.5-points
    # thickness and "gray30" color is used
    close="+D+p0.5p,gray30,dashed",
    pen="1p,gray30",
)

fig.plot(data=df_devi, style="c0.2c", pen="1p,gray30", fill="darkgray")

fig.shift_origin(xshift="w+1c")

# ------------------------------------------------------------------------------
# Right
fig.basemap(
    region=[0, 10, -1.5, 1.5],
    projection="X10c",
    # Use "\\053" to handle "+b" as a string not as a modifier
    frame=Frame(
        axes="wSnE",
        title="bounds \\053b",
        xaxis=Axis(annot=2, tick=1),
        yaxis=Axis(annot=1, tick=0.1),
    ),
)

# Plot an envelope based on the bounds ("+b")
fig.plot(data=df_bound, close="+b+p0.5p,gray30,dashed", pen="1p,gray30")

fig.plot(data=df_bound, style="c0.2c", pen="1p,gray30", fill="darkgray")

fig.show()
```

Total running time of the script: (0 minutes 0.273 seconds)

8.2.5 Fill area between curves

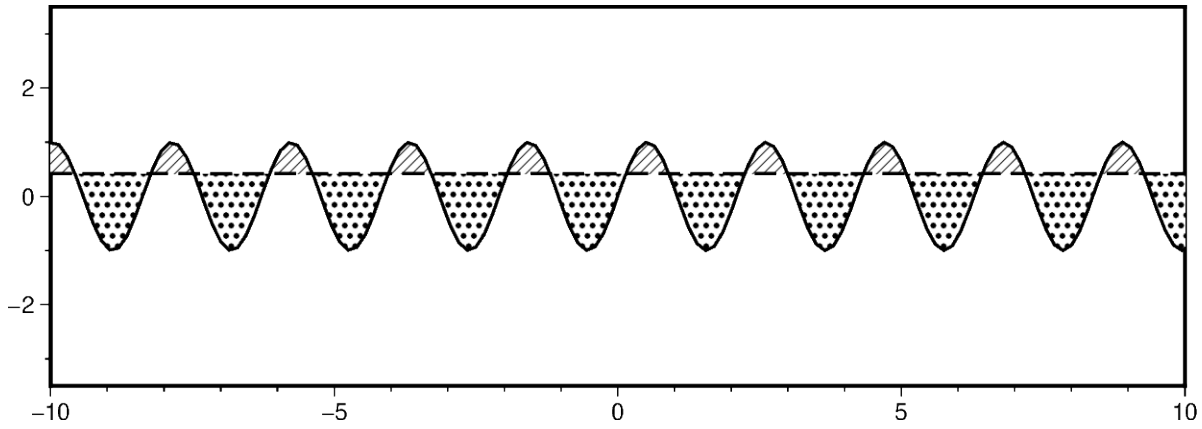
The *pygmt.Figure.fill_between* method fills the area between two curves y1 and y2. Different fills (colors or patterns) can be used for the areas y1 > y2 and y1 < y2. The two curves can be co-registered or have different x-coordinates. To plot an anomaly along a track use *pygmt.Figure.wiggle* and see the gallery example *Wiggle along tracks*.

For filling the areas between the two curves use the `fill` and `fill2` parameters to set different fills for the areas with y1 > y2 and y1 < y2, respectively. Use the `label` and `label2` parameters to set the corresponding legend entries. In addition to filling the areas, we can draw the outline curves. Use the `pen` and `pen2` parameters to set different lines for the two curves y1 and y2, respectively.

```
import numpy as np
import pygmt

# Generate some test data
x = np.arange(-10, 10.2, 0.15)
y1 = np.sin(3 * x)
y2 = np.sin(x / 2)

x3 = np.arange(-10, 10.4, 0.5) - 0.13
y3 = np.sin(x3 / 2)
```

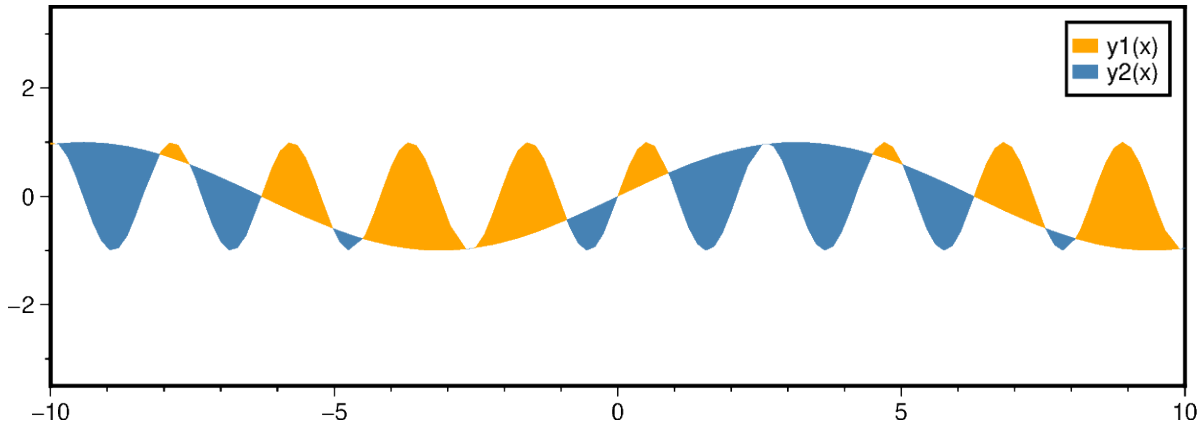
To compare a curve y1(x) to a horizontal line, pass the desired y-level to `y2`.

```
fig = pygmt.Figure()
fig.basemap(region=[-10, 10, -3.5, 3.5], projection="X15c/5c", frame=True)

fig.fill_between(
    x=x,
    y=y1,
    y2=0.42,
    fill="p8",
    fill2="p17",
    pen="1p,black,solid",
    pen2="1p,black,dashed",
)
fig.show()
```



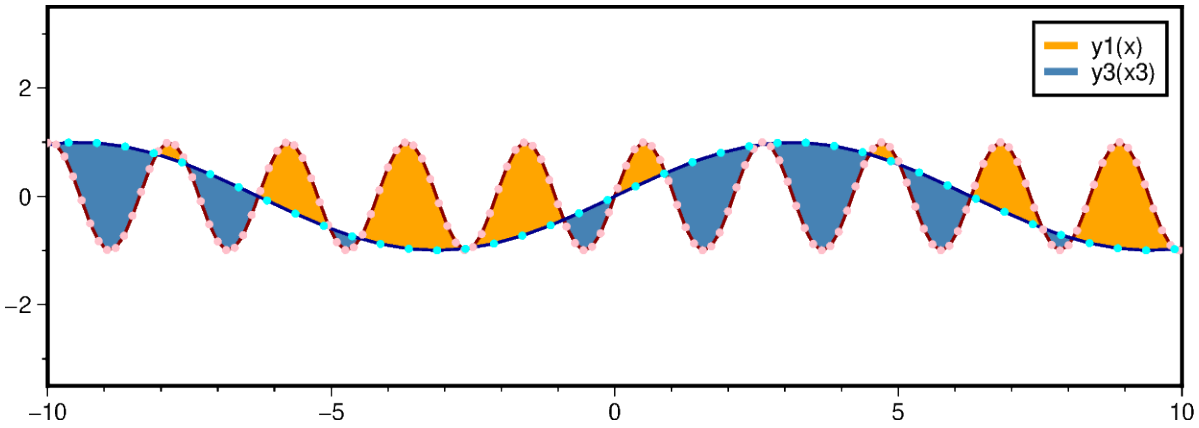
To compare two co-registered curves y1(x) and y2(x) pass a sequence with the same length as the inputs for `x` and `y` to `y2`.

```
fig = pygmt.Figure()
fig.basemap(region=[-10, 10, -3.5, 3.5], projection="X15c/5c", frame=True)

fig.fill_between(
    x=x, y=y1, y2=y2, fill="orange", fill2="steelblue", label="y1(x)", label2="y2(x)"
)
fig.legend()
fig.show()
```



Now, we use two non-co-registered curves, e.g., the two curves have different x-coordinates. For providing the x-coordinates for the second curve, use parameter `x2`. Via the `legend_pen` parameter the appearance in the legend can be changed to draw the legend entries as colored lines (using the fill colors) instead of filled and outlined boxes.

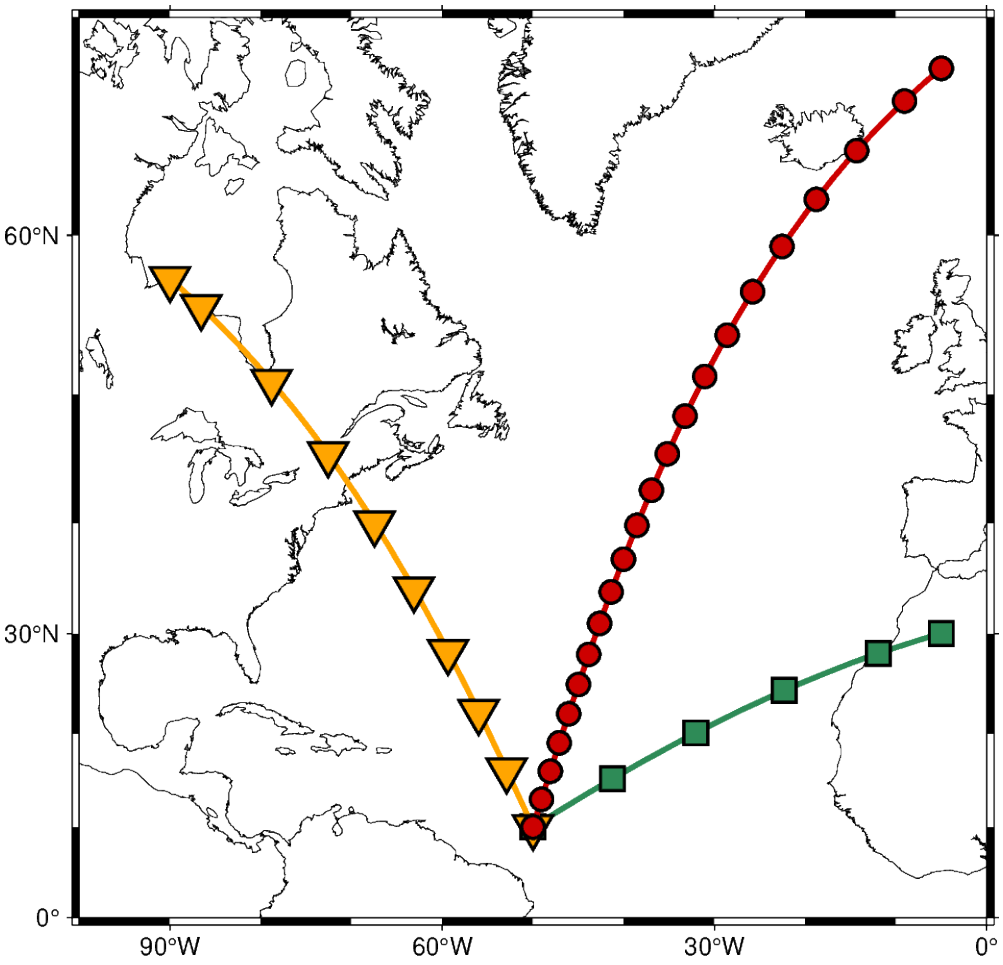
```
fig = pygmt.Figure()
fig.basemap(region=[-10, 10, -3.5, 3.5], projection="X15c/5c", frame=True)

fig.fill_between(
    x=x,
    y=y1,
    x2=x3,
    y2=y3,
    fill="orange",
    fill2="steelblue",
    pen="1p,darkred,solid",
    pen2="1p,darkblue,solid",
    label="y1(x)",
    label2="y3(x3)",
    legend_pen=True,
)

# Mark sampling points
fig.plot(x=x, y=y1, style="c0.1c", fill="pink")
fig.plot(x=x3, y=y3, style="c0.1c", fill="cyan")

fig.legend()
fig.show()
```



Total running time of the script: (0 minutes 0.317 seconds)

8.2.6 Generate points along great circles

The *pygmt.project* function can generate points along a great circle whose center and end points can be defined via the `center` and `endpoint` parameters, respectively. Using the `generate` parameter allows to generate (*r*, *s*, *p*) points every *dist* units of *p* along a profile as output. By default, all units (*r*, *s* and *p*) are set to degrees while `unit=True` allows to set the unit for *p* to km.

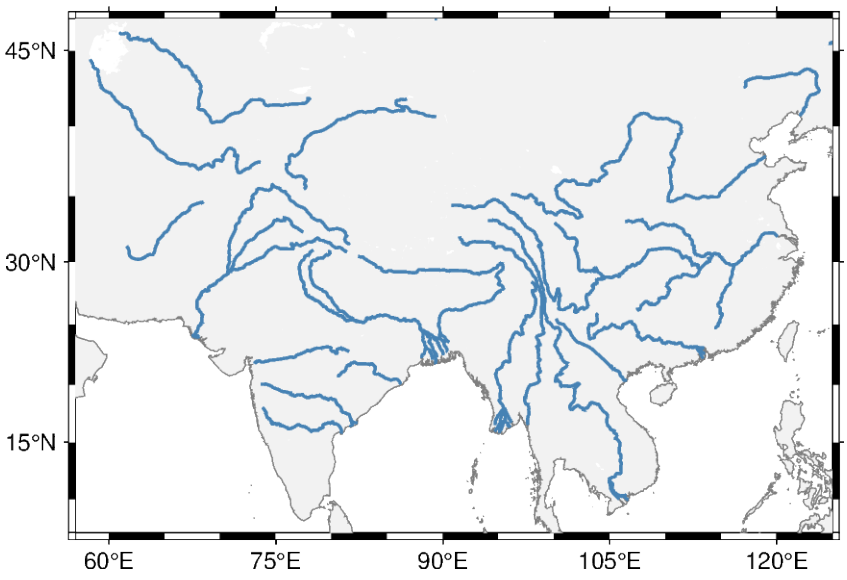


```
import pygmt

fig = pygmt.Figure()

# generate points every 10 degrees along a great circle from 10N,50W to 30N,5W
points1 = pygmt.project(center=[-50, 10], endpoint=[-5, 30], generate=10)
# generate points every 750 km along a great circle from 10N,50W to 57.5N,90W
points2 = pygmt.project(center=[-50, 10], endpoint=[-90, 57.5], generate=750,␣
↪unit=True)
# generate points every 350 km along a great circle from 10N,50W to 68N,5W
points3 = pygmt.project(center=[-50, 10], endpoint=[-5, 68], generate=350, unit=True)

# create a plot with coast and Mercator projection (M)
fig.basemap(region=[-100, 0, 0, 70], projection="M12c", frame=True)
fig.coast(shorelines=True, area_thresh=5000)

# plot individual points of first great circle as seagreen line
fig.plot(x=points1.r, y=points1.s, pen="2p,seagreen")
# plot individual points as seagreen squares atop
fig.plot(x=points1.r, y=points1.s, style="s.45c", fill="seagreen", pen="1p")

# plot individual points of second great circle as orange line
fig.plot(x=points2.r, y=points2.s, pen="2p,orange")
# plot individual points as orange inverted triangles atop
fig.plot(x=points2.r, y=points2.s, style="i.6c", fill="orange", pen="1p")

# plot individual points of third great circle as red3 line
fig.plot(x=points3.r, y=points3.s, pen="2p,red3")
# plot individual points as red3 circles atop
fig.plot(x=points3.r, y=points3.s, style="c.3c", fill="red3", pen="1p")

fig.show()
```

Total running time of the script: (0 minutes 0.173 seconds)

8.2.7 GeoPandas: Plotting lines with LineString or MultiLineString geometry

The *pygmt.Figure.plot* method allows us to plot geographical data such as lines with LineString or MultiLineString geometry types stored in a `geopandas.GeoDataFrame` object. Use `geopandas.read_file` to load data from any supported OGR format such as a shapefile (.shp), GeoJSON (.geojson), geopackage (.gpkg), etc. Then, pass the `geopandas.GeoDataFrame` object as an argument to the `data` parameter of *pygmt.Figure.plot*, and style the lines using the `pen` parameter.




```
import geopandas
import pygmt

# Read a sample dataset provided by Natural Earth. The dataset contains rivers stored
# as LineString/MultiLineString geometry types. Here will focus on Asia.
provider = "https://naciscdn.org/naturalearth"
rivers = geopandas.read_file(
    f"{provider}/50m/physical/ne_50m_rivers_lake_centerlines.zip"
)
rivers_asia = rivers.cx[57:125, 7:47].copy()

fig = pygmt.Figure()
fig.basemap(region=[57, 125, 7, 47], projection="M10c", frame=True)
fig.coast(land="gray95", shorelines="1/0.3p,gray50")

# Add rivers to map
fig.plot(data=rivers_asia, pen="1p,steelblue")

fig.show()
```

Total running time of the script: (0 minutes 0.279 seconds)

8.2.8 Horizontal and vertical lines

The *pygmt.Figure.hlines* and *pygmt.Figure.vlines* methods allow to plot horizontal and vertical lines in Cartesian, geographic and polar coordinate systems.

Cartesian coordinate system

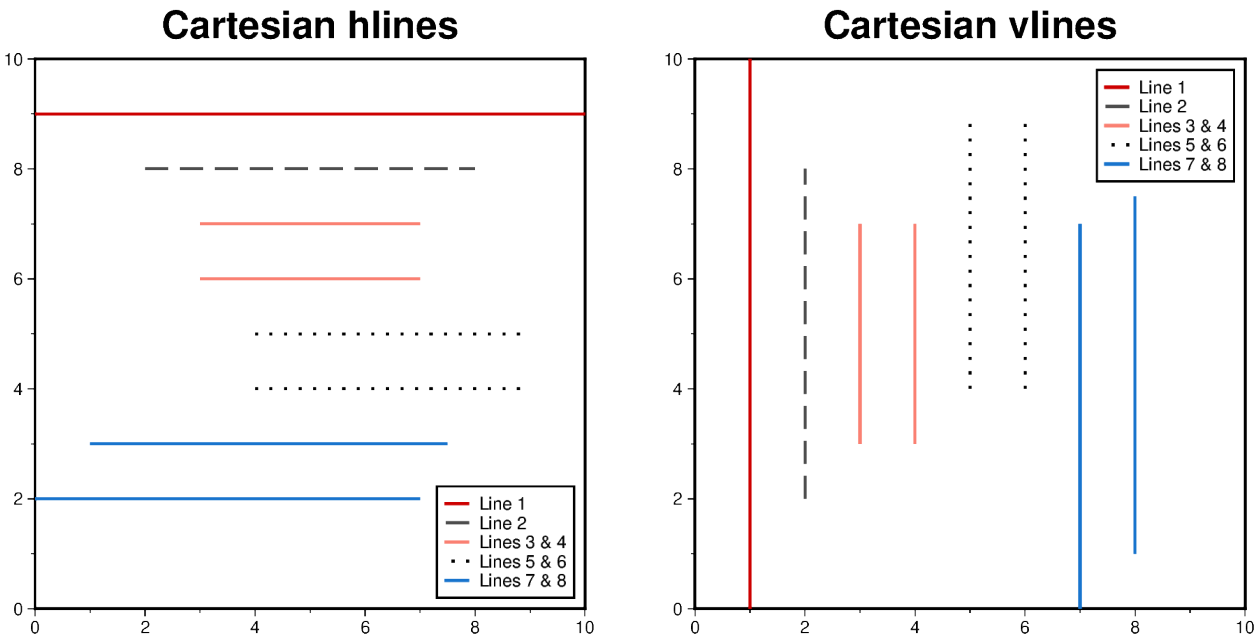
In Cartesian coordinate systems lines are plotted as straight lines.

```
import pygmt
from pygmt.params import Axis, Box, Frame, Position

fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

```
fig.basemap(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Frame(title="Cartesian hlines", axis=Axis(annot=True, tick=True)),
)

# Add a horizontal line at y=9
fig.hlines(y=9, pen="1.5p,red3", label="Line 1")
# Add a horizontal line at y=8 with x from 2 to 8
fig.hlines(y=8, xmin=2, xmax=8, pen="1.5p,gray30,-", label="Line 2")
# Add two horizontal lines at y=6 and y=7 both with x from 3 to 7
fig.hlines(y=[6, 7], xmin=3, xmax=7, pen="1.5p,salmon", label="Lines 3 & 4")
# Add two horizontal lines at y=4 and y=5 both with x from 4 to 9
fig.hlines(y=[4, 5], xmin=4, xmax=9, pen="1.5p,black,.", label="Lines 5 & 6")
# Add two horizontal lines at y=2 and y=3 with different x limits
fig.hlines(
    y=[2, 3], xmin=[0, 1], xmax=[7, 7.5], pen="1.5p,dodgerblue3", label="Lines 7 & 8"
)
fig.legend(position=Position("BR", offset=0.2), box=Box(pen="1p", fill="white"))

fig.shift_origin(xshift="w+2c")

fig.basemap(
    region=[0, 10, 0, 10],
    projection="X10c/10c",
    frame=Frame(title="Cartesian vlines", axis=Axis(annot=True, tick=True)),
)
# Add a vertical line at x=1
fig.vlines(x=1, pen="1.5p,red3", label="Line 1")
# Add a vertical line at x=2 with y from 2 to 8
fig.vlines(x=2, ymin=2, ymax=8, pen="1.5p,gray30,-", label="Line 2")
# Add two vertical lines at x=3 and x=4 both with y from 3 to 7
fig.vlines(x=[3, 4], ymin=3, ymax=7, pen="1.5p,salmon", label="Lines 3 & 4")
# Add two vertical lines at x=5 and x=6 both with y from 4 to 9
fig.vlines(x=[5, 6], ymin=4, ymax=9, pen="1.5p,black,.", label="Lines 5 & 6")
# Add two vertical lines at x=7 and x=8 with different y limits
fig.vlines(
    x=[7, 8], ymin=[0, 1], ymax=[7, 7.5], pen="1.5p,dodgerblue3", label="Lines 7 & 8"
)
fig.legend()

fig.show()
```



Geographic coordinate system

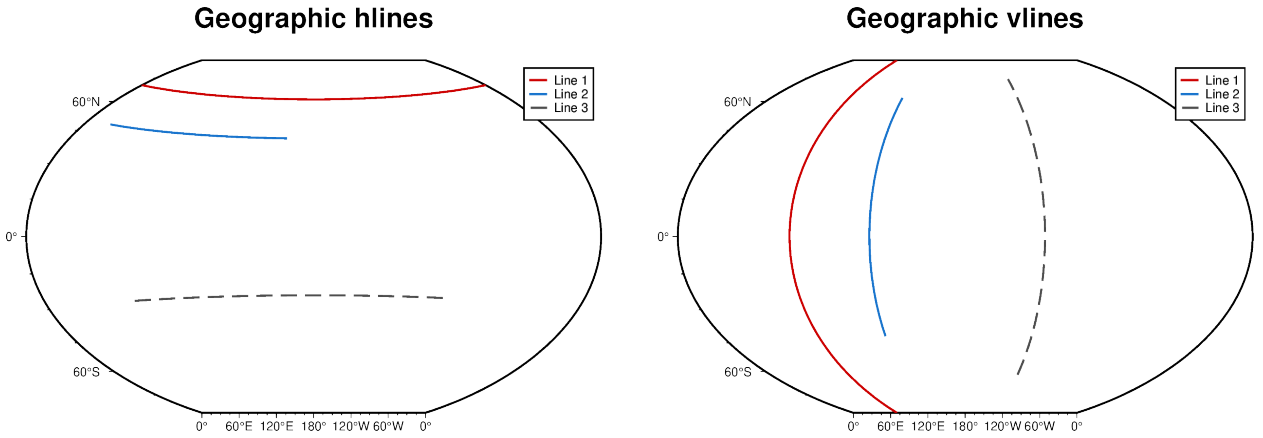
The same can be done in geographic coordinate systems where "horizontal" means lines are plotted along parallels (constant latitude) while "vertical" means lines are plotted along meridians (constant longitude).

```
fig = pygmt.Figure()

fig.basemap(
    region="g",
    projection="R15c",
    frame=Frame(title="Geographic hlines", axis=Axis(annot=True, tick=True)),
)
# Add a line at 70°N
fig.hlines(y=70, pen="1.5p,red3", label="Line 1")
# Add a line at 50°N with longitude limits at 20°E and 160°E
fig.hlines(y=50, xmin=20, xmax=160, pen="1.5p,dodgerblue3", label="Line 2")
# Add a line at 30°S with longitude limits at 60°E and 270°E
fig.hlines(y=-30, xmin=60, xmax=270, pen="1.5p,gray30,-", label="Line 3")
fig.legend()

fig.shift_origin(xshift="w+2c")

fig.basemap(
    region="g",
    projection="R15c",
    frame=Frame(title="Geographic vlines", axis=Axis(annot=True, tick=True)),
)
# Add a line at 70°E
fig.vlines(x=70, pen="1.5p,red3", label="Line 1")
# Add a line at 20°E with latitude limits at 50°S and 70°N
fig.vlines(x=120, ymin=-50, ymax=70, pen="1.5p,dodgerblue3", label="Line 2")
# Add a line at 230°E with latitude limits at 70°S and 80°N
fig.vlines(x=230, ymin=-70, ymax=80, pen="1.5p,gray30,-", label="Line 3")
```

(continues on next page)

(continued from previous page)

```
fig.legend()

fig.show()
```



Polar coordinate system

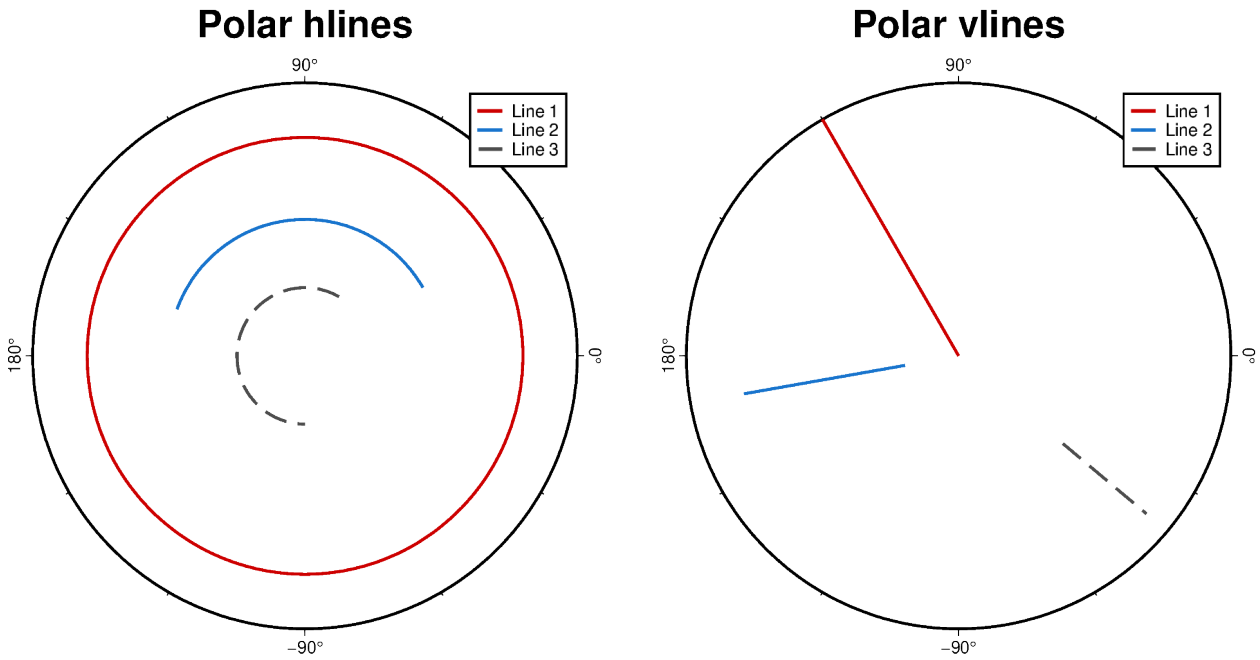
When using polar coordinate systems "horizontal" means lines are plotted as arcs along a constant radius while "vertical" means lines are plotted as straight lines along radius at a specified azimuth.

```
fig = pygmt.Figure()

fig.basemap(
    region=[0, 360, 0, 1],
    projection="P10c",
    frame=Frame(title="Polar hlines", axis=Axis(annot=True, tick=True)),
)
# Add a line along radius=0.8
fig.hlines(y=0.8, pen="1.5p,red3", label="Line 1")
# Add a line along radius=0.5 with azimuth limits at 30° and 160°
fig.hlines(y=0.5, xmin=30, xmax=160, pen="1.5p,dodgerblue3", label="Line 2")
# Add a line along radius=0.25 with azimuth limits at 60° and 270°
fig.hlines(y=0.25, xmin=60, xmax=270, pen="1.5p,gray30,-", label="Line 3")
fig.legend()

fig.shift_origin(xshift="w+2c")

fig.basemap(
    region=[0, 360, 0, 1],
    projection="P10c",
    frame=Frame(title="Polar vlines", axis=Axis(annot=True, tick=True)),
)
# Add a line along azimuth=120°
fig.vlines(x=120, pen="1.5p,red3", label="Line 1")
# Add a line along azimuth=190° with radius limits at 0.2 and 0.8
fig.vlines(x=190, ymin=0.2, ymax=0.8, pen="1.5p,dodgerblue3", label="Line 2")
# Add a line along azimuth=320 with radius limits at 0.5 and 0.9
fig.vlines(x=320, ymin=0.5, ymax=0.9, pen="1.5p,gray30,-", label="Line 3")
fig.legend()
```

(continues on next page)

(continued from previous page)

```
fig.show()
```

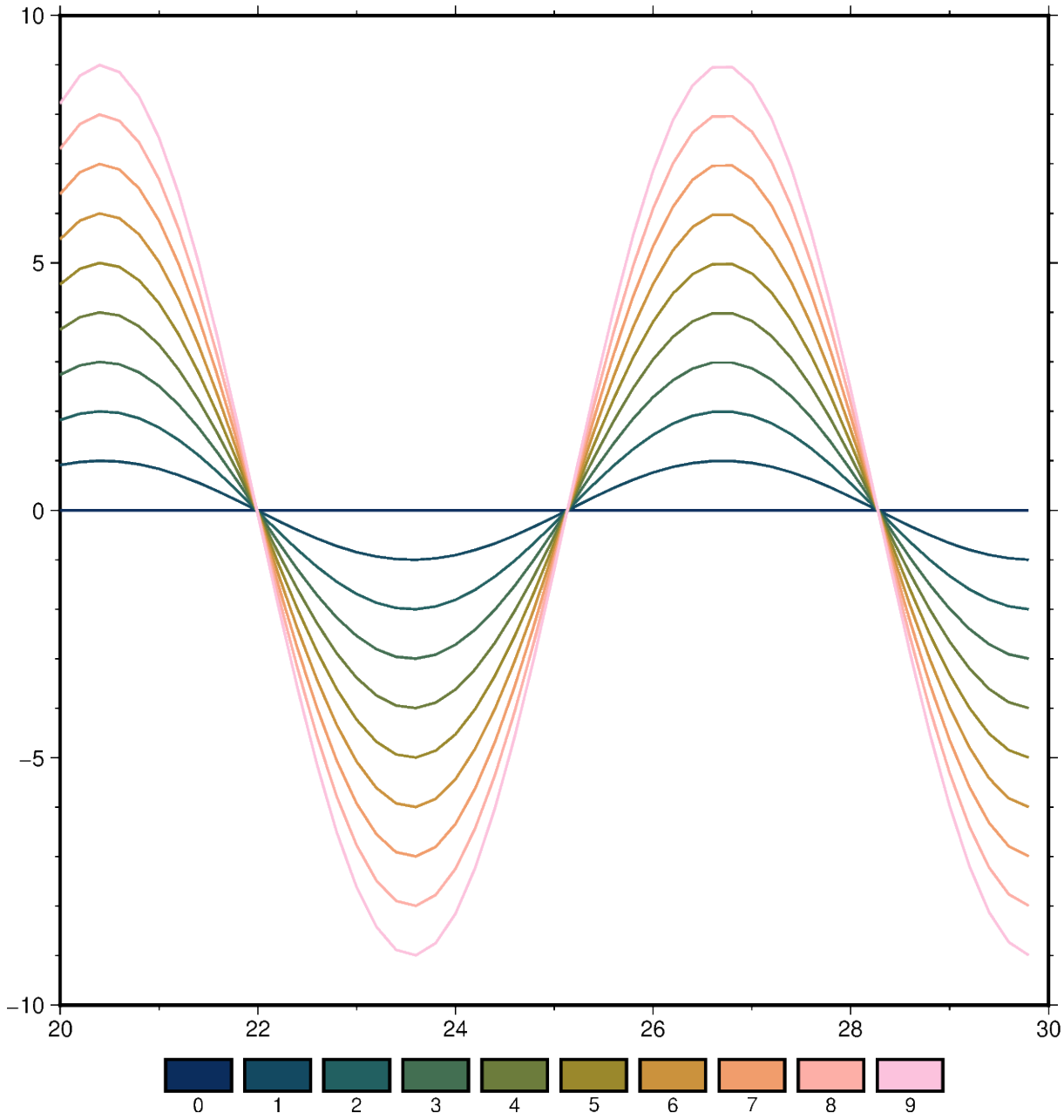


Total running time of the script: (0 minutes 0.671 seconds)

8.2.9 Line colors with a custom CPT

The color of the lines made by *pygmt.Figure.plot* can be set according to a custom CPT and assigned with the `pen` parameter.

The custom CPT can be used by setting the plot command's `cmap` parameter to `True`. The `zvalue` parameter sets the z-value (color) to be used from the custom CPT, and the line color is set as the z-value by using **+z** when setting the `pen` color.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# Create a list of values between 20 and 30 with 0.2 intervals
x = np.arange(start=20, stop=30, step=0.2)

fig = pygmt.Figure()
fig.basemap(
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)), region=[20, 30, -10,␣
↪10]
)
```

(continues on next page)

(continued from previous page)

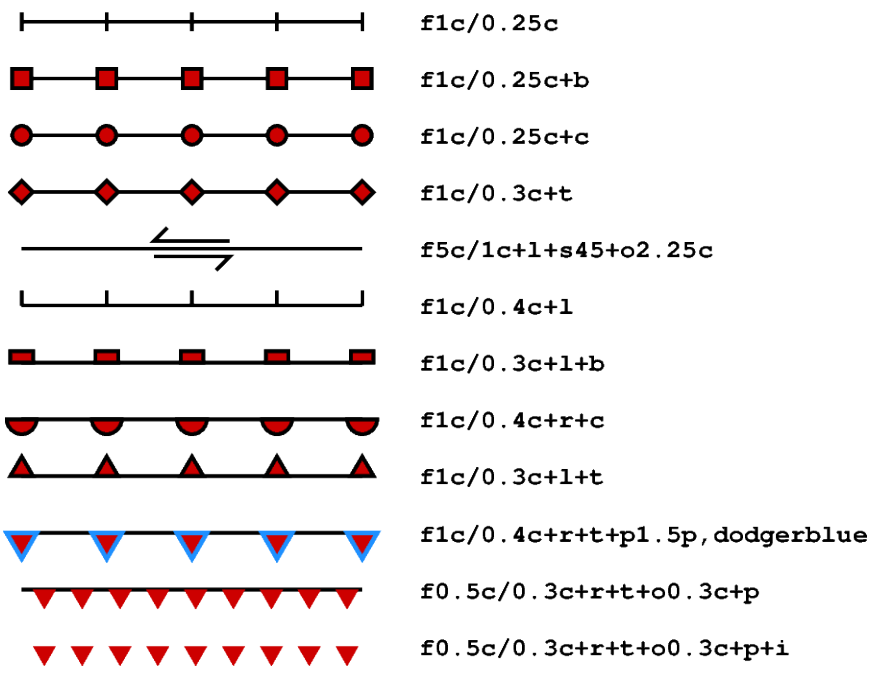
```
# Create a custom CPT with the batlow CPT and 10 discrete z-values (colors),
# use color_model="+c0-9" to write the color palette in categorical format and
# add labels (0) to (9) for the colorbar legend
pygmt.makecpt(cmap="SCM/batlow", series=[0, 9, 1], color_model="+c0-9")

# Plot 10 lines and set a different z-value for each line
for zvalue in range(10):
    y = zvalue * np.sin(x)
    fig.plot(x=x, y=y, cmap=True, zvalue=zvalue, pen="thick,+z,-")

# Color bar to show the custom CPT and the associated z-values
fig.colorbar()
fig.show()
```

Total running time of the script: (0 minutes 0.199 seconds)

8.2.10 Line fronts

Using the *pygmt.Figure.plot* method you can draw a so-called *front* which allows to plot specific symbols distributed along a line or curve. Typical use cases are weather fronts, fault lines, subduction zones, and more. To draw general symbols along a line or curve, i.e., a so-called *decorated line*, see the *Decorated lines example*.

A front can be drawn by passing **f**[±]*gap*[/*size*] to the `style` parameter where *gap* defines the distance gap between the symbols and *size* the symbol size. If *gap* is negative, it is interpreted to mean the number of symbols along the front instead. If *gap* has a leading + then we use the value exactly as given [Default will start and end each line with a symbol, hence the *gap* is adjusted to fit]. If *size* is missing it is set to 30% of the *gap*, except when *gap* is negative and *size* is thus required. Append **+l** or **+r** to plot symbols on the left or right side of the front [Default is centered]. Append **+***type* to specify which symbol to plot: **b**ox, **c**ircle, **f**ault [Default], **s**lip, or **t**riangle. Slip means left-lateral or right-lateral strike-slip arrows (centered is not an option). The **+s** modifier optionally accepts the angle used to draw the vector [Default is 20 degrees]. Alternatively, use **+S** which draws arcuate arrow heads. Append **+o***offset* to offset the first symbol from the beginning of the front by that amount [Default is 0]. The chosen symbol is drawn with the same pen as set for the line (i.e., via the `pen` parameter). To use an alternate pen, append **+p***pen*. To skip the outline, just use **+p** with no argument. To make the main front line invisible, add **+i**. For modifying the main front line via the `pen` parameter, see the *Line styles example*.




```
import numpy as np
import pygmt
from pygmt.params import Frame

# Generate a two-point line for plotting
x = np.array([1, 4])
y = np.array([20, 20])

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 20], projection="X15c/15c", frame=Frame(title="Line Fronts")
)

# Plot the line using different front styles
for frontstyle in [
    # line with "faults" front style, same as +f [Default]
    "f1c/0.25c",
    # line with box front style
    "f1c/0.25c+b",
    # line with circle front style
    "f1c/0.25c+c",
    # line with triangle front style
    "f1c/0.3c+t",
    # line with left-lateral ("+l") slip ("+s") front style, angle is set to
    # 45 degrees and offset to 2.25 cm
    "f5c/1c+l+s45+o2.25c",
    # line with "faults" front style, symbols are plotted on the left side of
    # the front
```

(continues on next page)

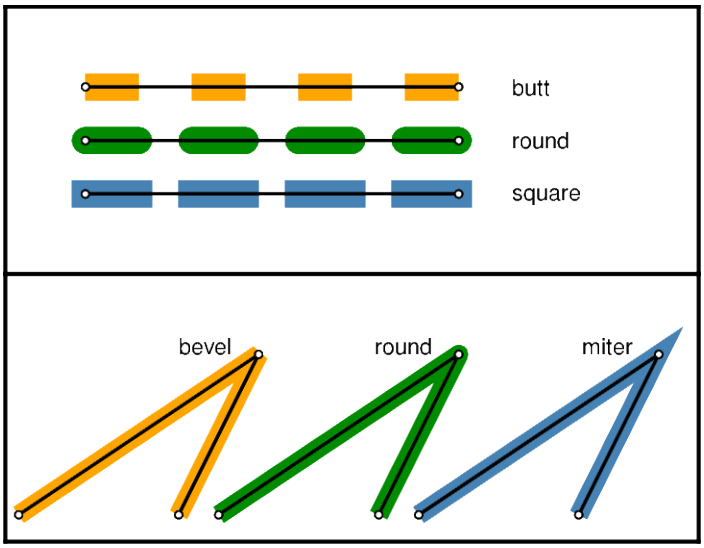
(continued from previous page)

```
    "f1c/0.4c+l",
    # line with box front style, symbols are plotted on the left side of the
    # front
    "f1c/0.3c+l+b",
    # line with circle front style, symbols are plotted on the right side of
    # the front
    "f1c/0.4c+r+c",
    # line with triangle front style, symbols are plotted on the left side of
    # the front
    "f1c/0.3c+l+t",
    # line with triangle front style, symbols are plotted on the right side of
    # the front, use other pen for the outline of the symbol
    "f1c/0.4c+r+t+p1.5p,dodgerblue",
    # line with triangle front style, symbols are plotted on the right side of
    # the front and offset is set to 0.3 cm, skip the outline
    "f0.5c/0.3c+r+t+o0.3c+p",
    # line with triangle front style, symbols are plotted on the right side of
    # the front and offset is set to 0.3 cm, skip the outline and make the main
    # front line invisible
    "f0.5c/0.3c+r+t+o0.3c+p+i",
]:
    y -= 1  # move the current line down
    fig.plot(x=x, y=y, pen="1.25p", style=frontstyle, fill="red3")
    fig.text(
        x=x[-1],
        y=y[-1],
        text=frontstyle,
        font="Courier-Bold",
        justify="ML",
        offset=(0.75, 0),
    )

fig.show()
```

Total running time of the script: (0 minutes 0.182 seconds)

8.2.11 Line segment caps and joints

PyGMT offers different appearances of line segment caps and joints. The desired appearance can be set via the GMT default parameters PS_LINE_CAP (`"butt"`, `"round"`, or `"square"` [Default]) as well as PS_LINE_JOIN (`"bevel"`, `"round"`, and `"miter"` [Default]) and PS_MITER_LIMIT (limit on the angle at the mitered joint below which a bevel is applied).




```
import numpy as np
import pygmt
from pygmt.params import Frame

# Set up dictionary for colors
dict_col = {
    "round": "green4",
    "square": "steelblue",
    "butt": "orange",
    "miter": "steelblue",
    "bevel": "orange",
}

# Create new Figure instance
fig = pygmt.Figure()

# ------------------------------------------------------------------------------
# Top: PS_LINE_CAP

# Create sample data
x = np.array([30, 170])
y = np.array([70, 70])

fig.basemap(region=[0, 260, 0, 100], projection="x1p", frame=Frame(axes="rltb"))

for line_cap in ["butt", "round", "square"]:
    # Change GMT default locally
    with pygmt.config(PS_LINE_CAP=line_cap):
        color = dict_col[line_cap]
        # Draw a 10-point thick line with 20-point long segments and gaps
        # Use the local PS_LINE_CAP setting
        fig.plot(x=x, y=y, pen=f"10p,{color},20_20")

    # Draw a 1-point thick black solid line to highlight segment cap appearance
    fig.plot(x=x, y=y, pen="1p,black,solid")
```

(continues on next page)

(continued from previous page)

```
    # Plot data points as circles
    fig.plot(x=x, y=y, style="c0.1c", fill="white", pen="0.5p,")
    # Add label for PS_LINE_CAP setting
    fig.text(text=line_cap, x=x[-1] + 20, y=y[-1], justify="LM")

    y = y - 20

fig.shift_origin(yshift="-h")

# --------------------------------------------------------------------------
# Bottom: PS_LINE_JOIN and PS_MITER_LIMIT

x = np.array([5, 95, 65])
y = np.array([10, 70, 10])

fig.basemap(region=[0, 260, 0, 100], projection="x1p", frame=Frame(axes="rltb"))

for line_join in ["bevel", "round", "miter"]:
    with pygmt.config(PS_LINE_JOIN=line_join, PS_MITER_LIMIT=1):
        color = dict_col[line_join]
        # Draw a 7-point thick solid line
        # Use the local PS_LINE_JOIN and PS_MITER_LIMIT settings
        fig.plot(x=x, y=y, pen=f"7p,{color},solid")

    fig.plot(x=x, y=y, pen="1p,black,solid")
    fig.plot(x=x, y=y, style="c0.1c", fill="white", pen="0.5p")
    fig.text(text=line_join, x=x[1] - 10, y=y[1], justify="RB")

    x = x + 75

fig.show()
```

Total running time of the script: (0 minutes 0.208 seconds)

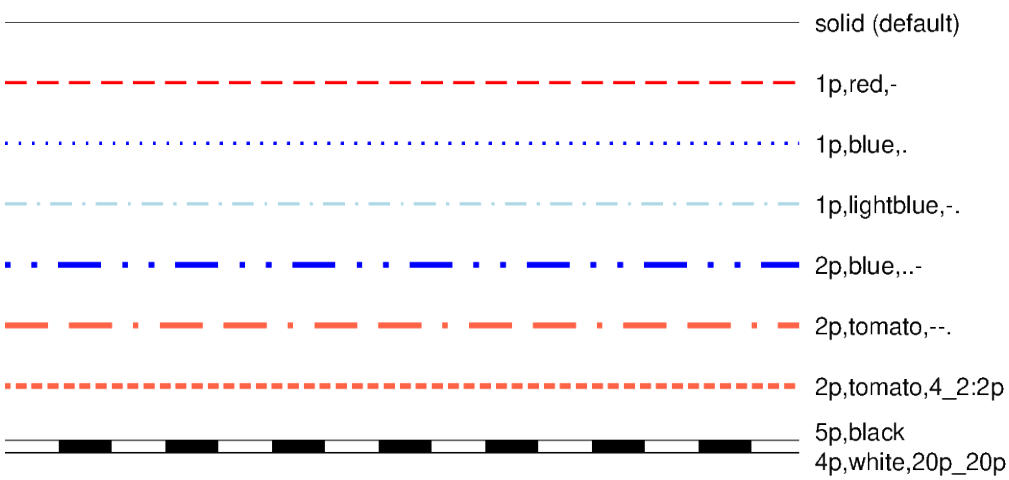
8.2.12 Line styles

The *pygmt.Figure.plot* method can plot lines in different styles. The default line style is a 0.25-point wide, black, solid line, and can be customized with the `pen` parameter.

A *pen* in GMT has three attributes: *width*, *color*, and *style*. The *style* attribute controls the appearance of the line. Giving `"dotted"` or `"."` yields a dotted line, whereas a dashed pen is requested with `"dashed"` or `"-"`. Also combinations of dots and dashes, like `".-"` for a dot-dashed line, are allowed.

For more advanced *pen* attributes, see the GMT Technical Reference https://docs.generic-mapping-tools.org/6.7/reference/features.html#wpen-attrib.

Line Styles




```
import numpy as np
import pygmt
from pygmt.params import Frame

# Generate a two-point line for plotting
x = np.array([0, 7])
y = np.array([9, 9])

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 10], projection="X15c/8c", frame=Frame(title="Line Styles")
)

# Plot the line using the default line style
fig.plot(x=x, y=y)
fig.text(x=x[-1], y=y[-1], text="solid (default)", justify="ML", offset=(0.2, 0))

# Plot the line using different line styles
for linestyle in [
    "1p,red,-",  # dashed line
    "1p,blue,.",  # dotted line
    "1p,lightblue,-.",  # dash-dotted line
    "2p,blue,..-",  # dot-dot-dashed line
    "2p,tomato,--.",  # dash-dash-dotted line
    # A pattern of 4-point-long line segments and 2-point-long gaps between
    # segments, with pattern offset by 2 points from the origin
    "2p,tomato,4_2:2p",
]:
    y -= 1  # Move the current line down
    fig.plot(x=x, y=y, pen=linestyle)
    fig.text(x=x[-1], y=y[-1], text=linestyle, justify="ML", offset=(0.2, 0))

# Plot the line like a railway track (black/white).
# The trick here is plotting the same line twice but with different line styles
```

(continues on next page)

(continued from previous page)

```
y -= 1  # move the current line down
fig.plot(x=x, y=y, pen="5p,black")
fig.plot(x=x, y=y, pen="4p,white,20p_20p")
fig.text(x=x[-1], y=y[-1], text="5p,black", justify="ML", offset=(0.2, 0.2))
fig.text(x=x[-1], y=y[-1], text="4p,white,20p_20p", justify="ML", offset=(0.2, -0.2))

fig.show()
```

Total running time of the script: (0 minutes 0.175 seconds)

8.2.13 Quoted lines

To plot a so-called *quoted line*, i.e., labels along a line or curve, use the `style` parameter of the *pygmt.Figure.plot* method with the argument `"q"` and the desired modifiers. A colon (`":"`) is used to separate the algorithm settings from the label information. This example shows how to adjust the labels. For modifying the main quoted line via the `pen` parameter, see the *Line styles example*. For details on the input data see the upstream GMT documentation at https://docs.generic-mapping-tools.org/6.7/plot.html#s.

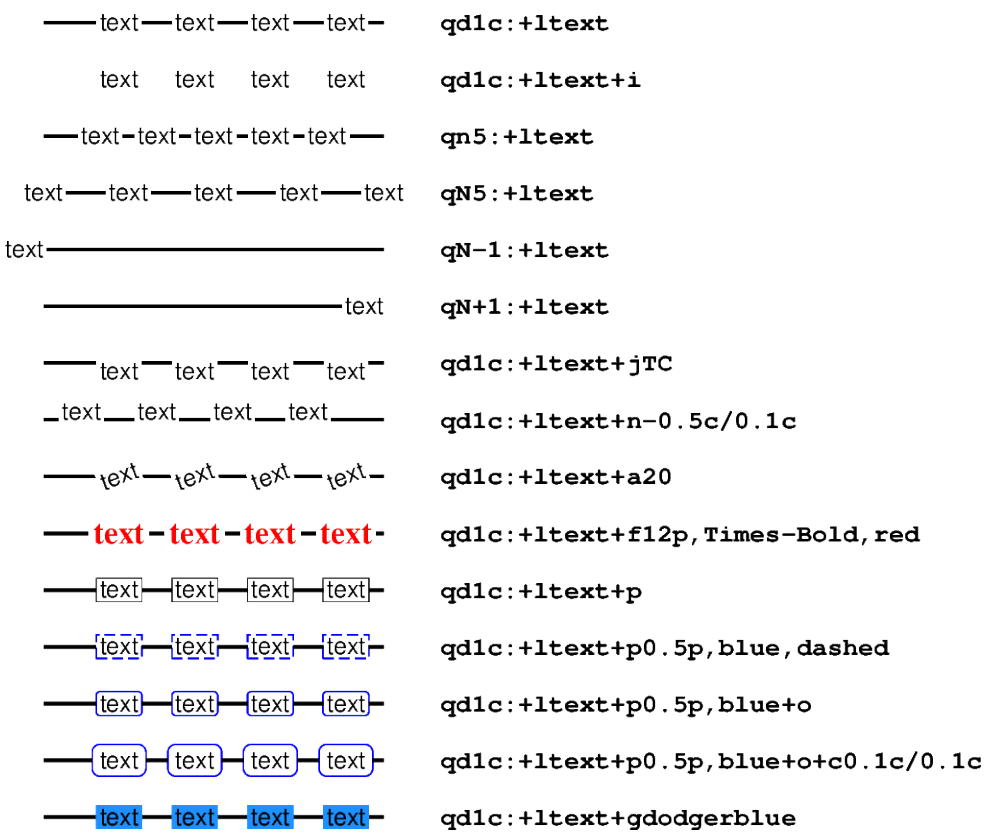
```
import numpy as np
import pygmt
from pygmt.params import Frame

# Generate a two-point line for plotting
x = np.array([1, 4])
y = np.array([20, 20])

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 20], projection="X15c/15c", frame=Frame(title="Quoted Lines")
)

# Plot different quoted lines
for quotedline in [
    # Line with labels ("+l") "text" in distance ("d") of 1 centimeter
    "qd1c:+ltext",
    # Suppress the main quoted line by appending "+i"
    "qd1c:+ltext+i",
    # Give the number of equally spaced labels by using "n" instead of "d"
    "qn5:+ltext",
    # Use uppercase "N" to have labels at the start and end of the line
    "qN5:+ltext",
    # To only plot a label at the start of the line use "N-1"
    "qN-1:+ltext",
    # To only plot a label at the end of the line use "N+1"
    "qN+1:+ltext",
    # Adjust the justification of the labels via "+j", here Top Center
    "qd1c:+ltext+jTC",
    # Shift labels using "+n" in x- and y-directions relative to the main
    # quoted line
    "qd1c:+ltext+n-0.5c/0.1c",
    # Rotate labels via "+a" (counter-clockwise from horizontal)
    "qd1c:+ltext+a20",
```

(continues on next page)

(continued from previous page)

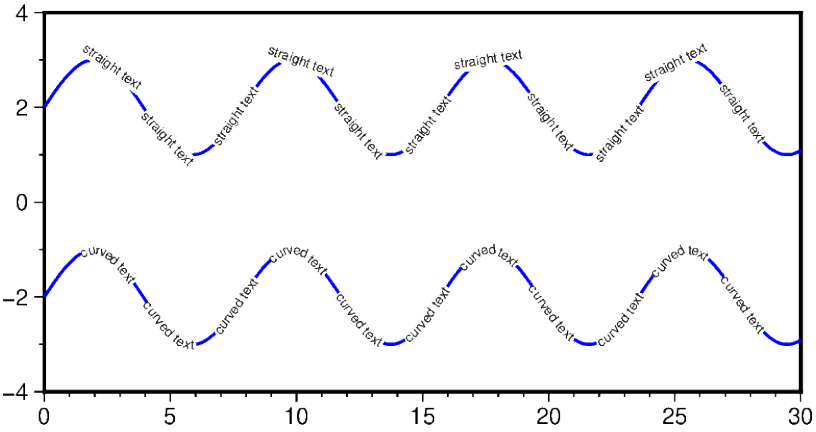
```
    # Adjust size, type, and color of the font via "+f"
    "qd1c:+ltext+f12p,Times-Bold,red",
    # Add a box around the label via "+p"
    "qd1c:+ltext+p",
    # Adjust thickness, color, and style of the outline
    "qd1c:+ltext+p0.5p,blue,dashed",
    # Append "+o" to get a box with rounded edges
    "qd1c:+ltext+p0.5p,blue+o",
    # Adjust the space between label and box in x- and y-directions via "+c"
    "qd1c:+ltext+p0.5p,blue+o+c0.1c/0.1c",
    # Give a fill of the box via "+g" together with the desired color
    "qd1c:+ltext+gdodgerblue",
]:
    y -= 1  # Move current line down
    fig.plot(x=x, y=y, pen="1.25p", style=quotedline)
    fig.text(
        x=x[-1],
        y=y[-1],
        text=quotedline,
        font="Courier-Bold",
        justify="ML",
        offset=(0.75, 0),
    )

fig.show()
```

Quoted Lines



For curved labels following the line, append `"+v"` to the argument passed to the `style` parameter.

```
# Generate sine curve
x = np.arange(0, 10 * np.pi, 0.1)
y = np.sin(0.8 * x)

fig = pygmt.Figure()

fig.basemap(region=[0, 30, -4, 4], projection="X10c/5c", frame=True)

fig.plot(x=x, y=y + 2, style="qd1.2c:+lstraight text+f5p", pen="1p,blue")

fig.plot(
    x=x,
    y=y - 2,
    # Append "+v" to force curved labels
    style="qd1.2c:+lcurved text+f5p+v",
    pen="1p,blue",
)

fig.show()
```



Total running time of the script: (0 minutes 1.000 seconds)

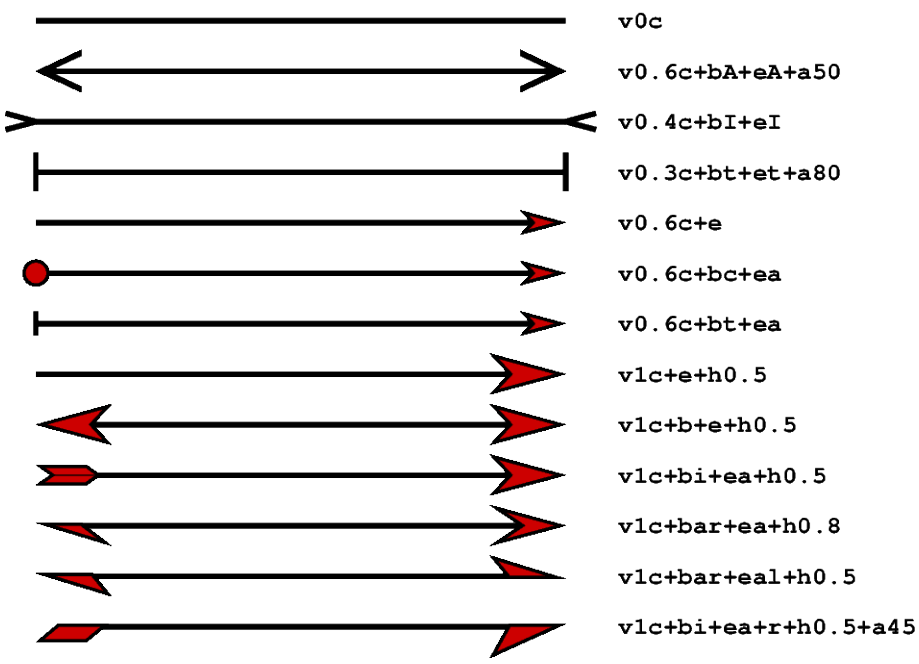
8.2.14 Vector heads and tails

Many methods in PyGMT allow plotting vectors with individual heads and tails. For this purpose, several modifiers may be appended to the corresponding vector-producing parameters for specifying the placement of vector heads and tails, their shapes, and the justification of the vector.

To place a vector head at the beginning of the vector path simply append **+b** to the vector-producing option (use **+e** to place one at the end). Optionally, append **t** for a terminal line, **c** for a circle, **a** for arrow (default), **i** for tail, **A** for plain open arrow, and **I** for plain open tail. Further append **l** or **r** (e.g. `"+bar"`) to only draw the left or right half-sides of the selected head/tail (default is both sides) or use **+l** or **+r** to apply simultaneously to both beginning and end. In this context, left and right refer to the side of the vector line when viewed from the beginning point to the end point of a line segment. The angle of the vector head apex can be set using **+a***angle* (default is 30). The shape of the vector head can be adjusted using **+h***shape* (e.g. `"+h0.5"`).

In the following, we use the *`pygmt.Figure.plot`* method to plot vectors with individual heads and tails. We must specify the modifiers (together with the vector type, here `"v"` for Cartesian vector, see also the *Vector types example*) by passing the corresponding shortcuts to the `style` parameter.

For further modifiers see https://docs.generic-mapping-tools.org/6.7/plot.html#vector-attributes. Please note that vectors were completely redesigned for GMT 5 which separated the vector head (a polygon) from the vector stem (a line). In GMT 4, the entire vector was a polygon. Thus, to fill the entire vector with a color users have to follow the GMT 4 synthax, which can be found on the webpage mentioned above.




```
import pygmt
from pygmt.params import Frame

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 15],
    projection="X15c/10c",
    frame=Frame(title="Vector heads and tails"),
)

x = 1
y = 14
angle = 0  # in degrees, measured counter-clockwise from horizontal
length = 7

for vecstyle in [
    # vector without head and tail (line)
    "v0c",
    # plain open arrow at beginning and end, angle of the vector head apex is
    # set to 50
    "v0.6c+bA+eA+a50",
    # plain open tail at beginning and end
    "v0.4c+bI+eI",
    # terminal line at beginning and end, angle of vector head apex is set
    # to 80
    "v0.3c+bt+et+a80",
    # arrow head at end
    "v0.6c+e",
```

(continues on next page)

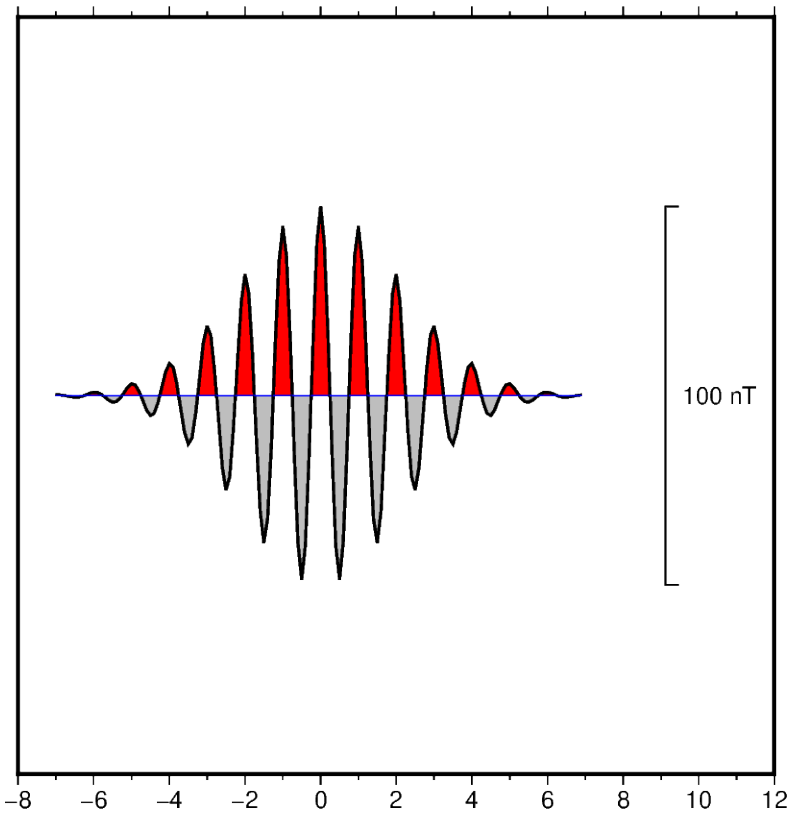
(continued from previous page)

```
    # circle at beginning and arrow head at end
    "v0.6c+bc+ea",
    # terminal line at beginning and arrow head at end
    "v0.6c+bt+ea",
    # arrow head at end, shape of vector head is set to 0.5
    "v1c+e+h0.5",
    # modified arrow heads at beginning and end
    "v1c+b+e+h0.5",
    # tail at beginning and arrow with modified vector head at end
    "v1c+bi+ea+h0.5",
    # half-sided arrow head (right side) at beginning and arrow at the end
    "v1c+bar+ea+h0.8",
    # half-sided arrow heads at beginning (right side) and end (left side)
    "v1c+bar+eal+h0.5",
    # half-sided tail at beginning and arrow at end (right side for both)
    "v1c+bi+ea+r+h0.5+a45",
]:
    fig.plot(
        x=x, y=y, style=vecstyle, direction=([angle], [length]), pen="2p", fill="red3"
    )
    fig.text(
        x=6, y=y, text=vecstyle, font="Courier-Bold", justify="ML", offset=(0.2, 0)
    )
    y -= 1  # move the next vector down

fig.show()
```

Total running time of the script: (0 minutes 0.364 seconds)

8.2.15 Wiggle along tracks

The *pygmt.Figure.wiggle* method can plot z = f(x,y) anomalies along tracks. `x`, `y`, `z` can be specified as 1-D arrays or within a specified file. The `scale` parameter can be used to set the scale of the anomaly in data/distance units. The positive and/or negative areas can be filled with color by setting the `positive_fill` and/or `negative_fill` parameters.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame, Position

# Create (x, y, z) triplets
x = np.arange(-7, 7, 0.1)
y = np.zeros(x.size)
z = 50 * np.exp(-((x / 3) ** 2)) * np.cos(2 * np.pi * x)

fig = pygmt.Figure()
fig.basemap(
    region=[-8, 12, -1, 1],
    projection="X10c",
    frame=Frame(axes="Snlr", xaxis=Axis(annot=2, tick=1)),
)
fig.wiggle(
    x=x,
    y=y,
    z=z,
    scale="20c",  # Set anomaly scale to 20 centimeters
    positive_fill="red",  # Fill positive areas red
    negative_fill="gray",  # Fill negative areas gray
    pen="1.0p",  # Set the outline width to 1.0 point
    track="0.5p,blue",  # Draw a blue track with a width of 0.5 points
    position=Position("MR"),  # Plot a vertical scale bar at Middle Right (MR).
    length=100,  # Bar length is 100 in data (z) units.
    label="nT",  # Set the z unit label to "nT".
```

(continues on next page)

(continued from previous page)

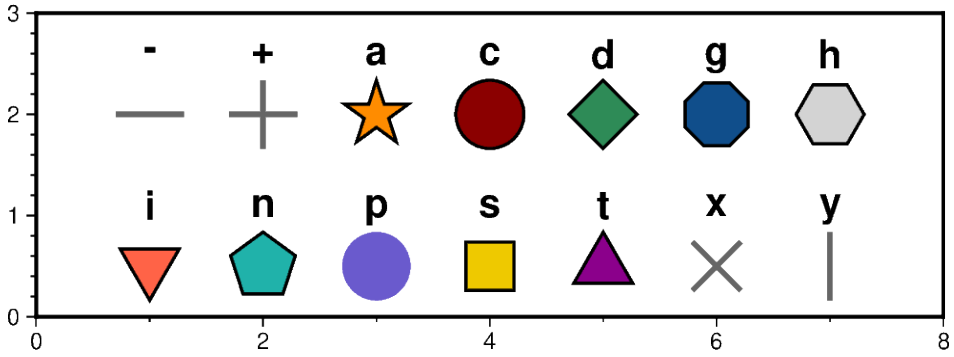
```
)
fig.show()
```

Total running time of the script: (0 minutes 0.133 seconds)

8.3 Symbols and markers

8.3.1 Basic geometric symbols

The *pygmt.Figure.plot* method can plot individual geometric symbols by passing the corresponding shortcuts to the `style` parameter. The 14 basic geometric symbols are shown underneath their corresponding shortcut codes. Four symbols (**-**, **+**, **x** and **y**) are line-symbols only for which we can adjust the linewidth via the `pen` parameter. The point symbol (**p**) only takes a color fill which we can define via the `fill` parameter. For the remaining symbols we may define a linewidth as well as a color fill.




```
import pygmt

fig = pygmt.Figure()
fig.basemap(region=[0, 8, 0, 3], projection="X12c/4c", frame=True)

# define fontstlye for annotations
font = "15p,Helvetica-Bold"

# upper row
y = 2

# use a dash in x-direction (-) with a size of 0.9 cm,
# linewidth is set to 2p and the linecolor to "gray40"
fig.plot(x=1, y=y, style="-0.9c", pen="2p,gray40")
fig.text(x=1, y=y + 0.6, text="-", font=font)

# use a plus (+) with a size of 0.9 cm,
# linewidth is set to 2p and the linecolor to "gray40"
fig.plot(x=2, y=y, style="+0.9c", pen="2p,gray40")
fig.text(x=2, y=y + 0.6, text="+", font=font)

# use a star (a) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" (default) and the
# color fill to "darkorange"
fig.plot(x=3, y=y, style="a0.9c", pen="1p,black", fill="darkorange")
```

(continues on next page)

(continued from previous page)

```
fig.text(x=3, y=y + 0.6, text="a", font=font)

# use a circle (c) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "darkred"
fig.plot(x=4, y=y, style="c0.9c", pen="1p,black", fill="darkred")
fig.text(x=4, y=y + 0.6, text="c", font=font)

# use a diamond (d) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "seagreen"
fig.plot(x=5, y=y, style="d0.9c", pen="1p,black", fill="seagreen")
fig.text(x=5, y=y + 0.6, text="d", font=font)

# use a octagon (g) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "dodgerblue4"
fig.plot(x=6, y=y, style="g0.9c", pen="1p,black", fill="dodgerblue4")
fig.text(x=6, y=y + 0.6, text="g", font=font)

# use a hexagon (h) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "lightgray"
fig.plot(x=7, y=y, style="h0.9c", pen="1p,black", fill="lightgray")
fig.text(x=7, y=y + 0.6, text="h", font=font)

# lower row
y = 0.5

# use an inverted triangle (i) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "tomato"
fig.plot(x=1, y=y, style="i0.9c", pen="1p,black", fill="tomato")
fig.text(x=1, y=y + 0.6, text="i", font=font)

# use pentagon (n) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "lightseagreen"
fig.plot(x=2, y=y, style="n0.9c", pen="1p,black", fill="lightseagreen")
fig.text(x=2, y=y + 0.6, text="n", font=font)

# use a point (p) with a size of 0.9 cm,
# color fill is set to "lightseagreen"
fig.plot(x=3, y=y, style="p0.9c", fill="slateblue")
fig.text(x=3, y=y + 0.6, text="p", font=font)

# use square (s) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "gold2"
fig.plot(x=4, y=y, style="s0.9c", pen="1p,black", fill="gold2")
fig.text(x=4, y=y + 0.6, text="s", font=font)
```

(continues on next page)

(continued from previous page)

```
# use triangle (t) with a size of 0.9 cm,
# linewidth is set to 1p, the linecolor to "black" and the
# color fill to "magenta4"
fig.plot(x=5, y=y, style="t0.9c", pen="1p,black", fill="magenta4")
fig.text(x=5, y=y + 0.6, text="t", font=font)

# use cross (x) with a size of 0.9 cm,
# linewidth is set to 2p and the linecolor to "gray40"
fig.plot(x=6, y=y, style="x0.9c", pen="2p,gray40")
fig.text(x=6, y=y + 0.6, text="x", font=font)

# use a dash in y-direction (y) with a size of 0.9 cm,
# linewidth is set to 2p and the linecolor to "gray40"
fig.plot(x=7, y=y, style="y0.9c", pen="2p,gray40")
fig.text(x=7, y=y + 0.6, text="y", font=font)

fig.show()
```

Total running time of the script: (0 minutes 0.428 seconds)

8.3.2 Bit and hachure patterns

In addition to colors, PyGMT also allows using bit and hachure patterns to fill symbols, polygons, and other areas, via the `fill` parameter or similar parameters.

GMT provides 90 predefined 1-bit patterns, which are numbered from 1 to 90. In addition, custom 1-, 8-, or 24-bit image raster files can also be used as patterns.

These patterns can be specified via the *pygmt.params.Pattern* class. The patterns can be customized with different resolution and different foreground and background colors. The foreground and background colors can also be inverted.

Bit and Hachure Patterns

```
import pygmt
from pygmt.params import Frame, Pattern

# A list of patterns that will be demonstrated.
# By default, a pattern is plotted in black and white with a resolution of 300 dpi.
patterns = [
    # Predefined 1-bit pattern 8.
    Pattern(8),
    # Predefined 1-bit pattern 19.
    Pattern(19),
    # Pattern 19 with custom background ("red3") and foreground ("lightbrown").
    Pattern(19, bgcolor="red3", fgcolor="lightbrown"),
    # Invert the background and foreground.
    Pattern(19, invert=True, bgcolor="red3", fgcolor="lightbrown"),
    # Same as above, but with a 100 dpi resolution.
    Pattern(19, bgcolor="red3", fgcolor="lightbrown", dpi=100),
    # Same as above, but with a transparent background by setting bgcolor to "".
    Pattern(19, bgcolor="", fgcolor="lightbrown", dpi=100),
]

fig = pygmt.Figure()
fig.basemap(
    region=[0, 10, 0, 12],
    projection="X18c/10c",
    frame=Frame(axes="rlbt", fill="lightgray", title="Bit and Hachure Patterns"),
)
y = 11
for pattern in patterns:
```

(continues on next page)

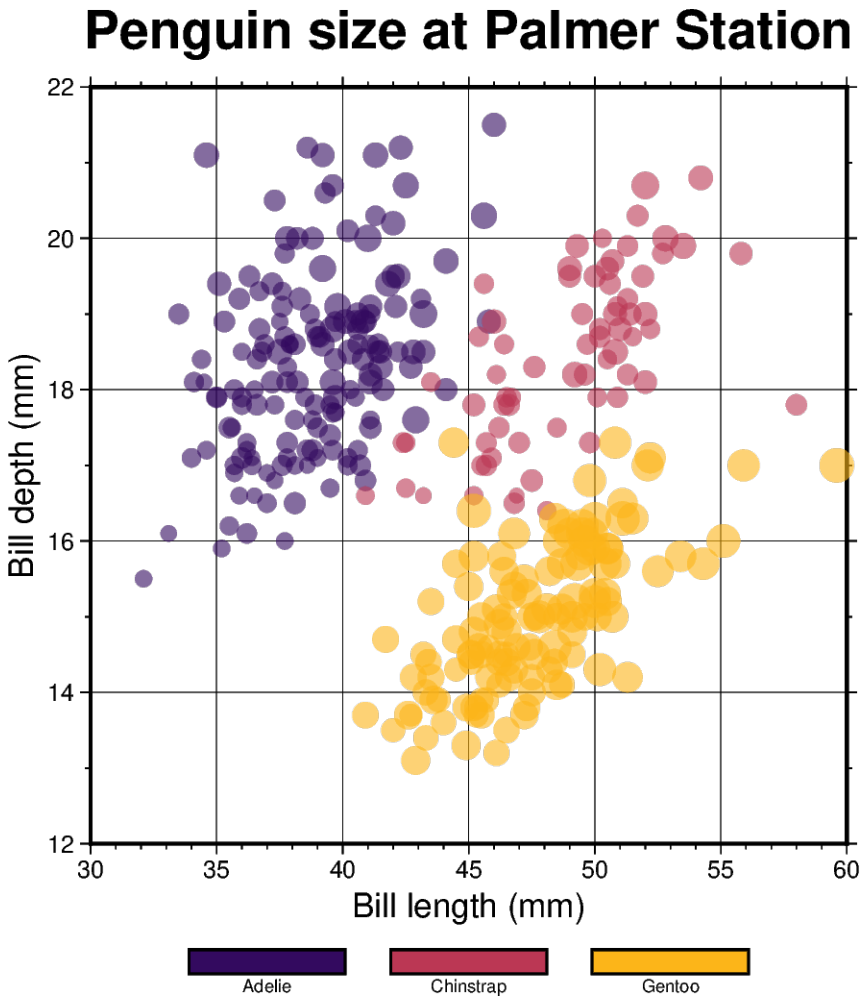
(continued from previous page)

```
    # Plot a square with the pattern as fill.
    # The square has a size of 2 centimeters with a 1 point thick, black outline.
    fig.plot(x=1, y=y, style="s2c", pen="1p,black", fill=pattern)
    # Add a description of the pattern.
    fig.text(x=2, y=y, text=str(repr(pattern)), font="Courier-Bold", justify="ML")
    y -= 2
fig.show()
```

Total running time of the script: (0 minutes 0.279 seconds)

8.3.3 Color points by categories

The *pygmt.Figure.plot* method can be used to plot symbols which are color-coded by categories. In the example below, we show how the Palmer Penguins dataset can be visualized. Here, we can pass the individual categories included in the "species" column directly to the `fill` parameter via `fill=df.species.cat.codes.astype(int)`. Additionally, we have to set `cmap=True`. A desired colormap can be selected via the *pygmt.makecpt* function.




```
import pandas as pd
import pygmt
from pygmt.params import Axis, Frame
```

(continues on next page)

(continued from previous page)

```
# Load sample penguins data
df = pd.read_csv("https://github.com/mwaskom/seaborn-data/raw/master/penguins.csv")

# Convert 'species' column to categorical dtype
# By default, pandas sorts the individual categories in an alphabetical order.
# For a non-alphabetical order, you have to manually adjust the list of
# categories. For handling and manipulating categorical data in pandas,
# have a look at:
# https://pandas.pydata.org/docs/user_guide/categorical.html
df.species = df.species.astype(dtype="category")

# Make a list of the individual categories of the 'species' column
# ['Adelie', 'Chinstrap', 'Gentoo']
# They are (corresponding to the categorical number code) by default in
# alphabetical order and later used for the colorbar annotations
cb_annots = list(df.species.cat.categories)

# Use pygmt.info to get region bounds (xmin, xmax, ymin, ymax)
# The below example will return a numpy array like [30.0, 60.0, 12.0, 22.0]
region = pygmt.info(
    data=df[["bill_length_mm", "bill_depth_mm"]],  # x and y columns
    per_column=True,  # Report the min/max values per column as a numpy array
    # Round the min/max values of the first two columns to the nearest multiple
    # of 3 and 2, respectively
    spacing=(3, 2),
)

# Make a 2-D categorical scatter plot, coloring each of the 3 species
# differently
fig = pygmt.Figure()

# Generate a basemap of 10 cm x 10 cm size
fig.basemap(
    region=region,
    projection="X10c/10c",
    frame=Frame(
        axes="WSen",
        title="Penguin size at Palmer Station",
        xaxis=Axis(annot=True, tick=True, grid=True, label="Bill length (mm)"),
        yaxis=Axis(annot=True, tick=True, grid=True, label="Bill depth (mm)"),
    ),
)

# Define a colormap for three categories, define the range of the
# new discrete CPT using series=(lowest_value, highest_value, interval),
# use color_model="+cAdelie,Chinstrap,Gentoo" to write the discrete color
# palette "inferno" in categorical format and add the species names as
# annotations for the colorbar
pygmt.makecpt(
    cmap="matplotlib/inferno",
    # Use the minimum and maximum of the categorical number code
    # to set the lowest_value and the highest_value of the CPT
```

(continues on next page)

(continued from previous page)

```
    series=(df.species.cat.codes.min(), df.species.cat.codes.max(), 1),
    # Convert ['Adelie', 'Chinstrap', 'Gentoo'] to 'Adelie,Chinstrap,Gentoo'
    color_model="+c" + ",".join(cb_annots),
)

fig.plot(
    # Use bill length and bill depth as x and y data input, respectively
    x=df.bill_length_mm,
    y=df.bill_depth_mm,
    # Vary symbol size according to the body mass, scaled by 7.5e-5
    size=df.body_mass_g * 7.5e-5,
    # Points colored by categorical number code (refers to the species)
    fill=df.species.cat.codes.astype(int),
    # Use colormap created by makecpt
    cmap=True,
    # Do not clip symbols that fall close to the plot bounds
    no_clip=True,
    # Use circles as symbols (the first "c") with diameter in
    # centimeters (the second "c")
    style="cc",
    # Set transparency level for all symbols to deal with overplotting
    transparency=40,
)

# Add colorbar legend
fig.colorbar()

fig.show()
```

Total running time of the script: (0 minutes 1.192 seconds)

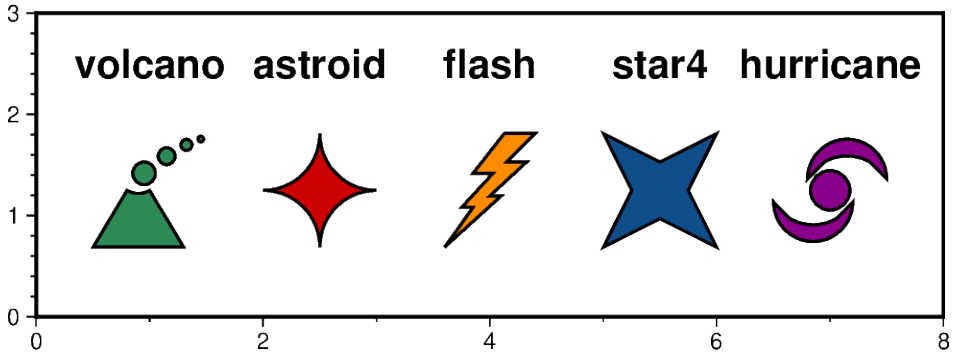
8.3.4 Custom symbols

The *pygmt.Figure.plot* method can plot individual custom symbols by passing the corresponding symbol name together with the **k** shortcut to the `style` parameter.

In total 41 custom symbols are already included of which the following plot shows five exemplary ones. The symbols are shown underneath their corresponding names. For the remaining symbols see the GMT Technical Reference https://docs.generic-mapping-tools.org/6.7/reference/custom-symbols.html.

Besides these built-in custom symbols, GMT allows users to define their own custom symbols. For this, a specific macro language is used. A detailed introduction can be found at https://docs.generic-mapping-tools.org/6.7/reference/custom-symbols.html#the-macro-language. After defining such a symbol it can be used in the same way as a built-in custom symbol.

Please note: Custom symbols cannot be used in auto-legends yet.




```
import pygmt

fig = pygmt.Figure()
fig.basemap(region=[0, 8, 0, 3], projection="X12c/4c", frame=True)

# Define pen and fontstyle for annotations
pen = "1p,black"
font = "15p,Helvetica-Bold"

# Use the volcano symbol with a size of 1.5c, fill color is set to "seagreen"
fig.plot(x=1, y=1.25, style="kvolcano/1.5c", pen=pen, fill="seagreen")
fig.text(x=1, y=2.5, text="volcano", font=font)

# Use the astroid symbol with a size of 1.5c, fill color is set to "red3"
fig.plot(x=2.5, y=1.25, style="kastroid/1.5c", pen=pen, fill="red3")
fig.text(x=2.5, y=2.5, text="astroid", font=font)

# Use the flash symbol with a size of 1.5c, fill color is set to "darkorange"
fig.plot(x=4, y=1.25, style="kflash/1.5c", pen=pen, fill="darkorange")
fig.text(x=4, y=2.5, text="flash", font=font)

# Use the star4 symbol with a size of 1.5c, fill color is set to "dodgerblue4"
fig.plot(x=5.5, y=1.25, style="kstar4/1.5c", pen=pen, fill="dodgerblue4")
fig.text(x=5.5, y=2.5, text="star4", font=font)

# Use the hurricane symbol with a size of 1.5c, fill color is set to "magenta4"
fig.plot(x=7, y=1.25, style="khurricane/1.5c", pen=pen, fill="magenta4")
fig.text(x=7, y=2.5, text="hurricane", font=font)

fig.show()
```

Total running time of the script: (0 minutes 0.119 seconds)

8.3.5 Datetime inputs

Datetime inputs of the following types are supported in PyGMT:

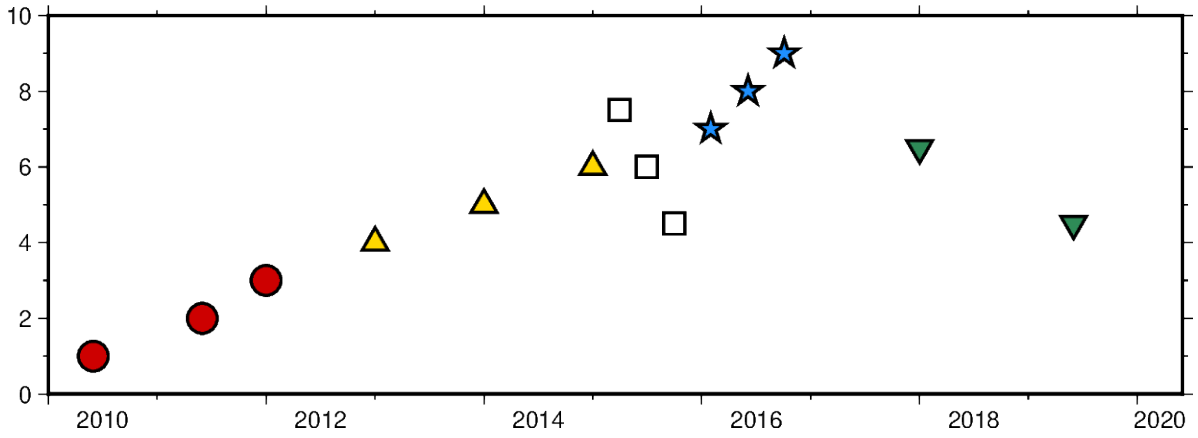
- `numpy.datetime64`
- `pandas.DatetimeIndex`
- `xarray.DataArray`: datetimes included in an *xarray.DataArray*
- raw datetime strings in ISO 8601 format (e.g. `"YYYY-MM-DD"`, `"YYYY-MM-DDTHH"`, and

"YYYY-MM-DDTHH:MM:SS")

- Python built-in `datetime.datetime` and `datetime.date`

We can pass datetime inputs based on one of the types listed above directly to the `x` and `y` parameters of e.g. the *pygmt.Figure.plot* method.

The `region` parameter has to include the x and y axis limits in the form [*date_min*, *date_max*, *ymin*, *ymax*]. Here *date_min* and *date_max* can be directly defined as datetime input.




```
import datetime as dt

import numpy as np
import pandas as pd
import pygmt
import xarray as xr
from pygmt.params import Axis, Frame

fig = pygmt.Figure()

# create a basemap with limits of 2010-01-01 to 2020-06-01 on the x-axis and
# 0 to 10 on the y-axis
fig.basemap(
    projection="X15c/5c",
    region=[dt.date(2010, 1, 1), dt.date(2020, 6, 1), 0, 10],
    frame=Frame(axes="WSen", axis=Axis(annot=True, tick=True)),
)

# numpy.datetime64 types
x = np.array(
    ["2010-06-01", "2011-06-01T12", "2012-01-01T12:34:56"], dtype=np.datetime64
)
y = [1, 2, 3]
fig.plot(x=x, y=y, style="c0.4c", pen="1p", fill="red3")

# pandas.DatetimeIndex
x = pd.date_range("2013", periods=3, freq="YS")
y = [4, 5, 6]
fig.plot(x=x, y=y, style="t0.4c", pen="1p", fill="gold")

# xarray.DataArray
```

(continues on next page)

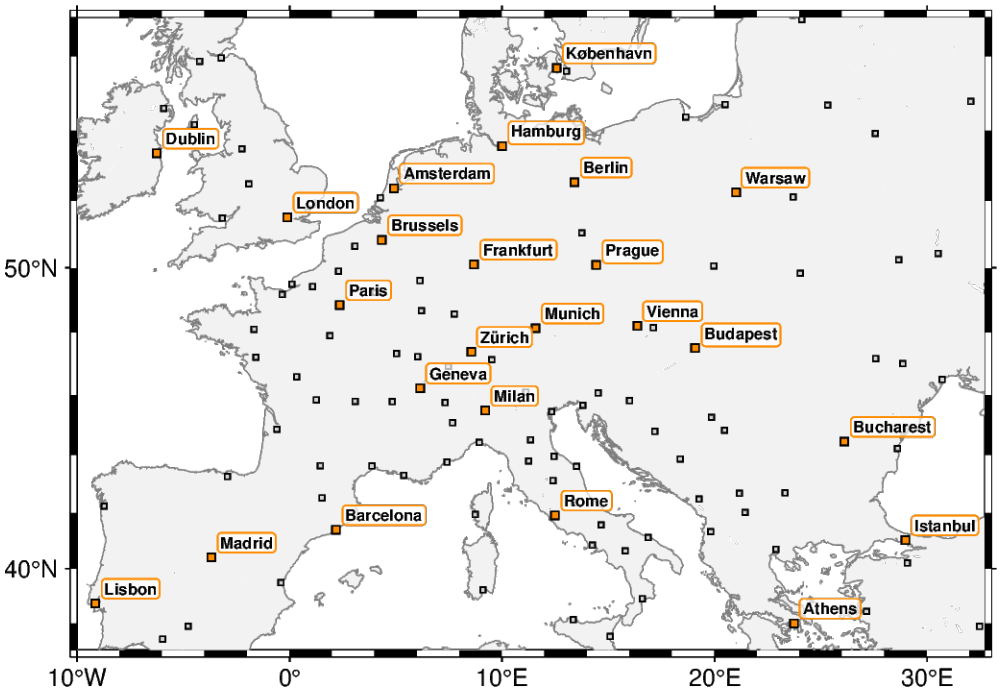
(continued from previous page)

```
x = xr.DataArray(data=pd.date_range(start="2015-03", periods=3, freq="QS"))
y = [7.5, 6, 4.5]
fig.plot(x=x, y=y, style="s0.4c", pen="1p")

# raw datetime strings
x = ["2016-02-01", "2016-06-04T14", "2016-10-04T00:00:15"]
y = [7, 8, 9]
fig.plot(x=x, y=y, style="a0.4c", pen="1p", fill="dodgerblue")

# the Python built-in datetime and date
x = [dt.date(2018, 1, 1), dt.datetime(2019, 6, 1, 20, 5, 45)]
y = [6.5, 4.5]
fig.plot(x=x, y=y, style="i0.4c", pen="1p", fill="seagreen")

fig.show()
```

Total running time of the script: (0 minutes 0.104 seconds)

8.3.6 GeoPandas: Plotting points with Point or MultiPoint geometry

The *pygmt.Figure.plot* method allows us to plot geographical data such as points with Point or MultiPoint geometry types stored in a `geopandas.GeoDataFrame` object. Use `geopandas.read_file` to load data from any supported OGR format such as a shapefile (.shp), GeoJSON (.geojson), geopackage (.gpkg), etc. Then, pass the `geopandas.GeoDataFrame` object as an argument to the `data` parameter of *pygmt.Figure.plot*, and style the points using the `fill` and `pen` parameters. Additionally, pass suitable columns of the `geopandas.GeoDataFrame` to the `x`, `y`, and `text` parameters of the *pygmt.Figure.text* method to label specific features.




```
import geopandas
import pygmt
```

(continues on next page)

(continued from previous page)

```
# Read a sample dataset provided by Natural Earth. The dataset contains cities stored
# as Point geometry type. In this example we focus on Europe.
provider = "https://naciscdn.org/naturalearth"
cities = geopandas.read_file(
    f"{provider}/50m/cultural/ne_50m_populated_places_simple.zip"
)
cities = cities[cities["name"] != "Vatican City"].copy()  # No overlap with label Rome
# Create two subsets for small and large cities
cities_small = cities[cities["worldcity"] != 1].copy()
cities_large = cities[cities["worldcity"] == 1].copy()

fig = pygmt.Figure()
fig.basemap(region=[-10, 32.7, 37, 57], projection="M12c", frame=True)
fig.coast(land="gray95", shorelines="1/0.3p,gray50")

# Plot the two subsets using squares with different sizes and fills.
fig.plot(data=cities_small, style="s0.1c", fill="lightgray", pen="0.5p")
fig.plot(data=cities_large, style="s0.15c", fill="darkorange", pen="0.5p")

# Label the larger cities with their names.
fig.text(
    x=cities_large.geometry.x,
    y=cities_large.geometry.y,
    text=cities_large["name"],
    offset=0.12,
    justify="BL",
    font="6p,Helvetica-Bold",
    fill="white@30",
    pen="0.5p,darkorange",
    clearance="0.05c+tO",
)

fig.show()
```

Total running time of the script: (0 minutes 0.636 seconds)

8.3.7 Multi-parameter symbols

The *pygmt.Figure.plot* method can plot individual multi-parameter symbols by passing the corresponding shortcuts (**e**, **j**, **r**, **R**, **w**) to the `style` parameter:

- **e**: ellipse
- **j**: rotated rectangle
- **r**: rectangle
- **R**: rounded rectangle
- **w**: pie wedge

```
import pygmt
from pygmt.params import Axis, Frame
```

We can plot multi-parameter symbols using the same symbol style. We need to define locations (lon, lat) via the `x` and

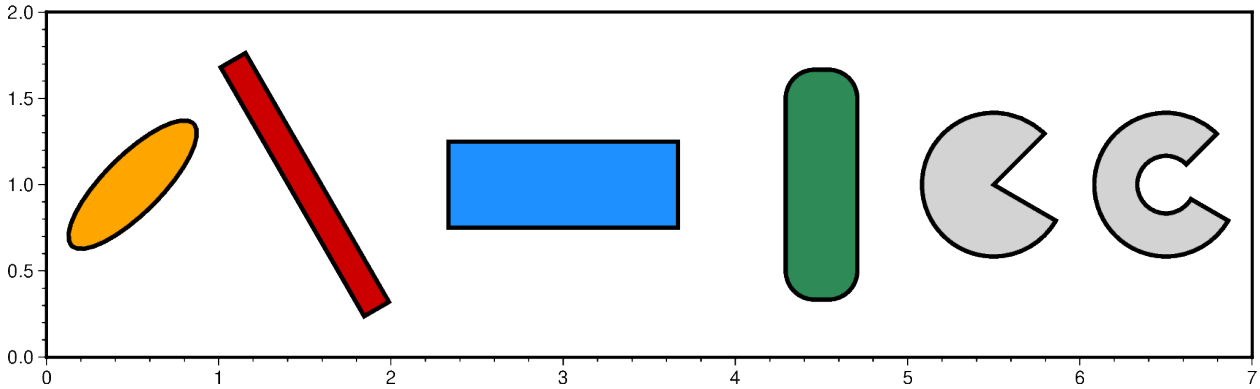
`y` parameters (scalar for a single symbol or 1-D list for several ones) and two or three symbol parameters after those shortcuts via the `style` parameter.

The multi-parameter symbols in the `style` parameter are defined as:

- **e**: ellipse, `direction/major_axis/minor_axis`
- **j**: rotated rectangle, `direction/width/height`
- **r**: rectangle, `width/height`
- **R**: rounded rectangle, `width/height/radius`
- **w**: pie wedge, `diameter/startdir/stopdir`, the last two arguments are directions given in degrees counter-clockwise from horizontal. Append **+i** and the desired value to apply an inner diameter.

Uppercase versions **E**, **J**, and **W** are similar to **e**, **j**, and **w** but expect geographic azimuths and distances.

```
fig = pygmt.Figure()
fig.basemap(region=[0, 7, 0, 2], projection="x3c", frame=True)

# Ellipse
fig.plot(x=0.5, y=1, style="e45/3/1", fill="orange", pen="2p,black")
# Rotated rectangle
fig.plot(x=1.5, y=1, style="j120/5/0.5", fill="red3", pen="2p,black")
# Rectangle
fig.plot(x=3, y=1, style="r4/1.5", fill="dodgerblue", pen="2p,black")
# Rounded rectangle
fig.plot(x=4.5, y=1, style="R1.25/4/0.5", fill="seagreen", pen="2p,black")
# Pie wedge
fig.plot(x=5.5, y=1, style="w2.5/45/330", fill="lightgray", pen="2p,black")
# Ring sector
fig.plot(x=6.5, y=1, style="w2.5/45/330+i1", fill="lightgray", pen="2p,black")

fig.show()
```



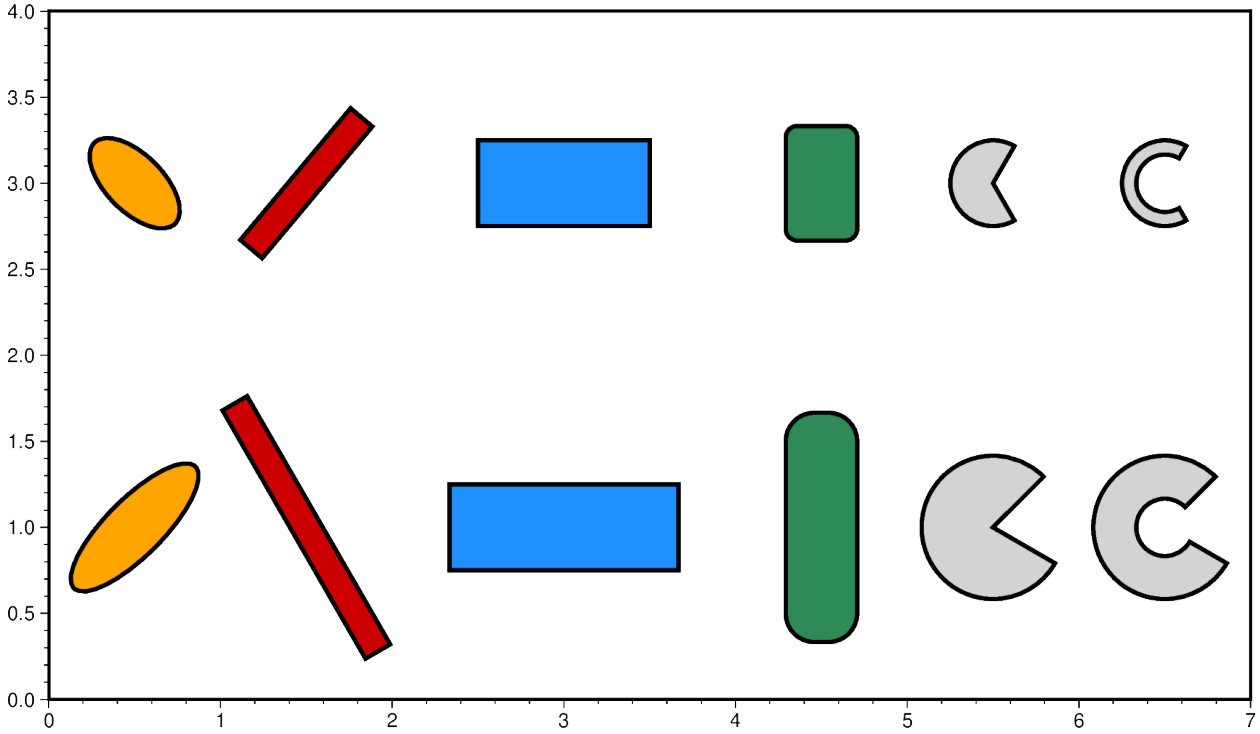
We can also plot symbols with varying parameters via defining those values in a 2-D list or numpy array (`[[parameters]]` for a single symbol or `[[parameters_1],[parameters_2],[parameters_i]]` for several ones) or using an appropriately formatted input file and passing it to `data`.

The symbol parameters in the 2-D list or numpy array are defined as:

- **e**: ellipse, `[[lon, lat, direction, major_axis, minor_axis]]`
- **j**: rotated rectangle, `[[lon, lat, direction, width, height]]`
- **r**: rectangle, `[[lon, lat, width, height]]`

- **R**: rounded rectangle, `[[lon, lat, width, height, radius]]`
- **w**: pie wedge, `[[lon, lat, diameter, startdir, stopdir]]`, the last two arguments are directions given in degrees counter-clockwise from horizontal

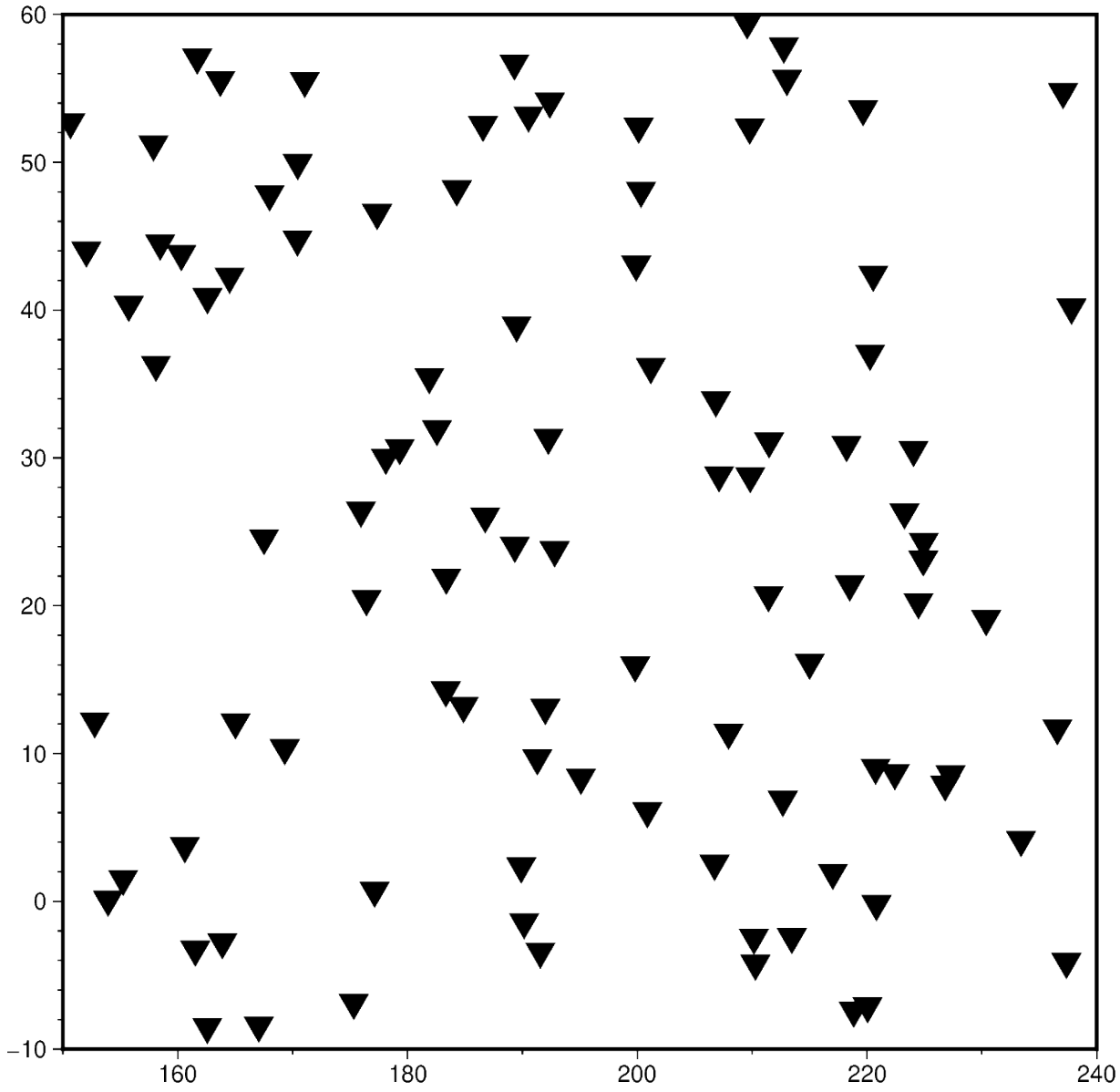
```
fig = pygmt.Figure()
fig.basemap(
    region=[0, 7, 0, 4],
    projection="x3c",
    frame=Frame(xaxis=Axis(annot=1, tick=0.2), yaxis=Axis(annot=0.5, tick=0.1)),
)

# Ellipse
data = [[0.5, 1, 45, 3, 1], [0.5, 3, 135, 2, 1]]
fig.plot(data=data, style="e", fill="orange", pen="2p,black")
# Rotated rectangle
data = [[1.5, 1, 120, 5, 0.5], [1.5, 3, 50, 3, 0.5]]
fig.plot(data=data, style="j", fill="red3", pen="2p,black")
# Rectangle
data = [[3, 1, 4, 1.5], [3, 3, 3, 1.5]]
fig.plot(data=data, style="r", fill="dodgerblue", pen="2p,black")
# Rounded rectangle
data = [[4.5, 1, 1.25, 4, 0.5], [4.5, 3, 1.25, 2.0, 0.2]]
fig.plot(data=data, style="R", fill="seagreen", pen="2p,black")
# Pie wedge
data = [[5.5, 1, 2.5, 45, 330], [5.5, 3, 1.5, 60, 300]]
fig.plot(data=data, style="w", fill="lightgray", pen="2p,black")
# Ring sector
data = [[6.5, 1, 2.5, 45, 330], [6.5, 3, 1.5, 60, 300]]
fig.plot(data=data, style="w+i1", fill="lightgray", pen="2p,black")

fig.show()
```



Total running time of the script: (0 minutes 0.338 seconds)

8.3.8 Points

The *pygmt.Figure.plot* method can plot data points. The symbol and size are set with the `style` parameter.




```
import numpy as np
import pygmt

# Generate a random set of points to plot
rng = np.random.default_rng(seed=42)
region = [150, 240, -10, 60]
x = rng.uniform(low=region[0], high=region[1], size=100)
y = rng.uniform(low=region[2], high=region[3], size=100)


fig = pygmt.Figure()
# Create a 15 cm x 15 cm basemap with a Cartesian projection (X) using the
# data region
fig.basemap(region=region, projection="X15c", frame=True)
# Plot using inverted triangles (i) of 0.5 cm size
```

(continues on next page)

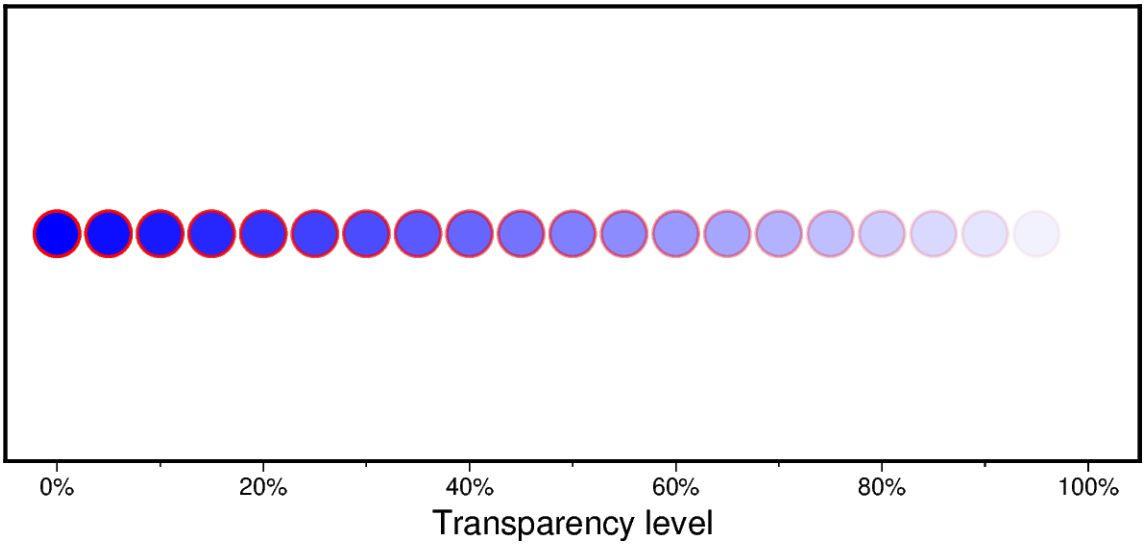
(continued from previous page)

```
fig.plot(x=x, y=y, style="i0.5c", fill="black")
fig.show()
```

Total running time of the script: (0 minutes 0.127 seconds)

8.3.9 Points with varying transparency

Points can be plotted with different transparency levels by passing in an array argument to the `transparency` parameter of *pygmt.Figure.plot*.



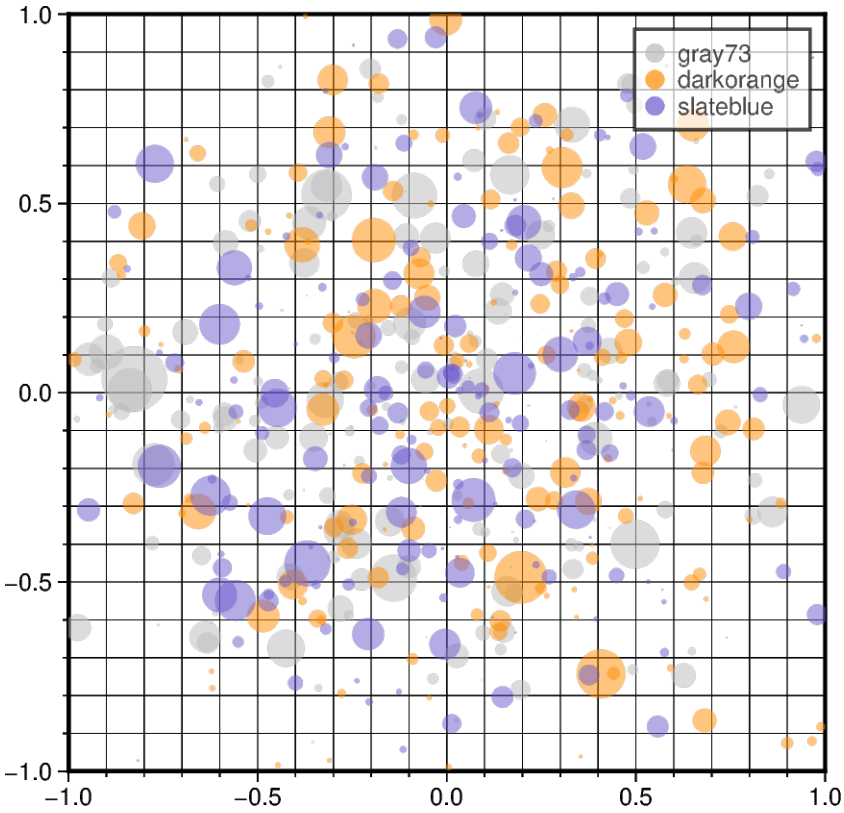

```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# prepare the input x and y data
x = np.arange(0, 105, 5)
y = np.ones(x.size)
# transparency level in percentage from 0 (i.e., opaque) to 100
transparency = x

fig = pygmt.Figure()
fig.basemap(
    region=[-5, 105, 0, 2],
    frame=Frame(
        axes="WSrt",
        xaxis=Axis(annot=True, tick=True, label="Transparency level", unit="%"),
    ),
    projection="X15c/6c",
)
fig.plot(x=x, y=y, style="c0.6c", fill="blue", pen="1p,red",␣
↪transparency=transparency)
fig.show()
```

Total running time of the script: (0 minutes 0.139 seconds)

8.3.10 Scatter plots with a legend

To create a scatter plot with a legend one may use a loop and create one scatter plot per item to appear in the legend and set the label accordingly.

Modified from the matplotlib example: https://matplotlib.org/stable/gallery/lines_bars_and_markers/scatter_with_legend.html




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

rng = np.random.default_rng(seed=19680801)
n = 200  # number of random data points

fig = pygmt.Figure()
fig.basemap(
    region=[-1, 1, -1, 1],
    projection="X10c/10c",
    frame=Frame(
        axes="WSrt",
        xaxis=Axis(annot=0.5, tick=True, grid=True),
        yaxis=Axis(annot=0.5, tick=True, grid=True),
    ),
)
for fill in ["gray73", "darkorange", "slateblue"]:
    # Generate standard normal distributions centered on 0
    # with standard deviations of 1
```

(continues on next page)

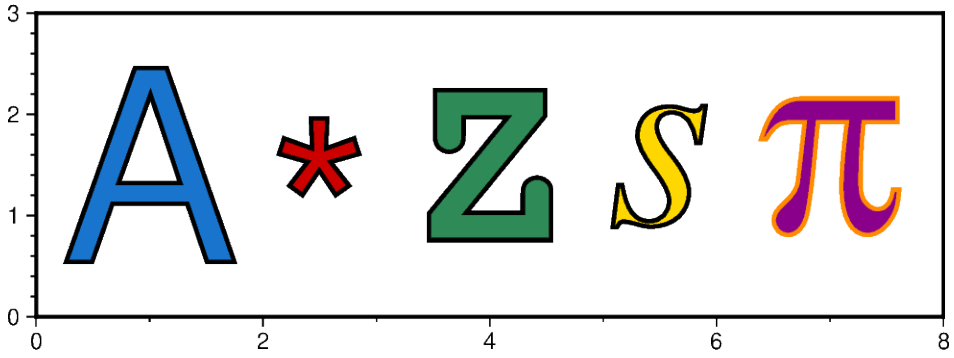
(continued from previous page)

```
    x = rng.normal(loc=0, scale=0.5, size=n)  # random x data
    y = rng.normal(loc=0, scale=0.5, size=n)  # random y data
    size = rng.normal(loc=0, scale=0.5, size=n) * 0.5  # random size, in cm

    # plot data points as circles (style="c"), with different sizes
    fig.plot(
        x=x,
        y=y,
        style="c",
        size=size,
        fill=fill,
        # Set the legend label,
        # and set the symbol size to be 0.25 cm (+S0.25c) in legend
        label=f"{fill}+S0.25c",
        transparency=50,  # set transparency level for all symbols
    )

fig.legend(transparency=30)  # set transparency level for legends
fig.show()
```

Total running time of the script: (0 minutes 0.197 seconds)

8.3.11 Text symbols

The *pygmt.Figure.plot* method allows to plot text symbols. Text is normally placed with the *pygmt.Figure.text* method but there are times we wish to treat a character or even a string as a plottable symbol. A text symbol can be drawn by passing **l***size*+**t***string* to the `style` parameter where *size* defines the size of the text symbol (note: the size is only approximate; no individual scaling is done for different characters) and *string* can be a letter or a text string (less than 256 characters). Optionally, you can append **+f***font,outlinecolor* to select a particular font [Default is FONT_ANNOT_PRIMARY] and outline color [Default is black] as well as **+j***justify* to change the justification [Default is CM]. For all supported fonts see *Supported Fonts*. The fill color of the text symbols can be set with the `fill` parameter, and the outline width can be customized with the `pen` parameter.




```
import pygmt

fig = pygmt.Figure()

fig.basemap(region=[0, 8, 0, 3], projection="X12c/4c", frame=True)

pen = "1.5p"
# plot an uppercase "A" of size 3.5c, color fill is set to "dodgerblue3"
```

(continues on next page)

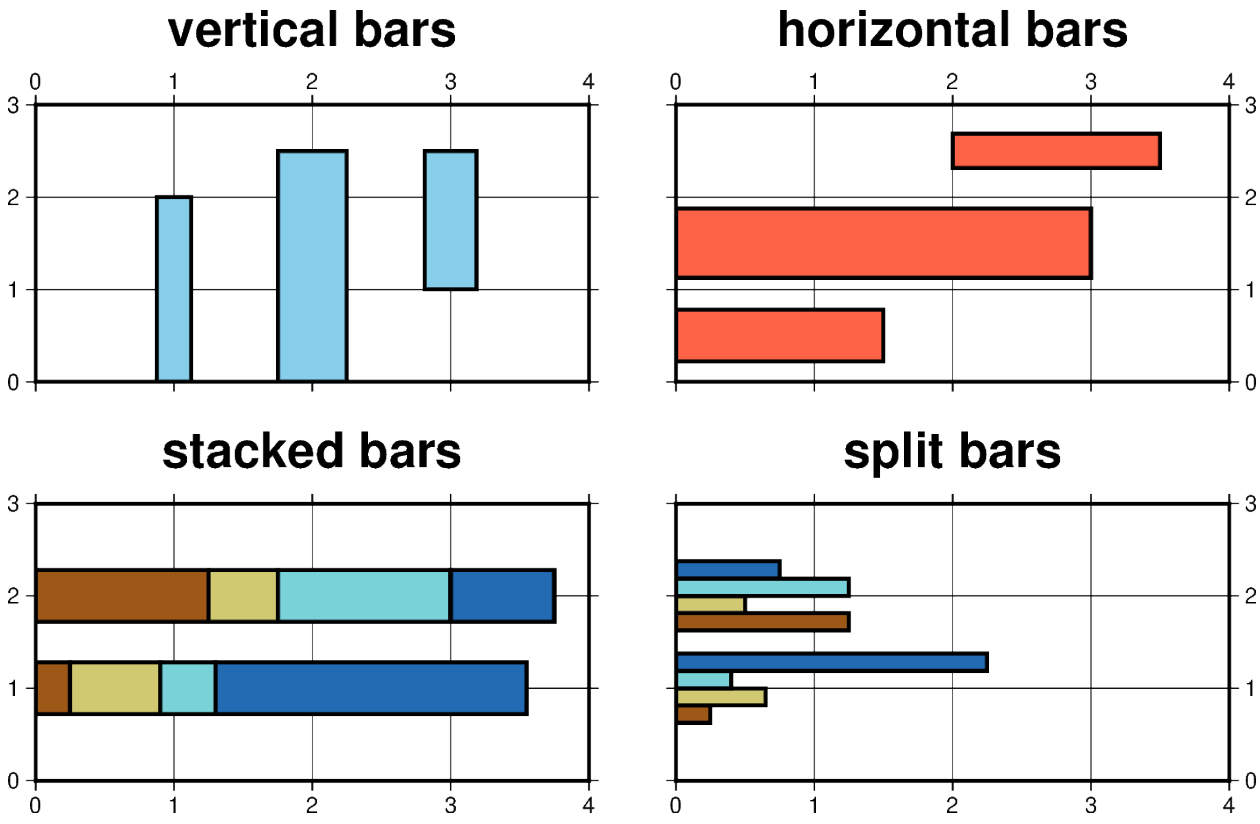
(continued from previous page)

```
fig.plot(x=1, y=1.5, style="l3.5c+tA", fill="dodgerblue3", pen=pen)
# plot an "asterisk" of size 3.5c, color fill is set to "red3"
fig.plot(x=2.5, y=1, style="l3.5c+t*", fill="red3", pen=pen)
# plot an uppercase "Z" of size 3.5c and use the "Courier-Bold" font,
# color fill is set to "seagreen"
fig.plot(x=4, y=1.5, style="l3.5c+tZ+fCourier-Bold", fill="seagreen", pen=pen)
# plot a lowercase "s" of size 3.5c and use the "Times-Italic" font,
# color fill is set to "gold"
fig.plot(x=5.5, y=1.5, style="l3.5c+ts+fTimes-Italic", fill="gold", pen=pen)
# plot the pi symbol of size 3.5c, the outline color of the symbol is set to
# "darkorange", the color fill is set to "magenta4"
fig.plot(x=7, y=1.5, style="l3.5c+tπ+fdarkorange", fill="magenta4", pen=pen)

fig.show()
```

Total running time of the script: (0 minutes 0.110 seconds)

8.3.12 Vertical or horizontal bars

The *pygmt.Figure.plot* method can plot vertical (**b**) or horizontal (**B**) bars by passing the corresponding shortcut to the `style` parameter. By default, *base* = 0 meaning that the bar is starting from 0. Append **+b**[*base*] to change this value. To plot multi-band bars, please append **+v**|**i***ny* (for vertical bars) or **+v**|**i***nx* (for horizontal ones), where *ny* or *nx* indicate the total number of bands in the bar (and hence the number of values required to follow the *x,y* coordinate pair in the input). Here, **+i** means we must accumulate the bar values from the increments *dy* or *dx*, while **+v** means we get the complete values relative to base. Normally, the bands are plotted as sections of a final single bar. Use **+s** to instead split the bar into *ny* or *nx* side-by-side, individual and thinner bars. Multi-band bars require `cmap=True` with one color per band.



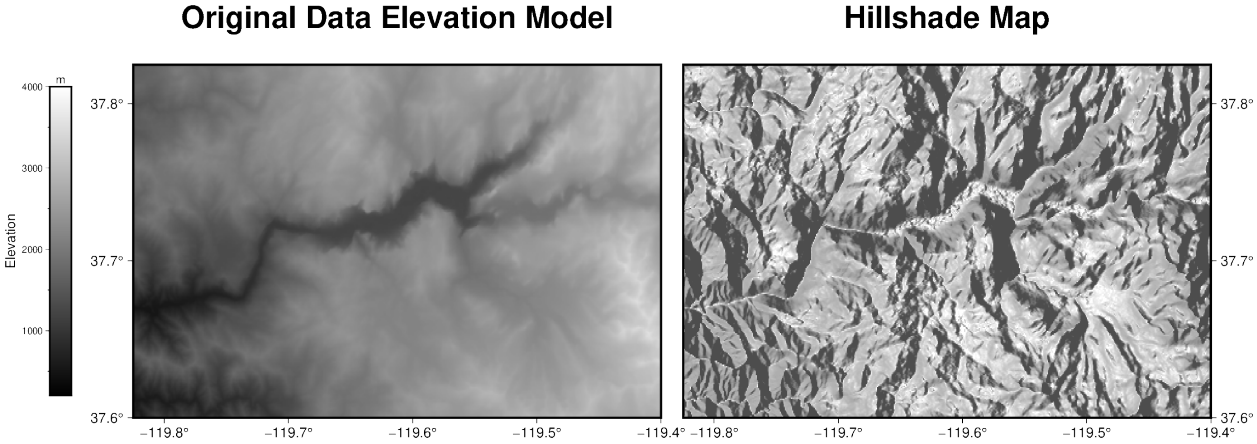
```
import pygmt
from pygmt.params import Axis, Frame

fig = pygmt.Figure()

pygmt.makecpt(cmap="SCM/roma", series=[0, 4, 1])

with fig.subplot(
    nrows=2,
    ncols=2,
    subsize=("8c", "4c"),
    frame=Axis(annot=True, grid=True),
    sharey=True,
    sharex=True,
    margins=["0.5c", "0.75c"],
):
    pen = "1.5p"
    with fig.set_panel(panel=0):
        fill = "skyblue"
        fig.basemap(region=[0, 4, 0, 3], frame=Frame(title="vertical bars"))
        fig.plot(x=1, y=2, style="b0.5c", fill=fill, pen=pen)
        fig.plot(x=2, y=2.5, style="b1c", fill=fill, pen=pen)
        # +b1 means that the bar is starting from y=1 here
        fig.plot(x=3, y=2.5, style="b0.75c+b1", fill=fill, pen=pen)

    with fig.set_panel(panel=1):
        fill = "tomato"
        fig.basemap(region=[0, 4, 0, 3], frame=Frame(title="horizontal bars"))
        fig.plot(x=1.5, y=0.5, style="B0.75c", fill=fill, pen=pen)
        fig.plot(x=3, y=1.5, style="B1c", fill=fill, pen=pen)
        # +b2 means that the bar is starting from x=2 here
        fig.plot(x=3.5, y=2.5, style="B0.5c+b2", fill=fill, pen=pen)

    # generate dictionary for plotting multi-band bars
    data = {
        "x1": [0.25, 1.25],
        "y": [1, 2],
        "x2": [0.65, 0.5],
        "x3": [0.4, 1.25],
        "x4": [2.25, 0.75],
    }

    with fig.set_panel(panel=2):
        fig.basemap(region=[0, 4, 0, 3], frame=Frame(title="stacked bars"))
        fig.plot(data=data, style="B0.75c+i4", cmap=True, pen=pen)

    with fig.set_panel(panel=3):
        fig.basemap(region=[0, 4, 0, 3], frame=Frame(title="split bars"))
        fig.plot(data=data, style="B1c+v4+s", cmap=True, pen=pen)

fig.show()
```

Total running time of the script: (0 minutes 0.179 seconds)

8.4 Images, contours, and fields

8.4.1 Calculating grid gradient and radiance

The *pygmt.grdgradient* function calculates the gradient of a grid file. In the example shown below we will see how to calculate a hillshade map based on a Data Elevation Model (DEM). As input *pygmt.grdgradient* gets an xarray.DataArray object or a path string to a grid file, calculates the respective gradient and returns it as an xarray.DataArray object. We will use the radiance parameter in order to set the illumination source direction and altitude.




```
import pygmt
from pygmt.params import Axis, Frame, Position

# Define region of interest around Yosemite valley
region = [-119.825, -119.4, 37.6, 37.825]

# Load sample grid (3 arc-seconds global relief) in target area
grid = pygmt.datasets.load_earth_relief(resolution="03s", region=region)

# calculate the reflection of a light source projecting from west to east
# (azimuth of 270 degrees) and at a latitude of 30 degrees from the horizon
dgrid = pygmt.grdgradient(grid=grid, radiance=[270, 30])

fig = pygmt.Figure()
# define figure configuration
pygmt.config(FORMAT_GEO_MAP="ddd.x", MAP_FRAME_TYPE="plain")

# --------------- plotting the original Data Elevation Model -----------

pygmt.makecpt(cmap="gmt/gray", series=[200, 4000, 10])
fig.grdimage(
    grid=grid,
    projection="M12c",
    frame=Frame(
        axes="WSrt",
        title="Original Data Elevation Model",
        xaxis=Axis(annot=0.1),
        yaxis=Axis(annot=0.1),
    ),
    cmap=True,
```

(continues on next page)

(continued from previous page)

```
)
fig.colorbar(
    position=Position("ML", cstype="outside", offset=(1.4, 0)),
    length=7,
    width=0.5,
    annot=1000,
    tick=500,
    label="Elevation",
    unit="m",
)

# --------------- plotting the hillshade map -----------

# Shift plot origin of the second map by 12.5 cm in x-direction
fig.shift_origin(xshift="12.5c")

pygmt.makecpt(cmap="gmt/gray", series=[-1.5, 0.3, 0.01])
fig.grdimage(
    grid=dgrid,
    projection="M12c",
    frame=Frame(
        axes="lSEt", title="Hillshade Map", xaxis=Axis(annot=0.1),␣
↪yaxis=Axis(annot=0.1)
    ),
    cmap=True,
)

fig.show()
```

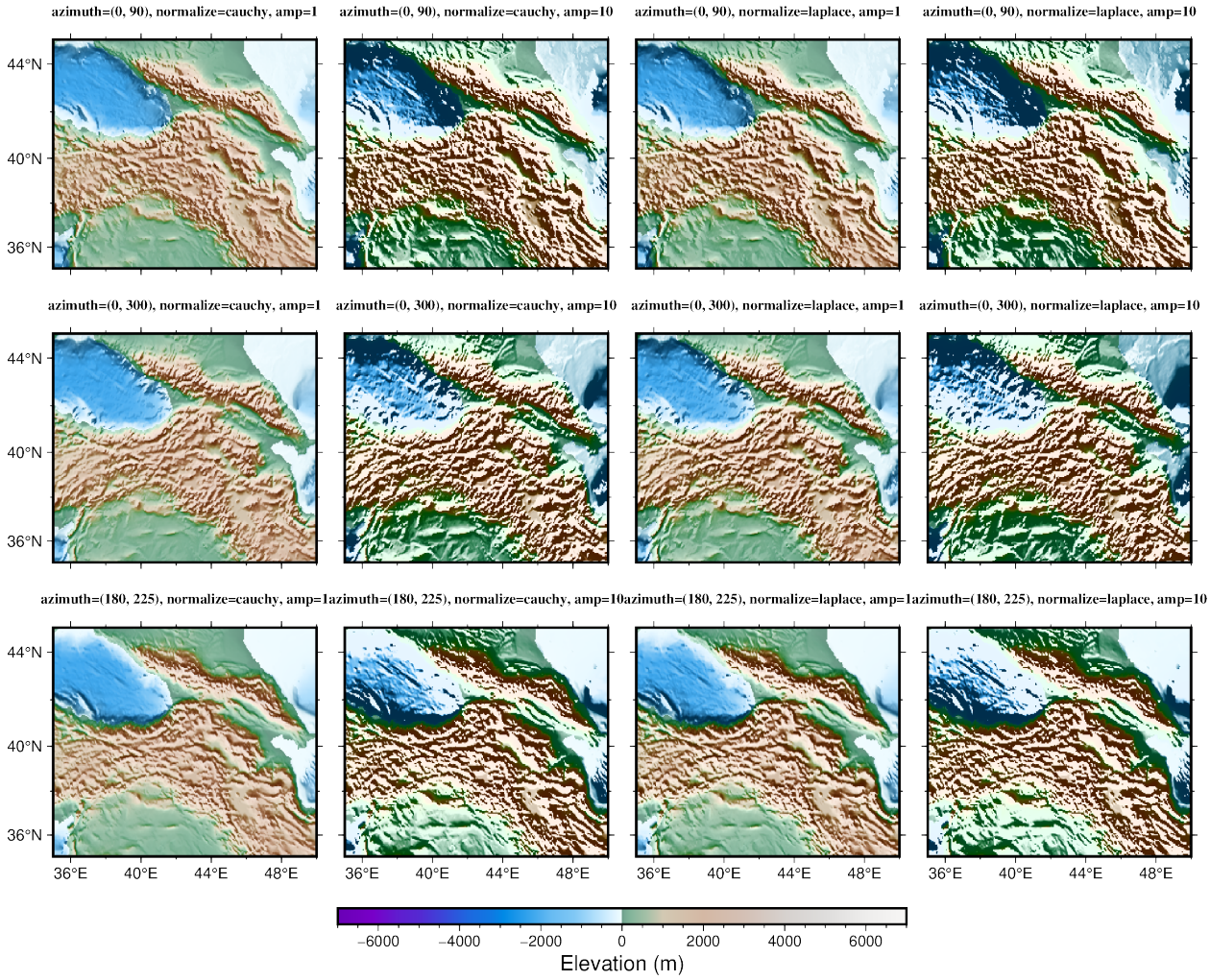
Total running time of the script: (0 minutes 0.368 seconds)

8.4.2 Calculating grid gradient with custom azimuth and normalization parameters

The *pygmt.grdgradient* function calculates the gradient of a grid file. As input, *pygmt.grdgradient* gets an xarray.DataArray object or a path string to a grid file. It then calculates the respective gradient and returns an xarray.DataArray object.

The `normalize` parameter calculates the azimuthal gradient of each point along a certain azimuth angle, then adjusts the brightness value of the color according to the positive/negative of the azimuthal gradient and the amplitude of each point. The example below shows how to customize the gradient by setting azimuth and normalization parameters.

- `azimuth` sets the illumination light source direction (0° is North, 90° is East, 180° is South, 270° is West).
- `normalize` sets the normalization method (e.g., Cauchy or Laplace distribution)
- `norm_amp` sets the amplitude of the normalization
- more parameters are available to further enhance the 3-D sense of the topography




```
grdblend [NOTICE]: Remote data courtesy of GMT data server␣
↪https://opentopography.s3.sdsc.edu/gmtdata␣
↪[https://opentopography.s3.sdsc.edu/gmtdata]
grdblend [NOTICE]: SRTM15 Earth Relief v2.7 at 03x03 arc minutes reduced by Gaussian␣
↪Cartesian filtering (15.7 km fullwidth) [Tozer et al., 2019].
grdblend [NOTICE]:   -> Download 90x90 degree grid tile (earth_relief_03m_g): N00E000
```

```
import pygmt
from pygmt.params import Axis, Frame, Position

# Load the 3 arc-minutes global relief grid in the target area around Caucasus
grid = pygmt.datasets.load_earth_relief(resolution="03m", region=[35, 50, 35, 45])

fig = pygmt.Figure()

# Define a colormap to be used for topography
```

(continues on next page)

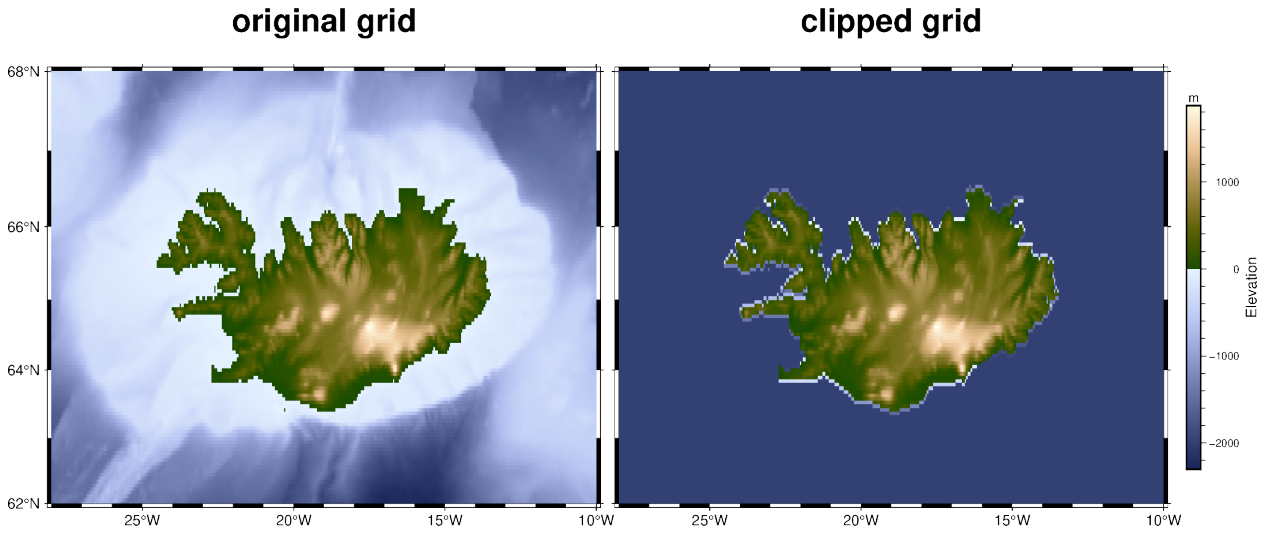
(continued from previous page)

```
pygmt.makecpt(cmap="gmt/terra", series=(-7000, 7000))

# Define figure configuration
pygmt.config(FONT_TITLE="10p,5", MAP_TITLE_OFFSET="1p", MAP_FRAME_TYPE="plain")

# Setup subplot panels with three rows and four columns
with fig.subplot(
    nrows=3,
    ncols=4,
    figsize=("28c", "21c"),
    sharex="b",
    sharey="l",
):
    # Setting azimuth angles, e.g. (0, 90) illuminates light source from the North␣
↪(top)
    # and East (right).
    for azi in [(0, 90), (0, 300), (180, 225)]:
        # "cauchy"/"laplace" sets cumulative Cauchy/Laplace distribution,␣
↪respectively.
        for normalize in ("cauchy", "laplace"):
            # amp (e.g., 1 or 10) controls the brightness value of the color.
            for amp in (1, 10):
                # Making an intensity DataArray using azimuth and normalize parameters
                shade = pygmt.grdgradient(
                    grid=grid, azimuth=azi, normalize=normalize, norm_amp=amp
                )
                fig.grdimage(
                    grid=grid,
                    shading=shade,
                    projection="M?",
                    frame=Frame(
                        title=f"azimuth={azi}, normalize={normalize}, amp={amp}",
                        axis=Axis(annot=4, tick=2),
                    ),
                    cmap=True,
                    panel=True,
                )

fig.colorbar(
    position=Position("BC", cstype="outside"),
    length=14,
    width=0.4,
    orientation="horizontal",
    annot=2000,
    tick=500,
    label="Elevation (m)",
)

fig.show()
```

Total running time of the script: (0 minutes 2.290 seconds)

8.4.3 Clipping grid values

The *pygmt.grdclip* function allows to clip defined ranges of grid values. In the example shown below we set all elevation values (grid points) smaller than 0 m (in general the bathymetric part of the grid) to a common value of -2000 m via the `below` parameter.




```
grdblend [NOTICE]: Remote data courtesy of GMT data server␣
↪https://opentopography.s3.sdsc.edu/gmtdata␣
↪[https://opentopography.s3.sdsc.edu/gmtdata]
grdblend [NOTICE]: SRTM15 Earth Relief v2.7 at 03x03 arc minutes reduced by Gaussian␣
↪Cartesian filtering (15.7 km fullwidth) [Tozer et al., 2019].
grdblend [NOTICE]:   -> Download 90x90 degree grid tile (earth_relief_03m_g): N00W090
```

```
import pygmt
from pygmt.params import Axis, Frame, Position

fig = pygmt.Figure()

# Define region of interest around Iceland
region = [-28, -10, 62, 68]

# Load sample grid (3 arc-minutes global relief) in target area
grid = pygmt.datasets.load_earth_relief(resolution="03m", region=region)

# Plot original grid
fig.basemap(
    region=region,
    projection="M12c",
    frame=Frame(
        axes="WSne",
        title="original grid",
        xaxis=Axis(annot=5, tick=1),
```

(continues on next page)

(continued from previous page)

```
        yaxis=Axis(annot=2, tick=1),
    ),
)
fig.grdimage(grid=grid, cmap="SCM/oleron")

# Shift plot origin of the second map by "width of the first map + 0.5 cm"
# in x-direction
fig.shift_origin(xshift="w+0.5c")

# Set all grid points < 0 m to a value of -2000 m.
grid = pygmt.grdclip(grid, below=[0, -2000])

# Plot clipped grid
fig.basemap(
    region=region,
    projection="M12c",
    frame=Frame(
        axes="wSne",
        title="clipped grid",
        xaxis=Axis(annot=5, tick=1),
        yaxis=Axis(annot=2, tick=1),
    ),
)
fig.grdimage(grid=grid)
fig.colorbar(
    label="Elevation",
    unit="m",
    position=Position("MR", cstype="outside", offset=(0.5, 0)),
    length=8,
)

fig.show()
```

Total running time of the script: (0 minutes 1.562 seconds)

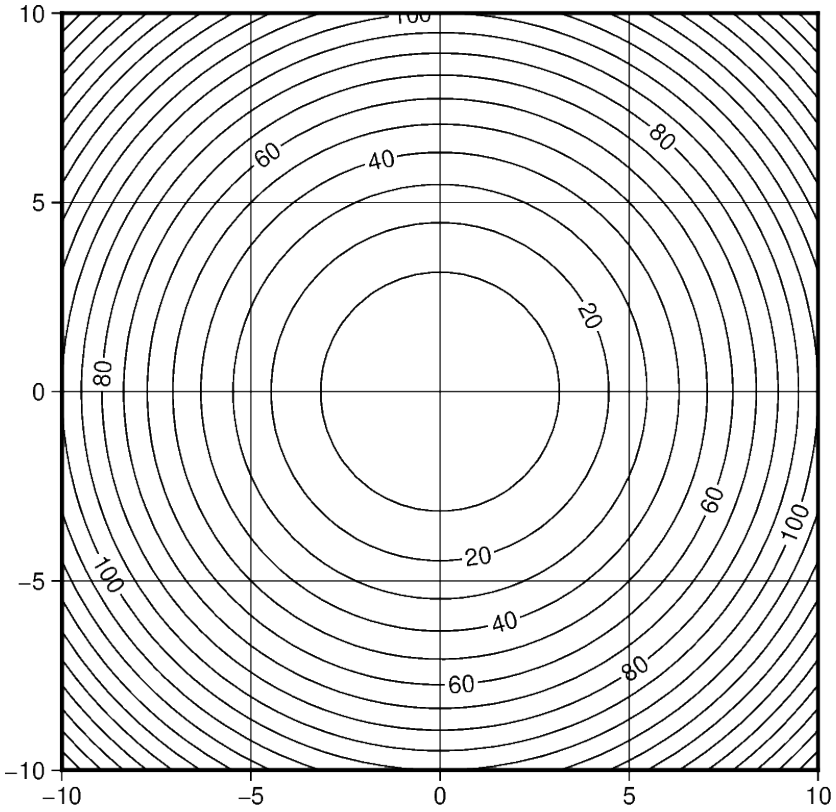
8.4.4 Contours

The *pygmt.Figure.contour* method can plot contour lines from a table of points by direct triangulation. The data for the triangulation can be provided using one of three methods:

1. `x`, `y`, `z` 1-D `numpy.ndarray` data columns.
2. `data` 2-D `numpy.ndarray` data matrix with 3 columns corresponding to `x`, `y`, `z`.
3. `data` path string to a file containing the `x`, `y`, `z` in a tabular format.

The parameters `levels` and `annotation` set the intervals of the contours and the annotations on the contours, respectively.

In this example we supply the data as 1-D `numpy.ndarray` with the `x`, `y`, and `z` parameters and draw the contours using a 0.5-points pen with contours every 10 z-values and annotations every 20 z-values.

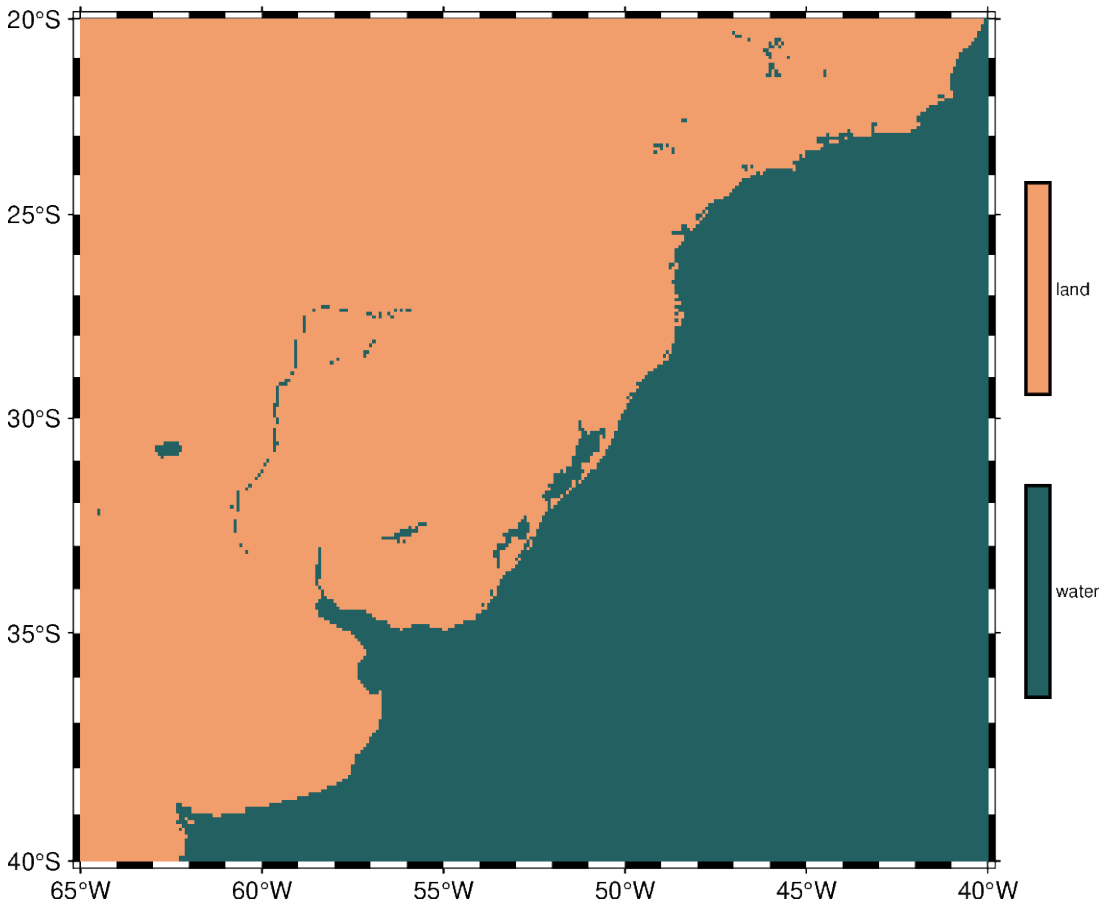



```
import numpy as np
import pygmt
from pygmt.params import Axis

# Build the contours underlying data with the function z = x^2 + y^2
X, Y = np.meshgrid(np.linspace(-10, 10, 50), np.linspace(-10, 10, 50))
Z = X**2 + Y**2
x, y, z = X.flatten(), Y.flatten(), Z.flatten()


fig = pygmt.Figure()
fig.contour(
    region=[-10, 10, -10, 10],
    projection="X10c/10c",
    frame=Axis(annot=True, grid=True),
    pen="0.5p",
    # Pass the data as 3 1-D data columns
    x=x,
    y=y,
    z=z,
    # Set the interval of the contours to 10
    levels=10,
    # Set the interval of the annotations to 20
    annotation=20,
)
fig.show()
```

Total running time of the script: (0 minutes 0.175 seconds)

8.4.5 Create 'wet-dry' mask grid

The *pygmt.grdlandmask* function allows setting all nodes on land or water to a specified value using the `mask_values` parameter.




```
grdimage [WARNING]: Your CPT is categorical. Enabling -nn+a to avoid interpolation␣
↪across categories.
```

```
import pygmt
from pygmt.params import Position

fig = pygmt.Figure()

# Define region of interest
region = [-65, -40, -40, -20]

# Assign a value of 0 for all water masses and a value of 1 for all land masses. Use
# shoreline data with low resolution and set the grid spacing to 5 arc-minutes in x-
# and y-directions.
grid = pygmt.grdlandmask(
```

(continues on next page)

(continued from previous page)

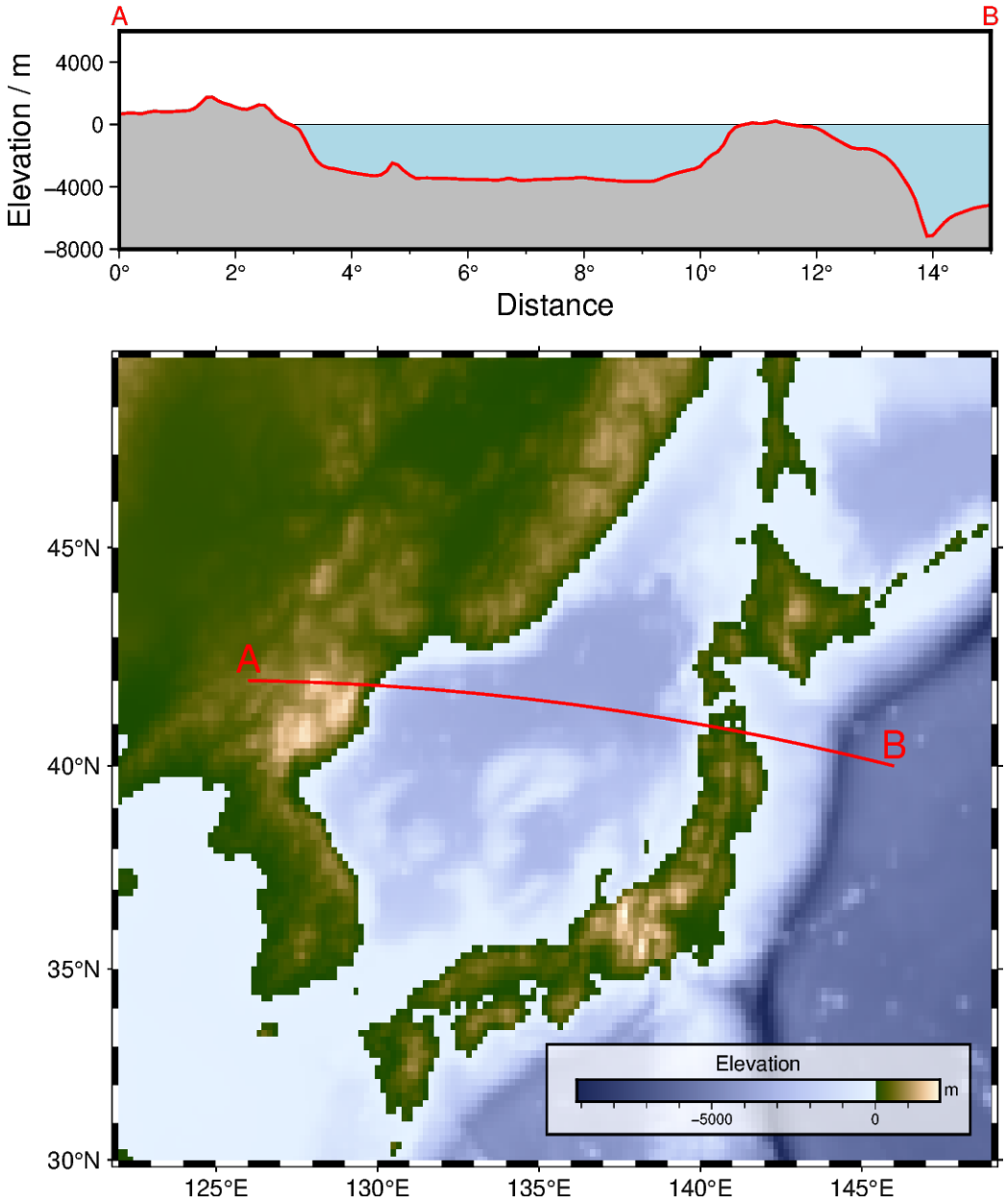
```
    region=region, spacing="5m", mask_values=[0, 1], resolution="low"
)

# Plot clipped grid
fig.basemap(region=region, projection="M12c", frame=True)

# Define a colormap to be used for two categories, define the range of the new␣
↪discrete
# CPT using series=(low, high, interval), use color_model="+cwater,land" to write the
# discrete color palette "SCM/batlow" in categorical format and add water/land as
# annotations for the colorbar.
pygmt.makecpt(cmap="SCM/batlow", series=(0, 1, 1), color_model="+cwater,land")

fig.grdimage(grid=grid, cmap=True)
fig.colorbar(position=Position("MR", cstype="outside", offset=(0.5, 0)), length=8)

fig.show()
```

Total running time of the script: (0 minutes 0.146 seconds)

8.4.6 Cross-section along a transect

pygmt.project and *pygmt.grdtrack* can be used to focus on a quantity and its variation along a desired survey line. In this example, the elevation is extracted from a grid provided via *pygmt.datasets.load_earth_relief*. The figure consists of two parts, a map of the elevation in the study area showing the survey line and a Cartesian plot showing the elevation along the survey line.

This example is orientated on an example in the GMT/China documentation: https://docs.gmt-china.org/latest/examples/ex026/




```
import pygmt
from pygmt.params import Axis, Box, Frame, Position

# Define region of study area
region_map = [122, 149, 30, 49]

# Chose a survey line with start point A and end point B
lonA, latA, lonB, latB = 126, 42, 146, 40  # ruff:␣
↪ignore[mixed-case-variable-in-global-scope]


fig = pygmt.Figure()
# --------------------------------------------------------------------------------
```

(continues on next page)

(continued from previous page)

```
# Bottom: Map of elevation in study area

# Set up basic map using a Mercator projection with a width of 12 centimeters
fig.basemap(region=region_map, projection="M12c", frame=Axis(annot=True, tick=True))

# Download grid for Earth relief with a resolution of 10 arc-minutes and gridline
# registration [Default]
grid_map = pygmt.datasets.load_earth_relief(resolution="10m", region=region_map)

# Plot the downloaded grid with color-coding based on the elevation
fig.grdimage(grid=grid_map, cmap="SCM/oleron")

# Add a colorbar for the elevation
fig.colorbar(
    # Place the colorbar inside the plot in the Bottom Right (BR) corner with an␣
↪offset
    # of 0.7 centimeters and 0.3 centimeters in x- or y-directions, respectively.
    position=Position("BR", offset=(0.7, 0.8)),
    length=5,
    width=0.3,
    orientation="horizontal",
    move_text="label",  # move the x-label above the horizontal colorbar.
    box=Box(pen="0.8p,black", fill="white@30"),
    label="Elevation",
    unit="m",
)

# Plot the survey line
fig.plot(x=[lonA, lonB], y=[latA, latB], pen="1p,red,solid")

# Add labels "A" and "B" for the start and end points of the survey line
fig.text(
    x=[lonA, lonB],
    y=[latA, latB],
    text=["A", "B"],
    offset=(0, 0.3),  # Move text 0.3 centimeters up (y-direction)
    font="15p,red",  # Use a red font with a size of 15 points
)

# ---------------------------------------------------------------------------
# Top: Elevation along survey line

# Shift plot origin to the top by the height of the map ("+h") plus 1.5 centimeters
fig.shift_origin(yshift="h+1.5c")

# Cartesian projection with a width of 12 centimeters and a height of 3 centimeters
fig.basemap(region=[0, 15, -8000, 6000], projection="X12c/3c", frame=0)

# Add labels "A" and "B" for the start and end points of the survey line
fig.text(
    x=[0, 15],
    y=[7000, 7000],
```

(continues on next page)

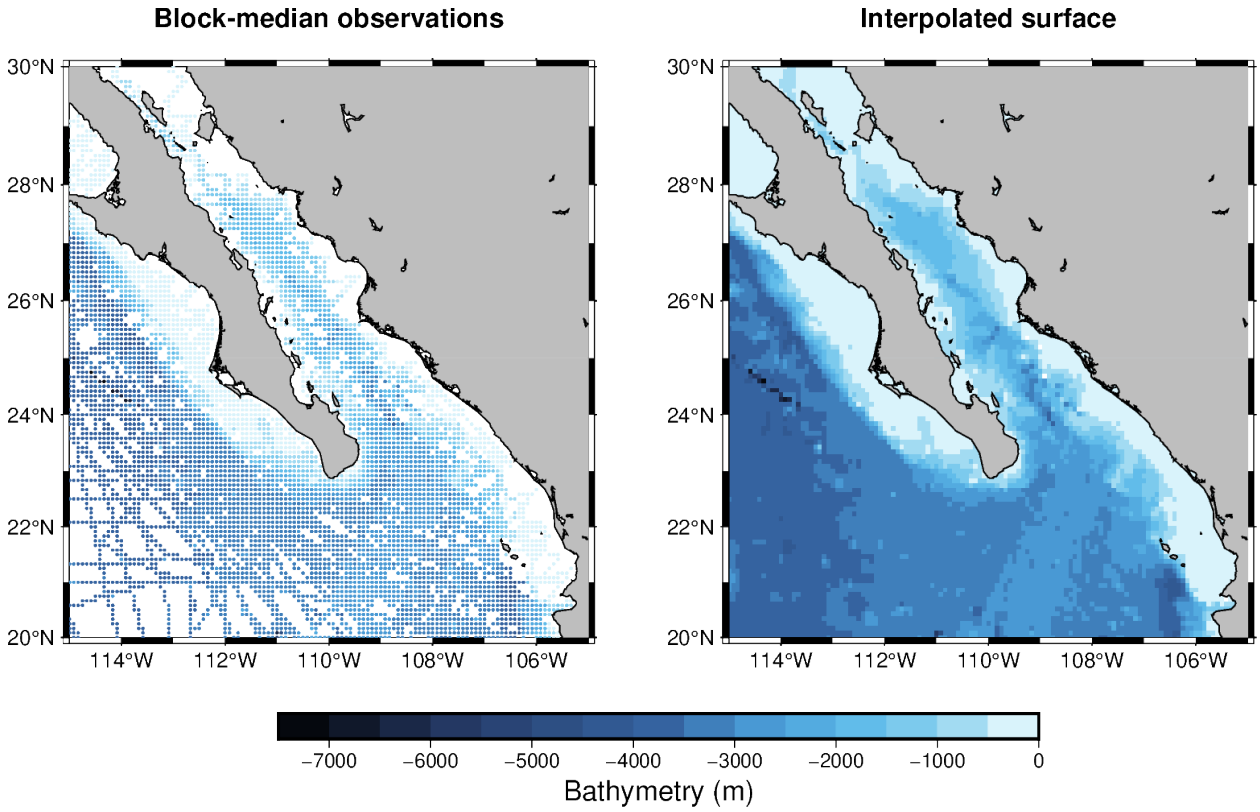
(continued from previous page)

```
    text=["A", "B"],
    no_clip=True,  # Do not clip text that fall outside the plot bounds
    font="10p,red",
)

# Generate points along a great circle corresponding to the survey line and store them
# in a pandas.DataFrame
track_df = pygmt.project(
    center=[lonA, latA],  # Start point of survey line (longitude, latitude)
    endpoint=[lonB, latB],  # End point of survey line (longitude, latitude)
    generate=0.1,  # Output data in steps of 0.1 degrees
)

# Extract the elevation at the generated points from the downloaded grid and add it as
# new column "elevation" to the pandas.DataFrame
track_df = pygmt.grdtrack(grid=grid_map, points=track_df, newcolname="elevation")

# Plot water masses down to -8000 meters below sea level as a light blue area.
fig.fill_between(x=[0, 15], y=[0, 0], y2=-8000, pen="0.25p", fill="lightblue")

# Plot elevation along the survey line, filled down to -8000 meters below sea level.
fig.fill_between(
    x=track_df.p, y=track_df.elevation, y2=-8000, pen="1p,red", fill="gray"
)

# Add plot frame
# Add annotations ("a") and ticks ("f") as well as labels ("+l") at the west or left
# and south or bottom sides ("WSrt")
fig.basemap(
    frame=Frame(
        axes="WSrt",
        xaxis=Axis(annot=2, tick=1, label="Distance", unit="°"),
        yaxis=Axis(annot=4000, label="Elevation / m"),
    )
)

fig.show()
```

Total running time of the script: (0 minutes 0.257 seconds)

8.4.7 Gridding scattered data

The *pygmt.surface* function interpolates irregularly spaced (x, y, z) observations onto a regular grid. Before interpolation, it's usually recommended to preprocess the data using one of *pygmt.blockmean*, *pygmt.blockmedian*, and *pygmt.blockmode*, to avoid aliasing short wavelengths, by returning one value for every occupied block.




```
import pygmt
from pygmt.params import Axis, Frame, Position

# Load irregular ship-track observations of bathymetry off Baja California.
data = pygmt.datasets.load_sample_data(name="bathymetry")

region = [245, 255, 20, 30]
spacing = "5m"  # 5 arc-minutes

# Calculate one representative median value for each occupied block.
reduced_data = pygmt.blockmedian(data=data, region=region, spacing=spacing,␣
↪center=True)

# Interpolate the reduced observations onto a regular grid. The value "d"
# constrains the grid to the minimum and maximum input data values.
grid = pygmt.surface(
    data=reduced_data, region=region, spacing=spacing, lower="d", upper="d"
)

fig = pygmt.Figure()
pygmt.makecpt(cmap="gmt/geo", series=[-7500, 0, 500])
pygmt.config(FONT_TITLE="12p", MAP_TITLE_OFFSET="4p")

with fig.subplot(nrows=1, ncols=2, figsize=("20c", "9c")):
    # Block-median observations.
    with fig.set_panel(panel=0):
        fig.basemap(
```

(continues on next page)

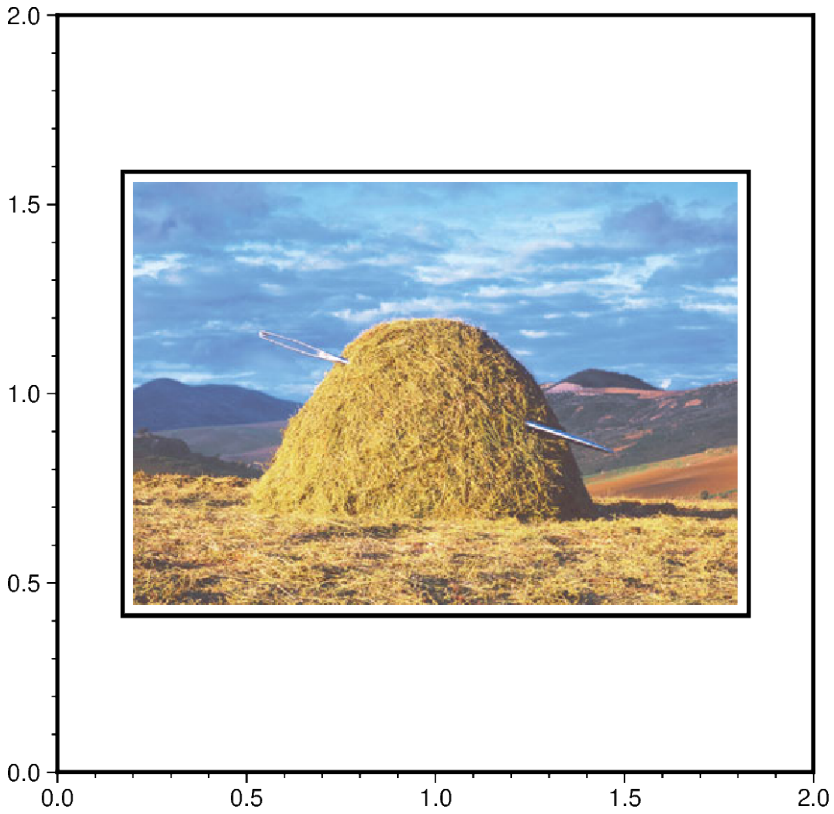
(continued from previous page)

```
            region=region,
            projection="M?",
            frame=Frame(
                axis=Axis(annot=2, tick=1),
                title="Block-median observations",
            ),
        )
        fig.plot(
            x=reduced_data.longitude,
            y=reduced_data.latitude,
            fill=reduced_data.bathymetry,
            style="c0.05c",
            cmap=True,
        )
        fig.coast(land="gray", shorelines="0.5p,black")

    # Interpolated surface grid.
    with fig.set_panel(panel=1):
        fig.grdimage(
            grid=grid,
            region=region,
            projection="M?",
            frame=Frame(
                axis=Axis(annot=2, tick=1),
                title="Interpolated surface",
            ),
            cmap=True,
        )
        fig.coast(land="gray", shorelines="0.5p,black")

# Position the color bar using explicit plot coordinates:
# 10 cm is the horizontal center of the 20 cm wide subplot.
fig.colorbar(
    position=Position(("10c", "-1.2c"), cstype="plotcoords", anchor="TC"),
    length=12,
    width=0.4,
    orientation="horizontal",
    annot=1000,
    label="Bathymetry (m)",
)

fig.show()
```

Total running time of the script: (0 minutes 0.400 seconds)

8.4.8 Image on a plot

The *pygmt.Figure.image* method can be used to read and place an image file in many formats (e.g., png, jpg, eps, pdf) on a plot. We must specify the filename via the `imagefile` parameter or simply use the filename as the first argument. You can also use a full URL pointing to your desired image. The `position` parameter allows us to place the image at a specific location on the plot.




```
from pathlib import Path

import pygmt
from pygmt.params import Position

fig = pygmt.Figure()
fig.basemap(region=[0, 2, 0, 2], projection="X10c", frame=True)

# Place the center of the image "needle.jpg" provided by GMT to the position (1, 1) on
# the current plot, scale it to a width of 8 centimeters and draw a rectangular border
# around it.
fig.image(
    imagefile=
↪"https://github.com/GenericMappingTools/gmtserver-admin/raw/master/cache/needle.jpg
↪",
    position=Position((1, 1), cstype="mapcoords", anchor="MC"),
    width="8c",
    box=True,
)

fig.show()

# Clean up the downloaded image in the current directory
Path("needle.jpg").unlink()
```

Total running time of the script: (0 minutes 1.386 seconds)

8.4.9 RGB image

The *pygmt.Figure.grdimage* method can be used to plot Red, Green, Blue (RGB) images, or any 3-band false color combination. Here, we'll use `rioxarray.open_rasterio` to read a GeoTIFF file into an `xarray.DataArray` format, and plot it on a map.

The example below shows a Worldview 2 satellite image over Lāhainā, Hawai'i during the August 2023 wildfires. Data is sourced from a Cloud-Optimized GeoTIFF (COG) file hosted on OpenAerialMap under a CC BY-NC 4.0 license.

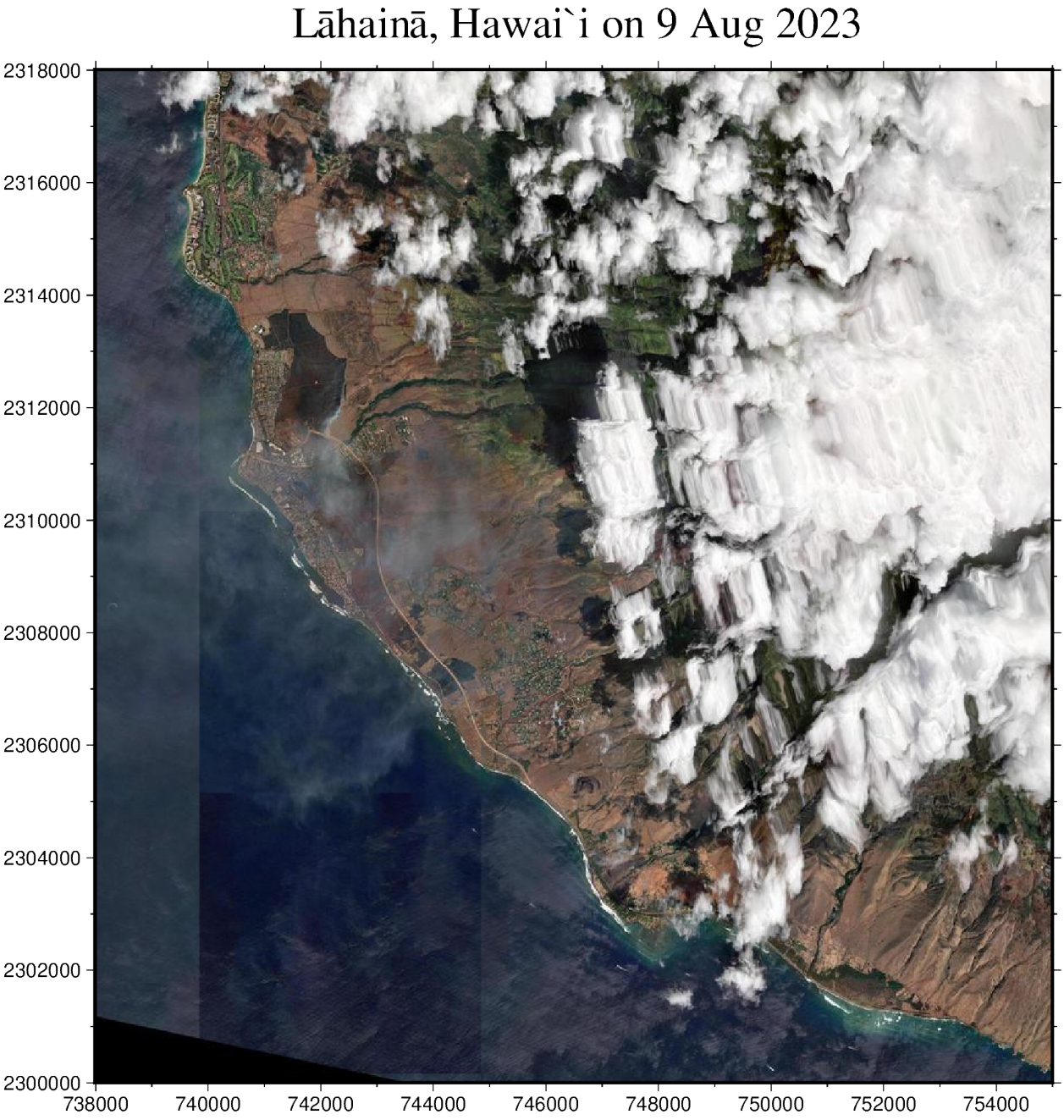
```
import pygmt
import rioxarray
from pygmt.params import Axis, Frame
```

Read 3-band data from GeoTIFF into an xarray.DataArray object:

```
with rioxarray.open_rasterio(
    filename=
↪"https://oin-hotosm-temp.s3.us-east-1.amazonaws.com/64d6a49a19cb3a000147a65b/0/64d6a49a19cb3a00014
↪",
    overview_level=5,
) as img:
    # Subset to area of Lāhainā in EPSG:32604 coordinates
    image = img.rio.clip_box(minx=738000, maxx=755000, miny=2300000, maxy=2318000)
    image = image.load()  # Force loading the DataArray into memory
image
```

Plot the RGB imagery:

```
fig = pygmt.Figure()
with pygmt.config(FONT_TITLE="Times-Roman"):  # Set title font to Times-Roman
    fig.grdimage(
        grid=image,
        # Use a map scale where 1 cm on the map equals 1 km on the ground
        projection="x1:100000",
        frame=Frame(
            axes="WSne",
            title=r"L@!a¯hain@!a¯, Hawai`i on 9 Aug 2023",
            axis=Axis(annot=True, tick=True),
        ),
    )
fig.show()
```

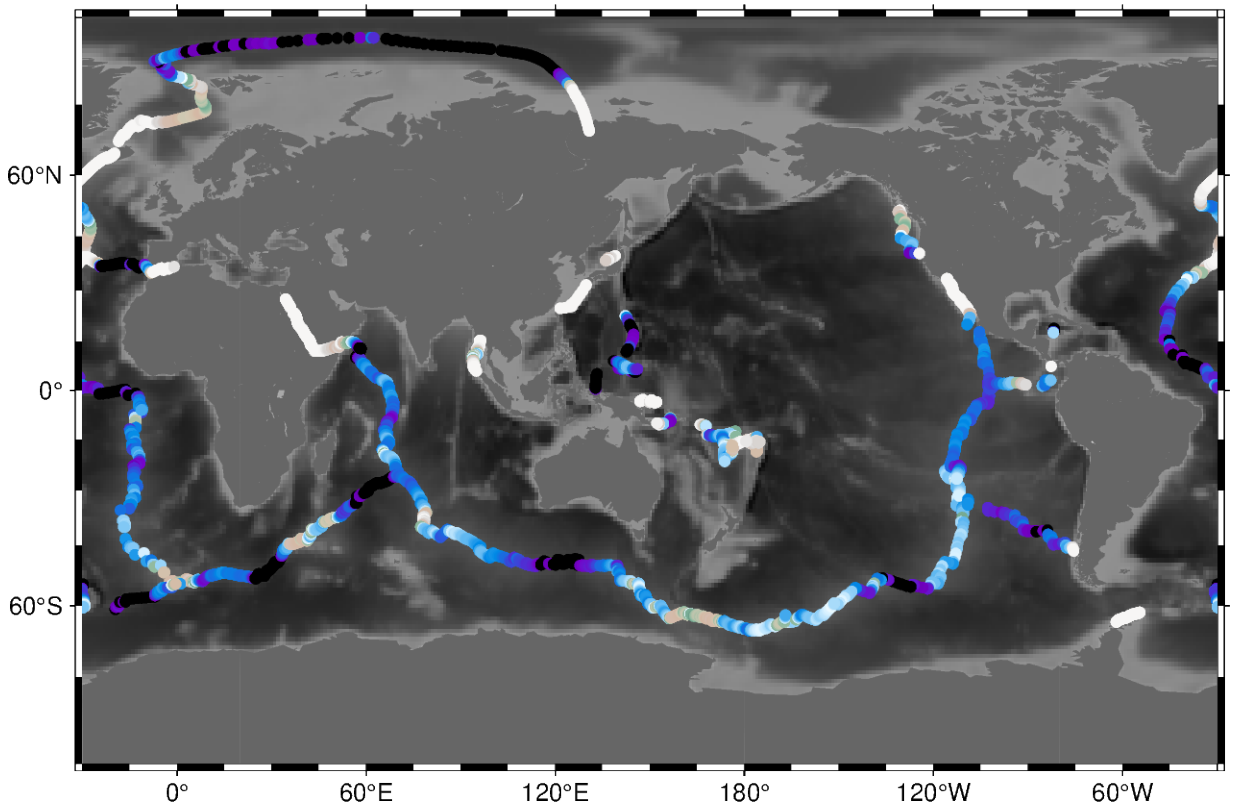


Total running time of the script: (0 minutes 4.014 seconds)

8.4.10 Sampling along tracks

The *pygmt.grdtrack* function samples a raster grid's value along specified points. We will need to input a 2-D raster to `grid` which can be an `xarray.DataArray`. The argument passed to the `points` parameter can be a `pandas.DataFrame` table where the first two columns are x and y (or longitude and latitude). Note also that there is a `newcolname` parameter that will be used to name the new column of values sampled from the grid.

Alternatively, a netCDF file path can be passed to `grid`. An ASCII file path can also be accepted for `points`. To save an output ASCII file, a file name argument needs to be passed to the `outfile` parameter.




```
import pygmt

# Load sample grid and point datasets
grid = pygmt.datasets.load_earth_relief()
points = pygmt.datasets.load_sample_data(name="ocean_ridge_points")
# Sample the bathymetry along the world's ocean ridges at specified track
# points
track = pygmt.grdtrack(points=points, grid=grid, newcolname="bathymetry")

fig = pygmt.Figure()
# Plot the earth relief grid on Cylindrical Stereographic projection, masking
# land areas
fig.basemap(region="g", projection="Cyl_stere/150/-20/15c", frame=True)
fig.grdimage(grid=grid, cmap="gmt/gray")
fig.coast(land="#666666")
# Plot the sampled bathymetry points using circles (c) of 0.15 cm size
# Points are colored using elevation values (normalized for visual purposes)
fig.plot(
    x=track.longitude,
    y=track.latitude,
    style="c0.15c",
    cmap="gmt/terra",
    fill=(track.bathymetry - track.bathymetry.mean()) / track.bathymetry.std(),
)
fig.show()
```

Total running time of the script: (0 minutes 0.286 seconds)

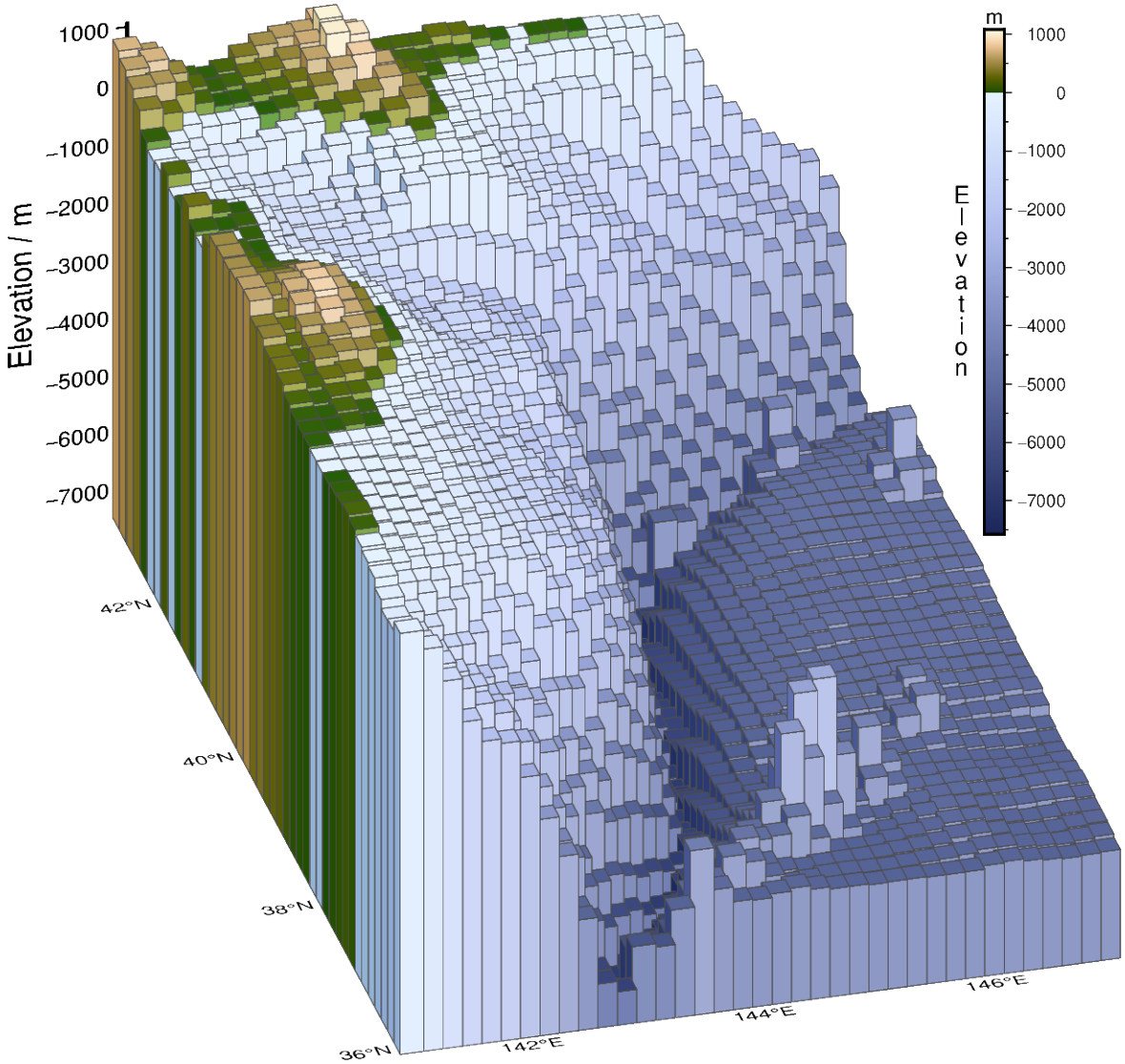
8.5 3-D Plots

8.5.1 3-D bar plot

A 3-D bar plot can be created from any collection of three-dimensional tabular data. The data points can lie on a regular grid or be irregularly scattered. A special case is creating such a 3-D bar plot based on a grid. This can be done in two steps:

1. Converting the grid into a table via *pygmt.grd2xyz*, with columns "x", "y", and "z" for longitude, latitude, and the quantity displayed by the grid, respectively.
2. Plotting this table as bars in 3-D using *pygmt.Figure.plot3d*.

The bars can be outlined, and the fill can be one color or based on a quantity using a colormap. For the latter, a fourth column needs to be added containing the values of the quantity for the color-coding.




```
import pygmt
from pygmt.params import Axis, Frame, Position
```

(continues on next page)

(continued from previous page)

```
# Define a study area around northern Japan with large elevation changes
region = [141, 147, 36, 43]

# Download a grid for the Earth relief with a resolution of 10 arc-minutes
grid = pygmt.datasets.load_earth_relief(resolution="10m", region=region)

# Convert the grid into a pandas DataFrame, with columns for longitude ("x"), latitude
# ("y") and elevation ("z")
grd_df = pygmt.grd2xyz(grid=grid)
zmin = grd_df["z"].min() - 50
zmax = grd_df["z"].max() + 50

# Add a fourth column "color" for the quantity used for the color-coding of the bars,
# here we use the elevation ("z")
grd_df["color"] = grd_df["z"]

# Create a 3-D bar plot with color-coding
fig = pygmt.Figure()

fig.basemap(
    region=[*region, zmin, zmax],
    projection="M10c",
    zsize="8c",
    frame=Frame(
        axes="WSneZ",
        xaxis=Axis(annot=True, tick=True),
        yaxis=Axis(annot=True, grid=True),
        zaxis=Axis(annot=1000, tick=500, label="Elevation / m"),
    ),
    perspective=(195, 30),
)

pygmt.makecpt(cmap="SCM/oleron", series=(zmin, zmax))
fig.plot3d(
    data=grd_df,
    # Use "o" to plot bars and give the desired size
    # The base of the bars is set via "+b"
    style=f"o0.34c+b{zmin}",
    cmap=True,
    pen="0.01p,gray30",
    perspective=True,
)
fig.colorbar(
    annot=1000,
    tick=500,
    label="Elevation",
    unit="m",
    position=Position("TR", cstype="inside", offset=1.4),
    orientation="vertical",
    length=7,
    move_text="label",
    label_as_column=True,
```

(continues on next page)

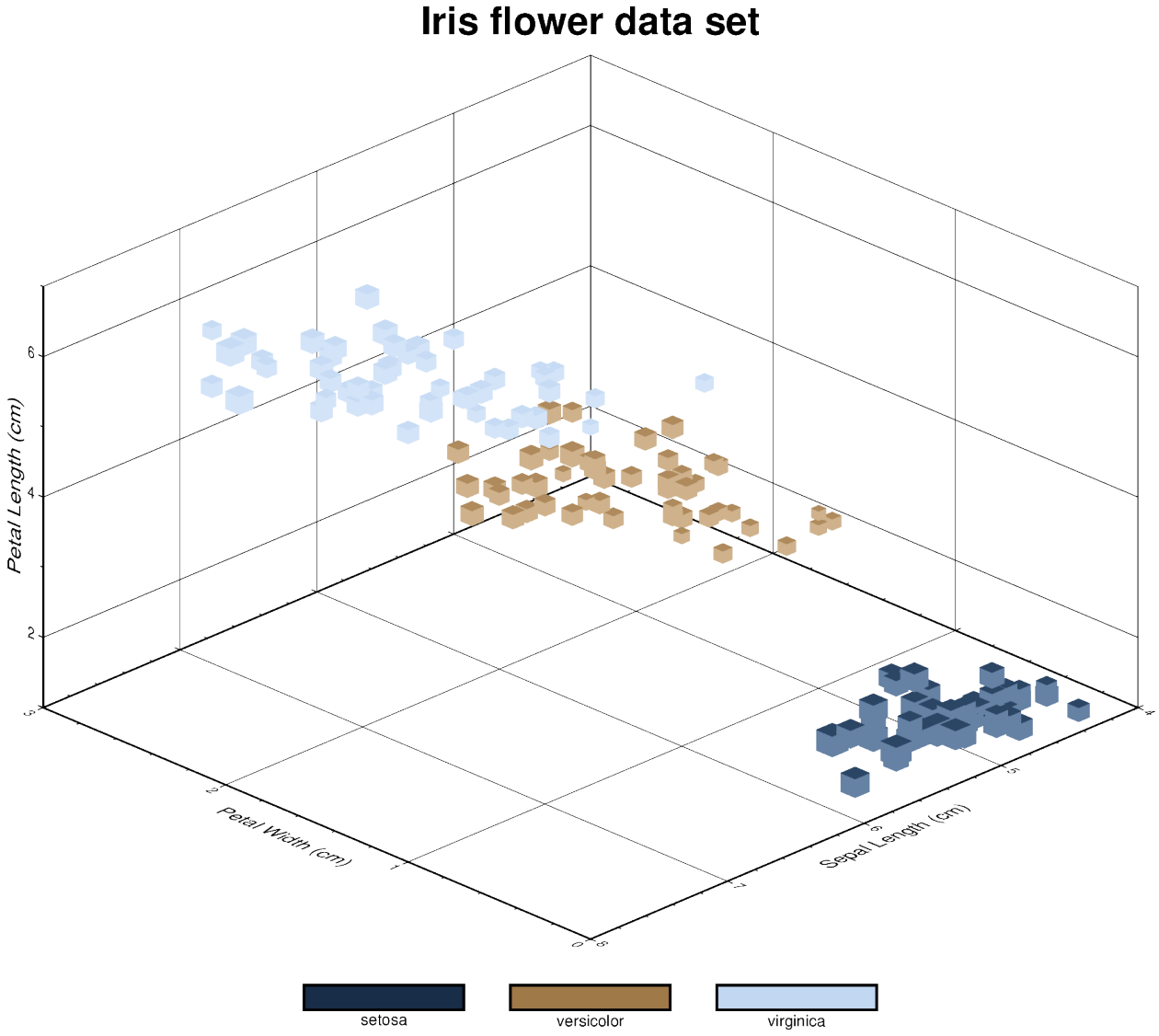
(continued from previous page)

```
)

fig.show()
```

Total running time of the script: (0 minutes 0.744 seconds)

8.5.2 3-D scatter plots

The *pygmt.Figure.plot3d* method can be used to plot symbols in 3-D. In the example below, we show how the Iris flower dataset can be visualized using a perspective 3-D plot. The `region` parameter has to include the x, y, z axis limits in the form of (xmin, xmax, ymin, ymax, zmin, zmax), which can be done automatically using *pygmt.info*. To plot the z-axis frame, set `frame` as a minimum to something like `Frame(axes="WsNeZ", zaxis=Axis(annot=True, tick=True))`. Use `perspective` to control the azimuth and elevation angle of the view, and `zscale` to adjust the vertical exaggeration factor.




```
import pandas as pd
import pygmt
```

(continues on next page)

(continued from previous page)

```
from pygmt.params import Axis, Frame

# Load sample iris data
df = pd.read_csv("https://github.com/mwaskom/seaborn-data/raw/master/iris.csv")

# Convert 'species' column to categorical dtype
# By default, pandas sorts the individual categories in an alphabetical order.
# For a non-alphabetical order, you have to manually adjust the list of
# categories. For handling and manipulating categorical data in pandas,
# have a look at:
# https://pandas.pydata.org/docs/user_guide/categorical.html
df.species = df.species.astype(dtype="category")

# Make a list of the individual categories of the 'species' column
# ['setosa', 'versicolor', 'virginica']
# They are (corresponding to the categorical number code) by default in
# alphabetical order and later used for the colorbar labels
labels = list(df.species.cat.categories)

# Use pygmt.info to get region bounds (xmin, xmax, ymin, ymax, zmin, zmax)
# The below example will return a numpy array [0.0, 3.0, 4.0, 8.0, 1.0, 7.0]
region = pygmt.info(
    data=df[["petal_width", "sepal_length", "petal_length"]],  # x, y, z columns
    per_column=True,  # Report the min/max values per column as a numpy array
    # Round the min/max values of the first three columns to the nearest
    # multiple of 1, 2 and 0.5, respectively
    spacing=(1, 2, 0.5),
)

# Make a 3-D scatter plot, coloring each of the 3 species differently
fig = pygmt.Figure()

# Define a colormap for three categories, define the range of the
# new discrete CPT using series=(lowest_value, highest_value, interval),
# use color_model="+csetosa,versicolor,virginica" to write the discrete color
# palette "cubhelix" in categorical format and add the species names as
# annotations for the colorbar
pygmt.makecpt(
    cmap="cpt-city/cubhelix",
    # Use the minimum and maximum of the categorical number code
    # to set the lowest_value and the highest_value of the CPT
    series=(df.species.cat.codes.min(), df.species.cat.codes.max(), 1),
    # Convert ['setosa', 'versicolor', 'virginica'] to
    # 'setosa,versicolor,virginica'
    color_model="+c" + ",".join(labels),
)

fig.plot3d(
    # Use petal width, sepal length, and petal length as x, y, and z
    # data input, respectively
    x=df.petal_width,
    y=df.sepal_length,
```

(continues on next page)

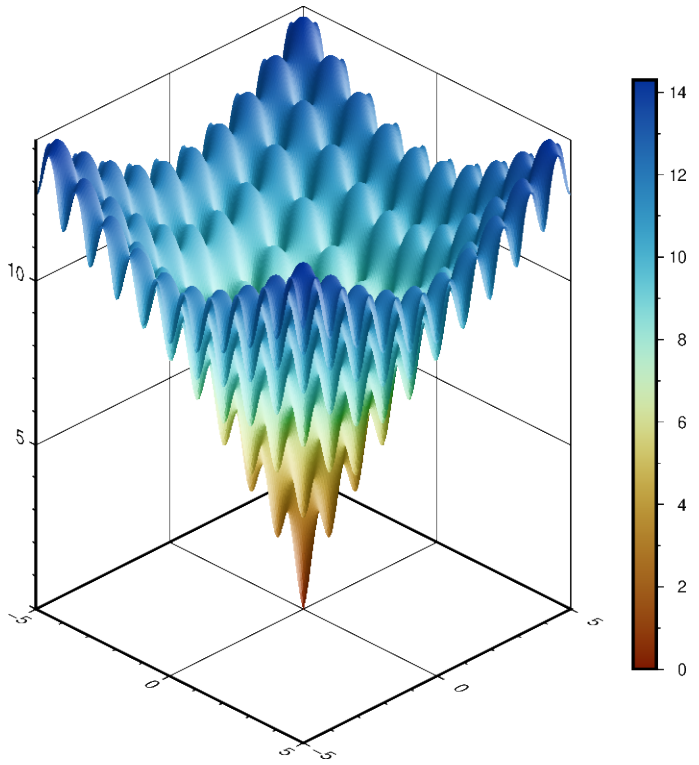
(continued from previous page)

```
    z=df.petal_length,
    # Vary each symbol size according to the sepal width, scaled by 0.1
    size=0.1 * df.sepal_width,
    # Use 3-D cubes ("u") as symbols with size in centimeters ("c")
    style="uc",
    # Points colored by categorical number code (refers to the species)
    fill=df.species.cat.codes.astype(int),
    # Use colormap created by makecpt
    cmap=True,
    # Set plot dimensions (xmin, xmax, ymin, ymax, zmin, zmax)
    region=region,
    # Set frame parameters
    frame=Frame(
        axes="WsNeZ3",  # z axis label positioned on 3rd corner
        title="Iris flower data set",
        xaxis=Axis(annot=True, tick=True, grid=True, label="Petal Width (cm)"),
        yaxis=Axis(annot=True, tick=True, grid=True, label="Sepal Length (cm)"),
        zaxis=Axis(annot=True, tick=True, grid=True, label="Petal Length (cm)"),
    ),
    # Set perspective to azimuth NorthWest (315°), at elevation 25°
    perspective=[315, 25],
    # Vertical exaggeration factor
    zscale=1.5,
)

# Shift the plot origin in x-direction temporarily and add the colorbar
with fig.shift_origin(xshift=3.1):
    fig.colorbar()

fig.show()
```

Total running time of the script: (0 minutes 1.083 seconds)

8.5.3 Plotting a surface

The *pygmt.Figure.grdview* method can plot 3-D surfaces with `surftype="surface"`. Here, we supply the data as an `xarray.DataArray` with the coordinate vectors `x` and `y` defined. Note that the `perspective` parameter here controls the azimuth and elevation angle of the view. Specifying the same scale for the `projection` and `zscale` parameters ensures equal axis scaling. The `shading` parameter specifies illumination; here we choose an azimuth of 45° with `shading="+a45"`.




```
import numpy as np
import pygmt
import xarray as xr
from pygmt.params import Axis, Frame, Position


# Define an interesting function of two variables, see:
# https://en.wikipedia.org/wiki/Ackley_function
def ackley(x, y):
    """
    Ackley function.
    """
    return (
        -20 * np.exp(-0.2 * np.sqrt(0.5 * (x**2 + y**2)))
        - np.exp(0.5 * (np.cos(2 * np.pi * x) + np.cos(2 * np.pi * y)))
        + np.exp(1)
        + 20
    )


# Create gridded data
INC = 0.05
x = np.arange(-5, 5 + INC, INC)
y = np.arange(-5, 5 + INC, INC)
data = xr.DataArray(ackley(*np.meshgrid(x, y)), coords=(x, y))

fig = pygmt.Figure()

# Plot grid as a 3-D surface
```

(continues on next page)

(continued from previous page)

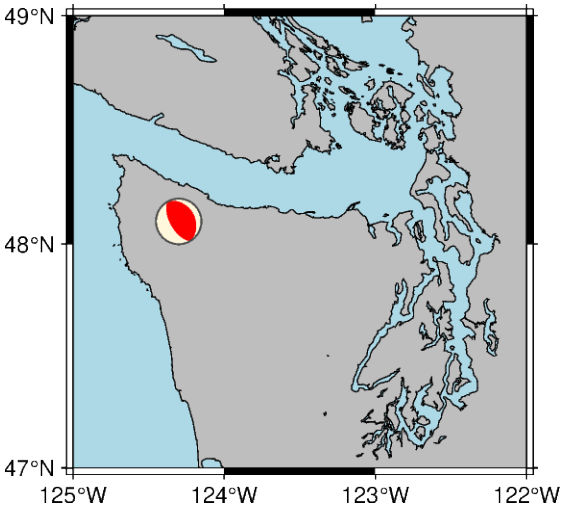
```
SCALE = 0.5  # in centimeters
fig.grdview(
    data,
    # Set annotations and gridlines in steps of five, and tick marks in steps of one
    frame=Frame(
        axis=Axis(annot=5, tick=1, grid=5),  # x and y axes
        zaxis=Axis(annot=5, tick=1, grid=5),
    ),
    projection=f"x{SCALE}c",
    zscale=f"{SCALE}c",
    surftype="surface",
    cmap="SCM/roma",
    perspective=[135, 30],  # Azimuth southeast (135°), at elevation 30°
    shading="+a45",
)

# Add colorbar for gridded data in the Middle Right corner.
fig.colorbar(annot=2, tick=1, position=Position("MR", cstype="outside"))

fig.show()
```

Total running time of the script: (0 minutes 2.311 seconds)

8.6 Seismology and geodesy

8.6.1 Focal mechanisms

The *pygmt.Figure.meca* method can plot focal mechanisms or beachballs. We can specify the focal mechanism nodal planes or moment tensor components as a dictionary using the `spec` parameter (or they can be specified as a 1-D or 2-D array, or within a file). The size of the beachballs can be set using the `scale` parameter. The compressive and extensive quadrants can be filled either with a color or a pattern via the `compression_fill` and `extension_fill` parameters, respectively. Use the `pen` parameter to adjust the outline of the beachballs.




```
import pygmt
from pygmt.params import Axis
```

(continues on next page)

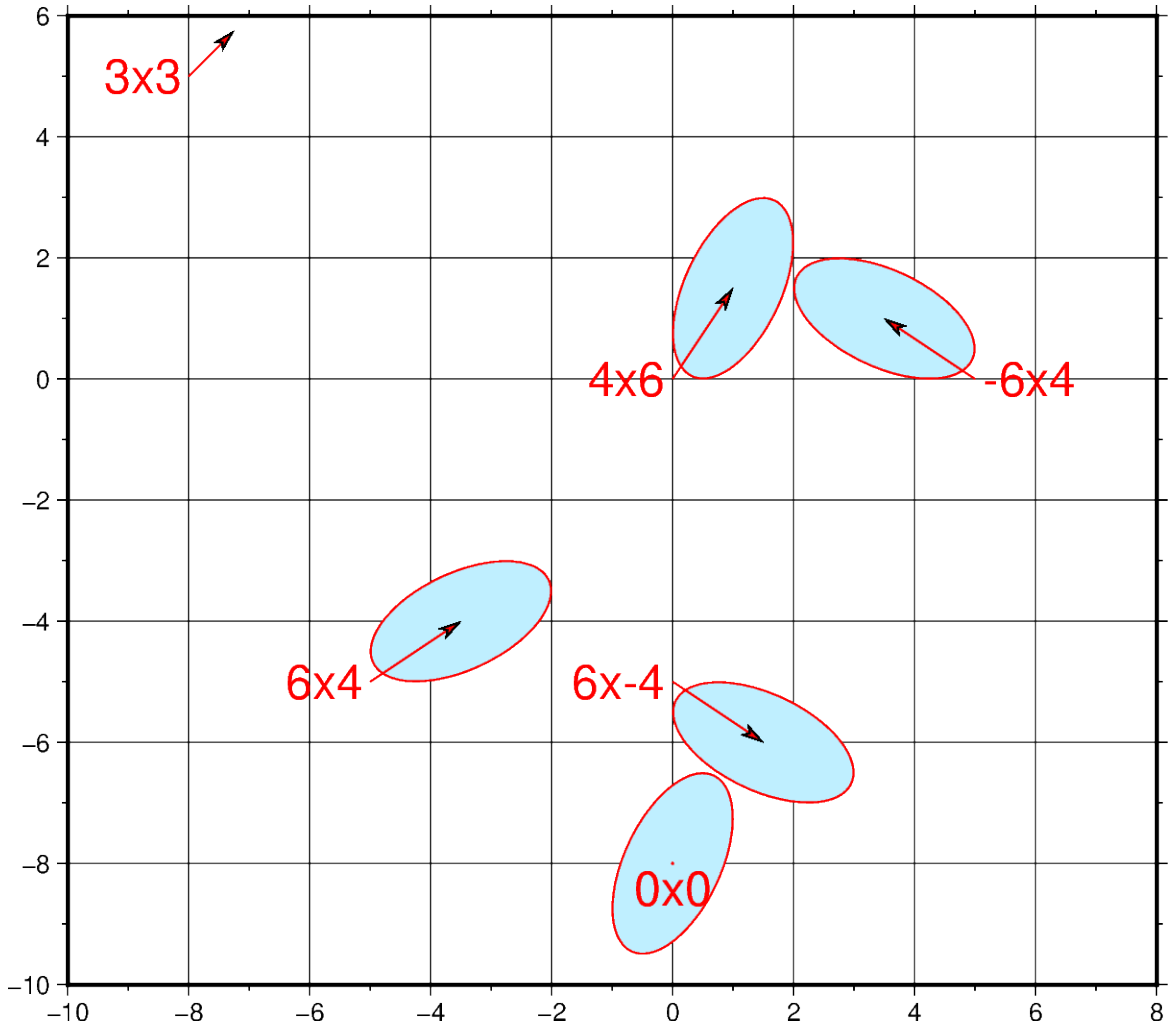
(continued from previous page)

```
fig = pygmt.Figure()

# Generate a map near Washington State showing land, water, and shorelines
fig.coast(
    region=[-125, -122, 47, 49],
    projection="M6c",
    land="grey",
    water="lightblue",
    shorelines=True,
    frame=Axis(annot=True),
)

# Store focal mechanism parameters in a dictionary based on the Aki & Richards
# convention
focal_mechanism = {"strike": 330, "dip": 30, "rake": 90, "magnitude": 3}

# Pass the focal mechanism data through the spec parameter. In addition provide
# scale, event location, and event depth
fig.meca(
    spec=focal_mechanism,
    scale="1c",  # in centimeters
    longitude=-124.3,
    latitude=48.1,
    depth=12.0,
    # Fill compressive quadrants with color "red"
    # [Default is "black"]
    compression_fill="red",
    # Fill extensive quadrants with color "cornsilk"
    # [Default is "white"]
    extension_fill="cornsilk",
    # Draw a 0.5-point thick dark gray ("gray30") solid outline via
    # the pen parameter [Default is "0.25p,black,solid"]
    pen="0.5p,gray30,solid",
)

fig.show()
```

Total running time of the script: (0 minutes 0.122 seconds)

8.6.2 Velocity arrows and confidence ellipses

The *pygmt.Figure.velo* method can be used to plot mean velocity arrows and confidence ellipses. The example below plots red velocity arrows with lightblue confidence ellipses outlined in red with the east_velocity x north_velocity used for the station names. Note that the velocity arrows are scaled by 0.2 and the 39% confidence limit will give an ellipse which fits inside a rectangle of dimension east_sigma by north_sigma.




```
import pandas as pd
import pygmt
from pygmt.params import Axis, Frame

fig = pygmt.Figure()
df = pd.DataFrame(
    data={
        "x": [0, -8, 0, -5, 5, 0],
        "y": [-8, 5, 0, -5, 0, -5],
        "east_velocity": [0, 3, 4, 6, -6, 6],
        "north_velocity": [0, 3, 6, 4, 4, -4],
        "east_sigma": [4, 0, 4, 6, 6, 6],
        "north_sigma": [6, 0, 6, 4, 4, 4],
        "correlation_EN": [0.5, 0.5, 0.5, 0.5, -0.5, -0.5],
        "SITE": ["0x0", "3x3", "4x6", "6x4", "-6x4", "6x-4"],
    }
)
fig.velo(
    data=df,
    region=[-10, 8, -10, 6],
```

(continues on next page)

(continued from previous page)

```
    projection="x0.8c",
    frame=Frame(axes="WSne", axis=Axis(annot=2, grid=2, tick=True)),
    spec="e0.2/0.39+f18",
    uncertainty_fill="lightblue1",
    pen="0.6p,red",
    line=True,
    vector="0.3c+p1p+e+gred",
)

fig.show()
```

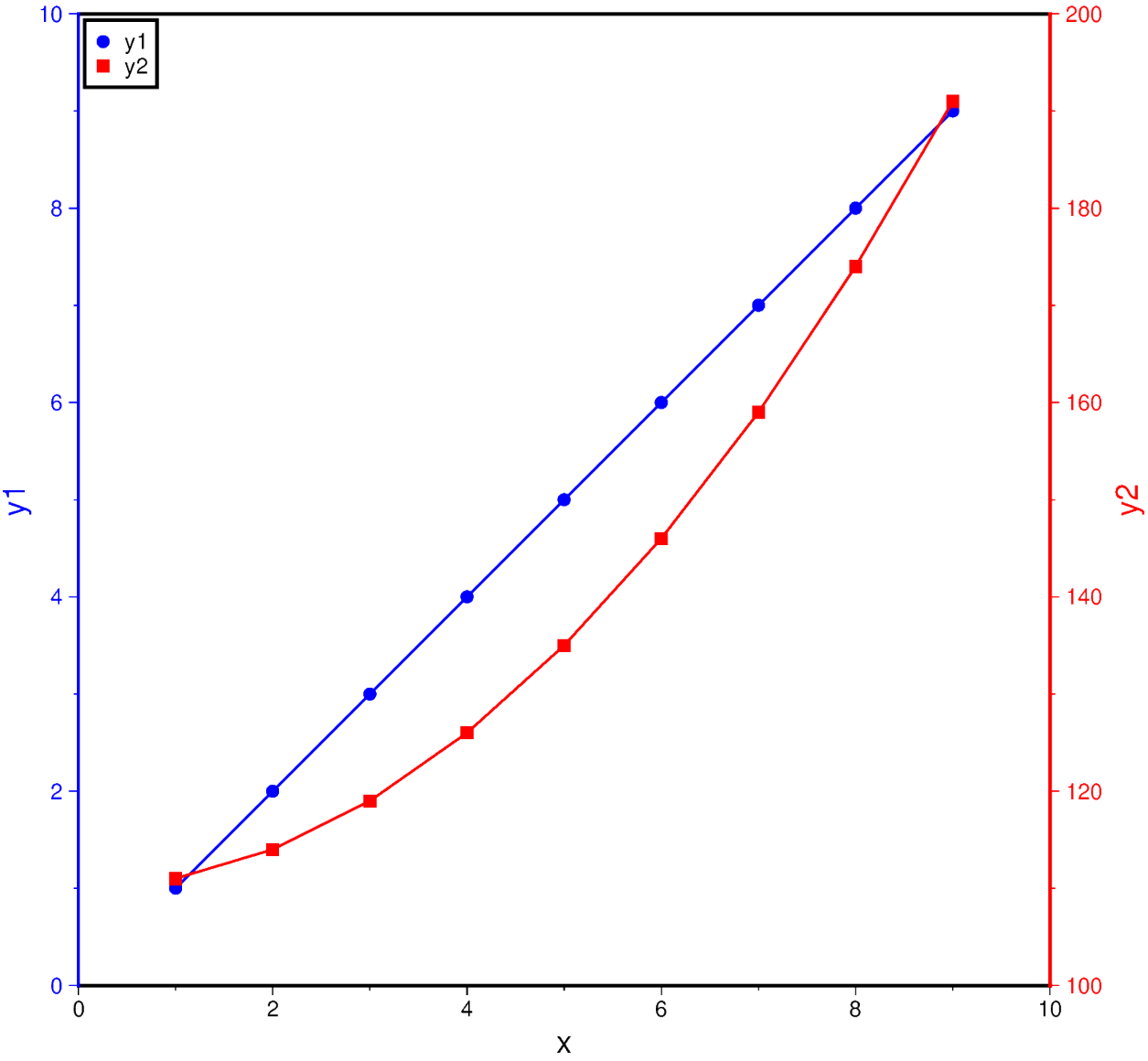
Total running time of the script: (0 minutes 0.164 seconds)

8.7 Base maps

8.7.1 Double Y-axes graph

The `frame` parameter of the plotting methods of the *`pygmt.Figure`* class can control which axes should be plotted and optionally show annotations, tick marks, and gridlines. By default, all 4 axes are plotted, along with annotations and tick marks (denoted **W**, **S**, **E**, **N**). Lowercase versions (**w**, **s**, **e**, **n**) can be used to denote to only plot the axes with tick marks. We can also only plot the axes without annotations and tick marks using **l** (left axis), **r** (right axis), **t** (top axis), **b** (bottom axis). When `frame` is used to change the frame settings, any axes that are not defined using one of these three options are not drawn.

To plot a double Y-axes graph using PyGMT, we need to plot at least two base maps separately. The base maps should share the same projection parameter and x-axis limits, but different y-axis limits.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame, Position

# Generate two sample Y-data from one common X-data
x = np.linspace(1.0, 9.0, num=9)
y1 = x
y2 = x**2 + 110

fig = pygmt.Figure()

# Plot the common X-axes
# The bottom axis (S) is plotted with annotations and tick marks
# The top axis (t) is plotted without annotations and tick marks
# The left and right axes are not drawn
fig.basemap(
```

(continues on next page)

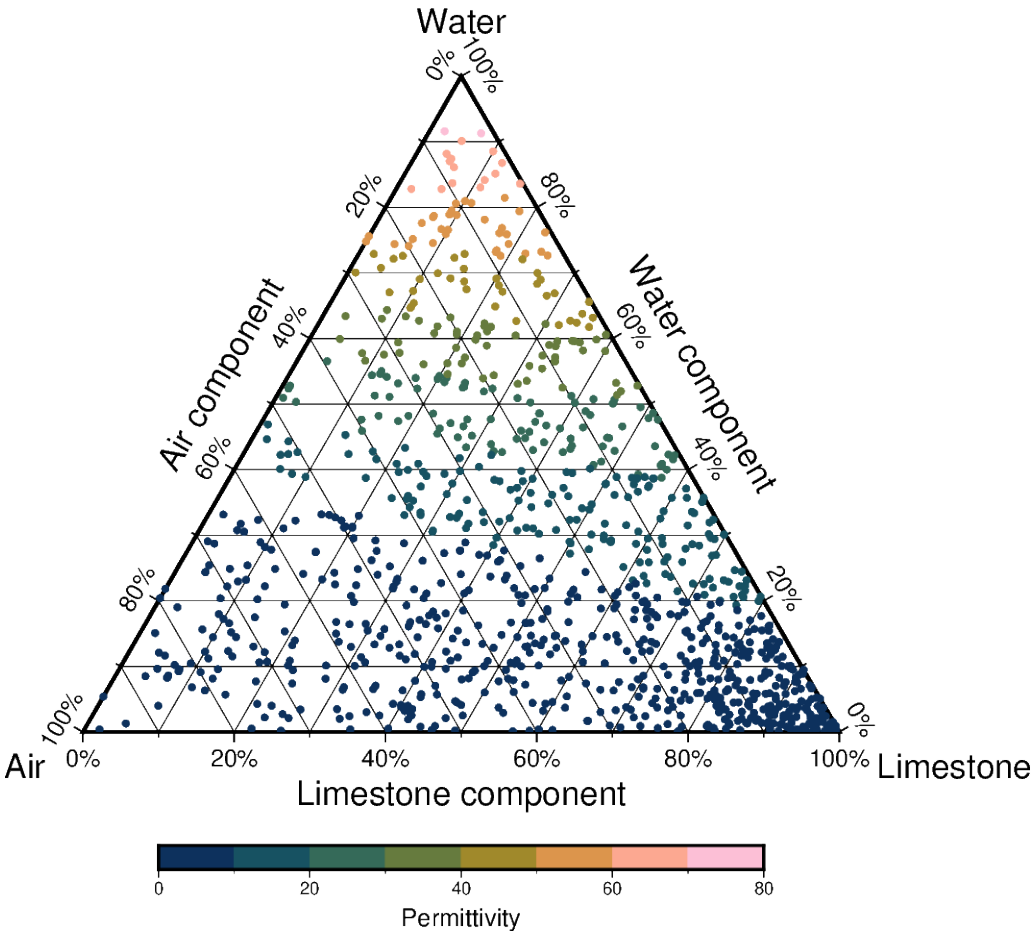
(continued from previous page)

```
    region=[0, 10, 0, 10],
    projection="X15c/15c",
    frame=Frame(axes="St", xaxis=Axis(annot=True, tick=True, label="x")),
)

# Plot the Y-axis for y1-data
# The left axis (W) is plotted with blue annotations, ticks, and label
with pygmt.config(
    MAP_FRAME_PEN="blue",
    MAP_TICK_PEN="blue",
    FONT_ANNOT_PRIMARY="blue",
    FONT_LABEL="blue",
):
    fig.basemap(frame=Frame(axes="W", yaxis=Axis(annot=True, tick=True, label="y1")))

# Plot the line for y1-data
fig.plot(x=x, y=y1, pen="1p,blue")
# Plot points for y1-data
fig.plot(x=x, y=y1, style="c0.2c", fill="blue", label="y1")

# Plot the Y-axis for y2-data
# The right axis (E) is plotted with red annotations, ticks, and label
with pygmt.config(
    MAP_FRAME_PEN="red",
    MAP_TICK_PEN="red",
    FONT_ANNOT_PRIMARY="red",
    FONT_LABEL="red",
):
    fig.basemap(
        region=[0, 10, 100, 200],
        frame=Frame(axes="E", yaxis=Axis(annot=True, tick=True, label="y2")),
    )
# Plot the line for y2-data
fig.plot(x=x, y=y2, pen="1p,red")
# Plot points for y2-data
fig.plot(x=x, y=y2, style="s0.28c", fill="red", label="y2")

# Create a legend in the Top Left (TL) corner of the plot with a 0.1-cm offset.
fig.legend(position=Position("TL", offset=0.1), box=True)

fig.show()
```

Total running time of the script: (0 minutes 0.191 seconds)

8.7.2 Ternary diagram

The *pygmt.Figure.ternary* method can draw ternary diagrams. The example shows how to plot circles with a diameter of 0.1 centimeters (`style="c0.1c"`) on a 10-centimeters-wide (`width="10c"`) ternary diagram at the positions listed in the first three columns of the sample dataset `rock_compositions`, with default annotations and gridline spacings, using the specified labeling defined via `alabel`, `blabel`, and `clabel`. Points are colored based on the values given in the fourth column of the sample dataset via `cmap=True`.




```
import pygmt
from pygmt.params import Axis, Frame, Position

fig = pygmt.Figure()

# Load sample data
data = pygmt.datasets.load_sample_data(name="rock_compositions")

# Define a colormap to be used for the values given in the fourth column
# of the input dataset
pygmt.makecpt(cmap="SCM/batlow", series=[0, 80, 10])

fig.ternary(
    data,
    region=[0, 100, 0, 100, 0, 100],
    width="10c",
    style="c0.1c",
    alabel="Limestone",
    blabel="Water",
    clabel="Air",
    cmap=True,
    frame=Frame(
        xaxis=Axis(
```

(continues on next page)

(continued from previous page)

```
            annot=True, tick=True, grid=True, label="Limestone component", unit="%"
        ),
        yaxis=Axis(annot=True, tick=True, grid=True, label="Water component", unit="%
↪"),
        zaxis=Axis(annot=True, tick=True, grid=True, label="Air component", unit="%"),
    ),
)

# Add a colorbar indicating the values given in the fourth column of the input dataset
fig.colorbar(
    position=Position("BC", cstype="outside", offset=(0, 1.5)), label="Permittivity"
)
fig.show()
```

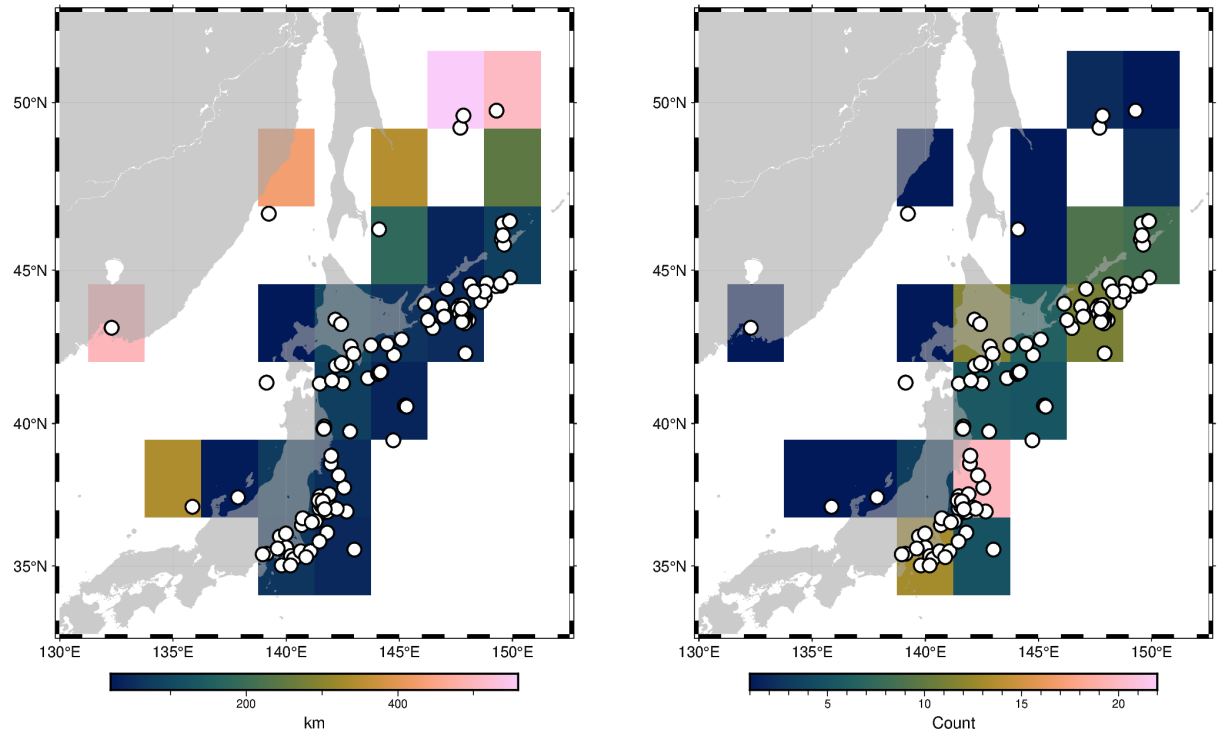
Total running time of the script: (0 minutes 0.161 seconds)

8.8 Histograms

8.8.1 Blockmean

The *pygmt.blockmean* function calculates different quantities inside blocks/bins whose dimensions are defined via the `spacing` parameter. The following examples show how to calculate the averages of the given values inside each bin and how to report the number of points inside each bin.

Mean earthquake depth inside each block **Number of points inside each block**




```
import pygmt
from pygmt.params import Axis, Frame
```

(continues on next page)

(continued from previous page)

```
# Load sample data
data = pygmt.datasets.load_sample_data(name="japan_quakes")
# Select only needed columns
data = data[["longitude", "latitude", "depth_km"]]

# Set the region for the plot
region = [130, 152.5, 32.5, 52.5]
# Define spacing in x- and y-directions (150x150 arc-minute blocks)
spacing = "150m"

fig = pygmt.Figure()

# Calculate mean depth in kilometers from all events within
# 150x150 arc-minute bins using blockmean
df = pygmt.blockmean(data=data, region=region, spacing=spacing)
# Convert to grid
grd = pygmt.xyz2grd(data=df, region=region, spacing=spacing)

fig.basemap(
    region=region,
    projection="M11c",
    frame=Frame(
        axis=Axis(annot=True, tick=True),
        title="Mean earthquake depth inside each block",
    ),
)
fig.grdimage(grid=grd, cmap="SCM/batlow")
# Plot slightly transparent landmasses on top
fig.coast(land="darkgray", transparency=40)
# Plot original data points
fig.plot(x=data.longitude, y=data.latitude, style="c0.3c", fill="white", pen="1p,black
↪")
fig.colorbar(label="km")

fig.shift_origin(xshift="w+5c")

# Calculate number of total locations within 150x150 arc-minute bins
# with blockmean's summary parameter
df = pygmt.blockmean(data=data, region=region, spacing=spacing, summary="n")
grd = pygmt.xyz2grd(data=df, region=region, spacing=spacing)

fig.basemap(
    region=region,
    projection="M11c",
    frame=Frame(
        axis=Axis(annot=True, tick=True), title="Number of points inside each block"
    ),
)
fig.grdimage(grid=grd, cmap="SCM/batlow")
fig.coast(land="darkgray", transparency=40)
fig.plot(x=data.longitude, y=data.latitude, style="c0.3c", fill="white", pen="1p,black
```

(continues on next page)

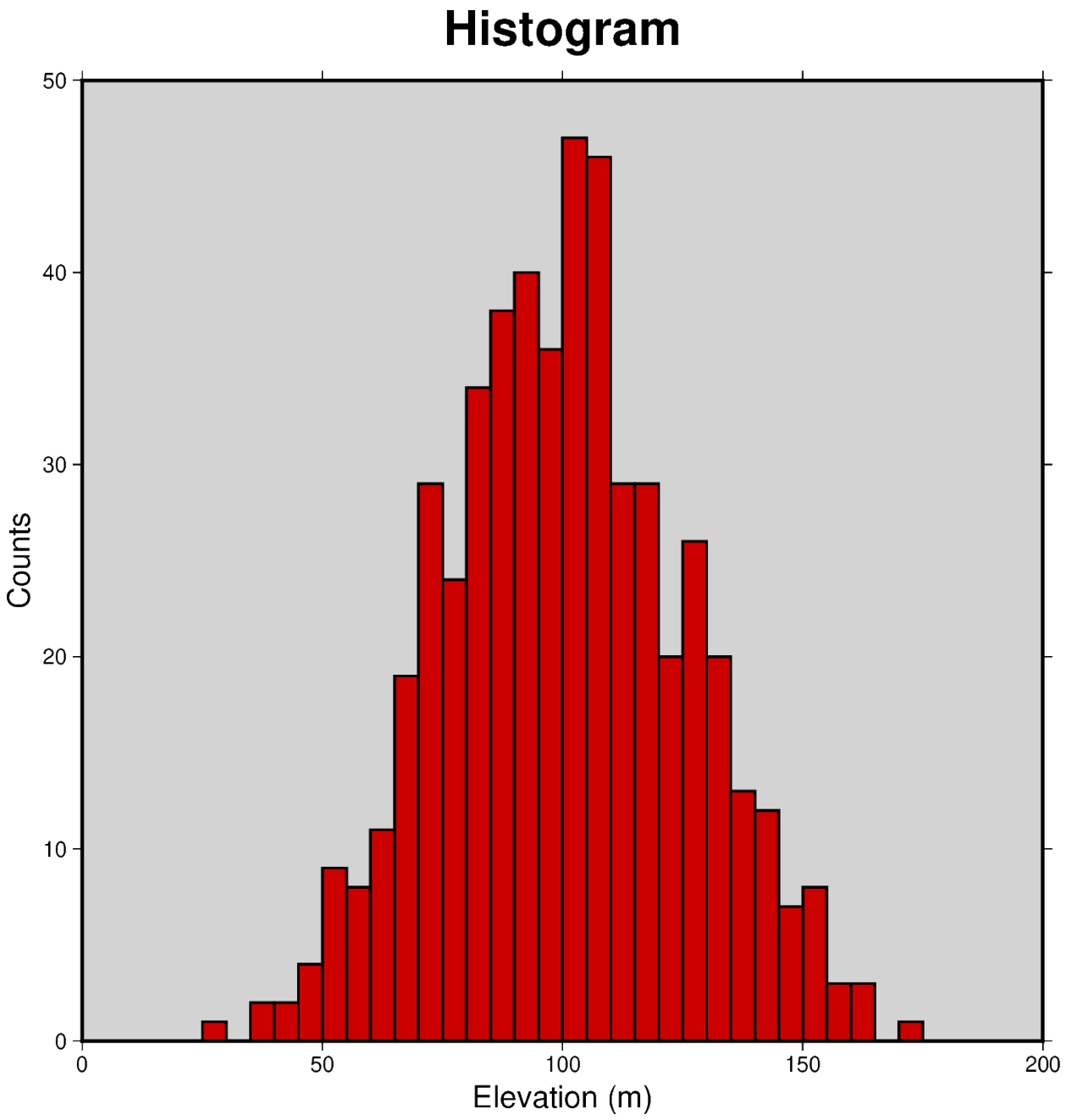
(continued from previous page)

```
↪")
fig.colorbar(label="Count")

fig.show()
```

Total running time of the script: (0 minutes 0.423 seconds)

8.8.2 Histogram

The *pygmt.Figure.histogram* method can plot regular histograms. Using the `series` parameter allows to set the interval for the width of each bar. The quantity each bar represents (a count or a percentage) can be selected via the `histtype` parameter.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# Generate random elevation data from a normal distribution
rng = np.random.default_rng(seed=100)
mean = 100  # mean of distribution
stddev = 25  # standard deviation of distribution
data = rng.normal(loc=mean, scale=stddev, size=521)
```

(continues on next page)

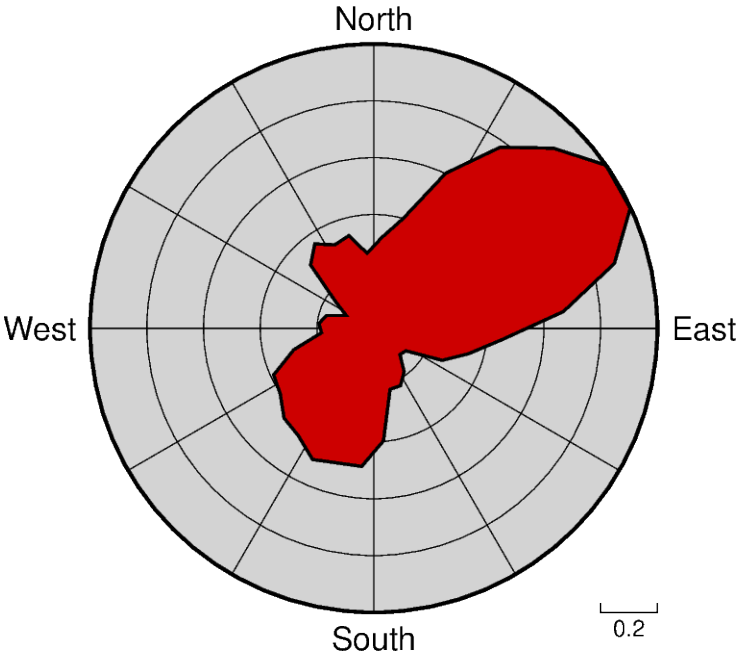
(continued from previous page)

```
fig = pygmt.Figure()

fig.histogram(
    data=data,
    # Define the frame, add a title, and set the background color to "lightgray".
    # Add labels to the x-axis and y-axis
    frame=Frame(
        axes="WSne",
        title="Histogram",
        fill="lightgray",
        xaxis=Axis(label="Elevation (m)"),
        yaxis=Axis(label="Counts"),
    ),
    # Generate evenly spaced bins by increments of 5
    series=5,
    # Use "red3" as color fill for the bars
    fill="red3",
    # Use the pen parameter to draw the outlines with a width of 1 point
    pen="1p",
)

fig.show()
```

Total running time of the script: (0 minutes 0.168 seconds)

8.8.3 Rose diagram

The *pygmt.Figure.rose* method can plot windrose diagrams or polar histograms.




```
import pygmt
from pygmt.params import Axis, Frame
```

(continues on next page)

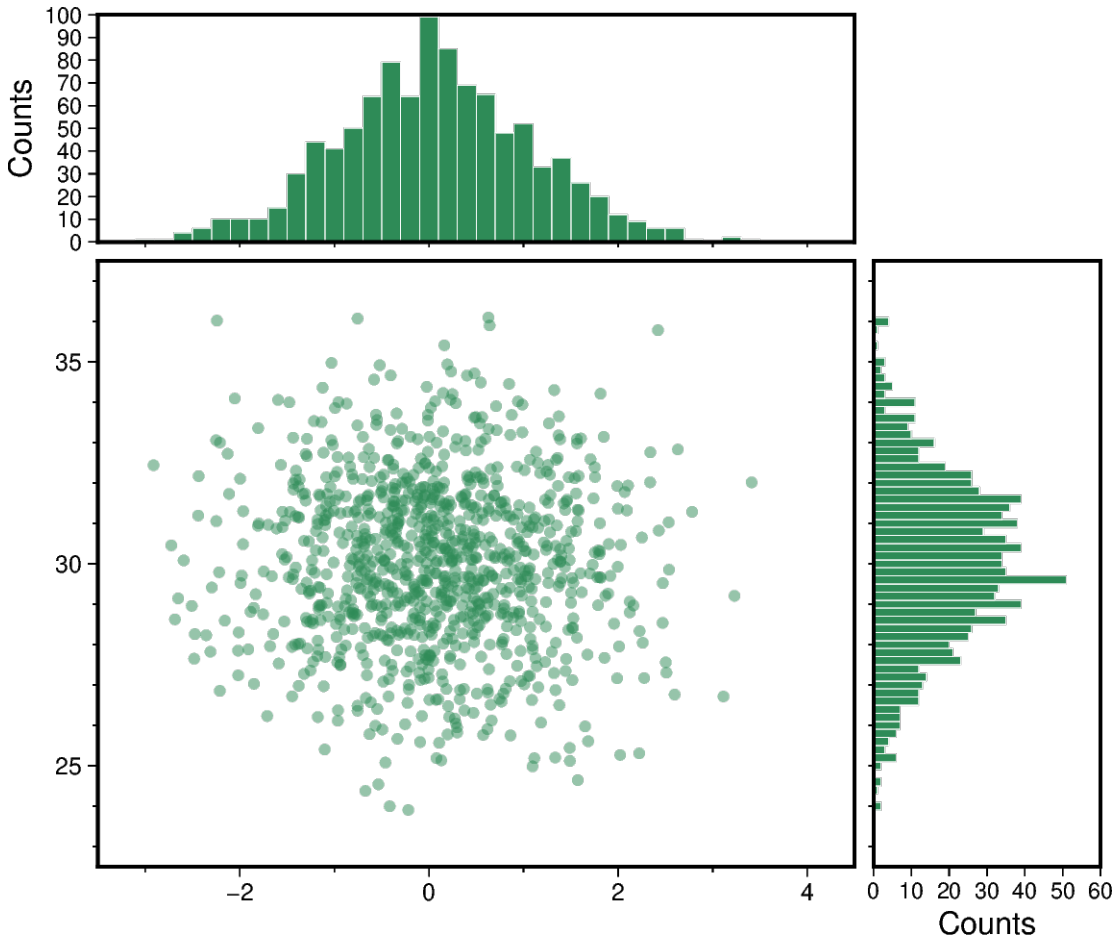
(continued from previous page)

```
# Load sample compilation of fracture lengths and azimuth as
# hypothetically digitized from geological maps
data = pygmt.datasets.load_sample_data(name="fractures")

fig = pygmt.Figure()

fig.rose(
    # use columns of the sample dataset as input for the length and azimuth
    # parameters
    length=data.length,
    azimuth=data.azimuth,
    # specify the "region" of interest in the (r,azimuth) space
    # [r0, r1, az0, az1], here, r0 is 0 and r1 is 1, for azimuth, az0 is 0 and
    # az1 is 360 which means we plot a full circle between 0 and 360 degrees
    region=[0, 1, 0, 360],
    # set the diameter of the rose diagram to 7.5 cm
    diameter="7.5c",
    # define the sector width in degrees, we append +r here to draw a rose
    # diagram instead of a sector diagram
    sector="10+r",
    # normalize bin counts by the largest value so all bin counts range from
    # 0 to 1
    norm=True,
    # use red3 as color fill for the sectors
    fill="red3",
    # define the frame with gridlines every 0.2 length unit in radial direction
    # and every 30 degrees in azimuthal direction, set background color to lightgray
    frame=Frame(xaxis=Axis(grid=0.2), yaxis=Axis(grid=30), fill="lightgray"),
    # use a pen size of 1p to draw the outlines
    pen="1p",
)

fig.show()
```

Total running time of the script: (0 minutes 0.102 seconds)

8.8.4 Scatter plot with histograms

To create a scatter plot with histograms at the sides of the plot one can use *pygmt.Figure.plot* in combination with *pygmt.Figure.histogram*. The positions of the histograms are plotted by offsetting them from the main scatter plot using *pygmt.Figure.shift_origin*.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# Generate random x-, y-coordinates from a standard normal distribution.
# x-values are centered on 0 with a standard deviation of 1, and y-values are centered
# on 30 with a standard deviation of 2.
rng = np.random.default_rng()
x = rng.normal(loc=0, scale=1, size=1000)
y = rng.normal(loc=30, scale=2, size=1000)

# Get axis limits from the data limits. Extend the limits by 0.5 to add some margin.
xmin = np.floor(x.min()) - 0.5
xmax = np.ceil(x.max()) + 0.5
ymin = np.floor(y.min()) - 0.5
ymax = np.ceil(y.max()) + 0.5

# Set fill color for symbols and bars.
fill = "seagreen"

# Set the dimensions of the scatter plot.
width, height = 10, 8
```

(continues on next page)

(continued from previous page)

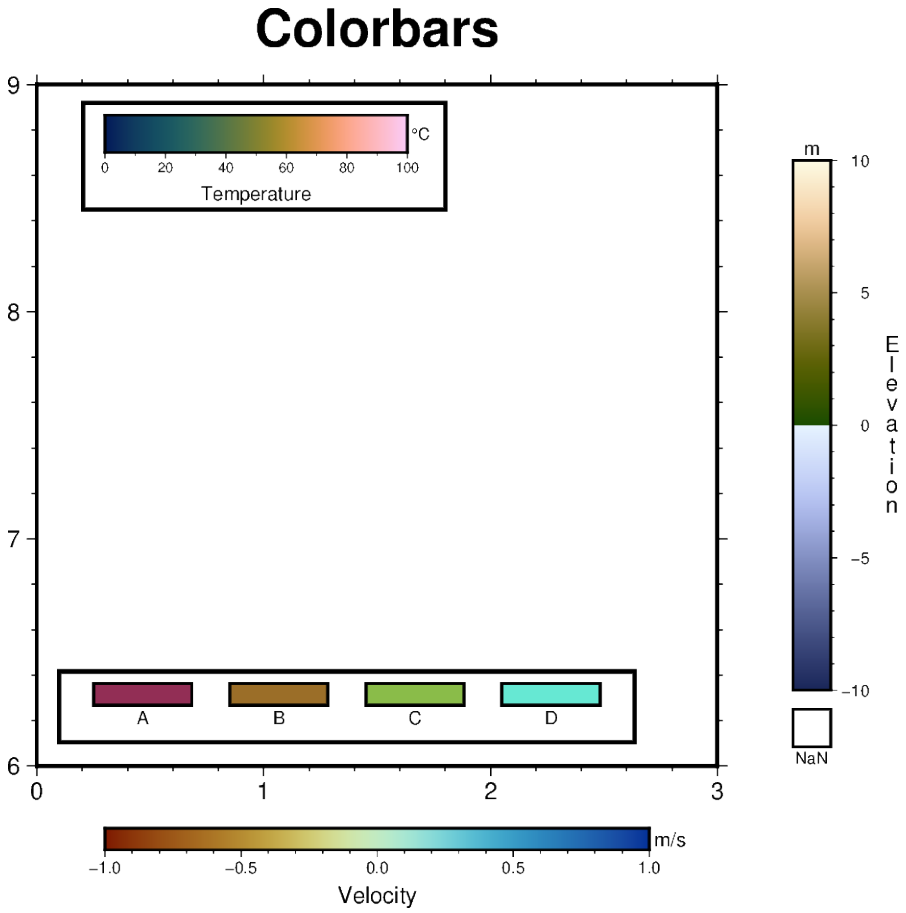
```
fig = pygmt.Figure()
fig.basemap(
    region=[xmin, xmax, ymin, ymax],
    projection=f"X{width}/{height}",
    frame=Frame(axes="WSrt", axis=Axis(annot=True, tick=True)),
)

# Plot data points as circles with a diameter of 0.15 centimeters and set transparency
# level for all circles to deal with overplotting.
fig.plot(x=x, y=y, style="c0.15c", fill=fill, transparency=50)

# Shift the plot origin in y-direction temporarily and add top margin histogram.
with fig.shift_origin(yshift=height + 0.25):
    fig.histogram(
        projection=f"X{width}/3",
        frame=Frame(
            axes="Wsrt",
            xaxis=Axis(tick=True),
            yaxis=Axis(annot=True, tick=True, label="Counts"),
        ),
        # Give the same value for ymin and ymax to have them calculated automatically.
        region=[xmin, xmax, 0, 0],
        data=x,
        fill=fill,
        pen="0.1p,white",
        series=0.2,
    )

# Shift the plot origin in x-direction temporarily and add right margin histogram.
with fig.shift_origin(xshift=width + 0.25):
    # Plot the horizontal histogram.
    fig.histogram(
        horizontal=True,
        projection=f"X3/{height}",
        # Note that the x- and y-axes are flipped, with the y-axis plotted␣
↪horizontally.
        frame=Frame(
            axes="wSrt",
            xaxis=Axis(tick=True),
            yaxis=Axis(annot=True, tick=True, label="Counts"),
        ),
        region=[ymin, ymax, 0, 0],
        data=y,
        fill=fill,
        pen="0.1p,white",
        series=0.2,
    )
fig.show()
```

Total running time of the script: (0 minutes 0.259 seconds)

8.9 Plot embellishments

8.9.1 Colorbar

The *pygmt.Figure.colorbar* method creates a color scalebar. The colormap is set via the `cmap` parameter. A full list of available color palette tables can be found at https://docs.generic-mapping-tools.org/6.7/reference/cpts.html. Use the `label` and `unit` parameters to set the labels along the long and short dimensions of the colorbar, respectively. To set the annotation and tick intervals, use the `annot` and `tick` parameters. The placement of the colorbar is set by passing a *pygmt.params.Position* object to the `position` parameter.




```
import pygmt
from pygmt.params import Axis, Frame, Position

fig = pygmt.Figure()
fig.basemap(
    region=[0, 3, 6, 9],
    projection="x3c",
    frame=Frame(axes="WSne", title="Colorbars", axis=Axis(annot=True, tick=True)),
)

# ============
# Create a colorbar designed for seismic tomography - roma
# Colorbar is placed at Bottom Center (BC) by default if no position is given
fig.colorbar(cmap="SCM/roma", annot=0.5, tick=0.1, label="Velocity", unit="m/s")
```

(continues on next page)

(continued from previous page)

```
# ============
# Create a colorbar showing the scientific rainbow - batlow
fig.colorbar(
    cmap="SCM/batlow",
    # A horizontal colorbar positioned at map coordinates (0.3, 8.7).
    position=Position((0.3, 8.7), cstype="mapcoords"),
    length=4,
    width=0.5,
    orientation="horizontal",
    box=True,
    label="Temperature",
    unit="°C",
    scale=100,
)

# ============
# Create a colorbar suitable for surface topography - oleron
fig.colorbar(
    cmap="SCM/oleron",
    # Colorbar placed at Middle Right (MR) outside the plot bounding box, offset by 1␣
↪cm
    # horizontally and 0 cm vertically from anchor point, with a rectangle for NaN
    # values. Note that the label 'Elevation' is plotted vertically as a column of
    # characters.
    position=Position("MR", cstype="outside", offset=(1, 0)),
    length=7,
    width=0.5,
    nan=True,
    label_as_column=True,
    label="Elevation",
    unit="m",
    scale=10,
)

# ============
# Create a colorbar suitable for categorical data - hawaii
# Set up the colormap
pygmt.makecpt(
    cmap="SCM/hawaii",
    series=[0, 3, 1],
    # Comma-separated string for the annotations of the colorbar
    color_model="+cA,B,C,D",
)
# Plot the colorbar
fig.colorbar(
    cmap=True,  # Use colormap set up above
    # Colorbar placed in the Bottom Left (BL) corner inside the plot bounding box,␣
↪with
    # an offset by 0.5 cm horizontally and 0.8 cm vertically from the anchor point,␣
↪and
    # plotted horizontally.
```

(continues on next page)

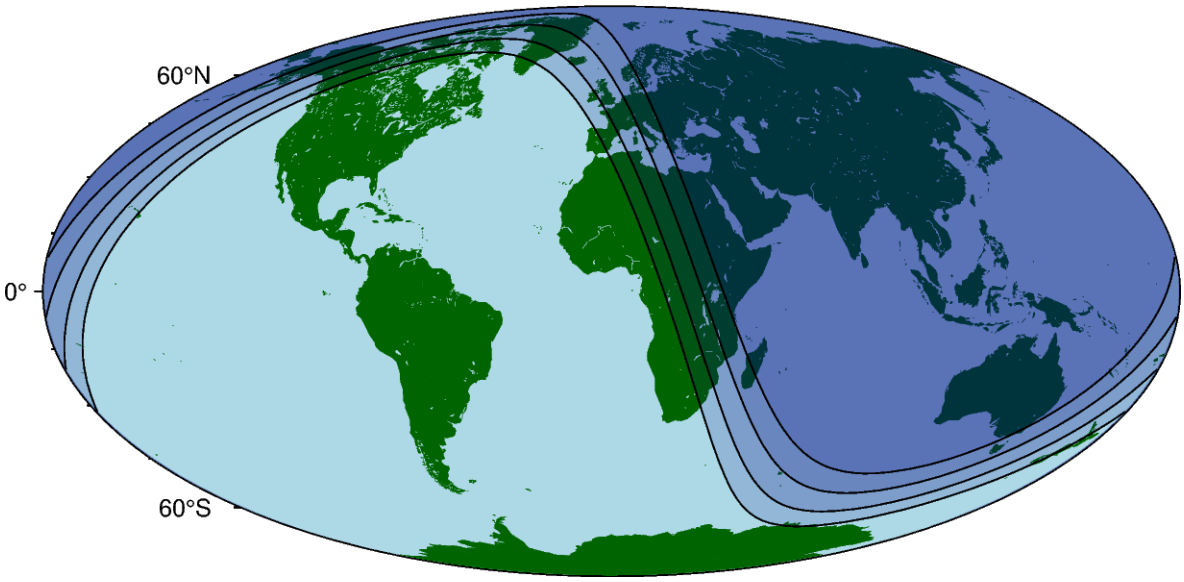
(continued from previous page)

```
    position=Position("BL", offset=(0.5, 0.8)),
    orientation="horizontal",
    box=True,
    # Divide colorbar into equal-sized rectangles
    equalsize=0.5,
)

fig.show()
```

Total running time of the script: (0 minutes 0.182 seconds)

8.9.2 Day-night terminator and twilights

Use *pygmt.Figure.solar* to show the different transition stages between daytime and nighttime. The parameter `terminator` is used to set the twilight stage, and can be either `"day_night"` (brightest), `"civil"`, `"nautical"`, or `"astronomical"` (darkest). Refer to https://en.wikipedia.org/wiki/Twilight for more information.




```
import datetime as dt

import pygmt

fig = pygmt.Figure()
# Create a global map using the Mollweide projection, centered at 0°E, with a width of
# 15 centimeters.
fig.basemap(region="d", projection="W15c", frame=True)
fig.coast(land="darkgreen", water="lightblue")

# Set a time for the day-night terminator and twilights to 17:00 UTC on January 1,␣
↪2000
reference_time = dt.datetime(year=2000, month=1, day=1, hour=17, minute=0, second=0)

# Plot the day-night terminator and twilights
for terminator in ["day_night", "civil", "nautical", "astronomical"]:
    fig.solar(
```

(continues on next page)

(continued from previous page)

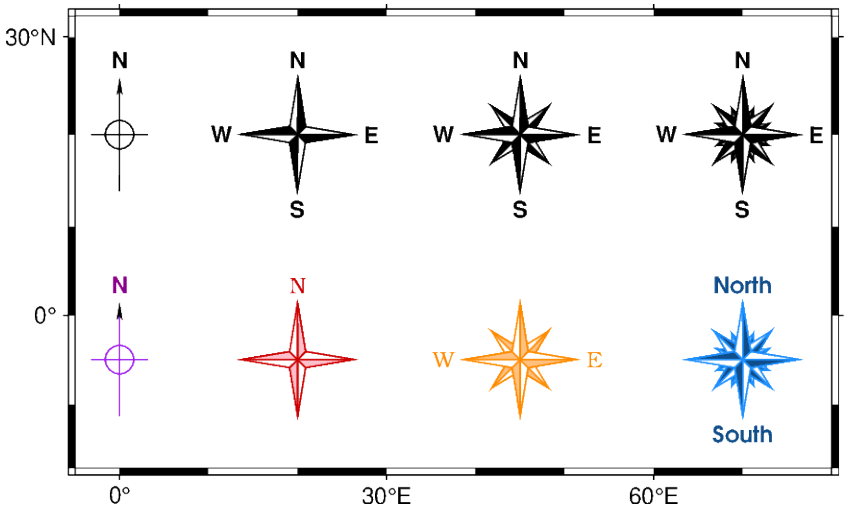
```
        terminator=terminator,
        terminator_datetime=reference_time,
        # Set the fill for the night area to navy blue with 85% transparency
        fill="navyblue@85",
        pen="0.5p",  # Set the terminator line to be 0.5-point thick
    )

fig.show()
```

Total running time of the script: (0 minutes 0.350 seconds)

8.9.3 Directional map rose

The *pygmt.Figure.directional_rose* method allows to add directional roses on maps. Using the method without any arguments will plot a rose at the Top Right corner, but this example will focus on customizing its position and appearance.

Colors of the map roses can be adjusted using MAP_DEFAULT_PEN and MAP_TICK_PEN_PRIMARY via *pygmt.config*. Customizing label font and color can be done via FONT_TITLE.




```
import pygmt
from pygmt.params import Position

fig = pygmt.Figure()

y0 = 20
y1 = -5
width = "1.5c"

fig.basemap(region=[-5, 80, -17, 32], projection="M10c", frame=True)

# Plain rose of 1.5 cm width showing an arrow towards North, a cross indicating the
# cardinal directions, and a label for the North direction
fig.directional_rose(
    width=width, labels=True, position=Position((0, y0), cstype="mapcoords")
)
```

(continues on next page)

(continued from previous page)

```
# Fancy, 1.5 cm wide rose of level 1 and labels indicating the different directions
fig.directional_rose(
    width=width,
    labels=True,
    position=Position((20, y0), cstype="mapcoords"),
    fancy=True,
)

# Fancy, 1.5 cm wide rose of level 2 and labels indicating the different directions
fig.directional_rose(
    width=width,
    labels=True,
    position=Position((45, y0), cstype="mapcoords"),
    fancy=2,
)

# Fancy, 1.5 cm wide rose of level 3 and labels indicating the different directions
fig.directional_rose(
    width=width,
    labels=True,
    position=Position((70, y0), cstype="mapcoords"),
    fancy=3,
)

# Plain rose of 1.5 cm width showing an arrow towards North, a cross indicating the
# cardinal directions, and a label for the North direction. Colors of the rose and
# labels are defined via MAP_TICK_PEN_PRIMARY and FONT_TITLE, respectively
with pygmt.config(MAP_TICK_PEN_PRIMARY="purple", FONT_TITLE="8p,darkmagenta"):
    fig.directional_rose(
        width=width,
        labels=True,
        position=Position((0, y1), cstype="mapcoords"),
    )

# Fancy, 1.5 cm wide rose of level 1 with only one label indicating the North␣
↪direction.
# Colors of the rose and labels are defined via MAP_DEFAULT_PEN, MAP_TICK_PEN_PRIMARY
# and FONT_TITLE, respectively.
with pygmt.config(
    MAP_DEFAULT_PEN="default,pink",
    MAP_TICK_PEN_PRIMARY="red3",
    FONT_TITLE="8p,Bookman-Light,red3",
):
    fig.directional_rose(
        width=width,
        labels=["", "", "", "N"],
        position=Position((20, y1), cstype="mapcoords"),
        fancy=True,
    )

# Fancy, 1.5 cm wide rose of level 2 with two labels indicating the West and East
```

(continues on next page)

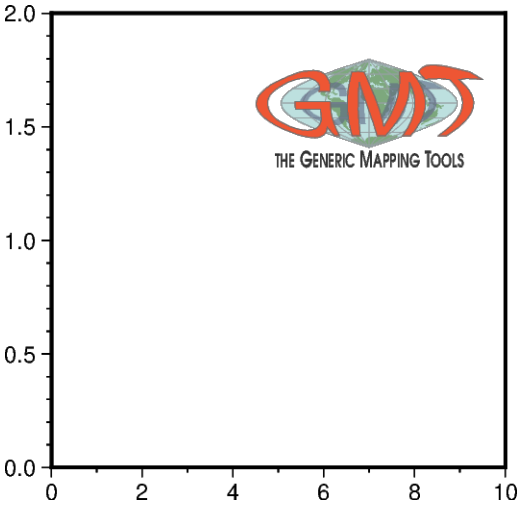
(continued from previous page)

```
# directions
with pygmt.config(
    MAP_DEFAULT_PEN="default,lightorange",
    MAP_TICK_PEN_PRIMARY="darkorange",
    FONT_TITLE="8p,Bookman-Light,darkorange",
):
    fig.directional_rose(
        width=width,
        labels=["W", "E", "", ""],
        position=Position((45, y1), cstype="mapcoords"),
        fancy=2,
    )

# Fancy, 1.5 cm wide rose of level 3 with two labels indicating the North and South
# directions
with pygmt.config(
    MAP_DEFAULT_PEN="default,Dodgerblue4",
    MAP_TICK_PEN_PRIMARY="Dodgerblue",
    FONT_TITLE="8p,AvantGarde-Demi,Dodgerblue4",
):
    fig.directional_rose(
        width=width,
        labels=["", "", "South", "North"],
        position=Position((70, y1), cstype="mapcoords"),
        fancy=3,
    )

fig.show()
```

Total running time of the script: (0 minutes 0.238 seconds)

8.9.4 GMT logo

The *pygmt.Figure.logo* method allows to place the GMT logo on a plot. For plotting the PyGMT logo, see *PyGMT logo*.

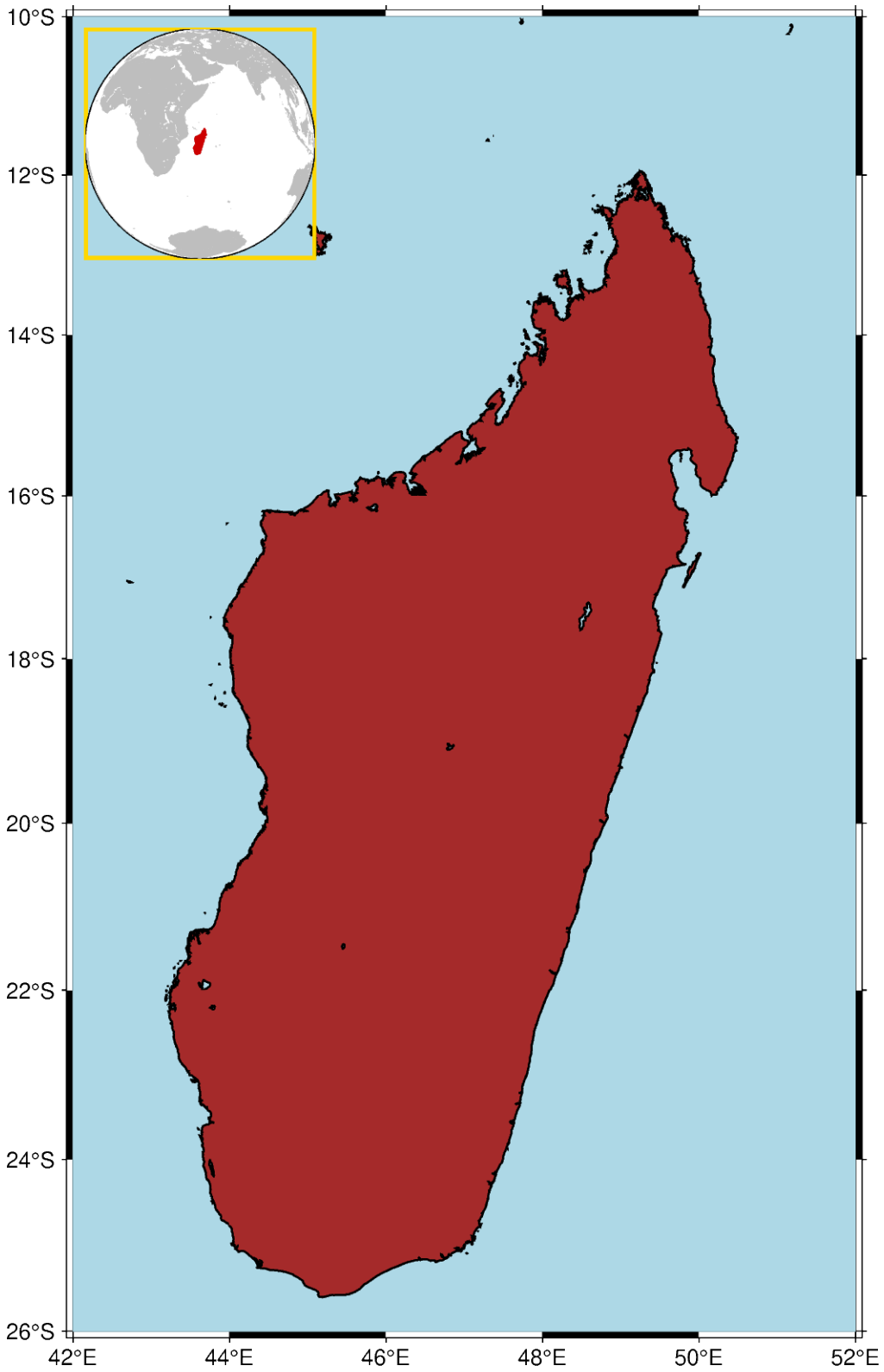


```
import pygmt
from pygmt.params import Position

fig = pygmt.Figure()
fig.basemap(region=[0, 10, 0, 2], projection="X6c", frame=True)

# Add the GMT logo in the Top Right (TR) corner of the current plot, scaled up to be 3
# centimeters wide and offset by 0.3 cm in x-direction and 0.6 cm in y-direction.
fig.logo(position=Position("TR", offset=(0.3, 0.6)), width="3c")
fig.show()
```

Total running time of the script: (0 minutes 0.164 seconds)

8.9.5 Inset

The *pygmt.Figure.inset* method adds an inset figure inside a larger figure. The method is called using a `with` statement. Plotting methods called within the `with` statement are applied to the inset figure.




```
import pygmt
from pygmt.params import Axis, Box, Position
```

(continues on next page)

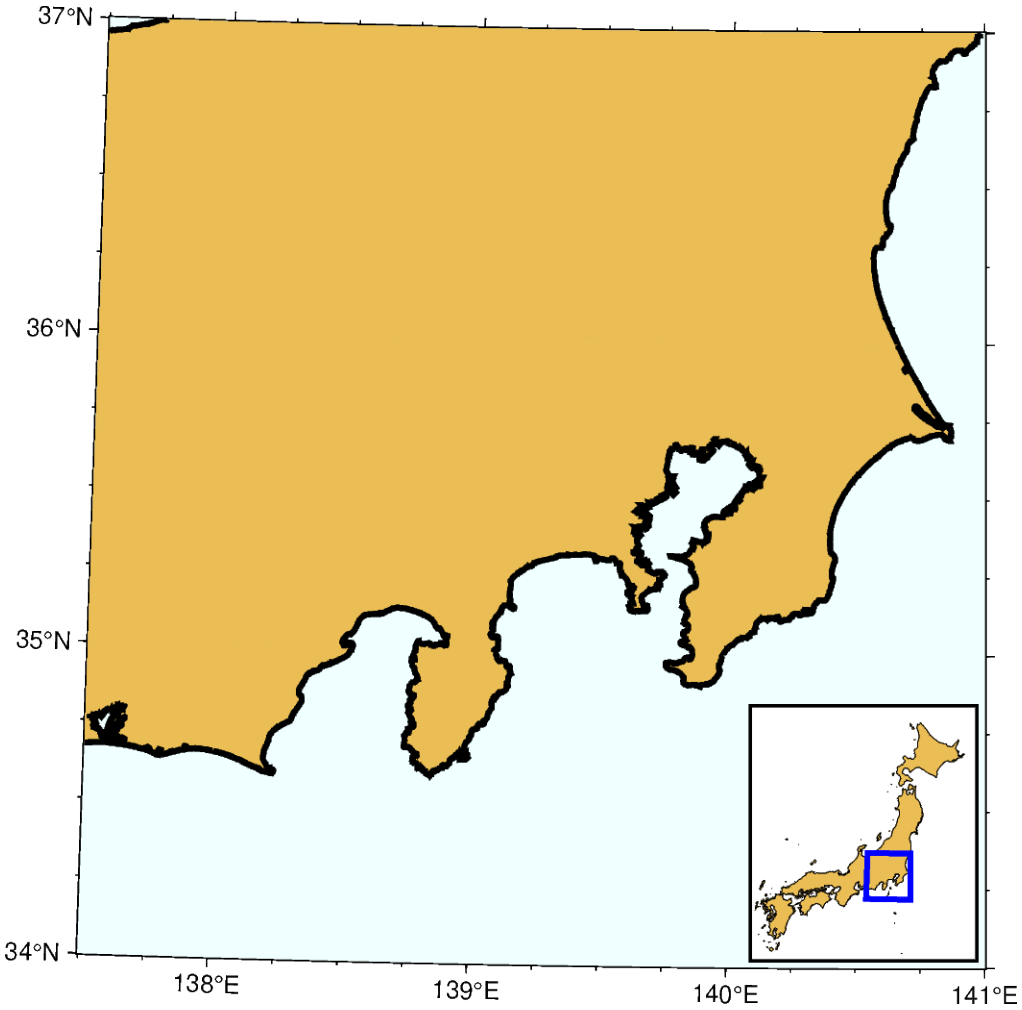
(continued from previous page)

```
fig = pygmt.Figure()
# Create the primary figure, setting the region to Madagascar
fig.basemap(region="MG+r2", projection="M12c", frame=Axis(annot=True))
# Set the land color to "brown", the water color to "lightblue", and the shorelines
# width to "thin"
fig.coast(land="brown", water="lightblue", shorelines="thin")

# Create an inset, placing it in the Top Left (TL) corner with a width of 3.5 cm and␣
↪x-
# and y-offsets of 0.2 cm. The clearance is set to 0, and the border is "gold" with a
# pen thickness of 1.5 points.
with fig.inset(
    position=Position("TL", offset=0.2),
    width=3.5,
    clearance=0,
    box=Box(pen="1.5p,gold"),
):
    # Create a map within the inset. This example uses the azimuthal orthogonal
    # projection centered at 47 E, 20 S. The question mark is required for the
    # automatic size determination by PyGMT
    fig.basemap(region="g", projection="G47/-20/?", frame=0)
    # Madagascar is highlighted in "red3"
    fig.coast(land="gray", water="white", dcw="MG+gred3")
fig.show()
```

Total running time of the script: (0 minutes 0.544 seconds)

8.9.6 Inset map showing a rectangular region

The *pygmt.Figure.inset* method adds an inset figure inside a larger figure. The method is called using a `with` statement. Plotting methods called within the `with` statement plot into the inset figure.




```
import pygmt
from pygmt.params import Axis, Box, Frame, Position

# Set the region of the main figure
region = [137.5, 141, 34, 37]

fig = pygmt.Figure()
# Plot the base map of the main figure. Universal Transverse Mercator (UTM) projection
# is used and the UTM zone is set to be "54S".
fig.basemap(
    region=region,
    projection="U54S/12c",
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
)

# Set the land color to "lightbrown", the water color to "azure1", the shoreline width
# to "2p", and the area threshold to 1000 km^2 for the main figure.
fig.coast(land="lightbrown", water="azure1", shorelines="2p", area_thresh=1000)

# Create an inset map, placing it in the Bottom Right (BR) corner with x- and␣
↪y-offsets
```

(continues on next page)

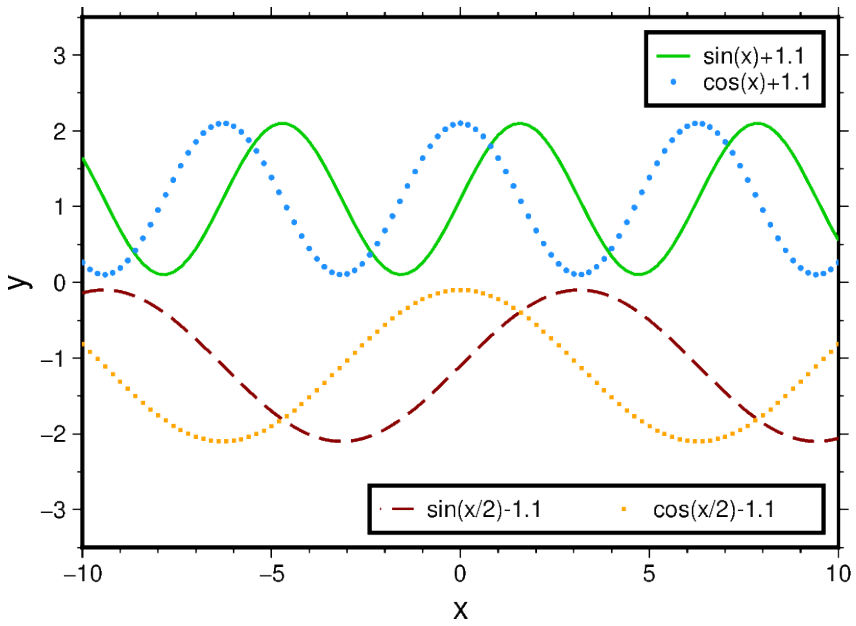
(continued from previous page)

```
# of 0.1 cm, respectively. The inset map contains the Japan main land. "U54S/3c" means
# UTM projection with a map width of 3 cm. The inset width and height are␣
↪automatically
# calculated from the specified ``region`` and ``projection`` parameters. Draws a
# rectangular box around the inset with a fill color of "white" and a pen of "1p".
with fig.inset(
    position=Position("BR", offset=0.1),
    box=Box(fill="white", pen="1p"),
    region=[129, 146, 30, 46],
    projection="U54S/3c",
):
    # Highlight the Japan area in "lightbrown" and draw its outline with a pen of
↪"0.2p"
    fig.coast(dcw="JP+glightbrown+p0.2p", area_thresh=10000)
    # Plot a rectangle ("r") in the inset map to show the area of the main figure.
    # "+s" means that the first two columns are the longitude and latitude of the␣
↪bottom
    # left corner of the rectangle, and the last two columns the longitude and␣
↪latitude
    # of the upper right corner.
    rectangle = [[region[0], region[2], region[1], region[3]]]
    fig.plot(data=rectangle, style="r+s", pen="2p,blue")
fig.show()
```

Total running time of the script: (0 minutes 0.285 seconds)

8.9.7 Legend

The *pygmt.Figure.legend* method can automatically create a legend for symbols plotted using *pygmt.Figure.plot*. A legend entry is only added when the `label` parameter is used to state the desired text. Optionally, to adjust the legend, users can append different modifiers. A list of all available modifiers can be found at https://docs.generic-mapping-tools.org/6.7/gmt.html#l-full. To create a multiple-column legend **+N** is used with the desired number of columns. For more complicated legends, users may want to write an ASCII file with instructions for the layout of the legend items and pass it to the `spec` parameter of *pygmt.Figure.legend*. For details on how to set up such a file, please see the GMT documentation at https://docs.generic-mapping-tools.org/6.7/legend.html#legend-codes.




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame, Position

# Set up some test data
x = np.arange(-10, 10.2, 0.2)
y1 = np.sin(x) + 1.1
y2 = np.cos(x) + 1.1
y3 = np.sin(x / 2) - 1.1
y4 = np.cos(x / 2) - 1.1

# Create new Figure() object
fig = pygmt.Figure()

fig.basemap(
    projection="X10c/7c",
    region=[-10, 10, -3.5, 3.5],
    frame=Frame(
        axes="WSne",
        xaxis=Axis(annot=True, tick=True, label="x"),
        yaxis=Axis(annot=1, tick=0.5, label="y"),
    ),
)

# ------------------------------------------------------------------------
# Top: Vertical legend (one column, default)

# Use the label parameter to state the text label for the legend entry
fig.plot(x=x, y=y1, pen="1p,green3", label="sin(x)+1.1")
fig.plot(x=x, y=y2, style="c0.07c", fill="dodgerblue", label="cos(x)+1.1")

# Add a legend to the plot at the Top Right (TR) corner with a 0.2-cm offset in x- and
# y-directions; surround the legend with a box.
```

(continues on next page)

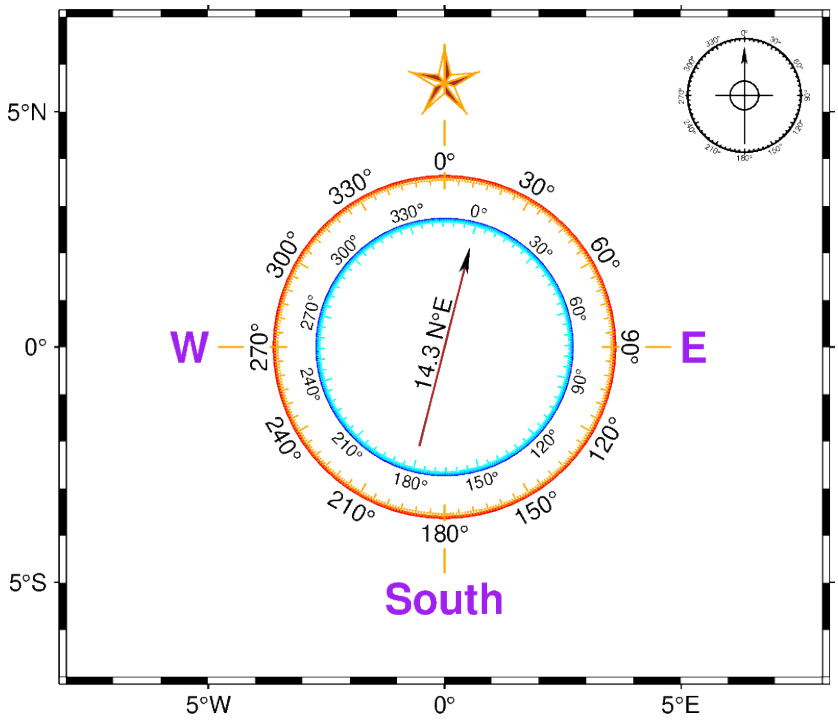
(continued from previous page)

```
fig.legend(position=Position("TR", offset=0.2), box=True)

# ----------------------------------------------------------------------------
# Bottom: Horizontal legend (here two columns)

# +N sets the number of columns corresponding to the given number, here 2
fig.plot(x=x, y=y3, pen="1p,darkred,-", label="sin(x/2)-1.1+N2")

fig.plot(x=x, y=y4, style="s0.07c", fill="orange", label="cos(x/2)-1.1")

# For a multi-column legend, users have to provide the width, here it is set to 6 cm;
# the legend is placed at the Bottom Right (BR) corner.
fig.legend(position=Position("BR", offset=0.2), width="6c", box=True)

fig.show()
```

Total running time of the script: (0 minutes 0.168 seconds)

8.9.8 Magnetic rose

The method *pygmt.Figure.magnetic_rose* can be used to add a magnetic rose to a map. This example shows how such a magnetic rose can be customized.




```
import pygmt

fig = pygmt.Figure()
fig.basemap(region=[-8, 8, -7, 7], projection="M10c", frame=True)

# Add a magnetic rose
```

(continues on next page)

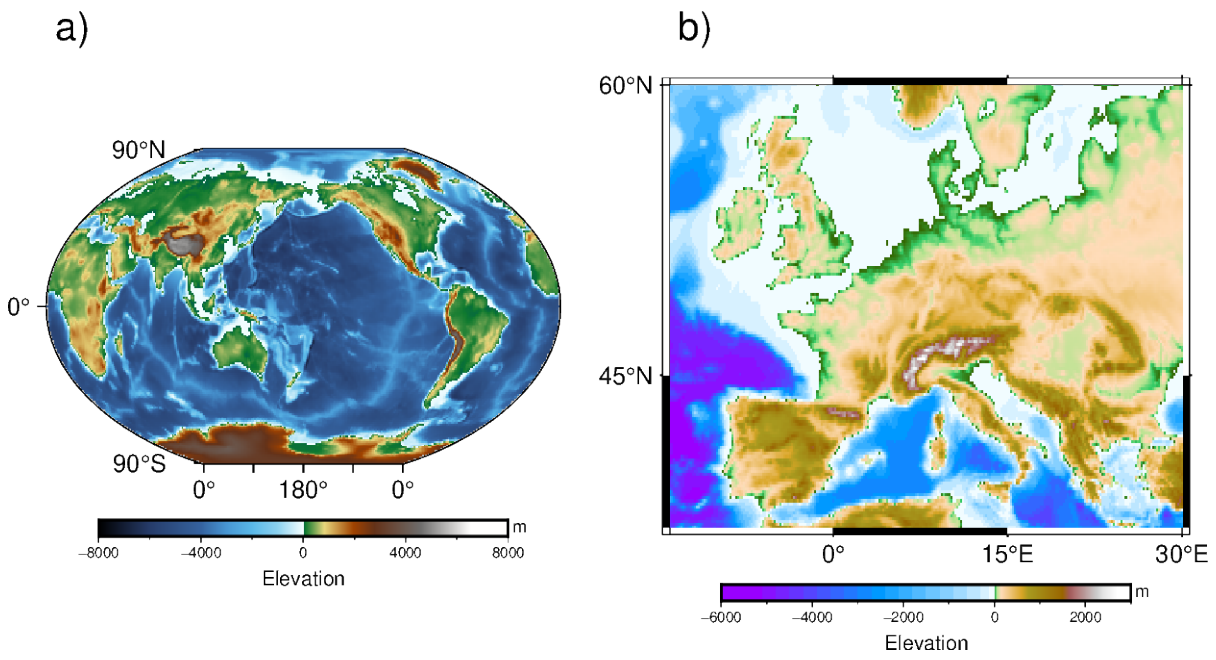
(continued from previous page)

```
# By default, it's placed in the Top Right corner
fig.magnetic_rose()

# Add a magnetic rose with several adjustments
with pygmt.config(
    # Pen used for the ticks on the outer circle and the outline of the North star
    MAP_TICK_PEN_SECONDARY="orange",
    # Pen used for the ticks related to the inner circle
    MAP_TICK_PEN_PRIMARY="cyan",
    # Pen used for the stem of the declination arrow and the fill of the North star
    MAP_DEFAULT_PEN="brown",
    # Font used for the labels for the geographic directions
    FONT_TITLE="purple",
):
    fig.magnetic_rose(
        position="MC",  # placed at MiddleCenter
        width=4.5,  # width of the rose
        # If a declination value is given, an arrow showing the declination is plotted
        # instead of the simple North arrow
        declination=14.3,
        # Adjust the label added to the declination arrow
        declination_label="14.3 N°E",
        # Add labels for the geographic directions. Use a * to get a North star and
        # "" to skip a label
        labels=["W", "E", "South", "*"],
        # Draw an outer circle with the provided pen
        outer_pen="1p,red",
        # Draw an inner circle with the provided pen
        inner_pen="1p,blue",
    )

fig.show()
```

Total running time of the script: (0 minutes 0.216 seconds)

8.9.9 Multiple colormaps

This gallery example shows how to create multiple colormaps for different subplots. To better understand how GMT modern mode maintains several levels of colormaps, please refer to https://docs.generic-mapping-tools.org/6.7/reference/features.html#gmt-modern-mode-hierarchical-levels for details.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()

# Load Earth relief data for the entire globe and a subset region
grid_globe = pygmt.datasets.load_earth_relief(resolution="01d")
subset_region = [-14, 30, 35, 60]
grid_subset = pygmt.datasets.load_earth_relief(resolution="10m", region=subset_region)

# Define a 1-row, 2-column subplot layout. The overall figure dimensions are set to be
# 15 cm wide and 8 cm high. Each subplot is automatically tagged. The space between␣
↪the
# subplots is set to be 0.5 cm.
with fig.subplot(nrows=1, ncols=2, figsize=("15c", "8c"), tag=True, margins=0.5):
    # Activate the first panel so that the colormap created by the makecpt function is
    # a panel-level CPT
    with fig.set_panel(panel=0):
        pygmt.makecpt(cmap="gmt/geo", series=[-8000, 8000])
        # "R?" means Winkel Tripel projection with map width automatically determined
        # from the subplot width.
        fig.basemap(projection="R?", region="g", frame=Axis(annot=True))
        fig.grdimage(grid=grid_globe)
        fig.colorbar(annot=4000, tick=2000, label="Elevation", unit="m")
    # Activate the second panel so that the colormap created by the makecpt function␣
↪is
    # a panel-level CPT
    with fig.set_panel(panel=1):
        pygmt.makecpt(cmap="gmt/globe", series=[-6000, 3000])
        # "M?" means Mercator projection with map width also automatically determined
        # from the subplot width.
        fig.basemap(projection="M?", region=subset_region, frame=Axis(annot=True))
```

(continues on next page)

(continued from previous page)

```
        fig.grdimage(grid=grid_subset)
        fig.colorbar(annot=2000, tick=1000, label="Elevation", unit="m")

fig.show()
```

Total running time of the script: (0 minutes 0.330 seconds)

8.9.10 PyGMT logo

Beside the *GMT logo*, there is a specific logo for PyGMT which can be plotted and added to a figure using *pygmt.Figure.pygmtlogo*. The design of the logo itself is kindly provided by @sfrooti and consists of an icon and the wordmark "PyGMT".

The visual logo is available in circle and hexagon shapes. It supports both colored and monochrome black-and-white palettes, alongside light and dark theme modes, combining to yield eight distinct standalone logo variants. Additionally, the visual logo can optionally include the wordmark "PyGMT"; if enabled, the wordmark can be positioned either to the right of the icon or beneath it.

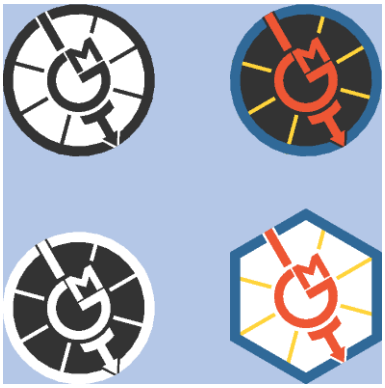
```
import pygmt
from pygmt.params import Frame
```

Plot the PyGMT logo without any arguments.

```
fig = pygmt.Figure()
fig.pygmtlogo()
fig.show()
```

Via the `color`, `theme`, `shape` parameters the appearance of the logo can be changed.

```
fig = pygmt.Figure()
fig.basemap(region=[-1, 1] * 2, projection="X5c/5c", frame=Frame(fill="180/199/231"))

fig.pygmtlogo(color=False, position="TL")

fig.pygmtlogo(theme="dark", position="TR")

fig.pygmtlogo(color=False, theme="dark", position="BL")

fig.pygmtlogo(shape="hexagon", position="BR")

fig.show()
```



Via the `wordmark` parameter the text "PyGMT" can be added on the right side or at the bottom of the visual. Use the `width` and `height` parameters to adjust the size of the logo.

```
fig = pygmt.Figure()
fig.basemap(region=[-1, 1] * 2, projection="X7c/5c", frame=Frame(fill="180/199/231"))

fig.pygmtlogo(wordmark="horizontal", position="TC", width="6c")

fig.pygmtlogo(theme="dark", wordmark="vertical", position="BC", height="3c")

fig.show()
```

Total running time of the script: (0 minutes 0.986 seconds)

8.9.11 Scale bar

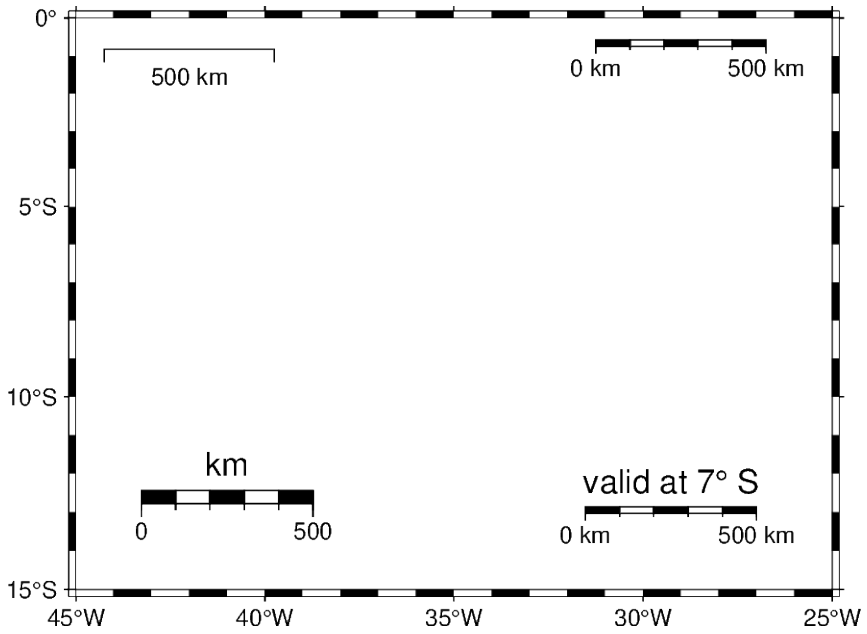
The *pygmt.Figure.scalebar* method can be used to add a scale bar to a map. This example shows how such a scale bar can be customized.

```
import pygmt
from pygmt.params import Axis, Box, Frame, Position

# Create a new Figure instance
fig = pygmt.Figure()

# Mercator projection with 10 centimeters width
fig.basemap(
    region=[-45, -25, -15, 0],
    projection="M0/0/10c",
```

(continues on next page)

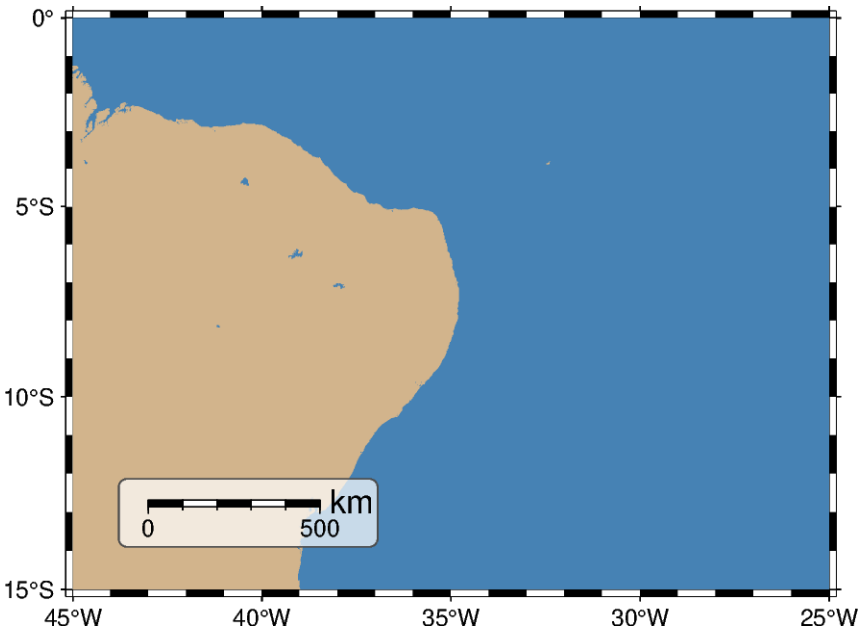
(continued from previous page)

```
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
)

# --- Top Left: Add a plain scale bar ---
# It is placed based on geographic coordinates 42° West and 1° South, applies at the
# reference point by default, and represents a length of 500 kilometers.
fig.scalebar(length="500k", position=Position((-42, -1), cstype="mapcoords"))

# --- Top Right: Add a fancy scale bar ---
# It is placed based on normalized bounding box coordinates. Use a fancy style to get␣
↪a
# scale bar that looks like train tracks. Add the distance unit to the single distance
# values.
fig.scalebar(
    position=Position((0.8, 0.95), cstype="boxcoords"),
    length="500k",
    fancy=True,
    unit=True,
)

# --- Bottom Left: Add a thick scale bar ---
# It applies at the middle of the map (scale_loc is set to True). Use the height
# parameter to adjust the thickness of the scale bar Without providing text, the label
# parameter adds the distance unit as label.
fig.scalebar(
    position=Position((0.2, 0.15), cstype="boxcoords"),
    scale_loc=True,
    length="500k",
    height="10p",
    fancy=True,
    label=True,
)

# --- Bottom Right: Add a scale bar valid for a specific location ---
# It is placed at BottomRight using MiddleRight as anchor point with an offset of 1
# centimeter in both x- and y-directions. It applies at -7° South. A customized label
# is added via the label parameter.
fig.scalebar(
    position=Position("BR", anchor="MR", offset=1),
    scale_loc=-7,
    length="500k",
    fancy=True,
    unit=True,
    label="valid at 7° S",
)

fig.show()
```



The `box` parameter allows surrounding the scale bar. This can be useful when adding a scale bar to a colorful map to improve contrast and readability.

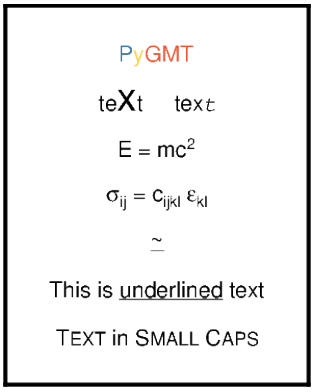
```
fig = pygmt.Figure()

fig.basemap(
    region=[-45, -25, -15, 0],
    projection="M10c",
    frame=Frame(axes="WSne", axis=Axis(annot=True, tick=True)),
)
fig.coast(land="tan", water="steelblue")
fig.scalebar(
    position=Position("BL", cstype="inside", offset=1),
    scale_loc=-7,
    length="500k",
    fancy=True,
    label="km",
    label_alignment="right",
    # Fill the box in white with a transparency of 30 percent, add a solid outline in
    # darkgray (gray30) with a thickness of 0.5 points, and use rounded edges with a
    # radius of 3 points
    box=Box(fill="white@30", pen="0.5p,gray30,solid", radius="3p"),
)

fig.show()
```



Total running time of the script: (0 minutes 0.270 seconds)

8.9.12 Text formatting

There are various options for formatting text contained in a plot, e.g., text added via *pygmt.Figure.text*, the plot title, labels of colorbars as well as Cartesian axes, and legend entries. It's also possible to change the font as well as its color and size only for specific characters of a longer text. The supported fonts are listed at *Supported Fonts*. For an complete overview see *Text Formatting*.




```
import pygmt

fig = pygmt.Figure()
fig.basemap(region=[-1, 1, -4, 4], projection="X4c/5c", frame=0)

# Change font color for specific letters of the word "PyGMT":
# blue for "P", yellow for "y", and red for "GMT"
fig.text(x=0, y=3, text="@;63/124/173;P@;;@;255/212/59;y@;;@;238/86/52;GMT@;;")

# Change font size and style for one single character, respectively
fig.text(x=0, y=2, text="te@:15:x@::t     tex@%Courier-Oblique%t@%%")
```

(continues on next page)

(continued from previous page)

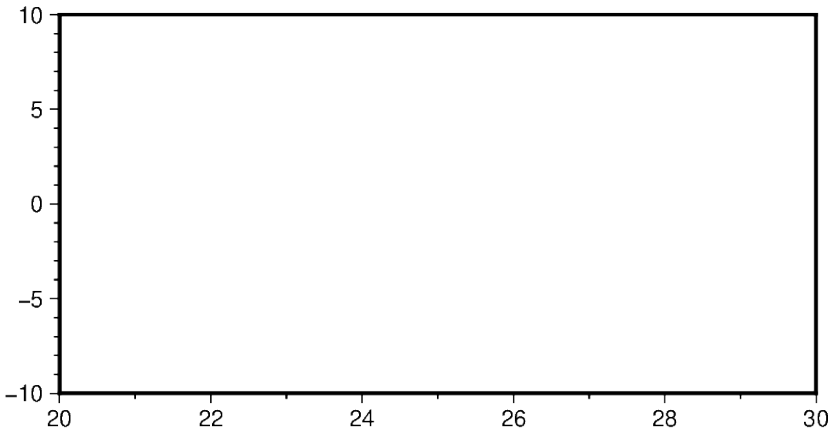
```
# Use superscript
fig.text(x=0, y=1, text="E = mc@+2@+")

# Use subscripts and Greek letters
fig.text(x=0, y=0, text="@~s@~@-ij@- = c@-ijkl@- @~e@~@-kl@-")

# Combine two characters above each other
fig.text(x=0, y=-1, text="@!_~")

# Underline the text
fig.text(x=0, y=-2, text="This is @_underlined@_ text")

# Use small caps
fig.text(x=0, y=-3, text="T@#ext@# in S@#mall@# C@#aps@#")

fig.show()
```

Total running time of the script: (0 minutes 0.106 seconds)

8.9.13 Timestamp

The *pygmt.Figure.timestamp* method can draw the GMT timestamp logo on the plot. The timestamp will always be shown relative to the Bottom Left (BL) corner of the plot. By default, the `offset` and `justify` parameters are set to `("-54p", "-54p")` (x-, y-directions) and `"BL"` (Bottom Left), respectively.

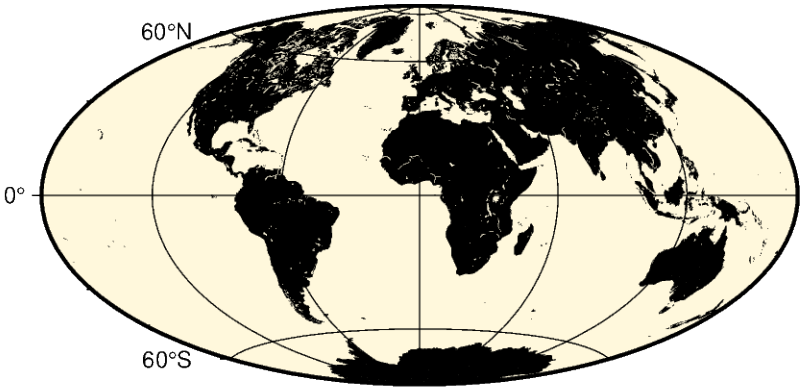
```
import os

import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.basemap(region=[20, 30, -10, 10], projection="X10c/5c", frame=True)
fig.timestamp()
fig.show()
```



Additionally, a custom label can be added via the `label` parameter. The font can be defined via the `font` parameter and

the timestamp string format via `timefmt`.

```
os.environ["TZ"] = "Pacific/Honolulu"  # optionally set the time zone

fig = pygmt.Figure()
fig.coast(
    region="d",
    projection="H10c",
    land="black",
    water="cornsilk",
    frame=Axis(annot=True, tick=True, grid=True),
)
fig.timestamp(
    label="Powered by PyGMT",
    justify="TL",
    font="Times-Bold",
    timefmt="%Y-%m-%dT%H:%M:%S%z",
)
fig.show()
```



GMT 2026-10-22T04:47:28-1000 Powered by PyGMT

Total running time of the script: (0 minutes 0.269 seconds)

CHAPTER

NINE

PROJECTIONS

PyGMT supports many map projections; see *GMT Map Projections* for an overview. Use the `projection` parameter to specify which one you want to use in all plotting methods. The projection is specified by a one-letter code along with (sometimes optional) reference longitude and latitude and the width of the map (for example, **A***lon0/lat0*[*/horizon*]*/width*). The map height is determined based on the region and projection.

These are all the available projections:

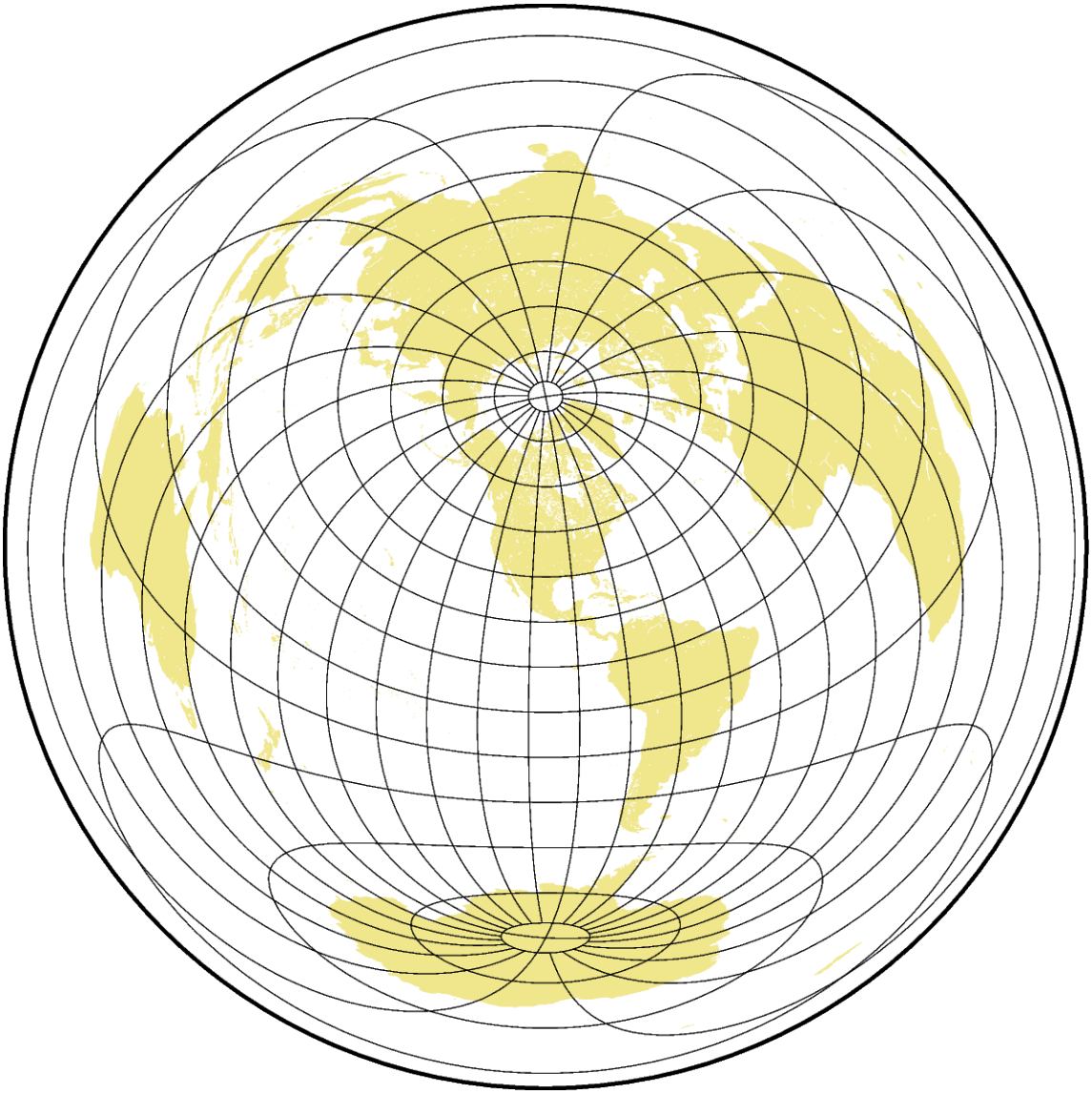
9.1 Azimuthal Projections

9.1.1 Azimuthal equidistant projection

The main advantage of this projection is that distances from the projection center are displayed in correct proportions. Also directions measured from the projection center are correct. It is very useful for a global view on locations that lie within a certain distance or for comparing distances of different locations relative to the projection center.

e*lon0/lat0*[*/horizon*]*/scale* or **E***lon0/lat0*[*/horizon*]*/width*

- **e** or **E**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *horizon*: The maximum distance from the projection center to the edge of the map (i.e. the visible portion of the rest of the world map) in degrees (<= 180°) [Optional, default is 180°].
- *scale* or *width*: Sets the map size.



```
coast [WARNING]: Fill/clip continent option (-G) may not work for this projection.
coast [WARNING]: If the antipode (0/4.65063e-310) is in the ocean then chances are␣
↪good it will work.
coast [WARNING]: Otherwise, avoid projection center coordinates that are exact␣
↪multiples of 80 degrees.
```

```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region="g",
```

(continues on next page)

(continued from previous page)

```
    projection="E-100/40/15c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
fig.show()
```

Total running time of the script: (0 minutes 0.549 seconds)

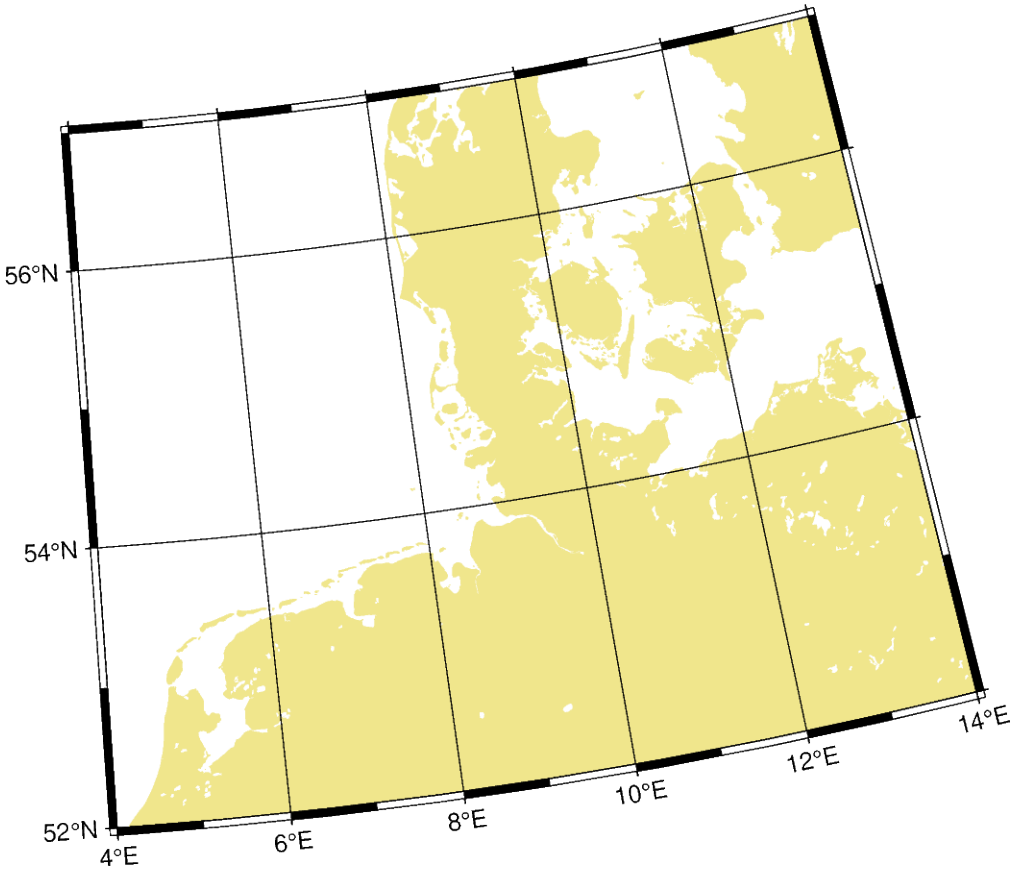
Tags: equidistant

9.1.2 General stereographic projection

This map projection is a conformal, azimuthal projection. It is mainly used with a projection center in one of the poles. Then meridians appear as straight lines and cross latitudes at a right angle. Unlike the azimuthal equidistant projection, the distances in this projection are not displayed in correct proportions. It is often used as a hemisphere map like the Lambert Azimuthal Equal Area projection.

s*lon0/lat0*[*/horizon*]*/scale* or **S***lon0/lat0*[*/horizon*]*/width*

- **s** or **S**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *horizon*: Sets the maximum distance from the projection center in degrees (< 180°) [Optional, default is 90°].
- *scale* or *width*: Sets the map size.



```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[4, 14, 52, 57],
    projection="S0/90/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
fig.show()
```

Total running time of the script: (0 minutes 0.151 seconds)

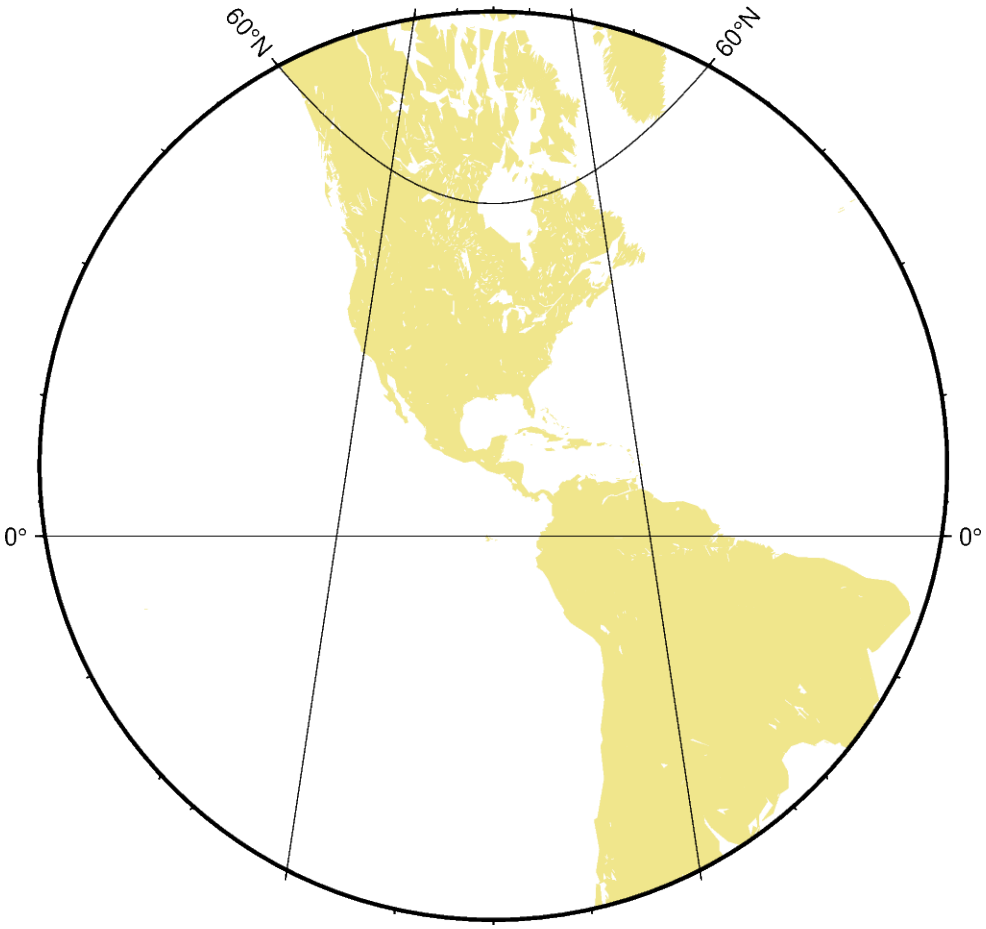
Tags: conformal

9.1.3 Gnomonic projection

The point of perspective of the gnomonic projection lies at the center of the Earth. As a consequence great circles (orthodromes) on the surface of the Earth are displayed as straight lines, which makes it suitable for distance estimation for navigational purposes. It is neither conformal nor equal-area and the distortion increases greatly with distance to the projection center. It follows that the scope of application is restricted to a small area around the projection center (at a maximum of 60°).

f*lon0/lat0*[*/horizon*]*/scale* or **F***lon0/lat0*[*/horizon*]*/width*

- **f** or **F**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *horizon*: Sets the maximum distance from the projection center in degrees (< 90°) [Optional, default is 60°].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="F-90/15/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
fig.show()
```

Total running time of the script: (0 minutes 0.182 seconds)

9.1.4 Lambert azimuthal equal-area projection

This projection was developed by Johann Heinrich Lambert in 1772 and is typically used for mapping large regions like continents and hemispheres. It is an azimuthal, equal-area projection, but is not perspective. Distortion is zero at the center of the projection, and increases radially away from this point.

a*lon0/lat0*[*/horizon*]*/scale* or **A***lon0/lat0*[*/horizon*]*/width*

- **a** or **A**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.

- *horizon*: Sets the maximum distance from the projection center in degrees (<= 180°) [Optional, default is 90°].
- *scale* or *width*: Sets the map size.

```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="A30/-20/60/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
fig.show()
```

Total running time of the script: (0 minutes 0.165 seconds)

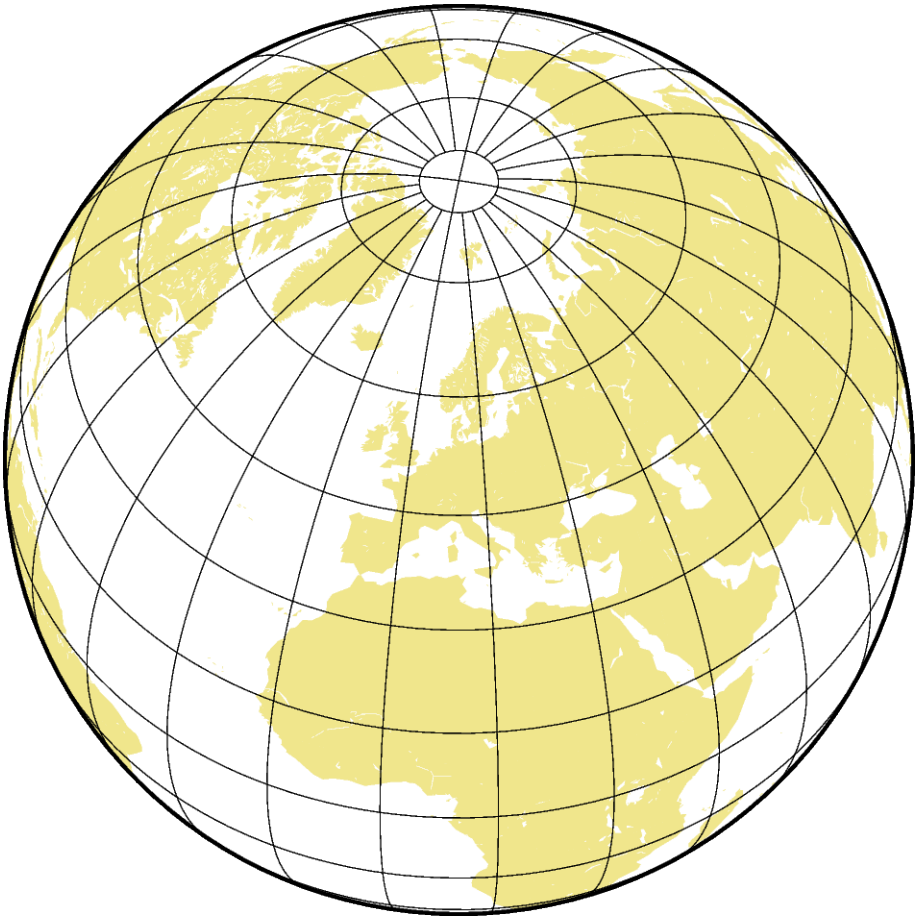
Tags: equal-area

9.1.5 Orthographic projection

This is a perspective projection like the general perspective, but with the difference that the point of perspective lies in infinite distance. It is therefore often used to give the appearance of a globe viewed from outer space, were one hemisphere can be seen as a whole. It is neither conformal nor equal-area and the distortion increases near the edges.

g*lon0/lat0*[*/horizon*]*/scale* or **G***lon0/lat0*[*/horizon*]*/width*

- **g** or **G**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *horizon*: Sets the maximum distance from the projection center in degrees (<= 90°) [Optional, default is 90°].
- *scale* or *width*: Sets the map size.



```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="G10/52/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
```

(continues on next page)

(continued from previous page)

```
fig.show()
```

Total running time of the script: (0 minutes 0.191 seconds)

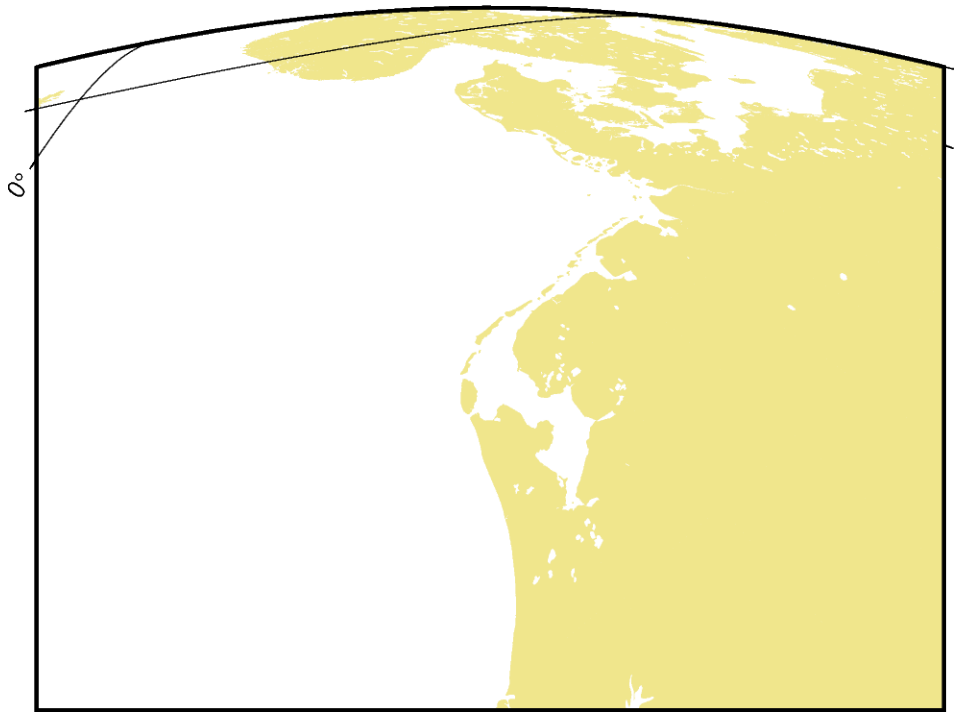
9.1.6 Perspective projection

The perspective projection imitates the view of the Earth from a finite point in space. In a full view of the earth one third of its surface area can be seen.

g*lon0/lat0/scale*[**+a***azimuth*][**+t***tilt*][**+v***vwidth/vheight*][**+w***twist*][**+z***altitude*] or **G***lon0/lat0/width*[**+a***azimuth*][**+t***tilt*][**+v***vwidth/vheight*][**+v**

- **g** or **G**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *scale* or *width*: Sets the map size.
- **+a***azimuth*: The direction in which you are looking, measured clockwise from north in degrees [Optional].
- **+t***tilt*: The viewing angle relative to zenith in degrees. A tilt of 0° is looking straight down, 60° is looking 30° above horizon [Optional].
- **+v***vwidth/vheight*: The viewport angle in degrees [Optional].
- **+w***twist*: The clockwise rotation of the image in degrees [Optional].
- **+z***altitude*: The height of the viewpoint above local sea level in km. If altitude is less than 10, it is the distance from the center of the earth to the viewpoint in earth radii [Optional].

The example shows the coast of Northern Europe viewed from 250 km above sea level looking 30° from north at a tilt of 45°. The height and width of the viewing angle is both 60°, which imitates viewing with naked eye.




```
import pygmt
from pygmt.params import Axis
```

(continues on next page)

(continued from previous page)

```
fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="G4/52/12c+a30+t45+v60/60+w0+z250",
    frame=Axis(annot=True, tick=True, grid=True),
    land="khaki",
    water="white",
)
fig.show()
```

Total running time of the script: (0 minutes 1.125 seconds)

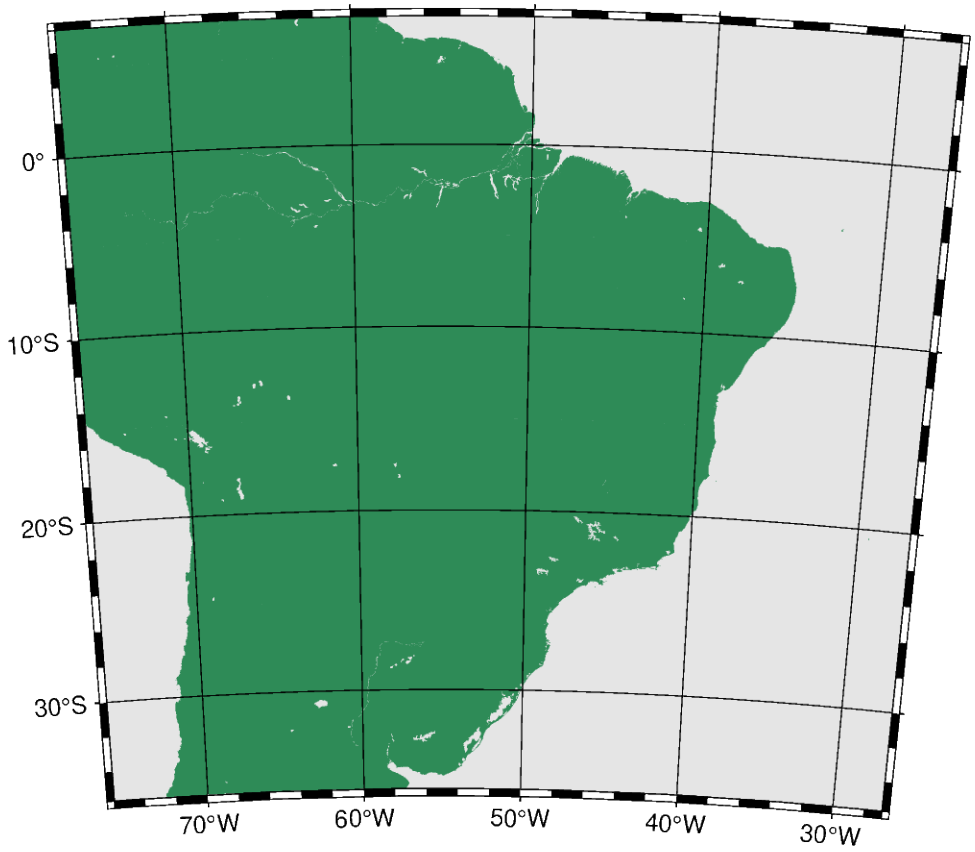
9.2 Conic Projections

9.2.1 Albers conic equal-area projection

This projection, developed by Heinrich C. Albers in 1805, is predominantly used to map regions of large east-west extent, in particular the United States. It is a conic, equal-area projection, in which parallels are unequally spaced arcs of concentric circles, more closely spaced at the north and south edges of the map. Meridians, on the other hand, are equally spaced radii about a common center, and cut the parallels at right angles. Distortion in scale and shape vanishes along the two standard parallels. Between them, the scale along parallels is too small; beyond them it is too large. The opposite is true for the scale along meridians.

b*lon0/lat0/lat1/lat2/scale* or **B***lon0/lat0/lat1/lat2/width*

- **b** or **B**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *lat1/lat2*: Sets the two standard parallels.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use the ISO country code for Brazil and add a padding of 2 degrees (+R2)
fig.coast(
    region="BR+R2",
    projection="B-55/-15/-25/0/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="seagreen",
    water="gray90",
)
fig.show()
```

Total running time of the script: (0 minutes 0.235 seconds)

Tags: equal-area

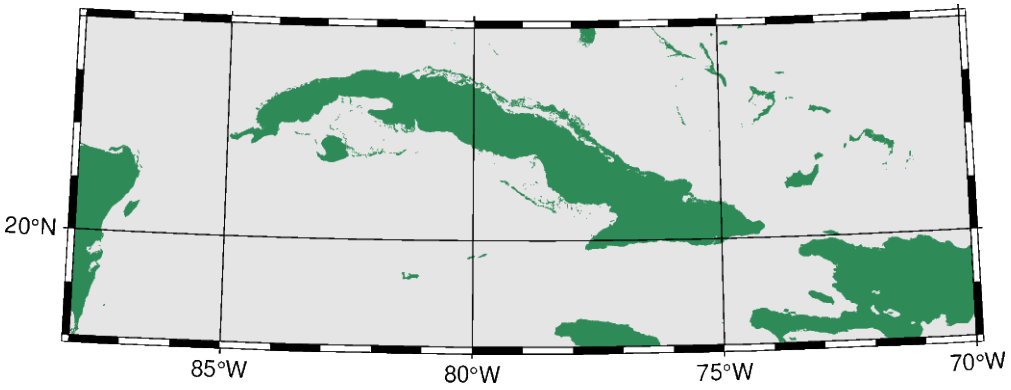
9.2.2 Equidistant conic projection

The equidistant conic projection was described by the Greek philosopher Claudius Ptolemy about A.D. 150. It is neither conformal or equal-area, but serves as a compromise between them. The scale is true along all meridians and the standard parallels.

d*lon0/lat0/lat1/lat2/scale* or **D***lon0/lat0/lat1/lat2/width*

- **d** or **D**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.

- *lat1/lat2*: Sets the two standard parallels.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[-88, -70, 18, 24],
    projection="D-79/21/19/23/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="seagreen",
    water="gray90",
)
fig.show()
```

Total running time of the script: (0 minutes 0.122 seconds)

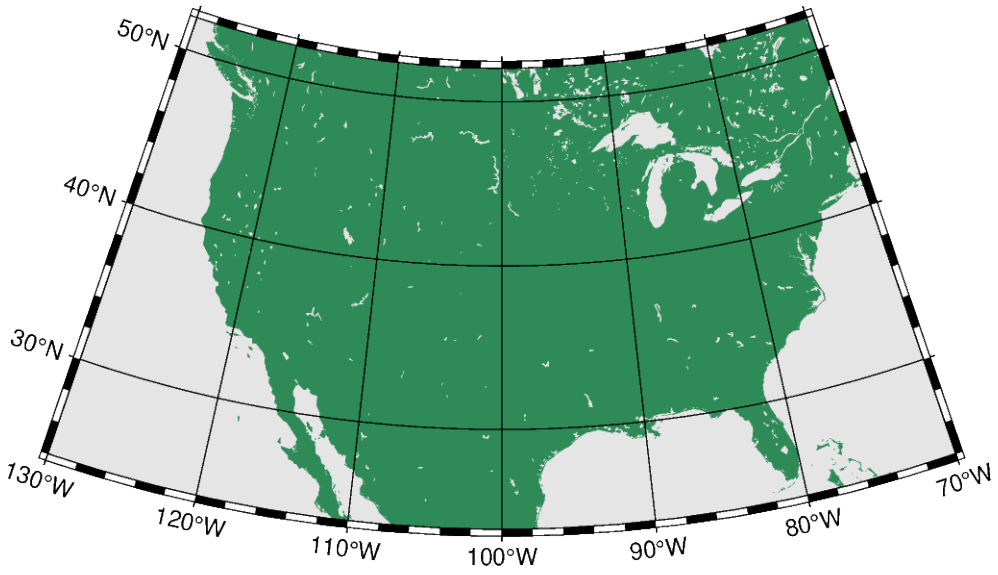
Tags: equidistant

9.2.3 Lambert conic conformal projection

This conic projection was designed by the Alsatian mathematician Johann Heinrich Lambert (1772) and has been used extensively for mapping of regions with predominantly east-west orientation, just like the Albers projection. Unlike the Albers projection, Lambert's conformal projection is not equal-area. The parallels are arcs of circles with a common origin, and meridians are the equally spaced radii of these circles. As with Albers projection, it is only the two standard parallels that are distortion-free.

l*lon0/lat0/lat1/lat2/scale* or **L***lon0/lat0/lat1/lat2/width*

- **l** or **L**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *lat1/lat2*: Sets the two standard parallels.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[-130, -70, 24, 52],
    projection="L-100/35/33/45/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="seagreen",
    water="gray90",
)
fig.show()
```

Total running time of the script: (0 minutes 0.145 seconds)

Tags: conformal

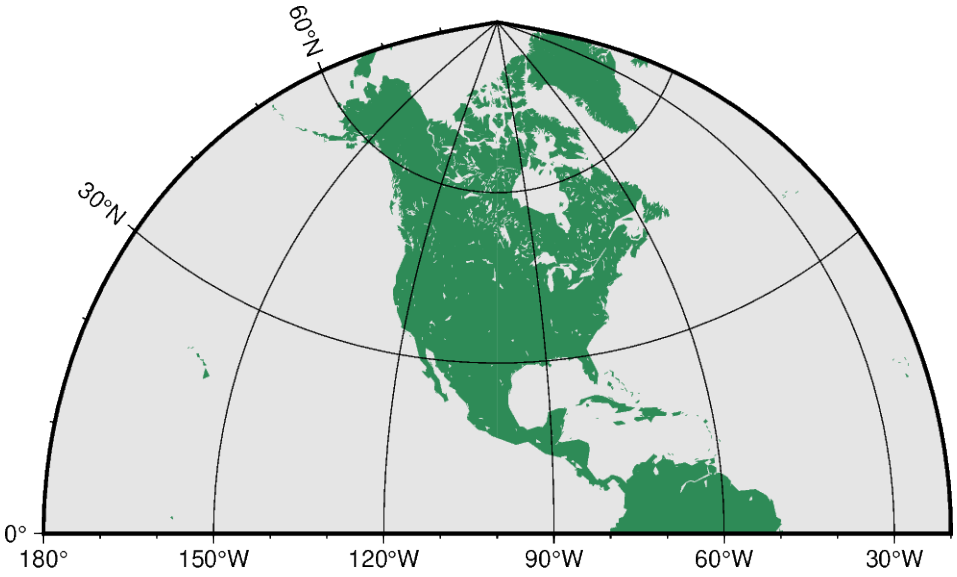
9.2.4 Polyconic projection

The polyconic projection, in Europe usually referred to as the American polyconic projection, was introduced shortly before 1820 by the Swiss-American cartographer Ferdinand Rodulph Hassler (1770-1843). As head of the Survey of the Coast, he was looking for a projection that would give the least distortion for mapping the coast of the United States. The projection acquired its name from the construction of each parallel, which is achieved by projecting the parallel onto the cone while it is rolled around the globe, along the central meridian, tangent to that parallel. As a consequence, the projection involves many cones rather than a single one used in regular conic projections.

The polyconic projection is neither equal-area, nor conformal. It is true to scale without distortion along the central meridian. Each parallel is true to scale as well, but the meridians are not as they get further away from the central meridian. As a consequence, no parallel is standard because conformity is lost with the lengthening of the meridians.

poly/[*lon0*/[*lat0*/]]*scale* or **Poly**/[*lon0*/[*lat0*/]]*width*

- **poly** or **Poly**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *lat0*: Sets the reference parallel [Optional]. When supplied, *lon0* must be supplied as well.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[-180, -20, 0, 90],
    projection="Poly/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="seagreen",
    water="gray90",
)
fig.show()
```

Total running time of the script: (0 minutes 0.132 seconds)

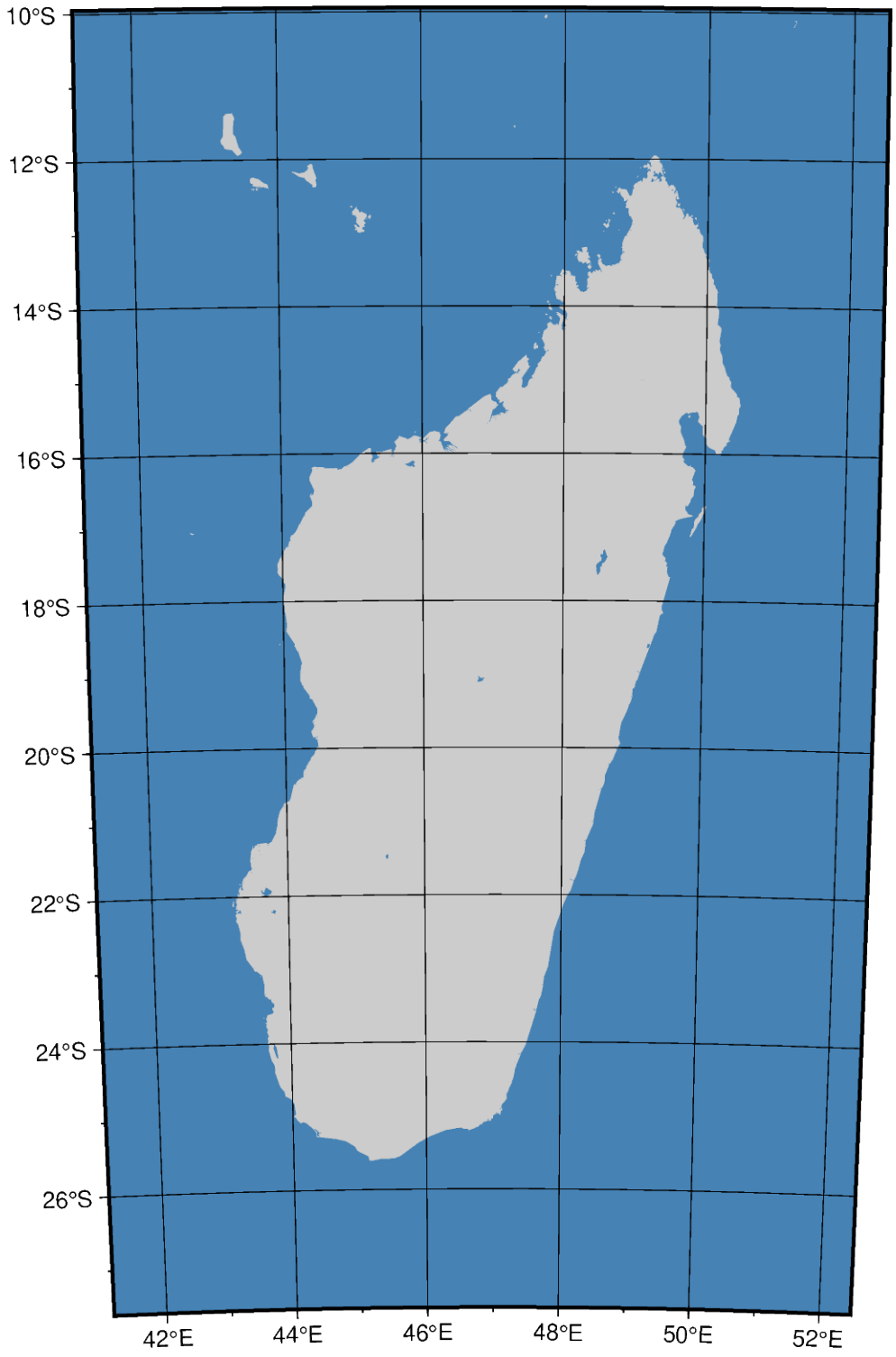
9.3 Cylindric Projections

9.3.1 Cassini cylindrical projection

This cylindrical projection was developed in 1745 by César-François Cassini de Thury for the survey of France. It is occasionally called Cassini-Soldner since the latter provided the more accurate mathematical analysis that led to the development of the ellipsoidal formulae. The projection is neither conformal nor equal-area, and behaves as a compromise between the two end-members. The distortion is zero along the central meridian. It is best suited for mapping regions of north-south extent. The central meridian, each meridian 90° away, and equator are straight lines; all other meridians and parallels are complex curves.

c*lon0/lat0/scale* or **C***lon0/lat0/width*

- **c** or **C**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use the ISO code for Madagascar (MG) and pad it by 2 degrees (+R2)
fig.coast(
```

(continues on next page)

(continued from previous page)

```
    region="MG+R2",
    projection="C47/-19/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

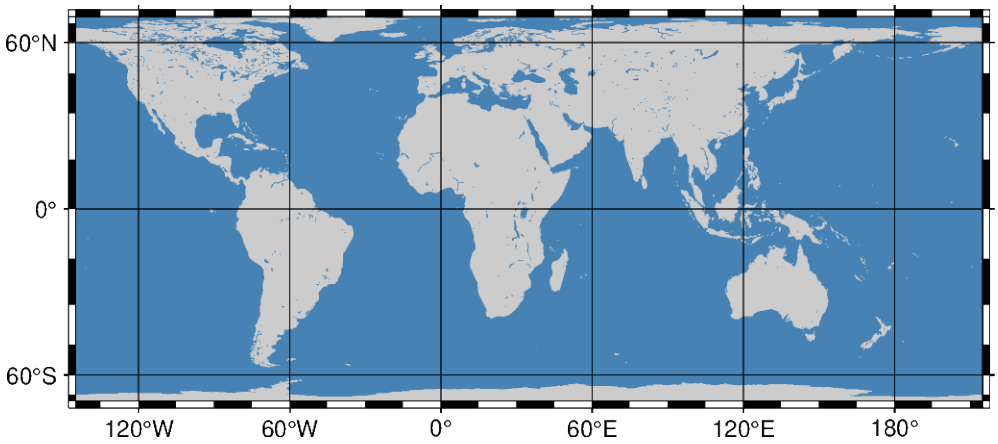
Total running time of the script: (0 minutes 0.242 seconds)

9.3.2 Cylindrical equal-area projection

This cylindrical projection is actually several projections, depending on what latitude is selected as the standard parallel. However, they are all equal area and hence non-conformal. All meridians and parallels are straight lines.

y*lon0/lat0/scale* or **Y***lon0/lat0/width*

- **y** or **Y**: Sets the projection type.
- *lon0/lat0*: Sets the projection center.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="Y35/30/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.148 seconds)

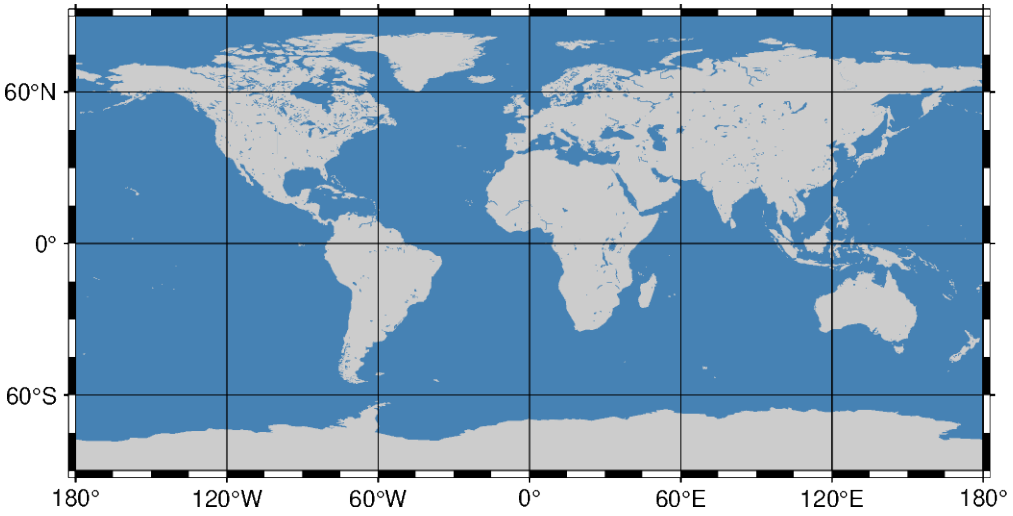
Tags: equal-area

9.3.3 Cylindrical equidistant projection

This simple cylindrical projection is really a linear scaling of longitudes and latitudes. The most common form is the Plate Carrée projection, where the scaling of longitudes and latitudes is the same. All meridians and parallels are straight lines.

q[*lon0*/[*lat0*/]]*scale* or **Q**[*lon0*/[*lat0*/]]*width*

- **q** or **Q**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional, default is the middle of the map].
- *lat0*: Sets the standard parallel [Optional, default is the equator]. When supplied, *lon0* must be supplied as well.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="Q12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.153 seconds)

Tags: equidistant

9.3.4 Cylindrical stereographic projection

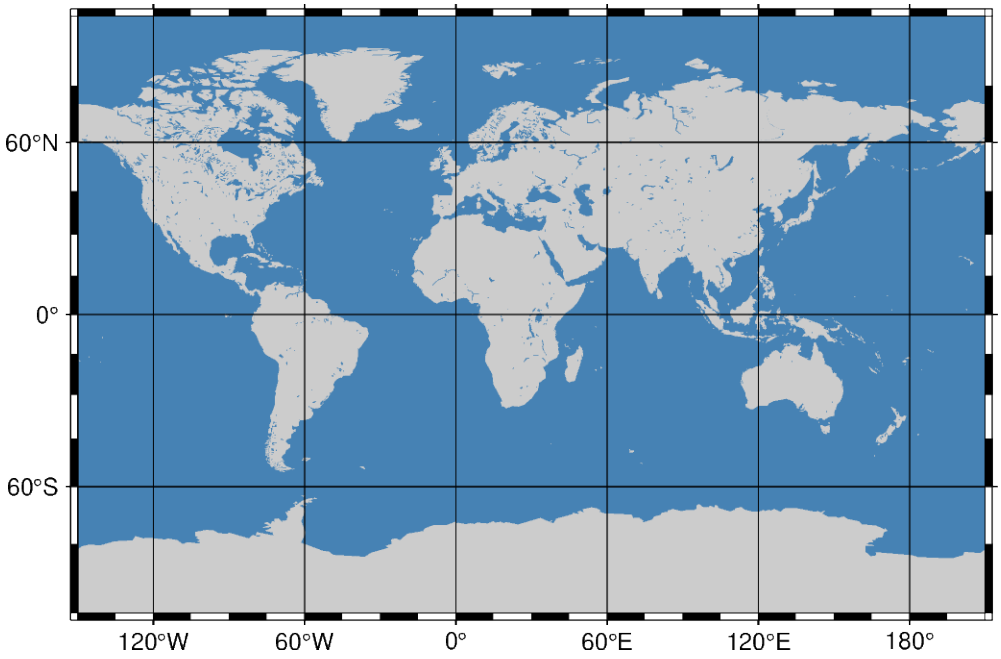
The cylindrical stereographic projections are certainly not as notable as other cylindrical projections, but are still used because of their relative simplicity and their ability to overcome some of the downsides of other cylindrical projections, like extreme distortions of the higher latitudes. The stereographic projections are perspective projections, projecting the sphere onto a cylinder in the direction of the antipodal point on the equator. The cylinder crosses the sphere at two standard parallels, equidistant from the equator.

cyl_stere/[*lon0*/[*lat0*/]]*scale* or **Cyl_stere/**[*lon0*/[*lat0*/]]*width*

- **cyl_stere** or **Cyl_stere**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *lat0*: Sets the standard parallel [Optional]. When supplied, *lon0* must be supplied as well.
- *scale* or *width*: Sets the map size.

The standard parallel is typically one of these (but can be any value):

- 66.159467 - Miller's modified Gall
- 55 - Kamenetskiy's First
- 45 - Gall's Stereographic
- 30 - Bolshoi Sovietskii Atlas Mira or Kamenetskiy's Second
- 0 - Braun's Cylindrical




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region="g",
    projection="Cyl_stere/30/-20/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

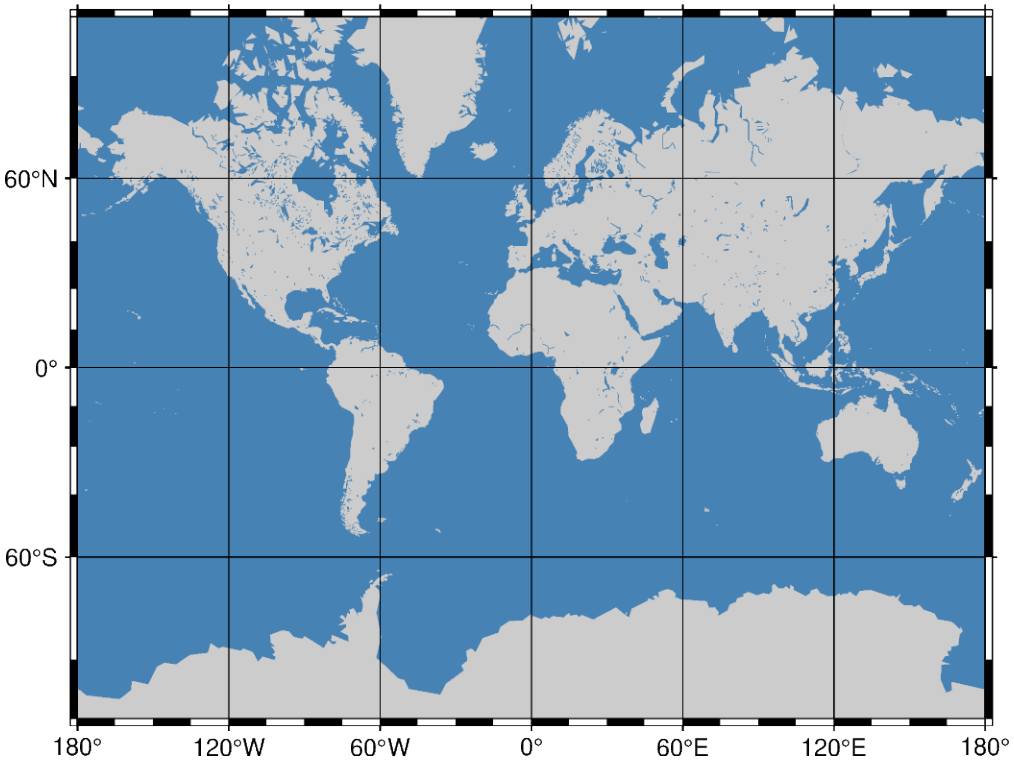
Total running time of the script: (0 minutes 0.161 seconds)

9.3.5 Mercator projection

The Mercator projection takes its name from the Flemish cartographer Gheert Cremer, better known as Gerardus Mercator, who presented it in 1569. The projection is a cylindrical and conformal, with no distortion along the equator. A major navigational feature of the projection is that a line of constant azimuth is straight. Such a line is called a rhumb line or loxodrome. Thus, to sail from one point to another one only had to connect the points with a straight line, determine the azimuth of the line, and keep this constant course for the entire voyage. The Mercator projection has been used extensively for world maps in which the distortion towards the polar regions grows rather large.

m[*lon0*/[*lat0*/]]*scale* or **M**[*lon0*/[*lat0*/]]*width*

- **m** or **M**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *lat0*: Sets the standard parallel [Optional]. When supplied, *lon0* must be supplied as well.
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[0, 360, -80, 80],
    projection="M0/0/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.168 seconds)

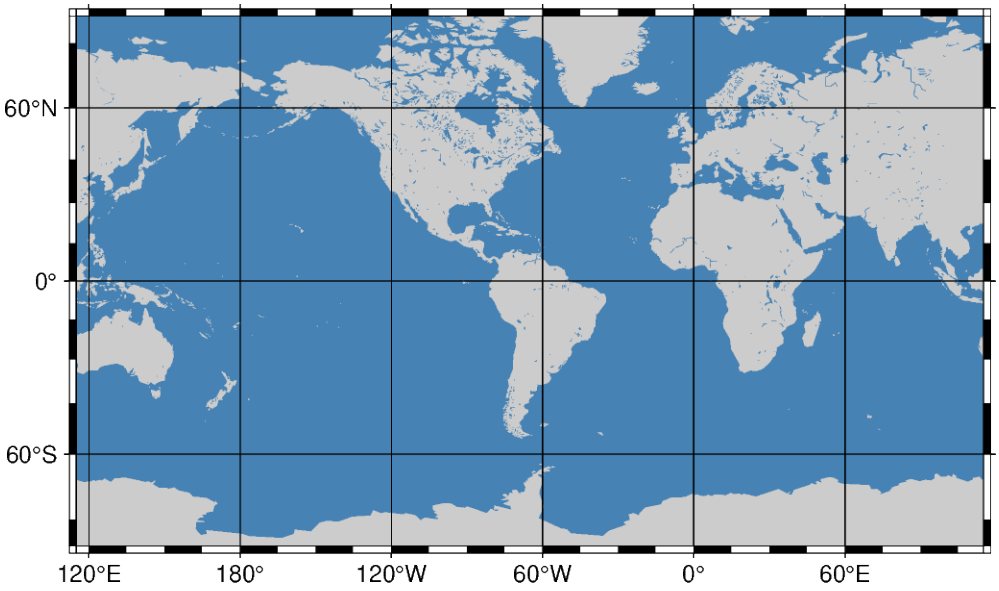
Tags: conformal

9.3.6 Miller cylindrical projection

This cylindrical projection, presented by Osborn Maitland Miller of the American Geographic Society in 1942, is neither equal nor conformal. All meridians and parallels are straight lines. The projection was designed to be a compromise between Mercator and other cylindrical projections. Specifically, Miller spaced the parallels by using Mercator's formula with 0.8 times the actual latitude, thus avoiding the singular poles; the result was then divided by 0.8.

j[*lon0/*]*/scale* or **J**[*lon0/*]*/width*

- **j** or **J**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    region=[-180, 180, -80, 80],
    projection="J-65/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.158 seconds)

9.3.7 Oblique Mercator projection

Oblique configurations of the cylinder give rise to the oblique Mercator projection. It is particularly useful when mapping regions of large lateral extent in an oblique direction. Both parallels and meridians are complex curves. The projection was developed in the early 1900s by several workers.

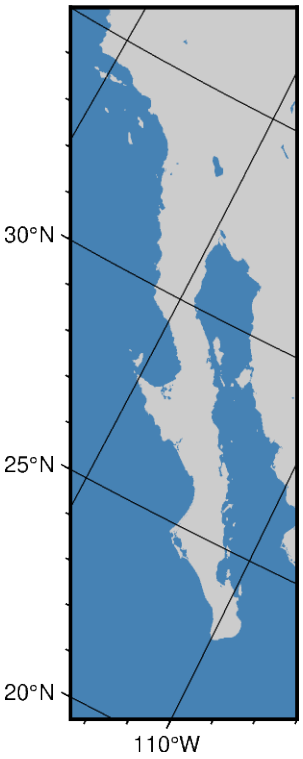
The projection is set with **o** or **O**. There are three different specification ways (**a**|**A**, **b**|**B**, **c**|**C**) available. For all three definitions, the uppercase letter means the projection pole is set in the southern hemisphere [Default is northern hemisphere].

1. Using the origin and azimuth

oa|**oA***lon0/lat0/azimuth/scale*[**+v**] or **Oa**|**OA***lon0/lat0/azimuth/width*[**+v**]

- *lon0/lat0*: Sets the projection center.
- *azimuth*: Sets the azimuth of the oblique equator.
- *scale* or *width*: Sets the map size.
- **+v**: Aligns the oblique equator with the y-axis [Optional].

```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
fig.coast(
    projection="Oa-120/25/-30/3c+v",
    # Set bottom left and top right coordinates of the map with "+r"
    region="-122/35/-107/22+r",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

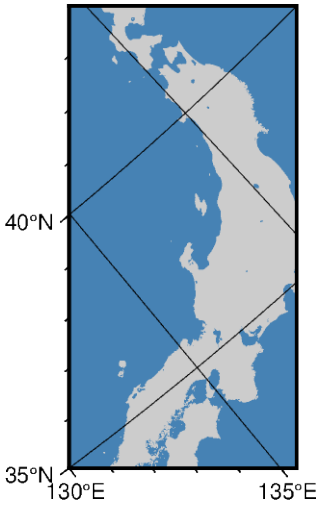


2. Using two points

ob|oB*lon0/lat0/lon1/lat1/scale*[**+v**] or **Ob|OB***lon0/lat0/lon1/lat1/width*[**+v**]

- *lon0/lat0*: Sets the projection center.
- *lon1/lat1*: Sets a second point on the oblique equator.
- *scale* or *width*: Sets the map size.
- **+v**: Aligns the oblique equator with the y-axis [Optional].

```
fig = pygmt.Figure()
fig.coast(
    projection="Ob130/35/25/35/3c",
    region="130/35/145/40+r",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```



3. Using the origin and projection pole

oc|oC*lon0/lat0/lonp/latp/scale*[**+v**] or **Oc|OC***lon0/lat0/lonp/latp/width*[**+v**]

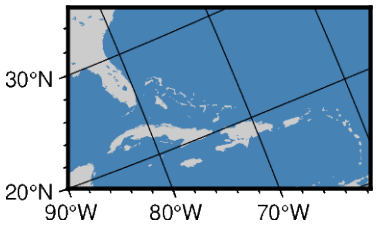
- *lon0/lat0*: Sets the projection center.
- *lonp/latp*: Sets the projection pole.
- *scale* or *width*: Sets the map size.
- **+v**: Aligns the oblique equator with the y-axis [Optional].

```
fig = pygmt.Figure()
fig.coast(
    projection="Oc280/25.5/22/69/4c",
    region="270/20/305/25+r",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
```

(continues on next page)

(continued from previous page)

```
)
fig.show()
```



Total running time of the script: (0 minutes 0.314 seconds)

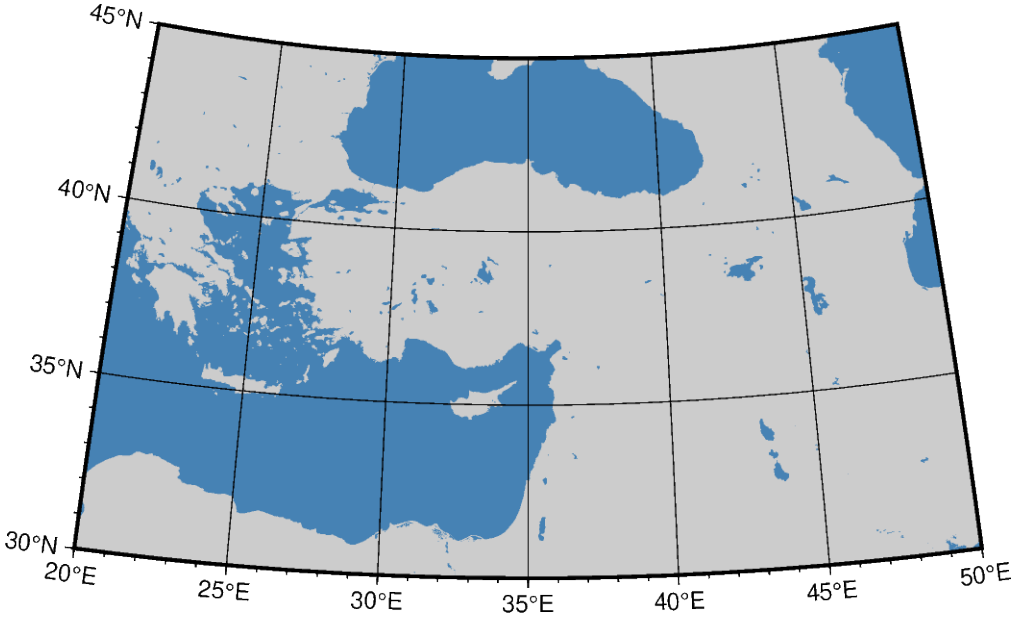
Tags: conformal

9.3.8 Transverse Mercator projection

The transverse Mercator was invented by Johann Heinrich Lambert in 1772. In this projection the cylinder touches a meridian along which there is no distortion. The distortion increases away from the central meridian and goes to infinity at 90° from center. The central meridian, each meridian 90° away from the center, and equator are straight lines; other parallels and meridians are complex curves.

t*lon0*[/*lat0*]/*scale* or **T***lon0*[/*lat0*]/*width*

- **t** or **T**: Sets the projection type.
- *lon0*: Sets the central meridian.
- *lat0*: Sets the latitude of origin [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
```

(continues on next page)

(continued from previous page)

```
fig.coast(
    region=[20, 50, 30, 45],
    projection="T35/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.141 seconds)

Tags: conformal

9.3.9 Universal Transverse Mercator projection

A particular subset of the *transverse Merctor* is the Universal Transverse Mercator (UTM) which was adopted by the US Army for large-scale military maps. Here, the globe is divided into 60 zones between 84°S and 84°N, most of which are 6° (in longitude) wide. Each of these UTM zones have their unique central meridian. Furthermore, each zone is divided into latitude bands but these are not needed to specify the projection for most cases. See Figure *Universal Transverse Mercator* for all zone designations.

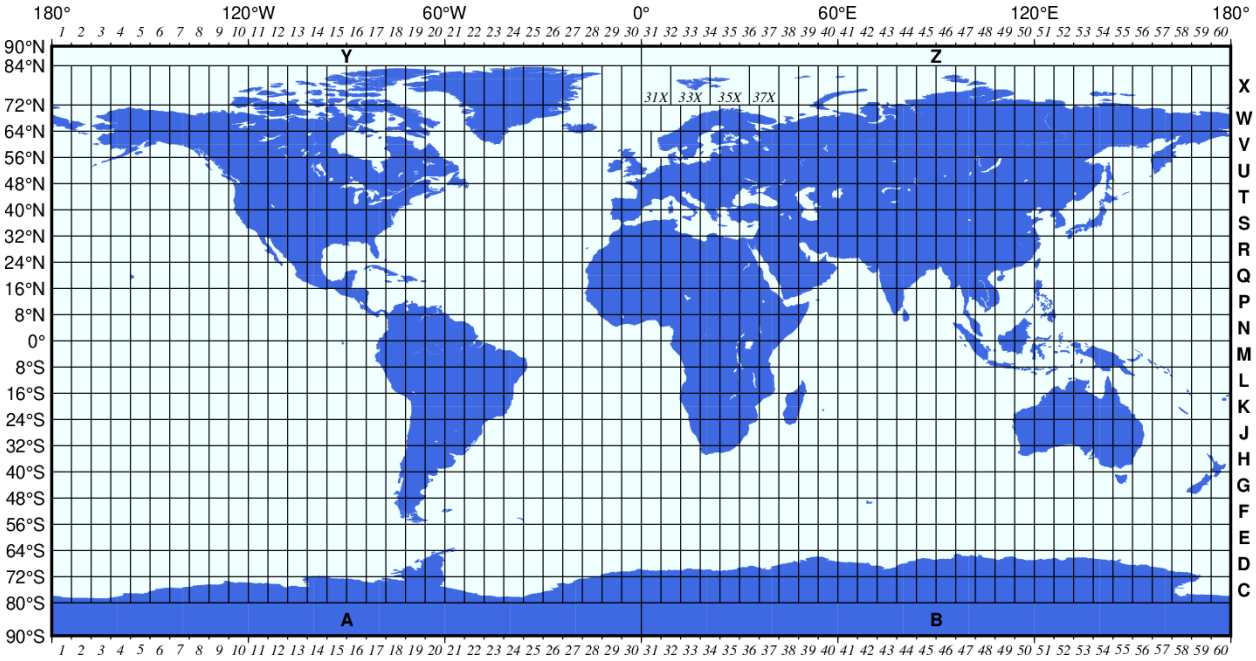


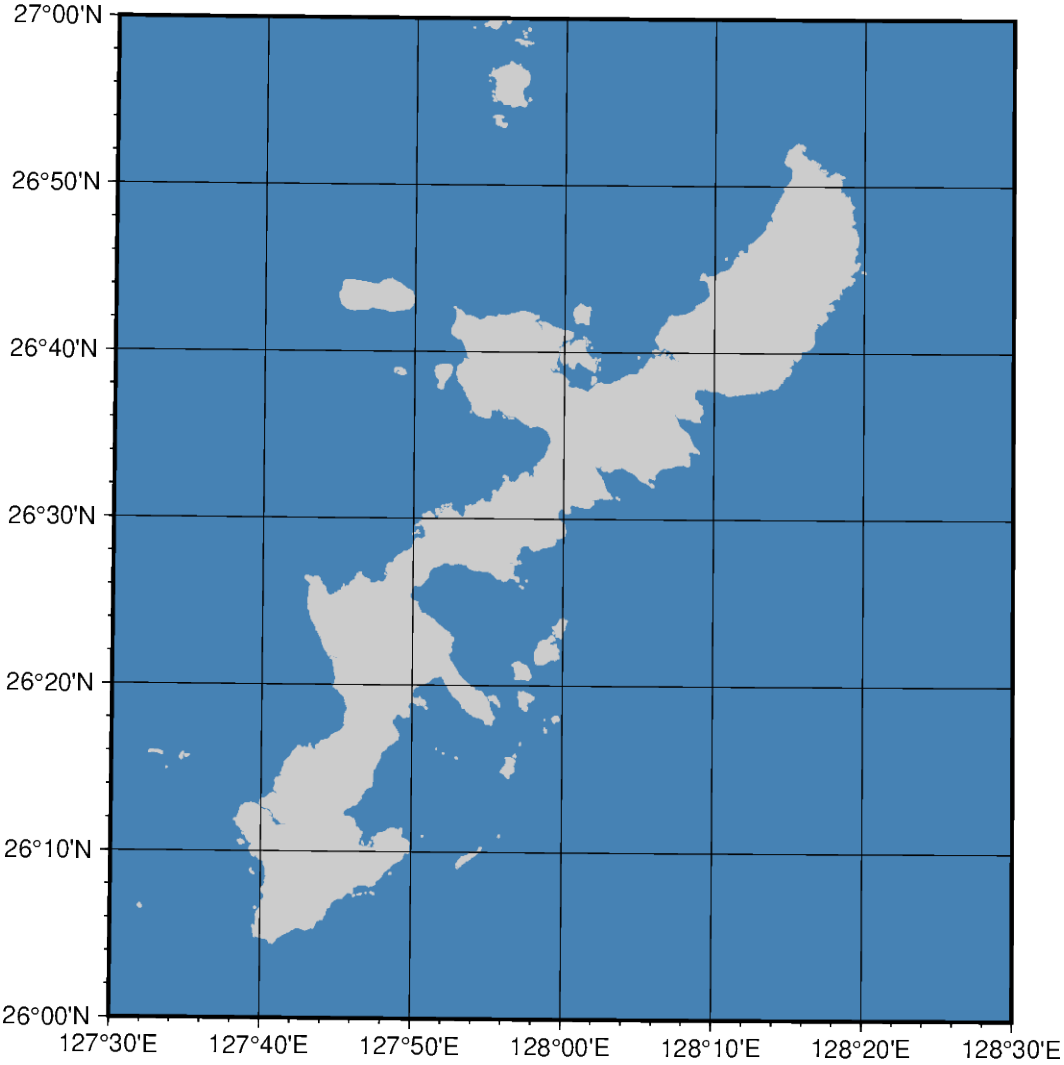
Fig. 1: Universal Transverse Mercator zone layout.

In order to minimize the distortion in any given zone, a scale factor of 0.9996 has been factored into the formulae (although a standard, you can change this with PROJ_SCALE_FACTOR). This makes the UTM projection a *secant* projection and not a *tangent* projection like the *transverse Merctor*. The scale only varies by 1 part in 1,000 from true scale at equator. The ellipsoidal projection expressions are accurate for map areas that extend less than 10° away from the central meridian. For larger regions we use the conformal latitude in the general spherical formulae instead.

u*zone/scale* or **U***zone/width*

- **u** or **U**: Sets the projection type.
- *zone*: Sets the UTM zone.

- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# UTM Zone is set to 52R
fig.coast(
    region=[127.5, 128.5, 26, 27],
    projection="U52R/12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="gray80",
    water="steelblue",
)
fig.show()
```

Total running time of the script: (0 minutes 0.161 seconds)

Tags: conformal

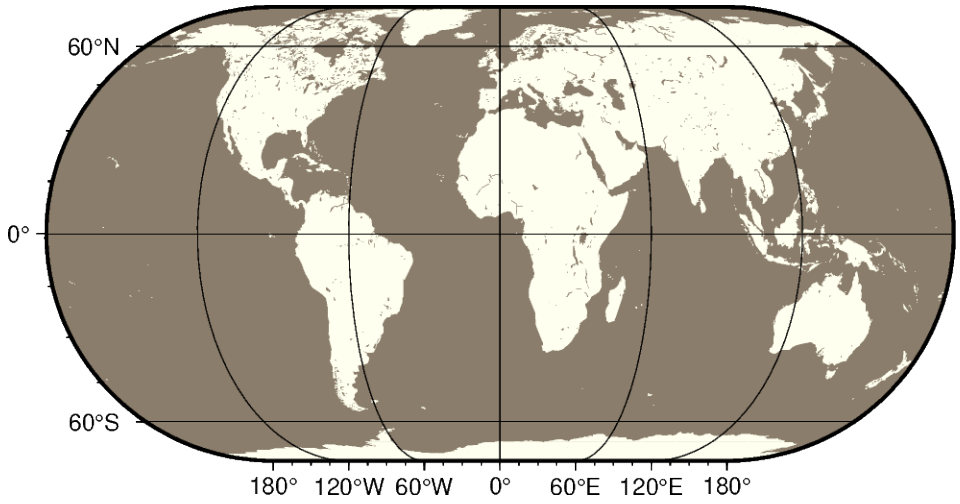
9.4 Miscellaneous Projections

9.4.1 Eckert IV equal-area projection

The Eckert IV projection, presented by the German cartographer Max Eckert-Greiffendorff in 1906, is a pseudo-cylindrical equal-area projection. Central meridian and all parallels are straight lines; other meridians are equally spaced elliptical arcs. The scale is true along latitude 40°30'.

kf[*lon0/*]*scale* or **Kf**[*lon0/*]*width*

- **kf** or **Kf**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="Kf12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.170 seconds)

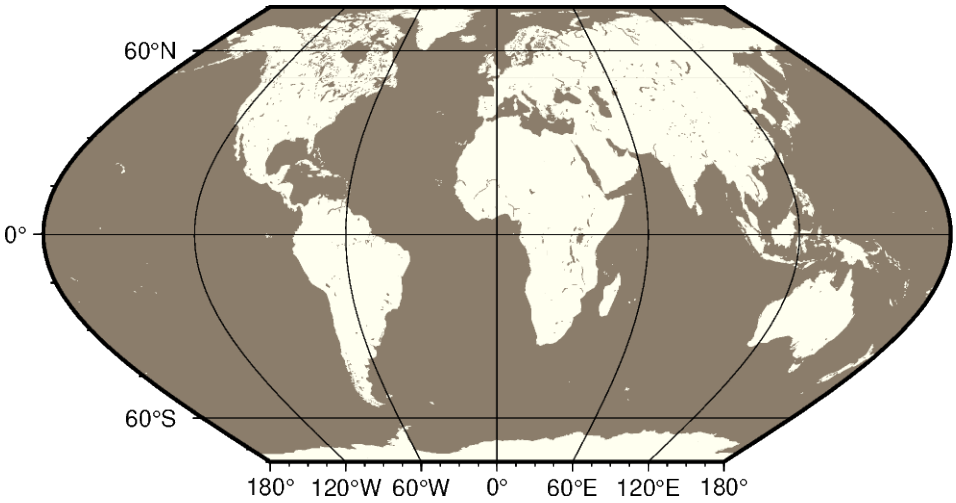
Tags: equal-area

9.4.2 Eckert VI equal-area projection

The Eckert VI projections, presented by the German cartographer Max Eckert-Greiffendorff in 1906, is a pseudo-cylindrical equal-area projection. Central meridian and all parallels are straight lines; other meridians are equally spaced sinusoids. The scale is true along latitude 49°16'.

ks[*lon0/*]*scale* or **Ks**[*lon0/*]*width*

- **ks** or **Ks**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="Ks12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.163 seconds)

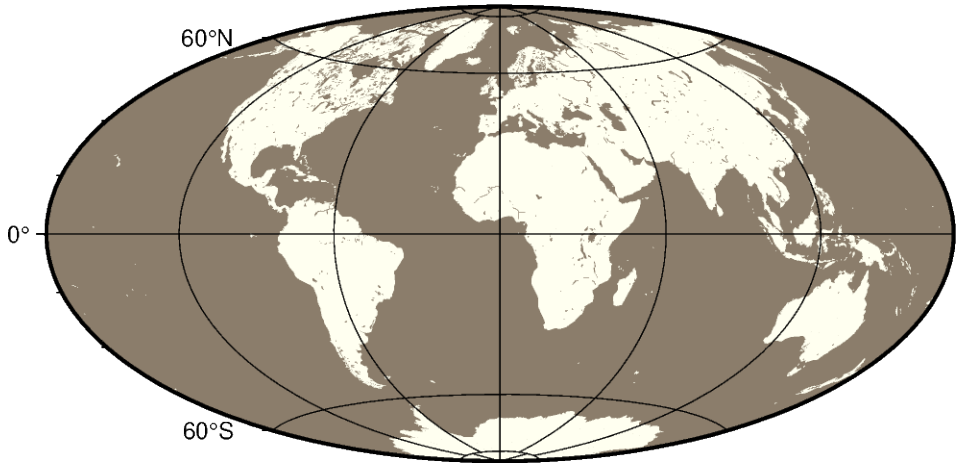
Tags: equal-area

9.4.3 Hammer projection

The equal-area Hammer projection, first presented by the German mathematician Ernst von Hammer in 1892, is also known as Hammer-Aitoff (the Aitoff projection looks similar, but is not equal-area). The border is an ellipse, equator and central meridian are straight lines, while other parallels and meridians are complex curves.

h[*lon0/*]*scale* or **H**[*lon0/*]*width*

- **h** or **H**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="H12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.160 seconds)

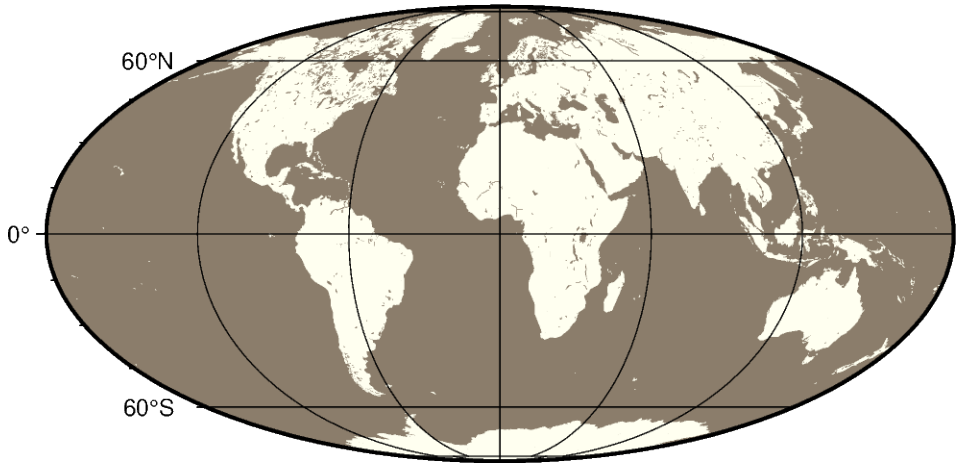
Tags: equal-area

9.4.4 Mollweide projection

This pseudo-cylindrical, equal-area projection was developed by the German mathematician and astronomer Karl Brandan Mollweide in 1805. Parallels are unequally spaced straight lines with the meridians being equally spaced elliptical arcs. The scale is only true along latitudes 40°44' north and south. The projection is used mainly for global maps showing data distributions. It is occasionally referenced under the name homalographic projection.

w[*lon0/*]*scale* or **W**[*lon0/*]*width*

- **w** or **W**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="W12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.171 seconds)

Tags: equal-area

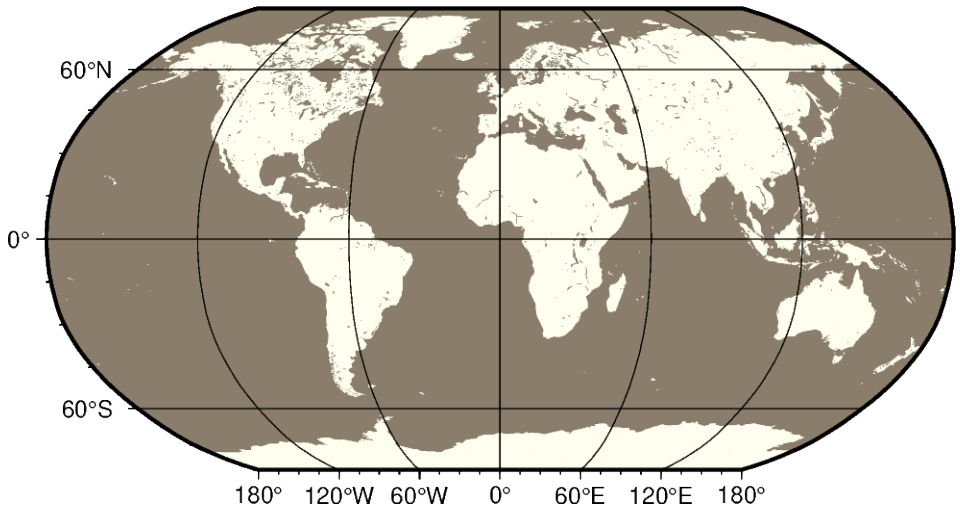
9.4.5 Robinson projection

The Robinson projection, presented by the American geographer and cartographer Arthur H. Robinson in 1963, is a modified cylindrical projection that is neither conformal nor equal-area. Central meridian and all parallels are straight lines; other meridians are curved. It uses lookup tables rather than analytic expressions to make the world map "look" right[1]. The scale is true along latitudes 38° north and south. The projection was originally developed for use by Rand McNally and is currently used by the National Geographic Society.

n[*lon0/*]*scale* or **N**[*lon0/*]*width*

- **n** or **N**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.

[1] Robinson provided a table of y-coordinates for latitudes every 5°. To project values for intermediate latitudes one must interpolate the table. Different interpolants may result in slightly different maps. GMT uses the interpolant selected by the parameter GMT_INTERPOLANT in the gmt.conf file.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="N12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

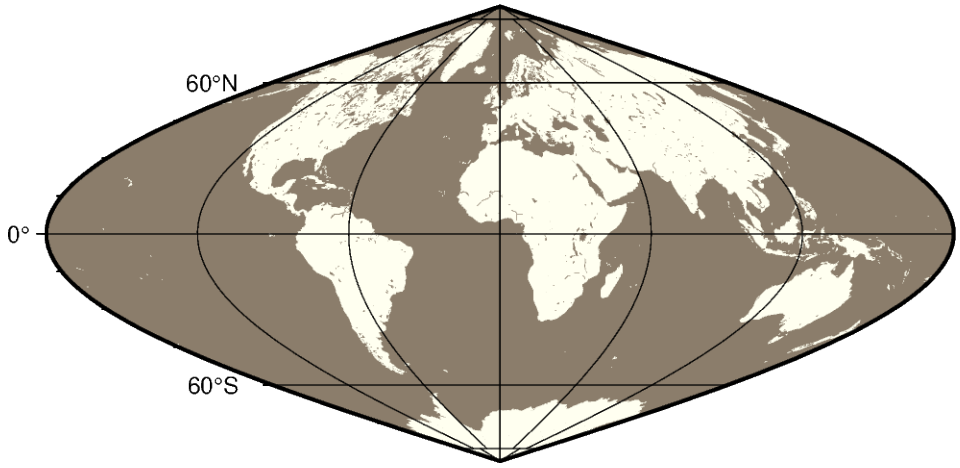
Total running time of the script: (0 minutes 0.159 seconds)

9.4.6 Sinusoidal projection

The sinusoidal projection is one of the oldest known projections, is equal-area, and has been used since the mid-16th century. It has also been called the "Equal-area Mercator" projection. The central meridian is a straight line; all other meridians are sinusoidal curves. Parallels are all equally spaced straight lines, with scale being true along all parallels (and central meridian).

i[*lon0/*]*scale* or **I**[*lon0/*]*width*

- **i** or **I**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="I12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.156 seconds)

Tags: equal-area

9.4.7 Spilhaus projection

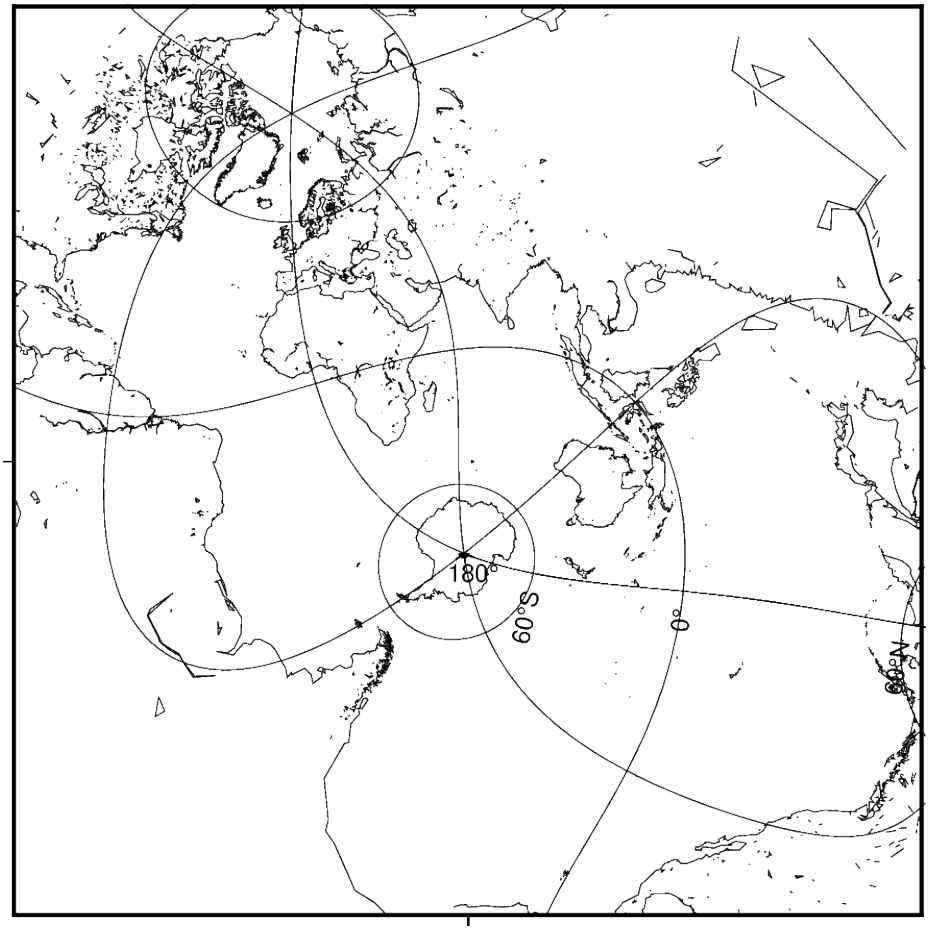
The Spilhaus projection is a world map projection that presents the world's oceans as one contiguous body of water, with Antarctica at the top. It was developed by Athelstan Spilhaus and is useful for oceanographic studies.

+proj=spilhaus+scale=*scale* or **+proj=spilhaus+width=***width*

- **+proj=spilhaus**: Sets the projection type.
- *scale* or *width*: Sets the map size.

Note

This projection works well for coastlines but currently has issues for filling land and water masses.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use the PROJ string to set the Spilhaus projection
fig.coast(
    region="d",
    projection="+proj=spilhaus+width=12c",
    frame=Axis(annot=True, tick=True, grid=True),
    shorelines=True,
)
fig.show()
```

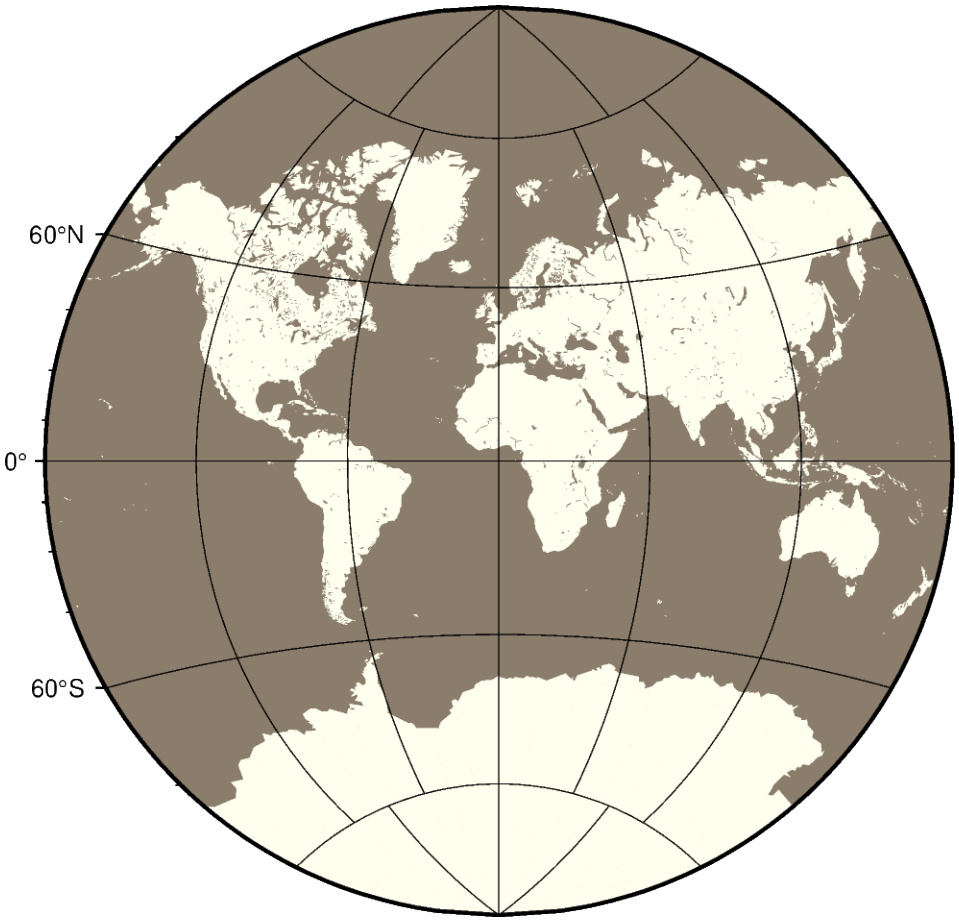
Total running time of the script: (0 minutes 0.212 seconds)

9.4.8 Van der Grinten projection

The Van der Grinten projection, presented by Alphons J. van der Grinten in 1904, is neither equal-area nor conformal. Central meridian and Equator are straight lines; other meridians are arcs of circles. The scale is true along the Equator only. Its main use is to show the entire world enclosed in a circle.

v[*lon0/*]*scale* or **V**[*lon0/*]*width*

- **v** or **V**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="V12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.187 seconds)

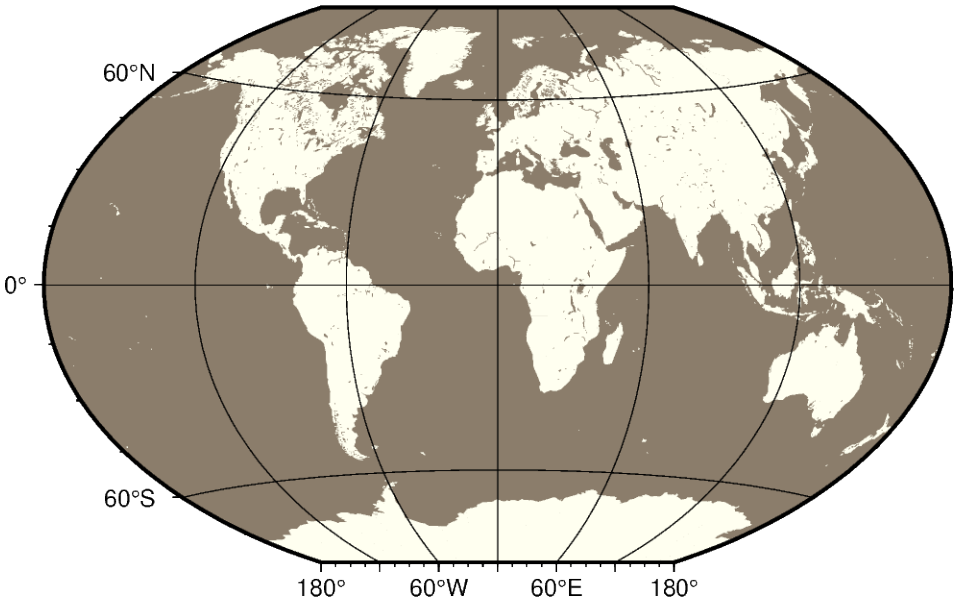
9.4.9 Winkel Tripel projection

In 1921, the German mathematician Oswald Winkel made a projection that was to strike a compromise between the properties of three elements (area, angle and distance). The German word "tripel" refers to this junction of where each of these elements are least distorted when plotting global maps. The projection was popularized when Bartholomew and Son started to use it in its world-renowned "The Times Atlas of the World" in the mid-20th century. In 1998, the National Geographic Society made the Winkel Tripel as its map projection of choice for global maps.

Naturally, this projection is neither conformal, nor equal-area. Central meridian and equator are straight lines; other parallels and meridians are curved. The projection is obtained by averaging the coordinates of the Equidistant Cylindrical and Aitoff (not Hammer-Aitoff) projections. The poles map into straight lines 0.4 times the length of equator.

r[*lon0/*]*scale* or **R**[*lon0/*]*width*

- **r** or **R**: Sets the projection type.
- *lon0*: Sets the central meridian [Optional].
- *scale* or *width*: Sets the map size.




```
import pygmt
from pygmt.params import Axis

fig = pygmt.Figure()
# Use region "d" to specify global region (-180/180/-90/90)
fig.coast(
    region="d",
    projection="R12c",
    frame=Axis(annot=True, tick=True, grid=True),
    land="ivory",
    water="bisque4",
)
fig.show()
```

Total running time of the script: (0 minutes 0.188 seconds)

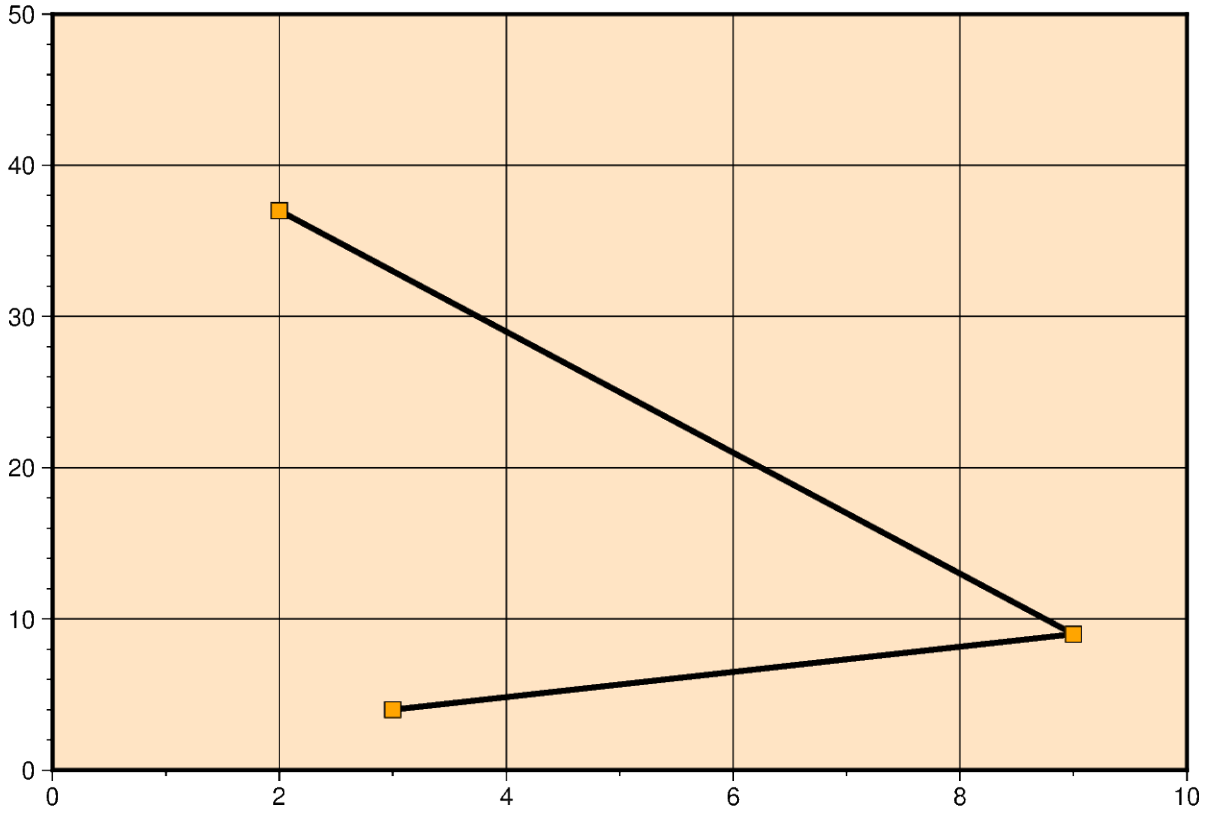
9.5 Non-geographic Projections

9.5.1 Cartesian linear

X*width*[/*height*] or **x***x-scale*[/*y-scale*]

- **X** or **x**: Sets the projection type.
- *width* or *x-scale*: Sets the plot size.
- *height* or *y-scale*: Sets the plot height [Optional].

The Cartesian linear projection is primarily designed for regular floating point data. To plot geographical data in a linear projection, see the upstream GMT documentation Geographic coordinates. To make the linear plot using calendar date/time as input coordinates, see the tutorial *Plotting datetime charts*. GMT documentation Calendar time coordinates.




```
import pygmt
from pygmt.params import Axis, Frame

fig = pygmt.Figure()
# The region parameter is specified as xmin, xmax, ymin, ymax
fig.basemap(
    region=[0, 10, 0, 50],
    projection="X15c/10c",
    frame=Frame(fill="bisque", axis=Axis(annot=True, tick=True, grid=True)),
)
fig.plot(x=[3, 9, 2], y=[4, 9, 37], pen="2p,black")
# Plot data points on top of the line
# Use squares with a size of 0.3 centimeters, an "orange" fill and a "black" outline
```

(continues on next page)

(continued from previous page)

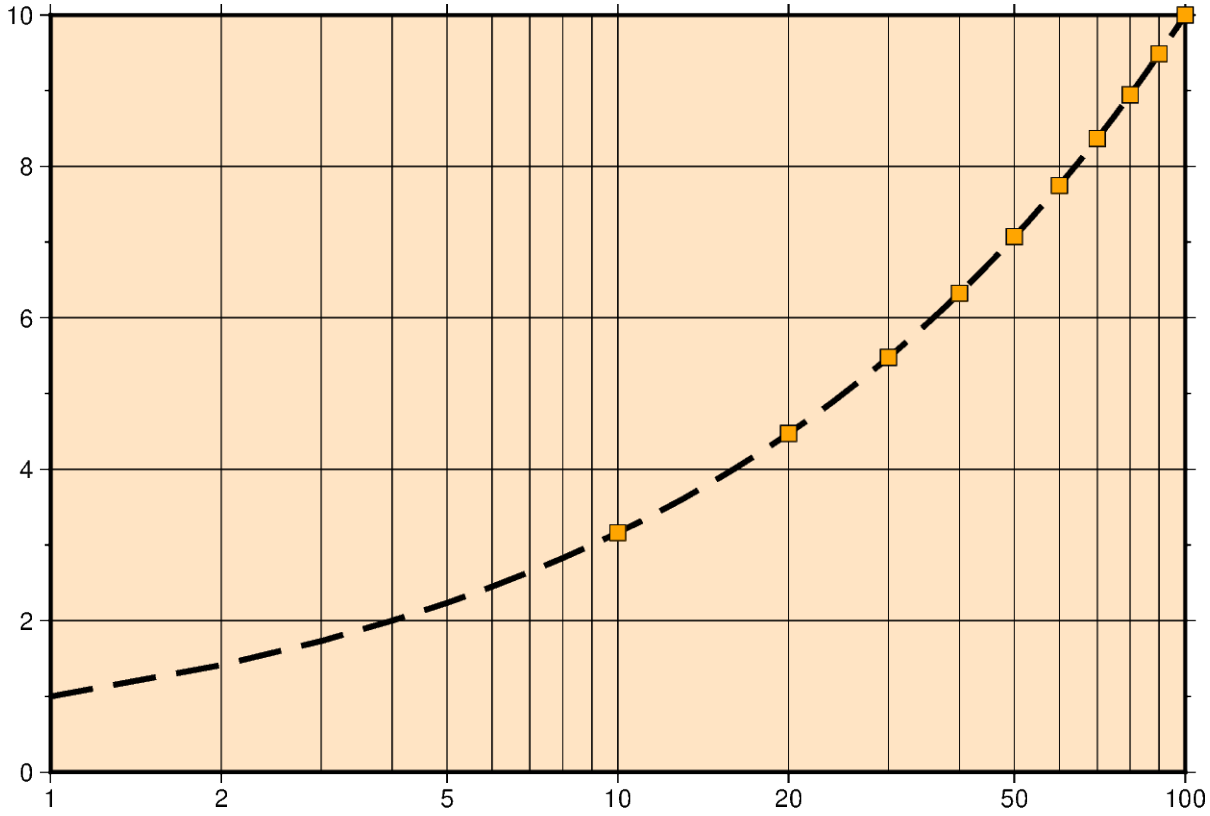
```
fig.plot(x=[3, 9, 2], y=[4, 9, 37], style="s0.3c", fill="orange", pen="black")
fig.show()
```

Total running time of the script: (0 minutes 0.116 seconds)

9.5.2 Cartesian logarithmic

X*width*[**l**][/*height*[**l**]] or **x***x-scale*[**l**][/*y-scale*[**l**]]

- **X** or **x**: Sets the projection type.
- *width* or *x-scale*: Sets the plot size.
- *height* or *y-scale*: Sets the plot height [Optional].
- **l**: Applies a logarithmic transformation to an axis. Append it after the corresponding size argument [Optional].




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# Create a list of x-values 0-100
xline = np.arange(0, 101)
# Create a list of y-values that are the square root of the x-values
yline = xline**0.5
# Create a list of x-values for every 10 in 0-100
xpoints = np.arange(0, 101, 10)
# Create a list of y-values that are the square root of the x-values
```

(continues on next page)

(continued from previous page)

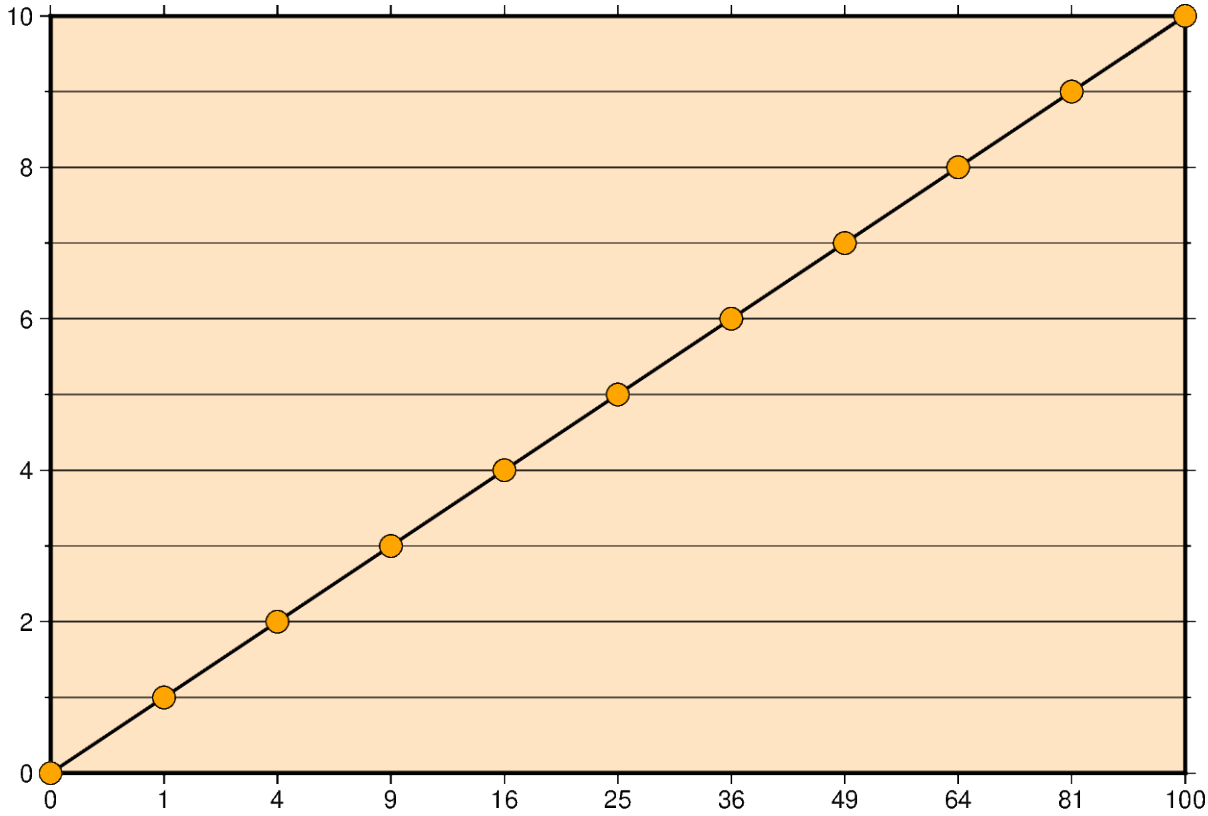
```
ypoints = xpoints**0.5

fig = pygmt.Figure()
fig.basemap(
    region=[1, 100, 0, 10],
    # Set a logarithmic transformation on the x-axis
    projection="X15cl/10c",
    # Set the plot's frame and color as well as annotations, ticks, and gridlines
    frame=Frame(
        axes="WSne",
        fill="bisque",
        xaxis=Axis(annot=2, grid=3),
        yaxis=Axis(annot=2, tick=1, grid=2),
    ),
)

# Set the line thickness to "2p", the color to "black", and the style to "dashed"
fig.plot(x=xline, y=yline, pen="2p,black,dashed")

# Plot the square root values on top of the line
# Use squares with a size of 0.3 centimeters, an "orange" fill and a "black" outline
# Symbols are not clipped if they go off the plot frame
fig.plot(x=xpoints, y=ypoints, style="s0.3c", fill="orange", pen="black",␣
↪no_clip=True)
fig.show()
```

Total running time of the script: (0 minutes 0.125 seconds)

9.5.3 Cartesian power

X*width*[**p***pvalue*][/*height*[**p***pvalue*]] or **x***x-scale*[**p***pvalue*][/*y-scale*[**p***pvalue*]]

- **X** or **x**: Sets the projection type.
- *width* or *x-scale*: Sets the plot size.
- *height* or *y-scale*: Sets the plot height [Optional].
- **p***pvalue*: Applies a power transformation with exponent *pvalue* to an axis. Append it after the corresponding size argument [Optional].




```
import numpy as np
import pygmt
from pygmt.params import Axis, Frame

# Create a list of y-values 0-10
yvalues = np.arange(0, 11)
# Create a list of x-values that are the square of the y-values
xvalues = yvalues**2

fig = pygmt.Figure()
fig.basemap(
    region=[0, 100, 0, 10],
    # Set the power transformation of the x-axis, with a power of 0.5
    projection="X15cp0.5/10c",
    # Set the figures frame as well as annotations and ticks
    # The "p" forces to show only square numbers as annotations of the x-axis
    frame=Frame(
        axes="WSne",
        fill="bisque",
        xaxis=Axis(annot="1p", tick=True, grid=True),
        yaxis=Axis(annot=2, tick=1, grid=True),
    ),
)

# Set the line thickness to "thick" (equals "1p", i.e. 1 point)
# Use as color "black" (default) and as style "solid" (default)
fig.plot(x=xvalues, y=yvalues, pen="thick,black,solid")
```

(continues on next page)

(continued from previous page)

```
# Plot the data points on top of the line
# Use circles with 0.3 centimeters diameter, with an "orange" fill and a "black"␣
↪outline
# Symbols are not clipped if they go off the plot frame
fig.plot(x=xvalues, y=yvalues, style="c0.3c", fill="orange", pen="black",␣
↪no_clip=True)
fig.show()
```

Total running time of the script: (0 minutes 0.120 seconds)

9.5.4 Polar

Polar projections allow plotting polar coordinate data (e.g. angle θ and radius r).

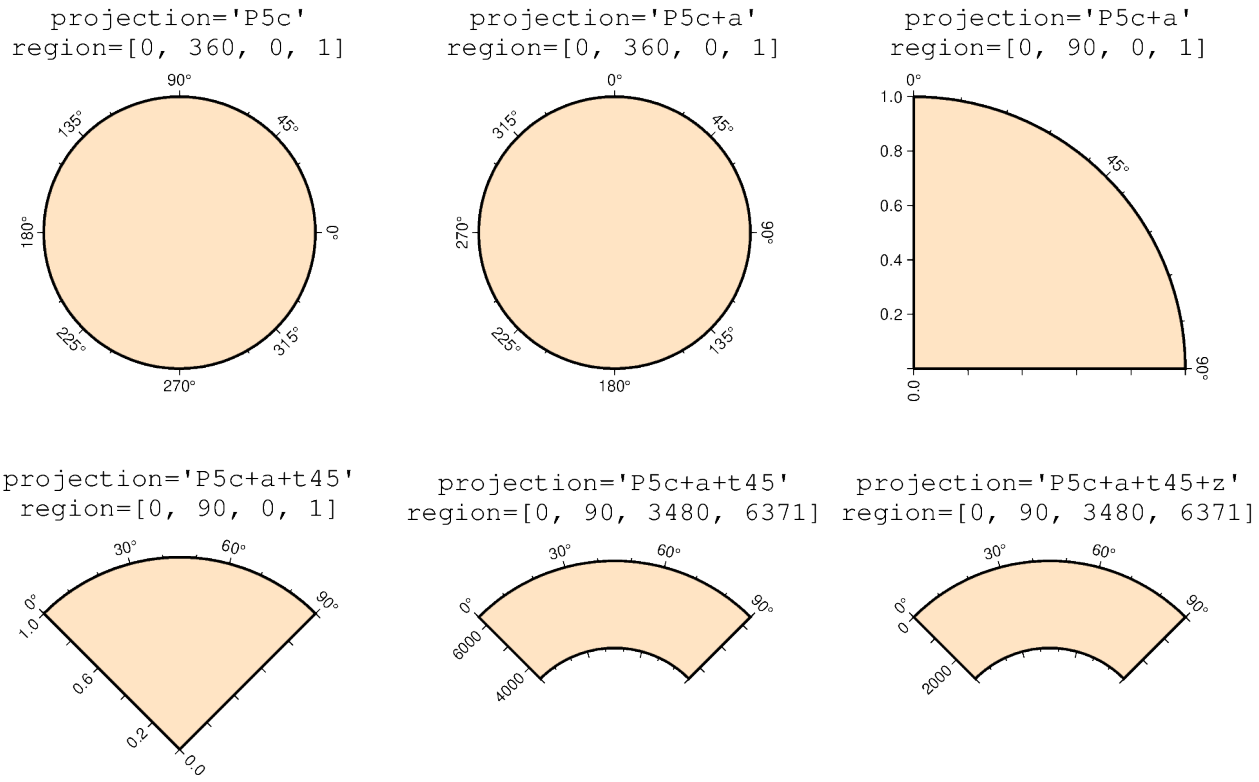
The full syntax for polar projections is:

P*width*[**+a**][**+f**[**e**|**p**|*radius*]][**+r***offset*][**+t***origin*][**+z**[**p**|*radius*]]

Limits are set via the `region` parameter ([*theta_min*, *theta_max*, *radius_min*, *radius_max*]). When using **P***width* you have to give the *width* of the plot. The lowercase version **p** is similar to **P** but expects a *scale* instead of a width (**p***scale*).

The following customizing modifiers are available:

- **+a**: by default, θ refers to the angle that is equivalent to a counterclockwise rotation with respect to the east direction (standard definition); **+a** indicates that the input data are rotated clockwise relative to the north direction (geographical azimuth angle) [Optional].
- **+r***offset*: represents the offset of the r-axis. This modifier allows you to offset the center of the circle from r=0 [Optional].
- **+t***origin*: sets the angle corresponding to the east direction which is equivalent to rotating the entire coordinate axis clockwise; if the **+a** modifier is used, setting the angle corresponding to the north direction is equivalent to rotating the entire coordinate axis counterclockwise [Optional].
- **+f**: reverses the radial direction [Optional].
  - Append **e** to indicate that the r-axis is an elevation angle, and the range of the r-axis should be between 0° and 90°.
  - Appending **p** sets the current Earth's radius (determined by PROJ_ELLIPSOID) to the maximum value of the r-axis when the r-axis is reversed.
  - Append *radius* to set the maximum value of the r-axis.
- **+z**: indicates that the r-axis is marked as depth instead of radius (e.g., r = *radius* - z) [Optional].
  - Append **p** to set radius to the current Earth's radius.
  - Append *radius* to set the value of the radius.




```
import pygmt
from pygmt.params import Axis, Frame

fig = pygmt.Figure()

pygmt.config(FONT_TITLE="14p,Courier,black", FORMAT_GEO_MAP="+D")

# ============
# Top left
fig.basemap(
    # Set plot limits to theta_min = 0, theta_max = 360, radius_min = 0, radius_max =␣
↪1
    region=[0, 360, 0, 1],
    # Set plot width to 5 cm
    projection="P5c",
    # Set the frame and title; @^ allows for a line break within the title
    frame=Frame(
        fill="bisque",
        title="projection='P5c' @^ region=[0, 360, 0, 1]",
        xaxis=Axis(annot=45, tick=True),
    ),
)

fig.shift_origin(xshift="w+3c")

# ============
# Top middle
fig.basemap(
```

(continues on next page)

(continued from previous page)

```
    # Set plot limits to theta_min = 0, theta_max = 360, radius_min = 0, radius_max =␣
↪1
    region=[0, 360, 0, 1],
    # Set plot width to 5 cm and interpret input data as geographic azimuth instead of
    # standard angle
    projection="P5c+a",
    # Set the frame and title; @^ allows for a line break within the title
    frame=Frame(
        fill="bisque",
        title="projection='P5c+a' @^ region=[0, 360, 0, 1]",
        xaxis=Axis(annot=45, tick=True),
    ),
)

fig.shift_origin(xshift="w+3c")

# ============
# Top right
fig.basemap(
    # Set plot limits to theta_min = 0, theta_max = 90, radius_min = 0, radius_max = 1
    region=[0, 90, 0, 1],
    # Set plot width to 5 cm and interpret input data as geographic azimuth instead of
    # standard angle
    projection="P5c+a",
    # Set the frame and title; @^ allows for a line break within the title
    frame=Frame(
        axes="WNe",
        fill="bisque",
        title="projection='P5c+a' @^ region=[0, 90, 0, 1]",
        xaxis=Axis(annot=45, tick=True),
        yaxis=Axis(annot=0.2),
    ),
)

fig.shift_origin(xshift="-2w-6c", yshift="-h-2c")

# ============
# Bottom left
fig.basemap(
    # Set plot limits to theta_min = 0, theta_max = 90, radius_min = 0, radius_max = 1
    region=[0, 90, 0, 1],
    # Set plot width to 5 cm and interpret input data as geographic azimuth instead of
    # standard angle, rotate coordinate system counterclockwise by 45 degrees
    projection="P5c+a+t45",
    # Set the frame and title; @^ allows for a line break within the title
    frame=Frame(
        axes="WNe",
        fill="bisque",
        title="projection='P5c+a+t45' @^ region=[0, 90, 0, 1]",
        xaxis=Axis(annot=30, tick=True),
        yaxis=Axis(annot=0.2),
    ),
```

(continues on next page)

(continued from previous page)

```
)

fig.shift_origin(xshift="w+3c", yshift="1.3c")

# ============
# Bottom middle
fig.basemap(
    # Set plot limits to theta_min = 0, theta_max = 90, radius_min = 3480,
    # radius_max = 6371 (Earth's radius)
    region=[0, 90, 3480, 6371],
    # Set plot width to 5 cm and interpret input data as geographic azimuth instead of
    # standard angle, rotate coordinate system counterclockwise by 45 degrees
    projection="P5c+a+t45",
    # Set the frame, and title; @^ allows for a line break within the title
    frame=Frame(
        axes="WNse",
        fill="bisque",
        title="projection='P5c+a+t45' @^ region=[0, 90, 3480, 6371]",
        xaxis=Axis(annot=30, tick=True),
        yaxis=Axis(annot=True),
    ),
)

fig.shift_origin(xshift="w+3c")

# ============
# Bottom right
fig.basemap(
    # Set plot limits to theta_min = 0, theta_max = 90, radius_min = 3480,
    # radius_max = 6371 (Earth's radius)
    region=[0, 90, 3480, 6371],
    # Set plot width to 5 cm and interpret input data as geographic azimuth instead of
    # standard angle, rotate coordinate system counterclockwise by 45 degrees, r-axis
    # is marked as depth
    projection="P5c+a+t45+z",
    # Set the frame, and title; @^ allows for a line break within the title
    frame=Frame(
        axes="WNse",
        fill="bisque",
        title="projection='P5c+a+t45+\\z' @^ region=[0, 90, 3480, 6371]",
        xaxis=Axis(annot=30, tick=True),
        yaxis=Axis(annot=True),
    ),
)

fig.show()
```

Total running time of the script: (0 minutes 0.227 seconds)

CHAPTER

TEN

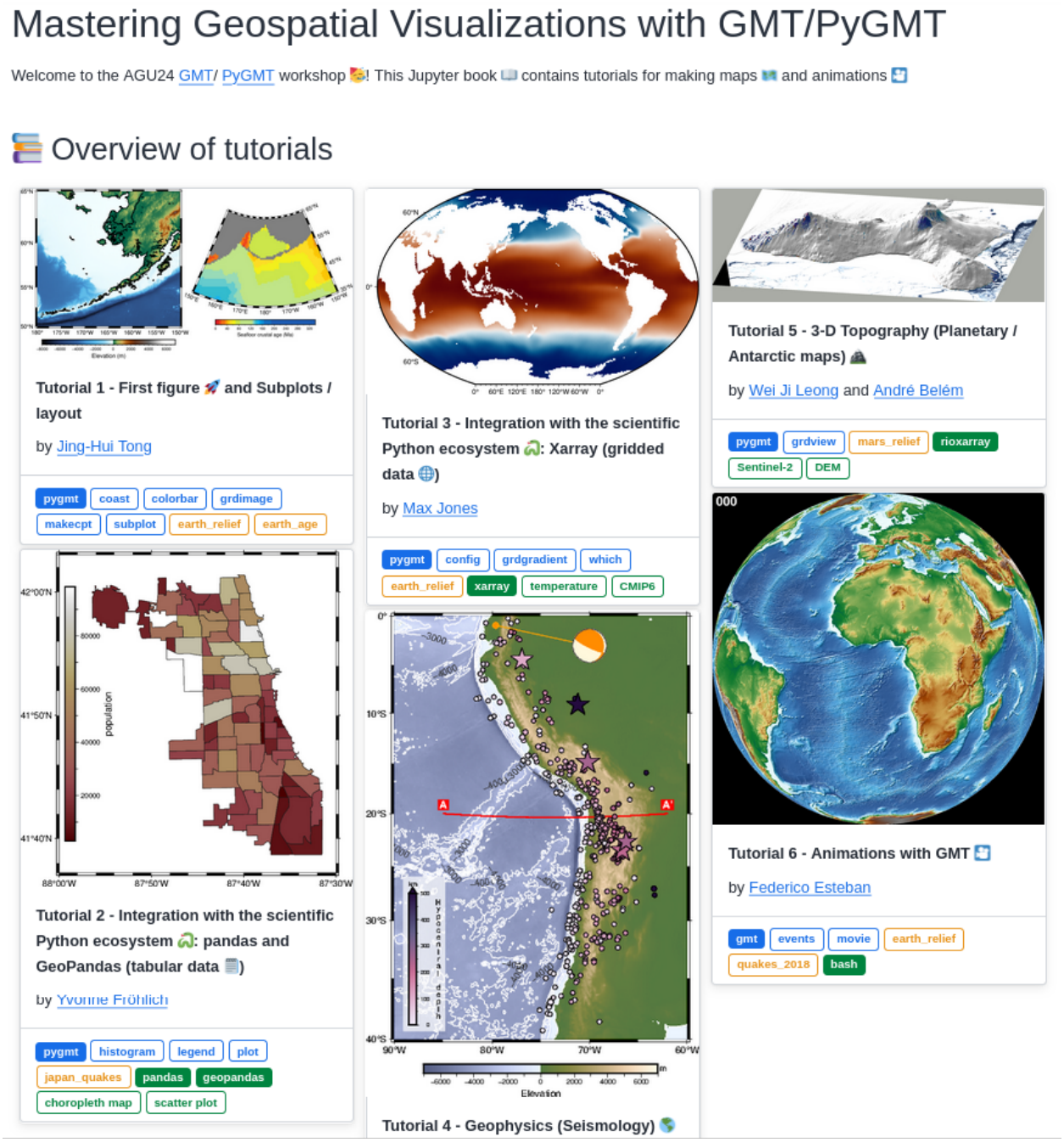
EXTERNAL RESOURCES

Below is a curated collection of external PyGMT resources.

To add your contribution to this collection, follow these instructions *to submit a pull request with your recommended addition to the External Resources file.*

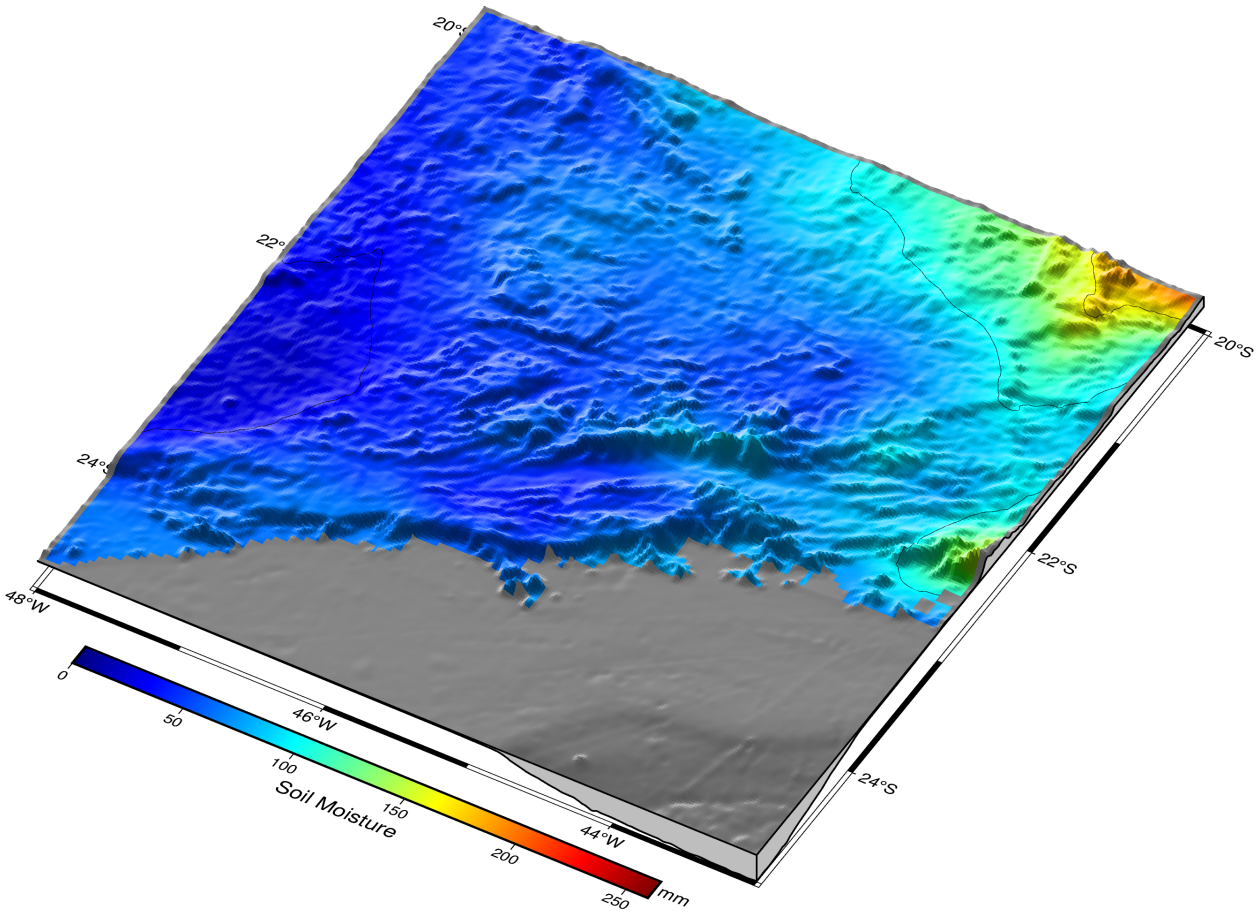
10.1 Tutorials

2024 AGU PREWS9: Mastering Geospatial Visualizations with GMT/PyGMT



Wei Ji Leong, Yvonne Fröhlich, Jing-Hui Tong, Federico Esteban, Max Jones, Andre Belem

https://www.generic-mapping-tools.org/agu24workshop/ **2024 PyGMT Webinar using Google Colab (in Portuguese)**



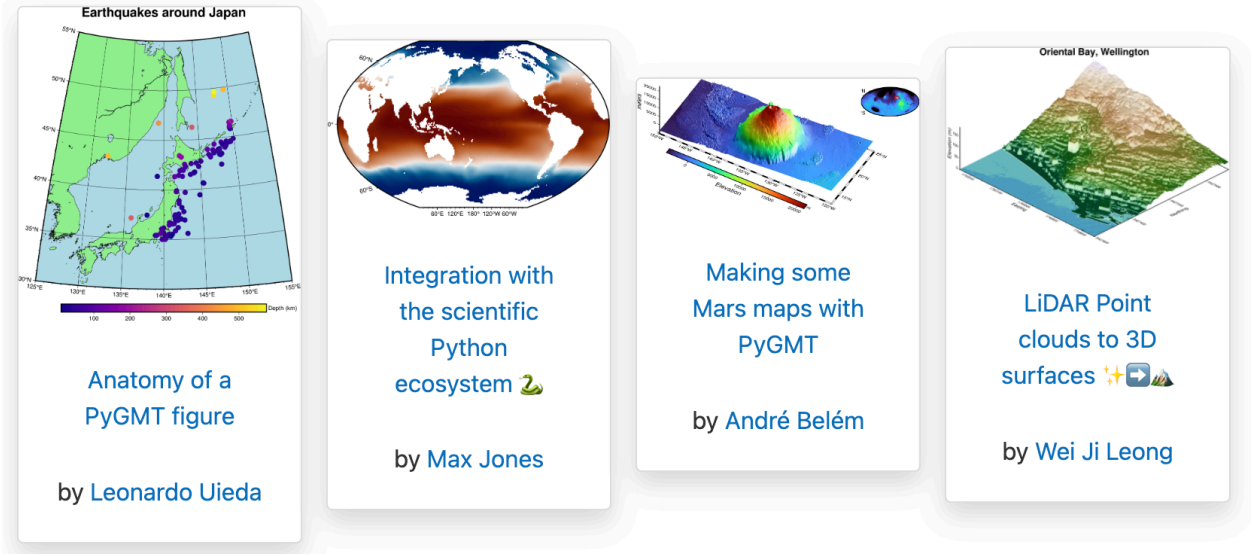
Andre Belem

https://github.com/andrebelem/Oficina_PyGMT **2022 EGU SC5.2: Crafting beautiful maps with PyGMT**

🌍 Crafting beautiful maps with PyGMT

Welcome to the EGU22 PyGMT short course 🥳

This Jupyter book 📖 contains PyGMT tutorials for producing maps 🗺️ and doing geospatial data processing 🌐



Wei Ji Leong, Leonardo Uieda, Max Jones, Andre Belem

https://www.generic-mapping-tools.org/egu22pygmt/ **2021 PyGMT Tutorial**

PyGMT Tutorial in 2021

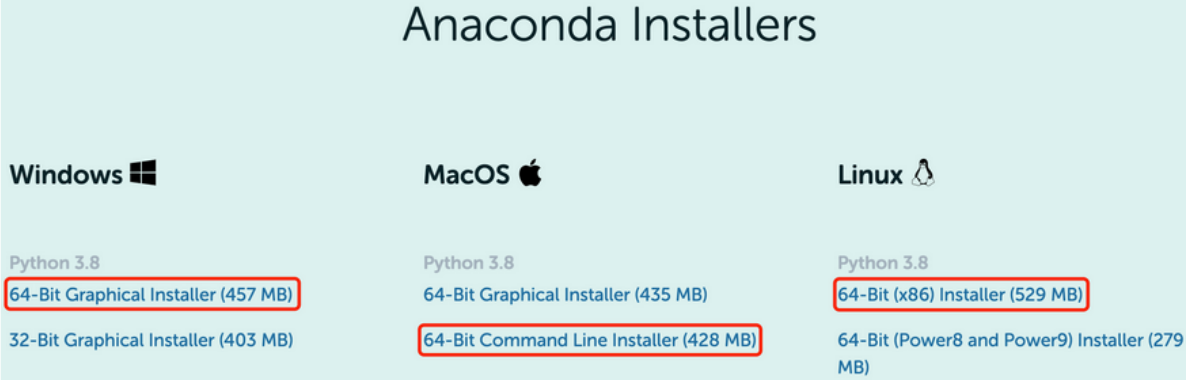
A preliminary introduction to PyGMT for MIG members.

Installation instructions

Anaconda is a Python distribution for scientific computing and is **strongly recommended**.

1. Download Anaconda

   Go to Anaconda download, you can see a downloading page similar to the below snapshot. Choose the corresponding package based on your operating system. Those indicated in red boxes are usually used.



MIGG-NTU

https://github.com/MIGG-NTU/PyGMT2021 **2021 PyGMT course at the UAF Geophysical Institute**

2021 GI PyGMT short course (lecture notebook)

Liam Toney

Geophysical Institute,
University of Alaska Fairbanks

1 March 2021

---

1. Basics

First, let's import the PyGMT Python package:

```
[1]: import pygmt
```

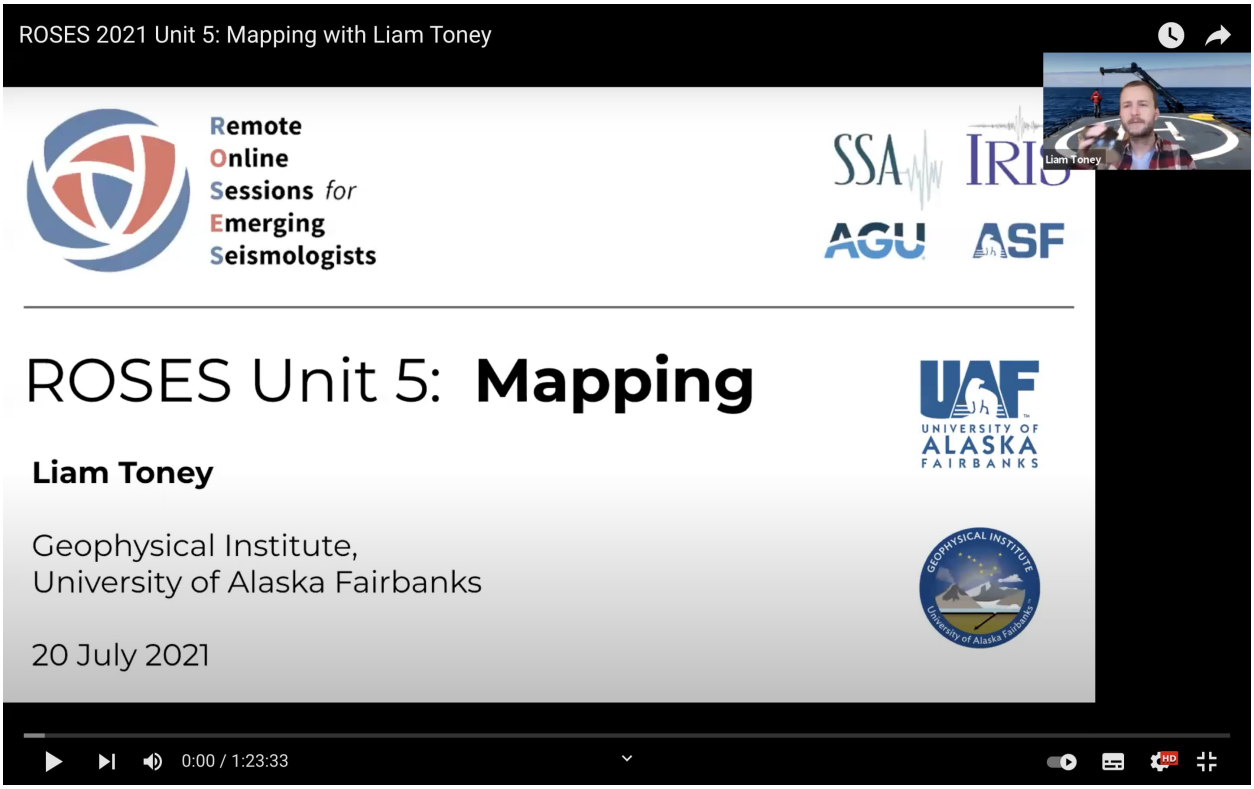
In PyGMT, every plot or map must start with the creation of a `pygmt.Figure` instance. This is similar to the Matplotlib command

```
fig = plt.figure()
```

which creates a `matplotlib.figure.Figure` instance. (Throughout this lecture, I'll try to connect PyGMT commands to Matplotlib commands.)

Liam Toney

https://github.com/liamtoney/gi-pygmt-2021 **2021 Remote Online Sessions for Emerging Seismologists (ROSES): Unit 5 - Mapping**



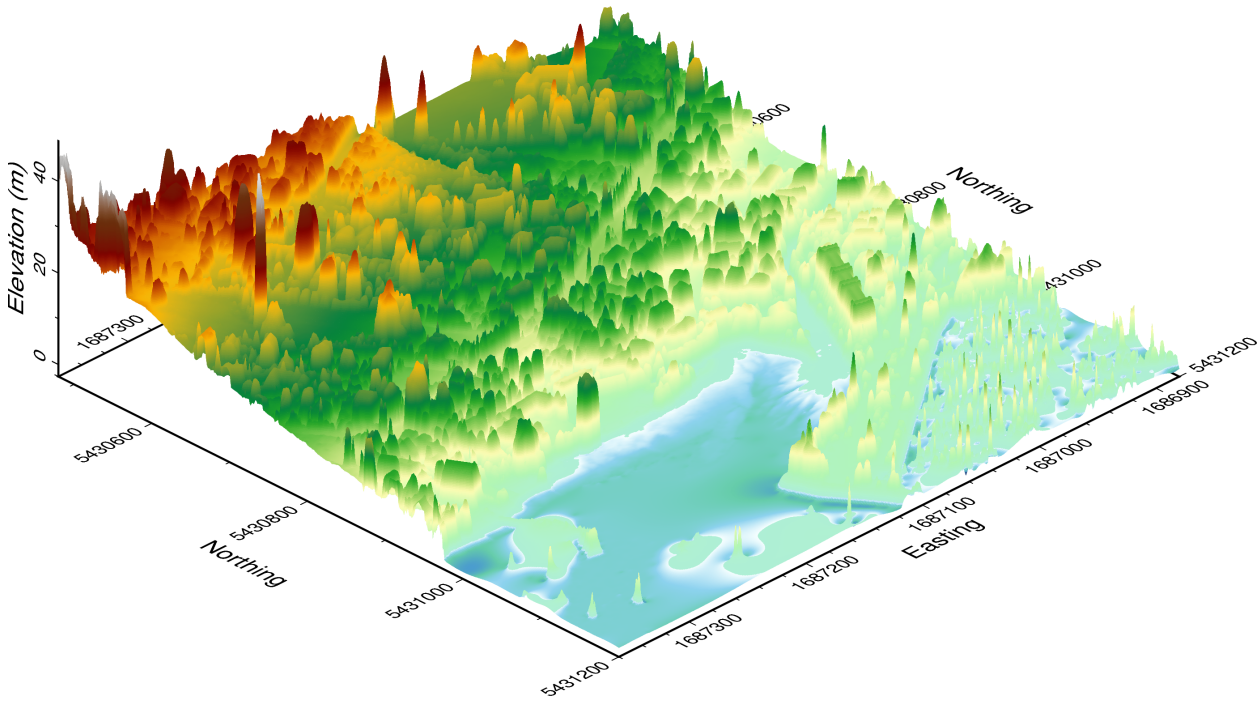
Liam Toney

https://www.youtube.com/watch?v=Zvcy7VDuhiw **2020 Remote Online Sessions for Emerging Seismologists (ROSES): Unit 8 - PyGMT**

Liam Toney

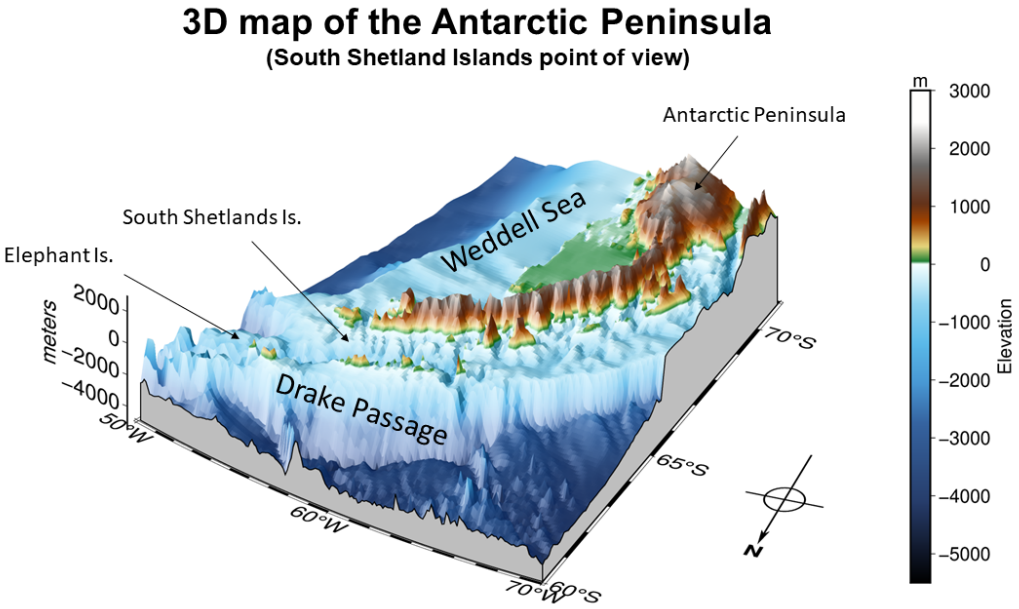
https://www.iris.edu/hq/inclass/lesson/728 **2019 PyGMT Workshop at FOSS4G Oceania**

Picton Harbour



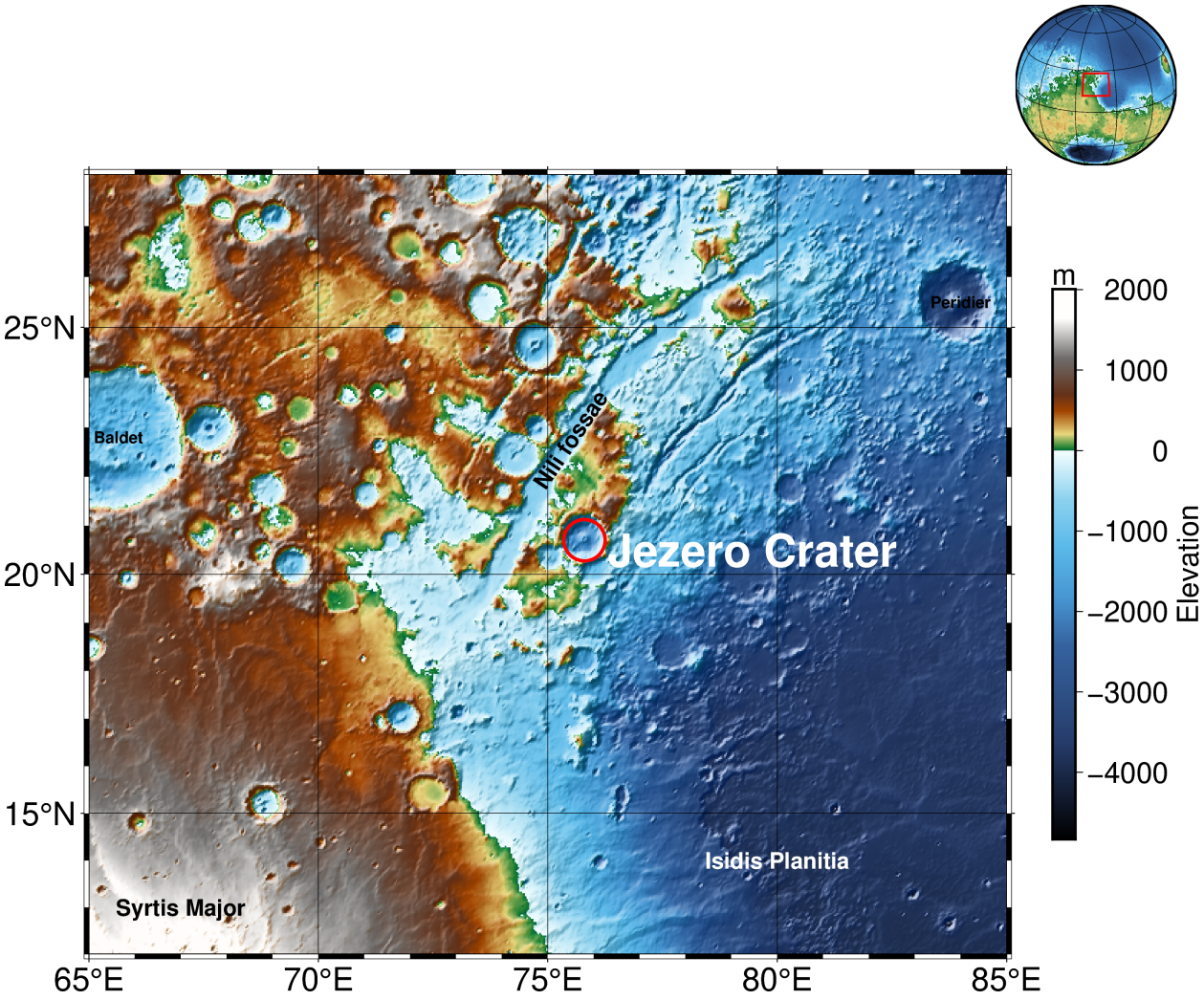
Wei Ji Leong

https://github.com/GenericMappingTools/foss4g2019oceania **Crafting 3D maps of Antarctica with PyGMT and the IBCSO V2**



Andre Belem

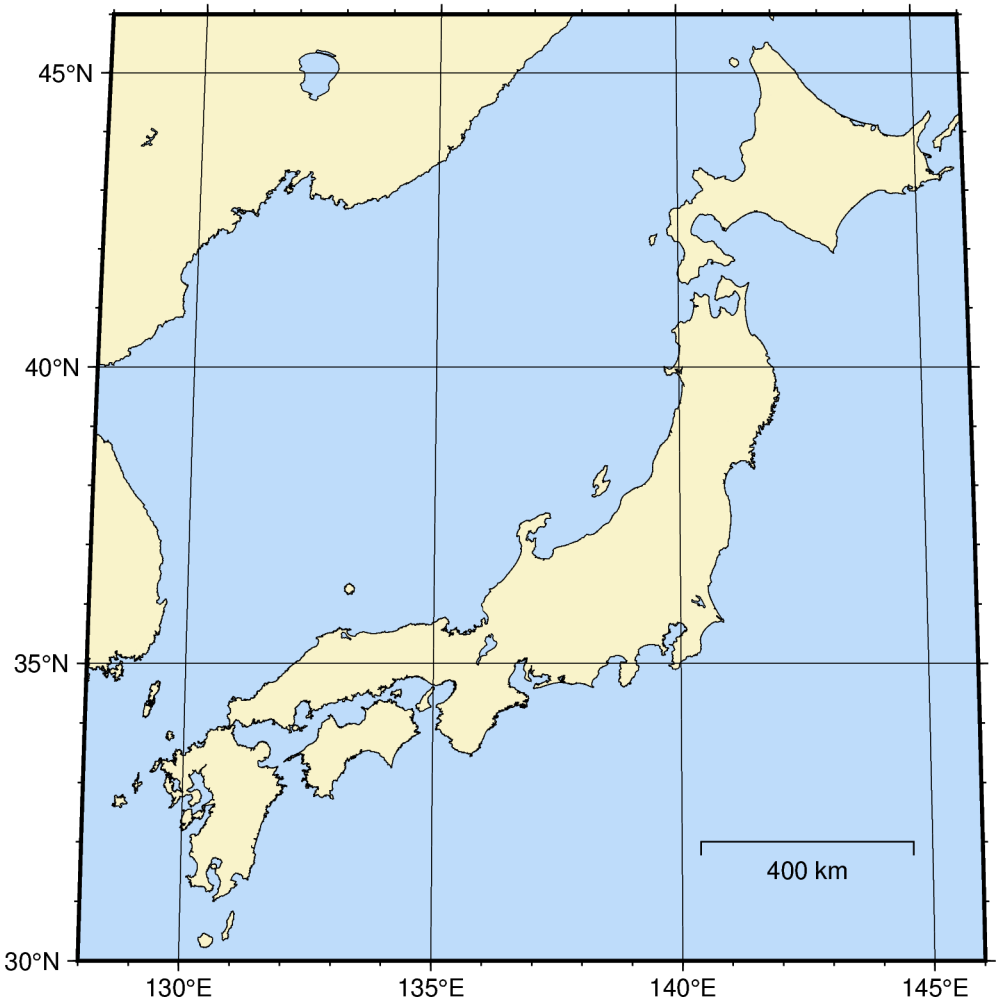
https://github.com/andrebelem/3D-Antarctic-maps **Planetary Maps (in PyGMT)**



Andre Belem

https://github.com/andrebelem/PlanetaryMaps **PyGMT-HOWTO**

Figure

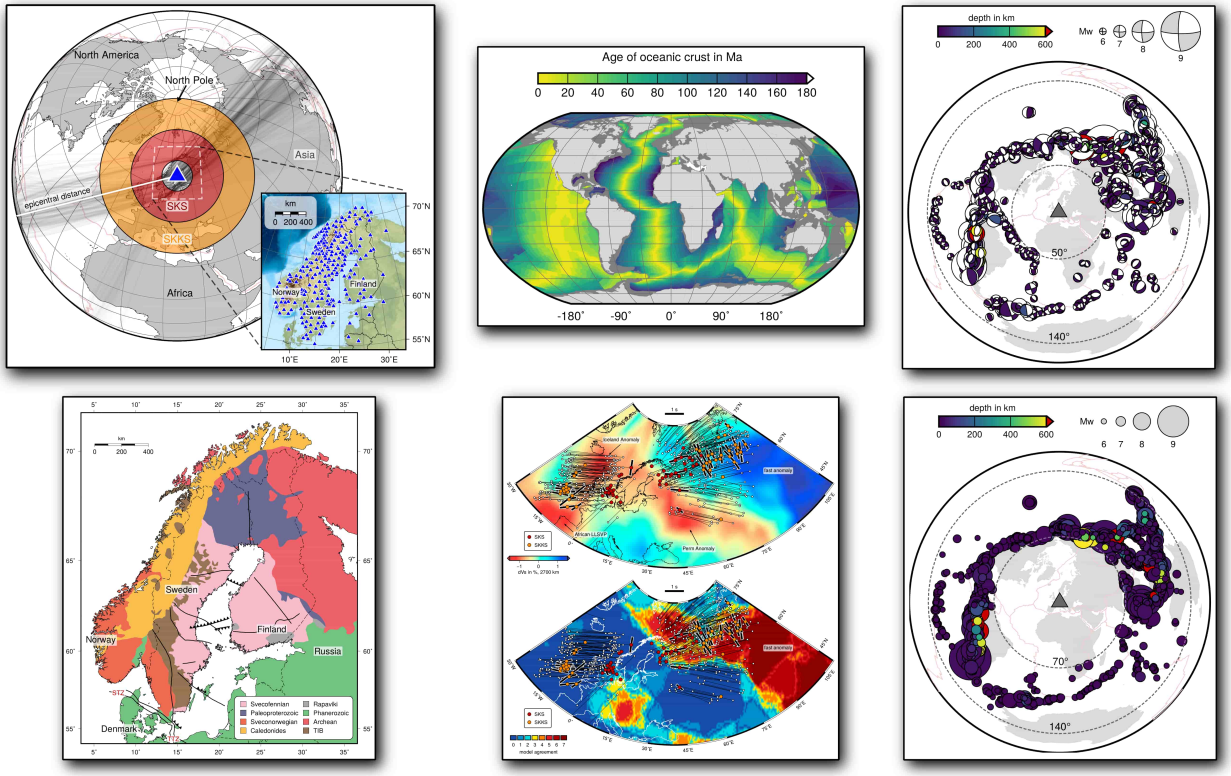


Takuto Maeda

https://tktmyd.github.io/pygmt-howto-jp/

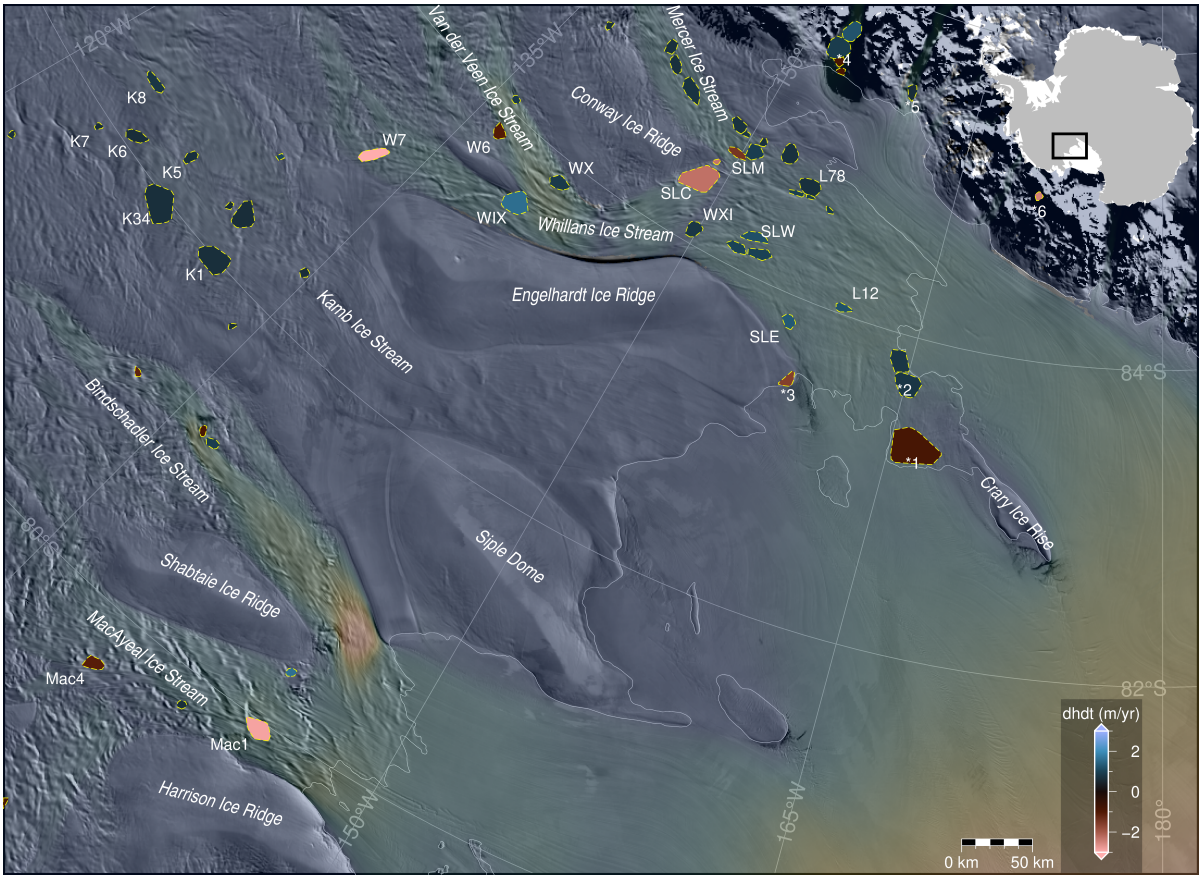
10.2 Examples from Publications and Posters

GMT and PyGMT plotting examples



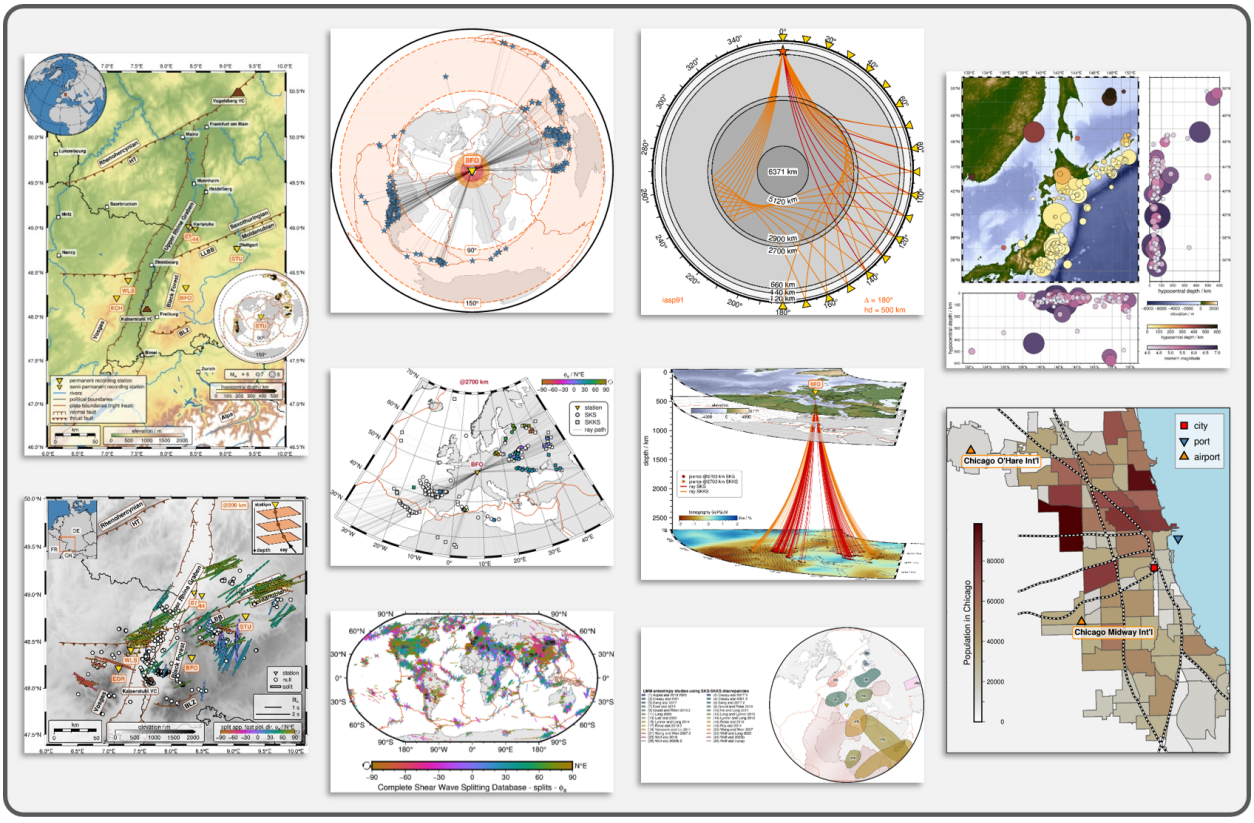
Michael Grund

https://github.com/michaelgrund/GMT-plotting **NZ Antarctic Science Conference 2021 poster**



Wei Ji Leong

https://github.com/weiji14/nzasc2021 **PyGMT plotting examples**



Yvonne Fröhlich

https://github.com/yvonnefroehlich/gmt-pygmt-plotting

CHAPTER

ELEVEN

API REFERENCE

This page gives an overview of all public PyGMT objects, functions and methods.

11.1 Plotting

11.1.1 Figure class overview

All plotting is handled through the *pygmt.Figure* class and its methods.

Figure()	A GMT figure to handle all plotting.

pygmt.Figure

class pygmt.**Figure**

A GMT figure to handle all plotting.

Use the plotting methods of this class to add elements to the figure. You can preview the figure using *pygmt.Figure.show* and save the figure to a file using *pygmt.Figure.savefig*.

Examples

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 360, -90, 90], projection="W15c", frame=True)
>>> fig.savefig("my-figure.png")
>>> # Make sure the figure file is generated and clean it up
>>> from pathlib import Path
>>> assert Path("my-figure.png").exists()
>>> Path("my-figure.png").unlink()
```

The plot region can be specified through ISO country codes (for example, `"JP"` for Japan):

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region="JP", projection="M7c", frame=True)
>>> # The fig.region attribute shows the WESN bounding box for the figure
>>> print(", ".join(f"{i:.2f}" for i in fig.region))
122.94, 145.82, 20.53, 45.52
```

Attributes

`Figure.`**`region`**

The geographic WESN bounding box for the current figure.

Methods Summary

Figure.basemap([projection, zscale, zsize, ...])	Plot base maps and frames.
Figure.choropleth(data, column[, cmap, ...])	Plot a choropleth map.
Figure.coast([resolution, land, water, ...])	Plot continents, countries, shorelines, rivers, and borders.
Figure.colorbar([cmap, position, length, ...])	Plot gray scale or color scale bar.
Figure.contour([data, x, y, z, no_clip, ...])	Contour table data by direct triangulation.
Figure.directional_rose([position, width, ...])	Add a directional rose on the map.
Figure.fill_between(x, y[, y2, x2, fill, ...])	Fill the area between two horizontal curves.
Figure.grdcontour(grid[, projection, ...])	Make contour map using a grid.
Figure.grdimage(grid[, cmap, monochrome, ...])	Project and plot grids or images.
Figure.grdview(grid[, cmap, drape_grid, ...])	Create 3-D perspective image or surface mesh from a grid.
Figure.histogram(data[, weights, histtype, ...])	Calculate and plot histograms.
Figure.hlines(y[, xmin, xmax, pen, label, ...])	Plot one or multiple horizontal line(s).
Figure.image(imagefile[, position, width, ...])	Plot raster or EPS images.
Figure.inset([position, width, height, box, ...])	Manage figure inset setup and completion.
Figure.legend([spec, position, width, ...])	Plot a legend.
Figure.logo([position, width, height, box, ...])	Plot the GMT logo.
Figure.magnetic_rose([position, width, ...])	Add a magnetic rose to the map.
Figure.meca(spec, scale[, convention, ...])	Plot focal mechanisms.
Figure.paragraph(x, y, text, parwidth, ...)	Typeset one or multiple paragraphs.
Figure.plot([data, x, y, size, symbol, ...])	Plot lines, polygons, and symbols in 2-D.
Figure.plot3d([data, x, y, z, size, symbol, ...])	Plot lines, polygons, and symbols in 3-D.
Figure.psconvert([prefix, dpi, gs_option, ...])	Convert [E]PS file(s) to other formats using Ghostscript.
Figure.pygmtlogo([shape, theme, wordmark, ...])	Plot the PyGMT logo.
Figure.rose([data, length, azimuth, cmap, ...])	Plot a polar histogram (rose, sector, windrose diagrams).
Figure.savefig(fname[, transparent, crop, ...])	Save the figure to an image file.
Figure.scalebar(length[, height, position, ...])	Add a scale bar on the plot.
Figure.set_panel([panel, tag, verbose])	Set the current subplot panel to plot on.
Figure.shift_origin([xshift, yshift])	Shift the plot origin in x- and/or y-directions.
Figure.show([method, dpi, width, waiting])	Display a preview of the figure.
Figure.solar([terminator, ...])	Plot day-night terminators and other sunlight parameters.
Figure.subplot([nrows, ncols, tag, ...])	Manage figure subplot configuration and selection.
Figure.ternary(data[, cmap, fill, width, ...])	Plot data on ternary diagrams.
Figure.text([textfiles, x, y, position, ...])	Plot or typeset text.
Figure.tilemap(region[, zoom, source, ...])	Plot an XYZ tile map.
Figure.timestamp([text, label, justify, ...])	Plot the GMT timestamp logo.
Figure.velo([data, cmap, no_clip, ...])	Plot velocity vectors, strain crosses, anisotropy bars, and wedges.
Figure.vlines(x[, ymin, ymax, pen, label, ...])	Plot one or multiple vertical line(s).
Figure.wiggle([data, x, y, z, position, ...])	Plot z=f(x,y) anomalies along tracks.

pygmt.Figure.basemap

`Figure.`**`basemap`**(*projection=None*, *zscale=None*, *zsize=None*, *region=None*, *frame=False*, *map_scale=None*, *compass=None*, *rose=None*, *box=False*, *verbose=False*, *panel=False*, *coltypes=None*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot base maps and frames.

Creates a basic or fancy basemap with axes, fill, and title. Several map projections are available, and separate tick-mark intervals for axis annotation, ticks, and gridlines can be specified.

If not in subplot mode (see *pygmt.Figure.subplot*), at least one of the parameters `frame`, `map_scale`, `rose`, or `compass` must be specified.

Note

Parameters `map_scale`, `rose`, and `compass` are deprecated since 0.19.0 in favor of the dedicated higher-level methods:

- *pygmt.Figure.scalebar*: Add a scale bar on the plot.
- *pygmt.Figure.directional_rose*: Add a directional rose on the plot.
- *pygmt.Figure.magnetic_rose*: Add a magnetic rose on the plot.

These methods provide more comprehensive and flexible APIs for their respective plot elements. The `box` parameter in *pygmt.Figure.basemap* is retained only as a compatibility parameter for these legacy parameters. For new code, prefer the `box` parameter on the dedicated methods instead.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/basemap.html.

Aliases

- B = frame
- F = box
- J = projection
- Jz = zscale
- JZ = zsize
- L = map_scale
- R = region
- Td = rose
- Tm = compass
- V = verbose
- c = panel
- f = coltypes
- p = perspective
- t = transparency

Parameters

- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest. *Required if this is the first plot command.*
- **frame** (*Frame* | *Axis* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **zscale** (`float` | `str` | `None`, default: `None`)
- **zsize** (`float` | `str` | `None`, default: `None`) – Set z-axis scaling or z-axis size.
- **map_scale** (`str` | `None`, default: `None`) – Draw a map scale bar on the plot.

  Deprecated since version 0.19.0: Use *pygmt.Figure.scalebar* instead. This parameter is maintained for backward compatibility and accepts raw GMT CLI strings for the `-L` option.

- **compass** (str | None, default: None) – Draw a map magnetic rose on the map.

  Deprecated since version 0.19.0: Use *pygmt.Figure.magnetic_rose* instead. This parameter is maintained for backward compatibility and accepts raw GMT CLI strings for the -Tm option.

- **rose** (str | None, default: None) – Draw a map directional rose on the map.

  Deprecated since version 0.19.0: Use *pygmt.Figure.directional_rose* instead. This parameter is maintained for backward compatibility and accepts raw GMT CLI strings for the -Td option.

- **box** (*Box* | str | bool, default: False) – Draw a background box behind the scalebar, directional rose, or magnetic rose.

  Deprecated since version 0.19.0: Use the box parameter in *pygmt.Figure.scalebar*, *pygmt.Figure.directional_rose*, or *pygmt.Figure.magnetic_rose* instead. This parameter is maintained for backward compatibility and accepts either a *pygmt.params.Box* object, a raw GMT CLI string, or True for the -F option. On *pygmt.Figure.basemap*, it only applies when used together with the legacy map_scale, rose, or compass parameters.

- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.

  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.

- **coltypes** (str | None, default: None) – Specify the types of the coordinates given via region. See https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full for details.

- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).

  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

See also

pygmt.Figure.directional_rose
Add a map directional rose.

pygmt.Figure.magnetic_rose
Add a map magnetic rose.

pygmt.Figure.scalebar
Add a scale bar.

Examples

Plot a basic map with a specified region, projection, and frame settings.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region="g", projection="H15c", frame=True)
>>> fig.show()
```

Examples using `pygmt.Figure.basemap`

pygmt.Figure.choropleth

`Figure.`**`choropleth`**(*data*, *column*, *cmap=True*, *intensity=None*, *pen=None*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Plot a choropleth map.

This method creates a choropleth map by filling each polygon according to an attribute value. It requires the input data to be a geo-like Python object that implements `__geo_interface__` and includes the values used to fill the polygons as an attribute field (e.g. a `geopandas.GeoDataFrame`), or an OGR_GMT file containing the geometry and data to plot.

Aliases

- B = frame
- C = cmap
- I = intensity
- J = projection
- N = no_clip
- R = region
- V = verbose
- W = pen
- a = column
- c = panel
- p = perspective
- t = transparency

Parameters

- **`data`** (`GeoDataFrame` | `str` | `PathLike`) – A geo-like Python object which implements `__geo_interface__` and includes the values used to fill the polygons as an attribute field (e.g. a `geopandas.GeoDataFrame`), or an OGR_GMT file containing the geometry and data to plot.
- **`column`** (`str`) – The name of the data column to use for the fill.
- **`cmap`** (`str` | `bool`, default: `True`) – The CPT to use for filling the polygons. If set to `True`, the current CPT will be used.
- **`intensity`** (`float` | `None`, default: `None`) – The intensity (nominally in the ±1 range) to modulate the fill color by simulating illumination [Default is no illumination].

- **no_clip** (bool, default: False) – Do **not** clip polygons that fall outside the frame boundaries.
- **pen** (str | None, default: None) – Set pen attributes for lines or the outline of symbols.
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot a choropleth map of the world based on population estimates.

```
>>> import geopandas
>>> import pygmt
>>> world = geopandas.read_file(
...
↪"https://naciscdn.org/naturalearth/110m/cultural/ne_110m_admin_0_countries.zip"
... )
>>> world["POP_EST"] *= 1e-6  # Population in millions
>>>
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-19.5, 53, -38, 37.5], projection="M15c", frame=True)
>>> pygmt.makecpt(cmap="bilbao", series=(0, 270, 10), reverse=True)
>>> fig.choropleth(world, column="POP_EST", pen="0.3p,gray10")
>>> fig.colorbar(frame=True)
>>> fig.show()
```

Examples using `pygmt.Figure.choropleth`

pygmt.Figure.coast

`Figure.`**`coast`**(*resolution=None*, *land=None*, *water=None*, *rivers=None*, *borders=None*, *shorelines=False*, *lakes=None*, *river_lakes=None*, *area_thresh=None*, *map_scale=None*, *box=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot continents, countries, shorelines, rivers, and borders.

Plots grayshaded, colored, or textured land masses [or water masses] on maps and [optionally] draws coastlines, rivers, and political boundaries. The data files come in 5 different resolutions: (**f**)ull, (**h**)igh, (**i**)ntermediate, (**l**)ow, and (**c**)rude. The full resolution files amount to more than 55 Mb of data and provide great detail; for maps of larger geographical extent it is more economical to use one of the other resolutions. If the user selects to paint the land areas and does not specify fill of water areas then the latter will be transparent (i.e., earlier graphics drawn in those areas will not be overwritten). Likewise, if the water areas are painted and no land fill is set then the land areas will be transparent.

A map projection must be supplied.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/coast.html.

Aliases

- E = dcw
- A = area_thresh
- B = frame
- C = lakes, river_lakes
- D = resolution
- F = box
- G = land
- I = rivers
- J = projection
- L = map_scale
- R = region
- S = water
- V = verbose
- c = panel
- p = perspective
- t = transparency

Parameters

- **`area_thresh`** (`float` | `str` | `None`, default: `None`) – *min_area*[/*min_level*/*max_level*][**+a**[**g**|**i**][**s**|**S**]][**+l**|**r**][**+p***percent*]. Features with an area smaller than *min_area* in km^2 or of hierarchical level that is lower than *min_level* or higher than *max_level* will not be plotted [Default is `"0/0/4"` (all features)].

- **resolution** (Literal['auto', 'full', 'high', 'intermediate', 'low', 'crude', None], default: None) – Select the resolution of the coastline dataset to use. The available resolutions from highest to lowest are: "full", "high", "intermediate", "low", and "crude", which drops by 80% between levels. Default is "auto" to automatically select the most suitable resolution given the chosen map scale.
- **land** (str | None, default: None) – Select filling of "dry" areas.
- **water** (str | None, default: None) – Select filling of "wet" areas.
- **lakes** (str | None, default: None) – Select filling of lakes. If not specified, will use the fill for "wet" areas set by the water parameter.
- **river_lakes** (str | None, default: None) – Select filling of river-lakes. If not specified, will use the fill for "wet" areas set by the water parameter.
- **rivers** (int | str | Sequence[int | str] | None, default: None) – Draw rivers. Specify the type of rivers to draw, and optionally append a pen attribute, in the format *river*/*pen* [Default pen is "0.25p,black,solid"]. Pass a sequence of river types or *river*/*pen* strings to draw different river types with different pens.

  Choose from the following river types:

  - 0: Double-lined rivers (river-lakes)
  - 1: Permanent major rivers
  - 2: Additional major rivers
  - 3: Additional rivers
  - 4: Minor rivers
  - 5: Intermittent rivers - major
  - 6: Intermittent rivers - additional
  - 7: Intermittent rivers - minor
  - 8: Major canals
  - 9: Minor canals
  - 10: Irrigation canals

  Or choose from the following preconfigured river groups:

  - "a": All rivers and canals (types 0 - 10)
  - "A": Rivers and canals except river-lakes (types 1 - 10)
  - "r": Permanent rivers (types 0 - 4)
  - "R": Permanent rivers except river-lakes (types 1 - 4)
  - "i": Intermittent rivers (types 5 - 7)
  - "c": Canals (types 8 - 10)

  Example usage:

  - rivers=1: Draw permanent major rivers with default pen.
  - rivers="1/0.5p,blue": Draw permanent major rivers with a 0.5-point blue pen.
  - rivers=["1/0.5p,blue", "5/0.3p,cyan,dashed"]: Draw permanent major rivers with a 0.5-point blue pen and intermittent major rivers with a 0.3-point dashed cyan pen.

- **map_scale** (str | None, default: None) – Draw a map scale bar on the plot.

  Deprecated since version 0.19.0: Use *pygmt.Figure.scalebar* instead. This parameter is maintained for backward compatibility and accepts raw GMT CLI strings for the -L option.

- **box** (*Box* | bool, default: False) – Draw a background box behind the map scale or rose. If set to True, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.

- **borders** (int | str | Sequence[int | str] | None, default: None) – Draw political boundaries. Specify the type of boundary to draw, and optionally append a pen attribute, in the format *border/pen* [Default pen is "0.25p,black,solid"]. Pass a sequence of border types or *border/pen* strings to draw different border types with different pens.

  Choose from the following border types:

  - 1: National boundaries
  - 2: State boundaries within the Americas
  - 3: Marine boundaries
  - "a": All boundaries (types 1 - 3)

  Example usage:

  - borders=1: Draw national boundaries with default pen.
  - borders="1/0.5p,red": Draw national boundaries with a 0.5-point red pen.
  - borders=["1/0.5p,red", "2/0.3p,blue,dashed"]: Draw national boundaries with a 0.5-point red pen and state boundaries with a 0.3-point dashed blue pen.

- **shorelines** (bool | str | Sequence[int | str], default: False) – Draw shorelines. Specify the pen attributes for shorelines [Default pen is "0.25p,black,solid"]. Shorelines have four levels; by default, the same pen is used for all levels. To specify the shoreline level, use the format *level/pen*. Pass a sequence of *level/pen* strings to draw different shoreline levels with different pens. When specific level pens are set, those not listed will not be drawn [Default draws all levels]. shorelines=True draws all levels with the default pen.

  Choose from the following shoreline levels:

  - 1: Coastline
  - 2: Lakeshore
  - 3: Island-in-lake shore
  - 4: Lake-in-island-in-lake shore

  Example usage:

  - shorelines=True: Draw all shoreline levels with default pen.
  - shorelines="0.5p,blue": Draw all shoreline levels with a 0.5-point blue pen.
  - shorelines="1/0.5p,black": Draw only coastlines with a 0.5-point black pen.
  - shorelines=["1/0.8p,black", "2/0.4p,blue"]: Draw coastlines with a 0.8-point black pen and lakeshores with a 0.4-point blue pen.

- **dcw** (*str or list*) – *code1,code2,...*[**+g***fill*][**+p***pen*][**+z**]. Select painting country polygons from the Digital Chart of the World. Append one or more comma-separated countries using the 2-letter ISO 3166-1 alpha-2 convention. To select a state of a country (if available), append *.state*, (e.g, "US.TX" for Texas). To specify a whole continent, prepend **=** to any of the

continent codes (e.g. `"=EU"` for Europe). Append **+p***pen* to draw polygon outlines [Default is no outline] and **+g***fill* to fill them [Default is no fill].

- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest. *Required if this is the first plot command.*
- **frame** (*Frame* | *Axis* | `Literal`['none'] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

See also

pygmt.Figure.directional_rose
Add a map directional rose.

pygmt.Figure.magnetic_rose
Add a map magnetic rose.

pygmt.Figure.scalebar
Add a scale bar.

Examples

Plot a map with coastlines, land, water, and other features.

```
>>> import pygmt
>>> from pygmt.params import Axis
>>> fig = pygmt.Figure()
>>> fig.coast()
...     projection="M10c",  # Set the Mercator projection and width to 10 cm
...     region=[-10, 30, 30, 60],  # Set the map region
...     frame=Axis(annot=True),
...     land="darkgreen",  # Set the color of the land area
...     water="lightblue",  # Set the color of the water area
...     borders="1/1p,black",  # Draw national borders with a 1-point black line
... )
>>> fig.show()
```

Examples using `pygmt.Figure.coast`

pygmt.Figure.colorbar

Figure.**colorbar**(*cmap=False*, *position=None*, *length=None*, *width=None*, *orientation=None*, *label=None*, *unit=None*, *annot=False*, *tick=False*, *grid=False*, *annot_angle=None*, *annot_prefix=None*, *annot_unit=None*, *reverse=False*, *nan=False*, *nan_position=None*, *bg_triangle=False*, *fg_triangle=False*, *triangle_height=None*, *move_text=None*, *label_as_column=False*, *box=False*, *truncate=None*, *shading=False*, *log=False*, *scale=None*, *monochrome=False*, *dpi=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot gray scale or color scale bar.

Both horizontal and vertical colorbars are supported. For CPTs with gradational colors (i.e., the lower and upper boundary of an interval have different colors) we will interpolate to give a continuous scale. Variations in intensity due to shading/illumination may be displayed by setting the `shading` parameter. Colors may be spaced according to a linear scale, all be equal size, or by providing a file with individual tile widths.

Note

For GMT >=6.5.0, the fontsizes of the colorbar x-label, x-annotations, and y-label are scaled based on the width of the colorbar following $\sqrt{colorbar_width/15}$. To set a desired fontsize via the GMT default parameters FONT_ANNOT_PRIMARY, FONT_ANNOT_SECONDARY, and FONT_LABEL (or jointly FONT) users have to divide the desired fontsize by the value calculated with the formula given above before passing it to the

default parameters. To only affect fontsizes related to the colorbar, the defaults can be changed locally only using `with pygmt.config(...):`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/colorbar.html.

Aliases

- L = equalsize
- Z = zfile
- C = cmap
- F = box
- G = truncate
- I = shading
- J = projection
- M = monochrome
- N = dpi
- Q = log
- R = region
- V = verbose
- W = scale
- c = panel
- p = perspective
- t = transparency
- B = label, unit, annot, tick, grid, annot_angle, annot_prefix, annot_unit
- D = position, **+w**: length/width, **+h/+v**: orientation, **+r**: reverse, **+n**: nan/nan_position, **+e**: fg_triangle/bg_triangle/triangle_height, **+m**: move_text/label_as_column

Parameters

- **cmap** (str | bool, default: False) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **position** (*Position* | Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None, default: None) – Position of the colorbar on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to bottom-center outside of the plot.
- **length** (float | str | None, default: None)
- **width** (float | str | None, default: None) – Length and width of the colorbar. If length is given with a unit `%` then it is in percentage of the corresponding plot side dimension (i.e., plot width for a horizontal colorbar, or plot height for a vertical colorbar). If width is given with unit `%` then it is in percentage of the bar length. [Length defaults to 80% of the corresponding plot side dimension, and width defaults to 4% of the bar length].
- **label** (str | None, default: None)
- **unit** (str | None, default: None) – Set the label and unit for the colorbar. The label is placed along the colorbar and the unit is placed at the end of the colorbar.
- **annot** (float | bool, default: False)
- **tick** (float | bool, default: False)
- **grid** (float | bool, default: False) – Intervals for annotations, ticks, and gridlines. Refer to *pygmt.params.Axis* for more details on how these parameters work.
- **annot_prefix** (str | None, default: None)

- **annot_unit** (str | None, default: None)
- **annot_angle** (float | None, default: None) – The prefix, unit and angle for the annotations. The prefix is placed before the annotation text; the unit is placed after the annotation text; and the angle is the angle of the annotation text.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set colorbar boundary frame, labels, and axes attributes. If frame=True, a default frame will be drawn. If frame="none", no frame will be drawn. Raw GMT strings or sequences of strings are also supported for backward compatibility. For more control over the frame attributes, use parameters such as annot, tick, grid, annot_angle, annot_prefix, annot_unit, label, and unit instead.
- **orientation** (Literal['horizontal', 'vertical'] | None, default: None) – Set the colorbar orientation to either "horizontal" or "vertical". [Default is vertical, unless position is set to bottom-center or top-center with cstype="outside" or cstype="inside", then horizontal is the default].
- **reverse** (bool, default: False) – Reverse the positive direction of the bar.
- **nan** (bool, default: False) – Draw a rectangle filled with the NaN color (via the **N** entry in the CPT or COLOR_NAN if no such entry) at the start or end of the colorbar (controlled via nan_position). If a string is given, use that string as the label for the NaN color.
- **nan_position** (Literal['start', 'end'] | None, default: None) – Set the position of the NaN rectangle. Choose either "start" or "end" to place the NaN color rectangle at the start or end of the colorbar [Default is "start"].
- **bg_triangle** (bool, default: False)
- **fg_triangle** (bool, default: False) – If True, draw triangles for the back- or foreground colors [Default is no triangles]. The back- and/or foreground colors are taken from the **B** and **F** entries in the CPT. If no such entries exist, then the system default colors for **B** and **F** are used instead (COLOR_BACKGROUND and COLOR_FOREGROUND).
- **triangle_height** (float | None, default: None) – Height of the triangles for back- and foreground colors [Default is half of the bar width].
- **move_text** (Literal['annotations', 'label', 'unit'] | Sequence[str] | None, default: None) – Move text (annotations, label, and unit) to opposite side. Accept a sequence of strings containing one or more of "annotations", "label", and "unit". The default placements of these texts depend on the colorbar orientation and position.
- **label_as_column** (bool, default: False) – Print a vertical label as a column of characters (does not work with special characters).
- **box** (*Box* | bool, default: False) – Draw a background box behind the colorbar. If set to True, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **truncate** (Sequence[float] | None, default: None) – (*zlow*, *zhigh*). Truncate the incoming CPT so that the lowest and highest z-levels are to *zlow* and *zhigh*. If one of these equal NaN then we leave that end of the CPT alone. The truncation takes place before the plotting.
- **scale** (float | None, default: None) – Multiply all z-values in the CPT by the provided scale. By default, the CPT is used as is.
- **shading** (float | Sequence[float] | bool, default: False) – Add illumination effects [Default is no illumination].

  – If True, a default intensity range of -1 to +1 is used.

  - Passing a single numerical value *max_intens* sets the range of intensities from *-max_intens* to *+max_intens*.
  - Passing a sequence of two numerical values (*low*, *high*) sets the intensity range from *low* to *high* to specify an asymmetric range.

- **equalsize** (*float or str*) – [**i**][*gap*]. Equal-sized color rectangles. By default, the rectangles are scaled according to the z-range in the CPT (see also `zfile`). If *gap* is appended and the CPT is discrete each annotation is centered on each rectangle, using the lower boundary z-value for the annotation. If **i** is prepended the interval range is annotated instead. If `shading` is used each rectangle will have its constant color modified by the specified intensity.
- **log** (`bool`, default: `False`) – Select logarithmic scale and power of ten annotations. All z-values in the CPT will be converted to $p = \log_{10}(z)$ and only integer p values will be annotated using the 10^p format [Default is linear scale].
- **zfile** (*str*) – File with colorbar-width per color entry. By default, the width of the entry is scaled to the color range, i.e., z = 0-100 gives twice the width as z = 100-150 (see also `equalsize`). **Note**: The widths may be in plot distance units or given as relative fractions and will be automatically scaled so that the sum of the widths equals the requested colorbar length.
- **monochrome** (`bool`, default: `False`) – Force a monochrome graybar using the (television) YIQ transformation.
- **dpi** (`int` | `None`, default: `None`) – Control how the color scale should be encoded graphically.
  - Use a positive integer to draw the color scale as image and set the effective dots-per-inch for rasterization of color scales, which is useful for continuous colormaps.
  - Use `dpi=0` to draw color rectangles, which is useful for discrete colormaps.

  If not specified, GMT uses its default encoding behavior, and the default dpi is 600 if the colorbar is drawn as image.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence[int]` | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].

  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **`transparency`** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Add a colorbar with a specified colormap, label, and unit on a plot.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 3], projection="X10c/3c", frame=True)
>>> fig.colorbar(cmap="SCM/roma", label="Velocity", unit="m/s")
>>> fig.show()
```

Examples using `pygmt.Figure.colorbar`

pygmt.Figure.contour

`Figure.`**`contour`**(*data=None*, *x=None*, *y=None*, *z=None*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Contour table data by direct triangulation.

Takes a matrix, (x, y, z) triplets, or a file name as input and plots, lines, polygons, or symbols at those locations on a plot.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/contour.html.

Aliases

- A = annotation
- C = levels
- G = label_placement
- L = triangular_mesh_pen
- S = skip
- W = pen
- b = binary
- d = nodata
- e = find
- h = header
- l = label
- B = frame
- J = projection
- N = no_clip
- R = region
- V = verbose
- c = panel
- f = coltypes
- i = incols
- p = perspective
- t = transparency

Parameters

- **`data`** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.

- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **annotation** (*float, list, or str*) – Specify or disable annotated contour levels, modifies annotated contours specified in `levels`.
  - Specify a fixed annotation interval.
  - Specify a list of annotation levels.
  - Disable all annotations by setting `annotation="n"`.
  - Adjust the appearance by appending different modifiers, e.g., `"annot_int+f10p+gred"` gives annotations with a font size of 10 points and a red filled box. For all available modifiers see https://docs.generic-mapping-tools.org/6.7/contour.html#a.
- **levels** (*float, list, or str*) – Specify the contour lines to generate.
  - The file name of a CPT file where the color boundaries will be used as contour levels.
  - The file name of a 2 (or 3) column file containing the contour levels (col 0), (**C**)ontour or (**A**)nnotate (col 1), and optional angle (col 2).
  - A fixed contour interval.
  - A list of contour levels.
- **D** (*str*) – Dump contour coordinates.
- **E** (*str*) – Network information.
- **label_placement** (*str*) – [**d**|**f**|**n**|**l**|**L**|**x**|**X**]*args*. Control the placement of labels along the quoted lines. It supports five controlling algorithms. See https://docs.generic-mapping-tools.org/6.7/contour.html#g for details.
- **I** (*bool*) – Color the triangles using the CPT if given via `levels`.
- **triangular_mesh_pen** (*str*) – Pen to draw the underlying triangulation [Default is `None`].
- **no_clip** (`bool`, default: `False`) – Do **not** clip contours or colored triangles at the frame boundaries [Default is `False` to fit inside `region`].
- **Q** (*float or str*) – [*cut*][**+z**]. Do not draw contours with less than *cut* number of points.
- **skip** (*bool or str*) – [**p**|**t**]. Skip input points outside region.
- **pen** (*str or list*) – [*type*]*pen*[**+c**[**l**|**f**]]. *type*, if present, can be **a** for annotated contours or **c** for regular contours [Default]. The pen sets the attributes for the particular line. Default pen for annotated contours is `"0.75p,black"` and for regular contours `"0.25p,black"`. Normally, all contours are drawn with a fixed color determined by the pen setting. If **+cl** is appended the colors of the contour lines are taken from the CPT (see `levels`). If **+cf** is appended the colors from the CPT file are applied to the contour annotations. Select **+c** for both effects.
- **label** (*str*) – Add a legend entry for the contour being plotted. Normally, the annotated contour is selected for the legend. You can select the regular contour instead, or both of them, by considering the label to be of the format [*annotcontlabel*][/*contlabel*]. If either label contains a slash (/) character then use `|` as the separator for the two labels instead.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i|o**[*ncols*][*type*][**w**][**+l|b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l|b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **nodata** (*str*) – **i|o***nodata*. Substitute specific values with NaN (for tabular data). For example, nodata="-9999" will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.

- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence[int | str]` | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].

  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot contours every 20 units and annotate every other contour (i.e., every 40 units) from data points given as three 1-D arrays:

```
>>> import numpy as np
>>> import pygmt
>>> x, y = np.meshgrid(np.linspace(-10, 10, 41), np.linspace(-10, 10, 41))
>>> z = x**2 + y**2
>>> fig = pygmt.Figure()
>>> fig.contour(
...     x=x.ravel(),
...     y=y.ravel(),
...     z=z.ravel(),
...     region=[-10, 10, -10, 10],
...     projection="X10c",
...     frame=True,
...     levels=20,
...     annotation=40,
...     pen="0.5p",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.contour`

pygmt.Figure.directional_rose

Figure.**directional_rose**(*position=None*, *width=None*, *fancy=False*, *labels=False*, *upright_labels=False*, *box=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Add a directional rose on the map.

Parameters

- **position** (*Position* | Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None, default: None) – Position of the directional rose on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.

- A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

If not specified, defaults to the Top Right corner of the plot.

- **width** (`float` | `str` | `None`, default: `None`) – Width of the rose in plot coordinates, or append unit `%` for a size in percentage of plot width [Default is 10%].
- **fancy** (`Literal[1, 2, 3]` | `bool`, default: `False`) – Get a fancy rose. The fanciness level can be set to 1, 2, or 3:
  - Level 1 draws the two principal E-W, N-S orientations
  - Level 2 adds the two intermediate NW-SE and NE-SW orientations
  - Level 3 adds the four minor orientations WNW-ESE, NNW-SSE, NNE-SSW, and ENE-WSW

  If set to `True`, defaults to level 1.
- **labels** (`Sequence[str]` | `bool`, default: `False`) – A sequence of four strings to label the cardinal points W, E, S, N. Use an empty string to skip a specific label. If set to `True`, default labels are used (`["W", "E", "S", "N"]` for a fancy rose and `["", "", "", "N"]` for a simple rose).
- **upright_labels** (`bool`, default: `False`) – Force the labels to be upright (more readable from the south) if the projection has extensive rotation of the directions. Requires `fancy` and `labels` to be set.
- **box** (*Box* | `bool`, default: `False`) – Draw a background box behind the directional rose. If set to `True`, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence[int]` | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Add a directional rose to a map with default settings on a map.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 80, 0, 30], projection="M10c", frame=True)
>>> fig.directional_rose()
>>> fig.show()
```

Examples using `pygmt.Figure.directional_rose`

pygmt.Figure.fill_between

`Figure.`**`fill_between`**(*x*, *y*, *y2=0*, *x2=None*, *fill=None*, *fill2=None*, *pen=None*, *pen2=None*, *label=None*, *label2=None*, *legend_pen=False*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Fill the area between two horizontal curves.

This method is a high-level wrapper around *`pygmt.Figure.plot`* to fill the area between a primary curve `y(x)` and a secondary curve `y2(x)`. The `y2` parameter can be either a single value [Default is 0] or a sequence. It can share the same `x` coordinates as `y` or use a separate `x2` coordinate sequence.

Parameters

- **x** (`Sequence`[`float`]) – X-coordinates of the primary curve.
- **y** (`Sequence`[`float`]) – Y-coordinates of the primary curve.
- **y2** (`float` | `Sequence`[`float`], default: `0`) – Y-coordinates of the secondary curve. It can be a scalar value for a horizontal reference line, or a sequence with the same length as `x` and `y` when `x2` is not used. Default is 0.
- **x2** (`Sequence`[`float`] | `None`, default: `None`) – X-coordinates of the secondary curve. Use this parameter only when `y2` is a sequence sampled at different x-coordinates from `y`. In that case, `y2` must have the same length as `x2`.
- **fill** (`str` | *`Pattern`* | `None`, default: `None`) – Fill for areas where the primary curve is greater than the secondary curve.
- **fill2** (`str` | *`Pattern`* | `None`, default: `None`) – Fill for areas where the secondary curve is greater than the primary curve.
- **pen** (`str` | `None`, default: `None`) – Pen attributes for the primary curve.
- **pen2** (`str` | `None`, default: `None`) – Pen attributes for the secondary curve.
- **label** (`str` | `None`, default: `None`) – Label for the primary curve, to be displayed in the legend.

- **label2** (str | None, default: None) – Label for the secondary curve, to be displayed in the legend.
- **legend_pen** (bool | str, default: False) – Draw legend entries as colored lines instead of filled boxes. By default, legend entries use fill and fill2 for the box fills and pen and pen2 for the box outlines. Set legend_pen=True to switch to colored lines that use the fill colors. Pass a pen thickness to control the legend line thickness.

  Note

  Due to an upstream GMT bug (https://github.com/GenericMappingTools/gmt/pull/9070), GMT <= 6.6.0 always uses a 2.5p pen thickness for these legend lines, regardless of the value passed to legend_pen.

- **no_clip** (bool, default: False) – Do **not** clip curves that fall outside the frame boundaries.
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Fill between two curves with specified colors and draw the curves with custom pen styles.

```
>>> import numpy as np
>>> import pygmt
>>> x = np.linspace(0, 2 * np.pi, 200)
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 2 * np.pi, -1.2, 1.2], projection="X10c/4c",␣
↪frame=True)
>>> fig.fill_between(
...     x=x,
...     y=np.sin(2 * x),
...     y2=np.sin(3 * x),
...     fill="lightblue",
...     pen="1p,blue",
...     fill2="lightred",
...     pen2="1p,red",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.fill_between`

pygmt.Figure.grdcontour

Figure.**grdcontour**(*grid*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Make contour map using a grid.

Takes a grid file name or an `xarray.DataArray` object as input.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdcontour.html.

Aliases

- A = annotation
- C = levels
- G = label_placement
- L = limit
- Q = cut
- S = resample
- W = pen
- l = label
- B = frame
- J = projection
- R = region
- V = verbose
- c = panel
- f = coltypes
- p = perspective
- t = transparency

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **levels** (*float, list, or str*) – Specify the contour lines to generate.
  - The file name of a CPT file where the color boundaries will be used as contour levels.
  - The file name of a 2 (or 3) column file containing the contour levels (col 0), (**C**)ontour or (**A**)nnotate (col 1), and optional angle (col 2).
  - A fixed contour interval.
  - A list of contour levels.
- **annotation** (*float, list, or str*) – Specify or disable annotated contour levels, modifies annotated contours specified in `levels`.
  - Specify a fixed annotation interval.
  - Specify a list of annotation levels.
  - Disable all annotations by setting `annotation="n"`.
  - Adjust the appearance by appending different modifiers, e.g., `"annot_int+f10p+gred"` gives annotations with a font size of 10 points and a red filled box. For all available modifiers see https://docs.generic-mapping-tools.org/6.7/grdcontour.html#a.
- **limit** (*str or list of 2 ints*) – *low*/*high*. Do no draw contours below *low* or above *high*, specify as string
- **cut** (*str or int*) – Do not draw contours with less than *cut* number of points.
- **resample** (*str or int*) – Resample smoothing factor.
- **label_placement** (*str*) – [**d**|**f**|**n**|**l**|**L**|**x**|**X**]*args*. Control the placement of labels along the quoted lines. It supports five controlling algorithms. See https://docs.generic-mapping-tools.org/6.7/grdcontour.html#g for details.
- **pen** (*str or list*) – [*type*]*pen*[**+c**[**l**|**f**]]. *type*, if present, can be **a** for annotated contours or **c** for regular contours [Default]. The pen sets the attributes for the particular line. Default pen for annotated contours is `"0.75p,black"` and for regular contours `"0.25p,black"`. Normally, all contours are drawn with a fixed color determined by the pen setting. If **+cl** is appended the colors of the contour lines are taken from the CPT (see `levels`). If **+cf** is appended the colors from the CPT file are applied to the contour annotations. Select **+c** for both effects.
- **label** (*str*) – Add a legend entry for the contour being plotted. Normally, the annotated contour is selected for the legend. You can select the regular contour instead, or both of them, by considering the label to be of the format [*annotcontlabel*][/*contlabel*]. If either label contains a slash (/) character then use `|` as the separator for the two labels instead.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame

and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int | Sequence[int] | bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **coltypes** (`str | None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **perspective** (`float | Sequence[float] | str | bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (`float | None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

```
>>> import pygmt
>>> from pygmt.params import Axis
>>> # Load the 15 arc-minutes grid with "gridline" registration in a given region
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="15m",
...     region=[-92.5, -82.5, -3, 7],
...     registration="gridline",
... )
>>> fig = pygmt.Figure()
>>> fig.grdcontour(
...     grid=grid,s
...     levels=250,  # Set the interval for contour lines
```

(continues on next page)

(continued from previous page)

```
...     annotation=1000,  # Set the interval for annotated contour lines
...     frame=Axis(annot=True),
...     projection="M10c",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.grdcontour`

pygmt.Figure.grdimage

Figure.**grdimage**(*grid*, *cmap=False*, *monochrome=False*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, *cores=False*, *coltypes=None*, ***kwargs*)

Project and plot grids or images.

Reads a 2-D grid file and produces a gray-shaded (or colored) map by building a rectangular image and assigning pixels a gray-shade (or color) based on the z-value and the CPT file. Optionally, illumination may be added by providing a file with intensities in the (-1,+1) range or instructions to derive intensities from the input data grid. Values outside this range will be clipped. Such intensity files can be created from the grid using *pygmt.grdgradient* and, optionally, modified by https://docs.generic-mapping-tools.org/6.7/grdmath.html or *pygmt.grdhisteq*. Alternatively, pass *image* which can be an image file (geo-referenced or not). In this case the image can optionally be illuminated with the file provided via the `shading` parameter. Here, if image has no coordinates then those of the intensity file will be used.

When using map projections, the grid is first resampled on a new rectangular grid with the same dimensions. Higher resolution images can be obtained by using the `dpi` parameter. To obtain the resampled value (and hence shade or color) of each map pixel, its location is inversely projected back onto the input grid after which a value is interpolated between the surrounding input grid values. By default bi-cubic interpolation is used. Aliasing is avoided by also forward projecting the input grid nodes. If two or more nodes are projected onto the same pixel, their average will dominate in the calculation of the pixel value. Interpolation and aliasing is controlled with the `interpolation` parameter.

The `region` parameter can be used to select a plot region larger or smaller than that implied by the extent of the grid.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdimage.html.

Aliases

- D = img_in
- E = dpi
- G = bitcolor
- I = shading
- Q = nan_transparent
- n = interpolation
- B = frame
- C = cmap
- J = projection
- M = monochrome
- N = no_clip
- R = region
- V = verbose
- c = panel
- f = coltypes
- p = perspective
- t = transparency
- x = cores

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **cmap** (str | bool, default: False) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **img_in** (*str*) – [**r**]. GMT will automatically detect standard image files (Geotiff, TIFF, JPG, PNG, GIF, etc.) and will read those via GDAL. For very obscure image formats you may need to explicitly set img_in, which specifies that the grid is in fact an image file to be read via GDAL. Append **r** to assign the region specified by region to the image. For example, if you have used region="d" then the image will be assigned a global domain. This mode allows you to project a raw image (an image without referencing coordinates).
- **dpi** (*int*) – [**i***dpi*]. Set the resolution of the projected grid that will be created if a map projection other than Linear or Mercator was selected [Default is 100 dpi]. By default, the projected grid will be of the same size (rows and columns) as the input file. Specify **i** to use the PostScript image operator to interpolate the image at the device resolution.
- **bitcolor** (*str*) – *color*[**+b**|**f**]. This parameter only applies when a resulting 1-bit image otherwise would consist of only two colors: black (0) and white (255). If so, this parameter will instead use the image as a transparent mask and paint the mask with the given color. Append **+b** to paint the background pixels (1) or **+f** for the foreground pixels [Default is **+f**].
- **shading** (str or xarray.DataArray) – [*intensfile*|*intensity*|*modifiers*]. Give the name of a grid file or a DataArray with intensities in the (-1,+1) range, or a constant intensity to apply everywhere (affects the ambient light). Alternatively, derive an intensity grid from the input data grid via a call to *pygmt.grdgradient*; append **+a***azimuth*, **+n***args*, and **+m***ambient* to specify azimuth, intensity, and ambient arguments for that function, or just give **+d** to select the default arguments (+a-45+nt1+m0). If you want a more specific intensity scenario then run *pygmt.grdgradient* separately first. If we should derive intensities from another file than grid, specify the file with suitable modifiers [Default is no illumination]. **Note**: If the input data represent an *image* then an *intensfile* or constant *intensity* must be provided.
- **monochrome** (bool, default: False) – Force conversion to monochrome image using the (television) YIQ transformation. Cannot be used with nan_transparent.
- **no_clip** (bool, default: False) – Do **not** clip the image at the frame boundaries (only relevant for non-rectangular maps) [Default is False].
- **nan_transparent** (*bool or str*) – [**+z***value*][*color*] Make grid nodes with z = NaN transparent, using the color-masking feature in PostScript Level 3 (the PS device must support PS Level 3). If the input is a grid, use **+z** to select another grid value than NaN. If input is instead an image, append an alternate *color* to select another pixel value to be transparent [Default is "black"].
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.

- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).
- **cores** (int | bool, default: False) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Examples

Plot the global Earth relief using a Mollweide projection.

```
>>> import pygmt
>>> from pygmt.params import Axis
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = pygmt.datasets.load_earth_relief("30m", registration="gridline")
>>> fig = pygmt.Figure()
>>> fig.grdimage(
...     grid=grid,
...     cmap="gmt/geo",  # Use the built-in "gmt/geo" CPT
...     projection="W10c",  # Set the Mollweide projection and width of 10 cm
...     frame=Axis(annot=True, grid=True),
... )
>>> fig.show()
```

Examples using `pygmt.Figure.grdimage`

pygmt.Figure.grdview

`Figure.`**`grdview`**(*grid*, *cmap=None*, *drape_grid=None*, *surftype=None*, *dpi=None*, *nan_transparent=False*, *monochrome=False*, *contour_pen=None*, *mesh_fill=None*, *mesh_pen=None*, *plane=False*, *facade_fill=None*, *facade_pen=None*, *projection=None*, *smooth=None*, *zscale=None*, *zsize=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Create 3-D perspective image or surface mesh from a grid.

Reads a 2-D grid file and produces a 3-D perspective plot by drawing a mesh, painting a colored/gray-shaded surface made up of polygons, or by scanline conversion of these polygons to a raster image. Options include draping a data set on top of a surface, plotting of contours on top of the surface, and apply artificial illumination based on intensities provided in a separate grid file.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdview.html.

Aliases

- I = shading
- n = interpolation
- B = frame
- C = cmap
- G = drape_grid
- J = projection
- Jz = zscale
- JZ = zsize
- N = plane, facade_fill
- Q = surftype, dpi, mesh_fill, nan_transparent, **+m**: monochrome
- R = region
- S = smooth
- V = verbose
- Wc = contour_pen
- Wf = facade_pen
- Wm = mesh_pen
- c = panel
- f = coltypes
- p = perspective
- t = transparency

Parameters

- **`grid`** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **`cmap`** (`str` | `None`, default: `None`) – The name of the color palette table to use.

- **drapegrid** – The grid (a file name or a `xarray.DataArray`) or image to be draped on top of the grid surface provided by `grid` [Default determines colors from `grid`]. Note that `zscale` and `plane` always refer to `grid`. `drape_grid` only provides the information pertaining to colors. If it's a grid, the colors will be looked-up via the CPT (see `cmap`); if it's an image, `cmap` is not expected.
- **surftype** (`Literal['mesh', 'surface', 'surface+mesh', 'image', 'waterfall_x', 'waterfall_y']` | `None`, default: `None`) – Specify surface type for the grid. Valid values are:
  - `"mesh"`: mesh plot [Default].
  - `"surface"`: surface plot.
  - `"surface+mesh"`: surface plot with mesh lines drawn on top of the surface.
  - `"image"`: image plot.
  - `"waterfall_x"`/`"waterfall_y"`: waterfall plots (row or column profiles).
- **dpi** (`int` | `None`, default: `None`) – Effective dots-per-unit resolution for the rasterization for image plots (i.e., `surftype="image"`) [Default is GMT_GRAPHICS_DPU]
- **nan_transparent** (`bool`, default: `False`) – Make grid nodes with z = NaN transparent, using the color-masking feature in PostScript Level 3. Only applies when `surftype="image"`.
- **monochrome** (`bool`, default: `False`) – Force conversion to monochrome image using the (television) YIQ transformation.
- **contour_pen** (`str` | `None`, default: `None`) – Draw contour lines on top of surface or mesh (not image). Append pen attributes used for the contours.
- **mesh_pen** (`str` | `None`, default: `None`) – Set the pen attributes used for the mesh. Need to set `surftype` to `"mesh"`, or `"surface+mesh"` to draw meshlines.
- **mesh_fill** (`str` | *`Pattern`* | `None`, default: `None`) – Set the mesh fill in mesh plot or waterfall plots [Default is white].
- **plane** (`float` | `bool`, default: `False`) – Draw a plane at the specified z-level. If `True`, defaults to the minimum value in the grid. However, if `region` was used to set *zmin/zmax* then *zmin* is used if it is less than the grid minimum value. Use `facade_pen` and `facade_fill` to control the appearance of the plane. **Note**: For GMT<=6.7.0, *zmin* set in `region` has no effect due to a GMT bug.
- **facade_fill** (`str` | *`Pattern`* | `None`, default: `None`) – Fill for the frontal facade between the plane specified by `plane` and the data perimeter.
- **facade_pen** (`str` | `None`, default: `None`) – Set the pen attributes used for the facade.
- **shading** (*`str or float`*) – Provide the name of a grid file with intensities in the (-1,+1) range, or a constant intensity to apply everywhere (affects the ambient light). Alternatively, derive an intensity grid from the main input data grid by using *`pygmt.grdgradient`* first; append **+a***azimuth*, **+n***args*, and **+m***ambient* to specify azimuth, intensity, and ambient arguments for that function, or just give **+d** to select the default arguments [Default is `"+a-45+nt1+m0"`].
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **smooth** (`int` | `None`, default: `None`) – Sets the smooth factor used for smoothing the contours before plotting [Default is no smoothing].
- **zscale** (`float` | `str` | `None`, default: `None`)

- **zsize** (float | str | None, default: None) – Set z-axis scaling or z-axis size.
- **region** (Sequence[float | str] | str | None, default: None) – *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest. When used with perspective, optionally append */zmin/zmax* to indicate the range to use for the 3-D axes [Default is the region given by the input grid].
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding

`+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Create a 3-D surface plot of the Earth's relief.

```
>>> import pygmt
>>> from pygmt.params import Axis, Frame
>>> # Load the 30 arc-minutes grid with "gridline" registration in a given region
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m",
...     region=[-92.5, -82.5, -3, 7],
...     registration="gridline",
... )
>>> fig = pygmt.Figure()
>>> fig.grdview(
...     grid=grid,
...     # Set the perspective to an azimuth of 130° and an elevation of 30°
...     perspective=(130, 30),
...     # Add a frame to the x- and y-axes
...     # Specify annotations on the south and east borders of the plot
...     frame=Frame(axes="wSnE", xaxis=Axis(annot=True), yaxis=Axis(annot=True)),
...     # Set the projection of the 2-D map to Mercator with a 10 cm width
...     projection="M10c",
...     zsize="2c",  # Set the vertical scale (z-axis) to 2 cm
...     surftype="surface",  # Set "surface plot" to color the surface via a CPT
...     cmap="gmt/geo",  # Specify CPT to "geo"
... )
>>> fig.show()
```

Examples using `pygmt.Figure.grdview`

pygmt.Figure.histogram

Figure.**histogram**(*data*, *weights=False*, *histtype='count'*, *bar_width=None*, *bar_offset=None*, *cmap=False*, *pen=None*, *fill=None*, *annot=False*, *annot_position='end'*, *annot_font=None*, *annot_offset=None*, *annot_orientation='horizontal'*, *horizontal=False*, *center=False*, *out_range=None*, *stairs=False*, *cumulative=False*, *distribution=None*, *distribution_pen=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, ***kwargs*)

Calculate and plot histograms.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/histogram.html.

Aliases

- T = series
- b = binary
- d = nodata
- e = find
- h = header
- l = label
- w = wrap
- A = horizontal
- B = frame

- C = cmap, **+b**: fill
- D = annot, **+b**: annot_position, **+f**: annot_font, **+o**: annot_offset, **+r**: annot_orientation
- E = bar_width, **+o**: bar_offset
- G = fill
- J = projection
- L = out_range
- N = distribution, **+p**: distribution_pen
- Q = cumulative
- R = region
- S = stairs
- V = verbose
- W = pen
- Z = histtype, **+w**: weights
- c = panel
- i = incols
- p = perspective
- t = transparency

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame) – Pass in either a file name to an ASCII data table, a Python list, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data.
- **weights** (bool | Sequence[float], default: False) – Use weighted counts instead of pure counts [Default is False, i.e., pure counts are used]. It can be:
  - True: Weights are provided in the second column of data, if data is a file name or a 2-D sequence.
  - A 1-D array of weights, one per data point, requiring that data is a 1-D sequence of values.
- **histtype** (Literal['count', 'percent', 'count_log', 'percent_log', 'count_log10', 'percent_log10'], default: 'count') – The quantity each bar represents:
  - "count": number of values in the bin [Default]
  - "percent": percentage of values in the bin, i.e., the bar heights sum to 100
  - "count_log": natural logarithm of (1 + count)
  - "percent_log": natural logarithm of (1 + percent)
  - "count_log10": base-10 logarithm of (1 + count)
  - "percent_log10": base-10 logarithm of (1 + percent)
- **cmap** (*str*) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **pen** (str | None, default: None) – Draw bar outline (or stair-case curve) using the specified pen thickness [Default is no outline].
- **fill** (str | *Pattern* | Literal['position', 'value'] | None, default: None) – Set color or pattern for filling bars [Default is no fill]. Set it to one of the two special values to fill bars by looking up the color from a CPT instead of a constant color:
  - "position": look up the color using the mid-coordinate of the bin. This is the default when cmap is set.
  - "value": look up the color using the bin value, i.e., the bar count or frequency.

  The special values require a CPT (either the current CPT or explicitly set by cmap), and can't be used with fill.
- **cmap** – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **annot** (bool, default: False) – If True, annotate each bar with the value it represents. The remaining annot_* parameters control how the annotations look and are ignored if annot is not set.

- **annot_position** (Literal['start', 'end'], default: 'end') – Position of the annotations relative to the bars. Valid values are "start" and "end".
- **annot_font** (str | None, default: None) – Font of the annotations [Default is FONT_ANNOT_PRIMARY].
- **annot_offset** (float | str | None, default: None) – Offset between a bar and its annotation, with an optional *dimension unit* [Default is "6p"].
- **annot_orientation** (Literal['horizontal', 'vertical'], default: 'horizontal') – Orientation of the annotations. Valid values are "horizontal" and "vertical".
- **bar_width** (float | str | None, default: None) – Use an alternative histogram bar width than the default set via series. Give either an alternative width in data units, or the user may append a *dimension unit* for a fixed dimension instead.
- **bar_offset** (float | str | None, default: None) – Shift all bars along the axis by a constant value. It may be given in data units of plot dimension units by appending the relevant unit. Requires bar_width.
- **center** (bool, default: False) – Center bin on each value specified via series [Default uses the values to define the left edge of each bin].
- **out_range** (Literal['first', 'last', 'both'] | None, default: None) – Handle values that fall outside the range set by series. By default, these values are ignored. Valid values are:
  - "first": only include values below first bin into the first bin
  - "last": only include values above the last bin into that last bin
  - "both": include values into the first or last bins
- **cumulative** (bool | Literal['reverse'], default: False) – Pass True to draw a cumulative histogram, or set it to "reverse" to draw a reverse cumulative histogram instead.
- **stairs** (bool, default: False) – Draw a stairs-step diagram which does not include the internal bars of the default histogram.
- **horizontal** (bool, default: False) – Plot the histogram horizontally from x = 0 [Default is vertically from y = 0]. The plot dimensions remain the same, but the two axes are flipped, i.e., the x-axis is plotted vertically and the y-axis is plotted horizontally.
- **series** (*int, str, or list*) – [*min*/*max*/]*inc*[**+n**]. Set the interval for the width of each bar in the histogram.
- **distribution** (Literal['mean', 'median', 'lms'] | Sequence[Literal['mean', 'median', 'lms']] | None, default: None) – Draw the equivalent normal distribution. Select which central location and scale to use:
  - "mean": mean and standard deviation
  - "median": median and L1 scale (1.4826 * median absolute deviation)
  - "lms": least median of squares (LMS) mode and scale

  Pass a sequence of modes to draw several curves at once, e.g., ["mean", "lms"].

  Note: If wrap is used, only "mean" is available and the circular von Mises distribution is determined instead.
- **distribution_pen** (str | Sequence[str] | None, default: None) – Pen used to draw the distribution curve [Default is "0.25p,black,solid"]. Pass a sequence of pens to use a

different pen for each curve; a single pen is used for all of them. If `distribution` is not set, this parameter is ignored.

- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*`Frame`* | *`Axis`* | `Literal`['none'] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **binary** (*`bool or str`*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **label** (*str*) – Add a legend entry for the symbol or line being plotted. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#l-full.
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).

- *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
- *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
- *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **`transparency`** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).
- **`wrap`** (*`str`*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Examples

Plot a histogram of randomly generated data with custom fill color and pen style.

```
>>> import numpy as np
>>> import pygmt
>>> # Generate random data from a normal distribution
>>> rng = np.random.default_rng(seed=100)
>>> data = rng.normal(loc=100, scale=25, size=1024)
>>> fig = pygmt.Figure()
>>> fig.histogram(data=data, frame=True, series=5, fill="red3", pen="1p")
>>> fig.show()
```

Examples using `pygmt.Figure.histogram`

pygmt.Figure.hlines

Figure.**hlines**(*y*, *xmin=None*, *xmax=None*, *pen=None*, *label=None*, *no_clip=False*, *panel=False*, *perspective=None*)

Plot one or multiple horizontal line(s).

This method is a high-level wrapper around *pygmt.Figure.plot* that focuses on plotting horizontal lines at Y-coordinates specified by the `y` parameter. The `y` parameter can be a single value (for a single horizontal line) or a sequence of values (for multiple horizontal lines).

By default, the X-coordinates of the start and end points of the lines are set to be the X-limits of the current plot, but this can be overridden by specifying the `xmin` and `xmax` parameters. `xmin` and `xmax` can be either a single value or a sequence of values. If a single value is provided, it is applied to all lines. If a sequence is provided, the length of `xmin` and `xmax` must match the length of `y`.

The term "horizontal" lines can be interpreted differently in different coordinate systems:

- **Cartesian**: lines are plotted as straight lines.
- **Polar**: lines are plotted as arcs along a constant radius.
- **Geographic**: lines are plotted as arcs along parallels (i.e., constant latitude).

Parameters

- **y** (`float` | `Sequence`[`float`]) – Y-coordinates to plot the lines. It can be a single value (for a single line) or a sequence of values (for multiple lines).
- **xmin/xmax** – X-coordinates of the start/end point(s) of the line(s). If `None`, defaults to the X-limits of the current plot. `xmin` and `xmax` can be either a single value or a sequence of values. If a single value is provided, it is applied to all lines. If a sequence is provided, the length of `xmin` and `xmax` must match the length of `y`.
- **pen** (`str` | `None`, default: `None`) – Pen attributes for the line(s), in the format of *width,color,style*.
- **label** (`str` | `None`, default: `None`) – Label for the line(s), to be displayed in the legend.
- **no_clip** (`bool`, default: `False`) – Do **not** clip lines outside the plot region. Only makes sense in the Cartesian coordinate system. [Default is `False` to clip lines at the plot region.]
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`str` | `bool` | `None`, default: `None`) – Select perspective view and set the azimuth and elevation angle of the viewpoint. Refer to *pygmt.Figure.plot* for details.

Examples

Plot several horizontal lines at different y-coordinates with varying x-limits and pen styles. Lines are labeled and a legend is displayed.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c/10c", frame=True)
```

(continues on next page)

(continued from previous page)

```
>>> fig.hlines(y=1, pen="1p,black", label="Line at y=1")
>>> fig.hlines(y=2, xmin=2, xmax=8, pen="1p,red,-", label="Line at y=2")
>>> fig.hlines(y=[3, 4], xmin=3, xmax=7, pen="1p,black,.", label="Lines at y=3,4")
>>> fig.hlines(y=[5, 6], xmin=4, xmax=9, pen="1p,red", label="Lines at y=5,6")
>>> fig.hlines(
...     y=[7, 8], xmin=[0, 1], xmax=[7, 8], pen="1p,blue", label="Lines at y=7,8"
... )
>>> fig.legend()
>>> fig.show()
```

Examples using `pygmt.Figure.hlines`

pygmt.Figure.image

`Figure.`**`image`**(*imagefile*, *position=None*, *width=None*, *height=None*, *dpi=None*, *replicate=None*, *box=False*, *monochrome=False*, *invert=False*, *bgcolor=None*, *fgcolor=None*, *transparent_color=None*, *bitcolor=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot raster or EPS images.

Reads an Encapsulated PostScript file or a raster image file and plots it. The image can be scaled arbitrarily, and 1-bit raster images can be:

- inverted, i.e., black pixels (on) become white (off) and vice versa.
- colorized, by assigning different foreground and background colors.
- made transparent where either the back- or foreground is painted.

As an option, the user may choose to convert colored raster images to grayscale using TV's YIQ-transformation. For raster files, the user can select which color is made transparent. The user may also choose to replicate the image which, when preceded by appropriate clip paths, may allow larger custom-designed fill patterns to be implemented.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/image.html.

Aliases

- B = frame
- D = position, **+w**: width/height, **+r**: dpi, **+n**: replicate
- F = box
- G = **+b**: bgcolor, **+f**: fgcolor, **+t**: transparent_color, bitcolor
- I = invert
- J = projection
- M = monochrome
- R = region
- V = verbose
- c = panel
- p = perspective
- t = transparency

Parameters

- **`imagefile`** (`str` | `PathLike`) – An Encapsulated PostScript (EPS) file or a raster image file. An EPS file must contain an appropriate BoundingBox. A raster file can have a depth of 1, 8, 24, or 32 bits and is read via GDAL.
- **`position`** (*`Position`* | `Sequence`[`float` | `str`] | `Literal`[`'TL'`, `'TC'`, `'TR'`, `'ML'`, `'MC'`, `'MR'`, `'BL'`, `'BC'`, `'BR'`] | `None`, default: `None`) – Position of the image on the plot. It can be specified in multiple ways:
  - A *`pygmt.params.Position`* object to fully control the reference point, anchor point, and offset.

  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot (position `(0, 0)` with anchor `"BL"`).

- **width** (`float` | `str` | `None`, default: `None`)
- **height** (`float` | `str` | `None`, default: `None`) – Width (and height) of the image in plot dimension, with an optional *dimension unit*. If `height` (or `width`) is set to 0, then the original aspect ratio of the image is maintained. If `width` (or `height`) is negative, the absolute value is used to interpolate image to the device resolution using the PostScript image operator. If neither dimensions nor `dpi` are set then revert to the default dpi [GMT_GRAPHICS_DPU].
- **dpi** (`float` | `str` | `None`, default: `None`) – Set the dpi of the image in dots per inch, or append **c** to indicate this is dots per cm.
- **replicate** (`int` | `Sequence[int]` | `None`, default: `None`) – *nx* or (*nx*, *ny*). Replicate the (scaled) image *nx* times in the horizontal direction, and *ny* times in the vertical direction. If a single integer *nx* is given, *ny* = *nx*. [Default is (1, 1)].
- **box** (*Box* | `bool`, default: `False`) – Draw a background box behind the image. If set to `True`, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **bgcolor** (`str` | `None`, default: `None`)
- **fgcolor** (`str` | `None`, default: `None`) – For 1-bit images, set the background and foreground colors [Default is the image's own colors]. Setting either to an empty string makes those pixels transparent. Cannot be both empty.
- **transparent_color** (`str` | `None`, default: `None`) – For color images, set a single color that should be made transparent.
- **bitcolor** (`str` | `Sequence[str]` | `None`, default: `None`) – [*color*][**+b**|**f**|**t**]. Change certain pixel values to another color or make them transparent. For 1-bit images you can specify an alternate *color* for the background (**+b**) or the foreground (**+f**) pixels, or give no color to make those pixels transparent. Alternatively, for color images you can select a single *color* that should be made transparent instead (**+t**). Pass a list to specify multiple settings.

  Deprecated since version 0.20.0: Use `bgcolor`, `fgcolor`, or `transparent_color` instead. Will be removed in 0.24.0.
- **monochrome** (`bool`, default: `False`) – Convert color image to monochrome grayshades using the (television) YIQ-transformation.
- **invert** (`bool`, default: `False`) – Invert 1-bit image before plotting, i.e., black pixels (on) become white (off) and vice versa. Ignored if used with color images.

  Note: There was an upstream GMT bug, so this feature may not work correctly for some 1-bit images for GMT<=6.6.0. See PR #8837.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

```
>>> import pygmt
>>> from pygmt.params import Box
```

Plot a raster image at its original size (based on the default dpi):

```
>>> fig = pygmt.Figure()
>>> fig.image("@circuit.png")
>>> fig.show()
```

Place the image at the Top Right corner of an existing basemap, scale it to a width of 3 centimeters, and draw a box around it:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c", frame=True)
>>> fig.image("@circuit.png", position="TR", width="3c", box=Box(pen="thin,blue"))
>>> fig.show()
```

Examples using `pygmt.Figure.image`

pygmt.Figure.inset

`Figure.`**`inset`**(*position=None*, *width=None*, *height=None*, *box=False*, *no_clip=False*, *projection=None*, *region=None*, *verbose=False*, ***kwargs*)

Manage figure inset setup and completion.

This method carves out a sub-region of the current plot canvas and restrict further plotting to that section of the canvas. Plotting methods that are called within the context manager are added to the inset figure.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/inset.html.

Aliases

- C = clearance
- D = position, **+w**: width/height
- F = box
- J = projection
- N = no_clip
- R = region
- V = verbose

Parameters

- **position** (*Position* | Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None, default: None) – Position of the inset on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot.
- **width** (float | str | None, default: None)
- **height** (float | str | None, default: None) – Width and height of the inset. Width must be specified unless `projection` and `region` are given, and height is set to be equal to width if not specified.
- **box** (*Box* | bool, default: False) – Draw a background box behind the inset. If set to `True`, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **clearance** (*float, str, or list*) – This is clearance that is added around the inside of the inset. Plotting will take place within the inner region only. The margins can be a single value, a pair of values separated (for setting separate horizontal and vertical margins), or the full set of four margins (for setting separate left, right, bottom, and top margins). When passing multiple values, it can be either a list or a string with the values separated by forward slashes [Default is no margins].

- **no_clip** (bool, default: False) – Do **not** clip features extruding outside the inset frame boundaries [Default is False].
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Examples

```
>>> import pygmt
>>> from pygmt.params import Box, Position
>>>
>>> fig = pygmt.Figure()
>>> fig.coast(region="MG+r2", water="lightblue", shorelines="thin")
>>> # Use a "with" statement to initialize the inset context manager.
>>> # Setting the position to Top Left and a width of 3.5 centimeters
>>> with fig.inset(
...     position=Position("TL", offset=0.2),
...     width="3.5c",
...     clearance=0,
...     box=Box(pen="green"),
... ):  # Map elements under the "with" statement are plotted in the inset
...     fig.coast(
...         region="g",
...         projection="G47/-20/?",
...         land="gray",
...         water="white",
...         dcw="MG+gred",
...     )
>>> # Map elements outside the "with" statement are plotted in the main figure
>>> fig.logo(position=Position("BR", offset=0.2), width="3c")
>>> fig.show()
```

Examples using `pygmt.Figure.inset`

pygmt.Figure.legend

Figure.**legend**(*spec=None*, *position=None*, *width=None*, *height=None*, *line_spacing=None*, *box=False*, *scale=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot a legend.

Makes legends that can be overlaid on plots. It reads specific legend-related information from an input file, a io.StringIO object, or automatically creates legend entries from plotted symbols that have labels. Unless otherwise noted, annotations will be made using the primary annotation font and size in effect (i.e., FONT_ANNOT_PRIMARY).

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/legend.html.

Aliases

- B = frame
- D = position, **+w**: width/height, **+l**: line_spacing
- F = box
- J = projection
- R = region
- S = scale
- V = verbose
- c = panel
- p = perspective
- t = transparency

Parameters

- **spec** (`str` | `PathLike` | `StringIO` | `None`, default: `None`) – The legend specification. It can be:
  - `None` which means using the automatically generated legend specification file
  - Path to the legend specification file
  - A `io.StringIO` object containing the legend specification

  See https://docs.generic-mapping-tools.org/6.7/legend.html for the definition of the legend specification.
- **position** (*Position* | `Sequence`[`float` | `str`] | `Literal`['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | `None`, default: `None`) – Position of the legend on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Top Right corner inside the plot with a 0.2-cm offset.
- **width** (`float` | `str` | `None`, default: `None`)
- **height** (`float` | `str` | `None`, default: `None`) – Width and height of the legend box. If not given, the width and height are computed automatically based on the contents of the legend specification. If unit is `%` (percentage) then width is computed as that fraction of the plot width. If height is given as percentage then height is recomputed as that fraction of the legend width (not plot height).

  Note: Currently, the automatic width calculation only works when legend codes **D**, **H**, **L**, **S**, or **V** are used and that the number of symbol columns (**N**) is 1. If height is zero or not given, GMT estimates it based on the expected vertical extent of the items to be placed. A legend containing paragraph text may require an explicit height.
- **line_spacing** (`float` | `None`, default: `None`) – The line-spacing factor between legend entries in units of the current font size [Default is 1.1].
- **box** (*Box* | `bool`, default: `False`) – Draw a background box behind the legend. If set to `True`, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **scale** (`float` | `None`, default: `None`) – Scale all symbol sizes by a common scale [Default is 1.0, i.e., no scaling].
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.

- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Create a legend automatically from the plotted symbols and lines that have labels:

```
>>> import pygmt
>>> from pygmt.params import Box, Position
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c", frame=True)
>>> fig.plot(x=[2, 5, 8], y=[3, 6, 4], style="c0.3c", fill="red", label="Circles")
>>> fig.plot(x=[2, 5, 8], y=[7, 2, 8], pen="1p,blue", label="Line")
```

(continues on next page)

(continued from previous page)

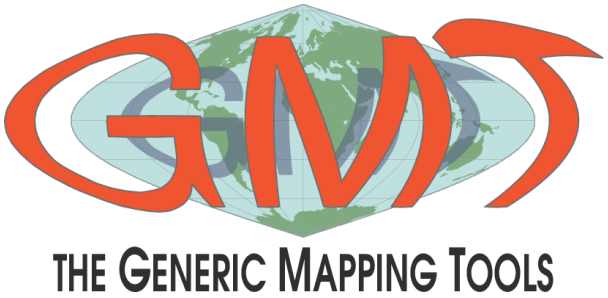
```
>>> fig.legend(
...     position=Position("BL", offset=0.2),
...     width="4c",
...     box=Box(pen="1p,gray30", fill="lightyellow", radius="4p"),
... )
>>> fig.show()
```

Examples using `pygmt.Figure.legend`

pygmt.Figure.logo

`Figure.`**`logo`**(*position=None*, *width=None*, *height=None*, *box=False*, *style='standard'*, *projection=None*, *region=None*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot the GMT logo.



By default, the GMT logo is 2 inches wide and 1 inch high and will be positioned relative to the current plot origin.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/gmtlogo.html.

Aliases

- D = position, **+w**: width, **+h**: height
- F = box
- J = projection
- R = region
- S = style
- V = verbose
- c = panel
- p = perspective
- t = transparency

Parameters

- **position** (*Position* | Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None, default: None) – Position of the GMT logo on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., (1, 2) or ("1c", "2c").
  - A *2-character justification code* for a position inside the plot, e.g., "TL" for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot (position (0, 0) with anchor "BL").
- **width** (float | str | None, default: None)

- **height** (float | str | None, default: None) – Width or height of the GMT logo. Since the aspect ratio is fixed, only one of the two can be specified. [Default is 2 inches wide and 1 inch high].
- **box** (*Box* | bool, default: False) – Draw a background box behind the logo. If set to True, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **style** (Literal['standard', 'url', 'no_label'], default: 'standard') – Control what is written beneath the map portion of the logo.
  - "standard": The text label "The Generic Mapping Tools".
  - "no_label": Skip the text label.
  - "url": The URL to the GMT website.
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

See also

pygmt.Figure.pygmtlogo
Plot the PyGMT logo.

Examples

```
>>> import pygmt
```

The simplest way to plot the GMT logo is to just call the method without any arguments.

```
>>> fig = pygmt.Figure()
>>> fig.logo()
>>> fig.show()
```

To plot the GMT logo at the Top Right corner on an existing basemap:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-90, -70, 0, 20], projection="M10c", frame=True)
>>> fig.logo(position="TR", width="3c")
>>> fig.show()
```

Examples using `pygmt.Figure.logo`

pygmt.Figure.magnetic_rose

Figure.**magnetic_rose**(*position=None*, *width=None*, *labels=False*, *outer_pen=False*, *inner_pen=False*, *declination=None*, *declination_label=None*, *intervals=None*, *box=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Add a magnetic rose to the map.

Parameters

- **position** (*Position* | Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None, default: None) – Position of the magnetic rose on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., (1, 2) or ("1c", "2c").
  - A *2-character justification code* for a position inside the plot, e.g., "TL" for Top Left corner inside the plot.

  If not specified, defaults to the Top Right corner of the plot.
- **width** (float | str | None, default: None) – Width of the rose in plot coordinates, or append unit % for a size in percentage of plot width [Default is 15%].
- **labels** (Sequence[str] | bool, default: False) – A sequence of four strings to label the cardinal points W, E, S, N. Use an empty string to skip a specific label. If the north label is "*", then a north star is plotted instead of the north label. If set to True, use the default labels ["W", "E", "S", "N"].

- **outer_pen** (str | bool, default: False) – Draw the outer circle of the magnetic rose, using the given pen attributes.
- **inner_pen** (str | bool, default: False) – Draw the inner circle of the magnetic rose, using the given pen attributes.
- **declination** (float | None, default: None) – Magnetic declination in degrees. By default, only a geographic north is plotted. With this parameter set, a magnetic north is also plotted. A magnetic compass needle is drawn inside the rose to indicate the direction to magnetic north.
- **declination_label** (str | None, default: None) – Label for the magnetic compass needle. Default is to format a label based on declination. To bypass the label, set to "-".
- **intervals** (Sequence[float] | None, default: None) – Specify the annotation and tick intervals for the geographic and magnetic directions. It can be a sequence of three or six values. If three values are given, they are used for both geographic and magnetic directions. If six values are given, the first three are used for geographic directions and the last three for magnetic directions. [Default is (30, 5, 1)]. **Note**: If MAP_EMBELLISHMENT_MODE is "auto" and the compass size is smaller than 2.5 cm then the interval defaults are reset to (90,30, 3, 45, 15, 3).
- **box** (*Box* | bool, default: False) – Draw a background box behind the magnetic rose. If set to True, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot a magnetic rose with custom position, width, labels, intervals, and pen styles.

```
>>> import pygmt
>>> from pygmt.params import Position
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-10, 10, -10, 10], projection="M15c", frame=True)
>>> fig.magnetic_rose(
...     position=Position((-5, -5), cstype="mapcoords"),
...     width="4c",
...     labels=["W", "E", "S", "*"],
...     intervals=(45, 15, 3, 60, 20, 4),
...     outer_pen="1p,red",
...     inner_pen="1p,blue",
...     declination=11.5,
...     declination_label="11.5°E",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.magnetic_rose`

pygmt.Figure.meca

Figure.**meca**(*spec*, *scale*, *convention=None*, *component='full'*, *longitude=None*, *latitude=None*, *depth=None*, *plot_longitude=None*, *plot_latitude=None*, *event_name=None*, *cmap=None*, *extension_fill=None*, *compression_fill=None*, *pen=None*, *no_clip=False*, *projection=None*, *frame=False*, *region=None*, *verbose=False*, *panel=False*, *transparency=None*, *perspective=False*, ***kwargs*)

Plot focal mechanisms.

The following focal mechanism conventions are supported:

Table 3: Supported focal mechanism conventions.

Convention	Description	Focal parameters	Remark
`"aki"`	Aki and Richard	*strike*, *dip*, *rake*, *magnitude*	angles in degrees
`"gcmt"`	global centroid moment tensor	*strike1*, *dip1*, *rake1*, *strike2*, *dip2*, *rake2*, *mantissa*, *exponent*	angles in degrees; seismic moment is $mantissa * 10^{exponent}$ in dyn cm
`"mt"`	seismic moment tensor	*mrr*, *mtt*, *mff*, *mrt*, *mrf*, *mtf*, *exponent*	moment components in $10^{exponent}$ dyn cm
`"partial"`	partial focal mechanism	*strike1*, *dip1*, *strike2*, *fault_type*, *magnitude*	angles in degrees; *fault_type* means +1/-1 for normal/reverse fault
`"principal_axis"`	principal axis	*t_value*, *t_azimuth*, *t_plunge*, *n_value*, *n_azimuth*, *n_plunge*, *p_value*, *p_azimuth*, *p_plunge*, *exponent*	values in $10^{exponent}$ dyn cm; azimuths and plunges in degrees

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/supplements/seis/meca.html.

Aliases

- A = offset
- Fr = label_box
- L = outline
- T = nodal
- B = frame
- C = cmap
- E = extension_fill
- G = compression_fill
- J = projection
- N = no_clip
- R = region
- S = scale/convention/component
- V = verbose
- W = pen
- c = panel
- p = perspective
- t = transparency

Parameters

- **spec** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame`) – Data that contain focal mechanism parameters.

  `spec` can be specified in either of the following types:

  – *str*: a file name containing focal mechanism parameters as columns. The meaning of each column is:

  * Columns 1 and 2: event longitude and latitude
  * Column 3: event depth (in kilometers)
  * Columns 4 to 3+n: focal mechanism parameters. The number of columns *n* depends on the choice of `convention` (see the table above for the supported conventions).
  * Columns 4+n and 5+n: longitude and latitude at which to place the beachball. `0 0` plots the beachball at the longitude and latitude given in the columns 1 and 2. [optional; requires `offset=True`].

    * Last Column: text string to appear near the beachball [optional].

  - *1-D np.array*: focal mechanism parameters of a single event. The meanings of columns are the same as above.
  - *2-D np.array*: focal mechanism parameters of multiple events. The meanings of columns are the same as above.
  - *dict* or `pandas.DataFrame`: The dict keys or `pandas.DataFrame` column names determine the focal mechanism convention. For the different conventions, the combination of keys / column names as given in the table above are required.

    A dict may contain values for a single focal mechanism or lists of values for multiple focal mechanisms.

    Both dict and `pandas.DataFrame` may optionally contain the keys / column names: `latitude`, `longitude`, `depth`, `plot_longitude`, `plot_latitude`, and/or `event_name`.

  If `spec` is either a str or a 1-D or 2-D numpy array, the `convention` parameter is required to interpret the columns. If `spec` is a dict or a `pandas.DataFrame`, `convention` is not needed and ignored if specified.

- **`scale`** (*`float or str`*) – *scale*[**+a***angle*][**+f***font*][**+j***justify*][**+l**][**+m**][**+o***dx*[/*dy*]][**+s***reference*]. Adjust scaling of the radius of the beachball, which is proportional to the magnitude. By default, *scale* defines the size for magnitude = 5 (i.e., scalar seismic moment M0 = 4.0E23 dyn cm). If **+l** is used the radius will be proportional to the seismic moment instead. Use **+s** and give a *reference* to change the reference magnitude (or moment), and use **+m** to plot all beachballs with the same size. A text string can be specified to appear near the beachball (corresponding to column or parameter `event_name`). Append **+a***angle* to change the angle of the text string; append **+f***font* to change its font (size,fontname,color); append **+j***justify* to change the text location relative to the beachball [Default is `"TC"`, i.e., Top Center]; append **+o** to offset the text string by *dx*/*dy*.
- **`convention`** (`Literal['aki', 'gcmt', 'mt', 'partial', 'principal_axis'] | None`, default: `None`) – Specify the focal mechanism convention of the input data. Ignored if `spec` is a dict or `pandas.DataFrame`. See the table above for the supported conventions.
- **`component`** (`Literal['full', 'dc', 'deviatoric']`, default: `'full'`) – The component of the seismic moment tensor to plot. Valid values are:
  - `"full"`: the full seismic moment tensor
  - `"dc"`: the closest double couple defined from the moment tensor (zero trace and zero determinant)
  - `"deviatoric"`: deviatoric part of the moment tensor (zero trace)
- **`longitude/latitude/depth`** – Longitude(s), latitude(s), and depth(s) of the event(s). The length of each must match the number of events. These parameters are only used if `spec` is a dictionary or a `pandas.DataFrame`, and they override any existing `longitude`, `latitude`, or `depth` values in `spec`.
- **`plot_longitude/plot_latitude`** – Longitude(s) and latitude(s) at which to place the beachball(s). The length of each must match the number of events. These parameters are only used if `spec` is a dictionary or a `pandas.DataFrame`, and they override any existing `plot_longitude` or `plot_latitude` values in `spec`.
- **`event_name`** (`str | Sequence[str] | None`, default: `None`) – Text string(s), such as event name(s), to appear near the beachball(s). The length must match the number of events. This parameter is only used if `spec` is a dictionary or a `pandas.DataFrame`, and it overrides any existing `event_name` labels in `spec`.

- **label_box** (*bool or str*) – [*fill*]. Draw a box behind the label if given via `event_name`. Use *fill* to give a fill color [Default is `"white"`].
- **offset** (*bool or str*) – [**+p***pen*][**+s***size*]. Offset beachball(s) to the longitude(s) and latitude(s) specified in the last two columns of the input file or array, or by `plot_longitude` and `plot_latitude` if provided. A line from the beachball to the initial location is drawn. Use **+s***size* to plot a small circle at the initial location and to set the diameter of this circle [Default is no circle]. Use **+p***pen* to set the pen attributes for this feature [Default is set via `pen`]. The fill of the circle is set via `compression_fill` or `cmap`, i.e., corresponds to the fill of the compressive quadrants.
- **compression_fill** (`str` | *Pattern* | `None`, default: `None`) – Set color or pattern for filling compressive quadrants [Default is `"black"`]. This setting also applies to the fill of the circle defined via `offset`.
- **extension_fill** (`str` | *Pattern* | `None`, default: `None`) – Set color or pattern for filling extensive quadrants [Default is `"white"`].
- **pen** (`str` | `None`, default: `None`) – Set (default) pen attributes for all lines related to the beachball [Default is `"0.25p,black,solid"`]. This setting applies to `outline`, `nodal`, and `offset`, unless overruled by arguments passed to those parameters. Draws the circumference of the beachball.
- **outline** (*bool or str*) – [*pen*]. Draw circumference and nodal planes of the beachball. Use *pen* to set the pen attributes for this feature [Default is set via `pen`].
- **nodal** (*bool, int, or str*) – [*nplane*][/*pen*]. Plot the nodal planes and outline the bubble which is transparent. If *nplane* is
  - `0` or `True`: both nodal planes are plotted [Default].
  - `1`: only the first nodal plane is plotted.
  - `2`: only the second nodal plane is plotted.

  Use */pen* to set the pen attributes for this feature [Default is set via `pen`]. For double couple mechanisms, `nodal` renders the beachball transparent by drawing only the nodal planes and the circumference. For non-double couple mechanisms, `nodal=0` overlays best double couple transparently.
- **cmap** (`str` | `None`, default: `None`) – File name of a CPT file or a series of comma-separated colors (e.g., *color1,color2,color3*) to build a linear continuous CPT from those colors automatically. The color of the compressive quadrants is determined by the z-value (i.e., event depth or the third column for an input file). This setting also applies to the fill of the circle defined via `offset`.
- **no_clip** (`bool`, default: `False`) – Do **not** skip symbols that fall outside the frame boundaries [Default is `False`, i.e., plot symbols inside the frame boundaries only].
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame

and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int | Sequence[int] | bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float | Sequence[float] | str | bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (`float | None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot a single focal mechanism using the Aki and Richards convention:

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-124, -122, 47, 49], projection="M10c", frame=True)
>>> fig.coast(land="gray80", water="lightblue", shorelines="0.5p,gray30")
>>> fig.meca(
...     spec={"strike": 330, "dip": 30, "rake": 90, "magnitude": 3},
...     scale="1c",
...     longitude=-124.3,
...     latitude=48.1,
...     depth=12.0,
... )
>>> fig.show()
```

Examples using `pygmt.Figure.meca`

pygmt.Figure.paragraph

Figure.**paragraph**(*x*, *y*, *text*, *parwidth*, *linespacing*, *font=None*, *angle=None*, *justify=None*, *fill=None*, *pen=None*, *alignment='left'*, *tab_width=4*, *blank_line=False*, *verbose=False*, *panel=False*, *transparency=None*)

Typeset one or multiple paragraphs.

This method typesets one or multiple paragraphs of text at a given position. The text is flowed within a given paragraph width and with a specified line spacing, and can be aligned left, center, right, or justified.

Multiple paragraphs can be provided as a sequence of strings, where each string represents a separate paragraph, or as a single string with a blank line (`\n\n`) separating the paragraphs.

The text string is typeset following the What You Type Is What You Get principle, meaning that the text is rendered exactly as it appears in the input string. This allows for precise control over the formatting of the text, including the use of multiple spaces and tabs. By default, a tab is replaced with four spaces, but this can be changed by setting the `tab_width`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/text.html.

Parameters

- **x/y** – The x, y coordinates of the paragraph.
- **text** (`str` | `Sequence`[`str`]) – The paragraph text to typeset. If a sequence of strings is provided, each string is treated as a separate paragraph.
- **parwidth** (`float` | `str`) – The width of the paragraph.
- **linespacing** (`float` | `str`) – The spacing between lines.
- **font** (`str` | `None`, default: `None`) – The font of the text.
- **angle** (`float` | `None`, default: `None`) – The angle of the text.
- **justify** (`Literal`['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | `None`, default: `None`) – Set the alignment of the block of text, relative to the given x, y position. Choose a *2-character justification code*.
- **alignment** (`Literal`['left', 'center', 'right', 'justified'], default: 'left') – Set the alignment of the text. Valid values are "left", "center", "right", and "justified".
- **fill** (`str` | *Pattern* | `None`, default: `None`) – Set color for filling the paragraph box [Default is no fill].
- **pen** (`str` | `None`, default: `None`) – Set the pen for the paragraph box [Default is "0.25p,black,solid"].
- **tab_width** (`int`, default: `4`) – Number of spaces used to expand tab characters in `text` when typesetting. Must be a non-negative integer. Use `0` to remove tab characters instead of replacing them with spaces.
- **blank_line** (`bool`, default: `False`) – If `True`, use a blank line between paragraphs. [Default is `False`, i.e., no blank line between paragraphs.]
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.

- `True` to advance to the next panel in the selected order.
- *index* to specify the index of the desired panel.
- (*row*, *col*) to specify the row and column of the desired panel.

The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.

- **`transparency`** (`float` | `Sequence`[`float`] | `bool` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Typeset a paragraph of text within a specified width and line spacing on a plot.

```
>>> import pygmt
>>>
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c/10c", frame=True)
>>> fig.paragraph(
...     x=4,
...     y=4,
...     text="This is a long paragraph. " * 10,
...     parwidth="5c",
...     linespacing="12p",
...     font="12p",
... )
>>> fig.show()
```

pygmt.Figure.plot

`Figure.`**`plot`**(*data=None*, *x=None*, *y=None*, *size=None*, *symbol=None*, *direction=None*, *cmap=False*, *straight_line=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Plot lines, polygons, and symbols in 2-D.

Takes a matrix, (x,y) pairs, or a file name as input and plots lines, polygons, or symbols at those locations on a plot.

Must provide either `data` or `x`/`y`.

If providing data through `x`/`y`, `fill` can be a 1-D array that will be mapped to a colormap.

If a symbol is selected and no symbol size given, then plot will interpret the third column of the input data as symbol size. Symbols whose size is <= 0 are skipped. If no symbols are specified then the symbol code (see `style` below) must be present as last column in the input. If `style` is not used, a line connecting the data points will be drawn instead. To explicitly close polygons, use `close`. Select a fill with `fill`. If `fill` is set, `pen` will control whether the polygon outline is drawn or not. If a symbol is selected, `fill` and `pen` determine the fill and outline/no outline, respectively.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/plot.html.

Aliases

- D = offset
- E = error_bar
- F = connection
- G = fill
- I = intensity
- L = close
- N = no_clip
- S = style
- W = pen
- Z = zvalue
- a = aspatial
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- l = label
- w = wrap
- A = straight_line
- B = frame
- C = cmap
- J = projection
- R = region
- V = verbose
- c = panel
- f = coltypes
- i = incols
- p = perspective
- t = transparency

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in either a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data. Use parameter incols to choose which columns are x, y, fill, and size, respectively.
- **x/y** (*float or 1-D arrays*) – The x and y coordinates, or arrays of x and y coordinates of the data points
- **size** (*1-D array*) – The size of the data points in units specified using style. Only valid if using x/y.
- **symbol** (*1-D array*) – The symbols of the data points. Only valid if using x/y.
- **direction** (*list of two 1-D arrays*) – If plotting vectors (using style="V" or style="v"), then should be a list of two 1-D arrays with the vector directions. These can be angle and length, azimuth and length, or x and y components, depending on the style options chosen.
- **straight_line** (bool | Literal['x', 'y'], default: False) – By default, line segments are drawn as straight lines in the Cartesian and polar coordinate systems, and as great circle arcs (by resampling coarse input data along such arcs) in the geographic coordinate system. The straight_line parameter can control the drawing of line segments. Valid values are:
  - True: Draw line segments as straight lines in geographic coordinate systems.
  - "x": Draw line segments by first along *x*, then along *y*.
  - "y": Draw line segments by first along *y*, then along *x*.

  Here, *x* and *y* have different meanings depending on the coordinate system:
  - **Cartesian** coordinate system: *x* and *y* are the X- and Y-axes.
  - **Polar** coordinate system: *x* and *y* are theta and radius.
  - **Geographic** coordinate system: *x* and *y* are parallels and meridians.

  Attention

  There exits a bug in GMT<=6.5.0 that, in geographic coordinate systems, the meaning of *x* and *y* is reversed, i.e., *x* means meridians and *y* means parallels. The bug is fixed by upstream PR #8648.

- **cmap** (*str | bool*, default: `False`) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **offset** (*str*) – *dx*/*dy*. Offset the plot symbol or line locations by the given amounts *dx*/*dy* [Default is no offset]. If *dy* is not given it is set equal to *dx*.
- **error_bar** (*bool or str*) – [**x**|**y**|**X**|**Y**][**+a**|**A**][**+cl**|**f**][**+n**][**+w***cap*][**+p***pen*]. Draw error bars. Full documentation is at https://docs.generic-mapping-tools.org/6.7/plot.html#e.
- **connection** (*str*) – [**c**|**n**|**p**][**a**|**r**|**s**|**t**|*refpoint*]. Alter the way points are connected (by specifying a *scheme*) and data are grouped (by specifying a *method*). Append one of three line connection schemes:
  - **c**: Draw continuous line segments for each group [Default].
  - **n**: Draw networks of line segments between all points in each group.
  - **p**: Draw line segments from a reference point reset for each group.

  Optionally, append the one of four segmentation methods to define the group:
  - **a**: Ignore all segment headers, i.e., let all points belong to a single group, and set group the reference point to the very first point of the first file.
  - **r**: Segment headers are honored so each segment is a group; the group reference point is reset after each record to the previous point (this method is only available with the `connection="p"` scheme).
  - **s**: Same as **r**, but the group reference point is reset to the first point of each incoming segment [Default].
  - **t**: Consider all data in each file to be a single separate group and reset the group reference point to the first point of each group.

  Instead of the codes **a**|**r**|**s**|**t** you may append the coordinates of a *refpoint* which will serve as a fixed external reference point for all groups.
- **fill** (*str*) – Set color or pattern for filling symbols or polygons [Default is no fill]. *fill* can be a 1-D array, but it is only valid if using `x`/`y` and `cmap=True` is also required.
- **intensity** (*float, bool, or 1-D array*) – Provide an *intensity* value (nominally in the -1 to +1 range) to modulate the fill color by simulating illumination. If using `intensity=True`, we will instead read *intensity* from the first data column after the symbol parameters (if given). *intensity* can also be a 1-D array to set varying intensity for symbols, but it is only valid for `x`/`y` pairs.
- **close** (*str*) – [**+b**|**d**|**D**][**+xl**|**r**|*x0*][**+yl**|**r**|*y0*][**+p***pen*]. Force closed polygons. Full documentation is at https://docs.generic-mapping-tools.org/6.7/plot.html#l.
- **no_clip** (*bool or str*) – [**c**|**r**]. Do **not** clip symbols that fall outside the frame boundaries [Default plots points whose coordinates are strictly inside the frame boundaries only]. The parameter does not apply to lines and polygons which are always clipped to the plot region. For periodic (360-longitude) maps we must plot all symbols twice in case they are clipped by the repeating boundary. `no_clip=True` will turn off clipping and not plot repeating symbols. Use `no_clip="r"` to turn off clipping but retain the plotting of such repeating symbols, or use `no_clip="c"` to retain clipping but turn off plotting of repeating symbols.
- **style** (*str*) – Plot symbols (including vectors, pie slices, fronts, decorated or quoted lines).
- **pen** (*str*) – Set pen attributes for lines or the outline of symbols.

- **zvalue** (*str*) – *value*|*file*. Instead of specifying a symbol or polygon fill and outline color via `fill` and `pen`, give both a *value* via `zvalue` and a color lookup table via `cmap`. Alternatively, give the name of a *file* with one z-value (read from the last column) for each polygon in the input data. To apply it to the fill color, use `fill="+z"`. To apply it to the pen color, append **+z** to `pen`.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal`['none'] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.

- `True` to advance to the next panel in the selected order.
- *index* to specify the index of the desired panel.
- (*row*, *col*) to specify the row and column of the desired panel.

The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:
  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.

  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.

- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **label** (*str*) – Add a legend entry for the symbol or line being plotted. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#l-full.
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (`float` | `Sequence`[`float`] | `bool` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF

or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing). `transparency` can also be a 1-D array to set varying transparency for symbols, but this option is only valid if using `x`/`y`.

- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

See also

pygmt.Figure.choropleth
Plot a choropleth map.

pygmt.Figure.fill_between
Fill the area between two curves.

pygmt.Figure.hlines
Plot horizontal lines.

pygmt.Figure.vlines
Plot vertical lines.

Examples

Plot a blue line connecting the data points and then plot the data points as red circles on top:

```
>>> import pygmt
>>> x = [1, 3, 5, 7, 9]
>>> y = [2, 6, 4, 8, 5]
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c", frame=True)
>>> fig.plot(x=x, y=y, pen="1p,blue")  # Line
>>> fig.plot(x=x, y=y, style="c0.3c", fill="cyan", pen="0.5p,black")  # Circle
>>> fig.show()
```

Examples using `pygmt.Figure.plot`

pygmt.Figure.plot3d

Figure.**plot3d**(*data=None*, *x=None*, *y=None*, *z=None*, *size=None*, *symbol=None*, *direction=None*, *cmap=False*, *straight_line=False*, *projection=None*, *zscale=None*, *zsize=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Plot lines, polygons, and symbols in 3-D.

Takes a matrix, (x, y, z) triplets, or a file name as input and plots lines, polygons, or symbols at those locations in 3-D.

Must provide either `data` or `x`, `y`, and `z`.

If providing data through `x`, `y`, and `z`, `fill` can be a 1-D array that will be mapped to a colormap.

If a symbol is selected and no symbol size given, then plot3d will interpret the fourth column of the input data as symbol size. Symbols whose size is <= 0 are skipped. If no symbols are specified then the symbol code (see `style` below) must be present as last column in the input. If `style` is not used, a line connecting the data points will be drawn instead. To explicitly close polygons, use `close`. Select a fill with `fill`. If `fill` is set, `pen` will control whether the polygon outline is drawn or not. If a symbol is selected, `fill` and `pen` determine the fill and outline/no outline, respectively.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/plot3d.html.

Aliases

- D = offset
- G = fill
- I = intensity
- L = close
- N = no_clip
- Q = no_sort
- S = style
- W = pen
- Z = zvalue
- a = aspatial
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- l = label
- w = wrap
- A = straight_line
- B = frame
- C = cmap
- J = projection
- Jz = zscale
- JZ = zsize
- R = region
- V = verbose
- c = panel
- f = coltypes
- i = incols
- p = perspective
- t = transparency

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Either a data file name, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data. Optionally, use parameter `incols` to specify which columns are x, y, z, fill, and size, respectively.
- **x/y/z** (*float or 1-D arrays*) – The x, y, and z coordinates, or arrays of x, y and z coordinates of the data points.
- **size** (*1-D array*) – The size of the data points in units specified in `style`. Only valid if using `x`/`y`/`z`.
- **symbol** (*1-D array*) – The symbols of the data points. Only valid if using `x`/`y`.
- **direction** (*list of two 1-D arrays*) – If plotting vectors (using `style="V"` or `style="v"`), then should be a list of two 1-D arrays with the vector directions. These can be angle and length, azimuth and length, or x and y components, depending on the style options chosen.
- **straight_line** (`bool` | `Literal['x', 'y']`, default: `False`) – By default, line segments are drawn as straight lines in the Cartesian and polar coordinate systems, and as great circle

arcs (by resampling coarse input data along such arcs) in the geographic coordinate system. The `straight_line` parameter can control the drawing of line segments. Valid values are:

– `True`: Draw line segments as straight lines in geographic coordinate systems.

– `"x"`: Draw line segments by first along *x*, then along *y*.

– `"y"`: Draw line segments by first along *y*, then along *x*.

Here, *x* and *y* have different meanings depending on the coordinate system:

– **Cartesian** coordinate system: *x* and *y* are the X- and Y-axes.

– **Polar** coordinate system: *x* and *y* are theta and radius.

– **Geographic** coordinate system: *x* and *y* are parallels and meridians.

NOTE: The `straight_line` parameter requires constant *z*-coordinates.

Attention

There exits a bug in GMT<=6.5.0 that, in geographic coordinate systems, the meaning of *x* and *y* is reversed, i.e., *x* means meridians and *y* means parallels. The bug is fixed by upstream PR #8648.

- **cmap** (*str* | *bool*, default: `False`) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **offset** (*str*) – *dx*/*dy*[/*dz*]. Offset the plot symbol or line locations by the given amounts *dx*/*dy*[/*dz*] [Default is no offset].
- **fill** (*str*) – Set color or pattern for filling symbols or polygons [Default is no fill]. *fill* can be a 1-D array, but it is only valid if using `x`/`y` and `cmap=True` is also required.
- **intensity** (*float, bool, or 1-D array*) – Provide an *intensity* value (nominally in the -1 to +1 range) to modulate the fill color by simulating illumination. If using `intensity=True`, we will instead read *intensity* from the first data column after the symbol parameters (if given). *intensity* can also be a 1-D array to set varying intensity for symbols, but it is only valid for `x`/`y`/`z`.
- **close** (*str*) – [**+b**|**d**|**D**][**+xl**|**r**|*x0*][**+yl**|**r**|*y0*][**+p***pen*]. Force closed polygons. Full documentation is at https://docs.generic-mapping-tools.org/6.7/plot3d.html#l.
- **no_clip** (*bool or str*) – [**c**|**r**]. Do **not** clip symbols that fall outside the frame boundaries [Default plots points whose coordinates are strictly inside the frame boundaries only]. This parameter does not apply to lines and polygons which are always clipped to the plot region. For periodic (360° longitude) maps we must plot all symbols twice in case they are clipped by the repeating boundary. `no_clip=True` will turn off clipping and not plot repeating symbols. Use `no_clip="r"` to turn off clipping but retain the plotting of such repeating symbols, or use `no_clip="c"` to retain clipping but turn off plotting of repeating symbols.
- **no_sort** (*bool*) – Turn off the automatic sorting of items based on their distance from the viewer. The default is to sort the items so that items in the foreground are plotted after items in the background.
- **style** (*str*) – Plot symbols. Full documentation is at https://docs.generic-mapping-tools.org/6.7/plot3d.html#s.
- **pen** (*str*) – Set pen attributes for lines or the outline of symbols.

- **zvalue** (*str*) – *value*|*file*. Instead of specifying a symbol or polygon fill and outline color via `fill` and `pen`, give both a *value* via **zvalue** and a color lookup table via `cmap`. Alternatively, give the name of a *file* with one z-value (read from the last column) for each polygon in the input data. To apply it to the fill color, use `fill="+z"`. To apply it to the pen color, append **+z** to `pen`.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **zscale** (`float` | `str` | `None`, default: `None`)
- **zsize** (`float` | `str` | `None`, default: `None`) – Set z-axis scaling or z-axis size.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal`['none'] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **panel** (`int` | `Sequence[int]` | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:
  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.

- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **label** (*str*) – Add a legend entry for the symbol or line being plotted. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#l-full.
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `Sequence[float]` | `bool` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing). `transparency` can also be a 1-D array to set varying transparency for symbols, but this option is only valid if using `x`/`y`/`z`.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Examples

Plot a 3-D helix viewed from an azimuth of 135° and an elevation of 30°:

```
>>> import numpy as np
>>> import pygmt
>>> from pygmt.params import Axis, Frame
>>> t = np.linspace(0, 6 * np.pi, 200)
>>> fig = pygmt.Figure()
>>> fig.plot3d(
...     x=np.cos(t),
...     y=np.sin(t),
...     z=t,
...     region=[-1.2, 1.2, -1.2, 1.2, 0, 20],
...     projection="X8c",
...     zsize="6c",
...     perspective=(135, 30),
...     pen="1.5p,blue",
...     frame=Frame(
...         axes="WSneZ",
...         axis=Axis(annot=True, tick=True),
...         zaxis=Axis(annot=True, tick=True),
...     ),
... )
>>> fig.show()
```

Examples using `pygmt.Figure.plot3d`

pygmt.Figure.psconvert

`Figure.`**`psconvert`**(*prefix=None*, *dpi=None*, *gs_option=None*, *gs_path=None*, *verbose=False*, ***kwargs*)

Convert [E]PS file(s) to other formats using Ghostscript.

Converts one or more PostScript files to other formats (BMP, EPS, JPEG, PDF, PNG, PPM, TIFF) using Ghostscript.

If no input files are given, will convert the current active figure (see *pygmt.Figure*). In this case, an output name must be given using parameter `prefix`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/psconvert.html.

Aliases

- A = crop
- I = resize
- N = bb_style
- Q = anti_aliasing
- T = fmt
- C = gs_option
- E = dpi
- F = prefix
- G = gs_path
- V = verbose

Parameters

- **`crop`** (*str or bool*) – Adjust the BoundingBox and HiResBoundingBox to the minimum required by the image content. Default is `True`. Append **+u** to first remove any GMT-produced time-stamps. Append **+r** to *round* the HighResBoundingBox instead of using the `ceil` function. This is going against Adobe Law but can be useful when creating very small images where the difference of one pixel might matter. If `verbose` is used we also report the dimensions of the final illustration.
- **`gs_path`** (`str` | `None`, default: `None`) – Full path to the Ghostscript executable.
- **`gs_option`** (`str` | `Sequence[str]` | `None`, default: `None`) – Specify one or a list of custom options that will be passed on to Ghostscript as is.
- **`dpi`** (`int` | `None`, default: `None`) – Set raster resolution in dpi [Default is 720 for PDF, 300 for others]. **Note**: Ghostscript limits the final width and height pixel dimensions of a raster file to be less than or equal to 65536.
- **`prefix`** (`str` | `None`, default: `None`) – Force the output file name. By default output names are constructed using the input names as base, which are appended with an appropriate extension. Use this parameter to provide a different name, but without extension. Extension is still determined automatically.
- **`resize`** (*str*) – [**+m***margins*][**+s**[**m**]*width*[*/height*]][**+S***scale*]. Adjust the BoundingBox and HiResBoundingBox by scaling and/or adding margins. Append **+m** to specify extra margins to extend the bounding box. Give either one (uniform), two (x and y) or four (individual sides) margins; append a *dimension unit* [Default is set by PROJ_LENGTH_UNIT]. Append **+s***width* to resize the output image to exactly *width* units. The default unit is set by PROJ_LENGTH_UNIT but you can append a new unit and/or impose different width and height (**Note**: This may change the image aspect ratio). What happens here is that Ghostscript will do the re-interpolation work and the final image will retain the DPI resolution set by `dpi`. Append **+sm** to set a maximum size and the new *width* is only imposed if the original figure width exceeds it. Append */height* to also impose a maximum height in addition to the width. Alternatively, append **+S***scale* to scale the image by a constant factor.
- **`bb_style`** (*str*) – Set optional BoundingBox fill color, fading, or draw the outline of the BoundingBox. Append **+f***fade* to fade the entire plot towards black (100%) [no fading, 0].

Append **+g***paint* to paint the BoundingBox behind the illustration and append **+p**[*pen*] to draw the BoundingBox outline (append a pen or accept the default pen of `"0.25p,black,solid"`). **Note**: If both **+g** and **+f** are used then we use *paint* as the fade color instead of black. Append **+i** to enforce gray-shades by using ICC profiles.

- **anti_aliasing** (*str*) – [**g**|**p**|**t**][**1**|**2**|**4**]. Set the anti-aliasing options for **g**raphics or **t**ext. Append the size of the subsample box (1, 2, or 4) [Default is `"4"`]. [Default is no anti-aliasing (same as bits = 1).]
- **fmt** (*str*) – Set the output format, where **b** means BMP, **e** means EPS, **E** means EPS with PageSize command, **f** means PDF, **F** means multi-page PDF, **j** means JPEG, **g** means PNG, **G** means transparent PNG (untouched regions are transparent), **m** means PPM, and **t** means TIFF [Default is JPEG]. To **b**|**j**|**g**|**t**, optionally append **+m** in order to get a monochrome (grayscale) image. The EPS format can be combined with any of the other formats. For example, **ef** creates both an EPS and a PDF file. Using **F** creates a multi-page PDF file from the list of input PS or PDF files. It requires the `prefix` parameter.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

pygmt.Figure.pygmtlogo

`Figure.`**pygmtlogo**(*shape='circle'*, *theme='light'*, *wordmark='none'*, *color=True*, *width=None*, *height=None*, *position=None*, *box=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Plot the PyGMT logo.

The design of the logo is kindly provided by @sfrooti and consists of a visual and the wordmark "PyGMT". The outer shape (in Python blue) represents Earth, with compass lines (in Python yellow) surrounding the letters "GMT" (in the red used for the GMT logo). The letter "T" aligns with the needle of the compass. By default, the PyGMT logo is plotted without wordmark, with a size of 2 centimeters for the circular version of the icon.

Parameters

- **shape** (`Literal['circle', 'hexagon']`, default: `'circle'`) – Shape of the visual logo. Use `"circle"` for a circle shape [Default] or `"hexagon"` for a hexagon shape.
- **theme** (`Literal['light', 'dark']`, default: `'light'`) – Use `"light"` for light mode (i.e., a white background) [Default] and `"dark"` for dark mode (i.e., a darkgray background).
- **wordmark** (`Literal['none', 'horizontal', 'vertical']`, default: `'none'`) – Add the wordmark "PyGMT" and adjust its orientation relative to the visual. Valid values are:
  - `"none"`: no wordmark [Default].
  - `"horizontal"`: wordmark at the right side of the visual.
  - `"vertical"`: wordmark below the visual.
- **color** (`bool`, default: `True`) – `True` for a color logo, and `False` for a black and white logo.
- **position** (*Position* | `Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None`, default: `None`) – Position of the PyGMT logo on the plot. It can be specified in multiple ways:

  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot (position `(0, 0)` with anchor `"BL"`).

- **width** (`float` | `str` | `None`, default: `None`)
- **height** (`float` | `str` | `None`, default: `None`) – Width or height of the PyGMT logo. Since the aspect ratio is fixed, only one of the two can be specified. If not specified, the default size of the visual logo is set to 2 cm.
- **box** (*Box* | `bool`, default: `False`) – Draw a background box behind the logo. If set to `True`, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence[int]` | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

See also

pygmt.Figure.logo
Plot the GMT logo.

Examples

```
>>> import pygmt
```

The simplest way to plot the PyGMT logo is to call the method without any arguments.

```
>>> fig = pygmt.Figure()
>>> fig.pygmtlogo()
>>> fig.show()
```

Plot the PyGMT logo with the wordmark "PyGMT" with a height of 1 centimeter at the right side in the Bottom Right corner on an existing basemap:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-90, -70, 0, 20], projection="M10c", frame=True)
>>> fig.pygmtlogo(wordmark="horizontal", position="BR", height="1c")
>>> fig.show()
```

Examples using `pygmt.Figure.pygmtlogo`

pygmt.Figure.rose

`Figure.`**`rose`**(*data=None*, *length=None*, *azimuth=None*, *cmap=False*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot a polar histogram (rose, sector, windrose diagrams).

Takes a matrix, (length,azimuth) pairs, or a file name as input and plots windrose diagrams or polar histograms (sector diagram or rose diagram).

Must provide either `data` or `length` and `azimuth`.

Options include full circle and half circle plots. The outline of the windrose is drawn with the same color as MAP_DEFAULT_PEN.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/rose.html.

Aliases

- A = sector
- D = shift
- Em = vectors
- F = no_scale
- G = fill
- I = inquire
- JX = diameter
- L = labels
- M = vector_params
- Q = alpha
- S = norm
- T = orientation
- W = pen
- Z = scale
- b = binary
- d = nodata
- e = find
- h = header
- w = wrap
- B = frame
- C = cmap
- R = region
- V = verbose
- c = panel
- i = incols
- p = perspective
- t = transparency

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in either a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data. Use parameter incols to choose which columns are length and azimuth, respectively. If a file with only azimuths is given, use incols to indicate the single column with azimuths; then all lengths are set to unity (see scale="u" to set actual lengths to unity as well).
- **length/azimuth** (*float or 1-D arrays*) – Length and azimuth values, or arrays of length and azimuth values.
- **orientation** (*bool*) – Specify that the input data are orientation data (i.e., have a 180 degree ambiguity) instead of true 0-360 degree directions [Default is 0-360 degrees]. We compensate by counting each record twice: First as azimuth and second as azimuth +180. Ignored if region is given as (-90, 90) or (0, 180).
- **region** (Sequence[float | str] | str | None, default: None) – [*r0*, *r1*, *az0*, *az1*]. *Required if this is the first plot command.* Specify the region of interest in (*r*, *azimuth*) space. Here, *r0* is 0 and *r1* is the maximal length in units. For *az0* and *az1*, specify either (-90, 90) or (0, 180) for half circle plot or (0, 360) for full circle.
- **diameter** (*str*) – Set the diameter of the rose diagram. If not given, then we default to a diameter of 7.5 cm.
- **sector** (*float or str*) – Give the sector width in degrees for sector and rose diagram. Default 0 means windrose diagram. Append **+r** to draw rose diagram instead of sector diagram (e.g. "10+r").
- **norm** (*bool*) – Normalize input radii (or bin counts if sector is used) by the largest value so all radii (or bin counts) range from 0 to 1.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full. Remember that here *x* is the radial distance and *y* is the azimuth. The y label may be used to plot a figure caption. The scale bar length is determined by the radial gridline spacing.
- **scale** (*float or str*) – Multiply the data radii by scale. E.g., use scale=0.001 to convert your data from m to km. To exclude the radii from consideration, set them all to unity with scale="u" [Default is no scaling].
- **fill** (*str*) – Set color or pattern for filling sectors [Default is no fill].
- **cmap** (str | bool, default: False) – Give a CPT. The *r*-value for each sector is used to look-up the sector color. Cannot be used with a rose diagram.
- **pen** (*str*) – Set pen attributes for sector outline or rose plot, e.g. pen="0.5p". [Default is no outline]. To change pen used to draw vector (requires vectors) [Default is same as sector outline] use e.g. pen="v0.5p".
- **labels** (*str*) – *wlabel,elabel,slabel,nlabel*. Specify labels for the 0, 90, 180, and 270 degree marks. For full-circle plot the default is "West,East,South,North" and for half-circle the default is "90W,90E,-,0". A "-" in any entry disables that label (e.g. labels="W,E,-,N"). Use labels="" to disable all four labels. Note that the GMT_LANGUAGE setting will affect the words used.

- **no_scale** (*bool*) – Do NOT draw the scale length bar (`no_scale=True`). Default plots scale in lower right corner provided `frame` is used.
- **shift** (*bool*) – Shift sectors so that they are centered on the bin interval (e.g., first sector is centered on 0 degrees).
- **vectors** (*str*) – *mode_file*. Plot vectors showing the principal directions given in the *mode_file* file. Alternatively, specify `vectors` to compute and plot mean direction. See `vector_params` to control the vector attributes. Finally, to instead save the computed mean direction and other statistics, use `vectors="+wmode_file"`. The eight items saved to a single record are: *mean_az*, *mean_r*, *mean_resultant*, *max_r*, *scaled_mean_r*, *length_sum*, *n*, *sign@alpha*, where the last term is 0 or 1 depending on whether the mean resultant is significant at the level of confidence set via `alpha`.
- **vector_params** (*str*) – Used with `vectors` to modify vector parameters. For vector heads, append vector head size [Default is 0, i.e., a line]. See https://docs.generic-mapping-tools.org/6.7/rose.html#vector-attributes for specifying additional attributes. If `vectors` is not given and the current plot mode is to draw a windrose diagram then using `vector_params` will add vector heads to all individual directions using the supplied attributes.
- **alpha** (*float or str*) – Set the confidence level used to determine if the mean resultant is significant (i.e., Lord Rayleigh test for uniformity) [Default is `alpha=0.05`]. **Note**: The critical values are approximated [Berens, 2009] and requires at least 10 points; the critical resultants are accurate to at least 3 significant digits. For smaller data sets you should consult exact statistical tables.

  Berens, P., 2009, CircStat: A MATLAB Toolbox for Circular Statistics, *J. Stat. Software*, 31(10), 1-21, https://doi.org/10.18637/jss.v031.i10.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i|o**[*ncols*][*type*][**w**][**+l|b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l|b** to indicate that the entire data file should be read as little- or big-endian, respectively.

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.

  - **+d** to divide the input values by the factor *divisor* [Default is 1].
  - **+s** to multiple the input values by the factor *scale* [Default is 1].
  - **+o** to add the given *offset* to the input values [Default is 0].

- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).

  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:

  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Examples

Load a dataset on fracture lengths and azimuths into a pandas DataFrame.

```
>>> import pygmt
>>> from pygmt.params import Axis, Frame
>>>
>>> data = pygmt.datasets.load_sample_data(name="fractures")
```

Create a sector diagram with a diameter of 7.5 centimeters. Use a sector width of 5 degrees. Fill the sectors in bisque and outline them in brown.

```
>>> fig = pygmt.Figure()
>>> fig.rose(
...     data=data,
...     region=[0, 1, 0, 360],
...     diameter="7.5c",
...     sector="5",
...     norm=True,
...     frame=True,
...     fill="bisque",
...     pen="1p,brown",
... )
>>> fig.show()
```

Create a rose diagram by appending "+r" to the sector width, here 10 degrees. Add gridlines in steps of 0.2 and 30 degrees for the (normalized) radial and azimuthal coordinates, respectively.

```
>>> fig = pygmt.Figure()
>>> fig.rose(
...     length=data.length,
...     azimuth=data.azimuth,
...     region=[0, 1, 0, 360],
...     diameter="7.5c",
...     sector="10+r",
...     norm=True,
...     fill="bisque",
...     frame=Frame(xaxis=Axis(grid=0.2), yaxis=Axis(grid=30)),
...     pen="1p,brown",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.rose`

pygmt.Figure.savefig

`Figure.`**`savefig`**(*fname*, *transparent=False*, *crop=True*, *anti_alias=True*, *show=False*, *worldfile=False*, ***kwargs*)

Save the figure to an image file.

Supported image formats and their extensions:

Raster image formats

- BMP (`.bmp`)
- JPEG (`.jpg` or `.jpeg`)
- GeoTIFF (`.tiff`)
- PNG (`.png`)
- PPM (`.ppm`)
- TIFF (`.tif`)

Vector image formats

- EPS (`.eps`)
- PDF (`.pdf`)

Besides the above formats, you can also save the figure to a KML file (`.kml`), with a companion PNG file generated automatically. The KML file can be viewed in Google Earth.

You can pass in any keyword arguments that *pygmt.Figure.psconvert* accepts.

Parameters

- **fname** (`str` | `PathLike`) – The desired figure file name, including the extension. See the list of supported formats and their extensions above.
- **transparent** (`bool`, default: `False`) – Use a transparent background for the figure. Only valid for PNG format and the PNG file associated with KML format.
- **crop** (`bool`, default: `True`) – Crop the figure canvas (page) to the plot area.
- **anti_alias** (`bool`, default: `True`) – Use anti-aliasing when creating raster images. Ignored if creating vector images. More specifically, it passes the arguments `"t2"` and `"g2"` to the `anti_aliasing` parameter of *pygmt.Figure.psconvert*.
- **show** (`bool`, default: `False`) – Display the figure in an external viewer.
- **worldfile** (`bool`, default: `False`) – Create a companion world file for the figure. The world file will have the same name as the figure file but with different extension (e.g., `.tfw` for `.tif`). See https://en.wikipedia.org/wiki/World_file#Filename_extension for the convention of world file extensions. This parameter only works for raster image formats (except GeoTIFF).
- ****kwargs** (*dict*) – Additional keyword arguments passed to *pygmt.Figure.psconvert*. Valid parameters are `dpi`, `gs_path`, `gs_option`, `resize`, `bb_style`, and `verbose`.

Return type

`None`

pygmt.Figure.scalebar

`Figure.`**scalebar**(*length*, *height=None*, *position=None*, *scale_loc=False*, *label=False*, *label_alignment=None*, *unit=False*, *fancy=False*, *vertical=False*, *box=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*)

Add a scale bar on the plot.

Parameters

- **length** (`float` | `str`) – Length of the scale bar in kilometers. Append a suffix to specify another unit. Supported units are: **e**, **f**, **k**, **M**, **n**, and **u**. See *Distance Units* for meanings of the units.
- **height** (`float` | `str` | `None`, default: `None`) – Height of the scale bar [Default is `"5p"`].
- **position** (*Position* | `Sequence`[`float` | `str`] | `Literal`[`'TL'`, `'TC'`, `'TR'`, `'ML'`, `'MC'`, `'MR'`, `'BL'`, `'BC'`, `'BR'`] | `None`, default: `None`) – Position of the scale bar on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.
  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot with a 0.2-cm and 0.4-cm offset in the x- and y-directions, respectively.

- **scale_loc** (float | Sequence[float] | bool, default: False) – Specify the location where the map scale is calculated. It can be:
  - *slat*: Map scale is calculated for latitude *slat*.
  - (*slon*, *slat*): Map scale is calculated for latitude *slat* and longitude *slon*, which is useful for oblique projections.
  - True: Map scale is calculated for the middle of the map.
  - False: Default to the location of the reference point.
- **label** (str | bool, default: False) – Text string for the scale bar label. If False, no label is added. If True, the distance unit provided in the length parameter is used [Default is "km"]. Requires fancy=True.
- **label_alignment** (Literal['left', 'right', 'top', 'bottom'] | None, default: None) – Alignment of the scale bar label. Choose from "left", "right", "top", or "bottom" [Default is "top" for horizontal scale bars and "right" for vertical scale bars].
- **fancy** (bool, default: False) – If True, draw a "fancy" scale bar, which is a segmented bar with alternating black and white rectangles. If False, draw a plain scale bar. Doesn't work for Cartesian projections for GMT<6.7.0.
- **unit** (bool, default: False) – If True, append the unit to all distance annotations along the scale. For a plain scale, this will instead select the unit to be appended to the distance length. The unit is determined from the suffix provided to the length parameter or defaults to "km".
- **vertical** (bool, default: False) – If True, plot a vertical rather than a horizontal scale. Only supported for Cartesian projections.
- **box** (*Box* | bool, default: False) – Draw a background box behind the scale bar. If set to True, a simple rectangular box is drawn using MAP_FRAME_PEN. To customize the box appearance, pass a *pygmt.params.Box* object to control style, fill, pen, and other box properties.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].

– *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Add a scale bar to a map with specified length, position, and styling options.

```
>>> import pygmt
>>> from pygmt.params import Position
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 80, -30, 30], projection="M10c", frame=True)
>>> fig.scalebar(
...     length=1000,
...     position=Position((10, 10), cstype="mapcoords"),
...     fancy=True,
...     label="Scale",
...     unit=True,
... )
>>> fig.show()
```

Examples using `pygmt.Figure.scalebar`

pygmt.Figure.set_panel

`Figure.`**`set_panel`**(*panel=None*, *tag=None*, *verbose=False*, ***kwargs*)

Set the current subplot panel to plot on.

Before you start plotting you must first select the active subplot. **Note**: If any *projection* option is passed with the question mark **?** as scale or width when plotting subplots, then the dimensions of the plot are automatically determined by the subplot size and your region. For Cartesian plots: If you want the scale to apply equally to both dimensions then you must specify `projection="x"` [The default `projection="X"` will fill the subplot by using unequal scales].

Aliases

- C = clearance
- A = tag
- V = verbose

Parameters

- **panel** (`int` | `Sequence[int]` | `None`, default: `None`) – *index* or (*row*, *col*). Sets the current subplot until further notice. **Note**: First *row* or *col* is 0, not 1. If not given we go to the next subplot by order specified via `autolabel` in *`pygmt.Figure.subplot`*. As an alternative, you may bypass using *`pygmt.Figure.set_panel`* and instead supply the common option **panel**=(*row*, *col*) to the first plot command you issue in that subplot. GMT maintains information about the current figure and subplot. Also, you may give the one-dimensional *index* instead which starts at 0 and follows the row or column order set via `autolabel` in *`pygmt.Figure.subplot`*.

- **tag** (`str` | `None`, default: `None`) – Tag for the current subplot. It overrides the automatic tag set by the *pygmt.Figure.subplot* method. Use `tag="-"` to skip the tag for this panel.
- **clearance** (*str or list*) – [*side*]*clearance*. Reserve a space of dimension *clearance* between the margin and the subplot on the specified side, using *side* values from **w**, **e**, **s**, or **n**. The option is repeatable to set aside space on more than one side (e.g. `clearance=["w1c", "s2c"]` would set a clearance of 1 cm on west side and 2 cm on south side). Such space will be left untouched by the main plotting but can be accessed by methods that plot scales, bars, text, etc. This setting overrides the common clearances set by `clearance` in the initial *pygmt.Figure.subplot* call.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Examples using `pygmt.Figure.set_panel`

pygmt.Figure.shift_origin

`Figure.`**`shift_origin`**(*xshift=None*, *yshift=None*)

Shift the plot origin in x- and/or y-directions.

The shifts can be permanent or temporary. If used as a standalone method, the shifts are permanent and apply to all subsequent plots. If used as a context manager, the shifts are temporary and only apply to the block of code within the context manager.

1. Use as a standalone method to shift the plot origin permanently:

```
fig.shift_origin(...)
...  # Other plot commands
```

2. Use as a context manager to shift the plot origin temporarily:

```
with fig.shift_origin(...):
    ...  # Other plot commands
    ...
```

The shifts *xshift* and *yshift* in x- and y-directions are relative to the current plot origin. The default unit for shifts is centimeters (**c**) but can be changed to other units via PROJ_LENGTH_UNIT. Optionally, append the length unit (see *Dimension Units*) to the shifts.

For *xshift*, character **w** can be used, which represents the bounding box **width** of the last plotting object. The full syntax is [[±][*f*]**w**[/*d*]±]*xoff*, where optional signs, factor *f* and divisor *d* can be used to compute an offset that may be adjusted further by ±*xoff*. Assuming that the last plotting object has a width of 10 centimeters, here are some example values for *xshift*:

- `"w"`: x-shift is 10 cm
- `"w+2c"`: x-shift is 10+2=12 cm
- `"2w+3c"`: x-shift is 2*10+3=23 cm
- `"w/2-2c"`: x-shift is 10/2-2=3 cm

Similarly, for *yshift*, character **h** can be used, which is the bounding box **height** of the last plotting object.

Note: The last plotting object can be a basemap, image, logo, legend, colorbar, etc.

Parameters

- **xshift** (`float` | `str` | `None`, default: `None`) – Shift plot origin in x-direction.

- **yshift** (float | str | None, default: None) – Shift plot origin in y-direction.

Examples

Shifting the plot origin permanently:

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 5, 0, 5], projection="X5c/5c", frame=True)
>>> # Shift the plot origin in x-direction by 6 cm
>>> fig.shift_origin(xshift=6)
<contextlib._GeneratorContextManager object at ...>
>>> fig.basemap(region=[0, 7, 0, 5], projection="X7c/5c", frame=True)
>>> # Shift the plot origin in x-direction based on the previous plot width.
>>> # Here, the width is 7 cm, and xshift is 8 cm.
>>> fig.shift_origin(xshift="w+1c")
<contextlib._GeneratorContextManager object at ...>
>>> fig.basemap(region=[0, 10, 0, 5], projection="X10c/5c", frame=True)
>>> fig.show()
```

Shifting the plot origin temporarily:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 5, 0, 5], projection="X5c/5c", frame=True)
>>> # Shift the plot origin in x-direction by 6 cm temporarily. The plot origin␣
↪will
>>> # revert back to the original plot origin after the block of code is executed.
>>> with fig.shift_origin(xshift=6):
...     fig.basemap(region=[0, 5, 0, 5], projection="X5c/5c", frame=True)
>>> # Shift the plot origin in y-direction by 6 cm temporarily.
>>> with fig.shift_origin(yshift=6):
...     fig.basemap(region=[0, 5, 0, 5], projection="X5c/5c", frame=True)
>>> # Shift the plot origin in x- and y-directions by 6 cm temporarily.
>>> with fig.shift_origin(xshift=6, yshift=6):
...     fig.basemap(region=[0, 5, 0, 5], projection="X5c/5c", frame=True)
>>> fig.show()
```

Examples using `pygmt.Figure.shift_origin`

pygmt.Figure.show

Figure.**show**(*method=None*, *dpi=300*, *width=500*, *waiting=0.5*, ***kwargs*)

Display a preview of the figure.

Inserts the preview in the Jupyter notebook output if available, otherwise opens it in the default viewer for your operating system (falls back to the default web browser).

Use *pygmt.set_display* to select the default display method ("notebook", "external", "none" or None).

The method parameter allows to override the default display method for the current figure. The parameters dpi and width can be used to control the resolution and dimension of the figure in the notebook.

The external viewer can be disabled by setting the environment variable *PYGMT_USE_EXTERNAL_DISPLAY* to "false". This is useful when running tests and building the documentation to avoid popping up windows.

The external viewer does not block the current process, thus it's necessary to suspend the execution of the current process for a short while after launching the external viewer, so that the preview image won't be deleted before

the external viewer tries to open it. Set the `waiting` parameter to a larger number if the image viewer on your computer is slow to open the figure.

Parameters

- **method** (`Literal['external', 'notebook', 'none', None]`, default: `None`) – The method to display the current image preview. Choose from:
  - `"external"`: External PDF preview using the default PDF viewer
  - `"notebook"`: Inline PNG preview in the current notebook
  - `"none"`: Disable image preview
  - `None`: Reset to the default display method

  The default display method is `"external"` in Python consoles and `"notebook"` in Jupyter notebooks, but can be changed by *pygmt.set_display*.
- **dpi** (`int`, default: `300`) – The image resolution (dots per inch) in Jupyter notebooks.
- **width** (`int`, default: `500`) – The image width (in pixels) in Jupyter notebooks.
- **waiting** (`float`, default: `0.5`) – Suspend the execution of the current process for a given number of seconds after launching an external viewer. Only works if `method="external"`.
- ****kwargs** (*dict*) – Additional keyword arguments passed to *pygmt.Figure.psconvert*. Valid parameters are `gs_path`, `gs_option`, `resize`, `bb_style`, and `verbose`.

Return type

`None`

Examples using `pygmt.Figure.show`

pygmt.Figure.solar

`Figure.`**`solar`**(*terminator='day_night'*, *terminator_datetime=None*, *fill=None*, *pen=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot day-night terminators and other sunlight parameters.

This method can plot the day-night terminator, and the civil, nautical, and astronomical twilights.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/solar.html.

Aliases

- B = frame
- G = fill
- J = projection
- R = region
- T = terminator, **+d/+z**: terminator_datetime
- V = verbose
- W = pen
- c = panel
- p = perspective
- t = transparency

Parameters

- **terminator** (`Literal['astronomical', 'civil', 'day_night', 'nautical']`, default: `'day_night'`) – Set the type of terminator. Choose one of the following:
  - `"astronomical"`: Astronomical twilight
  - `"civil"`: Civil twilight
  - `"day_night"`: Day-night terminator

- `"nautical"`: Nautical twilight

Refer to https://en.wikipedia.org/wiki/Twilight for the definitions of different types of twilight.

- **terminator_datetime** (`str` | `date` | `datetime` | `datetime64` | `Timestamp` | `None`, default: `None`) – Set the date and time for the terminator calculation. It can be provided as a string or any datetime-like object recognized by `pandas.to_datetime`. The time can be specified in UTC or with a UTC offset of any precision [Default is the current UTC date and time].
- **fill** (`str` | *`Pattern`* | `None`, default: `None`) – Set color or pattern for filling terminators [Default is no fill].
- **pen** (`str` | `None`, default: `None`) – Set pen attributes for lines [Default is `"0.25p,black,solid"`].
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*`Frame`* | *`Axis`* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding

`+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot the day-night terminator at the current UTC date and time.

```
>>> import datetime
>>> import pygmt
```

```
>>> fig = pygmt.Figure()
>>> fig.coast(land="darkgreen", water="lightblue", projection="W10c", region="d")
>>> fig.solar()
>>> fig.show()
```

Plot the astronomical twilight at 8:52:18 on June 24, 1997 (time in UTC), with the night-section filled with navyblue at 75% transparency.

```
>>> import datetime
>>> # Create a datetime object at 8:52:18 on June 24, 1997 (time in UTC)
>>> date = datetime.datetime(
...     year=1997, month=6, day=24, hour=8, minute=52, second=18
... )
>>> fig = pygmt.Figure()
>>> fig.coast(land="darkgreen", water="lightblue", projection="W10c", region="d")
>>> fig.solar(
...     terminator="astronomical",
...     terminator_datetime=date,
...     # Fill the night-section with navyblue at 75% transparency
...     fill="navyblue@75",
...     pen="1p,black",  # Draw the terminator with a 1-point black line
... )
>>> fig.show()
```

Examples using `pygmt.Figure.solar`

pygmt.Figure.subplot

`Figure.`**`subplot`**(*nrows=1*, *ncols=1*, *tag=False*, *tag_position=None*, *tag_box=None*, *tag_orientation=None*, *tag_number_style=None*, *tag_font=None*, *autolabel=False*, *margins=None*, *title=None*, *projection=None*, *frame=False*, *region=None*, *verbose=False*, ***kwargs*)

Manage figure subplot configuration and selection.

This method is used to split the current figure into a rectangular layout of subplots that each may contain a single self-contained figure. Begin by defining the layout of the entire multi-panel illustration. Several parameters are available to specify the systematic layout, labeling, dimensions, and more for the subplots.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/subplot.html#synopsis-begin-mode.

Aliases

- C = clearance
- Ff = figsize
- Fs = subsize
- SC = sharex
- SR = sharey
- B = frame
- J = projection
- M = margins
- R = region
- T = title
- V = verbose

Parameters

- **nrows** (int, default: 1) – Number of vertical rows of the subplot grid.
- **ncols** (int, default: 1) – Number of horizontal columns of the subplot grid.
- **figsize** (*list*) – Specify the final figure dimensions as [*width*, *height*].
- **subsize** (*list*) – Specify the dimensions of each subplot directly as [*width*, *height*]. Note that only one of figsize or subsize can be provided at once.
- **tag** (str | bool, default: False) – Specify automatic tagging of each subplot. It can accept a number, or a letter. The number or letter can be surrounded by parentheses on any side if these should be typeset as part of the tag. This sets the tag of the first, top-left subplot and others follow sequentially. If set to True, default to "a)".

  Examples are:

  - tag="a": tags are a, b, c, …
  - tag="1": tags are 1, 2, 3, …
  - tag="a)": tags are a), b), c), …
  - tag="(c)": tags are (c), (d), (e), …
  - tag=True: same as tag="a)".
- **tag_position** (Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | *Position* | None, default: None) – Position of the subplot tag on the plot. It can be specified in two ways:

  - A *2-character justification code* for a position inside the plot, e.g., "TL" for Top Left corner inside the plot.
  - A *pygmt.params.Position* object to fully control the position and offset. **Note**: the refpoint property of the Position object must be an two-character justification code, and cstype must be set to either "inside" or "outside",

  If not specified, defaults to Top Left corner inside the plot with the offset default to ("4p", "4p"), i.e., 20% of the FONT_TAG size.
- **tag_box** (*Box* | None, default: None) – Draw a box around the subplot tag. See *pygmt.params.Box* for details on how to specify the box.

  Notes on the use of the ``Box`` class:

  - The property clearance only accept one or two values.
  - The properties inner_pen, inner_gap, radius are not supported.
- **tag_number_style** (Literal['arabic', 'roman', 'Roman'] | None, default: None) – Style of the subplot tag numbers. It can be:

  - "arabic": Arabic numerals: 1, 2, 3, … [Default].
  - "roman": Lowercase Roman numerals: i, ii, iii, …

- `"Roman"`: Uppercase Roman numerals: I, II, III, …

- **tag_orientation** (`Literal['horizontal', 'vertical'] | None`, default: `None`) – Orientation of the subplot tag. It can be:
  - `"horizontal"`: Increase tag numbers horizontally across rows [Default].
  - `"vertical"`: Increase tag numbers vertically down columns.

- **tag_font** (`str | None`, default: `None`) – Font for the subplot tag [Default to `"20p,Helvetica,black"`].

- **autolabel** (`str | bool`, default: `False`) – Specify automatic tag of each subplot.

  Deprecated since version 0.19.0: Use the parameters `tag`, `tag_position`, `tag_box`, `tag_number_style`, `tag_orientation`, and `tag_font` instead.

- **clearance** (*str or list*) – [*side*]*clearance*. Reserve a space of dimension *clearance* between the margin and the subplot on the specified side, using *side* values from **w**, **e**, **s**, or **n**; or **x** for both **w** and **e**; or **y** for both **s** and **n**. No *side* means all sides (i.e. `clearance="1c"` would set a clearance of 1 cm on all sides). The option is repeatable to set aside space on more than one side (e.g. `clearance=["w1c", "s2c"]` would set a clearance of 1 cm on west side and 2 cm on south side). Such space will be left untouched by the main plotting but can be accessed by methods that plot scales, bars, text, etc.

- **margins** (`float | str | Sequence[float | str] | None`, default: `None`) – Margin space that is added between neighboring subplots (i.e., the interior margins) in addition to the automatic space added for tick marks, annotations, and labels. The margins can be specified as either:
  - a single value (for same margin on all sides). E.g. `"5c"`.
  - a pair of values (for separate horizontal and vertical margins). E.g., `("5c", "3c")`.
  - a set of four values (for separate left, right, bottom, and top margins). E.g., `("1c", "2c", "3c", "4c")`.

  The actual gap created is always a sum of the margins for the two opposing sides (e.g., east plus west or south plus north margins) [Default is half the primary annotation font size, giving the full annotation font size as the default gap].

- **sharex** (*bool or str*) – Set subplot layout for shared x-axes. Use when all subplots in a column share a common *x*-range. If `sharex=True`, the first (i.e., **t**op) and the last (i.e., **b**ottom) rows will have *x*-annotations; use `sharex="t"` or `sharex="b"` to select only one of those two rows [both]. Append **+l** if annotated *x*-axes should have a label [none]; optionally append the label if it is the same for the entire subplot. Append **+t** to make space for subplot titles for each row; use **+tc** for top row titles only [no subplot titles].

- **sharey** (*bool or str*) – Set subplot layout for shared y-axes. Use when all subplots in a row share a common *y*-range. If `sharey=True`, the first (i.e., **l**eft) and the last (i.e., **r**ight) columns will have *y*-annotations; use `sharey="l"` or `sharey="r"` to select only one of those two columns [both]. Append **+l** if annotated *y*-axes will have a label [none]; optionally, append the label if it is the same for the entire subplot. Append **+p** to make all annotations axis-parallel [horizontal]; if not used you may have to set `clearance` to secure extra space for long horizontal annotations.

  Notes for `sharex`/`sharey`:
  - Labels and titles that depends on which row or column are specified as usual via a subplot's own `frame` setting.
  - Append **+w** to the `figsize` or `subsize` parameter to draw horizontal and vertical lines between interior panels using selected pen [no lines].

- **title** (str | None, default: None) – Set the overarching heading of the entire figure [Default is no heading]. Font is determined by FONT_HEADING. Individual subplot can have titles set by sharex/sharey or frame.
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | Literal['none'] | str | Sequence[str] | bool, default: False) – Set frame and axes attributes for the plot. It can be a bool, "none", a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If frame=True, the frame will be drawn with the default attributes. If frame="none", no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Examples

Create a figure with two subplots in one row, showing regional maps of Japan and New Zealand in the Mercator projection. The subplots are tagged as "a)" and "b)" at the Top Left corner outside each map frame. The "?" in projection means the width of each map is automatically determined from the subplot dimensions:

```
>>> import pygmt
>>> from pygmt.params import Position
>>> fig = pygmt.Figure()
>>> with fig.subplot(
...     nrows=1,
...     ncols=2,
...     figsize=("15c", "6c"),
...     tag=True,
...     tag_position=Position("TL", cstype="outside"),
...     margins="0.5c",
... ):
...     fig.coast(
...         region=[128, 146, 30, 46],
...         projection="M?",
...         land="gray",
...         water="lightblue",
...         frame=True,
...         panel=0,
...     )
...     fig.coast(
...         region=[165, 180, -48, -34],
...         projection="M?",
...         land="gray",
...         water="lightblue",
...         frame=True,
...         panel=1,
...     )
>>> fig.show()
```

Examples using `pygmt.Figure.subplot`

pygmt.Figure.ternary

`Figure.`**`ternary`**(*data*, *cmap=False*, *fill=None*, *width=None*, *pen=None*, *alabel=None*, *blabel=None*, *clabel=None*, *no_clip=False*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot data on ternary diagrams.

Reads (*a*,*b*,*c*[,*z*]) records from *data* and plots symbols at those locations on a ternary diagram. If a symbol is selected and no symbol size given, then we will interpret the fourth column of the input data as symbol size. Symbols whose *size* is <= 0 are skipped. If no symbols are specified then the symbol code (see `style` below) must be present as last column in the input. If `style` is not specified then we instead plot lines or polygons.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/ternary.html.

Aliases

- S = style
- B = frame
- C = cmap
- G = fill
- JX = width
- L = alabel/blabel/clabel
- N = no_clip
- R = region
- V = verbose
- W = pen
- c = panel
- p = perspective
- t = transparency

Parameters

- **`data`** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame`) – Pass in either a file name to an ASCII data table, a Python list, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **`width`** (`float` | `str` | `None`, default: `None`) – Set the width of the plot by passing a number followed by a *dimension unit*. Use a negative width to indicate that positive axes directions be clock-wise [Default lets the a, b, c axes be positive in a counter-clockwise direction].
- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*amin*, *amax*, *bmin*, *bmax*, *cmin*, *cmax*]. Give the min and max limits for each of the three axes **a**, **b**, and **c**.
- **`frame`** (*`Frame`* | *`Axis`* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *`pygmt.params.Frame`* or *`pygmt.params.Axis`* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full. For ternary diagrams, use *`pygmt.params.Frame`* `xaxis`, `yaxis`, and `zaxis` attributes to set the **a**, **b**, and **c** axes, respectively.
- **`cmap`** (`str` | `bool`, default: `False`) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **`alabel`** (`str` | `None`, default: `None`) – Set the label for the *a* vertex where the component is 100%. The label is placed at a distance of three times the MAP_LABEL_OFFSET setting from the corner.
- **`blabel`** (`str` | `None`, default: `None`) – Same as `alabel` but for the *b* vertex.
- **`clabel`** (`str` | `None`, default: `None`) – Same as `alabel` but for the *c* vertex.

- **style** (*str*) – *symbol*[*size*]. Plot individual symbols in a ternary diagram.
- **pen** (str | None, default: None) – Set pen attributes for the outlines of symbols or lines.
- **fill** (str | *Pattern* | None, default: None) – Set color or pattern for filling of symbols [Default is no fill].
- **no_clip** (bool, default: False) – Do not clip symbols to the ternary diagram [Default plots points whose coordinates are strictly inside the plot border].
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with zsize or zscale. [Default is at the bottom of the z-axis].

  Alternatively, set perspective=True to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding +w or +v modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.
- **transparency** (float | None, default: None) – Set transparency level, in [0-100] percent range [Default is 0, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot five data points as red circles on a ternary diagram:

```
>>> import numpy as np
>>> import pygmt
>>> from pygmt.params import Axis, Frame
>>> data = np.array(
...     [[20, 30, 50], [60, 20, 20], [33, 33, 34], [10, 70, 20], [45, 10, 45]]
... )
>>> fig = pygmt.Figure()
>>> fig.ternary(
...     data=data,
```

(continues on next page)

(continued from previous page)

```
...     region=[0, 100, 0, 100, 0, 100],
...     width="8c",
...     style="c0.2c",
...     fill="red",
...     pen="0.5p",
...     alabel="A",
...     blabel="B",
...     clabel="C",
...     frame=Frame(axis=Axis(annot=True, tick=True, grid=True)),
... )
>>> fig.show()
```

Examples using `pygmt.Figure.ternary`

pygmt.Figure.text

`Figure.`**`text`**(*textfiles=None*, *x=None*, *y=None*, *position=None*, *text=None*, *angle=False*, *font=False*, *fill=None*, *pen=None*, *justify=None*, *offset=None*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Plot or typeset text.

Must provide at least one of the following combinations as input:

- `textfiles`
- `x`/`y`, and `text`
- `position` and `text`

The text strings passed via the `text` parameter can contain ASCII characters and non-ASCII characters defined in the Adobe ISOLatin1+, Adobe Symbol, Adobe ZapfDingbats and ISO-8859-x (x can be 1-11, 13-16) encodings. Refer to *Supported Encodings and Non-ASCII Characters* for the full list of supported non-ASCII characters.

For typesetting one or more paragraphs of text, see *`pygmt.Figure.paragraph`*.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/text.html.

Aliases

- C = clearance
- a = aspatial
- e = find
- h = header
- it = use_word
- w = wrap
- B = frame
- D = offset
- F = **+a**: angle, **+c**: position, **+j**: justify, **+f**: font
- G = fill
- J = projection
- N = no_clip
- R = region
- V = verbose
- W = pen
- c = panel
- f = coltypes
- p = perspective
- t = transparency

Parameters

- **`textfiles`** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – A file name or a list of file names containing one or more records. Each record has the following columns:
  - *x*: X coordinate or longitude
  - *y*: Y coordinate or latitude

  - *angle*: Angle in degrees counter-clockwise from horizontal
  - *font*: Text size, font, and color
  - *justify*: *2-character justification code*
  - *text*: The text string to typeset

  The *angle*, *font*, and *justify* columns are optional and can be set by using the `angle`, `font`, and `justify` parameters, respectively. If these parameters are set to `True`, then the corresponding columns must be present in the input file(s) and the columns must be in the order mentioned above.

- **x/y** (*float or 1-D arrays*) – The x and y coordinates, or an array of x and y coordinates to plot the text.
- **position** (`Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None`, default: `None`) – Set reference point on the plot for the text by using x, y coordinates extracted from `region` instead of providing them through `x`/`y`. Specify with a *2-character justification code*. For example, `position="TL"` plots the text at the Top Left corner of the plot.
- **text** (`str | Sequence[str] | ndarray | StringArray | None`, default: `None`) – The text string, or an array of strings to plot.
- **angle** (`float | Sequence[float] | bool`, default: `False`) – Set the angle measured in degrees counter-clockwise from horizontal (e.g. 30 sets the text at 30 degrees). If no angle is explicitly given (i.e. `angle=True`) then the input to `textfiles` must have this as a column.
- **font** (`str | Sequence[str] | ndarray | StringArray | bool`, default: `False`) – Set the font specification with format *size*,*font*,*color* where *size* is text size in points, *font* is the font to use, and *color* sets the font color. For example, `font="12p,Helvetica-Bold,red"` selects a 12p, red, Helvetica-Bold font. If no font info is explicitly given (i.e. `font=True`), then the input to `textfiles` must have this information in one of its columns.
- **justify** (`bool | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | Sequence[Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR']] | None`, default: `None`) – Set the alignment which refers to the part of the text string that will be mapped onto the (x, y) point. Choose a *2-character justification code*, e.g., **BL** for Bottom Left. If no justification is explicitly given (i.e. `justify=True`), then the input to `textfiles` must have this as a column.
- **clearance** (*str*) – [*dx/dy*][**+to**|**O**|**c**|**C**]. Adjust the clearance between the text and the surrounding box [Default is 15% of the font size]. Only used if `pen` or `fill` are specified. Append a *dimension unit*; if not given we consult PROJ_LENGTH_UNIT. Alternatively, append % for a percentage of the font size. Optionally, use modifier **+t** to set the shape of the text box when using `fill` and/or `pen`. Append lowercase **o** to get a straight rectangle [Default is **o**]. Append uppercase **O** to get a rounded rectangle. In paragraph mode (*paragraph*) you can also append lowercase **c** to get a concave rectangle or append uppercase **C** to get a convex rectangle.
- **fill** (`str | Pattern | None`, default: `None`) – Set color for filling text boxes [Default is no fill].
- **pen** (`str | None`, default: `None`) – Set the pen used to draw a rectangle around the text string (see `clearance`) [Default is `"0.25p,black,solid"`].
- **offset** (`Sequence[float | str] | str | None`, default: `None`) – (*dx*, *dy*) or [**j**|**J**]*dx*[/*dy*][**+v**[*pen*]]. Offset the text from the projected (x, y) point by (*dx*, *dy*) [Default is (0, 0)]. If *dy* is not specified then it is set equal to *dx*. Use **j** to offset the text away from the point instead (i.e., the text justification will determine the direction of the shift). Using **J** will shorten diagonal offsets at corners by sqrt(2). Optionally, append **+v** which will draw a line from the original point to the shifted point; append a pen to change the attributes for this line.

- **no_clip** (bool, default: False) – Do **not** clip text at the frame boundaries [Default is False].
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest. *Required if this is the first plot command.*
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **panel** (int | Sequence[int] | bool, default: False) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - True to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **use_word** (*int*) – Select a specific word from the trailing text, with the first word being 0 [Default is the entire trailing text]. No numerical columns can be specified.
- **perspective** (float | Sequence[float] | str | bool, default: False) – Select perspective view and set the azimuth and elevation of the viewpoint.

Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).

- *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
- *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
- *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **`transparency`** (`float` | `Sequence`[`float`] | `bool` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing). `transparency` can also be a 1-D array to set varying transparency for texts, but this option is only valid if using `x`/`y` and `text`.
- **`wrap`** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

See also

pygmt.Figure.paragraph
Typeset one or multiple paragraphs.

Examples

```
>>> import pygmt
```

Plot the text "PyGMT" using the font AvantGarde-Book in blue and 20 points. Place the text at Middle Center. Add a box with a 1-point thick black outline.

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-5, 5, -5, 5], projection="X5c/2c", frame=True)
>>> fig.text(
...     text="PyGMT", position="MC", font="20p,AvantGarde-Book,blue", pen=
↪"1p,black"
... )
>>> fig.show()
```

Place the text using x- and y-coordinates with an offset of 1 centimeter in both x- and y directions. Add a box with tan fill.

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-5, 5, -5, 5], projection="X5c/2c", frame=True)
>>> fig.text(
...     text="PyGMT",
...     x=0,
...     y=-3,
...     offset=(1, 1),
...     font="20p,AvantGarde-Book,blue",
...     fill="tan",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.text`

pygmt.Figure.tilemap

Figure.**tilemap**(*region*, *zoom='auto'*, *source=None*, *lonlat=True*, *wait=0*, *max_retries=2*, *zoom_adjust=None*, *headers=None*, *monochrome=False*, *no_clip=False*, *projection=None*, *frame=False*, *verbose=False*, *panel=False*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot an XYZ tile map.

This method loads XYZ tile maps from a tile server or local file using *pygmt.datasets.load_tile_map* into a georeferenced form, and plots the tiles as a basemap or overlay using *pygmt.Figure.grdimage*.

Note: By default, standard web map tiles served in a Spherical Mercator (EPSG:3857) Cartesian format will be reprojected to a geographic coordinate reference system (OGC:CRS84) and plotted with longitude/latitude bounds when `lonlat=True`. If reprojection is not desired, please set `lonlat=False` and provide Spherical Mercator (EPSG:3857) coordinates to the `region` parameter.

Aliases

- E = dpi
- I = shading
- Q = nan_transparent
- B = frame
- J = projection
- M = monochrome
- N = no_clip
- V = verbose
- c = panel
- p = perspective
- t = transparency

Parameters

- **region** (Sequence[float]) – The bounding box of the map in the form of a list [*xmin*, *xmax*, *ymin*, *ymax*]. These coordinates should be in longitude/latitude if `lonlat=True` or Spherical Mercator (EPSG:3857) if `lonlat=False`.
- **zoom** (int | Literal['auto'], default: 'auto') – Level of detail. Higher levels (e.g. 22) mean a zoom level closer to the Earth's surface, with more tiles covering a smaller geographical

area and thus more detail. Lower levels (e.g. `0`) mean a zoom level further from the Earth's surface, with less tiles covering a larger geographical area and thus less detail. Default is `"auto"` to automatically determine the zoom level based on the bounding box region extent.

> **Note**
>
> The maximum possible zoom level may be smaller than `22`, and depends on what is supported by the chosen web tile provider source.

- **source** (`TileProvider` | `str` | `None`, default: `None`) – The tile source: web tile provider or path to a local file. Provide either:
  - A web tile provider in the form of a `xyzservices.TileProvider` object. See Contextily providers for a list of tile providers. Default is `xyzservices.providers.OpenStreetMap.HOT`, i.e. OpenStreetMap Humanitarian web tiles.
  - A web tile provider in the form of a URL. The placeholders for the XYZ in the URL need to be `{x}`, `{y}`, `{z}`, respectively. E.g. `https://tile.openstreetmap.org/{z}/{x}/{y}.png`.
  - A local file path. The file is read with rasterio and all bands are loaded into the basemap. See Working with local files.

> **Important**
>
> Tiles are assumed to be in the Spherical Mercator projection (EPSG:3857).

- **lonlat** (`bool`, default: `True`) – If `False`, coordinates in `region` are assumed to be Spherical Mercator as opposed to longitude/latitude.
- **wait** (`int`, default: `0`) – If the tile API is rate-limited, the number of seconds to wait between a failed request and the next try.
- **max_retries** (`int`, default: `2`) – Total number of rejected requests allowed before contextily will stop trying to fetch more tiles from a rate-limited API.
- **zoom_adjust** (`int` | `None`, default: `None`) – The amount to adjust a chosen zoom level if it is chosen automatically. Values outside of -1 to 1 are not recommended as they can lead to slow execution.
- **headers** (`dict[str, str]` | `None`, default: `None`) – HTTP headers to include with requests to the tile server. This can be useful for authentication or to set a custom User-Agent. When supported by `contextily` (>=1.7.0), PyGMT sets a default `User-Agent` header like `PyGMT/X.Y.Z (+https://www.pygmt.org)`.

> **Note**
>
> Requires `contextily>=1.7.0`.

- **kwargs** (*dict*) – Extra keyword arguments to pass to *pygmt.Figure.grdimage*.

Examples

```
>>> import contextily
>>> import pygmt
```

Plot a tile map of Honolulu, Hawaii using the default OpenStreetMap Humanitarian web tiles:

```
>>> fig = pygmt.Figure()
>>> fig.tilemap(
...     region=[-157.84, -157.8, 21.255, 21.285], projection="M12c", frame=True
... )
>>> fig.show()
```

Use the Esri World Imagery satellite tiles provided via the contextily library at a specified zoom level:

```
>>> fig = pygmt.Figure()
>>> fig.tilemap(
...     region=[-157.84, -157.8, 21.255, 21.285],
...     projection="M12c",
...     zoom=14,
...     source=contextily.providers.Esri.WorldImagery,
...     frame=True,
... )
>>> fig.show()
```

Examples using `pygmt.Figure.tilemap`

pygmt.Figure.timestamp

Figure.**timestamp**(*text=None*, *label=None*, *justify='BL'*, *offset=('-54p', '-54p')*, *font='Helvetica,black'*, *timefmt='%Y %b %d %H:%M:%S'*)

Plot the GMT timestamp logo.

Add the GMT timestamp logo with an optional label at the bottom-left corner relative to the current plot origin, with an offset of `("-54p", "-54p")`. The timestamp will be in the locale set by the environment variable *TZ* (generally local time but can be changed via `os.environ["TZ"]`) and its format is controlled by the `timefmt` parameter. It can also be replaced with any custom text string using the `text` parameter.

Parameters

- **text** (`str` | `None`, default: `None`) – If `None`, the current date and time are shown in the GMT timestamp logo. Set this parameter to replace the timestamp string with a custom text string instead. The text must be no longer than 64 characters; longer strings are truncated and raise a `RuntimeWarning`.
- **label** (`str` | `None`, default: `None`) – The text string shown after the GMT timestamp logo.
- **justify** (`Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR']`, default: `'BL'`) – Specify a *2-character justification code* for the timestamp box relative to the current

plot origin. The default is the Bottom Left (`"BL"`) corner.

- **offset** (`float` | `str` | `Sequence`[`float` | `str`], default: `('-54p', '-54p')`) – *offset* or (*offset_x*, *offset_y*). Offset the anchor point of the timestamp box by *offset_x* and *offset_y*. If a single value *offset* is given, *offset_y* = *offset_x* = *offset*.
- **font** (`str`, default: `'Helvetica,black'`) – Font of the timestamp and the optional label. Since the GMT logo has a fixed height, the font sizes are fixed to be 8-point for the timestamp and 7-point for the label.
- **timefmt** (`str`, default: `'%Y %b %d %H:%M:%S'`) – Format string for the timestamp. The format string is parsed by the C function `strftime`, so that virtually any text can be used (even not containing any time information).

Examples

Plot the GMT timestamp logo.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.timestamp()
>>> fig.show()
```

Plot the GMT timestamp logo with a custom label.

```
>>> fig = pygmt.Figure()
>>> fig.timestamp(label="Powered by PyGMT")
>>> fig.show()
```

Examples using `pygmt.Figure.timestamp`

pygmt.Figure.velo

`Figure.`**velo**(*data=None*, *cmap=False*, *no_clip=False*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *perspective=False*, *transparency=None*, ***kwargs*)

Plot velocity vectors, strain crosses, anisotropy bars, and wedges.

Reads data values from files, `numpy.ndarray` or `pandas.DataFrame` and plots the selected geodesy symbol on a plot. You may choose from velocity vectors and their uncertainties, rotational wedges and their uncertainties, anisotropy bars, or strain crosses. Symbol fills or their outlines may be colored based on constant parameters or via color lookup tables.

Must provide `data` and `spec`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/supplements/geodesy/velo.html.

Aliases

- A = vector
- D = rescale
- E = uncertainty_fill
- G = fill
- H = scale
- I = shading
- L = line
- S = spec
- W = pen
- Z = zvalue
- d = nodata
- e = find
- h = header
- B = frame
- C = cmap
- J = projection
- N = no_clip
- R = region
- V = verbose
- c = panel
- i = incols
- p = perspective
- t = transparency

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in either a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data. Note that text columns are only supported with file or pandas.DataFrame inputs.
- **spec** (*str*) – Select the meaning of the columns in the data file and the figure to be plotted. In all cases, the scales are in data units per length unit and sizes are in length units (default length unit is controlled by PROJ_LENGTH_UNIT unless a *dimension unit* is appended).
  - **e**[*velscale*/]*confidence*[**+f***font*]

    Velocity ellipses in (N,E) convention. The *velscale* sets the scaling of the velocity arrows. If *velscale* is not given then we read it from the data file as an extra column. The *confidence* sets the 2-dimensional confidence limit for the ellipse, e.g. 0.95 for 95% confidence ellipse. Use **+f** to set the font and size of the text [Default is 9p,Helvetica,black]; give **+f**0 to deactivate labeling. The arrow will be drawn with the pen attributes specified by the pen parameter and the arrow-head can be colored via fill. The ellipse will be filled with the color or pattern specified by the uncertainty_fill parameter [Default is transparent], and its outline will be drawn if line is selected using the pen selected (by pen if not given by line). Parameters are expected to be in the following columns:

    * **1**,**2**: longitude, latitude of station
    * **3**,**4**: eastward, northward velocity
    * **5**,**6**: uncertainty of eastward, northward velocities (1-sigma)
    * **7**: correlation between eastward and northward components
    * **Trailing text**: name of station (optional)
  - **n**[*barscale*]

    Anisotropy bars. *barscale* sets the scaling of the bars. If *barscale* is not given then we read it from the data file as an extra column. Parameters are expected to be in the following columns:

    * **1**,**2**: longitude, latitude of station
    * **3**,**4**: eastward, northward components of anisotropy vector
  - **r**[*velscale*/]*confidence*[**+f***font*]

    Velocity ellipses in rotated convention. The *velscale* sets the scaling of the velocity arrows. If *velscale* is not given then we read it from the data file as an extra column. The *confidence* sets the 2-dimensional confidence limit for the ellipse, e.g. 0.95 for 95% confidence ellipse. Use **+f** to set the font and size of the text [Default is 9p,Helvetica,black]; give **+f**0 to deactivate labeling. The arrow will be drawn with the pen attributes specified by the pen parameter and the arrow-head can be colored via fill. The ellipse will be filled with the color or pattern specified by the uncertainty_fill parameter [Default is transparent], and its outline will be drawn if line is selected using the pen selected (by pen if not given by line). Parameters are expected to be in the following columns:

    * **1**,**2**: longitude, latitude of station
    * **3**,**4**: eastward, northward velocity
    * **5**,**6**: semi-major, semi-minor axes
    * **7**: counter-clockwise angle, in degrees, from horizontal axis to major axis of ellipse.

* **Trailing text**: name of station (optional)

– **w**[*wedgescale/*]*wedgemag*

Rotational wedges. The *wedgescale* sets the size of the wedges. If *wedgescale* is not given then we read it from the data file as an extra column. Rotation values are multiplied by *wedgemag* before plotting. For example, setting *wedgemag* to 1.e7 works well for rotations of the order of 100 nanoradians/yr. Use `fill` to set the fill color or pattern for the wedge, and `uncertainty_fill` to set the color or pattern for the uncertainty. Parameters are expected to be in the following columns:

* **1**,**2**: longitude, latitude of station
* **3**: rotation in radians
* **4**: rotation uncertainty in radians

– **x**[*cross_scale*]

Strain crosses. The *cross_scale* sets the size of the cross. If *cross_scale* is not given then we read it from the data file as an extra column. Parameters are expected to be in the following columns:

* **1**,**2**: longitude, latitude of station
* **3**: eps1, the most extensional eigenvalue of strain tensor, with extension taken positive.
* **4**: eps2, the most compressional eigenvalue of strain tensor, with extension taken positive.
* **5**: azimuth of eps2 in degrees CW from North.

- **vector** (*bool or str*) – Modify vector parameters. For vector heads, append vector head *size* [Default is 9p]. See https://docs.generic-mapping-tools.org/6.7/supplements/geodesy/velo.html#vector-attributes for specifying additional attributes.
- **cmap** (`str` | `bool`, default: `False`) – File name of a CPT file or a series of comma-separated colors (e.g., *color1*,*color2*,*color3*) to build a linear continuous CPT from those colors automatically.
- **rescale** (*str*) – Can be used to rescale the uncertainties of velocities (`spec="e"` and `spec="r"`) and rotations (`spec="w"`). Can be combined with the `confidence` variable.
- **uncertainty_fill** (*str*) – Set color or pattern for filling uncertainty wedges (`spec="w"`) or velocity error ellipses (`spec="e"` or `spec="r"`). If `uncertainty_fill` is not specified, the uncertainty regions will be transparent. **Note**: Using `cmap` and `zvalue="+e"` will update the uncertainty fill color based on the selected measure in `zvalue` [Default is magnitude error]. More details at https://docs.generic-mapping-tools.org/6.7/reference/features.html#gfill-attrib.
- **fill** (*str*) – Set color or pattern for filling symbols [Default is no fill]. **Note**: Using `cmap` (and optionally `zvalue`) will update the symbol fill color based on the selected measure in `zvalue` [Default is magnitude]. More details at https://docs.generic-mapping-tools.org/6.7/reference/features.html#gfill-attrib.
- **scale** (*float or bool*) – [*scale*]. Scale symbol sizes and pen widths on a per-record basis using the *scale* read from the data set, given as the first column after the (optional) *z* and *size* columns [Default is no scaling]. The symbol size is either provided by `spec` or via the input *size* column. Alternatively, append a constant *scale* that should be used instead of reading a scale column.
- **shading** (*float or bool*) – *intens*. Use the supplied *intens* value (nominally in the -1 to +1 range) to modulate the symbol fill color by simulating illumination [Default is none]. If *intens*

is not provided we will instead read the intensity from an extra data column after the required input columns determined by `spec`.

- **line** (*str*) – [*pen*[**+c**[**f**|**l**]]]. Draw lines. Ellipses and rotational wedges will have their outlines drawn using the current pen (see `pen`). Alternatively, append a separate pen to use for the error outlines. If the modifier **+cl** is appended then the color of the pen is updated from the CPT (see `cmap`). If instead modifier **+cf** is appended then the color from the cpt file is applied to error fill only [Default]. Use just **+c** to set both pen and fill color.
- **no_clip** (`bool`, default: `False`) – Do **not** skip symbols that fall outside the frame boundaries [Default is `False`, i.e., plot symbols inside the frame boundaries only].
- **pen** (*str*) – [*pen*][**+c**[**f**|**l**]]. Set pen attributes for velocity arrows, ellipse circumference and fault plane edges [Default is `"0.25p,black,solid"`]. If the modifier **+cl** is appended then the color of the pen is updated from the CPT (see `cmap`). If instead modifier **+cf** is appended then the color from the cpt file is applied to symbol fill only [Default]. Use just **+c** to set both pen and fill color.
- **zvalue** (*str*) – [**m**|**e**|**n**|**u**][**+e**]. Select the quantity that will be used with the CPT given via `cmap` to set the fill color. Choose from **m**agnitude (vector magnitude or rotation magnitude), **e**ast-west velocity, **n**orth-south velocity, or **u** (user-supplied data column after the required columns). To instead use the corresponding error estimates (i.e., vector or rotation uncertainty) to lookup the color and paint the error ellipse or wedge instead, append **+e**.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal`[`'none'`] | `str` | `Sequence`[`str`] | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to

instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.

- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **perspective** (`float` | `Sequence`[`float`] | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).
  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).

Examples

Plot mean velocity arrows with confidence ellipses. Fill the ellipses in bisque and outline them in darkbrown. Color the arrows in magenta and fill their heads in purple. Add the labels with fonsize 10. The velocity arrows are scaled by 0.2 and the 39% confidence limit will give an ellipse which fits inside a rectangle of dimension east_sigma by north_sigma.

```
>>> import pandas as pd
>>> import pygmt
>>>
>>> # Setup some dummy data as pandas DataFrame with two entries.
>>> df = pd.DataFrame(
...     data={
...         "x": [2, -2],
...         "y": [0, -1],
...         "east_velocity": [4, 6],
...         "north_velocity": [6, 2],
...         "east_sigma": [3, 4],
...         "north_sigma": [3, 1],
...         "correlation_EN": [0.5, -0.5],
...         "SITE": ["4x6", "6x2"],
...     }
... )
>>>
>>> fig = pygmt.Figure()
>>> fig.velo(
...     data=df,
...     region=[-5, 5, -5, 5],
...     projection="X10c",
...     frame=True,
...     spec="e0.2/0.39+f10",
...     uncertainty_fill="bisque",
...     pen="1p,magenta",
...     line="darkbrown",
...     vector="0.5c+p1p+e+gpurple",
... )
>>>
>>> fig.show()
```

Examples using `pygmt.Figure.velo`

pygmt.Figure.vlines

`Figure.`**`vlines`**(*x*, *ymin=None*, *ymax=None*, *pen=None*, *label=None*, *no_clip=False*, *panel=False*, *perspective=None*)

Plot one or multiple vertical line(s).

This method is a high-level wrapper around *pygmt.Figure.plot* that focuses on plotting vertical lines at X-coordinates specified by the `x` parameter. The `x` parameter can be a single value (for a single vertical line) or a sequence of values (for multiple vertical lines).

By default, the Y-coordinates of the start and end points of the lines are set to be the Y-limits of the current plot, but this can be overridden by specifying the `ymin` and `ymax` parameters. `ymin` and `ymax` can be either a single value or a sequence of values. If a single value is provided, it is applied to all lines. If a sequence is provided, the length of `ymin` and `ymax` must match the length of `x`.

The term "vertical" lines can be interpreted differently in different coordinate systems:

- **Cartesian**: lines are plotted as straight lines.
- **Polar**: lines are plotted as straight lines along a constant azimuth.
- **Geographic**: lines are plotted as arcs along meridians (i.e., constant longitude).

Parameters

- **x** (`float` | `Sequence`[`float`]) – X-coordinates to plot the lines. It can be a single value (for a single line) or a sequence of values (for multiple lines).
- **ymin/ymax** – Y-coordinates of the start/end point(s) of the line(s). If `None`, defaults to the Y-limits of the current plot. `ymin` and `ymax` can either be a single value or a sequence of values. If a single value is provided, it is applied to all lines. If a sequence is provided, the length of `ymin` and `ymax` must match the length of `x`.
- **pen** (`str` | `None`, default: `None`) – Pen attributes for the line(s), in the format of *width,color,style*.
- **label** (`str` | `None`, default: `None`) – Label for the line(s), to be displayed in the legend.
- **no_clip** (`bool`, default: `False`) – Do **not** clip lines outside the plot region. Only makes sense in the Cartesian coordinate system. [Default is `False` to clip lines at the plot region.]
- **panel** (`int` | `Sequence`[`int`] | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *Figure.subplot* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *Figure.subplot* method. *row*, *col* and *index* all start at 0.
- **perspective** (`str` | `bool` | `None`, default: `None`) – Select perspective view and set the azimuth and elevation angle of the viewpoint. Refer to *pygmt.Figure.plot* for details.

Examples

Plot several vertical lines at different x-coordinates with varying y-limits and pen styles. Lines are labeled and a legend is displayed.

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c/10c", frame=True)
>>> fig.vlines(x=1, pen="1p,black", label="Line at x=1")
```

(continues on next page)

(continued from previous page)

```
>>> fig.vlines(x=2, ymin=2, ymax=8, pen="1p,red,-", label="Line at x=2")
>>> fig.vlines(x=[3, 4], ymin=3, ymax=7, pen="1p,black,.", label="Lines at x=3,4")
>>> fig.vlines(x=[5, 6], ymin=4, ymax=9, pen="1p,red", label="Lines at x=5,6")
>>> fig.vlines(
...     x=[7, 8], ymin=[0, 1], ymax=[7, 8], pen="1p,blue", label="Lines at x=7,8"
... )
>>> fig.legend()
>>> fig.show()
```

Examples using `pygmt.Figure.vlines`

pygmt.Figure.wiggle

`Figure.`**`wiggle`**(*data=None*, *x=None*, *y=None*, *z=None*, *position=None*, *length=None*, *label_alignment=None*, *positive_fill=None*, *negative_fill=None*, *projection=None*, *region=None*, *frame=False*, *verbose=False*, *panel=False*, *incols=None*, *label=None*, *perspective=False*, *transparency=None*, *coltypes=None*, ***kwargs*)

Plot z=f(x,y) anomalies along tracks.

Takes a matrix, (x, y, z) triplets, or a file name as input and plots z as a function of distance along track.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/wiggle.html.

Aliases

- T = track
- W = pen
- Z = scale
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- w = wrap
- B = frame
- D = **+w**: length, **+l**: label, **+a**: label_alignment
- G = **+p**: positive_fill, **+n**: negative_fill
- J = projection
- R = region
- V = verbose
- c = panel
- f = coltypes
- i = incols
- p = perspective
- t = transparency

Parameters

- **x/y/z** (*1-D arrays*) – The arrays of x and y coordinates and z data points.
- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in either a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data. Use parameter `incols` to choose which columns are x, y, z, respectively.
- **position** (*Position* | `Sequence`[`float` | `str`] | `Literal`['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | `None`, default: `None`) – Position of the vertical scale on the plot. It can be specified in multiple ways:
  - A *pygmt.params.Position* object to fully control the reference point, anchor point, and offset.
  - A sequence of two values representing the x- and y-coordinates in plot coordinates, e.g., `(1, 2)` or `("1c", "2c")`.

  - A *2-character justification code* for a position inside the plot, e.g., `"TL"` for Top Left corner inside the plot.

  If not specified, defaults to the Bottom Left corner of the plot with a 0.2-cm offset.

- **length** (`float` | `str` | `None`, default: `None`) – Length of the vertical scale bar in data (z) units.
- **label** (`str` | `None`, default: `None`) – Set the z unit label that is used in the scale label [Default is no unit].
- **label_alignment** (`Literal['left', 'right']` | `None`, default: `None`) – Set the alignment of the scale label. Choose from `"left"` or `"right"` [Default is `"left"`].
- **scale** (*str or float*) – Give anomaly scale in data-units/dimension-unit. Append a *dimension unit*; if no unit is given we use the default unit that is controlled by PROJ_LENGTH_UNIT.
- **positive_fill** (`str` | *Pattern* | `None`, default: `None`) – Set color or pattern for filling positive wiggles [Default is no fill].
- **negative_fill** (`str` | *Pattern* | `None`, default: `None`) – Set color or pattern for filling negative wiggles [Default is no fill].
- **track** (*str*) – Draw track [Default is no track]. Append pen attributes to use [Default is `"0.25p,black,solid"`].
- **pen** (*str*) – Specify outline pen attributes [Default is no outline].
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **frame** (*Frame* | *Axis* | `Literal['none']` | `str` | `Sequence[str]` | `bool`, default: `False`) – Set frame and axes attributes for the plot. It can be a bool, `"none"`, a *pygmt.params.Frame* or *pygmt.params.Axis* object. Raw GMT strings or sequences of strings are also supported for backward compatibility. If `frame=True`, the frame will be drawn with the default attributes. If `frame="none"`, no frame will be drawn. Use a *pygmt.params.Frame* or *pygmt.params.Axis* object for more control over the attributes of the frame and axes. A tutorial is available at *frame and axes attributes*. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#b-full.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)

- **L**: uint64_t (8-byte unsigned int)
- **f**: 4-byte single-precision float
- **d**: 8-byte double-precision float
- **x**: use to skip *ncols* anywhere in the record

For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:

- **w** after any item to force byte-swapping.
- **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **panel** (`int` | `Sequence[int]` | `bool`, default: `False`) – Select a specific subplot panel. Only allowed when used in *`Figure.subplot`* mode.
  - `True` to advance to the next panel in the selected order.
  - *index* to specify the index of the desired panel.
  - (*row*, *col*) to specify the row and column of the desired panel.

  The panel order is determined by the *`Figure.subplot`* method. *row*, *col* and *index* all start at 0.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.

  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:

  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.

- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:

  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.

- **incols** (`int` | `str` | `Sequence[int | str]` | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].

  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    * **+l** to take the *log10* of the input values.
    * **+d** to divide the input values by the factor *divisor* [Default is 1].
    * **+s** to multiple the input values by the factor *scale* [Default is 1].
    * **+o** to add the given *offset* to the input values [Default is 0].

- **perspective** (`float` | `Sequence[float]` | `str` | `bool`, default: `False`) – Select perspective view and set the azimuth and elevation of the viewpoint.

  Accepts a single value or a sequence of two or three values: *azimuth*, (*azimuth*, *elevation*), or (*azimuth*, *elevation*, *zlevel*).

  - *azimuth*: Azimuth angle of the viewpoint in degrees [Default is 180, i.e., looking from south to north].
  - *elevation*: Elevation angle of the viewpoint above the horizon [Default is 90, i.e., looking straight down at nadir].
  - *zlevel*: Z-level at which 2-D elements (e.g., the plot frame) are drawn. Only applied when used together with `zsize` or `zscale`. [Default is at the bottom of the z-axis].

  Alternatively, set `perspective=True` to reuse the perspective setting from the previous plotting method, or pass a string following the full GMT syntax for finer control (e.g., adding `+w` or `+v` modifiers to select an axis location other than the plot origin). See https://docs.generic-mapping-tools.org/6.7/gmt.html#perspective-full for details.

- **transparency** (`float` | `None`, default: `None`) – Set transparency level, in [0-100] percent range [Default is `0`, i.e., opaque]. Only visible when PDF or raster format output is selected. Only the PNG format selection adds a transparency layer in the image (for further processing).
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Examples

Draw a wiggle plot with positive and negative fills, a track, and a vertical scale.

```
>>> import numpy as np
>>> import pygmt
>>>
>>> x = np.arange(-7, 7, 0.01)
>>> y = np.zeros(x.size)
>>> z = np.exp(-((x / 2) ** 2)) * np.sin(2 * np.pi * x)
>>>
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[-8, 10, -1, 1], projection="X10c/5c", frame=True)
>>> fig.wiggle(
...     x=x,
...     y=y,
...     z=z,
...     scale="0.5c",
...     length=1,
...     position="RM",
...     pen="0.5p,",
```

(continues on next page)

(continued from previous page)

```
...     positive_fill="lightorange",
...     negative_fill="lightgray",
...     track="0.5p,red",
... )
>>> fig.show()
```

Examples using `pygmt.Figure.wiggle`

Examples using `pygmt.Figure`

11.1.2 Plotting figure elements

Figure.basemap([projection, zscale, zsize, ...])	Plot base maps and frames.
Figure.choropleth(data, column[, cmap, ...])	Plot a choropleth map.
Figure.coast([resolution, land, water, ...])	Plot continents, countries, shorelines, rivers, and borders.
Figure.colorbar([cmap, position, length, ...])	Plot gray scale or color scale bar.
Figure.directional_rose([position, width, ...])	Add a directional rose on the map.
Figure.hlines(y[, xmin, xmax, pen, label, ...])	Plot one or multiple horizontal line(s).
Figure.inset([position, width, height, box, ...])	Manage figure inset setup and completion.
Figure.legend([spec, position, width, ...])	Plot a legend.
Figure.logo([position, width, height, box, ...])	Plot the GMT logo.
Figure.magnetic_rose([position, width, ...])	Add a magnetic rose to the map.
Figure.paragraph(x, y, text, parwidth, ...)	Typeset one or multiple paragraphs.
Figure.pygmtlogo([shape, theme, wordmark, ...])	Plot the PyGMT logo.
Figure.scalebar(length[, height, position, ...])	Add a scale bar on the plot.
Figure.solar([terminator, ...])	Plot day-night terminators and other sunlight parameters.
Figure.text([textfiles, x, y, position, ...])	Plot or typeset text.
Figure.timestamp([text, label, justify, ...])	Plot the GMT timestamp logo.
Figure.vlines(x[, ymin, ymax, pen, label, ...])	Plot one or multiple vertical line(s).

11.1.3 Plotting tabular data

Figure.contour([data, x, y, z, no_clip, ...])	Contour table data by direct triangulation.
Figure.fill_between(x, y[, y2, x2, fill, ...])	Fill the area between two horizontal curves.
Figure.histogram(data[, weights, histtype, ...])	Calculate and plot histograms.
Figure.meca(spec, scale[, convention, ...])	Plot focal mechanisms.
Figure.plot([data, x, y, size, symbol, ...])	Plot lines, polygons, and symbols in 2-D.
Figure.plot3d([data, x, y, z, size, symbol, ...])	Plot lines, polygons, and symbols in 3-D.
Figure.rose([data, length, azimuth, cmap, ...])	Plot a polar histogram (rose, sector, windrose diagrams).
Figure.ternary(data[, cmap, fill, width, ...])	Plot data on ternary diagrams.
Figure.velo([data, cmap, no_clip, ...])	Plot velocity vectors, strain crosses, anisotropy bars, and wedges.
Figure.wiggle([data, x, y, z, position, ...])	Plot z=f(x,y) anomalies along tracks.

11.1.4 Plotting raster data

Figure.grdcontour(grid[, projection, ...])	Make contour map using a grid.

continues on next page

Table 6 – continued from previous page

Figure.grdimage(grid[, cmap, monochrome, ...])	Project and plot grids or images.
Figure.grdview(grid[, cmap, drape_grid, ...])	Create 3-D perspective image or surface mesh from a grid.
Figure.image(imagefile[, position, width, ...])	Plot raster or EPS images.
Figure.tilemap(region[, zoom, source, ...])	Plot an XYZ tile map.

11.1.5 Configuring layout

Figure.set_panel([panel, tag, verbose])	Set the current subplot panel to plot on.
Figure.shift_origin([xshift, yshift])	Shift the plot origin in x- and/or y-directions.
Figure.subplot([nrows, ncols, tag, ...])	Manage figure subplot configuration and selection.

11.1.6 Saving and displaying the figure

Figure.savefig(fname[, transparent, crop, ...])	Save the figure to an image file.
Figure.show([method, dpi, width, waiting])	Display a preview of the figure.
Figure.psconvert([prefix, dpi, gs_option, ...])	Convert [E]PS file(s) to other formats using Ghostscript.

11.1.7 Configuring the display settings

The following function is provided directly through the `pygmt` top level package.

set_display([method])	Set the display method when calling *pygmt.Figure.show*.

pygmt.set_display

pygmt.**set_display**(*method=None*)

Set the display method when calling *pygmt.Figure.show*.

Parameters

method (Literal['external', 'notebook', 'none', None], default: None) – The method to display an image preview. Choose from:

- "external": External PDF preview using the default PDF viewer
- "notebook": Inline PNG preview in the current notebook
- "none": Disable image preview
- None: Reset to the default display method, which is either "external" in Python consoles or "notebook" in Jupyter notebooks.

Return type

None

Examples

Let's assume that you're using a Jupyter Notebook:

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c/5c", frame=True)
>>> fig.show()  # Will display a PNG image in the current notebook
>>>
>>> # Set the display method to "external"
>>> pygmt.set_display(method="external")
>>> fig.show()  # Will display a PDF image using the default PDF viewer
>>>
>>> # Set the display method to "none"
>>> pygmt.set_display(method="none")
>>> fig.show()  # Will not show any image
>>>
>>> # Reset to the default display method
>>> pygmt.set_display(method=None)
>>> fig.show()  # Again, will show a PNG image in the current notebook
```

11.1.8 Color palette table generation

The following functions are provided directly through the `pygmt` top level package.

grd2cpt(grid[, cmap, truncate, overrule_bg, ...])	Make linear or histogram-equalized color palette table from grid.
makecpt([cmap, truncate, overrule_bg, ...])	Make GMT color palette tables.

pygmt.grd2cpt

pygmt.**grd2cpt**(*grid*, *cmap=False*, *truncate=None*, *overrule_bg=False*, *no_bg=False*, *log=False*, *continuous=False*, *region=None*, *verbose=False*, ***kwargs*)

Make linear or histogram-equalized color palette table from grid.

This function will help you to make static color palette tables (CPTs). By default, the CPT will be saved as the current CPT of the session, figure, subplot, panel, or inset depending on which level *pygmt.grd2cpt* is called (for details on how GMT modern mode maintains different levels of colormaps please see https://docs.generic-mapping-tools.org/6.7/reference/features.html#gmt-modern-mode-hierarchical-levels). You can use `output` to save the CPT to a file. The CPT is based on an existing dynamic master CPT of your choice, and the mapping from data value to colors is through the data's cumulative distribution function (CDF), so that the colors are histogram equalized. Thus if the grid(s) and the resulting CPT are used in *pygmt.Figure.grdimage* with a linear projection, the colors will be uniformly distributed in area on the plot. Let z be the data values in the grid. Define CDF(Z) = (# of z < Z) / (# of z in grid). (NaNs are ignored). These z-values are then normalized to the master CPT and colors are sampled at the desired intervals.

The CPT includes three additional colors beyond the range of z-values. These are the background color (B) assigned to values lower than the lowest *z*-value, the foreground color (F) assigned to values higher than the highest *z*-value, and the NaN color (N) painted wherever values are undefined. For color tables beyond the standard GMT offerings, visit cpt-city and Scientific Colour-Maps.

If the master CPT includes B, F, and N entries, these will be copied into the new master file. If not, the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN from the gmt.conf file will be used. This default behavior can be overruled using the parameters `background`, `overrule_bg` or `no_bg`.

The color model (RGB, HSV or CMYK) of the palette created by *pygmt.grd2cpt* will be the same as specified in the header of the master CPT. When there is no COLOR_MODEL entry in the master CPT, the COLOR_MODEL specified in the gmt.conf file or the `color_model` parameter will be used.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grd2cpt.html.

Aliases

- A = transparency
- D = background
- E = nlevels
- F = color_model
- H = output
- I = reverse
- L = limit
- T = series
- W = categorical
- Ww = cyclic
- C = cmap
- G = truncate
- M = overrule_bg
- N = no_bg
- Q = log
- R = region
- V = verbose
- Z = continuous

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **transparency** (*float or str*) – Set a constant level of transparency (0-100) for all color slices. Append **+a** to also affect the foreground, background, and NaN colors [Default is no transparency, i.e., `0` (opaque)].

- **cmap** (`str` | `bool`, default: `False`) – Select the master color palette table (CPT) to use in the interpolation. Full list of built-in color palette tables can be found at https://docs.generic-mapping-tools.org/6.7/reference/cpts.html#built-in-color-palette-tables-cpt.

- **background** (*bool or str*) – Select the back- and foreground colors to match the colors for lowest and highest *z*-values in the output CPT [Default (`background=True` or `background="o"`) uses the colors specified in the master file, or those defined by the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN]. Use `background="i"` to match the colors for the lowest and highest values in the input (instead of the output) CPT.

- **color_model** (*str*) – [**R**|**r**|**h**|**c**][**+c**[*label*|*start*[**-**]]]. Force output CPT to be written with r/g/b codes, gray-scale values or color name (**R**, default) or r/g/b codes only (**r**), or h-s-v codes (**h**), or c/m/y/k codes (**c**). Optionally or alternatively, append **+c** to write discrete palettes in categorical format. If *label* is appended then we create labels for each category to be used when the CPT is plotted. The *label* may be a comma-separated list of category names (you can skip a category by not giving a name), or give *start*, where we automatically build monotonically increasing labels from *start* (a single letter or an integer). Additionally append **-** to build ranges *start-start+1* as labels instead.

- **nlevels** (*bool, int, or str*) – Set to `True` to create a linear color table by using the grid z-range as the new limits in the CPT. Alternatively, set *nlevels* to resample the color table into *nlevels* equidistant slices.

- **series** (*list or str*) – [*min/max/inc*[**+b**|**l**|**n**]|*file*|*list*]. Define the range of the new CPT by giving the lowest and highest z-value (and optionally an interval). If this is not given, the existing range in the master CPT will be used intact. The values produced defines the color slice boundaries. If **+n** is used it refers to the number of such boundaries and not the number of slices. For details on array creation, see https://docs.generic-mapping-tools.org/6.7/makecpt.html#generate-1d-array.

- **truncate** (`Sequence`[`float`] | `None`, default: `None`) – (*zlow*, *zhigh*). Truncate the incoming CPT so that the lowest and highest z-levels are to *zlow* and *zhigh*. If one of these equals NaN, then we leave that end of the CPT alone. The truncation takes place before any resampling. See also https://docs.generic-mapping-tools.org/6.7/reference/features.html#manipulating-cpts.

- **output** (*str*) – Optional. The file name with extension .cpt to store the generated CPT file. If not given or `False` [Default], saves the CPT as the current CPT of the session, figure, subplot, panel, or inset depending on which level *pygmt.grd2cpt* is called.
- **reverse** (*str*) – Set this to `True` or **c** [Default] to reverse the sense of color progression in the master CPT. Set this to **z** to reverse the sign of z-values in the color table. Note that this change of z-direction happens before `truncate` and `series` values are used so the latter must be compatible with the changed z-range. See also https://docs.generic-mapping-tools.org/6.7/reference/features.html#manipulating-cpts.
- **overrule_bg** (`bool`, default: `False`) – Overrule background, foreground, and NaN colors specified in the master CPT with the values of the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN specified in the gmt.conf file or by *pygmt.config*. When combined with `background`, only COLOR_NAN is considered.
- **no_bg** (`bool`, default: `False`) – Do not write out the background, foreground, and NaN-color fields [Default will write them, i.e. `no_bg=False`].
- **log** (`bool`, default: `False`) – For logarithmic interpolation scheme with input given as logarithms. Expects input z-values provided via `series` to be log10(z), assigns colors, and writes out z.
- **continuous** (`bool`, default: `False`) – Force a continuous CPT when building from a list of colors and a list of z-values [Default is False, i.e. discrete CPT].
- **categorical** (*bool*) – Do not interpolate the input color table but pick the output colors starting at the beginning of the color table, until colors for all intervals are assigned. This is particularly useful in combination with a categorical color table, like `cmap="gmt/categorical"`.
- **cyclic** (*bool*) – Produce a wrapped (cyclic) color table that endlessly repeats its range. Note that `cyclic=True` cannot be set together with `categorical=True`.
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

Examples

```
>>> import pygmt
>>> # load the 30 arc-minutes grid with "gridline" registration
>>> grid = pygmt.datasets.load_earth_relief("30m", registration="gridline")
>>> # create a plot
>>> fig = pygmt.Figure()
>>> # create a CPT from the grid object with grd2cpt
>>> pygmt.grd2cpt(grid=grid)
>>> # plot the grid object, the CPT will be automatically used
>>> fig.grdimage(grid=grid)
>>> # show the plot
>>> fig.show()
```

pygmt.makecpt

pygmt.**makecpt**(*cmap=False*, *truncate=None*, *overrule_bg=False*, *no_bg=False*, *log=False*, *continuous=False*, *verbose=False*, ***kwargs*)

Make GMT color palette tables.

This function will help you to make static color palette tables (CPTs). By default, the CPT will be saved as the current CPT of the session, figure, subplot, panel, or inset depending on which level *pygmt.makecpt* is called (for details on how GMT modern mode maintains different levels of colormaps please see https://docs.generic-mapping-tools.org/6.7/reference/features.html#gmt-modern-mode-hierarchical-levels). You can use `output` to save the CPT to a file. You define an equidistant set of contour intervals or pass your own z-table or list, and create a new CPT based on an existing master (dynamic) CPT. The resulting CPT can be reversed relative to the master cpt, and can be made continuous or discrete. For color tables beyond the standard GMT offerings, visit cpt-city and Scientific Colour-Maps.

The CPT includes three additional colors beyond the range of z-values. These are the background color (B) assigned to values lower than the lowest *z*-value, the foreground color (F) assigned to values higher than the highest *z*-value, and the NaN color (N) painted wherever values are undefined.

If the master CPT includes B, F, and N entries, these will be copied into the new master file. If not, the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN from the gmt.conf file will be used. This default behavior can be overruled using the parameters `background`, `overrule_bg` or `no_bg`.

The color model (RGB, HSV or CMYK) of the palette created by *pygmt.makecpt* will be the same as specified in the header of the master CPT. When there is no COLOR_MODEL entry in the master CPT, the COLOR_MODEL specified in the gmt.conf file will be used.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/makecpt.html.

Aliases

- A = transparency
- D = background
- F = color_model
- H = output
- I = reverse
- T = series
- W = categorical
- Ww = cyclic
- C = cmap
- G = truncate
- M = overrule_bg
- N = no_bg
- Q = log
- V = verbose
- Z = continuous

Parameters

- **transparency** (*float or str*) – Set a constant level of transparency (0-100) for all color slices. Append **+a** to also affect the foreground, background, and NaN colors [Default is no transparency, i.e., `0` (opaque)].
- **cmap** (`str` | `bool`, default: `False`) – Select the master color palette table (CPT) to use in the interpolation. Full list of built-in color palette tables can be found at https://docs.generic-mapping-tools.org/6.7/reference/cpts.html#built-in-color-palette-tables-cpt.
- **background** (*bool or str*) – Select the back- and foreground colors to match the colors for lowest and highest *z*-values in the output CPT [Default (`background=True` or `background="o"`) uses the colors specified in the master file, or those defined by the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN]. Use `background="i"` to match the colors for the lowest and highest values in the input (instead of the output) CPT.
- **color_model** (*str*) – [**R**|**r**|**h**|**c**][**+c**[*label*|*start*[**-**]]]. Force output CPT to be written with r/g/b codes, gray-scale values or color name (**R**, default) or r/g/b codes only (**r**), or h-s-v codes (**h**), or c/m/y/k codes (**c**). Optionally or alternatively, append **+c** to write discrete palettes in categorical format. If *label* is appended then we create labels for each category to be used when

the CPT is plotted. The *label* may be a comma-separated list of category names (you can skip a category by not giving a name), or give *start*, where we automatically build monotonically increasing labels from *start* (a single letter or an integer). Additionally append **-** to build ranges *start-start+1* as labels instead.

- **series** (*list or str*) – [*min/max/inc*[**+b**|**l**|**n**]|*file*|*list*]. Define the range of the new CPT by giving the lowest and highest z-value (and optionally an interval). If this is not given, the existing range in the master CPT will be used intact. The values produced defines the color slice boundaries. If **+n** is used it refers to the number of such boundaries and not the number of slices. For details on array creation, see https://docs.generic-mapping-tools.org/6.7/makecpt.html#generate-1d-array.
- **truncate** (Sequence[float] | None, default: None) – (*zlow*, *zhigh*). Truncate the incoming CPT so that the lowest and highest z-levels are to *zlow* and *zhigh*. If one of these equals NaN, then we leave that end of the CPT alone. The truncation takes place before any resampling. See also https://docs.generic-mapping-tools.org/6.7/reference/features.html#manipulating-cpts.
- **output** (*str*) – Optional. The file name with extension .cpt to store the generated CPT file. If not given or False [Default], saves the CPT as the current CPT of the session, figure, subplot, panel, or inset depending on which level *pygmt.makecpt* is called.
- **reverse** (*str*) – Set this to True or **c** [Default] to reverse the sense of color progression in the master CPT. Also the foreground and background colors are exchanged, including those specified by COLOR_BACKGROUND and COLOR_FOREGROUND. Set this to **z** to reverse the sign of z-values in the color table. Note that this change of the z-direction happens before the truncate and series values are used, so the latter must be compatible with the changed z-range. See also https://docs.generic-mapping-tools.org/6.7/reference/features.html#manipulating-cpts.
- **overrule_bg** (bool, default: False) – Overrule background, foreground, and NaN colors specified in the master CPT with the values of the parameters COLOR_BACKGROUND, COLOR_FOREGROUND, and COLOR_NAN specified in the gmt.conf file or by *pygmt.config*. When combined with background, only COLOR_NAN is considered.
- **no_bg** (bool, default: False) – Do not write out the background, foreground, and NaN-color fields [Default will write them, i.e. no_bg=False].
- **log** (bool, default: False) – For logarithmic interpolation scheme with input given as logarithms. Expects input z-values provided via series to be log10(z), assigns colors, and writes out z.
- **continuous** (bool, default: False) – Force a continuous CPT when building from a list of colors and a list of z-values [Default is False, i.e. discrete CPT].
- **categorical** (*bool*) – Do not interpolate the input color table but pick the output colors starting at the beginning of the color table, until colors for all intervals are assigned. This is particularly useful in combination with a categorical color table, like cmap="gmt/categorical".
- **cyclic** (*bool*) – Produce a wrapped (cyclic) color table that endlessly repeats its range. Note that cyclic=True cannot be set together with categorical=True.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Examples using `pygmt.makecpt`

11.2 Data Processing

11.2.1 Operations on tabular data

`binstats`(data[, outgrid, spacing, ...])	Bin spatial data and determine statistics per bin.
`blockmean`([data, x, y, z, output_type, ...])	Block average (x, y, z) data tables by mean estimation.
`blockmedian`([data, x, y, z, output_type, ...])	Block average (x, y, z) data tables by median estimation.
`blockmode`([data, x, y, z, output_type, ...])	Block average (x, y, z) data tables by mode estimation.
`filter1d`(data[, output_type, outfile, verbose])	Time domain filtering of 1-D data tables.
`nearneighbor`([data, x, y, z, outgrid, ...])	Grid table data using a "Nearest neighbor" algorithm.
`project`([data, x, y, z, output_type, ...])	Project data onto lines or great circles, or generate tracks.
`select`([data, output_type, outfile, ...])	Select data table subsets based on multiple spatial criteria.
`sph2grd`(data[, outgrid, spacing, region, ...])	Compute grid from spherical harmonic coefficients.
`sphdistance`([data, x, y, outgrid, spacing, ...])	Create Voronoi distance, node, or natural nearest-neighbor grid on a sphere.
`sphinterpolate`(data[, outgrid, spacing, ...])	Spherical gridding in tension of data on a sphere.
`surface`([data, x, y, z, outgrid, spacing, ...])	Grid table data using adjustable tension continuous curvature splines.
`triangulate`()	Delaunay triangulation or Voronoi partitioning and gridding of Cartesian data.
`triangulate.regular_grid`([data, x, y, z, ...])	Delaunay triangle based gridding of Cartesian data.
`triangulate.delaunay_triples`([data, x, y, ...])	Delaunay triangle based gridding of Cartesian data.
`xyz2grd`([data, x, y, z, outgrid, spacing, ...])	Convert data table to a grid.

pygmt.binstats

pygmt.**binstats**(*data*, *outgrid=None*, *spacing=None*, *statistic='number'*, *quantile_value=50*, *region=None*, *verbose=False*, *incols=None*, *registration=False*, ***kwargs*)

Bin spatial data and determine statistics per bin.

Reads arbitrarily located (x,y[,z][,w]) points (2-4 columns) from `data` and for each node in the specified grid layout determines which points are within the given radius. These points are then used in the calculation of the specified statistic. The results may be presented as is or may be normalized by the circle area to perhaps give density estimates.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/gmtbinstats.html.

Aliases

- E = empty
- N = normalize
- S = search_radius
- W = weight
- a = aspatial
- b = binary
- h = header
- C = statistic
- G = outgrid
- I = spacing
- R = region
- V = verbose
- i = incols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame`) – A file name of an ASCII data table or a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **statistic** (Literal['mean', 'mad', 'full', 'interquartile', 'min', 'minpos', 'median', 'number', 'lms', 'mode', 'quantile', 'rms', 'stddev', 'max', 'maxneg', 'sum'], default: 'number') – Choose the statistic that will be computed per node based on the points that are within *radius* distance of the node. Select one of:
  - "mean": Mean (i.e., average).
  - "mad": Median absolute deviation (MAD).
  - "full": The full (max-min) range.
  - "interquartile": The 25-75% interquartile range.
  - "min": Minimum (lowest value).
  - "minpos": Minimum of positive values only.
  - "median": Median value.
  - "number": The number of values per bin.
  - "lms": Least median square (LMS) scale.
  - "mode": Mode (maximum likelihood estimate).
  - "quantile": Selected quantile. The quantile value is specified by the quantile_value parameter and is in the 0-100% range.
  - "rms": Root mean square (RMS).
  - "stddev": Standard deviation.
  - "max": Maximum (highest value).
  - "maxneg": Maximum of negative values only.
  - "sum": The sum of the values.
- **quantile_value** (float, default: 50) – The quantile value if statistic="quantile".
- **empty** (*float*) – Set the value assigned to empty nodes [Default is NaN].
- **normalize** (*bool*) – Normalize the resulting grid values by the area represented by the search *radius* [Default is no normalization].
- **search_radius** (*float or str*) – Set the *search_radius* that determines which data points are considered close to a node. Append a *distance unit*. Not compatible with tiling.
- **weight** (*str*) – Input data have an extra column containing observation point weight. If weights are given then weighted statistical quantities will be computed while the count will be the sum of the weights instead of number of points. If the weights are actually uncertainties (one sigma) then append **+s** and weight = 1/sigma.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID).

If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

- **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.

- **+c** to add a header comment with column names to the output [Default is no column names].
- **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
- **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
- **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

Blank lines and lines starting with # are always skipped.

- **incols** (`int` | `str` | `Sequence[int | str]` | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.

Return type

`DataArray | None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

pygmt.blockmean

pygmt.**blockmean**(*data=None*, *x=None*, *y=None*, *z=None*, *output_type='pandas'*, *outfile=None*, *center=False*, *spacing=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Block average (x, y, z) data tables by mean estimation.

Reads arbitrarily located (x, y, z) triplets [or optionally weighted quadruplets (x, y, z, w)] and writes to the output a mean position and value for every non-empty block in a grid region defined by the `region` and `spacing` parameters.

Takes a matrix, (x, y, z) triplets, or a file name as input.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/blockmean.html.

Aliases

- S = summary
- a = aspatial
- b = binary
- d = nodata
- e = find
- h = header
- w = wrap
- C = center
- I = spacing
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.
- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **center** (`bool`, default: `False`) – Use the center of each block as the output location. By default, the mean x and y coordinates are used.
- **spacing** (`Sequence[float | str]` | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **summary** (*str*) – [**m**|**n**|**s**|**w**]. Type of summary values calculated by blockmean.
  - **m**: reports mean value [Default]
  - **n**: report the number of input points inside each block
  - **s**: report the sum of all z-values inside a block
  - **w**: report the sum of weights
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, nodata="-9999" will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.

- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (int | str | Sequence[int | str] | None, default: None) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **outcols** (int | str | Sequence[int | str] | None, default: None) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word

number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

Optionally, append one of the following modifiers to any column or column range to transform the output columns:

* **+l** to take the *log10* of the input values.
* **+d** to divide the output values by the factor *divisor* [Default is 1].
* **+s** to multiply the output values by the factor *scale* [Default is 1].
* **+o** to add the given *offset* to the output values [Default is 0].

- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
`DataFrame | ndarray | None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a table of ship observations of bathymetry off Baja California
>>> data = pygmt.datasets.load_sample_data(name="bathymetry")
>>> # Calculate block mean values within 5 by 5 arc-minute bins
>>> data_bmean = pygmt.blockmean(data=data, region=[245, 255, 20, 30], spacing="5m
↪")
```

Examples using `pygmt.blockmean`

pygmt.blockmedian

pygmt.**blockmedian**(*data=None*, *x=None*, *y=None*, *z=None*, *output_type='pandas'*, *outfile=None*, *center=False*, *spacing=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Block average (x, y, z) data tables by median estimation.

Reads arbitrarily located (x, y, z) triplets [or optionally weighted quadruplets (x, y, z, w)] and writes to the output a median position and value for every non-empty block in a grid region defined by the `region` and `spacing` parameters.

Takes a matrix, (x, y, z) triplets, or a file name as input.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/blockmedian.html.

Aliases

- a = aspatial
- b = binary
- d = nodata
- e = find
- h = header
- w = wrap
- C = center
- I = spacing
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.
- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **center** (`bool`, default: `False`) – Use the center of each block as the output location. By default, the median x and median y coordinates are used.
- **spacing** (`Sequence[float | str] | None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID).

If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

- **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.

- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **outcols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).

  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

    Optionally, append one of the following modifiers to any column or column range to transform the output columns:

    * **+l** to take the *log10* of the input values.
    * **+d** to divide the output values by the factor *divisor* [Default is 1].
    * **+s** to multiply the output values by the factor *scale* [Default is 1].
    * **+o** to add the given *offset* to the output values [Default is 0].

- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
`DataFrame | ndarray | None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in the file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a table of ship observations of bathymetry off Baja California
>>> data = pygmt.datasets.load_sample_data(name="bathymetry")
>>> # Calculate block median values within 5 by 5 arc-minute bins
```

(continues on next page)

(continued from previous page)

```
>>> data_bmedian = pygmt.blockmedian(
...     data=data, region=[245, 255, 20, 30], spacing="5m"
... )
```

Examples using `pygmt.blockmedian`

pygmt.blockmode

pygmt.**blockmode**(*data=None*, *x=None*, *y=None*, *z=None*, *output_type='pandas'*, *outfile=None*, *center=False*, *spacing=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Block average (x, y, z) data tables by mode estimation.

Reads arbitrarily located (x, y, z) triplets [or optionally weighted quadruplets (x, y, z, w)] and writes to the output a mode position and value for every non-empty block in a grid region defined by the `region` and `spacing` parameters.

Takes a matrix, (x, y, z) triplets, or a file name as input.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/blockmode.html.

Aliases

- a = aspatial
- b = binary
- d = nodata
- e = find
- h = header
- w = wrap
- C = center
- I = spacing
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.
- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **center** (`bool`, default: `False`) – Use the center of each block as the output location. By default, the modal x and y coordinates are used.
- **spacing** (`Sequence[float | str]` | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.

  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **`verbose`** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **`aspatial`** (*`bool or str`*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **`binary`** (*`bool or str`*) – **i|o**[*ncols*][*type*][**w**][**+l|b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l|b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **outcols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – *cols*[,…][,**t**[*word*]].

Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].

- For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
- For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

  Optionally, append one of the following modifiers to any column or column range to transform the output columns:

  * **+l** to take the *log10* of the input values.
  * **+d** to divide the output values by the factor *divisor* [Default is 1].
  * **+s** to multiply the output values by the factor *scale* [Default is 1].
  * **+o** to add the given *offset* to the output values [Default is 0].

- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type

`DataFrame | ndarray | None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in the file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a table of ship observations of bathymetry off Baja California
>>> data = pygmt.datasets.load_sample_data(name="bathymetry")
>>> # Calculate block mode values within 5 by 5 arc-minute bins
>>> data_bmode = pygmt.blockmode(data=data, region=[245, 255, 20, 30], spacing="5m
↪")
```

Examples using `pygmt.blockmode`

pygmt.filter1d

pygmt.**filter1d**(*data*, *output_type='pandas'*, *outfile=None*, *verbose=False*, ***kwargs*)

Time domain filtering of 1-D data tables.

A general time domain filter for multiple column time series data. The user specifies which column is the time (i.e., the independent variable) via `time_col`. The fastest operation occurs when the input time series are equally spaced and have no gaps or outliers and the special options are not needed. Read a table and output as a `numpy.ndarray`, `pandas.DataFrame`, or ASCII file.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/filter1d.html.

Aliases

- E = end
- F = filter_type
- N = time_col
- V = verbose

Parameters

- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.
- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **filter_type** (*str*) – **type***width*[**+h**]. Set the filter **type**. Choose among convolution and non-convolution filters. Append the filter code followed by the full filter *width* in same units as time column. By default, this performs a low-pass filtering; append **+h** to select high-pass filtering. Some filters allow for optional arguments and a modifier.

  Available convolution filter types are:
  - **b**: boxcar. All weights are equal.
  - **c**: cosine arch. Weights follow a cosine arch curve.
  - **g**: Gaussian. Weights are given by the Gaussian function.
  - **f**: custom. Instead of *width* give name of a one-column file with your own weight coefficients.

  Non-convolution filter types are:
  - **m**: median. Returns median value.

- **p**: maximum likelihood probability (a mode estimator). Return modal value. If more than one mode is found we return their average value. Append **+l** or **+u** if you rather want to return the lowermost or uppermost of the modal values.
- **l**: lower (absolute). Return the minimum of all values.
- **L**: lower. Return minimum of all positive values only.
- **u**: upper (absolute). Return maximum of all values.
- **U**: upper. Return maximum of all negative values only.

Uppercase type **B**, **C**, **G**, **M**, **P**, **F** will use robust filter versions: i.e., replace outliers (2.5 L1 scale off median, using 1.4826 * median absolute deviation [MAD]) with median during filtering.

In the case of **L**|**U** it is possible that no data passes the initial sign test; in that case the filter will return 0.0. Apart from custom coefficients (**f**), the other filters may accept variable filter widths by passing *width* as a two-column time-series file with filter widths in the second column. The filter-width file does not need to be co-registered with the data as we obtain the required filter width at each output location via interpolation. For multi-segment data files the filter file must either have the same number of segments or just a single segment to be used for all data segments.

- **end** (*bool*) – Include ends of time series in output. The default [False] loses half the filter-width of data at each end.
- **time_col** (*int*) – Indicate which column contains the independent variable (time). The left-most column is 0, while the right-most is (*n_cols* - 1) [Default is `0`].
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataFrame` | `ndarray` | `None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in the file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

pygmt.nearneighbor

pygmt.**nearneighbor**(*data=None*, *x=None*, *y=None*, *z=None*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, *incols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Grid table data using a "Nearest neighbor" algorithm.

pygmt.nearneighbor reads arbitrarily located (*x*, *y*, *z*[, *w*]) triplets [quadruplets] and uses a nearest neighbor algorithm to assign a weighted average value to each node that has one or more data points within a search radius centered on the node with adequate coverage across a subset of the chosen sectors. The node value is computed as a weighted mean of the nearest point from each sector inside the search radius. The weighting function and the averaging used is given by:

$$w(r_i) = \frac{w_i}{1 + d(r_i)^2}, \quad d(r) = \frac{3r}{R}, \quad \bar{z} = \frac{\sum_i^n w(r_i) z_i}{\sum_i^n w(r_i)}$$

where n is the number of data points that satisfy the selection criteria and r_i is the distance from the node to the *i*'th data point. If no data weights are supplied then $w_i = 1$.

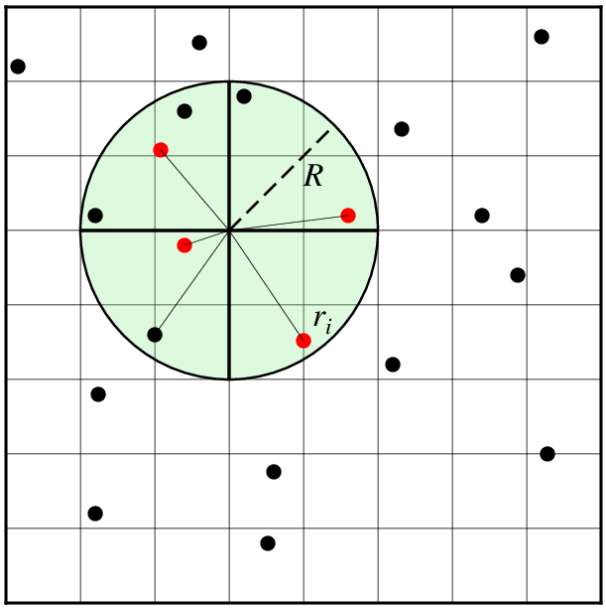


Fig. 1: Search geometry includes the search radius (R) which limits the points considered and the number of sectors (here 4), which restricts how points inside the search radius contribute to the value at the node. Only the closest point in each sector (red circles) contribute to the weighted estimate.

Takes a matrix, (x, y, z) triplets, or a file name as input.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/nearneighbor.html.

Aliases

- E = empty
- N = sectors
- S = search_radius
- a = aspatial
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- w = wrap
- G = outgrid
- I = spacing
- R = region
- V = verbose
- f = coltypes
- i = incols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **spacing** (`Sequence`[`float` | `str`] | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID).

If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

- **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **search_radius** (*str*) – Set the search radius that determines which data points are considered close to a node.
- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **empty** (*str*) – Optional. Set the value assigned to empty nodes. Defaults to NaN.
- **sectors** (*str*) – *sectors*[**+m***min_sectors*]|**n**. Optional. The circular search area centered on each node is divided into *sectors* sectors. Average values will only be computed if there is *at least* one value inside each of at least *min_sectors* of the sectors for a given node. Nodes that fail this test are assigned the value NaN (but see `empty`). If **+m** is omitted then *min_sectors* is set to be at least 50% of *sectors* (i.e., rounded up to next integer) [Default is a quadrant search with 100% coverage, i.e., *sectors* = *min_sectors* = 4]. Note that only the nearest value per sector enters into the averaging; the more distant points are ignored. Alternatively, use `sectors="n"` to call GDAL's nearest neighbor algorithm instead.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i|o**[*ncols*][*type*][**w**][**+l|b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)

  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:

  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:

  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:

  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.

- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence[int | str]` | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)

- **s**: second cycle (second)
- **c**: custom cycle (normalized)

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray`: if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a sample dataset of bathymetric x, y, and z values
>>> data = pygmt.datasets.load_sample_data(name="bathymetry")
>>> # Create a new grid with 5 arc-minutes spacing in the designated region
>>> # Set search_radius to only take points within 10 arc-minutes of a node
>>> output = pygmt.nearneighbor(
...     data=data,
...     spacing="5m",
...     region=[245, 255, 20, 30],
...     search_radius="10m",
... )
```

pygmt.project

pygmt.**project**(*data=None*, *x=None*, *y=None*, *z=None*, *output_type='pandas'*, *outfile=None*, *azimuth=None*, *center=None*, *endpoint=None*, *width=None*, *length=None*, *pole=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Project data onto lines or great circles, or generate tracks.

Project reads arbitrary $(x, y[, z])$ data and returns any combination of (x, y, z, p, q, r, s), where (p, q) are the coordinates in the projection, (r, s) is the position in the (x, y) coordinate system of the point on the profile ($q = 0$ path) closest to (x, y), and z is all remaining columns in the input (beyond the required x and y columns).

Alternatively, `project` may be used to generate (r, s, p) triplets at equal increments along a profile using the `generate` parameter. In this case, the value of `data` is ignored (you can use, e.g., `data=None`).

Projections are defined in any (but only) one of three ways:

1. By a `center` and an `azimuth` in degrees clockwise from North.
2. By a `center` and `endpoint` of the projection path.
3. By a `center` and a `pole` position.

To spherically project data along a great circle path, an oblique coordinate system is created which has its equator along that path, and the zero meridian through the Center. Then the oblique longitude (p) corresponds to the distance from the Center along the great circle, and the oblique latitude (q) corresponds to the distance perpendicular to the great circle path. When moving in the increasing (p) direction, (toward B or in the azimuth direction), the positive (q) direction is to your left. If a Pole has been specified, then the positive (q) direction is toward the pole.

To specify an oblique projection, use the `pole` parameter to set the pole. Then the equator of the projection is already determined and the `center` parameter is used to locate the $p = 0$ meridian. The center *cx/cy* will be taken

as a point through which the $p = 0$ meridian passes. If you do not care to choose a particular point, use the South pole (*cx* = 0, *cy* = -90).

Data can be selectively windowed by using the `length` and `width` parameters. If `width` is used, the projection width is set to use only data with $w_{min} < q < w_{max}$. If `length` is set, then the length is set to use only those data with $l_{min} < p < l_{max}$. If the `endpoint` parameter has been used to define the projection, then `length="w"` may be used to window the length of the projection to exactly the span from O to B.

Flat Earth (Cartesian) coordinate transformations can also be made. Set `flat_earth=True` and remember that azimuth is clockwise from North (the y-axis), NOT the usual cartesian theta, which is counterclockwise from the x-axis. azimuth = 90 - theta.

No assumptions are made regarding the units for $x, y, r, s, p, q, dist, l_{min}, l_{max}, w_{min}, w_{max}$. If `unit` is selected, map units are assumed and x, y, r, s must be in degrees and $p, q, dist, l_{min}, l_{max}, w_{min}, w_{max}$ will be in km.

Calculations of specific great-circle and geodesic distances or for back-azimuths or azimuths are better done using https://docs.generic-mapping-tools.org/6.7/mapproject as project is strictly spherical.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/project.html.

Aliases

- F = convention
- G = generate
- N = flat_earth
- Q = unit
- S = sort
- Z = ellipse
- A = azimuth
- C = center
- E = endpoint
- L = length
- T = pole
- V = verbose
- W = width
- f = coltypes

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x- and y-coordinates and z-values of the data points.
- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.
- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **center** (`Sequence[float | str] | None`, default: `None`) – Set the origin of the projection, in the form of (*cx*, *cy*), in Definitions 1 or 2. If Definition 3 is used, then (*cx*, *cy*) are the coordinates of a point through which the oblique zero meridian ($p = 0$) should pass. (*cx*, *cy*) is not required to be 90 degrees from the pole set by `pole`.
- **azimuth** (`float` | `None`, default: `None`) – Set the azimuth of the projection (Definition 1). The azimuth is clockwise from North (the y-axis) regardless of whether spherical or Cartesian coordinate transformation is applied.
- **endpoint** (`Sequence[float | str] | None`, default: `None`) – (*bx*, *by*). Set the end point of the projection path (Definition 2).

- **convention** (*str*) – Specify the desired output using any combination of **xyzpqrs**, in any order [Default is **xypqrsz**]. Do not space between the letters. Use lowercase. The output will be columns of values corresponding to your `convention`. The **z** flag is special and refers to all numerical columns beyond the leading **x** and **y** in your input record. The **z** flag also includes any trailing text (which is placed at the end of the record regardless of the order of **z** in `convention`). **Note**: If `generate` is True, then the output order is hardwired to be **rsp** and `convention` is not allowed.
- **generate** (*str*) – *dist* [*/colat*][**+c**|**h**]. Create (r, s, p) output data every *dist* units of p (See `unit` parameter). Alternatively, append */colat* for a small circle instead [Default is a colatitude of 90, i.e., a great circle]. If setting a pole with `pole` and you want the small circle to go through *cx*/*cy*, append **+c** to compute the required colatitude. Use `center` and `endpoint` to generate a circle that goes through the center and end point. Note, in this case the center and end point cannot be farther apart than 2|colat|. Finally, if you append **+h** then we will report the position of the pole as part of the segment header [Default is no header]. **Note**: No input is read and the value of `data`, `x`, `y`, and `z` is ignored if `generate` is used.
- **length** (`Sequence`[`float` | `str`] | `Literal`[`'limit'`] | `None`, default: `None`) – (*lmin*, *lmax*) or `"limit"`. Project only those data whose *p* coordinate is within $l_{min} < p < l_{max}$. If `endpoint` has been set, then you may alternatively set it to `"limit"` to stay within the distance from `center` to `endpoint`.
- **flat_earth** (*bool*) – Make a Cartesian coordinate transformation in the plane. [Default is `False`; plane created with spherical trigonometry.]
- **unit** (*bool*) – Set units for x, y, r, s to degrees and $p, q, dist, l_{min}, l_{max}, w_{min}, w_{max}$ to km. [Default is `False`; all arguments use the same units]
- **sort** (*bool*) – Sort the output into increasing p order. Useful when projecting random data into a sequential profile.
- **pole** (`Sequence`[`float` | `str`] | `None`, default: `None`) – (*px*, *py*). Set the position of the rotation pole of the projection. (Definition 3).
- **width** (`Sequence`[`float` | `str`] | `None`, default: `None`) – (*w_min*, *w_max*). Specify width controls for the projected points. Project only those points whose q coordinate is within $w_{min} < q < w_{max}$.
- **ellipse** (*str*) – *major*/*minor*/*azimuth* [**+e**|**n**]. Used in conjunction with `center` (sets its center) and `generate` (sets the distance increment) to create the coordinates of an ellipse with *major* and *minor* axes given in km (unless `flat_earth` is given for a Cartesian ellipse) and the *azimuth* of the major axis in degrees. Append **+e** to adjust the increment set via `generate` so that the ellipse has equal distance increments [Default uses the given increment and closes the ellipse]. Instead, append **+n** to set a specific number of unique equidistant data via `generate`. For degenerate ellipses you can just supply a single *diameter* instead. A geographic diameter may be specified in any desired unit other than km by appending the unit (e.g., 3-D for degrees) [Default is km]; the increment is assumed to be in the same unit. **Note**: For the Cartesian ellipse (which requires `flat_earth`), the *direction* is counter-clockwise from the horizontal instead of an *azimuth*.
- **verbose** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

`DataFrame` | `ndarray` | `None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples using `pygmt.project`

pygmt.select

pygmt.**select**(*data=None*, *output_type='pandas'*, *outfile=None*, *resolution=None*, *area_thresh=None*, *projection=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *coltypes=None*, ***kwargs*)

Select data table subsets based on multiple spatial criteria.

This is a filter that reads (x, y) or (longitude, latitude) positions from the first 2 columns of `data` and uses a combination of 1-7 criteria to pass or reject the records. Records can be selected based on whether or not they:

1. are inside a rectangular region (`region` [and `projection`])
2. are within *dist* km of any point in *pointfile* (`dist2pt`)
3. are within *dist* km of any line in *linefile* (`dist2line`)
4. are inside one of the polygons in *polygonfile* (`polygon`)
5. are inside geographical features (based on coastlines)
6. have z-values within a given range
7. are inside bins of a grid mask whose nodes are non-zero

The sense of the tests can be reversed for each of these 7 criteria by using the `reverse` parameter.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/gmtselect.html.

Aliases

- C = dist2pt
- F = polygon
- G = mask_grid
- I = reverse
- L = dist2line
- N = mask_values
- Z = z_subregion
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- s = skiprows
- w = wrap
- A = area_thresh
- D = resolution
- J = projection
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols

Parameters

- **`data`** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in either a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **`output_type`** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.
  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.

  - `file` will save the result to the file specified by the `outfile` parameter.

- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **area_thresh** (`float` | `str` | `None`, default: `None`) – *min_area*[/*min_level*/*max_level*][**+a**[**g**|**i**][**s**|**S**]][**+l**|**r**][**+p***percent*]. Features with an area smaller than *min_area* in km^2 or of hierarchical level that is lower than *min_level* or higher than *max_level* will not be plotted [Default is `"0/0/4"` (all features)].
- **dist2pt** (*str*) – *pointfile*|*lon*/*lat***+d***dist*. Pass all records whose locations are within *dist* of any of the points in the ASCII file *pointfile*. If *dist* is zero, the 3rd column of *pointfile* must have each point's individual radius of influence. If you only have a single point, you can specify *lon*/*lat* instead of *pointfile*. Distances are Cartesian and in user units. Alternatively, if `region` and `projection` are used, the geographic coordinates are projected to map coordinates (in centimeters, inches, meters, or points, as determined by PROJ_LENGTH_UNIT) before Cartesian distances are compared to *dist*.
- **dist2line** (*str*) – *linefile***+d***dist*[**+p**]. Pass all records whose locations are within *dist* of any of the line segments in the ASCII multiple-segment file *linefile*. If *dist* is zero, we will scan each sub-header in *linefile* for an embedded **-D***dist* setting that sets each line's individual distance value. Distances are Cartesian and in user units. Alternatively, if `region` and `projection` are used, the geographic coordinates are projected to map coordinates (in centimeters, inches, meters, or points, as determined by PROJ_LENGTH_UNIT) before Cartesian distances are compared to *dist*. Append **+p** to ensure only points whose orthogonal projections onto the nearest line-segment fall within the segment's endpoints [Default considers points "beyond" the line's endpoints].
- **polygon** (*str*) – *polygonfile*. Pass all records whose locations are within one of the closed polygons in the ASCII multiple-segment file *polygonfile*. For spherical polygons (lon, lat), make sure no consecutive points are separated by 180 degrees or more in longitude.
- **mask_grid** (*str*) – Pass all locations that are inside the valid data area of the grid *mask_grid*. Nodes that are outside are either NaN or zero.
- **reverse** (*str*) – [**cflrsz**]. Reverse the sense of the test for each of the criteria specified:
  - **c** select records NOT inside any point's circle of influence.
  - **f** select records NOT inside any of the polygons.
  - **g** will pass records inside the cells with z equal zero of the *mask_grid* in `mask_grid`.
  - **l** select records NOT within the specified distance of any line.
  - **r** select records NOT inside the specified rectangular region.
  - **s** select records NOT considered inside as specified by `mask_values` (and `area_thresh`, `resolution`).
  - **z** select records NOT within the range specified by `z_subregion`.
- **mask_values** (*str or list*) – Pass all records whose location is inside specified geographical features. Specify if records should be skipped (s) or kept (k) using 1 of 2 formats:
  - *wet/dry*.
  - *ocean/land/lake/island/pond*.

  [Default is s/k/s/k/s (i.e., s/k), which passes all points on dry land].
- **resolution** (`Literal['auto', 'full', 'high', 'intermediate', 'low', 'crude', None]`, default: `None`) – Ignored unless `mask_values` is set. Select the resolution of the

coastline dataset to use. The available resolutions from highest to lowest are: `"full"`, `"high"`, `"intermediate"`, `"low"`, and `"crude"`, which drops by 80% between levels. Alternatively, choose `"auto"` to automatically select the most suitable resolution given the chosen region. Note that because the coastlines differ in details, a node in a mask file using one resolution is not guaranteed to remain inside [or outside] when a different resolution is selected. If `None`, the low resolution is used by default.

- **`z_subregion`** (*`str or list`*) – *min*[/*max*][**+a**][**+c***col*][**+i**]. Pass all records whose 3rd column (*z*; *col* = 2) lies within the given range or is NaN (use `skiprows` to skip NaN records). If *max* is omitted then we test if *z* equals *min* instead. This means equality within 5 ULPs (unit of least precision; https://en.wikipedia.org/wiki/Unit_in_the_last_place). Input file must have at least three columns. To indicate no limit on *min* or *max*, specify a hyphen (-). If your 3rd column is absolute time then remember to supply `coltypes="2T"`. To specify another column, append **+c***col*, and to specify several tests pass a list of arguments as you have columns to test. **Note**: When more than one `z_subregion` argument is given then the `reverse="z"` cannot be used. In the case of multiple tests you may use these modifiers as well: **+a** passes any record that passes at least one of your *z* tests [Default is all tests must pass], and **+i** reverses the tests to pass record with *z* value NOT in the given range. Finally, if **+c** is not used then it is automatically incremented for each new `z_subregion` argument, starting with 2.
- **`projection`** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **`verbose`** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **`binary`** (*`bool or str`*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:
  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.

- **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

Blank lines and lines starting with # are always skipped.

- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **outcols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

    Optionally, append one of the following modifiers to any column or column range to transform the output columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the output values by the factor *divisor* [Default is 1].
    - **+s** to multiply the output values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the output values [Default is 0].
- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data

column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:

  - **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.
  - **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].

- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
DataFrame | ndarray | None

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a table of ship observations of bathymetry off Baja California
>>> ship_data = pygmt.datasets.load_sample_data(name="bathymetry")
>>> # Only return the data points that lie within the region between
>>> # longitudes 246 and 247 and latitudes 20 and 21
>>> out = pygmt.select(data=ship_data, region=[246, 247, 20, 21])
```

pygmt.sph2grd

pygmt.**sph2grd**(*data*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, *incols=None*, *registration=False*, *cores=False*, ***kwargs*)

Compute grid from spherical harmonic coefficients.

Reads a spherical harmonics coefficient table with records of L, M, C[L,M], S[L,M] and evaluates the spherical harmonic model on the specified grid.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/sph2grd.html.

Aliases

- b = binary
- h = header
- G = outgrid
- I = spacing
- R = region
- V = verbose
- i = incols
- r = registration
- x = cores

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame) – Pass in data with L, M, C[L,M], S[L,M] values by providing a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data.
- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the registration, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If region=grdfile is used then the grid spacing and the registration have already been initialized; use spacing and registration to override these values.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)

- **l**: int64_t (8-byte signed int)
- **L**: uint64_t (8-byte unsigned int)
- **f**: 4-byte single-precision float
- **d**: 8-byte double-precision float
- **x**: use to skip *ncols* anywhere in the record

For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:

- **w** after any item to force byte-swapping.
- **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].

- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.
- **cores** (int | bool, default: False) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type
DataArray | None

Returns

ret – Return type depends on whether the outgrid parameter is set:

- xarray.DataArray if outgrid is not set
- None if outgrid is set (grid output will be stored in file set by outgrid)

Examples

```
>>> import pygmt
>>> # Create a new grid from the remote file "EGM96_to_36.txt",
>>> # set the grid spacing to 1 arc-degree, and the region to global ("g")
>>> new_grid = pygmt.sph2grd(data="@EGM96_to_36.txt", spacing=1, region="g")
```

pygmt.sphdistance

pygmt.**sphdistance**(*data=None*, *x=None*, *y=None*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, ***kwargs*)

Create Voronoi distance, node, or natural nearest-neighbor grid on a sphere.

Reads a table containing *lon, lat* columns and performs the construction of Voronoi polygons. These polygons are then processed to calculate the nearest distance to each node of the lattice and written to the specified grid.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/sphdistance.html.

Aliases

- C = single_form
- D = duplicate
- E = quantity
- L = unit
- N = node_table
- Q = voronoi
- G = outgrid
- I = spacing
- R = region
- V = verbose

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in (x, y) or (longitude, latitude) values by providing a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data.
- **x/y** (*1-D arrays*) – Arrays of x and y coordinates.
- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format

or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the registration, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If region=grdfile is used then the grid spacing and the registration have already been initialized; use spacing and registration to override these values.
- **single_form** (*bool*) – For large data sets you can save some memory (at the expense of more processing) by only storing one form of location coordinates (geographic or Cartesian 3-D vectors) at any given time, translating from one form to the other when necessary [Default keeps both arrays in memory]. Not applicable with voronoi.
- **duplicate** (*bool*) – Used to skip duplicate points since the algorithm cannot handle them. [Default assumes there are no duplicates].
- **quantity** (*str*) – **d**|**n**|**z**[*dist*]. Specify the quantity that should be assigned to the grid nodes [Default is **d**]:
  - **d**: compute distances to the nearest data point
  - **n**: assign the ID numbers of the Voronoi polygons that each grid node is inside
  - **z**: assign all nodes inside the polygon the z-value of the center node for a natural nearest-neighbor grid.

  Optionally, append the resampling interval along Voronoi arcs in spherical degrees.
- **unit** (*str*) – Specify the unit used for distance calculations. Valid values are **d**, **e**, **f**, **k**, **M**, **n**, and **u**. See *Distance Units* for meanings of the units.
- **node_table** (*str*) – Read the information pertaining to each Voronoi polygon (the unique node lon, lat and polygon area) from a separate file [Default acquires this information from the ASCII segment headers of the output file]. Required if binary input via *voronoi* is used.
- **voronoi** (*str*) – Append the name of a file with pre-calculated Voronoi polygons [Default performs the Voronoi construction on input data].
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Return type
DataArray | None

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- None if `outgrid` is set (grid output will be stored in file set by `outgrid`)

Examples

```
>>> import numpy as np
>>> import pygmt
>>> # Create an array of longitude/latitude coordinates
>>> coords_list = [[85.5, 22.3], [82.3, 22.6], [85.8, 22.4], [86.5, 23.3]]
>>> coords_array = np.array(coords_list)
>>> # Perform a calculation of the distance to
>>> # each point from Voronoi polygons
>>> grid = pygmt.sphdistance(
...     data=coords_array, spacing=[1, 2], region=[82, 87, 22, 24]
... )
```

pygmt.sphinterpolate

pygmt.**sphinterpolate**(*data*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, ***kwargs*)

Spherical gridding in tension of data on a sphere.

Reads a table containing *lon, lat, z* columns and performs a Delaunay triangulation to set up a spherical interpolation in tension. Several options may be used to affect the outcome, such as choosing local versus global gradient estimation or optimize the tension selection to satisfy one of four criteria.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/sphinterpolate.html.

Aliases

- G = outgrid
- I = spacing
- R = region
- V = verbose

Parameters

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n**

or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

– **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **`verbose`** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- None if `outgrid` is set (grid output will be stored in file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a table of Mars with longitude/latitude/radius columns
>>> mars_shape = pygmt.datasets.load_sample_data(name="mars_shape")
>>> # Perform Delaunay triangulation on the table data
>>> # to produce a grid with a 1 arc-degree spacing
>>> grid = pygmt.sphinterpolate(data=mars_shape, spacing=1, region="g")
```

pygmt.surface

pygmt.**surface**(*data=None*, *x=None*, *y=None*, *z=None*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, *registration=False*, *coltypes=None*, ***kwargs*)

Grid table data using adjustable tension continuous curvature splines.

Surface reads randomly-spaced (x, y, z) triplets and produces gridded values z(x,y) by solving:

$$(1-t)\nabla^2(z) + t\nabla(z) = 0$$

where t is a tension factor between 0 and 1, and ∇ indicates the Laplacian operator. Here, $t = 0$ gives the "minimum curvature" solution. Minimum curvature can cause undesired oscillations and false local maxima or minima (see Smith and Wessel, 1990), and you may wish to use $t > 0$ to suppress these effects. Experience suggests $t \sim 0.25$ usually looks good for potential field data and t should be larger ($t \sim 0.35$) for steep topography data. $t = 1$ gives a harmonic surface (no maxima or minima are possible except at control data points). It is recommended that the user preprocess the data with `pygmt.blockmean`, `pygmt.blockmedian`, or `pygmt.blockmode` to avoid

spatial aliasing and eliminate redundant data. You may impose lower and/or upper bounds on the solution. These may be entered in the form of a fixed value, a grid with values, or simply be the minimum/maximum input data values. Natural boundary conditions are applied at the edges, except for geographic data with 360-degree range where we apply periodic boundary conditions in the longitude direction.

Takes a matrix, (x, y, z) triplets, or a file name as input.

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/surface.html.

Reference: Smith, W. H. F., & Wessel, P. (1990). Gridding with continuous curvature splines in tension. *Geophysics*, 55(3), 293-305. https://doi.org/10.1190/1.1442837

Aliases

- C = convergence
- Ll = lower
- Lu = upper
- M = max_radius
- T = tension
- a = aspatial
- b = binary
- d = nodata
- e = find
- h = header
- i = incols
- w = wrap
- G = outgrid
- I = spacing
- R = region
- V = verbose
- f = coltypes
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **x/y/z** (*1-D arrays*) – Arrays of x and y coordinates and values z of the data points.
- **spacing** (`Sequence`[`float` | `str`] | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.
- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **convergence** (*float*) – Optional. Convergence limit. Iteration is assumed to have converged when the maximum absolute change in any grid value is less than `convergence`. (Units same as data z units). Alternatively, give limit in percentage of root-mean-square (rms) deviation by appending %. [Default is scaled to 10^{-4} of the rms deviation of the data from a best-fit (least-squares) plane.] This is the final convergence limit at the desired grid spacing; for intermediate (coarser) grids the effective convergence limit is divided by the grid spacing multiplier.
- **max_radius** (*float or str*) – Optional. After solving for the surface, apply a mask so that nodes farther than `max_radius` away from a data constraint are set to NaN [Default is no masking]. Append a *distance unit* if needed. One can also select the nodes to mask by using the *n_cells***c** form. Here *n_cells* means the number of cells around the node is controlled by a data point. As an example `"0c"` means that only the cell where the point lies is filled, `"1c"` keeps one cell beyond that (i.e. makes a 3x3 square neighborhood), and so on.
- **lower** (*float or str*) – Optional. Impose limits on the output solution. Parameter `lower` sets the lower bound. `lower` can be the name of a grid file with lower bound values, a fixed value, **d** to set to minimum input value, or **u** for unconstrained [Default]. Grid files used to set the limits may contain NaNs. In the presence of NaNs, the limit of a node masked with NaN is unconstrained.
- **upper** (*float or str*) – Optional. Impose limits on the output solution. Parameter `upper` sets the upper bound and can be the name of a grid file with upper bound values, a fixed value, **d** to set to maximum input value, or **u** for unconstrained [Default]. Grid files used to set the limits may contain NaNs. In the presence of NaNs, the limit of a node masked with NaN is unconstrained.
- **tension** (*float or str*) – [**b**|**i**]. Optional. Tension factor[s]. These must be between 0 and 1. Tension may be used in the interior solution (above equation, where it suppresses spurious oscillations) and in the boundary conditions (where it tends to flatten the solution approaching the edges). Add **i***tension* to set interior tension, and **b***tension* to set boundary tension. If you do not prepend **i** or **b**, both will be set to the same value. [Default is 0 for both and gives minimum curvature solution.]
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)

- **L**: uint64_t (8-byte unsigned int)
- **f**: 4-byte single-precision float
- **d**: 8-byte double-precision float
- **x**: use to skip *ncols* anywhere in the record

For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:

- **w** after any item to force byte-swapping.
- **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying

columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:

  * **+l** to take the *log10* of the input values.
  * **+d** to divide the input values by the factor *divisor* [Default is 1].
  * **+s** to multiple the input values by the factor *scale* [Default is 1].
  * **+o** to add the given *offset* to the input values [Default is 0].

- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
`DataArray | None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray`: if `outgrid` is not set
- None if `outgrid` is set (grid output will be stored in file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a sample table of topography
>>> topography = pygmt.datasets.load_sample_data(name="notre_dame_topography")
>>> # Perform gridding of topography data
>>> grid = pygmt.surface(data=topography, spacing=1, region=[0, 4, 0, 8])
```

Examples using `pygmt.surface`

pygmt.triangulate

class pygmt.**triangulate**

Delaunay triangulation or Voronoi partitioning and gridding of Cartesian data.

Triangulate reads in x,y[,z] data and performs Delaunay triangulation, i.e., it finds how the points should be connected to give the most equilateral triangulation possible. If a map projection (give `region` and `projection`) is chosen then it is applied before the triangulation is calculated. By default, the output is triplets of point id numbers that make up each triangle. The id numbers refer to the points position (line number, starting at 0 for the first line) in the input file. If `outgrid` and `spacing` are set a grid will be calculated based on the surface defined by the planar triangles. The actual algorithm used in the triangulations is either that of Watson [1982] or Shewchuk [1996] [Default is Shewchuk if installed; type `gmt get GMT_TRIANGULATE` on the command line to see which method is selected]. Furthermore, if the Shewchuk algorithm is installed then you can also perform the calculation of Voronoi polygons and optionally grid your data via the natural nearest neighbor algorithm.

Note

For geographic data with global or very large extent you should consider sphtriangulate instead since `triangulate` is a Cartesian or small-geographic area operator and is unaware of periodic or polar boundary conditions.

Methods Summary

triangulate.delaunay_triples([data, x, y, ...])	Delaunay triangle based gridding of Cartesian data.
triangulate.regular_grid([data, x, y, z, ...])	Delaunay triangle based gridding of Cartesian data.

pygmt.triangulate.delaunay_triples

static triangulate.**delaunay_triples**(*data=None*, *x=None*, *y=None*, *z=None*, ** (Keyword-only parameters separator (PEP 3102))*, *output_type='pandas'*, *outfile=None*, *spacing=None*, *projection=None*, *region=None*, *verbose=False*, *incols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Delaunay triangle based gridding of Cartesian data.

Reads in x,y[,z] data and performs Delaunay triangulation, i.e., it finds how the points should be connected to give the most equilateral triangulation possible. If a map projection (give `region` and `projection`) is chosen then it is applied before the triangulation is calculated. The actual algorithm used in the triangulations is either that of Watson [1982] or Shewchuk [1996] [Default if installed; type `gmt get GMT_TRIANGULATE` on the command line to see which method is selected).

Must provide either `data` or `x`, `y`, and `z`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/triangulate.html

Aliases

- b = binary
- d = nodata
- e = find
- h = header
- s = skiprows
- w = wrap
- I = spacing
- J = projection
- R = region
- V = verbose
- f = coltypes
- i = incols
- r = registration

Parameters

- **x/y/z** (numpy.ndarray) – Arrays of x and y coordinates and values z of the data points.

- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data.
- **output_type** (Literal['pandas', 'numpy', 'file'], default: 'pandas') – Desired output type of the result data.
  - pandas will return a pandas.DataFrame object.
  - numpy will return a numpy.ndarray object.
  - file will save the result to the file specified by the outfile parameter.
- **outfile** (str | PathLike | None, default: None) – File name for saving the result data. Required if output_type="file". If specified, output_type will be forced to be "file".
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, nodata="-9999" will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.

- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence[int | str]` | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:
  - **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.

  - **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].

- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
DataFrame | ndarray | None

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in the file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Note

For geographic data with global or very large extent you should consider sphtriangulate instead since `triangulate` is a Cartesian or small-geographic area operator and is unaware of periodic or polar boundary conditions.

pygmt.triangulate.regular_grid

static triangulate.**regular_grid**(*data=None*, *x=None*, *y=None*, *z=None*, *outgrid=None*, *spacing=None*, *projection=None*, *region=None*, *verbose=False*, *incols=None*, *registration=False*, *coltypes=None*, ***kwargs*)

Delaunay triangle based gridding of Cartesian data.

Reads in x,y[,z] data and performs Delaunay triangulation, i.e., it finds how the points should be connected to give the most equilateral triangulation possible. If a map projection (give `region` and `projection`) is chosen then it is applied before the triangulation is calculated. By setting `outgrid` and `spacing`, a grid will be calculated based on the surface defined by the planar triangles. The actual algorithm used in the triangulations is either that of Watson [1982] or Shewchuk [1996] [Default is Shewchuk if installed; type `gmt get GMT_TRIANGULATE` on the command line to see which method is selected]. This choice is made during the GMT installation. Furthermore, if the Shewchuk algorithm is installed then you can also perform the calculation of Voronoi polygons and optionally grid your data via the natural nearest neighbor algorithm.

Must provide either `data` or `x`, `y`, and `z`.

Must provide `region` and `spacing`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/triangulate.html.

Aliases

- b = binary
- d = nodata
- e = find
- h = header
- s = skiprows
- w = wrap
- G = outgrid
- I = spacing
- J = projection
- R = region
- V = verbose
- f = coltypes
- i = incols
- r = registration

Parameters

- **x/y/z** (numpy.ndarray) – Arrays of x and y coordinates and values z of the data points.
- **data** (str | PathLike | dict | ndarray | DataFrame | Dataset | GeoDataFrame | None, default: None) – Pass in (x, y[, z]) or (longitude, latitude[, elevation]) values by providing a file name to an ASCII data table, a 2-D numpy.ndarray, a pandas.DataFrame, an xarray.Dataset made up of 1-D xarray.DataArray data variables, or a geopandas.GeoDataFrame containing the tabular data.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the registration, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If region=grdfile is used then the grid spacing and the registration have already been initialized; use spacing and registration to override these values.
- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers. The interpolation is performed in the original coordinates, so if your triangles are close to the poles you are better off projecting all data to a local coordinate system before using triangulate (this is true of all gridding routines) or instead select sphtriangulate.
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:

  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:

  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.

- **incols** (int | str | Sequence[int | str] | None, default: None) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.
- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:
  - **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.
  - **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
DataArray | None

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is `None` [Default]
- `None` if `outgrid` is a str (grid output is stored in `outgrid`)

Note

For geographic data with global or very large extent you should consider sphtriangulate instead since `triangulate` is a Cartesian or small-geographic area operator and is unaware of periodic or polar boundary conditions.

pygmt.xyz2grd

`pygmt.`**`xyz2grd`**(*data=None*, *x=None*, *y=None*, *z=None*, *outgrid=None*, *spacing=None*, *projection=None*, *region=None*, *verbose=False*, *registration=False*, *coltypes=None*, ***kwargs*)

Convert data table to a grid.

Reads one or more tables with *x, y, z* columns and creates a binary grid file. *`pygmt.xyz2grd`* will report if some of the nodes are not filled in with data. Such unconstrained nodes are set to a value specified by the user [Default is NaN]. Nodes with more than one value will be set to the mean value.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/xyz2grd.html.

Aliases

- A = duplicate
- Z = convention
- b = binary
- d = nodata
- e = find
- h = header
- i = incols
- w = wrap
- G = outgrid
- I = spacing
- J = projection
- R = region
- V = verbose
- f = coltypes
- r = registration

Parameters

- **`data`** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in (x, y, z) or (longitude, latitude, elevation) values by providing a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.
- **`x/y/z`** (*`1-D arrays`*) – The arrays of x and y coordinates and z data points.
- **`outgrid`** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **`duplicate`** (*`str`*) – [**d**|**f**|**l**|**m**|**n**|**r**|**S**|**s**|**u**|**z**]. By default we will calculate mean values if multiple entries fall on the same node. Use `duplicate` to change this behavior, except it is ignored if `convention` is given. Append **f** or **s** to simply keep the first or last data point that was assigned to each node. Append **l** or **u** or **d** to find the lowest (minimum) or upper (maximum) value or the difference between the maximum and minimum values at each node, respectively. Append **m** or **r** or **S** to compute mean or RMS value or standard deviation at each node, respectively. Append **n** to simply count the number of data points that were assigned to each node (this only requires two input columns *x* and *y* as *z* is not consulted). Append **z** to sum multiple values that belong to the same node.

- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the registration, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If region=grdfile is used then the grid spacing and the registration have already been initialized; use spacing and registration to override these values.
- **convention** (*str*) – [*flags*]. Read a 1-column ASCII [or binary] table. This assumes that all the nodes are present and sorted according to specified ordering convention contained in *flags*. If incoming data represents rows, make *flags* start with **T**(op) if first row is y = ymax or **B**(ottom) if first row is y = ymin. Then, append **L** or **R** to indicate that first element is at left or right end of row. Likewise for column formats: start with **L** or **R** to position first column, and then append **T** or **B** to position first element in a row. **Note**: These two row/column indicators are only required for grids; for other tables they do not apply. For gridline registered grids: If data are periodic in x but the incoming data do not contain the (redundant) column at x = xmax, append **x**. For data periodic in y without redundant row at y = ymax, append **y**. Append **s***n* to skip the first *n* number of bytes (probably a header). If the byte-order or the words needs to be swapped, append **w**. Select one of several data types (all binary except **a**):
  - **A** ASCII representation of one or more floating point values per record
  - **a** ASCII representation of a single item per record
  - **c** int8_t, signed 1-byte character
  - **u** uint8_t, unsigned 1-byte character
  - **h** int16_t, signed 2-byte integer
  - **H** uint16_t, unsigned 2-byte integer
  - **i** int32_t, signed 4-byte integer
  - **I** uint32_t, unsigned 4-byte integer
  - **l** int64_t, long (8-byte) integer
  - **L** uint64_t, unsigned long (8-byte) integer
  - **f** 4-byte floating point single precision
  - **d** 8-byte floating point double precision

  [Default format is scanline orientation of ASCII numbers: **La**]. The difference between **A** and **a** is that the latter can decode both *date***T***clock* and *ddd:mm:ss[.xx]* formats but expects each input record to have a single value, while the former can handle multiple values per record

but can only parse regular floating point values. Translate incoming *z*-values via the `incols` parameter.

- **`projection`** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **`verbose`** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **`binary`** (*`bool or str`*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using `binary="i"`) or output (using `binary="o"`), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **`nodata`** (*`str`*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **`find`** (*`str`*) – [**~**]*"pattern"* | [**~**]*/regexp/*[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **`coltypes`** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **`header`** (*`str`*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:

- **+d** to remove existing header records.
- **+c** to add a header comment with column names to the output [Default is no column names].
- **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
- **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
- **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

Blank lines and lines starting with # are always skipped.

- **incols** – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
DataArray | None

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray`: if `outgrid` is not set
- None if `outgrid` is set (grid output will be stored in file set by `outgrid`)

Examples

```
>>> import numpy as np
>>> import pygmt
>>> # generate a grid for z=x**2+y**2, with an x-range of 0 to 3,
>>> # and a y-range of 10.5 to 12.5. The x- and y-spacings are 1.0 and 0.5.
>>> x, y = np.meshgrid([0, 1, 2, 3], [10.5, 11.0, 11.5, 12.0, 12.5])
>>> z = x**2 + y**2
>>> xx, yy, zz = x.flatten(), y.flatten(), z.flatten()
>>> grid = pygmt.xyz2grd(
...     x=xx, y=yy, z=zz, spacing=(1.0, 0.5), region=[0, 3, 10, 13]
... )
```

Examples using `pygmt.xyz2grd`

11.2.2 Operations on raster data

dimfilter(grid[, outgrid, spacing, region, ...])	Directional filtering of grids in the space domain.
grd2xyz(grid[, output_type, outfile, ...])	Convert grid to data table.
grdclip(grid[, outgrid, above, below, ...])	Clip the range of grid values.
grdcut(grid[, kind, outgrid, projection, ...])	Extract subregion from a grid or image or a slice from a cube.
grdfill(grid[, outgrid, constant_fill, ...])	Interpolate across holes in a grid.
grdfilter(grid[, outgrid, filter, width, ...])	Filter a grid in the space (or time) domain.
grdgradient(grid[, outgrid, azimuth, ...])	Compute directional gradients from a grid.
grdhisteq()	Perform histogram equalization for a grid.
grdhisteq.equalize_grid(grid[, outgrid, ...])	Perform histogram equalization for a grid.
grdhisteq.compute_bins(grid[, output_type, ...])	Perform histogram equalization for a grid.
grdlandmask([outgrid, spacing, mask_values, ...])	Create a "wet-dry" mask grid from shoreline database.
grdmask(data[, outgrid, spacing, region, ...])	Create mask grid from polygons or point coverage.
grdpaste(grid1, grid2[, outgrid, verbose, ...])	Join two grids along their common edge.
grdproject(grid[, outgrid, center, spacing, ...])	Forward and inverse map transformation of grids.
grdsample(grid[, outgrid, toggle, spacing, ...])	Resample a grid onto a new lattice.
grdtrack(grid[, points, output_type, ...])	Sample one or more grids at specified locations.
grdvolume(grid[, output_type, outfile, ...])	Calculate grid volume and area constrained by a contour.

pygmt.dimfilter

pygmt.**dimfilter**(*grid*, *outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, ***kwargs*)

Directional filtering of grids in the space domain.

Filter a grid in the space (or time) domain by dividing the given filter circle into the given number of sectors, applying one of the selected primary convolution or non-convolution filters to each sector, and choosing the final

outcome according to the selected secondary filter. It computes distances using Cartesian or Spherical geometries. The output grid can optionally be generated as a subregion of the input and/or with a new increment using `spacing`, which may add an "extra space" in the input data to prevent edge effects for the output grid. If the filter is low-pass, then the output may be less frequently sampled than the input. *`pygmt.dimfilter`* will not produce a smooth output as other spatial filters do because it returns a minimum median out of *N* medians of *N* sectors. The output can be rough unless the input data are noise-free. Thus, an additional filtering (e.g., Gaussian via *`pygmt.grdfilter`*) of the DiM-filtered data is generally recommended.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/dimfilter.html.

Reference: Kim, S.-S., & Wessel, P. (2008). Directional median filtering for regional-residual separation of bathymetry. *Geochemistry, Geophysics, Geosystems*, 9(3), 2007GC001850. https://doi.org/10.1029/2007GC001850

Aliases

- D = distance
- F = filter
- N = sectors
- G = outgrid
- I = spacing
- R = region
- V = verbose

Parameters

- **`grid`** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **`outgrid`** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **`distance`** (*`int or str`*) – Distance flag tells how grid (x,y) relates to filter width, as follows:

  – **0**: grid (x,y) in same units as *width*, Cartesian distances.

  – **1**: grid (x,y) in degrees, *width* in kilometers, Cartesian distances.

  – **2**: grid (x,y) in degrees, *width* in km, dx scaled by cos(middle y), Cartesian distances.

  The above options are fastest because they allow weight matrix to be computed only once. The next two options are slower because they recompute weights for each latitude.

  – **3**: grid (x,y) in degrees, *width* in km, dx scaled by cosine(y), Cartesian distance calculation.

  – **4**: grid (x,y) in degrees, *width* in km, Spherical distance calculation.

- **`filter`** (*`str`*) – **x***width*[**+l**|**u**]. Set the primary filter type. Choose among convolution and non-convolution filters. Use the filter code **x** followed by the full diameter *width*. Available convolution filters are:

  – **b**: boxcar. Aall weights are equal.

  – **c**: cosine arch. Weights follow a cosine arch curve.

  – **g**: Gaussian. Weights are given by the Gaussian function.

  Non-convolution filters are:

  – **m**: median. Returns median value.

  - **p**: maximum likelihood probability (a mode estimator). Return modal value. If more than one mode is found we return their average value. Append **+l** or **+h** to the filter width if you want to return the smallest or largest of each sector's modal values.

- **sectors** (*str*) – **x***sectors*[**+l**|**u**] Set the secondary filter type **x** and the number of bow-tie sectors. *sectors* must be integer and larger than 0. When *sectors* is set to 1, the secondary filter is not effective. Available secondary filters **x** are:
  - **l**: lower. Return the minimum of all filtered values.
  - **u**: upper. Return the maximum of all filtered values.
  - **a**: average. Return the mean of all filtered values.
  - **m**: median. Return the median of all filtered values.
  - **p**: mode. Return the mode of all filtered values. If more than one mode is found we return their average value. Append **+l** or **+h** to the sectors if you rather want to return the smallest or largest of the modal values.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc* [and optionally *y_inc*] is the output increment. Append **m** to indicate minutes, or **c** to indicate seconds. If the new *x_inc*, *y_inc* are **not** integer multiples of the old ones (in the input data), filtering will be considerably slower. [Default is same as the input.]
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*]. Define the region of the output points [Default is the same as the input].
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Return type

DataArray | None

Returns

ret – Return type depends on whether the outgrid parameter is set:

- xarray.DataArray if outgrid is not set
- None if outgrid is set (grid output will be stored in the file set by outgrid)

Examples

```
>>> import pygmt
>>> # Load a grid of Earth relief data
>>> grid = pygmt.datasets.load_earth_relief()
>>> # Create a filtered grid from an input grid.
>>> filtered_grid = pygmt.dimfilter(
...     grid=grid,
...     # Set filter type to "median" and the diameter width to 600 km
...     filter="m600",
...     # Set grid in degrees, width in km
...     distance=4,
...     # Create 6 sectors and return the lowest values in the sector
...     sectors="l6",
...     # Set the region longitude range from 55W to 51W, and the
...     # latitude range from 24S to 19S
...     region=[-55, -51, -24, -19],
... )
```

pygmt.grd2xyz

pygmt.**grd2xyz**(*grid*, *output_type='pandas'*, *outfile=None*, *region=None*, *verbose=False*, *outcols=None*, *coltypes=None*, ***kwargs*)

Convert grid to data table.

Read a grid and output xyz-triplets as a `numpy.ndarray`, `pandas.DataFrame`, or ASCII file.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grd2xyz.html.

Aliases

- C = cstyle
- W = weight
- Z = convention
- b = binary
- d = nodata
- h = header
- s = skiprows
- R = region
- V = verbose
- f = coltypes
- o = outcols

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.

  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.

- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.

- **cstyle** (*str*) – [**f**|**i**]. Replace the x- and y-coordinates on output with the corresponding column and row numbers. These start at 0 (C-style counting); append **f** to start at 1 (Fortran-style counting). Alternatively, append **i** to write just the two columns *index* and *z*, where *index* is the 1-D indexing that GMT uses when referring to grid nodes.

- **weight** (*str*) – [**a**[**+u***unit*]|*weight*]. Write out *x,y,z,w*, where *w* is the supplied *weight* (or 1 if not supplied) [Default writes *x,y,z* only]. Choose **a** to compute weights equal to the area each node represents. For Cartesian grids this is simply the product of the *x* and *y* increments (except for gridline-registered grids at all sides [half] and corners [quarter]). For geographic grids we default to a length unit of **k**. Change this by appending **+u***unit* (see *Distance Units*). For such grids, the area varies with latitude and also sees special cases for gridline-registered layouts at sides, corners, and poles.

- **convention** (*str*) – [*flags*]. Write a 1-column ASCII [or binary] table. Output will be organized according to the specified ordering convention contained in *flags*. If data should be written by rows, make *flags* start with **T** (op) if first row is y = ymax or **B** (ottom) if first row is y = ymin. Then, append **L** or **R** to indicate that first element should start at left or right end of row. Likewise for column formats: start with **L** or **R** to position first column, and then append **T** or **B** to position first element in a row. For gridline registered grids: If grid is periodic in x but the written data should not contain the (redundant) column at x = xmax, append **x**. For grid periodic in y, skip writing the redundant row at y = ymax by appending **y**. If the byte-order needs to be swapped, append **w**. Select one of several data types (all binary except **a**):

  - **a**: ASCII representation of a single item per record
  - **c**: int8_t, signed 1-byte character
  - **u**: uint8_t, unsigned 1-byte character
  - **h**: int16_t, short 2-byte integer
  - **H**: uint16_t, unsigned short 2-byte integer
  - **i**: int32_t, 4-byte integer
  - **I**: uint32_t, unsigned 4-byte integer
  - **l**: int64_t, long (8-byte) integer
  - **L**: uint64_t, unsigned long (8-byte) integer
  - **f**: 4-byte floating point single precision
  - **d**: 8-byte floating point double precision

  Default format is scanline orientation of ASCII numbers: **TLa**.

- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest. Adding region will select a subsection of the grid. If this subsection exceeds the boundaries of the grid, only the common region will be output.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **binary** (*bool or str*) – **i|o**[*ncols*][*type*][**w**][**+l|b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l|b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.

- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, `nodata="-9999"` will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **outcols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

    Optionally, append one of the following modifiers to any column or column range to transform the output columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the output values by the factor *divisor* [Default is 1].
    - **+s** to multiply the output values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the output values [Default is 0].
- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data

column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:

- **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.
- **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].

Return type

`DataFrame` | `ndarray` | `None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in the file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a pandas.DataFrame with the xyz data from an input grid
>>> xyz_dataframe = pygmt.grd2xyz(grid=grid, output_type="pandas")
>>> xyz_dataframe.head(n=2)
    lon   lat           z
0  10.0  25.0      965.5
1  10.5  25.0      876.5
```

Examples using `pygmt.grd2xyz`

pygmt.grdclip

`pygmt.`**`grdclip`**(*grid*, *outgrid=None*, *above=None*, *below=None*, *between=None*, *replace=None*, *region=None*, *verbose=False*, ***kwargs*)

Clip the range of grid values.

This function operates on the values of a grid. It can:

- Set values smaller than a threshold to a new value
- Set values larger than a threshold to a new value
- Set values within a range to a new value
- Replace individual values with a new value

Such operations are useful when you want all of a continent or an ocean to fall into one color or gray shade in image processing, when clipping the range of data values is required, or for reclassification of data values. The values can be any number or NaN (Not a Number).

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdclip.html.

Aliases

- G = outgrid
- R = region
- Sa = above
- Sb = below
- Si = between
- Sr = replace
- V = verbose

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **above** (`Sequence[float]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*high*, *above*), to set all node values greater than *high* to *above*.

- **below** (`Sequence[float]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*low*, *below*) to set all node values less than *low* to *below*.

- **between** (`Sequence[float]` | `Sequence[Sequence[float]]` | `None`, default: `None`) – Pass a sequence of three values in the form of (*low*, *high*, *between*) to set all node values between *low* and *high* to *between*. It can also accept a sequence of sequences (e.g., list of lists or 2-D numpy array) to set different values for different ranges.

- **replace** (`Sequence[float]` | `Sequence[Sequence[float]]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*old*, *new*) to replace all node values equal to *old* with *new*. It can also accept a sequence of sequences (e.g., list of lists or 2-D numpy array) to replace different old values with different new values. This is mostly useful when your data are known to be integer values.

- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load the 30 arc-minutes Earth relief grid, with a longitude range of 10° E␣
↪to
>>> # 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
```

(continues on next page)

(continued from previous page)

```
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Report the minimum and maximum data values
>>> [grid.data.min(), grid.data.max()]
[np.float32(183.5), np.float32(1807.0)]
>>> # Create a new grid from an input grid. Set all values below 1,000 to 0 and␣
↪all
>>> # values above 1,500 to 10,000
>>> new_grid = pygmt.grdclip(grid=grid, below=[1000, 0], above=[1500, 10000])
>>> # Report the minimum and maximum data values
>>> [new_grid.data.min(), new_grid.data.max()]
[np.float32(0.0), np.float32(10000.0)]
```

Examples using `pygmt.grdclip`

pygmt.grdcut

pygmt.**grdcut**(*grid*, *kind='grid'*, *outgrid=None*, *projection=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Extract subregion from a grid or image or a slice from a cube.

Produce a new `outgrid` file which is a subregion of `grid`. The subregion is specified with `region`; the specified range must not exceed the range of `grid` (but see `extend`). If in doubt, run *`pygmt.grdinfo`* to check range. Alternatively, define the subregion indirectly via a range check on the node values or via distances from a given point. Finally, you can give `projection` for oblique projections to determine the corresponding rectangular `region` that will give a grid that fully covers the oblique domain.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdcut.html.

Aliases

- N = extend
- S = circ_subregion
- Z = z_subregion
- G = outgrid
- J = projection
- R = region
- V = verbose
- f = coltypes

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **kind** (`Literal['grid', 'image']`, default: `'grid'`) – The raster data kind. Valid values are `"grid"` and `"image"`. When the input `grid` is a file name, it's difficult to determine if the file is a grid or an image, so we need to specify the raster kind explicitly. The default is `"grid"`.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **extend** (*bool or float*) – Allow grid to be extended if new `region` exceeds existing boundaries. Give a value to initialize nodes outside current region.

- **circ_subregion** (*str*) – *lon/lat/radius*[*unit*][**+n**]. Specify an origin (*lon* and *lat*) and *radius*; append a distance *unit* and we determine the corresponding rectangular region so that all grid nodes on or inside the circle are contained in the subset. If **+n** is appended we set all nodes outside the circle to NaN.
- **z_subregion** (*str*) – [*min/max*][**+n**|**N**|**r**]. Determine a new rectangular region so that all nodes outside this region are also outside the given z-range [-inf/+inf]. To indicate no limit on *min* or *max* only, specify a hyphen (-). Normally, any NaNs encountered are simply skipped and not considered in the range-decision. Append **+n** to consider a NaN to be outside the given z-range. This means the new subset will be NaN-free. Alternatively, append **+r** to consider NaNs to be within the data range. In this case we stop shrinking the boundaries once a NaN is found [Default simply skips NaNs when making the range decision]. Finally, if your core subset grid is surrounded by rows and/or columns that are all NaNs, append **+N** to strip off such columns before (optionally) considering the range of the core subset for further reduction of the area.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, with a longitude range of
>>> # 12° E to 15° E and a latitude range of 21° N to 24° N
>>> new_grid = pygmt.grdcut(grid=grid, region=[12, 15, 21, 24])
```

pygmt.grdfill

pygmt.**grdfill**(*grid*, *outgrid=None*, *constant_fill=None*, *grid_fill=None*, *neighbor_fill=None*, *spline_fill=None*, *inquire=False*, *hole=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Interpolate across holes in a grid.

Read a grid that presumably has unfilled holes that the user wants to fill in some fashion. Holes are identified by NaN values but this criteria can be changed via the `hole` parameter. There are several different algorithms that can be used to replace the hole values. If no holes are found the original unchanged grid is returned.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdfill.html.

Aliases

- Ac = constant_fill
- Ag = grid_fill
- An = neighbor_fill
- As = spline_fill
- G = outgrid
- L = inquire
- N = hole
- R = region
- V = verbose
- f = coltypes

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **constant_fill** (`float` | `None`, default: `None`) – Fill the holes with a constant value. Specify the constant value to use.

- **grid_fill** (`str` | `PathLike` | `DataArray` | `None`, default: `None`) – Fill the holes with values sampled from another (possibly coarser) grid. Specify the grid (a file name or an `xarray.DataArray`) to use for the fill.

- **neighbor_fill** (`float` | `bool` | `None`, default: `None`) – Fill the holes with the nearest neighbor. Specify the search radius in pixels. If set to `True`, the default search radius will be used ($r^2 = \sqrt{n^2 + m^2}$, where (*n,m*) are the node dimensions of the grid).

- **spline_fill** (`float` | `bool` | `None`, default: `None`) – Fill the holes with a bicubic spline. Specify the tension value to use. If set to `True`, no tension will be used.

- **hole** (`float` | `None`, default: `None`) – Set the node value used to identify a point as a member of a hole [Default is NaN].

- **inquire** (`bool`, default: `False`) – Output the bounds of each hole. The bounds are returned as a 2-D numpy array in the form of (west, east, south, north). No grid fill takes place and `outgrid` is ignored.

- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].

- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

`DataArray` | `ndarray` | `None`

Returns

ret – If `inquire` is `True`, return the bounds of each hole as a 2-D numpy array. Otherwise, the return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

Fill holes in a bathymetric grid with a constant value of 20.

```
>>> import pygmt
>>> # Load a bathymetric grid with missing data
>>> earth_relief_holes = pygmt.datasets.load_sample_data(name="earth_relief_holes
↪")
>>> # Fill the holes with a constant value of 20
>>> filled_grid = pygmt.grdfill(grid=earth_relief_holes, constant_fill=20)
```

Inquire the bounds of each hole.

```
>>> pygmt.grdfill(grid=earth_relief_holes, inquire=True)
array([[1.83333333, 6.16666667, 3.83333333, 8.16666667],
       [6.16666667, 7.83333333, 0.5       , 2.5       ]])
```

pygmt.grdfilter

pygmt.**grdfilter**(*grid*, *outgrid=None*, *filter=None*, *width=None*, *highpass=False*, *distance=None*, *spacing=None*, *nans=None*, *toggle=False*, *region=None*, *verbose=False*, *registration=False*, *cores=False*, *coltypes=None*, ***kwargs*)

Filter a grid in the space (or time) domain.

Filter a grid file in the space (or time) domain using one of the selected convolution or non-convolution isotropic or rectangular filters and compute distances using Cartesian or Spherical geometries. The output grid file can optionally be generated as a sub-region of the input (via `region`) and/or with new increment (via `spacing`) or registration (via `toggle`). In this way, one may have "extra space" in the input data so that the edges will not be used and the output can be within one half-width of the input edges. If the filter is low-pass, then the output may be less frequently sampled than the input.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdfilter.html.

Aliases

- D = distance
- F = filter, width, **+h**: highpass
- G = outgrid
- I = spacing
- N = nans
- R = region
- T = toggle
- V = verbose
- f = coltypes
- r = registration
- x = cores

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **filter** (Literal['boxcar', 'cosarch', 'gaussian', 'minall', 'minpos', 'maxall', 'maxneg'] | str | None, default: None) – The filter type. Choose among convolution and non-convolution filters.

  Convolution filters include:

  - "boxcar": All weights are equal.
  - "cosarch": Weights follow a cosine arch curve.
  - "gaussian": Weights are given by the Gaussian function, where filter width is 6 times the conventional Gaussian sigma.

  Non-convolution filters include:

  - "minall": Return minimum of all values.
  - "minpos": Return minimum of all positive values only.
  - "maxall": Return maximum of all values.
  - "maxneg": Return maximum of all negative values only.

  Note: There are still a few other filter types available in GMT (e.g., histogram and mode filters), but they are not implemented in PyGMT yet. As a workaround, pass the raw GMT command string to this parameter to use these other filter types. Refer to https://docs.generic-mapping-tools.org/6.7/grdfilter.html#f for the full syntax of this parameter.
- **width** (float | Sequence[float] | None, default: None) – The full diameter width of the filter. It can be a single value for an isotropic filter, or a pair of values for a rectangular filter (width in x- and y-directions, requiring distance be either "pixel" or "cartesian").
- **highpass** (bool, default: False) – By default, the filter is a low-pass filter. If True, then the filter is a high-pass filter. [Default is False].
- **distance** (Literal['pixel', 'cartesian', 'geo_cartesian', 'geo_flatearth1', 'geo_flatearth2', 'geo_spherical', 'geo_mercator'] | None, default: None) – Determine how grid (*x, y*) relates to filter *width* and how distances are calculated. Valid values are list below. The first four options are fastest because they allow weight matrix to be computed only once. The last three options are slower because they recompute weights for each latitude.

Value	Grid (x,y)	Width	Distance Calculation
"pixel"	Pixels (px, py)	Odd number of pixels	Cartesian
"cartesian"	Same units as *width*	Any	Cartesian
"geo_carte	Degrees	km	Cartesian
"geo_flate	Degrees	km	Cartesian, dx scaled by cos(middle y)
"geo_flate	Degrees	km	Cartesian, dx scaled by cos(y) per row
"geo_spher	Degrees	km	Spherical (great circle)
"geo_merca	Mercator **-Jm1** img units	km	Spherical

- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the registration, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If region=grdfile is used then the grid spacing and the registration have already been initialized; use spacing and registration to override these values.
- **nans** (Literal['ignore', 'replace', 'preserve'] | None, default: None) – Determine how NaN-values in the input grid affect the filtered output grid. Choose one of:
  - "ignore": Ignore all NaNs in the calculation of filtered value [Default].
  - "replace": Similar to "ignore" except if the input node was NaN then the output node will be set to NaN (only applied if both grids are co-registered).
  - "preserve": Force the filtered value to be NaN if any grid nodes with NaN-values are found inside the filter circle.
- **toggle** (bool, default: False) – Toggle the node registration for the output grid so as to become the opposite of the input grid [Default gives the same registration as the input grid]. Alternatively, use registration to set the registration explicitly.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.
- **cores** (int | bool, default: False) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type

DataArray | None

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> from pathlib import Path
>>> import pygmt
>>> # Apply a gaussian filter of 600 km (full width) to the @earth_relief_30m_g␣
↪grid
>>> # and return a filtered grid (saved as netCDF file).
>>> pygmt.grdfilter(
...     grid="@earth_relief_30m_g",
...     filter="gaussian",
...     width=600,
...     distance="geo_spherical",
...     region=[150, 250, 10, 40],
...     spacing=0.5,
...     outgrid="filtered_pacific.nc",
... )
>>> Path("filtered_pacific.nc").unlink()  # Cleanup file
>>> # Apply a Gaussian smoothing filter of 600 km to the input DataArray and␣
↪return
>>> # a filtered DataArray with the smoothed grid.
>>> grid = pygmt.datasets.load_earth_relief()
>>> smoothed = pygmt.grdfilter(
...     grid=grid, filter="gaussian", width=600, distance="geo_spherical"
... )
```

pygmt.grdgradient

pygmt.**grdgradient**(*grid*, *outgrid=None*, *azimuth=None*, *radiance=None*, *normalize=False*, *norm_amp=None*, *norm_ambient=None*, *norm_sigma=None*, *norm_offset=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Compute directional gradients from a grid.

Can accept `azimuth`, `direction`, and `radiance` input to create the resulting gradient.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdgradient.html.

Aliases

- D = direction
- Q = tiles
- S = slope_file
- n = interpolation
- A = azimuth
- E = radiance
- G = outgrid
- R = region
- V = verbose
- f = coltypes

- N = normalize, norm_amp, **+a**: norm_ambient, **+s**: norm_sigma, **+o**: norm_offset

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **azimuth** (`float` | `Sequence[float]` | `None`, default: `None`) – *azim* or (*azim*, *azim2*). Azimuthal direction for a directional derivative; *azim* is the angle in the x-y plane measured in degrees positive clockwise from north (the +y direction) toward east (the +x direction). The negative of the directional derivative, $-(\frac{dz}{dx}\sin(\text{azim}) + \frac{dz}{dy}\cos(\text{azim}))$, is found; negation yields positive values when the slope of $z(x, y)$ is downhill in the *azim* direction, the correct sense for shading the illumination of an image by (see *pygmt.Figure.grdimage* and *pygmt.Figure.grdview*) a light source above the x-y plane shining from the *azim* direction. Optionally, supply two azimuths, (*azim*, *azim2*), in which case the gradients in each of these directions are calculated and the one larger in magnitude is retained; this is useful for illuminating data with two directions of lineated structures, e.g., `(0, 270)` illuminates from the north (top) and west (left). Finally, if *azim* is a file it must be a grid of the same domain, spacing and registration as *grid* that will update the azimuth at each output node when computing the directional derivatives.
- **direction** (*str*) – [**a**][**c**][**o**][**n**]. Find the direction of the positive (up-slope) gradient of the data. The following options are supported:
  - **a**: Find the aspect (i.e., the down-slope direction)
  - **c**: Use the conventional Cartesian angles measured counterclockwise from the positive x (east) direction.
  - **o**: Report orientations (0-180) rather than directions (0-360).
  - **n**: Add 90 degrees to all angles (e.g., to give local strikes of the surface).
- **radiance** (`Sequence[float]` | `str` | `None`, default: `None`) – (*azim*, *elev*) or [**m**|**s**|**p**]*azim/elev*[**+a***ambient*] [**+d***diffuse*][**+p***specular*][**+s***shine*]. Compute Lambertian radiance appropriate to use with *pygmt.Figure.grdimage* and *pygmt.Figure.grdview*. The Lambertian Reflection assumes an ideal surface that reflects all the light that strikes it and the surface appears equally bright from all viewing directions. Here, *azim* and *elev* are the azimuth and elevation of the light vector. Optionally, supply *ambient* [0.55], *diffuse* [0.6], *specular* [0.4], or *shine* [10], which are parameters that control the reflectance properties of the surface. Default values are given in the brackets. Use **s** for a simpler Lambertian algorithm. Note that with this form you only have to provide azimuth and elevation. Alternatively, use **p** for the Peucker piecewise linear approximation (simpler but faster algorithm; in this case *azim* and *elev* are hardwired to 315 and 45 degrees. This means that even if you provide other values they will be ignored.).
- **normalize** (`Literal['laplace', 'cauchy']` | `bool`, default: `False`) – Normalize the output gradients. Valid values are:
  - `False`: No normalization is done [Default].
  - `True`: Normalize using max absolute value.
  - `"laplace"`: Normalize using cumulative Laplace distribution.
  - `"cauchy"`: Normalize using cumulative Cauchy distribution.

  The normalization process is controlled via the additional parameters `norm_amp`, `norm_ambient`, `norm_sigma`, and `norm_offset`.

Let g denote the actual gradients, g_n the normalized gradients, a the maximum output magnitude (`norm_amp`), o the offset value (`norm_offset`), and σ the sigma value (`norm_sigma`). The normalization is computed as follows:

- `True`: $g_n = a(\frac{g-o}{\max(|g-o|)})$
- `"laplace"`: $g_n = a(1 - \exp(\sqrt{2}\frac{g-o}{\sigma}))$
- `"cauchy"`: $g_n = \frac{2a}{\pi}\arctan(\frac{g-o}{\sigma})$

- **norm_amp** (`float` | `None`, default: `None`) – Set the maximum output magnitude [Default is 1].
- **norm_ambient** (`float` | `None`, default: `None`) – The ambient value to add to all nodes after gradient calculations are completed [Default is 0].
- **norm_offset** (`float` | `None`, default: `None`) – The offset value used in the normalization. If not given, it is set to the average of g.
- **norm_sigma** (`float` | `None`, default: `None`) – The *sigma* value used in the Laplace or Cauchy normalization. If not given, it is estimated from the L1 norm of $g - o$ for Laplace or the L2 norm of $g - o$ for Cauchy.
- **tiles** (*str*) – **c**|**r**|**R**. Control how normalization via `normalize` is carried out. When multiple grids should be normalized the same way (i.e., with the same *offset* and/or *sigma*), we must pass these values via `normalize`. However, this is inconvenient if we compute these values from a grid. Use **c** to save the results of *offset* and *sigma* to a statistics file; if grid output is not needed for this run then do not specify `outgrid`. For subsequent runs, just use **r** to read these values. Using **R** will read then delete the statistics file.
- **slope_file** (*str*) – Name of output grid file with scalar magnitudes of gradient vectors. Requires `direction` but makes `outgrid` optional.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, set the azimuth to 10 degrees,
>>> new_grid = pygmt.grdgradient(grid=grid, azimuth=10)
```

Examples using `pygmt.grdgradient`

pygmt.grdhisteq

class pygmt.**grdhisteq**

Perform histogram equalization for a grid.

Two common use cases of *pygmt.grdhisteq* are to find data values that divide a grid into patches of equal area (*pygmt.grdhisteq.compute_bins*) or to write a grid with statistics based on some kind of cumulative distribution function (*pygmt.grdhisteq.equalize_grid*).

Histogram equalization provides a way to highlight data that has most values clustered in a small portion of the dynamic range, such as a grid of flat topography with a mountain in the middle. Ordinary gray shading of this grid (using *pygmt.Figure.grdimage* or *pygmt.Figure.grdview*) with a linear mapping from topography to graytone will result in most of the image being very dark gray, with the mountain being almost white. *pygmt.grdhisteq.compute_bins* can provide a list of data values that divide the data range into divisions which have an equal area in the image [Default is 16 if `divisions` is not set]. The `pandas.DataFrame` or ASCII file output can be used to make a colormap with *pygmt.makecpt* and an image with *pygmt.Figure.grdimage* that has all levels of gray occurring equally.

pygmt.grdhisteq.equalize_grid provides a way to write a grid with statistics based on a cumulative distribution function. In this application, the `outgrid` has relative highs and lows in the same (x,y) locations as the `grid`, but the values are changed to reflect their place in the cumulative distribution.

Methods Summary

grdhisteq.compute_bins(grid[, output_type, ...])	Perform histogram equalization for a grid.
grdhisteq.equalize_grid(grid[, outgrid, ...])	Perform histogram equalization for a grid.

pygmt.grdhisteq.compute_bins

static grdhisteq.**compute_bins**(*grid*, *output_type='pandas'*, *outfile=None*, *region=None*, *verbose=False*, ***kwargs*)

Perform histogram equalization for a grid.

Histogram equalization provides a way to highlight data that has most values clustered in a small portion of the dynamic range, such as a grid of flat topography with a mountain in the middle. Ordinary gray shading of this grid (using *pygmt.Figure.grdimage* or *pygmt.Figure.grdview*) with a linear mapping from topography to graytone will result in most of the image being very dark gray, with the mountain being almost white. *pygmt.grdhisteq.compute_bins* can provide a list of data values that divide the data range into divisions which have an equal area in the image [Default is 16 if `divisions` is not set]. The `pandas.DataFrame` or ASCII file output can be used to make a colormap with *pygmt.makecpt* and an image with *pygmt.Figure.grdimage* that has all levels of gray occurring equally.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdhisteq.html.

Aliases

- C = divisions
- N = gaussian
- Q = quadratic
- h = header
- R = region
- V = verbose

Parameters

- **grid** (str | PathLike | DataArray) – Name of the input grid file or the grid loaded as a xarray.DataArray object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **output_type** (Literal['pandas', 'numpy', 'file'], default: 'pandas') – Desired output type of the result data.
  - pandas will return a pandas.DataFrame object.
  - numpy will return a numpy.ndarray object.
  - file will save the result to the file specified by the outfile parameter.
- **outfile** (str | PathLike | None, default: None) – File name for saving the result data. Required if output_type="file". If specified, output_type will be forced to be "file".
- **divisions** (*int*) – Set the number of divisions of the data range.
- **quadratic** (*bool*) – Perform quadratic equalization [Default is linear].
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.

Return type

DataFrame | ndarray | None

Returns

ret – Return type depends on outfile and output_type:

- None if outfile is set (output will be stored in file set by outfile)

- pandas.DataFrame or numpy.ndarray if outfile is not set (depends on output_type)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Find elevation intervals that split the data range into 5
>>> # divisions, each of which have an equal area in the original grid.
>>> bins = pygmt.grdhisteq.compute_bins(grid=grid, divisions=5)
>>> print(bins)
        start    stop
bin_id
0       183.5   395.0
1       395.0   472.0
2       472.0   575.0
3       575.0   709.5
4       709.5  1807.0
```

See also

pygmt.grd2cpt

Note

This method does a weighted histogram equalization for geographic grids to account for node area varying with latitude.

Examples using `pygmt.grdhisteq.compute_bins`

pygmt.grdhisteq.equalize_grid

static grdhisteq.**equalize_grid**(*grid*, *outgrid=None*, *region=None*, *verbose=False*, ***kwargs*)

Perform histogram equalization for a grid.

pygmt.grdhisteq.equalize_grid provides a way to write a grid with statistics based on a cumulative distribution function. The outgrid has relative highs and lows in the same (x,y) locations as the grid, but the values are changed to reflect their place in the cumulative distribution.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdhisteq.html.

Aliases

- C = divisions
- N = gaussian
- Q = quadratic
- h = header
- G = outgrid
- R = region
- V = verbose

Parameters

- **grid** (str | PathLike | DataArray) – Name of the input grid file or the grid loaded as a xarray.DataArray object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **divisions** (*int*) – Set the number of divisions of the data range.

- **gaussian** (*bool or int or float*) – *norm*. Produce an output grid with standard normal scores using gaussian=True or force the scores to fall in the ±*norm* range.

- **quadratic** (*bool*) – Perform quadratic equalization [Default is linear].

- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Return type

DataArray | None

Returns

ret – Return type depends on the outgrid parameter:

- xarray.DataArray if outgrid is None
- None if outgrid is a str (grid output is stored in outgrid)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range
>>> # of 10°E to 30°E, and a latitude range of 15°N to 25°N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid with a Gaussian data distribution
>>> grid = pygmt.grdhisteq.equalize_grid(grid=grid, gaussian=True)
```

See also

pygmt.grd2cpt

Note

This method does a weighted histogram equalization for geographic grids to account for node area varying with latitude.

Examples using `pygmt.grdhisteq.equalize_grid`

pygmt.grdlandmask

pygmt.**grdlandmask**(*outgrid=None*, *spacing=None*, *mask_values=None*, *border_values=None*, *resolution=None*, *area_thresh=None*, *region=None*, *registration=False*, *verbose=False*, *cores=False*, ***kwargs*)

Create a "wet-dry" mask grid from shoreline database.

Read the selected shoreline database and use that information to decide which nodes in the specified grid are over land or over water. The nodes defined by the selected region and lattice spacing will be set according to one of two criteria: (1) land vs water, or (2) the more detailed (hierarchical) ocean vs land vs lake vs island vs pond. A mask grid is created with the specified grid spacing.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdlandmask.html.

Aliases

- A = area_thresh
- D = resolution
- E = border_values
- G = outgrid
- I = spacing
- N = mask_values
- R = region
- V = verbose
- r = registration
- x = cores

Parameters

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **spacing** (Sequence[float | str] | None, default: None) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.
- **area_thresh** (float | str | None, default: None) – *min_area*[/*min_level*/*max_level*][**+a**[**g**|**i**][**s**|**S**]][**+l**|**r**][**+p***percent*]. Features with an area smaller than *min_area* in km^2 or of hierarchical level that is lower than *min_level* or higher than *max_level* will not be plotted [Default is `"0/0/4"` (all features)].
- **resolution** (Literal['auto', 'full', 'high', 'intermediate', 'low', 'crude', None], default: None) – Select the resolution of the coastline dataset to use. The available

resolutions from highest to lowest are: `"full"`, `"high"`, `"intermediate"`, `"low"`, and `"crude"`, which drops by 80% between levels. Alternatively, choose `"auto"` to automatically select the most suitable resolution given the chosen region. Note that because the coastlines differ in details, a node in a mask file using one resolution is not guaranteed to remain inside [or outside] when a different resolution is selected. If `None`, the low resolution is used by default.

- **mask_values** (`Sequence[float] | None`, default: `None`) – Set the values that will be assigned to nodes, in the form of [*wet*, *dry*], or [*ocean*, *land*, *lake*, *island*, *pond*]. Default is `[0, 1, 0, 1, 0]` (i.e., `[0, 1]`), meaning that all "wet" nodes will be assigned a value of 0 and all "dry" nodes will be assigned a value of 1. Values can be any number, or one of `None`, `"NaN"`, and `np.nan` for setting nodes to NaN.

  Use `border_values` to control how nodes on feature boundaries are handled.

- **border_values** (`bool | float | Sequence[float] | None`, default: `None`) – Sets the behavior for nodes that fall exactly on a polygon boundary. Valid values are:

  - `False`: Treat boundary nodes as inside [Default]
  - `True`: Treat boundary nodes as outside
  - A single value: Set all boundary nodes to the same value
  - A sequence of four values in the form of [*cborder*, *lborder*, *iborder*, *pborder*] to treat different kinds of boundary nodes as the specified values. *cborder* is for coastline, *lborder* for lake outline, *iborder* for islands-in-lakes outlines, and *pborder* for ponds-in-islands-in-lakes outlines.

  Values can be any number, or one of `None`, `"NaN"`, and `np.nan` for setting nodes to NaN.

- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.

- **cores** (`int | bool`, default: `False`) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type

`DataArray | None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Create a landmask grid with a longitude range of 125° E to 130° E, a
```

(continues on next page)

(continued from previous page)

```
>>> # latitude range of 30° N to 35° N, and a grid spacing of 1 arc-degree
>>> landmask = pygmt.grdlandmask(spacing=1, region=[125, 130, 30, 35])
```

Examples using `pygmt.grdlandmask`

pygmt.grdmask

pygmt.**grdmask**(*data*, *outgrid=None*, *spacing=None*, *region=None*, *outside=None*, *edge=None*, *inside=None*, *id_start=None*, *verbose=False*, ***kwargs*)

Create mask grid from polygons or point coverage.

Takes one or more polygons or data point coordinates, and creates a grid where nodes that fall inside, on the edge, or outside the polygons or within the search radius from data points are assigned values based on the `outside`, `edge`, and `inside` parameters.

The mask grid can be used to mask out specific regions from another grid. For masking based on coastline features, consider using *`pygmt.grdlandmask`* instead.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdmask.html.

Aliases

- G = outgrid
- I = spacing
- N = outside, edge, inside, id_start
- R = region
- V = verbose

Parameters

- **`data`** – Pass in either a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data containing the polygon(s) or data points. Input can be:
  - **Polygon mode**: One or more polygons with closed coordinates
  - **Point coverage mode**: Data points (used with `search_radius` parameter)
- **`outgrid`** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **`spacing`** (`Sequence[float | str]` | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The

resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **outside** (`float` | `None`, default: `None`)
- **edge** (`float` | `Literal['z', 'id']` | `None`, default: `None`)
- **inside** (`float` | `Literal['z', 'id']` | `None`, default: `None`) – Set the value assigned to nodes outside, on the edge, or inside the polygons. Can be any number, or one of `None`, `"NaN"`, and `np.nan` for NaN. Defaults are `0` for `outside`, `0` for `edge`, and `1` for `inside`. When setting these values, keep in mind you are creating a mask grid which is thought to be applied to a real grid in a second step.

  `edge` and `inside` can also be set to one of the following values:

  – `"z"`: Use the z-values from polygon data (segment header `-Zzval`, `-Lheader`, or via `-aZ=name`).

  – `"id"`: Use a running polygon ID number.

  To treat edges as inside, use the same value as `inside`.
- **id_start** (`float` | `None`, default: `None`) – The starting number for polygon IDs when `inside="id"` [Default is `0`]. Only valid when `inside="id"`.
- **region** (`Sequence[float | str] | str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> import numpy as np
>>> # Create a simple polygon as a triangle
>>> polygon = np.array([[125, 30], [130, 30], [130, 35], [125, 30]])
>>> # Create a mask grid with 1 arc-degree spacing
>>> mask = pygmt.grdmask(data=polygon, spacing=1, region=[125, 130, 30, 35])
>>> mask.values
array([[0., 0., 0., 0., 0., 0.],
       [0., 0., 1., 1., 1., 0.],
       [0., 0., 0., 1., 1., 0.],
       [0., 0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 0., 0.],
       [0., 0., 0., 0., 0., 0.]], dtype=float32)
```

pygmt.grdpaste

pygmt.**grdpaste**(*grid1*, *grid2*, *outgrid=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Join two grids along their common edge.

Combine `grid1` and `grid2` into a single grid by pasting them together along their common edge. The two input grids must have the same grid spacings and registration, and must have one edge in common. If in doubt, check with *`pygmt.grdinfo`* and use *`pygmt.grdcut`* and/or *`pygmt.grdsample`* if necessary to prepare the edge joint. Note: For geographical grids, you may have to use `coltypes` to handle periodic longitudes unless the input grids are properly recognized as such via their meta-data. For stitching multiple grids, see `grdblend` (not implemented in PyGMT yet) instead.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdpaste.html.

Aliases

- G = outgrid
- V = verbose
- f = coltypes

Parameters

- **grid1** (`str` | `PathLike` | `DataArray`)
- **grid2** (`str` | `PathLike` | `DataArray`) – The two grids to be pasted. Accepts either file names or `xarray.DataArray` objects.

  Notes:

  1. Both grids must be of the same type—mixing a file name with an `xarray.DataArray` is not allowed.
  2. Passing two `xarray.DataArray` objects requires GMT>6.6.0 due to an upstream GMT bug.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

pygmt.grdproject

pygmt.**grdproject**(*grid*, *outgrid=None*, *center=False*, *spacing=None*, *dpi=None*, *inverse=False*, *unit=None*, *scaling=False*, *projection=None*, *region=None*, *registration=False*, *verbose=False*, ***kwargs*)

Forward and inverse map transformation of grids.

This method will project a geographical gridded data set onto a rectangular grid. If `inverse` is `True`, it will project a rectangular coordinate system to a geographic system. To obtain the value at each new node, its location is inversely projected back onto the input grid after which a value is interpolated between the surrounding input grid values. By default bi-cubic interpolation is used. Aliasing is avoided by also forward projecting the input grid nodes. If two or more nodes are projected onto the same new node, their average will dominate in the calculation of the new node value. Interpolation and aliasing is controlled with the `interpolation` parameter. The new node spacing may be determined in one of several ways by specifying the grid spacing, number of nodes, or resolution. Nodes not constrained by input data are set to NaN. The `region` parameter can be used to select a plot region larger or smaller than that implied by the extent of the grid file.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdproject.html.

Aliases

- n = interpolation
- C = center
- D = spacing
- E = dpi
- F = scaling
- G = outgrid
- I = inverse
- J = projection
- M = unit
- R = region
- V = verbose
- r = registration

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **center** (`Sequence`[`float` | `str`] | `bool`, default: `False`) – If `True`, let the projected coordinates be relative to the projection center [Default is relative to the lower left corner]. Optionally, set offsets (*dx*, *dy*) in the projected units to be added (or subtracted when `inverse` is set) to (from) the projected coordinates, such as false eastings and northings for particular projection zones [Default is `(0, 0)`].

- **spacing** (`float` | `str` | `Sequence`[`float` | `str`] | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.

  – **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

  – **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The

resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **dpi** (`int` | `None`, default: `None`) – Set the resolution for the new grid in dots per inch.
- **inverse** (`bool`, default: `False`) – When set to `True`, do the inverse transformation, from rectangular to geographical [Default is `False`].
- **unit** (`Literal['c', 'i', 'p']` | `None`, default: `None`) – Set the projected measure unit. Valid values are **c**, **i**, and **p**. [Default is set by PROJ_LENGTH_UNIT]. Cannot be used with `scaling`. See *Dimension Units* for meanings of the units.
- **scaling** (`Literal['c', 'i', 'p', 'e', 'f', 'k', 'M', 'n', 'u']` | `bool`, default: `False`) – Force 1:1 scaling, i.e., output (or input, see `inverse`) data are in actual projected meters. To specify other units, set it to one of the supported units: **c**, **i**, **p**, **f**, **k**, **M**, **n**, and **u**. See *Dimension Units* and *Distance Units* for meanings of the units. Without `scaling`, the output (or input, see `inverse`) is in the units specified by PROJ_LENGTH_UNIT (but see `unit`).
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **registration** (`Literal['gridline', 'pixel']` | `bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> region = [10, 30, 15, 25]
>>> grid = pygmt.datasets.load_earth_relief(resolution="30m", region=region)
```

(continues on next page)

(continued from previous page)

```
>>> # Project the geographic gridded data onto a rectangular grid
>>> new_grid = pygmt.grdproject(grid=grid, projection="M10c", region=region)
```

pygmt.grdsample

pygmt.**grdsample**(*grid*, *outgrid=None*, *toggle=False*, *spacing=None*, *region=None*, *verbose=False*, *registration=False*, *cores=False*, *coltypes=None*, ***kwargs*)

Resample a grid onto a new lattice.

This function reads a grid and interpolates it to create a new grid. It can

- change the registration (via `toggle` or `registration`)
- change the grid-spacing or number of nodes (via `spacing`)
- set a new sub-region (via `region`)

When `region` is omitted, the output grid will cover the same region as the input grid. When `spacing` is omitted, the grid spacing of the output grid will be the same as the input grid. Either `registration` or `toggle` can be used to change the grid registration. When omitted, the output grid will have the same registration as the input grid.

A bicubic [Default], bilinear, B-spline, or nearest-neighbor interpolation is set with `interpolation`. Note that using `region` only is equivalent to *`pygmt.grdcut`* or `grdedit -S`. `grdsample` safely creates a fine mesh from a coarse one; the converse may suffer aliasing unless the data are filtered using *`pygmt.grdfilter`* or `grdfft`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdsample.html.

Aliases

- n = interpolation
- G = outgrid
- I = spacing
- R = region
- T = toggle
- V = verbose
- f = coltypes
- r = registration
- x = cores

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **spacing** (`Sequence`[`float` | `str`] | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.

  – **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

- **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **`toggle`** (`bool`, default: `False`) – Toggle between grid and pixel registration; if the input is grid-registered, the output will be pixel-registered and vice-versa. This is a *destructive* grid change; see https://docs.generic-mapping-tools.org/6.7/reference/options.html#switch-registrations. *Note**: `toggle` and `registration` are mutually exclusive.
- **`region`** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **`verbose`** (`Literal`[`'quiet'`, `'error'`, `'warning'`, `'timing'`, `'info'`, `'compat'`, `'debug'`] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **`coltypes`** (`str` | `None`, default: `None`) – [**i|o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **`interpolation`** (*`str`*) – [**b|c|l|n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **`registration`** (`Literal`[`'gridline'`, `'pixel'`] | `bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **`cores`** (`int` | `bool`, default: `False`) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, change the registration,
>>> # and set both x- and y-spacings to 0.5 arc-degrees
>>> new_grid = pygmt.grdsample(grid=grid, toggle=True, spacing=[0.5, 0.5])
```

pygmt.grdtrack

pygmt.**grdtrack**(*grid*, *points=None*, *output_type='pandas'*, *outfile=None*, *newcolname=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *coltypes=None*, ***kwargs*)

Sample one or more grids at specified locations.

Reads one or more grid files and a table (from file or an array input; but see `profile` for exception) with (x,y) [or (lon,lat)] positions in the first two columns (more columns may be present). It interpolates the grid(s) at the positions in the table and writes out the table with the interpolated values added as (one or more) new columns. Alternatively (`crossprofile`), the input is considered to be line-segments and we create orthogonal cross-profiles at each data point or with an equidistant separation and sample the grid(s) along these profiles. A bicubic [Default], bilinear, B-spline or nearest-neighbor interpolation is used, requiring boundary conditions at the limits of the region (see `interpolation`; Default uses "natural" conditions (second partial derivative normal to edge is zero) unless the grid is automatically recognized as periodic.)

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdtrack.html.

Aliases

- A = resample
- C = crossprofile
- D = dfile
- E = profile
- F = critical
- N = no_skip
- S = stack
- T = radius
- Z = z_only
- a = aspatial
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- j = distcalc
- n = interpolation
- s = skiprows
- w = wrap
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **points** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in either a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.

- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.

  – `pandas` will return a `pandas.DataFrame` object.

  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.

- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.
- **newcolname** (*str*) – Required if `points` is a `pandas.DataFrame`. The name for the new column in the track `pandas.DataFrame` table where the sampled values will be placed.
- **resample** (*str*) – **f**|**p**|**m**|**r**|**R**[**+l**] For track resampling (if `crossprofile` or `profile` are set) we can select how this is to be performed. Append **f** to keep original points, but add intermediate points if needed [Default], **m** as **f**, but first follow meridian (along y) then parallel (along x), **p** as **f**, but first follow parallel (along y) then meridian (along x), **r** to resample at equidistant locations; input points are not necessarily included in the output, and **R** as **r**, but adjust given spacing to fit the track length exactly. Finally, append **+l** if geographic distances should be measured along rhumb lines (loxodromes) instead of great circles. Ignored unless `crossprofile` is used.
- **crossprofile** (*str*) – *length*/*ds*[*/spacing*][**+a**|**+v**][**l**|**r**]. Use input line segments to create an equidistant and (optionally) equally-spaced set of crossing profiles along which we sample the grid(s) [Default simply samples the grid(s) at the input locations]. Specify two length scales that control how the sampling is done: *length* sets the full length of each cross-profile, while *ds* is the sampling spacing along each cross-profile. Optionally, append */spacing* for an equidistant spacing between cross-profiles [Default erects cross-profiles at the input coordinates]; see `resample` for how resampling the input track is controlled. By default, all cross-profiles have the same direction (left to right as we look in the direction of the input line segment). Append **+a** to alternate the direction of cross-profiles, or **v** to enforce either a "west-to-east" or "south-to-north" view. By default the entire profiles are output. Choose to only output the left or right halves of the profiles by appending **+l** or **+r**, respectively. Append a suitable *distance unit* to *length*; it sets the unit used for *ds* [and *spacing*]. The default unit for geographic grids is meters while Cartesian grids implies the user unit. The output columns will be *lon*, *lat*, *dist*, *azimuth*, *z1*, *z2*, …, *zn* (The *zi* are the sampled values for each of the *n* grids).
- **dfile** (*str*) – In concert with `crossprofile` we can save the (possibly resampled) original lines to *dfile* [Default only saves the cross-profiles]. The columns will be *lon*, *lat*, *dist*, *azimuth*, *z1*, *z2*, … (sampled value for each grid).
- **profile** (*str*) – *line*[,*line*,…][**+a***az*][**+c**][**+d**][**+g**][**+i***inc*][**+l***length*][**+n***np*][**+o***az*][**+r***radius*]. Instead of reading input track coordinates, specify profiles via coordinates and modifiers. The format of each *line* is *start*/*stop*, where *start* or *stop* are either *lon*/*lat* (*x*/*y* for Cartesian data) or a 2-character XY key that uses the text-style justification format to specify a point on the map as [LCR][BMT]. Each line will be a separate segment unless **+c** is used which will connect segments with shared joints into a single segment. In addition to line coordinates, you can use Z-, Z+ to mean the global minimum and maximum locations in the grid (only available if a single grid is given via **outfile**). You may append **+i***inc* to set the sampling interval; if not given then we default to half the minimum grid interval. For a *line* along parallels or meridians you can add **+g** to report degrees of longitude or latitude instead of great circle distances starting at zero. Instead of two coordinates you can specify an origin and one of **+a**, **+o**, or **+r**. The **+a** sets the azimuth of a profile of given length starting at the given origin, while **+o** centers the profile on the origin; both require **+l**. For circular sampling specify **+r** to define a circle of given radius centered on the origin; this option requires either **+n** or **+i**. The **+n***np* modifier sets the desired number of points, while **+l***length* gives the total length of the profile. Append **+d** to output the along-track distances after the coordinates. **Note**: No track file will be read. Also note that only one distance unit can be chosen. Giving different units will result in an error. If no units are specified we default to great circle distances in km (if geographic). If working with geographic data you can use `distcalc` to control distance

calculation mode [Default is Great Circle]. **Note**: If `crossprofile` is set and *spacing* is given then that sampling scheme overrules any modifier set in `profile`.

- **`critical`** (*`str`*) – [**+b**][**+n**][**+r**][**+z***z0*]. Find critical points along each cross-profile as a function of along-track distance. Requires `crossprofile` and a single input grid (*z*). We examine each cross-profile generated and report (*dist*, *lonc*, *latc*, *distc*, *azimuthc*, *zc*) at the center peak of maximum *z* value, (*lonl*, *latl*, *distl*) and (*lonr*, *latr*, *distr*) at the first and last non-NaN point whose *z*-value exceeds *z0*, respectively, and the *width* based on the two extreme points found. Here, *dist* is the distance along the original input `points` and the other 12 output columns are a function of that distance. When searching for the center peak and the extreme first and last values that exceed the threshold we assume the profile is positive up. If we instead are looking for a trough then you must use **+n** to temporarily flip the profile to positive. The threshold *z0* value is always given as >= 0; use **+z** to change it [Default is 0]. Alternatively, use **+b** to determine the balance point and standard deviation of the profile; this is the weighted mean and weighted standard deviation of the distances, with *z* acting as the weight. Finally, use **+r** to obtain the weighted rms about the cross-track center (*distc* == 0). **Note**: We round the exact results to the nearest distance nodes along the cross-profiles. We write 13 output columns per track: *dist, lonc, latc, distc, azimuthc, zc, lonl, latl, distl, lonr, latr, distr, width*.
- **`no_skip`** (*`bool`*) – Do *not* skip points that fall outside the domain of the grid(s) [Default only output points within the grid domain].
- **`stack`** (*`str or list`*) – *method/modifiers*. In conjunction with `crossprofile`, compute a single stacked profile from all profiles across each segment. Choose how stacking should be computed [Default method is **a**]:
  - **a** = mean (average)
  - **m** = median
  - **p** = mode (maximum likelihood)
  - **l** = lower
  - **L** = lower but only consider positive values
  - **u** = upper
  - **U** = upper but only consider negative values.

  The *modifiers* control the output; choose one or more among these choices:
  - **+a** : Append stacked values to all cross-profiles.
  - **+d** : Append stack deviations to all cross-profiles.
  - **+r** : Append data residuals (data - stack) to all cross-profiles.
  - **+s**[*file*] : Save stacked profile to *file* [Default file name is grdtrack_stacked_profile.txt].
  - **+c***fact* : Compute envelope on stacked profile as ±*fact* **deviation* [Default fact value is 2].

  Here are some notes:

  1. Deviations depend on *method* and are st.dev (**a**), L1 scale, i.e., 1.4826 * median absolute deviation (MAD) (for **m** and **p**), or half-range (upper-lower)/2.
  2. The stacked profile file contains a leading column plus groups of 4-6 columns, with one group for each sampled grid. The leading column holds cross distance, while the first four columns in a group hold stacked value, deviation, min value, and max value, respectively. If *method* is one of **a**|**m**|**p** then we also write the lower and upper confidence bounds (see **+c**). When one or more of **+a**, **+d**, and **+r** are used then we also append the stacking results to

the end of each row, for all cross-profiles. The order is always stacked value (**+a**), followed by deviations (**+d**) and finally residuals (**+r**). When more than one grid is sampled this sequence of 1-3 columns is repeated for each grid.

- **radius** (*bool, float, or str*) – [*radius*][**+e**|**p**]. To be used with normal grid sampling, and limited to a single, non-IMG grid. If the nearest node to the input point is NaN, search outwards until we find the nearest non-NaN node and report that value instead. Optionally specify a search radius which limits the consideration to points within this distance from the input point. To report the location of the nearest node and its distance from the input point, append **+e**. The default unit for geographic grid distances is spherical degrees. Use *radius* to change the unit and give *radius* = 0 if you do not want to limit the radius search. To instead replace the input point with the coordinates of the nearest node, append **+p**.
- **z_only** (*bool*) – Only write out the sampled z-values [Default writes all columns].
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, nodata="-9999" will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.

- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.
- **coltypes** (str | None, default: None) – [**i|o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x|y|z|d|X|Y|D***gap*[**u**][**+a**][**+c***col*][**+n|p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x|X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y|Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d|D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x|y|d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X|Y|D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:
  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.
- **header** (*str*) – [**i|o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.

- **incols** (int | str | Sequence[int | str] | None, default: None) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **distcalc** (*str*) – Determine how spherical distances are calculated [*Full usage*].
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **outcols** (int | str | Sequence[int | str] | None, default: None) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

    Optionally, append one of the following modifiers to any column or column range to transform the output columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the output values by the factor *divisor* [Default is 1].

* **+s** to multiply the output values by the factor *scale* [Default is 1].
* **+o** to add the given *offset* to the output values [Default is 0].

- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:
  - **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.
  - **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[/*phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type
DataFrame | ndarray | None

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # -118° E to -107° E, and a latitude range of -49° N to -42° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[-118, -107, -49, -42]
... )
>>> # Load a pandas dataframe with ocean ridge points
>>> points = pygmt.datasets.load_sample_data(name="ocean_ridge_points")
>>> # Create a pandas dataframe from an input grid and set of points
>>> # The output dataframe adds a column named "bathymetry"
>>> output_dataframe = pygmt.grdtrack(
...     points=points, grid=grid, newcolname="bathymetry"
... )
```

Examples using `pygmt.grdtrack`

pygmt.grdvolume

pygmt.**grdvolume**(*grid*, *output_type='pandas'*, *outfile=None*, *region=None*, *verbose=False*, ***kwargs*)

Calculate grid volume and area constrained by a contour.

Read a 2-D grid file and calculate the volume contained below the surface and above the plane specified by the given contour (or zero if not given) and return the contour, area, volume, and maximum mean height (volume/area). Alternatively, a range of contours can be specified to return the volume and area inside the contour for all contour values.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdvolume.html.

Aliases

- C = contour
- R = region
- S = unit
- V = verbose

Parameters

- **grid** (str | PathLike | DataArray) – Name of the input grid file or the grid loaded as a xarray.DataArray object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **output_type** (Literal['pandas', 'numpy', 'file'], default: 'pandas') – Desired output type of the result data.

  - pandas will return a pandas.DataFrame object.
  - numpy will return a numpy.ndarray object.
  - file will save the result to the file specified by the outfile parameter.

- **outfile** (str | PathLike | None, default: None) – File name for saving the result data. Required if output_type="file". If specified, output_type will be forced to be "file".

- **contour** (*str, float, or list*) – *cval*|*low*/*high*/*delta*|**r***low*/*high*|**r***cval*. Find area, volume and mean height (volume/area) inside and above the *cval* contour. Alternatively, search using all contours from *low* to *high* in steps of *delta*. [Default returns area, volume and mean height of the entire grid]. The area is measured in the plane of the contour. Adding the **r** prefix computes the volume below the grid surface and above the planes defined by *low* and *high*, or below *cval* and grid's minimum. Note that this is an *outside* volume whilst the other forms compute an *inside* (below the surface) area volume. Use this form to compute for example the volume of water between two contours. If no *contour* is given then there is no contour and the entire grid area, volume and the mean height is returned and *cval* will be reported as 0.

- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin*/*xmax*/*ymin*/*ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Return type

DataFrame | ndarray | None

Returns

ret – Return type depends on outfile and output_type:

- None if outfile is set (output will be stored in file set by outfile)
- pandas.DataFrame or numpy.ndarray if outfile is not set (depends on output_type)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a pandas dataframe that contains the contour, area, volume,
>>> # and maximum mean height above the plane specified by the given
>>> # contour and below the surface; set the minimum contour z-value to
>>> # 200, the maximum to 400, and the interval to 50.
>>> output_dataframe = pygmt.grdvolume(
...     grid=grid, contour=[200, 400, 50], output_type="pandas"
... )
>>> print(output_dataframe)
       0             1             2           3
0  200.0  2.323600e+12  8.523815e+14  366.836554
1  250.0  2.275864e+12  7.371655e+14  323.905736
2  300.0  2.166707e+12  6.258570e+14  288.851699
3  350.0  2.019284e+12  5.207732e+14  257.899955
4  400.0  1.870441e+12  4.236191e+14  226.480847
```

11.2.3 Crossover analysis with x2sys

x2sys_init(tag[, spacing, region, verbose])	Initialize a new x2sys track database.
x2sys_cross([tracks, outfile, region, verbose])	Calculate crossovers between track data files.

pygmt.x2sys_init

pygmt.**x2sys_init**(*tag*, *spacing=None*, *region=None*, *verbose=False*, ***kwargs*)

Initialize a new x2sys track database.

Serves as the starting point for x2sys and initializes a set of data bases that are particular to one kind of track data. These data, their associated data bases, and key parameters are given a short-hand notation called an x2sys TAG. The TAG keeps track of settings such as file format, whether the data are geographic or not, and the binning resolution for track indices.

Before you can run *pygmt.x2sys_init* you must set the environment variable *X2SYS_HOME* to a directory where you have write permission, which is where x2sys can keep track of your settings.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/supplements/x2sys/x2sys_init.html.

Reference: Wessel, P. (2010). Tools for analyzing intersecting tracks: The x2sys package. *Computers & Geosciences*, 36(3), 348-354. https://doi.org/10.1016/j.cageo.2009.05.009

Aliases

- D = fmtfile
- E = suffix
- F = force
- G = discontinuity
- N = units
- W = gap
- j = distcalc
- I = spacing
- R = region
- V = verbose

Parameters

- **tag** (*str*) – The unique name of this data type x2sys TAG.
- **fmtfile** (*str*) – Format definition file prefix for this data set (see GMT's Format Definition Files for more information). Specify full path if the file is not in the current directory.

  Some file formats already have definition files premade. These include:

  - **mgd77** (for plain ASCII MGD77 data files)
  - **mgd77+** (for enhanced MGD77+ netCDF files)
  - **gmt** (for old mgg supplement binary files)
  - **xy** (for plain ASCII x, y tables)
  - **xyz** (same, with one z-column)
  - **geo** (for plain ASCII longitude, latitude files)
  - **geoz** (same, with one z-column).
- **suffix** (*str*) – Specify the file extension (suffix) for these data files. If not given we use the format definition file prefix as the suffix (see `fmtfile`).
- **discontinuity** (*str*) – **d**|**g**. Select geographical coordinates. Append **d** for discontinuity at the Dateline (makes longitude go from -180° E to +180° E) or **g** for discontinuity at Greenwich (makes longitude go from 0° E to 360° E [Default]). If not given we assume the data are Cartesian.
- **spacing** (`Sequence`[`float` | `str`] | `None`, default: `None`) – *dx*[/*dy*]. *dx* and optionally *dy* is the grid spacing. Append **m** to indicate minutes or **s** to indicate seconds for geographic data. These spacings refer to the binning used in the track bin-index data base.
- **units** (*str or list*) – **d**|**s***unit*. Set the units used for distance and speed when requested by other programs. Append **d** for distance or **s** for speed, then give the desired *unit* as:

  - **c**: Cartesian userdist or userdist/usertime
  - **e**: meters or m/s
  - **f**: feet or ft/s
  - **k**: kilometers or km/hr
  - **m**: miles or mi/hr
  - **n**: nautical miles or knots
  - **u**: survey feet or sft/s

  [Default is `units=["dk", "se"]` (km and m/s) if `discontinuity` is set, and `units=["dc", "sc"]` otherwise (e.g., for Cartesian units)].
- **gap** (*str or list*) – **t**|**d***gap*. Give **t** or **d** and append the corresponding maximum time gap (in user units; this is typically seconds [Default is infinity]), or distance (for units, see `units`) gap [Default is infinity]) allowed between the two data points immediately on either

side of a crossover. If these limits are exceeded then a data gap is assumed and no COE will be determined.

- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **distcalc** (*str*) – Determine how spherical distances are calculated [*Full usage*].

pygmt.x2sys_cross

pygmt.**x2sys_cross**(*tracks=None*, *outfile=None*, *region=None*, *verbose=False*, ***kwargs*)

Calculate crossovers between track data files.

Determines all intersections between ("external cross-overs") or within ("internal cross-overs") tracks (Cartesian or geographic), and report the time, position, distance along track, heading and speed along each track segment, and the crossover error (COE) and mean values for all observables. By default, *pygmt.x2sys_cross* will look for both external and internal COEs. As an option, you may choose to project all data using one of the map projections prior to calculating the COE.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/supplements/x2sys/x2sys_cross.html.

Reference: Wessel, P. (2010). Tools for analyzing intersecting tracks: The x2sys package. *Computers & Geosciences*, 36(3), 348-354. https://doi.org/10.1016/j.cageo.2009.05.009

Aliases

- A = combitable
- C = runtimes
- D = override
- I = interpolation
- Q = coe
- S = speed
- T = tag
- W = numpoints
- Z = track_values
- R = region
- V = verbose

Parameters

- **tracks** (pandas.DataFrame, str, or list) – A table or a list of tables with (x, y) or (lon, lat) values in the first two columns. Track(s) can be provided as pandas.DataFrame tables or file names. Supported file formats are ASCII, native binary, or COARDS netCDF 1-D data. More columns may also be present.

  If the file names are missing their file extension, we will append the suffix specified for this TAG. Track files will be searched for first in the current directory and second in all directories listed in $X2SYS_HOME/TAG/TAG_paths.txt (if it exists). [If environment variable *X2SYS_HOME* is not set it will default to $GMT_SHAREDIR/x2sys]. (**Note**: MGD77 files will also be looked for via $MGD77_HOME/mgd77_paths.txt and .gmt files will be searched for via $GMT_SHAREDIR/mgg/gmtfile_paths).
- **outfile** (str | PathLike | None, default: None) – The file name for the output ASCII txt file to store the table in.
- **tag** (*str*) – Specify the x2sys TAG which identifies the attributes of this data type.
- **combitable** (*str*) – Only process the pair-combinations found in the file *combitable* [Default process all possible combinations among the specified files]. The file *combitable* is created by x2sys_get's -L option.
- **runtimes** (*bool or str*) – Compute and append the processing run-time for each pair to the progress message (use runtimes=True). Pass in a file name (e.g. runtimes="file.txt") to save these run-times to file. The idea here is to use the knowledge of run-times to

split the main process in a number of sub-processes that can each be launched in a different processor of your multi-core machine. See the MATLAB function split_file4coes.m.

- **override** (*bool or str*) – **S**|**N**. Control how geographic coordinates are handled (Cartesian data are unaffected). By default, we determine if the data are closer to one pole than the other, and then we use a cylindrical polar conversion to avoid problems with longitude jumps. You can turn this off entirely with `override` and then the calculations uses the original data (we have protections against longitude jumps). However, you can force the selection of the pole for the projection by appending **S** or **N** for the south or north pole, respectively. The conversion is used because the algorithm used to find crossovers is inherently a Cartesian algorithm that can run into trouble with data that has large longitudinal range at higher latitudes.
- **interpolation** (*str*) – **l**|**a**|**c**. Sets the interpolation mode for estimating values at the crossover. Choose among:
  - **l**: Linear interpolation [Default].
  - **a**: Akima spline interpolation.
  - **c**: Cubic spline interpolation.
- **coe** (*str*) – Use **e** for external COEs only, and **i** for internal COEs only [Default is all COEs].
- **speed** (*str or list*) – **l**|**u**|**h***speed*. Defines window of track speeds. If speeds are outside this window we do not calculate a COE. Specify:
  - **l** sets lower speed [Default is 0].
  - **u** sets upper speed [Default is infinity].
  - **h** does not limit the speed but sets a lower speed below which headings will not be computed (i.e., set to NaN) [Default calculates headings regardless of speed].

  For example, you can use `speed=["l0", "u10", "h5"]` to set a lower speed of 0, upper speed of 10, and disable heading calculations for speeds below 5.
- **numpoints** (*int*) – Give the maximum number of data points on either side of the crossover to use in the spline interpolation [Default is 3].
- **track_values** (*bool*) – Report the values of each track at the crossover [Default reports the crossover value and the mean value].
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataFrame | None`

Returns

crossover_errors – Table containing crossover error information. A `pandas.DataFrame` object is returned if `outfile` is not set, otherwise `None` is returned and output will be stored in file set by `outfile`.

11.3 GMT Defaults

Operations on GMT defaults:

config(*[, COLOR_BACKGROUND, ...])	Change GMT default settings globally or locally.

11.3.1 pygmt.config

class pygmt.**config**(*, *COLOR_BACKGROUND=None*, *COLOR_FOREGROUND=None*, *COLOR_CPT=None*, *COLOR_NAN=None*, *COLOR_MODEL=None*, *COLOR_HSV_MIN_S=None*, *COLOR_HSV_MAX_S=None*, *COLOR_HSV_MIN_V=None*, *COLOR_HSV_MAX_V=None*, *COLOR_SET=None*, *DIR_CACHE=None*, *DIR_DATA=None*, *DIR_DCW=None*, *DIR_GSHHG=None*, *FONT_ANNOT_PRIMARY=None*, *FONT_ANNOT_SECONDARY=None*, *FONT_HEADING=None*, *FONT_LABEL=None*, *FONT_LOGO=None*, *FONT_SUBTITLE=None*, *FONT_TAG=None*, *FONT_TITLE=None*, *FORMAT_CLOCK_IN=None*, *FORMAT_CLOCK_OUT=None*, *FORMAT_CLOCK_MAP=None*, *FORMAT_DATE_IN=None*, *FORMAT_DATE_OUT=None*, *FORMAT_DATE_MAP=None*, *FORMAT_GEO_OUT=None*, *FORMAT_GEO_MAP=None*, *FORMAT_FLOAT_OUT=None*, *FORMAT_FLOAT_MAP=None*, *FORMAT_TIME_PRIMARY_MAP=None*, *FORMAT_TIME_SECONDARY_MAP=None*, *FORMAT_TIME_STAMP=None*, *GMT_DATA_SERVER=None*, *GMT_DATA_SERVER_LIMIT=None*, *GMT_DATA_UPDATE_INTERVAL=None*, *GMT_COMPATIBILITY=None*, *GMT_CUSTOM_LIBS=None*, *GMT_EXPORT_TYPE=None*, *GMT_EXTRAPOLATE_VAL=None*, *GMT_FFT=None*, *GMT_GRAPHICS_DPU=None*, *GMT_GRAPHICS_FORMAT=None*, *GMT_HISTORY=None*, *GMT_INTERPOLANT=None*, *GMT_LANGUAGE=None*, *GMT_MAX_CORES=None*, *GMT_THEME=None*, *GMT_TRIANGULATE=None*, *GMT_VERBOSE=None*, *IO_COL_SEPARATOR=None*, *IO_FIRST_HEADER=None*, *IO_GRIDFILE_FORMAT=None*, *IO_GRIDFILE_SHORTHAND=None*, *IO_HEADER=None*, *IO_HEADER_MARKER=None*, *IO_N_HEADER_RECS=None*, *IO_NAN_RECORDS=None*, *IO_NC4_CHUNK_SIZE=None*, *IO_NC4_DEFLATION_LEVEL=None*, *IO_LONLAT_TOGGLE=None*, *IO_SEGMENT_BINARY=None*, *IO_SEGMENT_MARKER=None*, *MAP_ANNOT_MIN_ANGLE=None*, *MAP_ANNOT_MIN_SPACING=None*, *MAP_ANNOT_OBLIQUE=None*, *MAP_ANNOT_OFFSET_PRIMARY=None*, *MAP_ANNOT_OFFSET_SECONDARY=None*, *MAP_ANNOT_ORTHO=None*, *MAP_DEFAULT_PEN=None*, *MAP_DEGREE_SYMBOL=None*, *MAP_EMBELLISHMENT_MODE=None*, *MAP_FRAME_AXES=None*, *MAP_FRAME_PEN=None*, *MAP_FRAME_PERCENT=None*, *MAP_FRAME_TYPE=None*, *MAP_FRAME_WIDTH=None*, *MAP_GRID_CROSS_SIZE_PRIMARY=None*, *MAP_GRID_CROSS_SIZE_SECONDARY=None*, *MAP_GRID_PEN_PRIMARY=None*, *MAP_GRID_PEN_SECONDARY=None*, *MAP_HEADING_OFFSET=None*, *MAP_LABEL_MODE=None*, *MAP_LABEL_OFFSET=None*, *MAP_LINE_STEP=None*, *MAP_LOGO=None*, *MAP_LOGO_POS=None*, *MAP_ORIGIN_X=None*, *MAP_ORIGIN_Y=None*, *MAP_POLAR_CAP=None*, *MAP_SCALE_HEIGHT=None*, *MAP_TICK_LENGTH_PRIMARY=None*, *MAP_TICK_LENGTH_SECONDARY=None*, *MAP_TICK_PEN_PRIMARY=None*, *MAP_TICK_PEN_SECONDARY=None*, *MAP_TITLE_OFFSET=None*, *MAP_VECTOR_SHAPE=None*, *PROJ_AUX_LATITUDE=None*, *PROJ_DATUM=None*, *PROJ_ELLIPSOID=None*, *PROJ_GEODESIC=None*, *PROJ_LENGTH_UNIT=None*, *PROJ_MEAN_RADIUS=None*, *PROJ_SCALE_FACTOR=None*, *PS_CHAR_ENCODING=None*, *PS_COLOR_MODEL=None*, *PS_COMMENTS=None*, *PS_IMAGE_COMPRESS=None*, *PS_LINE_CAP=None*, *PS_LINE_JOIN=None*, *PS_MITER_LIMIT=None*, *PS_MEDIA=None*, *PS_PAGE_COLOR=None*, *PS_PAGE_ORIENTATION=None*, *PS_SCALE_X=None*, *PS_SCALE_Y=None*, *PS_TRANSPARENCY=None*, *TIME_EPOCH=None*, *TIME_IS_INTERVAL=None*, *TIME_INTERVAL_FRACTION=None*, *TIME_LEAP_SECONDS=None*, *TIME_REPORT=None*, *TIME_UNIT=None*, *TIME_WEEK_START=None*, *TIME_Y2K_OFFSET_YEAR=None*, *FONT=None*, *FONT_ANNOT=None*, *FORMAT_TIME_MAP=None*, *MAP_ANNOT_OFFSET=None*, *MAP_GRID_CROSS_SIZE=None*, *MAP_GRID_PEN=None*, *MAP_TICK_LENGTH=None*, *MAP_TICK_PEN=None*)

Change GMT default settings globally or locally.

Change GMT default settings globally:

```
pygmt.config(PARAMETER=value)
```

Change GMT default settings locally by using it as a context manager:

```
with pygmt.config(PARAMETER=value):
    ...
```

Full GMT defaults list at https://docs.generic-mapping-tools.org/6.7/gmt.conf.html.

Note

PS_CONVERT is not supported. To configure conversion options, please pass parameters to *pygmt.Figure.savefig* or *pygmt.Figure.show* instead.

Examples using `pygmt.config`

11.4 Metadata

Getting metadata from tabular or grid data:

info(data[, spacing, per_column, incols, ...])	Get information about data tables.
grdinfo(grid[, spacing, tiles, region, ...])	Extract information from 2-D grids or 3-D cubes.

11.4.1 pygmt.info

pygmt.**info**(*data*, *spacing=None*, *per_column=False*, *incols=None*, *registration=False*, *verbose=False*, *coltypes=None*, ***kwargs*)

Get information about data tables.

Reads from files and finds the extreme values in each of the columns reported as min/max pairs. It recognizes NaNs and will print warnings if the number of columns vary from record to record. As an option, it will find the extent of the first two columns rounded up and down to the nearest multiple of the supplied increments given by `spacing`. Such output will be in a `numpy.ndarray` form [*w*, *e*, *s*, *n*], which can be used directly as the `region` parameter for other modules (hence only *dx* and *dy* are needed). If the `per_column` parameter is combined with `spacing`, then the `numpy.ndarray` output will be rounded up/down for as many columns as there are increments provided in `spacing`. A similar parameter `nearest_multiple` will provide a `numpy.ndarray` in the form of [*zmin*, *zmax*, *dz*] for makecpt.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/gmtinfo.html.

Aliases

- T = nearest_multiple
- a = aspatial
- C = per_column
- I = spacing
- V = verbose
- f = coltypes
- i = incols
- r = registration

Parameters

- **data** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame`) – Pass in either a file name to an ASCII data table, a 1-D/2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.

- **per_column** (bool, default: False) – Report the min/max values per column in separate columns [Default is the format <min/max>].
- **spacing** (Sequence[float] | str | None, default: None) – [**b**|**p**|**f**|**s**]*dx*[/*dy*[/*dz*…]]. Compute the min/max values of the first n columns to the nearest multiple of the provided increments [default is 2 columns]. By default, output results in the form [w, e, s, n], unless per_column is set in which case we output each min and max value in separate output columns.
- **nearest_multiple** (*str*) – **dz**[**+c***col*]. Report the min/max of the first (0'th) column to the nearest multiple of dz and output this in the form [zmin, zmax, dz].
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **incols** (int | str | Sequence[int | str] | None, default: None) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in input order (e.g., incols=(1, 0) for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., incols="0:2,4+l" to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use incols="n" to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.

Return type

ndarray | str

Returns

output (numpy.ndarray or str) – Return type depends on whether any of the per_column, spacing, or nearest_multiple parameters are set.

- numpy.ndarray if either of the above parameters are used.
- str if none of the above parameters are used.

Examples using `pygmt.info`

11.4.2 pygmt.grdinfo

`pygmt.`**`grdinfo`**(*grid*, *spacing=None*, *tiles=False*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Extract information from 2-D grids or 3-D cubes.

Can read the grid from a file or given as an `xarray.DataArray` grid.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdinfo.html.

Aliases

- C = per_column
- F = geographic
- L = force_scan
- M = minmax_pos
- T = nearest_multiple
- D = tiles
- I = spacing
- R = region
- V = verbose
- f = coltypes

Parameters

- **`grid`** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **`per_column`** (*`str or bool`*) – **n|t**. Format the report using tab-separated fields on a single line. The output is name *w e s n z0 z1 dx dy nx ny* [*x0 y0 x1 y1*] [*med scale*] [*mean std rms*] [*n_nan*] *registration gtype*. The data in brackets are outputted depending on the `force_scan` and `minmax_pos` parameters. Use **t** to place file name at the end of the output record or, **n** or `True` to only output numerical columns. The registration is either 0 (gridline) or 1 (pixel), while gtype is either 0 (Cartesian) or 1 (geographic). The default value is `False`. This cannot be called if `geographic` is also set.

- **`tiles`** (`Sequence`[`float`] | `str` | `bool`, default: `False`) – *xoff*[/*yoff*][**+i**]. Divide a single grid's domain (or the `region` domain, if no grid given) into tiles of size dx times dy (set via `spacing`). You can specify overlap between tiles by appending *xoff*[/*yoff*]. If the single grid is given you may use the modifier **+i** to ignore tiles that have no data within each tile subregion. Default output is text region strings. Use `per_column` to instead report four columns with xmin xmax ymin ymax per tile, or use `per_column="t"` to also have the region string appended as trailing text.

- **`geographic`** (*`bool`*) – Report grid domain and x/y-increments in world mapping format. The default value is `False`. This cannot be called if `per_column` is also set.

- **`spacing`** (`Sequence`[`float`] | `str` | `None`, default: `None`) – *dx*[/*dy*]|**b**|**i**|**r**. Report the min/max of the region to the nearest multiple of dx and dy, and output this in the form w/e/s/n (unless `per_column` is set). To report the actual grid region, append **r**. For a grid produced by the img supplement (a Cartesian Mercator grid), the exact geographic region is given with **i** (if not found then we return the actual grid region instead). If no argument is given then we report the grid increment in the form *xinc*[/*yinc*]. If **b** is given we write each grid's bounding box polygon instead. Finally, if `tiles` is in effect then *dx* and *dy* are the dimensions of the desired tiles.

- **`force_scan`** (*`int or str`*) – **0|1|2|p|a**.

  - **0**: Report range of z after actually scanning the data, not just reporting what the header says.
  - **1**: Report median and L1 scale of z (L1 scale = 1.4826 * Median Absolute Deviation (MAD)).

  - **2**: Report mean, standard deviation, and root-mean-square (rms) of z.
  - **p**: Report mode (LMS) and LMS scale of z.
  - **a**: Include all of the above.
- **minmax_pos** (*bool*) – Include the x/y values at the location of the minimum and maximum z-values.
- **nearest_multiple** (*str*) – [*dz*][**+a**[*alpha*]][**+s**]. Determine minimum and maximum z-values. If *dz* is provided then we first round these values off to multiples of *dz*. To exclude the two tails of the distribution when determining the minimum and maximum you can add **+a** to set the *alpha* value (in percent): We then sort the grid, exclude the data in the 0.5**alpha* and 100 - 0.5**alpha* tails, and revise the minimum and maximum. To force a symmetrical range about zero, using minus/plus the maximum absolute value of the two extremes, append **+s**. We report the result via the text string *zmin/zmax* or *zmin/zmax/dz* (if *dz* was given) as expected by *pygmt.makecpt*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **coltypes** (str | None, default: None) – [**i|o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type
str

Returns
info (*str*) – A string with information about the grid.

11.5 xarray Integration

GMTBackendEntrypoint()	xarray backend to read raster grid/image files using 'gmt' engine.
GMTDataArrayAccessor(xarray_obj)	GMT accessor for xarray.DataArray.

11.5.1 pygmt.GMTBackendEntrypoint

class pygmt.**GMTBackendEntrypoint**

xarray backend to read raster grid/image files using 'gmt' engine.

Internally, GMT uses the netCDF C library to read netCDF files, and GDAL for GeoTIFF and other raster formats. See https://docs.generic-mapping-tools.org/6.7/reference/features.html#grid-file-format for more details about supported formats. This GMT engine can also read GMT remote datasets (file names starting with an @) directly, and pre-loads *pygmt.GMTDataArrayAccessor* properties (in the '.gmt' accessor) for easy access to GMT-specific metadata and features.

When using xarray.open_dataarray or xarray.load_dataarray with engine="gmt", the raster_kind parameter is required and can be either:

- "grid": for reading single-band raster grids
- "image": for reading multi-band raster images

Optionally, you can pass in a `region` in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code.

Examples

Read a single-band netCDF file using `raster_kind="grid"`

```
>>> import pygmt
>>> import xarray as xr
>>>
>>> da_grid = xr.open_dataarray(
...     "@static_earth_relief.nc", engine="gmt", raster_kind="grid"
... )
>>> da_grid
<xarray.DataArray 'z' (lat: 14, lon: 8)> Size: 448B
[112 values with dtype=float32]
Coordinates:
  * lat      (lat) float64 112B -23.5 -22.5 -21.5 -20.5 ... -12.5 -11.5 -10.5
  * lon      (lon) float64 64B -54.5 -53.5 -52.5 -51.5 -50.5 -49.5 -48.5 -47.5
:ivar Conventions: CF-1.7
:ivar title: Produced by grdcut
:ivar history: grdcut @earth_relief_01d_p -R-55/-47/-24/-10 -Gstatic_eart...
:ivar description: Reduced by Gaussian Cartesian filtering (111.2 km fullwidt...
:ivar actual_range: [190. 981.]
:ivar long_name: elevation (m)
```

Read a multi-band GeoTIFF file using `raster_kind="image"`

```
>>> da_image = xr.open_dataarray(
...     "@earth_night_01d", engine="gmt", raster_kind="image"
... )
>>> da_image
<xarray.DataArray 'z' (band: 3, y: 180, x: 360)> Size: 194kB
[194400 values with dtype=uint8]
Coordinates:
  * band     (band) uint8 3B 1 2 3
  * y        (y) float64 1kB 89.5 88.5 87.5 86.5 ... -86.5 -87.5 -88.5 -89.5
  * x        (x) float64 3kB -179.5 -178.5 -177.5 -176.5 ... 177.5 178.5 179.5
:ivar long_name: z
```

Load a single-band netCDF file using `raster_kind="grid"` over a bounding box region.

```
>>> da_grid = xr.load_dataarray(
...     "@tut_bathy.nc", engine="gmt", raster_kind="grid", region=[-64, -62, 32,␣
↪33]
... )
>>> da_grid
<xarray.DataArray 'z' (lat: 13, lon: 25)> Size: 1kB
array([[-4369., -4587., -4469., -4409., -4587., -4505., -4403., -4405.,
        -4466., -4595., -4609., -4608., -4606., -4607., -4607., -4597.,
...
        -4667., -4642., -4677., -4795., -4797., -4800., -4803., -4818.,
        -4820.]], dtype=float32)
Coordinates:
  * lat      (lat) float64 104B 32.0 32.08 32.17 32.25 ... 32.83 32.92 33.0
```

(continues on next page)

(continued from previous page)

```
  * lon      (lon) float64 200B -64.0 -63.92 -63.83 ... -62.17 -62.08 -62.0
:ivar Conventions: CF-1.7
:ivar title: ETOPO5 global topography
:ivar history: grdreformat -fg bermuda.grd bermuda.nc=ns
:ivar description: /home/elepaio5/data/grids/etopo5.i2
:ivar actual_range: [-4968. -4315.]
:ivar long_name: Topography
:ivar units: m
```

Attributes

GMTBackendEntrypoint.**description**: ClassVar[str] = **'Open raster (.grd, .nc or .tif) files in xarray via GMT.'**

GMTBackendEntrypoint.**open_dataset_parameters**: ClassVar[tuple | None] = **('filename_or_obj', 'raster_kind', 'region')**

GMTBackendEntrypoint.**supports_groups**: ClassVar[bool] = **False**

GMTBackendEntrypoint.**url**: ClassVar[str] = **'https://pygmt.org/dev/api/generated/pygmt.GMTBackendEntrypoint.html'**

Methods Summary

GMTBackendEntrypoint.guess_can_open(...)	Backend open_dataset method used by Xarray in open_dataset.
GMTBackendEntrypoint.open_dataset(...[, ...])	Backend open_dataset method used by xarray in open_dataset.
GMTBackendEntrypoint.open_datatree(...[, ...])	Backend open_datatree method used by Xarray in open_datatree.
GMTBackendEntrypoint.open_groups_as_dict(...)	Opens a dictionary mapping from group names to Datasets.

pygmt.GMTBackendEntrypoint.guess_can_open

GMTBackendEntrypoint.**guess_can_open**(*filename_or_obj*)

Backend open_dataset method used by Xarray in open_dataset.

Return type

bool

pygmt.GMTBackendEntrypoint.open_dataset

GMTBackendEntrypoint.**open_dataset**(*filename_or_obj*, *, *drop_variables=None*, *raster_kind*, *region=None*)

Backend open_dataset method used by xarray in open_dataset.

Parameters

- **filename_or_obj** (str | PathLike) – File path to a netCDF (.nc), GeoTIFF (.tif) or other grid/image file format that can be read by GMT via the netCDF or GDAL C libraries. See also https://docs.generic-mapping-tools.org/6.7/reference/features.html#grid-file-format.
- **raster_kind** (Literal['grid', 'image']) – Whether to read the file as a "grid" (single-band) or "image" (multi-band).

- **region** (Sequence[float] | str | None, default: None) – The subregion of the grid or image to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code.

Return type
Dataset

pygmt.GMTBackendEntrypoint.open_datatree

GMTBackendEntrypoint.**open_datatree**(*filename_or_obj*, *, *drop_variables=None*)

Backend open_datatree method used by Xarray in open_datatree.

If implemented, set the class variable supports_groups to True.

Return type
DataTree

pygmt.GMTBackendEntrypoint.open_groups_as_dict

GMTBackendEntrypoint.**open_groups_as_dict**(*filename_or_obj*, *, *drop_variables=None*)

Opens a dictionary mapping from group names to Datasets.

Called by open_groups. This function exists to provide a universal way to open all groups in a file, before applying any additional consistency checks or requirements necessary to create a *DataTree* object (typically done using from_dict).

If implemented, set the class variable supports_groups to True.

Return type
dict[str, Dataset]

11.5.2 pygmt.GMTDataArrayAccessor

class pygmt.**GMTDataArrayAccessor**(*xarray_obj*)

GMT accessor for xarray.DataArray.

The *gmt* accessor extends xarray.DataArray to store GMT-specific properties for grids and images, which are important for PyGMT to correctly process and plot them. The *gmt* accessor contains the following properties:

- registration: Grid registration type *pygmt.enums.GridRegistration*.
- gtype: Grid coordinate system type *pygmt.enums.GridType*.

The *gmt* accessor also provides a set of grid-operation methods that enables applying GMT's grid processing functionalities directly to the current xarray.DataArray object. See the summary table below for the list of available methods.

Notes

When accessed the first time, the *gmt* accessor will first be initialized to the default values (GridRegistration.GRIDLINE and GridType.CARTESIAN, i.e., a gridline-registered, Cartesian grid), and then the properties will be updated with the correct grid registration and type determined from the source encoding (i.e., grid.encoding["source"]), if it is available.

Due to the limitations of xarray accessors, the *gmt* accessor is created once per xarray.DataArray instance. Thus, the *gmt* accessor will be re-initialized in cases where the xarray.DataArray is manipulated (e.g., arithmetic and slice operations) or when accessing a xarray.DataArray from a xarray.Dataset. In these cases, the GMT-specific properties will result in incorrect values if the source encoding is not defined or is dropped due to operations, and users need to manually set these properties again.

Examples

For grids loaded from a file (e.g., via `xarray.load_dataarray`) and GMT's built-in remote datasets, the GMT-specific properties are automatically determined and you can access them as follows:

```
>>> from pygmt.datasets import load_earth_relief
>>> # Use the global Earth relief grid with 1 degree spacing
>>> grid = load_earth_relief(resolution="01d", registration="pixel")
>>> # See if grid uses Gridline or Pixel registration
>>> grid.gmt.registration
<GridRegistration.PIXEL: 1>
>>> # See if grid is in Cartesian or Geographic coordinate system
>>> grid.gmt.gtype
<GridType.GEOGRAPHIC: 1>
>>> grid.encoding["source"] is not None
True
```

Inplace assignment operators like `*=` don't create new instances, so the properties are still kept:

```
>>> grid *= 2.0
>>> grid.gmt.registration
<GridRegistration.PIXEL: 1>
>>> grid.gmt.gtype
<GridType.GEOGRAPHIC: 1>
```

Slice operation creates a new instance, but the source encoding is kept, so the properties are still kept:

```
>>> grid_slice = grid[0:30, 50:80]
>>> # grid source encoding is kept in slice operation
>>> grid_slice.encoding["source"] is not None
True
>>> # properties are still kept
>>> grid_slice.gmt.registration
<GridRegistration.PIXEL: 1>
>>> grid_slice.gmt.gtype
<GridType.GEOGRAPHIC: 1>
```

Other grid operations (e.g., arithmetic operations) create new instances and drop the source encoding, so the properties will be reset to the default values:

```
>>> grid2 = grid * 2.0
>>> # grid source encoding is dropped in arithmetic operation.
>>> "source" in grid2.encoding
False
>>> # properties are reset to the default values
>>> grid2.gmt.registration
<GridRegistration.GRIDLINE: 0>
>>> grid2.gmt.gtype
<GridType.CARTESIAN: 0>
>>> # Need to set these properties before passing the grid to PyGMT
>>> grid2.gmt.registration = grid.gmt.registration
>>> grid2.gmt.gtype = grid.gmt.gtype
>>> grid2.gmt.registration
<GridRegistration.PIXEL: 1>
```

(continues on next page)

(continued from previous page)

```
>>> grid2.gmt.gtype
<GridType.GEOGRAPHIC: 1>
```

For `xarray.DataArray` grids created from scratch, the source encoding is not available, so the properties will be set to the default values, and you need to manually set the correct properties before passing it to PyGMT functions:

```
>>> import numpy as np
>>> import xarray as xr
>>> import pygmt
>>> from pygmt.enums import GridRegistration, GridType
>>> # Create a DataArray in gridline coordinates of sin(lon) * cos(lat)
>>> interval = 2.5
>>> lat = np.arange(90, -90 - interval, -interval)
>>> lon = np.arange(0, 360 + interval, interval)
>>> longrid, latgrid = np.meshgrid(lon, lat)
>>> data = np.sin(np.deg2rad(longrid)) * np.cos(np.deg2rad(latgrid))
>>> grid = xr.DataArray(data, coords=[("latitude", lat), ("longitude", lon)])
>>> # Default to a gridline-registered Cartesian grid
>>> grid.gmt.registration
<GridRegistration.GRIDLINE: 0>
>>> grid.gmt.gtype
<GridType.CARTESIAN: 0>
>>> # Manually set it to a gridline-registered geographic grid
>>> grid.gmt.registration = GridRegistration.GRIDLINE
>>> grid.gmt.gtype = GridType.GEOGRAPHIC
>>> grid.gmt.registration
<GridRegistration.GRIDLINE: 0>
>>> grid.gmt.gtype
<GridType.GEOGRAPHIC: 1>
```

Accessing a `xarray.DataArray` from a `xarray.Dataset` always creates new instances, so these properties are always lost if the source encoding is not available. The workaround is to assign the `xarray.DataArray` into a variable:

```
>>> ds = xr.Dataset({"zval": grid})
>>> ds.zval.gmt.registration
<GridRegistration.GRIDLINE: 0>
>>> ds.zval.gmt.gtype
<GridType.CARTESIAN: 0>
>>> # Manually set these properties won't work as expected
>>> ds.zval.gmt.registration = GridRegistration.GRIDLINE
>>> ds.zval.gmt.gtype = GridType.GEOGRAPHIC
>>> ds.zval.gmt.registration, ds.zval.gmt.gtype
(<GridRegistration.GRIDLINE: 0>, <GridType.CARTESIAN: 0>)
>>> # workaround: assign the DataArray into a variable
>>> zval = ds.zval
>>> zval.gmt.registration, zval.gmt.gtype
(<GridRegistration.GRIDLINE: 0>, <GridType.CARTESIAN: 0>)
>>> zval.gmt.registration = GridRegistration.GRIDLINE
>>> zval.gmt.gtype = GridType.GEOGRAPHIC
>>> zval.gmt.registration, zval.gmt.gtype
(<GridRegistration.GRIDLINE: 0>, <GridType.GEOGRAPHIC: 1>)
```

Instead of calling a grid-processing function and passing the `xarray.DataArray` object as an input, you can call the corresponding method directly on the object. For example, the following two are equivalent:

```
>>> from pygmt.datasets import load_earth_relief
>>> grid = load_earth_relief(resolution="30m", region=[10, 30, 15, 25])
>>> # Create a new grid from an input grid. Set all values below 1,000 to 0 and␣
↪all
>>> # values above 1,500 to 10,000.
>>> # Option 1:
>>> new_grid = pygmt.grdclip(grid=grid, below=[1000, 0], above=[1500, 10000])
>>> # Option 2:
>>> new_grid = grid.gmt.clip(below=[1000, 0], above=[1500, 10000])
```

Attributes

`GMTDataArrayAccessor.`**`gtype`**

Grid coordinate system type *pygmt.enums.GridType*.

`GMTDataArrayAccessor.`**`registration`**

Grid registration type *pygmt.enums.GridRegistration*.

Methods Summary

GMTDataArrayAccessor.clip([outgrid, above, ...])	Clip the range of grid values.
GMTDataArrayAccessor.cut([kind, outgrid, ...])	Extract subregion from a grid or image or a slice from a cube.
GMTDataArrayAccessor.dimfilter([outgrid, ...])	Directional filtering of grids in the space domain.
GMTDataArrayAccessor.fill([outgrid, ...])	Interpolate across holes in a grid.
GMTDataArrayAccessor.filter([outgrid, ...])	Filter a grid in the space (or time) domain.
GMTDataArrayAccessor.gradient([outgrid, ...])	Compute directional gradients from a grid.
GMTDataArrayAccessor.histeq(grid[, outgrid, ...])	Perform histogram equalization for a grid.
GMTDataArrayAccessor.project([outgrid, ...])	Forward and inverse map transformation of grids.
GMTDataArrayAccessor.sample([outgrid, ...])	Resample a grid onto a new lattice.
GMTDataArrayAccessor.track([points, ...])	Sample one or more grids at specified locations.

pygmt.GMTDataArrayAccessor.clip

`GMTDataArrayAccessor.`**`clip`**(*outgrid=None*, *above=None*, *below=None*, *between=None*, *replace=None*, *region=None*, *verbose=False*, ***kwargs*)

Clip the range of grid values.

This function operates on the values of a grid. It can:

- Set values smaller than a threshold to a new value
- Set values larger than a threshold to a new value
- Set values within a range to a new value
- Replace individual values with a new value

Such operations are useful when you want all of a continent or an ocean to fall into one color or gray shade in image processing, when clipping the range of data values is required, or for reclassification of data values. The values can be any number or NaN (Not a Number).

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdclip.html.

Aliases

- G = outgrid
- R = region
- Sa = above
- Sb = below
- Si = between
- Sr = replace
- V = verbose

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **above** (`Sequence[float]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*high*, *above*), to set all node values greater than *high* to *above*.

- **below** (`Sequence[float]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*low*, *below*) to set all node values less than *low* to *below*.

- **between** (`Sequence[float]` | `Sequence[Sequence[float]]` | `None`, default: `None`) – Pass a sequence of three values in the form of (*low*, *high*, *between*) to set all node values between *low* and *high* to *between*. It can also accept a sequence of sequences (e.g., list of lists or 2-D numpy array) to set different values for different ranges.

- **replace** (`Sequence[float]` | `Sequence[Sequence[float]]` | `None`, default: `None`) – Pass a sequence of two values in the form of (*old*, *new*) to replace all node values equal to *old* with *new*. It can also accept a sequence of sequences (e.g., list of lists or 2-D numpy array) to replace different old values with different new values. This is mostly useful when your data are known to be integer values.

- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load the 30 arc-minutes Earth relief grid, with a longitude range of 10° E␣
↪to
>>> # 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
```

(continues on next page)

(continued from previous page)

```
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Report the minimum and maximum data values
>>> [grid.data.min(), grid.data.max()]
[np.float32(183.5), np.float32(1807.0)]
>>> # Create a new grid from an input grid. Set all values below 1,000 to 0 and␣
↪all
>>> # values above 1,500 to 10,000
>>> new_grid = pygmt.grdclip(grid=grid, below=[1000, 0], above=[1500, 10000])
>>> # Report the minimum and maximum data values
>>> [new_grid.data.min(), new_grid.data.max()]
[np.float32(0.0), np.float32(10000.0)]
```

pygmt.GMTDataArrayAccessor.cut

GMTDataArrayAccessor.**cut**(*kind='grid'*, *outgrid=None*, *projection=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Extract subregion from a grid or image or a slice from a cube.

Produce a new `outgrid` file which is a subregion of `grid`. The subregion is specified with `region`; the specified range must not exceed the range of `grid` (but see `extend`). If in doubt, run *`pygmt.grdinfo`* to check range. Alternatively, define the subregion indirectly via a range check on the node values or via distances from a given point. Finally, you can give `projection` for oblique projections to determine the corresponding rectangular `region` that will give a grid that fully covers the oblique domain.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdcut.html.

Aliases

- N = extend
- S = circ_subregion
- Z = z_subregion
- G = outgrid
- J = projection
- R = region
- V = verbose
- f = coltypes

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **kind** (`Literal['grid', 'image']`, default: `'grid'`) – The raster data kind. Valid values are `"grid"` and `"image"`. When the input `grid` is a file name, it's difficult to determine if the file is a grid or an image, so we need to specify the raster kind explicitly. The default is `"grid"`.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **extend** (*bool or float*) – Allow grid to be extended if new `region` exceeds existing boundaries. Give a value to initialize nodes outside current region.

- **circ_subregion** (*str*) – *lon/lat/radius*[*unit*][**+n**]. Specify an origin (*lon* and *lat*) and *radius*; append a distance *unit* and we determine the corresponding rectangular region so that all grid

nodes on or inside the circle are contained in the subset. If **+n** is appended we set all nodes outside the circle to NaN.

- **z_subregion** (*str*) – [*min/max*][**+n**|**N**|**r**]. Determine a new rectangular region so that all nodes outside this region are also outside the given z-range [-inf/+inf]. To indicate no limit on *min* or *max* only, specify a hyphen (-). Normally, any NaNs encountered are simply skipped and not considered in the range-decision. Append **+n** to consider a NaN to be outside the given z-range. This means the new subset will be NaN-free. Alternatively, append **+r** to consider NaNs to be within the data range. In this case we stop shrinking the boundaries once a NaN is found [Default simply skips NaNs when making the range decision]. Finally, if your core subset grid is surrounded by rows and/or columns that are all NaNs, append **+N** to strip off such columns before (optionally) considering the range of the core subset for further reduction of the area.
- **projection** (`str` | `None`, default: `None`) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (`Sequence`[`float` | `str`] | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

`DataArray` | `None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, with a longitude range of
>>> # 12° E to 15° E and a latitude range of 21° N to 24° N
>>> new_grid = pygmt.grdcut(grid=grid, region=[12, 15, 21, 24])
```

pygmt.GMTDataArrayAccessor.dimfilter

`GMTDataArrayAccessor.`**`dimfilter`**(*outgrid=None*, *spacing=None*, *region=None*, *verbose=False*, ***kwargs*)

Directional filtering of grids in the space domain.

Filter a grid in the space (or time) domain by dividing the given filter circle into the given number of sectors, applying one of the selected primary convolution or non-convolution filters to each sector, and choosing the final outcome according to the selected secondary filter. It computes distances using Cartesian or Spherical geometries. The output grid can optionally be generated as a subregion of the input and/or with a new increment using `spacing`,

which may add an "extra space" in the input data to prevent edge effects for the output grid. If the filter is low-pass, then the output may be less frequently sampled than the input. `pygmt.dimfilter` will not produce a smooth output as other spatial filters do because it returns a minimum median out of *N* medians of *N* sectors. The output can be rough unless the input data are noise-free. Thus, an additional filtering (e.g., Gaussian via `pygmt.grdfilter`) of the DiM-filtered data is generally recommended.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/dimfilter.html.

Reference: Kim, S.-S., & Wessel, P. (2008). Directional median filtering for regional-residual separation of bathymetry. *Geochemistry, Geophysics, Geosystems*, 9(3), 2007GC001850. https://doi.org/10.1029/2007GC001850

Aliases

- D = distance
- F = filter
- N = sectors
- G = outgrid
- I = spacing
- R = region
- V = verbose

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **distance** (*int or str*) – Distance flag tells how grid (x,y) relates to filter width, as follows:

  – **0**: grid (x,y) in same units as *width*, Cartesian distances.

  – **1**: grid (x,y) in degrees, *width* in kilometers, Cartesian distances.

  – **2**: grid (x,y) in degrees, *width* in km, dx scaled by cos(middle y), Cartesian distances.

  The above options are fastest because they allow weight matrix to be computed only once. The next two options are slower because they recompute weights for each latitude.

  – **3**: grid (x,y) in degrees, *width* in km, dx scaled by cosine(y), Cartesian distance calculation.

  – **4**: grid (x,y) in degrees, *width* in km, Spherical distance calculation.

- **filter** (*str*) – **x***width*[**+l**|**u**]. Set the primary filter type. Choose among convolution and non-convolution filters. Use the filter code **x** followed by the full diameter *width*. Available convolution filters are:

  – **b**: boxcar. Aall weights are equal.

  – **c**: cosine arch. Weights follow a cosine arch curve.

  – **g**: Gaussian. Weights are given by the Gaussian function.

  Non-convolution filters are:

  – **m**: median. Returns median value.

  – **p**: maximum likelihood probability (a mode estimator). Return modal value. If more than one mode is found we return their average value. Append **+l** or **+h** to the filter width if you want to return the smallest or largest of each sector's modal values.

- **sectors** (*str*) – **x***sectors*[**+l**|**u**] Set the secondary filter type **x** and the number of bow-tie sectors. *sectors* must be integer and larger than 0. When *sectors* is set to 1, the secondary filter is not effective. Available secondary filters **x** are:
  - **l**: lower. Return the minimum of all filtered values.
  - **u**: upper. Return the maximum of all filtered values.
  - **a**: average. Return the mean of all filtered values.
  - **m**: median. Return the median of all filtered values.
  - **p**: mode. Return the mode of all filtered values. If more than one mode is found we return their average value. Append **+l** or **+h** to the sectors if you rather want to return the smallest or largest of the modal values.
- **spacing** (`Sequence[float | str] | None`, default: `None`) – *x_inc* [and optionally *y_inc*] is the output increment. Append **m** to indicate minutes, or **c** to indicate seconds. If the new *x_inc*, *y_inc* are **not** integer multiples of the old ones (in the input data), filtering will be considerably slower. [Default is same as the input.]
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*]. Define the region of the output points [Default is the same as the input].
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type

`DataArray | None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of Earth relief data
>>> grid = pygmt.datasets.load_earth_relief()
>>> # Create a filtered grid from an input grid.
>>> filtered_grid = pygmt.dimfilter(
...     grid=grid,
...     # Set filter type to "median" and the diameter width to 600 km
...     filter="m600",
...     # Set grid in degrees, width in km
...     distance=4,
...     # Create 6 sectors and return the lowest values in the sector
...     sectors="l6",
...     # Set the region longitude range from 55W to 51W, and the
...     # latitude range from 24S to 19S
...     region=[-55, -51, -24, -19],
... )
```

pygmt.GMTDataArrayAccessor.fill

GMTDataArrayAccessor.**fill**(*outgrid=None*, *constant_fill=None*, *grid_fill=None*, *neighbor_fill=None*, *spline_fill=None*, *inquire=False*, *hole=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Interpolate across holes in a grid.

Read a grid that presumably has unfilled holes that the user wants to fill in some fashion. Holes are identified by NaN values but this criteria can be changed via the `hole` parameter. There are several different algorithms that can be used to replace the hole values. If no holes are found the original unchanged grid is returned.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdfill.html.

Aliases

- Ac = constant_fill
- Ag = grid_fill
- An = neighbor_fill
- As = spline_fill
- G = outgrid
- L = inquire
- N = hole
- R = region
- V = verbose
- f = coltypes

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **constant_fill** (`float` | `None`, default: `None`) – Fill the holes with a constant value. Specify the constant value to use.

- **grid_fill** (`str` | `PathLike` | `DataArray` | `None`, default: `None`) – Fill the holes with values sampled from another (possibly coarser) grid. Specify the grid (a file name or an `xarray.DataArray`) to use for the fill.

- **neighbor_fill** (`float` | `bool` | `None`, default: `None`) – Fill the holes with the nearest neighbor. Specify the search radius in pixels. If set to `True`, the default search radius will be used ($r^2 = \sqrt{n^2 + m^2}$, where (*n,m*) are the node dimensions of the grid).

- **spline_fill** (`float` | `bool` | `None`, default: `None`) – Fill the holes with a bicubic spline. Specify the tension value to use. If set to `True`, no tension will be used.

- **hole** (`float` | `None`, default: `None`) – Set the node value used to identify a point as a member of a hole [Default is NaN].

- **inquire** (`bool`, default: `False`) – Output the bounds of each hole. The bounds are returned as a 2-D numpy array in the form of (west, east, south, north). No grid fill takes place and `outgrid` is ignored.

- **region** (`Sequence[float | str]` | `str` | `None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.

- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug']` | `bool`, default: `False`) – Select verbosity level [*Full usage*].

- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

Return type

DataArray | ndarray | None

Returns

ret – If inquire is True, return the bounds of each hole as a 2-D numpy array. Otherwise, the return type depends on whether the outgrid parameter is set:

- xarray.DataArray if outgrid is not set
- None if outgrid is set (grid output will be stored in the file set by outgrid)

Examples

Fill holes in a bathymetric grid with a constant value of 20.

```
>>> import pygmt
>>> # Load a bathymetric grid with missing data
>>> earth_relief_holes = pygmt.datasets.load_sample_data(name="earth_relief_holes
↪")
>>> # Fill the holes with a constant value of 20
>>> filled_grid = pygmt.grdfill(grid=earth_relief_holes, constant_fill=20)
```

Inquire the bounds of each hole.

```
>>> pygmt.grdfill(grid=earth_relief_holes, inquire=True)
array([[1.83333333, 6.16666667, 3.83333333, 8.16666667],
       [6.16666667, 7.83333333, 0.5       , 2.5       ]])
```

pygmt.GMTDataArrayAccessor.filter

GMTDataArrayAccessor.**filter**(*outgrid=None*, *filter=None*, *width=None*, *highpass=False*, *distance=None*, *spacing=None*, *nans=None*, *toggle=False*, *region=None*, *verbose=False*, *registration=False*, *cores=False*, *coltypes=None*, ***kwargs*)

Filter a grid in the space (or time) domain.

Filter a grid file in the space (or time) domain using one of the selected convolution or non-convolution isotropic or rectangular filters and compute distances using Cartesian or Spherical geometries. The output grid file can optionally be generated as a sub-region of the input (via region) and/or with new increment (via spacing) or registration (via toggle). In this way, one may have "extra space" in the input data so that the edges will not be used and the output can be within one half-width of the input edges. If the filter is low-pass, then the output may be less frequently sampled than the input.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdfilter.html.

Aliases

- D = distance
- F = filter, width, **+h**: highpass
- G = outgrid
- I = spacing
- N = nans
- R = region
- T = toggle
- V = verbose
- f = coltypes
- r = registration
- x = cores

Parameters

- **grid** (str | PathLike | DataArray) – Name of the input grid file or the grid loaded as a xarray.DataArray object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **filter** (Literal['boxcar', 'cosarch', 'gaussian', 'minall', 'minpos', 'maxall', 'maxneg'] | str | None, default: None) – The filter type. Choose among convolution and non-convolution filters.

  Convolution filters include:

  - "boxcar": All weights are equal.
  - "cosarch": Weights follow a cosine arch curve.
  - "gaussian": Weights are given by the Gaussian function, where filter width is 6 times the conventional Gaussian sigma.

  Non-convolution filters include:

  - "minall": Return minimum of all values.
  - "minpos": Return minimum of all positive values only.
  - "maxall": Return maximum of all values.
  - "maxneg": Return maximum of all negative values only.

  Note: There are still a few other filter types available in GMT (e.g., histogram and mode filters), but they are not implemented in PyGMT yet. As a workaround, pass the raw GMT command string to this parameter to use these other filter types. Refer to https://docs.generic-mapping-tools.org/6.7/grdfilter.html#f for the full syntax of this parameter.

- **width** (float | Sequence[float] | None, default: None) – The full diameter width of the filter. It can be a single value for an isotropic filter, or a pair of values for a rectangular filter (width in x- and y-directions, requiring distance be either "pixel" or "cartesian").

- **highpass** (bool, default: False) – By default, the filter is a low-pass filter. If True, then the filter is a high-pass filter. [Default is False].

- **distance** (Literal['pixel', 'cartesian', 'geo_cartesian', 'geo_flatearth1', 'geo_flatearth2', 'geo_spherical', 'geo_mercator'] | None, default: None) – Determine how grid (*x, y*) relates to filter *width* and how distances are calculated. Valid values are list below. The first four options are fastest because they allow weight matrix to be computed only once. The last three options are slower because they recompute weights for each latitude.

Value	Grid (x,y)	Width	Distance Calculation
`"pixel"`	Pixels (px, py)	Odd number of pixels	Cartesian
`"cartesian"`	Same units as *width*	Any	Cartesian
`"geo_carte`	Degrees	km	Cartesian
`"geo_flate`	Degrees	km	Cartesian, dx scaled by cos(middle y)
`"geo_flate`	Degrees	km	Cartesian, dx scaled by cos(y) per row
`"geo_spher`	Degrees	km	Spherical (great circle)
`"geo_merca`	Mercator **-Jm1** img units	km	Spherical

- **spacing** (`Sequence[float | str] | None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.
  - **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.
  - **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.
- **nans** (`Literal['ignore', 'replace', 'preserve'] | None`, default: `None`) – Determine how NaN-values in the input grid affect the filtered output grid. Choose one of:
  - `"ignore"`: Ignore all NaNs in the calculation of filtered value [Default].
  - `"replace"`: Similar to `"ignore"` except if the input node was NaN then the output node will be set to NaN (only applied if both grids are co-registered).
  - `"preserve"`: Force the filtered value to be NaN if any grid nodes with NaN-values are found inside the filter circle.
- **toggle** (`bool`, default: `False`) – Toggle the node registration for the output grid so as to become the opposite of the input grid [Default gives the same registration as the input grid]. Alternatively, use `registration` to set the registration explicitly.
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].

- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.
- **cores** (int | bool, default: False) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type

DataArray | None

Returns

ret – Return type depends on whether the outgrid parameter is set:

- xarray.DataArray if outgrid is not set
- None if outgrid is set (grid output will be stored in the file set by outgrid)

Examples

```
>>> from pathlib import Path
>>> import pygmt
>>> # Apply a gaussian filter of 600 km (full width) to the @earth_relief_30m_g␣
↪grid
>>> # and return a filtered grid (saved as netCDF file).
>>> pygmt.grdfilter(
...     grid="@earth_relief_30m_g",
...     filter="gaussian",
...     width=600,
...     distance="geo_spherical",
...     region=[150, 250, 10, 40],
...     spacing=0.5,
...     outgrid="filtered_pacific.nc",
... )
>>> Path("filtered_pacific.nc").unlink()  # Cleanup file
>>> # Apply a Gaussian smoothing filter of 600 km to the input DataArray and␣
↪return
>>> # a filtered DataArray with the smoothed grid.
>>> grid = pygmt.datasets.load_earth_relief()
>>> smoothed = pygmt.grdfilter(
...     grid=grid, filter="gaussian", width=600, distance="geo_spherical"
... )
```

pygmt.GMTDataArrayAccessor.gradient

GMTDataArrayAccessor.**gradient**(*outgrid=None*, *azimuth=None*, *radiance=None*, *normalize=False*, *norm_amp=None*, *norm_ambient=None*, *norm_sigma=None*, *norm_offset=None*, *region=None*, *verbose=False*, *coltypes=None*, ***kwargs*)

Compute directional gradients from a grid.

Can accept `azimuth`, `direction`, and `radiance` input to create the resulting gradient.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdgradient.html.

Aliases

- D = direction
- Q = tiles
- S = slope_file
- n = interpolation
- A = azimuth
- E = radiance
- G = outgrid
- R = region
- V = verbose
- f = coltypes

- N = normalize, norm_amp, **+a**: norm_ambient, **+s**: norm_sigma, **+o**: norm_offset

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **azimuth** (`float` | `Sequence[float]` | `None`, default: `None`) – *azim* or (*azim*, *azim2*). Azimuthal direction for a directional derivative; *azim* is the angle in the x-y plane measured in degrees positive clockwise from north (the +y direction) toward east (the +x direction). The negative of the directional derivative, $-(\frac{dz}{dx}\sin(\text{azim}) + \frac{dz}{dy}\cos(\text{azim}))$, is found; negation yields positive values when the slope of $z(x, y)$ is downhill in the *azim* direction, the correct sense for shading the illumination of an image by (see *pygmt.Figure.grdimage* and *pygmt.Figure.grdview*) a light source above the x-y plane shining from the *azim* direction. Optionally, supply two azimuths, (*azim*, *azim2*), in which case the gradients in each of these directions are calculated and the one larger in magnitude is retained; this is useful for illuminating data with two directions of lineated structures, e.g., `(0, 270)` illuminates from the north (top) and west (left). Finally, if *azim* is a file it must be a grid of the same domain, spacing and registration as *grid* that will update the azimuth at each output node when computing the directional derivatives.

- **direction** (*str*) – [**a**][**c**][**o**][**n**]. Find the direction of the positive (up-slope) gradient of the data. The following options are supported:

  - **a**: Find the aspect (i.e., the down-slope direction)
  - **c**: Use the conventional Cartesian angles measured counterclockwise from the positive x (east) direction.
  - **o**: Report orientations (0-180) rather than directions (0-360).
  - **n**: Add 90 degrees to all angles (e.g., to give local strikes of the surface).

- **radiance** (`Sequence[float]` | `str` | `None`, default: `None`) – (*azim*, *elev*) or [**m**|**s**|**p**]*azim/elev*[**+a***ambient*] [**+d***diffuse*][**+p***specular*][**+s***shine*]. Compute Lambertian radiance appropriate to use with *pygmt.Figure.grdimage* and *pygmt.Figure.grdview*. The Lambertian Reflection assumes an ideal surface that reflects all the light that strikes it and the surface appears equally bright from all viewing directions. Here, *azim* and *elev* are the azimuth and elevation of the light vector. Optionally, supply *ambient* [0.55], *diffuse* [0.6], *specular* [0.4], or *shine* [10], which are parameters that control the reflectance properties of the

surface. Default values are given in the brackets. Use **s** for a simpler Lambertian algorithm. Note that with this form you only have to provide azimuth and elevation. Alternatively, use **p** for the Peucker piecewise linear approximation (simpler but faster algorithm; in this case *azim* and *elev* are hardwired to 315 and 45 degrees. This means that even if you provide other values they will be ignored.).

- **normalize** (Literal['laplace', 'cauchy'] | bool, default: False) – Normalize the output gradients. Valid values are:
  - False: No normalization is done [Default].
  - True: Normalize using max absolute value.
  - "laplace": Normalize using cumulative Laplace distribution.
  - "cauchy": Normalize using cumulative Cauchy distribution.

  The normalization process is controlled via the additional parameters norm_amp, norm_ambient, norm_sigma, and norm_offset.

  Let g denote the actual gradients, g_n the normalized gradients, a the maximum output magnitude (norm_amp), o the offset value (norm_offset), and σ the sigma value (norm_sigma). The normalization is computed as follows:
  - True: $g_n = a(\frac{g-o}{\max(|g-o|)})$
  - "laplace": $g_n = a(1 - \exp(\sqrt{2}\frac{g-o}{\sigma}))$
  - "cauchy": $g_n = \frac{2a}{\pi} \arctan(\frac{g-o}{\sigma})$
- **norm_amp** (float | None, default: None) – Set the maximum output magnitude [Default is 1].
- **norm_ambient** (float | None, default: None) – The ambient value to add to all nodes after gradient calculations are completed [Default is 0].
- **norm_offset** (float | None, default: None) – The offset value used in the normalization. If not given, it is set to the average of g.
- **norm_sigma** (float | None, default: None) – The *sigma* value used in the Laplace or Cauchy normalization. If not given, it is estimated from the L1 norm of $g - o$ for Laplace or the L2 norm of $g - o$ for Cauchy.
- **tiles** (*str*) – **c**|**r**|**R**. Control how normalization via normalize is carried out. When multiple grids should be normalized the same way (i.e., with the same *offset* and/or *sigma*), we must pass these values via normalize. However, this is inconvenient if we compute these values from a grid. Use **c** to save the results of *offset* and *sigma* to a statistics file; if grid output is not needed for this run then do not specify outgrid. For subsequent runs, just use **r** to read these values. Using **R** will read then delete the statistics file.
- **slope_file** (*str*) – Name of output grid file with scalar magnitudes of gradient vectors. Requires direction but makes outgrid optional.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **coltypes** (str | None, default: None) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.

- **interpolation** (*str*) – **[b|c|l|n][+a][+b***BC***][+c][+t***threshold***]**. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor

Return type
DataArray | None

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, set the azimuth to 10 degrees,
>>> new_grid = pygmt.grdgradient(grid=grid, azimuth=10)
```

pygmt.GMTDataArrayAccessor.histeq

GMTDataArrayAccessor.**histeq**(*grid*, *outgrid=None*, *region=None*, *verbose=False*, ***kwargs*)

Perform histogram equalization for a grid.

pygmt.grdhisteq.equalize_grid provides a way to write a grid with statistics based on a cumulative distribution function. The `outgrid` has relative highs and lows in the same (x,y) locations as the `grid`, but the values are changed to reflect their place in the cumulative distribution.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdhisteq.html.

Aliases

- C = divisions
- N = gaussian
- Q = quadratic
- h = header
- G = outgrid
- R = region
- V = verbose

Parameters

- **grid** (str | PathLike | DataArray) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (str | PathLike | None, default: None) – Name of the output netCDF grid file. If not specified, will return an xarray.DataArray object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.
- **divisions** (*int*) – Set the number of divisions of the data range.
- **gaussian** (*bool or int or float*) – *norm*. Produce an output grid with standard normal scores using gaussian=True or force the scores to fall in the ±*norm* range.
- **quadratic** (*bool*) – Perform quadratic equalization [Default is linear].
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].

Return type

DataArray | None

Returns

ret – Return type depends on the outgrid parameter:

- xarray.DataArray if outgrid is None
- None if outgrid is a str (grid output is stored in outgrid)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range
>>> # of 10°E to 30°E, and a latitude range of 15°N to 25°N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid with a Gaussian data distribution
>>> grid = pygmt.grdhisteq.equalize_grid(grid=grid, gaussian=True)
```

See also

pygmt.grd2cpt

Note

This method does a weighted histogram equalization for geographic grids to account for node area varying with latitude.

pygmt.GMTDataArrayAccessor.project

GMTDataArrayAccessor.**project**(*outgrid=None*, *center=False*, *spacing=None*, *dpi=None*, *inverse=False*, *unit=None*, *scaling=False*, *projection=None*, *region=None*, *registration=False*, *verbose=False*, ***kwargs*)

Forward and inverse map transformation of grids.

This method will project a geographical gridded data set onto a rectangular grid. If `inverse` is `True`, it will project a rectangular coordinate system to a geographic system. To obtain the value at each new node, its location is inversely projected back onto the input grid after which a value is interpolated between the surrounding input grid values. By default bi-cubic interpolation is used. Aliasing is avoided by also forward projecting the input grid nodes. If two or more nodes are projected onto the same new node, their average will dominate in the calculation of the new node value. Interpolation and aliasing is controlled with the `interpolation` parameter. The new node spacing may be determined in one of several ways by specifying the grid spacing, number of nodes, or resolution. Nodes not constrained by input data are set to NaN. The `region` parameter can be used to select a plot region larger or smaller than that implied by the extent of the grid file.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdproject.html.

Aliases

- n = interpolation
- C = center
- D = spacing
- E = dpi
- F = scaling
- G = outgrid
- I = inverse
- J = projection
- M = unit
- R = region
- V = verbose
- r = registration

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **center** (`Sequence`[`float` | `str`] | `bool`, default: `False`) – If `True`, let the projected coordinates be relative to the projection center [Default is relative to the lower left corner]. Optionally, set offsets (*dx*, *dy*) in the projected units to be added (or subtracted when `inverse` is set) to (from) the projected coordinates, such as false eastings and northings for particular projection zones [Default is `(0, 0)`].

- **spacing** (`float` | `str` | `Sequence`[`float` | `str`] | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.

  – **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

  – **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

  Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **dpi** (int | None, default: None) – Set the resolution for the new grid in dots per inch.
- **inverse** (bool, default: False) – When set to True, do the inverse transformation, from rectangular to geographical [Default is False].
- **unit** (Literal['c', 'i', 'p'] | None, default: None) – Set the projected measure unit. Valid values are **c**, **i**, and **p**. [Default is set by PROJ_LENGTH_UNIT]. Cannot be used with scaling. See *Dimension Units* for meanings of the units.
- **scaling** (Literal['c', 'i', 'p', 'e', 'f', 'k', 'M', 'n', 'u'] | bool, default: False) – Force 1:1 scaling, i.e., output (or input, see inverse) data are in actual projected meters. To specify other units, set it to one of the supported units: **c**, **i**, **p**, **f**, **k**, **M**, **n**, and **u**. See *Dimension Units* and *Distance Units* for meanings of the units. Without scaling, the output (or input, see inverse) is in the units specified by PROJ_LENGTH_UNIT (but see unit).
- **projection** (str | None, default: None) – *projcode*[*projparams*/]*width*|*scale*. Select map *projection*.
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **registration** (Literal['gridline', 'pixel'] | bool, default: False) – Select gridline or pixel node registration. Valid values are "gridline", "pixel", and bool. GMT default is gridline registration. If True, select pixel registration.

Return type

DataArray | None

Returns

ret – Return type depends on whether the outgrid parameter is set:

- xarray.DataArray if outgrid is not set
- None if outgrid is set (grid output will be stored in the file set by outgrid)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> region = [10, 30, 15, 25]
>>> grid = pygmt.datasets.load_earth_relief(resolution="30m", region=region)
>>> # Project the geographic gridded data onto a rectangular grid
>>> new_grid = pygmt.grdproject(grid=grid, projection="M10c", region=region)
```

pygmt.GMTDataArrayAccessor.sample

GMTDataArrayAccessor.**sample**(*outgrid=None*, *toggle=False*, *spacing=None*, *region=None*, *verbose=False*, *registration=False*, *cores=False*, *coltypes=None*, ***kwargs*)

Resample a grid onto a new lattice.

This function reads a grid and interpolates it to create a new grid. It can

- change the registration (via `toggle` or `registration`)
- change the grid-spacing or number of nodes (via `spacing`)
- set a new sub-region (via `region`)

When `region` is omitted, the output grid will cover the same region as the input grid. When `spacing` is omitted, the grid spacing of the output grid will be the same as the input grid. Either `registration` or `toggle` can be used to change the grid registration. When omitted, the output grid will have the same registration as the input grid.

A bicubic [Default], bilinear, B-spline, or nearest-neighbor interpolation is set with `interpolation`. Note that using `region` only is equivalent to *`pygmt.grdcut`* or `grdedit -S`. `grdsample` safely creates a fine mesh from a coarse one; the converse may suffer aliasing unless the data are filtered using *`pygmt.grdfilter`* or `grdfft`.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdsample.html.

Aliases

- n = interpolation
- G = outgrid
- I = spacing
- R = region
- T = toggle
- V = verbose
- f = coltypes
- r = registration
- x = cores

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **outgrid** (`str` | `PathLike` | `None`, default: `None`) – Name of the output netCDF grid file. If not specified, will return an `xarray.DataArray` object. For writing a specific grid file format or applying basic data operations to the output grid, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **spacing** (`Sequence[float | str]` | `None`, default: `None`) – *x_inc*[**+e**|**n**][/*y_inc*[**+e**|**n**]]. *x_inc* [and optionally *y_inc*] is the grid spacing.

  – **Geographical (degrees) coordinates**: Optionally, append an increment unit. Choose among **m** to indicate arc-minutes or **s** to indicate arc-seconds. If one of the units **e**, **f**, **k**, **M**, **n** or **u** is appended instead, the increment is assumed to be given in meter, foot, km, mile, nautical mile or US survey foot, respectively, and will be converted to the equivalent degrees longitude at the middle latitude of the region (the conversion depends on PROJ_ELLIPSOID). If *y_inc* is given but set to 0 it will be reset equal to *x_inc*; otherwise it will be converted to degrees latitude.

  – **All coordinates**: If **+e** is appended then the corresponding max *x* (*east*) or *y* (*north*) may be slightly adjusted to fit exactly the given increment [by default the increment may be adjusted slightly to fit the given domain]. Finally, instead of giving an increment you may specify the *number of nodes* desired by appending **+n** to the supplied integer argument; the increment is then recalculated from the number of nodes, the `registration`, and the domain. The

resulting increment value depends on whether you have selected a gridline-registered or pixel-registered grid; see GMT File Formats for details.

Note: If `region=grdfile` is used then the grid spacing and the registration have already been initialized; use `spacing` and `registration` to override these values.

- **toggle** (`bool`, default: `False`) – Toggle between grid and pixel registration; if the input is grid-registered, the output will be pixel-registered and vice-versa. This is a *destructive* grid change; see https://docs.generic-mapping-tools.org/6.7/reference/options.html#switch-registrations. *Note**: `toggle` and `registration` are mutually exclusive.
- **region** (`Sequence[float | str] | str | None`, default: `None`) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **verbose** (`Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool`, default: `False`) – Select verbosity level [*Full usage*].
- **coltypes** (`str | None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **registration** (`Literal['gridline', 'pixel'] | bool`, default: `False`) – Select gridline or pixel node registration. Valid values are `"gridline"`, `"pixel"`, and bool. GMT default is gridline registration. If `True`, select pixel registration.
- **cores** (`int | bool`, default: `False`) – Specify the number of active cores to be used in any OpenMP-enabled multi-threaded algorithms. By default, all available cores are used. Set a positive number *n* to use *n* cores (if too large it will be truncated to the maximum cores available); or set a negative number *-n* to select (all - *n*) cores (or at least 1 if *n* equals or exceeds all).

Return type

`DataArray | None`

Returns

ret – Return type depends on whether the `outgrid` parameter is set:

- `xarray.DataArray` if `outgrid` is not set
- `None` if `outgrid` is set (grid output will be stored in the file set by `outgrid`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # 10° E to 30° E, and a latitude range of 15° N to 25° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[10, 30, 15, 25]
... )
>>> # Create a new grid from an input grid, change the registration,
```

(continues on next page)

(continued from previous page)

```
>>> # and set both x- and y-spacings to 0.5 arc-degrees
>>> new_grid = pygmt.grdsample(grid=grid, toggle=True, spacing=[0.5, 0.5])
```

pygmt.GMTDataArrayAccessor.track

GMTDataArrayAccessor.**track**(*points=None*, *output_type='pandas'*, *outfile=None*, *newcolname=None*, *region=None*, *verbose=False*, *incols=None*, *outcols=None*, *coltypes=None*, ***kwargs*)

Sample one or more grids at specified locations.

Reads one or more grid files and a table (from file or an array input; but see `profile` for exception) with (x,y) [or (lon,lat)] positions in the first two columns (more columns may be present). It interpolates the grid(s) at the positions in the table and writes out the table with the interpolated values added as (one or more) new columns. Alternatively (`crossprofile`), the input is considered to be line-segments and we create orthogonal cross-profiles at each data point or with an equidistant separation and sample the grid(s) along these profiles. A bicubic [Default], bilinear, B-spline or nearest-neighbor interpolation is used, requiring boundary conditions at the limits of the region (see `interpolation`; Default uses "natural" conditions (second partial derivative normal to edge is zero) unless the grid is automatically recognized as periodic.)

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/grdtrack.html.

Aliases

- A = resample
- C = crossprofile
- D = dfile
- E = profile
- F = critical
- N = no_skip
- S = stack
- T = radius
- Z = z_only
- a = aspatial
- b = binary
- d = nodata
- e = find
- g = gap
- h = header
- j = distcalc
- n = interpolation
- s = skiprows
- w = wrap
- R = region
- V = verbose
- f = coltypes
- i = incols
- o = outcols

Parameters

- **grid** (`str` | `PathLike` | `DataArray`) – Name of the input grid file or the grid loaded as a `xarray.DataArray` object.

  For reading a specific grid file format or applying basic data operations, see https://docs.generic-mapping-tools.org/6.7/gmt.html#grd-inout-full for the available modifiers.

- **points** (`str` | `PathLike` | `dict` | `ndarray` | `DataFrame` | `Dataset` | `GeoDataFrame` | `None`, default: `None`) – Pass in either a file name to an ASCII data table, a 2-D `numpy.ndarray`, a `pandas.DataFrame`, an `xarray.Dataset` made up of 1-D `xarray.DataArray` data variables, or a `geopandas.GeoDataFrame` containing the tabular data.

- **output_type** (`Literal['pandas', 'numpy', 'file']`, default: `'pandas'`) – Desired output type of the result data.

  - `pandas` will return a `pandas.DataFrame` object.
  - `numpy` will return a `numpy.ndarray` object.
  - `file` will save the result to the file specified by the `outfile` parameter.

- **outfile** (`str` | `PathLike` | `None`, default: `None`) – File name for saving the result data. Required if `output_type="file"`. If specified, `output_type` will be forced to be `"file"`.

- **newcolname** (*str*) – Required if `points` is a `pandas.DataFrame`. The name for the new column in the track `pandas.DataFrame` table where the sampled values will be placed.
- **resample** (*str*) – **f**|**p**|**m**|**r**|**R**[**+l**] For track resampling (if `crossprofile` or `profile` are set) we can select how this is to be performed. Append **f** to keep original points, but add intermediate points if needed [Default], **m** as **f**, but first follow meridian (along y) then parallel (along x), **p** as **f**, but first follow parallel (along y) then meridian (along x), **r** to resample at equidistant locations; input points are not necessarily included in the output, and **R** as **r**, but adjust given spacing to fit the track length exactly. Finally, append **+l** if geographic distances should be measured along rhumb lines (loxodromes) instead of great circles. Ignored unless `crossprofile` is used.
- **crossprofile** (*str*) – *length*/*ds*[*/spacing*][**+a**|**+v**][**l**|**r**]. Use input line segments to create an equidistant and (optionally) equally-spaced set of crossing profiles along which we sample the grid(s) [Default simply samples the grid(s) at the input locations]. Specify two length scales that control how the sampling is done: *length* sets the full length of each cross-profile, while *ds* is the sampling spacing along each cross-profile. Optionally, append */spacing* for an equidistant spacing between cross-profiles [Default erects cross-profiles at the input coordinates]; see `resample` for how resampling the input track is controlled. By default, all cross-profiles have the same direction (left to right as we look in the direction of the input line segment). Append **+a** to alternate the direction of cross-profiles, or **v** to enforce either a "west-to-east" or "south-to-north" view. By default the entire profiles are output. Choose to only output the left or right halves of the profiles by appending **+l** or **+r**, respectively. Append a suitable *distance unit* to *length*; it sets the unit used for *ds* [and *spacing*]. The default unit for geographic grids is meters while Cartesian grids implies the user unit. The output columns will be *lon*, *lat*, *dist*, *azimuth*, *z1*, *z2*, …, *zn* (The *zi* are the sampled values for each of the *n* grids).
- **dfile** (*str*) – In concert with `crossprofile` we can save the (possibly resampled) original lines to *dfile* [Default only saves the cross-profiles]. The columns will be *lon*, *lat*, *dist*, *azimuth*, *z1*, *z2*, … (sampled value for each grid).
- **profile** (*str*) – *line*[,*line*,…][**+a***az*][**+c**][**+d**][**+g**][**+i***inc*][**+l***length*][**+n***np*][**+o***az*][**+r***radius*]. Instead of reading input track coordinates, specify profiles via coordinates and modifiers. The format of each *line* is *start*/*stop*, where *start* or *stop* are either *lon*/*lat* (*x*/*y* for Cartesian data) or a 2-character XY key that uses the text-style justification format to specify a point on the map as [LCR][BMT]. Each line will be a separate segment unless **+c** is used which will connect segments with shared joints into a single segment. In addition to line coordinates, you can use Z-, Z+ to mean the global minimum and maximum locations in the grid (only available if a single grid is given via **outfile**). You may append **+i***inc* to set the sampling interval; if not given then we default to half the minimum grid interval. For a *line* along parallels or meridians you can add **+g** to report degrees of longitude or latitude instead of great circle distances starting at zero. Instead of two coordinates you can specify an origin and one of **+a**, **+o**, or **+r**. The **+a** sets the azimuth of a profile of given length starting at the given origin, while **+o** centers the profile on the origin; both require **+l**. For circular sampling specify **+r** to define a circle of given radius centered on the origin; this option requires either **+n** or **+i**. The **+n***np* modifier sets the desired number of points, while **+l***length* gives the total length of the profile. Append **+d** to output the along-track distances after the coordinates. **Note**: No track file will be read. Also note that only one distance unit can be chosen. Giving different units will result in an error. If no units are specified we default to great circle distances in km (if geographic). If working with geographic data you can use `distcalc` to control distance calculation mode [Default is Great Circle]. **Note**: If `crossprofile` is set and *spacing* is given then that sampling scheme overrules any modifier set in `profile`.
- **critical** (*str*) – [**+b**][**+n**][**+r**][**+z***z0*]. Find critical points along each cross-profile as a function of along-track distance. Requires `crossprofile` and a single input grid (*z*). We examine each cross-profile generated and report (*dist*, *lonc*, *latc*, *distc*, *azimuthc*, *zc*) at the

center peak of maximum *z* value, (*lonl*, *latl*, *distl*) and (*lonr*, *latr*, *distr*) at the first and last non-NaN point whose *z*-value exceeds *z0*, respectively, and the *width* based on the two extreme points found. Here, *dist* is the distance along the original input `points` and the other 12 output columns are a function of that distance. When searching for the center peak and the extreme first and last values that exceed the threshold we assume the profile is positive up. If we instead are looking for a trough then you must use **+n** to temporarily flip the profile to positive. The threshold *z0* value is always given as >= 0; use **+z** to change it [Default is 0]. Alternatively, use **+b** to determine the balance point and standard deviation of the profile; this is the weighted mean and weighted standard deviation of the distances, with *z* acting as the weight. Finally, use **+r** to obtain the weighted rms about the cross-track center (*distc* == 0). **Note**: We round the exact results to the nearest distance nodes along the cross-profiles. We write 13 output columns per track: *dist, lonc, latc, distc, azimuthc, zc, lonl, latl, distl, lonr, latr, distr, width*.

- **no_skip** (*bool*) – Do *not* skip points that fall outside the domain of the grid(s) [Default only output points within the grid domain].
- **stack** (*str or list*) – *method*/*modifiers*. In conjunction with `crossprofile`, compute a single stacked profile from all profiles across each segment. Choose how stacking should be computed [Default method is **a**]:
  - **a** = mean (average)
  - **m** = median
  - **p** = mode (maximum likelihood)
  - **l** = lower
  - **L** = lower but only consider positive values
  - **u** = upper
  - **U** = upper but only consider negative values.

  The *modifiers* control the output; choose one or more among these choices:
  - **+a** : Append stacked values to all cross-profiles.
  - **+d** : Append stack deviations to all cross-profiles.
  - **+r** : Append data residuals (data - stack) to all cross-profiles.
  - **+s**[*file*] : Save stacked profile to *file* [Default file name is grdtrack_stacked_profile.txt].
  - **+c***fact* : Compute envelope on stacked profile as ±*fact* **deviation* [Default fact value is 2].

  Here are some notes:

  1. Deviations depend on *method* and are st.dev (**a**), L1 scale, i.e., 1.4826 * median absolute deviation (MAD) (for **m** and **p**), or half-range (upper-lower)/2.
  2. The stacked profile file contains a leading column plus groups of 4-6 columns, with one group for each sampled grid. The leading column holds cross distance, while the first four columns in a group hold stacked value, deviation, min value, and max value, respectively. If *method* is one of **a**|**m**|**p** then we also write the lower and upper confidence bounds (see **+c**). When one or more of **+a**, **+d**, and **+r** are used then we also append the stacking results to the end of each row, for all cross-profiles. The order is always stacked value (**+a**), followed by deviations (**+d**) and finally residuals (**+r**). When more than one grid is sampled this sequence of 1-3 columns is repeated for each grid.
- **radius** (*bool, float, or str*) – [*radius*][**+e**|**p**]. To be used with normal grid sampling, and limited to a single, non-IMG grid. If the nearest node to the input point is NaN, search

outwards until we find the nearest non-NaN node and report that value instead. Optionally specify a search radius which limits the consideration to points within this distance from the input point. To report the location of the nearest node and its distance from the input point, append **+e**. The default unit for geographic grid distances is spherical degrees. Use *radius* to change the unit and give *radius* = 0 if you do not want to limit the radius search. To instead replace the input point with the coordinates of the nearest node, append **+p**.

- **z_only** (*bool*) – Only write out the sampled z-values [Default writes all columns].
- **verbose** (Literal['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | bool, default: False) – Select verbosity level [*Full usage*].
- **region** (Sequence[float | str] | str | None, default: None) – [*xmin*, *xmax*, *ymin*, *ymax*] or *xmin/xmax/ymin/ymax*[**+r**][**+u***unit*]. Specify the *region* of interest.
- **aspatial** (*bool or str*) – [*col*=]*name*[,…]. Control how aspatial data are handled during input and output. Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#aspatial-full.
- **binary** (*bool or str*) – **i**|**o**[*ncols*][*type*][**w**][**+l**|**b**]. Select native binary input (using binary="i") or output (using binary="o"), where *ncols* is the number of data columns of *type*, which must be one of:
  - **c**: int8_t (1-byte signed char)
  - **u**: uint8_t (1-byte unsigned char)
  - **h**: int16_t (2-byte signed int)
  - **H**: uint16_t (2-byte unsigned int)
  - **i**: int32_t (4-byte signed int)
  - **I**: uint32_t (4-byte unsigned int)
  - **l**: int64_t (8-byte signed int)
  - **L**: uint64_t (8-byte unsigned int)
  - **f**: 4-byte single-precision float
  - **d**: 8-byte double-precision float
  - **x**: use to skip *ncols* anywhere in the record

  For records with mixed types, append additional comma-separated combinations of *ncols type* (no space). The following modifiers are supported:
  - **w** after any item to force byte-swapping.
  - **+l**|**b** to indicate that the entire data file should be read as little- or big-endian, respectively.

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#bi-full.
- **nodata** (*str*) – **i**|**o***nodata*. Substitute specific values with NaN (for tabular data). For example, nodata="-9999" will replace all values equal to -9999 with NaN during input and all NaN values with -9999 during output. Prepend **i** to the *nodata* value for input columns only. Prepend **o** to the *nodata* value for output columns only.
- **find** (*str*) – [**~**]*"pattern"* | [**~**]/*regexp*/[**i**]. Only pass records that match the given *pattern* or regular expressions [Default processes all records]. Prepend **~** to the *pattern* or *regexp* to instead only pass data expressions that do not match the pattern. Append **i** for case insensitive matching. This does not apply to headers or segment headers.

- **coltypes** (`str` | `None`, default: `None`) – [**i**|**o**]*colinfo*. Specify data types of input and/or output columns (time or geographical data). Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#f-full.
- **gap** (*str or list*) – **x**|**y**|**z**|**d**|**X**|**Y**|**D***gap*[**u**][**+a**][**+c***col*][**+n**|**p**]. Examine the spacing between consecutive data points in order to impose breaks in the line. To specify multiple criteria, provide a list with each item containing a string describing one set of criteria.
  - **x**|**X**: define a gap when there is a large enough change in the x coordinates (uppercase to use projected coordinates).
  - **y**|**Y**: define a gap when there is a large enough change in the y coordinates (uppercase to use projected coordinates).
  - **d**|**D**: define a gap when there is a large enough distance between coordinates (uppercase to use projected coordinates).
  - **z**: define a gap when there is a large enough change in the z data. Use **+c***col* to change the z data column [Default *col* is 2 (i.e., 3rd column)].

  A unit **u** may be appended to the specified *gap*:
  - For geographic data (**x**|**y**|**d**), valid units are **d**, **m**, **s**, **e**, **f**, **k**, **M**, and **n** [Default is **e**]. See *Distance Units* for meanings of the units.
  - For projected data (**X**|**Y**|**D**), valid units are **c**, **i**, and **p**. See *Dimension Units* for meanings of the units.

  Append modifier **+a** to specify that *all* the criteria must be met [default imposes breaks if any one criterion is met].

  One of the following modifiers can be appended:
  - **+n**: specify that the previous value minus the current column value must exceed *gap* for a break to be imposed.
  - **+p**: specify that the current value minus the previous value must exceed *gap* for a break to be imposed.
- **header** (*str*) – [**i**|**o**][*n*][**+c**][**+d**][**+m***segheader*][**+r***remark*][**+t***title*]. Specify that input and/or output file(s) have *n* header records [Default is 0]. Prepend **i** if only the primary input should have header records. Prepend **o** to control the writing of header records, with the following modifiers supported:
  - **+d** to remove existing header records.
  - **+c** to add a header comment with column names to the output [Default is no column names].
  - **+m** to add a segment header *segheader* to the output after the header block [Default is no segment header].
  - **+r** to add a *remark* comment to the output [Default is no comment]. The *remark* string may contain \n to indicate line-breaks.
  - **+t** to add a *title* comment to the output [Default is no title]. The *title* string may contain \n to indicate line-breaks.

  Blank lines and lines starting with # are always skipped.
- **incols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – Specify data columns for primary input in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default reads all columns in order, starting with the first (i.e., column 0)].

  - For a sequence: specify individual columns in input order (e.g., `incols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `incols="0:2,4+l"` to input the first three columns followed by the log10-transformed 5th column). To read from a given column until the end of the record, leave off *stop* when specifying the column range. To read trailing text, add the column **t**. Append the word number to **t** to ingest only a single word from the trailing text. Instead of specifying columns, use `incols="n"` to simply read numerical input and skip trailing text. Optionally, append one of the following modifiers to any column or column range to transform the input columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the input values by the factor *divisor* [Default is 1].
    - **+s** to multiple the input values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the input values [Default is 0].
- **distcalc** (*str*) – Determine how spherical distances are calculated [*Full usage*].
- **interpolation** (*str*) – [**b**|**c**|**l**|**n**][**+a**][**+b***BC*][**+c**][**+t***threshold*]. Select interpolation mode for grids. You can select the type of spline used:
  - **b** for B-spline
  - **c** for bicubic [Default]
  - **l** for bilinear
  - **n** for nearest-neighbor
- **outcols** (`int` | `str` | `Sequence`[`int` | `str`] | `None`, default: `None`) – *cols*[,…][,**t**[*word*]].

  Specify data columns for primary output in arbitrary order. Columns can be repeated and columns not listed will be skipped [Default writes all columns in order, starting with the first (i.e., column 0)].
  - For a sequence: specify individual columns in output order (e.g., `outcols=(1, 0)` for the 2nd column followed by the 1st column).
  - For a string: specify individual columns or column ranges in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified, with columns and/or column ranges separated by commas (e.g., `outcols="0:2,4"` to output the first three columns followed by the 5th column). To write from a given column until the end of the record, leave off *stop* when specifying the column range. To write trailing text, add the column **t**. Append the word number to **t** to write only a single word from the trailing text. Instead of specifying columns, use `outcols="n"` to simply read numerical input and skip trailing text. **Note**: If `incols` is also used then the columns given to `outcols` correspond to the order after the `incols` selection has taken place.

    Optionally, append one of the following modifiers to any column or column range to transform the output columns:
    - **+l** to take the *log10* of the input values.
    - **+d** to divide the output values by the factor *divisor* [Default is 1].
    - **+s** to multiply the output values by the factor *scale* [Default is 1].
    - **+o** to add the given *offset* to the output values [Default is 0].

- **skiprows** (*bool or str*) – [*cols*][**+a**][**+r**]. Suppress output for records whose *z*-value equals NaN [Default outputs all records]. Optionally, supply a comma-separated list of all columns or column ranges to consider for this NaN test [Default only considers the third data column (i.e., *cols* = *2*)]. Column ranges must be given in the format *start*[:*inc*]:*stop*, where *inc* defaults to 1 if not specified. The following modifiers are supported:
  - **+r** to reverse the suppression, i.e., only output the records whose *z*-value equals NaN.
  - **+a** to suppress the output of the record if just one or more of the columns equal NaN [Default skips record only if values in all specified *cols* equal NaN].
- **wrap** (*str*) – **y**|**a**|**w**|**d**|**h**|**m**|**s**|**c***period*[*/phase*][**+c***col*]. Convert the input *x*-coordinate to a cyclical coordinate, or a different column if selected via **+c***col*. The following cyclical coordinate transformations are supported:
  - **y**: yearly cycle (normalized)
  - **a**: annual cycle (monthly)
  - **w**: weekly cycle (day)
  - **d**: daily cycle (hour)
  - **h**: hourly cycle (minute)
  - **m**: minute cycle (second)
  - **s**: second cycle (second)
  - **c**: custom cycle (normalized)

  Full documentation is at https://docs.generic-mapping-tools.org/6.7/gmt.html#w-full.

Return type

`DataFrame` | `ndarray` | `None`

Returns

ret – Return type depends on `outfile` and `output_type`:

- `None` if `outfile` is set (output will be stored in file set by `outfile`)
- `pandas.DataFrame` or `numpy.ndarray` if `outfile` is not set (depends on `output_type`)

Examples

```
>>> import pygmt
>>> # Load a grid of @earth_relief_30m data, with a longitude range of
>>> # -118° E to -107° E, and a latitude range of -49° N to -42° N
>>> grid = pygmt.datasets.load_earth_relief(
...     resolution="30m", region=[-118, -107, -49, -42]
... )
>>> # Load a pandas dataframe with ocean ridge points
>>> points = pygmt.datasets.load_sample_data(name="ocean_ridge_points")
>>> # Create a pandas dataframe from an input grid and set of points
>>> # The output dataframe adds a column named "bathymetry"
>>> output_dataframe = pygmt.grdtrack(
...     points=points, grid=grid, newcolname="bathymetry"
... )
```

11.6 Class-style Parameters

Axis([annot, tick, grid, end_skip, label, ...])	Class for setting up one axis of a plot.
Box([clearance, fill, inner_gap, inner_pen, ...])	Class for specifying the box around GMT embellishments.
Frame([axes, title, subtitle, fill, axis, ...])	Class for setting up the frame and axes of a plot.
Pattern([pattern, dpi, bgcolor, fgcolor, invert])	Class for specifying bit and hachure patterns.
Position(refpoint[, cstype, anchor, offset])	Class for positioning embellishments on a plot.

11.6.1 pygmt.params.Axis

class pygmt.params.**Axis**(*annot=False*, *tick=False*, *grid=False*, *end_skip=False*, *label=None*, *alt_label=None*, *hlabel=None*, *alt_hlabel=None*, *prefix=None*, *unit=None*, *angle=None*)

Class for setting up one axis of a plot.

Examples

To specify the same attributes for x- and y-axes, with intervals of 4 for annotations, 2 for ticks, and 1 for gridlines:

```
>>> import pygmt
>>> from pygmt.params import Axis
>>> fig = pygmt.Figure()
>>> fig.basemap(
...     region=[0, 10, 0, 20],
...     projection="X10c/10c",
...     frame=Axis(annot=4, tick=2, grid=1),
... )
>>> fig.show()
```

annot: float | str | bool = False

Specify the interval for annotations. It can be `True` to let GMT decide the interval automatically; or a value to set a specific interval in the format of *stride*[±*phase*][*unit*], where, *stride* is the interval, *phase* is the offset to shift the annotations by that amount, and *unit* is one of the 18 supported unit codes related to time intervals.

tick: float | str | bool = False

Specify the interval for ticks. Same format as `annot`.

grid: float | str | bool = False

Specify the interval for gridlines. Same format as `annot`.

end_skip: Literal['lower', 'upper'] | bool = False

Skip annotations that fall exactly at the ends of the axis. Set to `True` to skip both ends, or use `"lower"`/`"upper"` to skip only the lower/upper end.

label: str | None = None

Label for the axis [Default is no label].

alt_label: str | None = None

Alternate label for the right or top axis [Default is the same as `label`].

hlabel: str | None = None

Horizontal label for y-axes, useful for very short labels.

alt_hlabel: str | None = None

Horizontal label for the right y-axis, useful for very short labels.

prefix: str | None = None

A leading text prefix for the axis annotations (e.g., dollar sign for plots related to money) [For Cartesian plots only].

unit: str | None = None

Unit to append to the axis annotations [For Cartesian plots only].

angle: float | None = None

Angle of the axis annotations.

Examples using `pygmt.params.Axis`

11.6.2 pygmt.params.Box

class pygmt.params.**Box**(*clearance=None*, *fill=None*, *inner_gap=None*, *inner_pen=None*, *pen=None*, *radius=False*, *shade_offset=None*, *shade_fill=None*)

Class for specifying the box around GMT embellishments.

Examples

```
>>> import pygmt
>>> from pygmt.params import Box
>>> fig = pygmt.Figure()
>>> fig.logo(box=Box(pen="1p", radius="5p", shade_offset=("5p", "5p")))
>>> fig.show()
```

clearance: float | str | Sequence[float | str] | None = None

Set clearances between the embellishment and the box border. It can be either a scalar value or a sequence of two/four values.

- a scalar value means a uniform clearance in all four directions.
- a sequence of two values means separate clearances in x- and y-directions.
- a sequence of four values means separate clearances for left/right/bottom/top.

fill: str | None = None

Fill for the box [Default is no fill].

inner_gap: float | str | None = None

Gap between the outer and inner borders [Default is "2p"].

inner_pen: str | None = None

Pen attributes for the inner border [Default to MAP_DEFAULT_PEN].

pen: str | None = None

Pen attributes for the box outline.

radius: str | bool = False

Draw a rounded rectangular border instead of sharp. Passing a value with unit to control the corner radius [Default is "6p"].

shade_offset: Sequence[float | str] | None = None

Place an offset background shaded region behind the box. A sequence of two values (dx, dy) indicates the shift relative to the foreground frame [Default is `("4p", "-4p")`].

shade_fill: str | None = None

Fill for the shaded region [Default is `"gray50"`].

Examples using `pygmt.params.Box`

11.6.3 pygmt.params.Frame

class pygmt.params.**Frame**(*axes=None*, *title=None*, *subtitle=None*, *fill=None*, *axis=None*, *axis2=None*, *xaxis=None*, *yaxis=None*, *zaxis=None*, *xaxis2=None*, *yaxis2=None*, *zaxis2=None*)

Class for setting up the frame and axes of a plot.

Examples

To draw the west and south axes with both ticks and annotations, and draw the east and north axes with ticks but without annotations:

```
>>> import pygmt
>>> from pygmt.params import Axis, Frame
>>> fig = pygmt.Figure()
>>> fig.basemap(
...     region=[0, 10, 0, 20], projection="X10c/10c", frame=Frame(axes="WSen")
... )
>>> fig.show()
```

To draw the west and south axes with both ticks and annotations, and draw the east and north axes without ticks and annotations. For west and south axes, specify the same attributes for both axes using the `axis` parameter, with intervals of 4 for annotations, 2 for ticks, and 1 for gridlines.

```
>>> fig = pygmt.Figure()
>>> fig.basemap(
...     region=[0, 10, 0, 20],
...     projection="X10c/10c",
...     frame=Frame(axes="WSrt", axis=Axis(annot=4, tick=2, grid=1)),
... )
>>> fig.show()
```

To specify the attributes for x- and y-axes separately:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(
...     region=[0, 10, 0, 20],
...     projection="X10c/10c",
...     frame=Frame(
...         axes="WSrt",
...         xaxis=Axis(annot=4, tick=2, grid=1, label="X Label"),
...         yaxis=Axis(annot=5, tick=2.5, grid=1, label="Y Label"),
...     ),
... )
>>> fig.show()
```

axes: str | None = None

Controls which axes are drawn and whether they are annotated, using a combination of the codes below. Axis omitted from the set will not be drawn.

For a 2-D plot, there are four axes: west, east, south, and north (or left, right, bottom, top if you prefer); For a 3-D plot, there is an extra Z-axis. They can be denoted by the following codes:

- **W** (west), **E** (east), **S** (south), **N** (north), **Z**: Draw axes with both ticks and annotations.
- **w** (west), **e** (east), **s** (south), **n** (north), **z**: Draw axes with ticks but without annotations.
- **l** (left), **r** (right), **b** (bottom), **t** (top), **u** (up): Draw axes without ticks or annotations.

For examples:

- `"WS"`: Draw the west and south axes with both ticks and annotations, but do not draw the east and north axes.
- `"WSen"`: Draw the west and south axes with both ticks and annotations, draw the east and north axes with ticks but without annotations.
- `"WSrt"`: Draw the west and south axes with both ticks and annotations, draw the east and north axes without ticks or annotations.
- `"WSrtZ"`: Draw the west and south axes with both ticks and annotations, draw the east and north axes without ticks or annotations, and draw the z-axis with both ticks and annotations.

For a 3-D plot, if the z-axis code is specified, a single vertical axis will be drawn at the most suitable corner by default. Append any combination of the corner IDs from 1 to 4 to draw one or more vertical axes at the corresponding corners (e.g., `"WSrtZ1234"`):

- **1**: the south-western (lower-left) corner
- **2**: the south-eastern (lower-right) corner
- **3**: the north-eastern (upper-right) corner
- **4**: the north-western (upper-left) corner

title: str | None = None

The title string centered above the plot frame [Default is no title].

subtitle: str | None = None

The subtitle string centered below the plot title. Requires `title` to be set. [Default is no subtitle].

fill: str | None = None

Fill for the interior of the frame with a color or a pattern [Default is no fill].

axis: *Axis* | None = None

Specify the attributes for axes by an *Axis* object.

The attributes for x and y axes can be specified in two ways: (1) specifying the same attributes for both axes using the `axis` parameter; (2) specifying the attributes for each axis separately using the `xaxis` and `yaxis` parameters. The attributes for z-axis can only be specified separately using the `zaxis` parameter.

GMT uses the notion of primary (the default) and secondary axes, while secondary axes are optional and mostly used for time axes annotations. To specify the same attributes for both secondary x and y axes, use the `axis2` parameter. To specify the attributes for the secondary axes separately, use the `xaxis2`, `yaxis2`, and `zaxis2` parameters.

Examples using `pygmt.params.Frame`

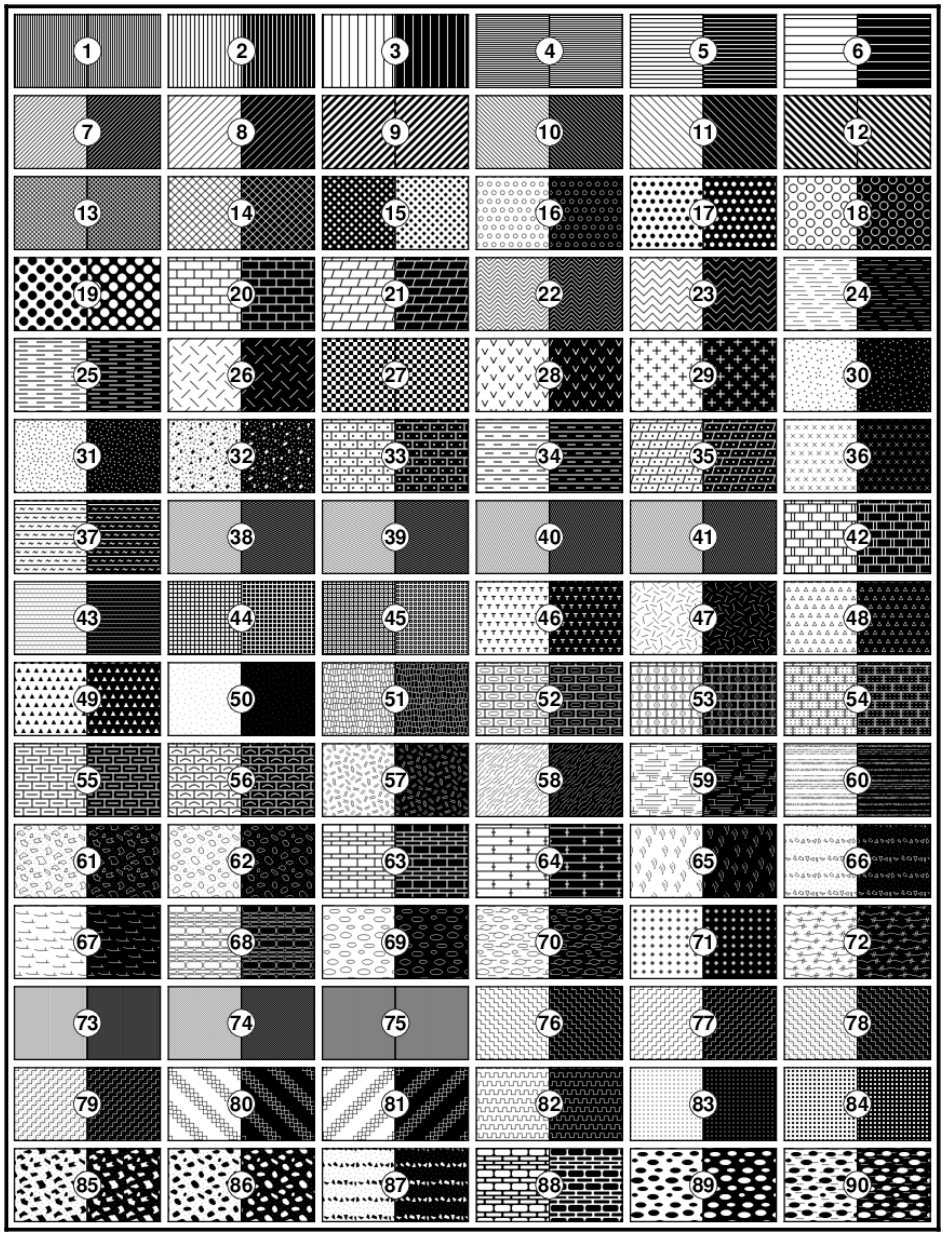
11.6.4 pygmt.params.Pattern

class pygmt.params.**Pattern**(*pattern=1*, *dpi=None*, *bgcolor=None*, *fgcolor=None*, *invert=False*)

Class for specifying bit and hachure patterns.

This class allows users to specify predefined bit-patterns or custom 1-, 8-, or 24-bit image raster files to fill symbols and polygons in various PyGMT plotting methods. The patterns can be customized with different resolution and different foreground and background colors. The foreground and background colors can also be inverted.

GMT provides 90 predefined patterns that can be used in PyGMT. The patterns are numbered from 1 to 90, and shown below:



Examples

Draw a global map with land areas filled with pattern 15 in a light red background and 200 dpi resolution:

```
>>> import pygmt
>>> from pygmt.params import Pattern
>>> fig = pygmt.Figure()
>>> fig.coast(
...     region="g",
...     projection="H10c",
...     frame=True,
...     land=Pattern(15, bgcolor="lightred", dpi=200),
...     shorelines=True,
... )
>>> fig.show()
```

pattern: int | str | PathLike = 1

The pattern to use. It can be specified in two forms:

- An integer in the range of 1-90, corresponding to one of 90 predefined 64x64 bit-patterns. [Default is 1].
- Name of a 1-, 8-, or 24-bit image raster file, to create customized, repeating images using image raster files.

dpi: int | None = None

Resolution of the pattern in dots per inch (DPI) [Default is 300].

bgcolor: str | None = None

The background color for predefined bit-patterns or 1-bit images. Setting either `bgcolor` or `fgcolor` to an empty string will yield a transparent background/foreground where only the foreground/background pixels will be painted. [Default is white for background and black for foreground].

fgcolor: str | None = None

The foreground color for predefined bit-patterns or 1-bit images.

invert: bool = False

If `True`, the pattern will be bit-inverted, i.e., white and black areas will be interchanged (only applies to predefined bit-patterns or 1-bit images).

Examples using `pygmt.params.Pattern`

11.6.5 pygmt.params.Position

class pygmt.params.**Position**(*refpoint*, *cstype=None*, *anchor=None*, *offset=None*)

Class for positioning embellishments on a plot.

This class provides flexible positioning for GMT embellishments (e.g., logo, scale, rose) by defining a *reference point* on the plot and an *anchor point* on the embellishment. The embellishment is positioned so these two points overlap.

Conceptual Model

Think of it like dropping an anchor from a boat:

1. The boat navigates to the *reference point* (a location on the plot)
2. The *anchor point* (a specific point on the embellishment) is aligned with the *reference point*
3. The embellishment is "dropped" at that position

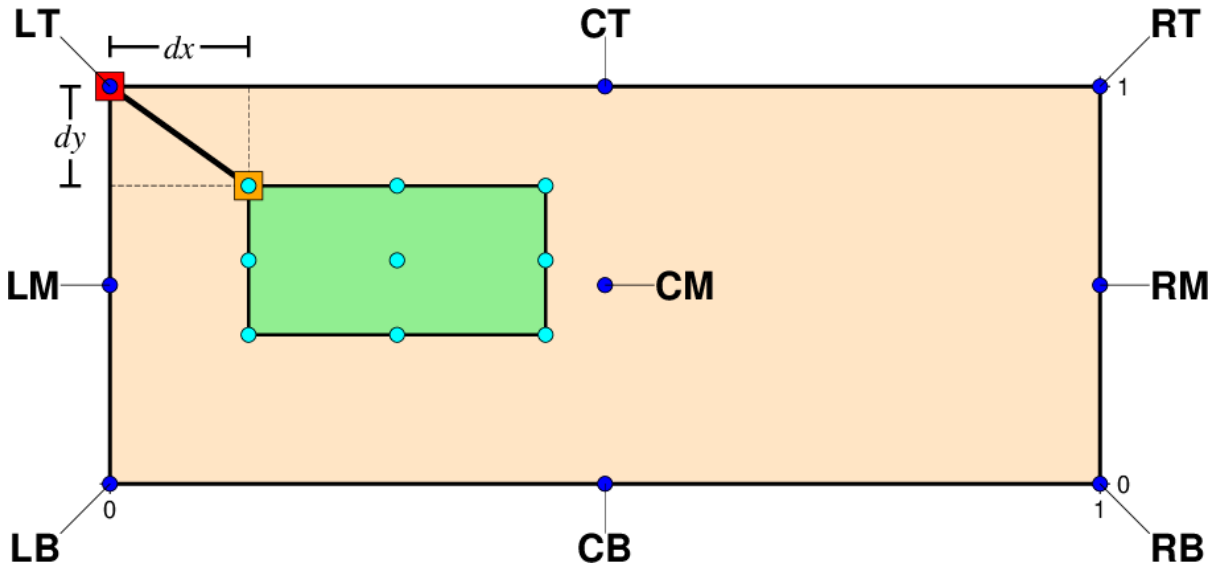


Fig. 2: The placement of a GMT embellishment (represented by a green rectangle) in relation to the underlying plot (represented by a bisque rectangle).

Reference Point

The *reference point* can be specified in five different ways using the `cstype` and `refpoint` attributes:

`cstype="mapcoords"` Map Coordinates
Use data/geographic coordinates. Specify `refpoint` as (*longitude*, *latitude*). Useful when tying the embellishment to a specific geographic location.

Example: `refpoint=(135, 20), cstype="mapcoords"`

`cstype="plotcoords"` Plot Coordinates
Use plot coordinates as distances from the lower-left plot origin. Specify `refpoint` as (*x*, *y*) with a *dimension unit*. Useful for precise layout control.

Example: `refpoint=("2c", "2.5c"), cstype="plotcoords"`

`cstype="boxcoords"` Normalized Coordinates
Use normalized coordinates where (0, 0) is the lower-left corner and (1, 1) is the upper-right corner of the bounding box of the current plot. Specify `refpoint` as (*nx*, *ny*). Useful for positioning relative to plot dimensions without units.

Example: `refpoint=(0.2, 0.1), cstype="boxcoords"`

`cstype="inside"` Inside Plot
Select one of the nine *justification codes* as the *reference point*. The *anchor point* defaults to be the same as the *reference point*, so the embellishment is placed inside the plot.

Example: `refpoint="TL", cstype="inside"`

`cstype="outside"` Outside Plot
Similar to `cstype="inside"`, but the *anchor point* defaults to the mirror opposite of the *reference point*. Useful for placing embellishments outside the plot boundaries (e.g., color bars).

Example: `refpoint="TL", cstype="outside"`

Anchor Point

The *anchor point* determines which part of the embellishment aligns with the *reference point*. It uses one of nine *justification codes*.

Set `anchor` explicitly to override these defaults. If not set, the default *anchor* behaviors are:

- `cstype="inside"`: Same as the *reference point* justification code
- `cstype="outside"`: Mirror opposite of the *reference point* justification code
- Other cstypes: `"MC"` (Middle Center) for map rose and scale, `"BL"` (Bottom Left) for other embellishments

Offset

The `offset` parameter shifts the *anchor point* from its default position. Offsets are applied to the projected plot coordinates, with positive values moving in the direction indicated by the *anchor point*'s justification code. It should be a single value (applied to both x and y) or as (*offset_x*, *offset_y*).

Examples

Position the GMT logo at map coordinates (3, 3) with the logo's middle-left point as the anchor, offset by (0.2, 0.2):

```
>>> import pygmt
>>> from pygmt.params import Position
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c", frame=True)
>>> fig.logo(
...     position=Position(
...         (3, 3), cstype="mapcoords", anchor="ML", offset=(0.2, 0.2)
...     ),
...     box=True,
... )
>>> fig.show()
```

Position the GMT logo in the top-left corner inside the plot:

```
>>> fig = pygmt.Figure()
>>> fig.basemap(region=[0, 10, 0, 10], projection="X10c", frame=True)
>>> fig.logo(position=Position("TL", cstype="inside", offset="0.2c"), box=True)
>>> fig.show()
```

refpoint: Sequence[float | str] | Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR']

Location of the reference point on the plot. The format depends on `cstype`:

- `cstype="mapcoords"`: (*longitude*, *latitude*)
- `cstype="plotcoords"`: (*x*, *y*) with plot units
- `cstype="boxcoords"`: (*nx*, *ny*)
- `cstype="inside"` or `"outside"`: *2-character justification codes*

cstype: Literal['mapcoords', 'inside', 'outside', 'boxcoords', 'plotcoords'] | None = None

cstype of the reference point. Valid values are:

- `"mapcoords"`: Map/Data coordinates
- `"plotcoords"`: Plot coordinates
- `"boxcoords"`: Normalized coordinates

- `"inside"` or `"outside"`: Justification codes

If not specified, defaults to `"inside"` if `refpoint` is a justification code; otherwise defaults to `"plotcoords"`.

anchor: Literal['TL', 'TC', 'TR', 'ML', 'MC', 'MR', 'BL', 'BC', 'BR'] | None = None

Anchor point on the embellishment using a *2-character justification code*. If `None`, defaults are applied based on `cstype` (see above).

offset: float | str | Sequence[float | str] | None = None

Offset for the anchor point as a single value or (*offset_x*, *offset_y*). If a single value is given, the offset is applied to both x- and y-directions.

Examples using `pygmt.params.Position`

11.7 Enums

GridRegistration	Enum for the grid registration.
GridType	Enum for the grid type.

11.7.1 pygmt.enums.GridRegistration

class pygmt.enums.**GridRegistration**(**values*)

Enum for the grid registration.

GRIDLINE = 0

Gridline registration

PIXEL = 1

Pixel registration

11.7.2 pygmt.enums.GridType

class pygmt.enums.**GridType**(**values*)

Enum for the grid type.

CARTESIAN = 0

Cartesian grid

GEOGRAPHIC = 1

Geographic grid

11.8 Miscellaneous

which(fname[, download, verbose])	Find full path to specified files.
show_versions([file])	Print various dependency versions which are useful when submitting bug reports.

11.8.1 pygmt.which

pygmt.**which**(*fname*, *download=False*, *verbose=False*, ***kwargs*)

Find full path to specified files.

Reports the full paths to the files given through `fname`. It looks for the file in (1) the current directory, (2) in $GMT_USERDIR (if defined), (3) in $GMT_DATADIR (if defined), or (4) in $GMT_CACHEDIR (if defined).

`fname` can also be a downloadable file (either a complete URL, an @file for downloading from the GMT data server, or any of the GMT remote datasets. In these cases, use the `download` parameter to set the desired behavior. If `download` is not used (or `False`), the file will not be found.

Full GMT docs at https://docs.generic-mapping-tools.org/6.7/gmtwhich.html.

Aliases

- G = download
- V = verbose

Parameters

- **fname** (`str` | `PathLike` | `Sequence`[`str` | `PathLike`]) – One or more file names to find the full path.
- **download** (`Literal`['auto', 'cache', 'local', 'user'] | `bool`, default: `False`) – Try to download the file if it is not found in your local data or cache directories and the file is downloadable. Here, downloadable files include:
  - a file specified by a complete URL
  - a GMT remote file on the GMT data server, specified with a leading `@`.
  - any of the GMT remote datasets

  Valid values are:
  - `False`: Do not download the file.
  - `True` or `"local"`: Download the file to the current directory.
  - `"cache"`: Download the file to the user cache directory.
  - `"user"`: Download the file to the user data directory but ignore any subdirectory structure.
  - `"auto"`: Download the file to appropriate folder under the user directory (this is where GMT normally places downloaded files).
- **verbose** (`Literal`['quiet', 'error', 'warning', 'timing', 'info', 'compat', 'debug'] | `bool`, default: `False`) – Select verbosity level [*Full usage*].

Return type
`str` | `list`[`str`]

Returns
path – The path(s) to the file(s), depending on the parameters used.

Raises
FileNotFoundError – If the file is not found.

11.8.2 pygmt.show_versions

pygmt.**show_versions**(*file=<_io.TextIOWrapper name='<stdout>' mode='w' encoding='utf-8'>*)

Print various dependency versions which are useful when submitting bug reports.

This includes information about:

- PyGMT itself
- System information (Python version, Operating System)
- Core dependency versions (NumPy, pandas, xarray, etc)
- GDAL and Ghostscript versions
- GMT library information

It also warns users if the installed Ghostscript version has serious bugs or is incompatible with the installed GMT version.

Return type

None

11.9 Datasets

PyGMT provides access to GMT's datasets through the `pygmt.datasets` module. These functions will download the datasets automatically the first time they are used and store them in GMT's user data directory.

datasets.list_sample_data()	Report datasets available for tests and documentation examples.
datasets.load_black_marble([resolution, region])	Load NASA Black Marble images in various resolutions.
datasets.load_blue_marble([resolution, region])	Load NASA Blue Marble images in various resolutions.
datasets.load_earth_age([resolution, ...])	Load the Earth seafloor crustal age dataset in various resolutions.
datasets.load_earth_deflection([resolution, ...])	Load the IGPP Earth east-west and north-south deflection datasets in various resolutions.
datasets.load_earth_dist([resolution, ...])	Load the GSHHG Earth distance to shoreline dataset in various resolutions.
datasets.load_earth_free_air_anomaly([...])	Load the IGPP Earth free-air anomaly and uncertainty datasets in various resolutions.
datasets.load_earth_geoid([resolution, ...])	Load the EGM2008 Earth geoid dataset in various resolutions.
datasets.load_earth_magnetic_anomaly([...])	Load the Earth magnetic anomaly datasets in various resolutions.
datasets.load_earth_mask([resolution, ...])	Load the GSHHG Earth mask dataset in various resolutions.
datasets.load_earth_mean_dynamic_topography	Load the CNES Earth mean dynamic topography dataset in various resolutions.
datasets.load_earth_mean_sea_surface([...])	Load the CNES Earth mean sea surface dataset in various resolutions.
datasets.load_earth_relief([resolution, ...])	Load the Earth relief datasets (topography and bathymetry) in various resolutions.
datasets.load_earth_vertical_gravity_gradi	Load the IGPP Earth vertical gravity gradient dataset in various resolutions.
datasets.load_mars_relief([resolution, ...])	Load the Mars relief dataset in various resolutions.
datasets.load_mercury_relief([resolution, ...])	Load the Mercury relief dataset in various resolutions.
datasets.load_moon_relief([resolution, ...])	Load the Moon relief dataset in various resolutions.

continues on next page

Table 24 – continued from previous page

datasets.load_pluto_relief([resolution, ...])	Load the Pluto relief dataset in various resolutions.
datasets.load_venus_relief([resolution, ...])	Load the Venus relief dataset in various resolutions.
datasets.load_sample_data(name)	Load an example dataset from the GMT server.

11.9.1 pygmt.datasets.list_sample_data

pygmt.datasets.**list_sample_data**()

Report datasets available for tests and documentation examples.

Return type
dict[str, str]

Returns
dict – Names and short descriptions of available sample datasets.

See also

load_sample_data
Load an example dataset from the GMT server.

11.9.2 pygmt.datasets.load_black_marble

pygmt.datasets.**load_black_marble**(*resolution='01d'*, *region=None*)

Load NASA Black Marble images in various resolutions.

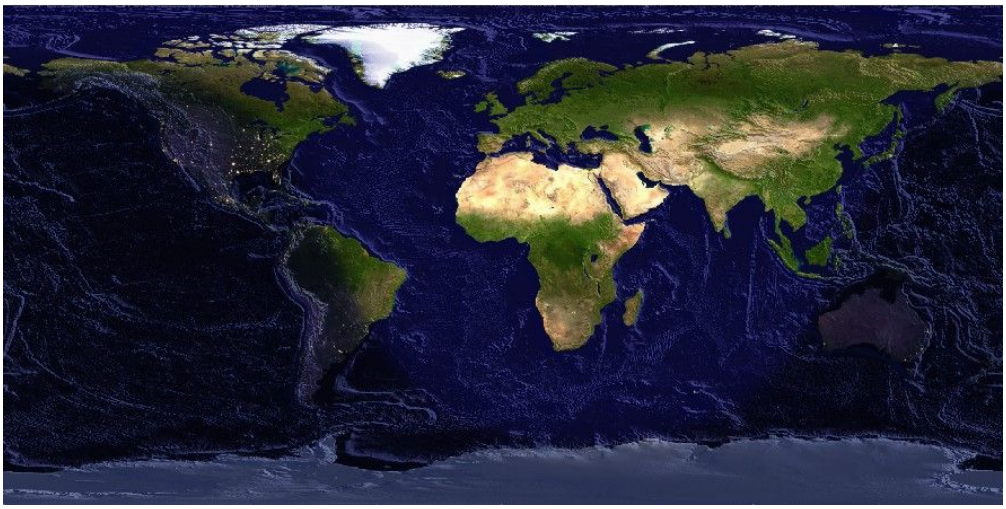


Fig. 3: Earth day/night dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_night/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_night**_*res*. *res* is the grid resolution. If *res* is omitted (i.e., `@earth_night`), GMT automatically selects a suitable resolution based on the current region and projection settings.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-daynight.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s'], default: '01d') – The image resolution. The suffix d, m, and s stand for arc-degrees, arc-minutes, and arc-seconds.
- **region** (Sequence[float] | str | None, default: None) – The subregion of the image to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*].

Return type

DataArray

Returns

image – The NASA Black Marble image. Coordinates are latitude and longitude in degrees.

Note

The registration and coordinate system type of the returned xarray.DataArray image can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_black_marble
>>> # Load the default image (pixel-registered 1 arc-degree image)
>>> image = load_black_marble()
```

11.9.3 pygmt.datasets.load_blue_marble

pygmt.datasets.**load_blue_marble**(*resolution='01d'*, *region=None*)

Load NASA Blue Marble images in various resolutions.

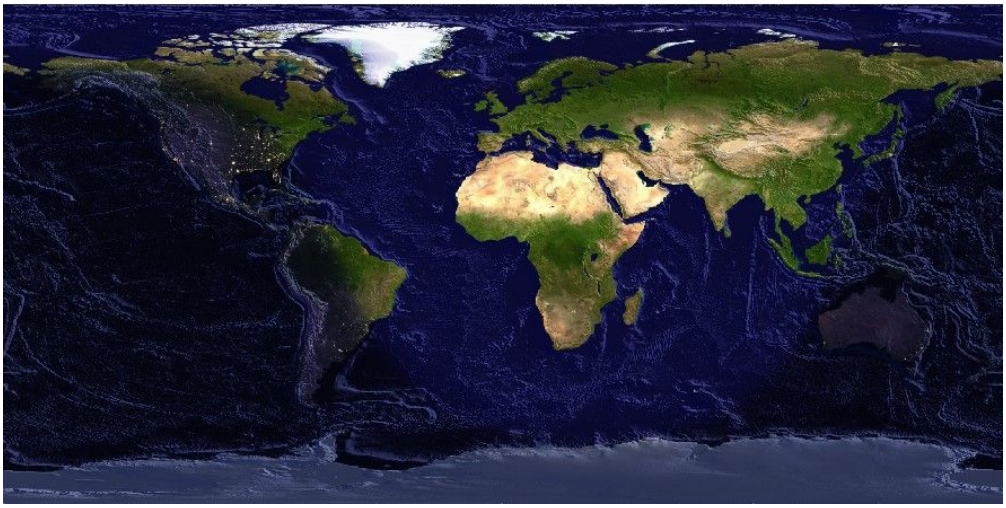


Fig. 4: Earth day/night dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually ~/.gmt/server/earth/earth_day/), and load the dataset as an xarray.DataArray. An internet connection is

required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any image processing function or plotting method, using the following file name format: **@earth_day**_*res*. *res* is the image resolution. If *res* is omitted (i.e., `@earth_day`), GMT automatically selects a suitable resolution based on the current region and projection settings.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-daynight.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s']`, default: `'01d'`) – The image resolution. The suffix `d`, `m`, and `s` stand for arc-degrees, arc-minutes, and arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the image to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*].

Return type

`DataArray`

Returns

image – The NASA Blue Marble image. Coordinates are latitude and longitude in degrees.

Note

The registration and coordinate system type of the returned `xarray.DataArray` image can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_blue_marble
>>> # Load the default image (pixel-registered 1 arc-degree image)
>>> image = load_blue_marble()
```

11.9.4 pygmt.datasets.load_earth_age

`pygmt.datasets.`**`load_earth_age`**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the Earth seafloor crustal age dataset in various resolutions.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_age/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_age**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_age_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_age`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@earth_age.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_age.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-age.html for more details about available datasets, including version information and references.

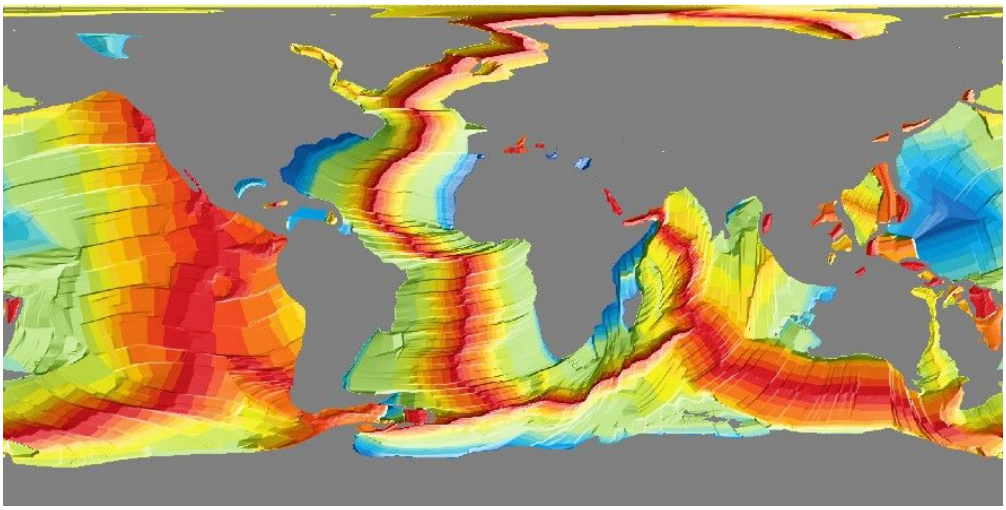


Fig. 5: Earth seafloor crustal age dataset.

Parameters

- **resolution** (Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m'], default: '01d') – The grid resolution. The suffix d and m stand for arc-degrees and arc-minutes.
- **region** (Sequence[float] | str | None, default: None) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., "05m").
- **registration** (Literal['gridline', 'pixel'], default: 'gridline') – Grid registration type. Either "pixel" for pixel registration or "gridline" for gridline registration.

Return type

DataArray

Returns

grid – The Earth seafloor crustal age grid. Coordinates are latitude and longitude in degrees. Age is in millions of years (Myr).

Note

The registration and coordinate system type of the returned xarray.DataArray grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_age
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_age()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_age(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_age(
...     resolution="05m",
```

(continues on next page)

(continued from previous page)

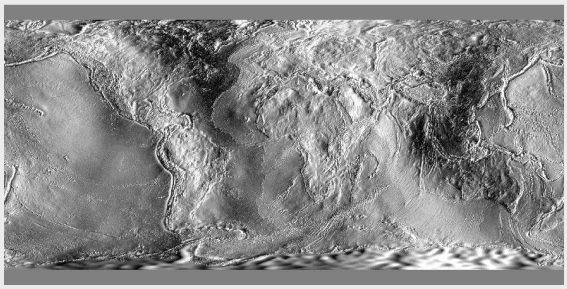
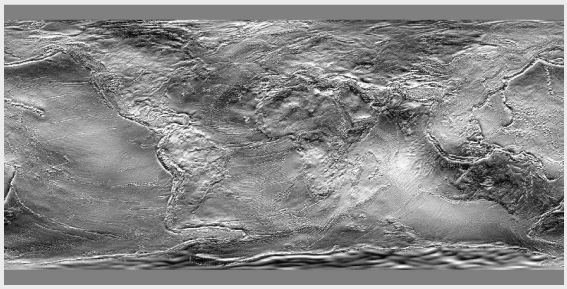
```
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

Examples using `pygmt.datasets.load_earth_age`

11.9.5 pygmt.datasets.load_earth_deflection

`pygmt.datasets.`**`load_earth_deflection`**(*resolution='01d'*, *region=None*, *registration=None*, *component='east'*)

Load the IGPP Earth east-west and north-south deflection datasets in various resolutions.

IGPP Earth east-west deflection	IGPP Earth north-south deflection
	

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_edefl/` and `~/.gmt/server/earth/earth_ndefl/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The datasets can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@***earth_defl_type_res_reg*. *earth_defl_type* is the GMT name for the dataset. The available options are **earth_edefl** and **earth_ndefl**; *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_edefl_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_edefl`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@earth_defl.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_defl.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-edefl.html and https://www.generic-mapping-tools.org/remote-datasets/earth-ndefl.html for more details about available datasets, including version information and references.

Parameters

- **`resolution`** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **`region`** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).

- **registration** (Literal['gridline', 'pixel', None], default: None) – Grid registration type. Either "pixel" for pixel registration or "gridline" for gridline registration. Default is None, which means "gridline" for all resolutions except "01m" which is "pixel" only.
- **component** (Literal['east', 'north'], default: 'east') – By default, the east-west deflection (component="east") is returned, set component="north" to return the north-south deflection.

Return type
DataArray

Returns
grid – The Earth east-west or north-south deflection grid. Coordinates are latitude and longitude in degrees. Deflection values are in micro-radians, where positive (negative) values indicate a deflection to the east or north (west or south).

Note

The registration and coordinate system type of the returned xarray.DataArray grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_deflection
>>> # Load the default grid for east-west deflection (gridline-registered
>>> # 1 arc-degree grid)
>>> grid = load_earth_deflection()
>>> # Load the default grid for north-south deflection
>>> grid = load_earth_deflection(component="north")
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_deflection(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_deflection(
...     resolution="05m", region=[120, 160, 30, 60], registration="gridline"
... )
```

11.9.6 pygmt.datasets.load_earth_dist

pygmt.datasets.**load_earth_dist**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the GSHHG Earth distance to shoreline dataset in various resolutions.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually ~/.gmt/server/earth/earth_dist/), and load the dataset as an xarray.DataArray. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_dist**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., @earth_dist_01d), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., @earth_dist), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, @earth_dist.cpt. To use the dataset-specific CPT when plotting the dataset, explicitly set cmap="@earth_dist.cpt", otherwise GMT's default CPT (*google/turbo*) will

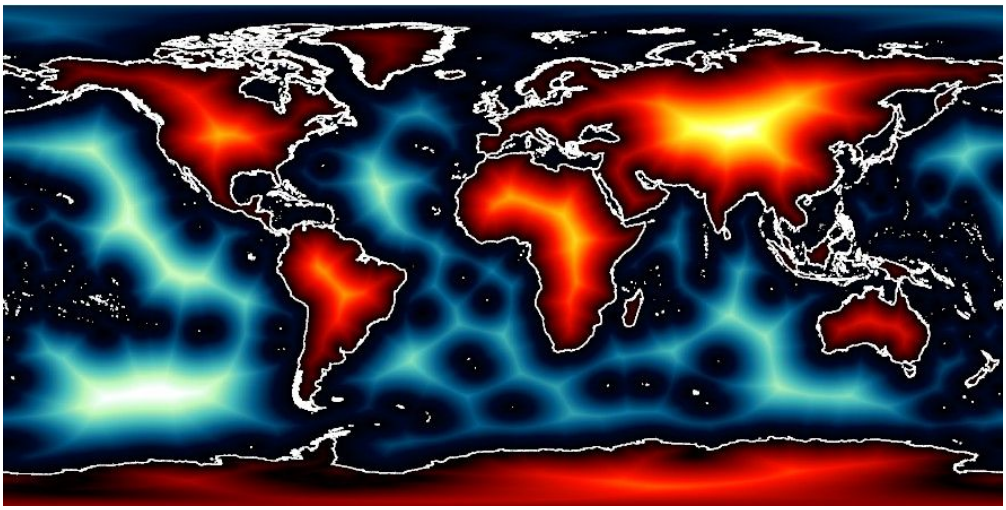


Fig. 6: GSHHG Earth distance to shoreline dataset.

be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-dist.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel']`, default: `'gridline'`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration.

Return type

`DataArray`

Returns

grid – The GSHHG Earth distance to shoreline grid. Coordinates are latitude and longitude in degrees. Distances are in kilometers, where positive (negative) values mean land to coastline (ocean to coastline).

> **Note**
>
> The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

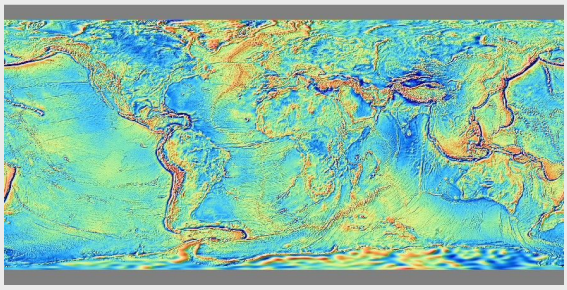
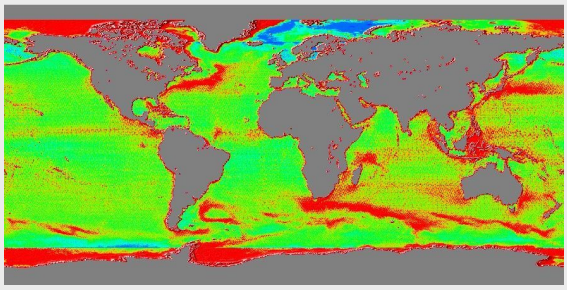
Examples

```
>>> from pygmt.datasets import load_earth_dist
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_dist()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_dist(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_dist(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.7 pygmt.datasets.load_earth_free_air_anomaly

pygmt.datasets.**load_earth_free_air_anomaly**(*resolution='01d'*, *region=None*, *registration=None*, *uncertainty=False*)

Load the IGPP Earth free-air anomaly and uncertainty datasets in various resolutions.

IGPP Earth free-air anomaly	IGPP Earth free-air anomaly uncertainty
	

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_faa/` or `~/.gmt/server/earth/earth_faaerror/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: *@earth_faa_type_res_reg*. *earth_faa_type* is the GMT name for the dataset. The available options are **earth_faa** and **earth_faaerror**. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_faa_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_faa`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with two color palette table (CPT) files, `@earth_faa.cpt` and `@earth_faaerror.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_faa.cpt"` or `cmap="@earth_faaerror.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-faa.html and https://www.generic-mapping-tools.org/remote-datasets/earth-faaerror.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except `"01m"` which is `"pixel"` only.
- **uncertainty** (`bool`, default: `False`) – By default, the Earth free-air anomaly values are returned. Set to `True` to return the related uncertainties instead.

Return type

`DataArray`

Returns

grid – The Earth free-air anomaly (uncertainty) grid. Coordinates are latitude and longitude in

degrees. Values and uncertainties are in mGal.

> **Note**
>
> The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_free_air_anomaly
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_free_air_anomaly()
>>> # Load the uncertainties related to the default grid
>>> grid = load_earth_free_air_anomaly(uncertainty=True)
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_free_air_anomaly(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_free_air_anomaly(
...     resolution="05m", region=[120, 160, 30, 60], registration="gridline"
... )
```

11.9.8 pygmt.datasets.load_earth_geoid

`pygmt.datasets.`**`load_earth_geoid`**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the EGM2008 Earth geoid dataset in various resolutions.

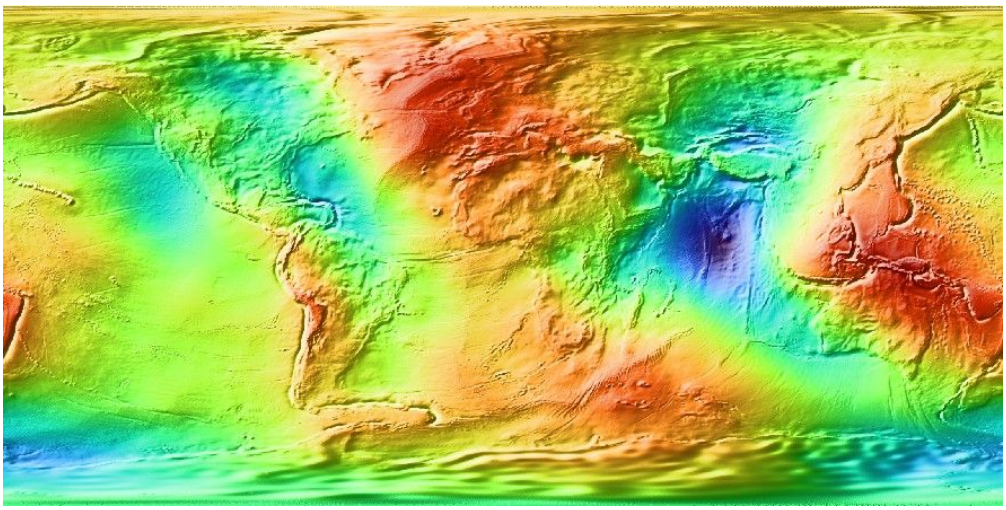


Fig. 7: EGM2008 Earth geoid dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_geoid/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_geoid**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_geoid_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting

functions. If *res* is also omitted (i.e., `@earth_geoid`), GMT automatically selects a suitable resolution based on the current region and projection settings.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-geoid.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel']`, default: `'gridline'`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration.

Return type

`DataArray`

Returns

grid – The Earth geoid grid. Coordinates are latitude and longitude in degrees. Units are in meters.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

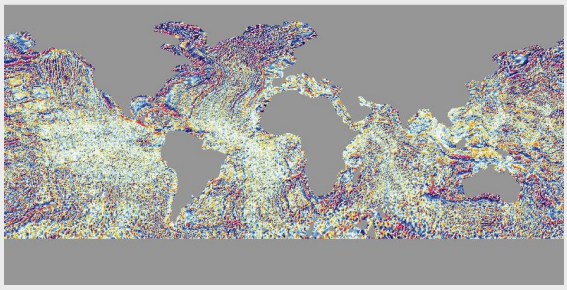
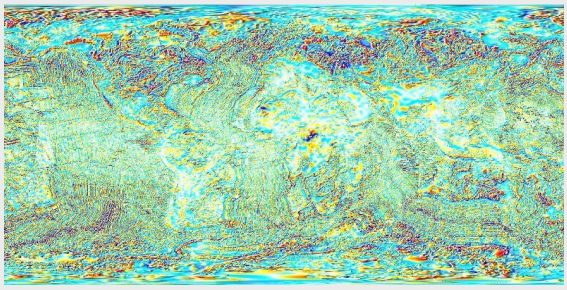
Examples

```
>>> from pygmt.datasets import load_earth_geoid
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_geoid()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_geoid(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_geoid(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.9 pygmt.datasets.load_earth_magnetic_anomaly

`pygmt.datasets.`**`load_earth_magnetic_anomaly`**(*resolution='01d'*, *region=None*, *registration=None*, *data_source='emag2'*)

Load the Earth magnetic anomaly datasets in various resolutions.

Earth Magnetic Anomaly Model (EMAG2)	World Digital Magnetic Anomaly Map (WDMAM)
	

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_mag/`, `~/.gmt/server/earth/earth_mag4km/`, `~/.gmt/server/earth/earth_wdmam/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@***earth_mag_type_res_reg*. *earth_mag_type* is the GMT name for the dataset. The available options are **earth_mag**, **earth_mag4km**, and **earth_wdmam**; *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_mag_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_mag`), GMT automatically selects a suitable resolution based on the current region and projection settings.

The default color palette tables (CPTs) for this dataset are `@earth_mag.cpt` for `data_source="emag2"` and `data_source="emag2_4km"`, and `@earth_wdmam.cpt` for `data_source="wdmam"`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_mag.cpt"` or `cmap="@earth_wdmam.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-mag.html and https://www.generic-mapping-tools.org/remote-datasets/earth-wdmam.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes. The resolution `"02m"` is not available for `data_source="wdmam"`.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except `"02m"` for `data_source="emag2"` or `data_source="emag2_4km"`, which are `"pixel"` only.
- **data_source** (`Literal['emag2', 'emag2_4km', 'wdmam']`, default: `'emag2'`) – Select the source of the magnetic anomaly data. Available options are:
  - `"emag2"`: EMAG2 Earth Magnetic Anomaly Model. It only includes data observed at sea level over oceanic regions. See https://www.generic-mapping-tools.org/remote-datasets/earth-mag.html.

- `"emag2_4km"`: Use a version of EMAG2 where all observations are relative to an altitude of 4 km above the geoid and include data over land.
- `"wdmam"`: World Digital Magnetic Anomaly Map (WDMAM). See https://www.generic-mapping-tools.org/remote-datasets/earth-wdmam.html.

Return type
`DataArray`

Returns
grid – The Earth magnetic anomaly grid. Coordinates are latitude and longitude in degrees. Units are in nano Tesla (nT).

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_magnetic_anomaly
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_magnetic_anomaly()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_magnetic_anomaly(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_magnetic_anomaly(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
>>> # Load the 20 arc-minutes grid of the emag2_4km dataset
>>> grid = load_earth_magnetic_anomaly(
...     resolution="20m", registration="gridline", data_source="emag2_4km"
... )
>>> # Load the 20 arc-minutes grid of the WDMAM dataset
>>> grid = load_earth_magnetic_anomaly(
...     resolution="20m", registration="gridline", data_source="wdmam"
... )
```

11.9.10 pygmt.datasets.load_earth_mask

`pygmt.datasets.`**`load_earth_mask`**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the GSHHG Earth mask dataset in various resolutions.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_mask/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_mask**_*res_reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_mask_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_mask`), GMT automatically selects a suitable resolution based on the current region and projection settings.

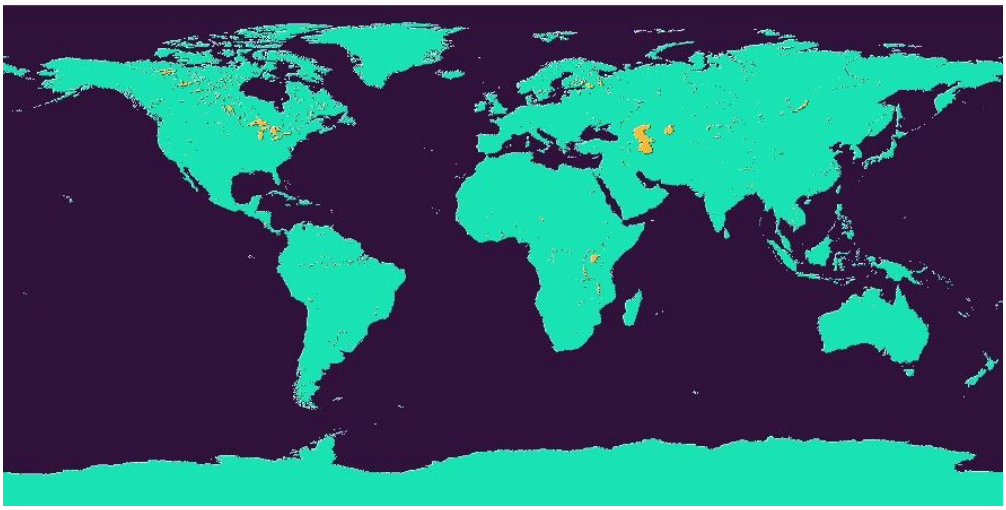


Fig. 8: GSHHG Earth mask dataset.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-mask.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s', '15s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m`, and `s` stand for arc-degrees, arc-minutes, and arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code.
- **registration** (`Literal['gridline', 'pixel']`, default: `'gridline'`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration.

Return type

`DataArray`

Returns

grid – The Earth mask grid. Coordinates are latitude and longitude in degrees. The node values in the mask grids are all in the 0-4 range and reflect different surface types:

- 0: Oceanic areas beyond the shoreline
- 1: Land areas inside the shoreline
- 2: Lakes inside the land areas
- 3: Islands in lakes in the land areas
- 4: Smaller lakes in islands that are found within lakes inside the land area

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_mask
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_mask()
>>> # location (120°E, 50°N) is in land area (1)
>>> grid.sel(lon=120, lat=50).values
array(1, dtype=int8)
>>> # location (170°E, 50°N) is in oceanic area (0)
>>> grid.sel(lon=170, lat=50).values
array(0, dtype=int8)
```

11.9.11 pygmt.datasets.load_earth_mean_dynamic_topography

pygmt.datasets.**load_earth_mean_dynamic_topography**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the CNES Earth mean dynamic topography dataset in various resolutions.

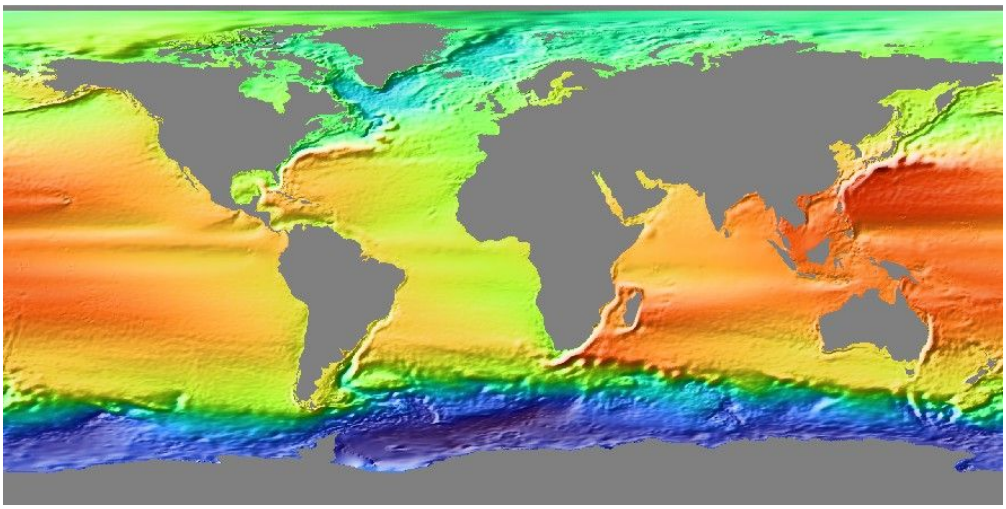


Fig. 9: CNES Earth mean dynamic topography dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_mdt/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_mdt**_*res_reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_mdt_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_mdt`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@earth_mdt.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_mdt.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-mdt.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (Literal['01d', '30m', '20m', '15m', '10m', '07m'], default: '01d') – The grid resolution. The suffix d and m stand for arc-degrees and arc-minutes. Note that "07m" refers to a resolution of 7.5 arc-minutes.
- **region** (Sequence[float] | str | None, default: None) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code.
- **registration** (Literal['gridline', 'pixel'], default: 'gridline') – Grid registration type. Either "pixel" for pixel registration or "gridline" for gridline registration.

Return type

DataArray

Returns

grid – The CNES Earth mean dynamic topography grid. Coordinates are latitude and longitude in degrees. Values are in meters.

Note

The registration and coordinate system type of the returned xarray.DataArray grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_mean_dynamic_topography
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_mean_dynamic_topography()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_mean_dynamic_topography(
...     resolution="30m", registration="gridline"
... )
>>> # Load high-resolution (7 arc-minutes) grid for a specific region
>>> grid = load_earth_mean_dynamic_topography(
...     resolution="07m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.12 pygmt.datasets.load_earth_mean_sea_surface

pygmt.datasets.**load_earth_mean_sea_surface**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the CNES Earth mean sea surface dataset in various resolutions.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually ~/.gmt/server/earth/earth_mss/), and load the dataset as an xarray.DataArray. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_mss**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., @earth_mss_01d), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., @earth_mss), GMT automatically selects a suitable resolution based on the current region and projection settings.

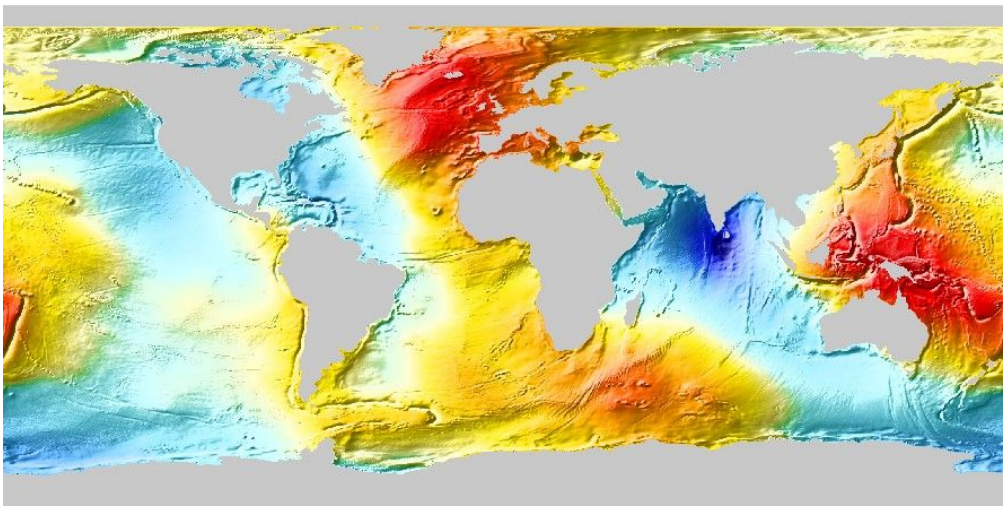


Fig. 10: CNES Earth mean sea surface dataset.

This dataset comes with a color palette table (CPT) file, `@earth_mss.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_mss.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-mss.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel']`, default: `'gridline'`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration.

Return type

`DataArray`

Returns

grid – The CNES Earth mean sea surface grid. Coordinates are latitude and longitude in degrees. Values are in meters.

> **Note**
>
> The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_mean_sea_surface
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_mean_sea_surface()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_mean_sea_surface(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_mean_sea_surface(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.13 pygmt.datasets.load_earth_relief

`pygmt.datasets.`**`load_earth_relief`**(*resolution='01d'*, *region=None*, *registration=None*, *data_source='igpp'*, *use_srtm=False*)

Load the Earth relief datasets (topography and bathymetry) in various resolutions.

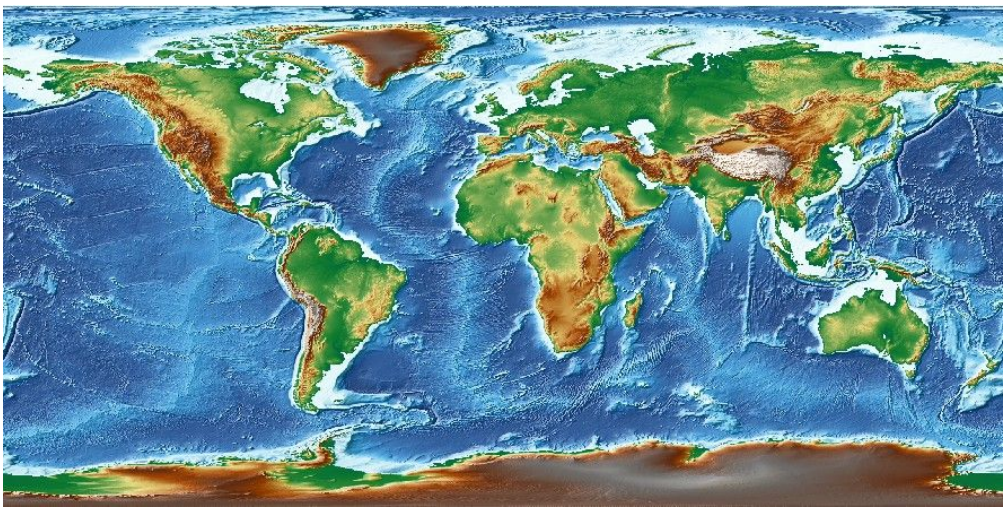


Fig. 11: Earth relief datasets (topography and bathymetry).

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_relief`, `~/.gmt/server/earth/earth_gebco`, `~/.gmt/server/earth/earth_gebcosi`, `~/.gmt/server/earth/earth_synbath`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: *@earth_relief_type_res_reg*. *earth_relief_type* is the GMT name for the dataset. The available options are **earth_relief**, **earth_gebco**, **earth_gebcosi**, and **earth_synbath**. *res* is the grid

resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid willcbe loaded for plotting functions. If *res* is also omitted (i.e., `@earth_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `gmt/geo`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="gmt/geo"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s', '15s', '03s', '01s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m` and `s` stand for arc-degrees, arc-minutes, and arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except `"15s"` which is `"pixel"` only.
- **data_source** (`Literal['igpp', 'gebco', 'gebcosi', 'synbath']`, default: `'igpp'`) – Select the source for the Earth relief data. Available options are:
  - `"igpp"`: IGPP Earth Relief. See https://www.generic-mapping-tools.org/remote-datasets/earth-relief.html.
  - `"synbath"`: IGPP Earth Relief dataset that uses statistical properties of young seafloor to provide a more realistic relief of young areas with small seamounts.
  - `"gebco"`: GEBCO Earth Relief with only observed relief and inferred relief via altimetric gravity. See https://www.generic-mapping-tools.org/remote-datasets/earth-gebco.html.
  - `"gebcosi"`: GEBCO Earth Relief that gives sub-ice (si) elevations.

  Notes: Only the `"igpp"` data source provides the highest resolutions `"03s"` and `"01s"`.
- **use_srtm** (`bool`, default: `False`) – For the resolutions `"03s"` and `"01s"`, by default the land-only SRTM tiles from NASA are used along with up-sampled SRTM15 tiles (with a resolution of 15 arc-seconds) to fill in the missing ocean values. If set to `True`, only the original land-only SRTM tiles are loaded without filling in the ocean values. Only available for `data_source="igpp"`.

Return type

`DataArray`

Returns

grid – The Earth relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
>>> # Load the original 3 arc-seconds land-only SRTM tiles from NASA
>>> grid = load_earth_relief(
...     resolution="03s",
...     region=[135, 136, 35, 36],
...     registration="gridline",
...     use_srtm=True,
... )
```

Examples using `pygmt.datasets.load_earth_relief`

11.9.14 pygmt.datasets.load_earth_vertical_gravity_gradient

`pygmt.datasets.`**`load_earth_vertical_gravity_gradient`**(*resolution='01d'*, *region=None*, *registration=None*)

Load the IGPP Earth vertical gravity gradient dataset in various resolutions.

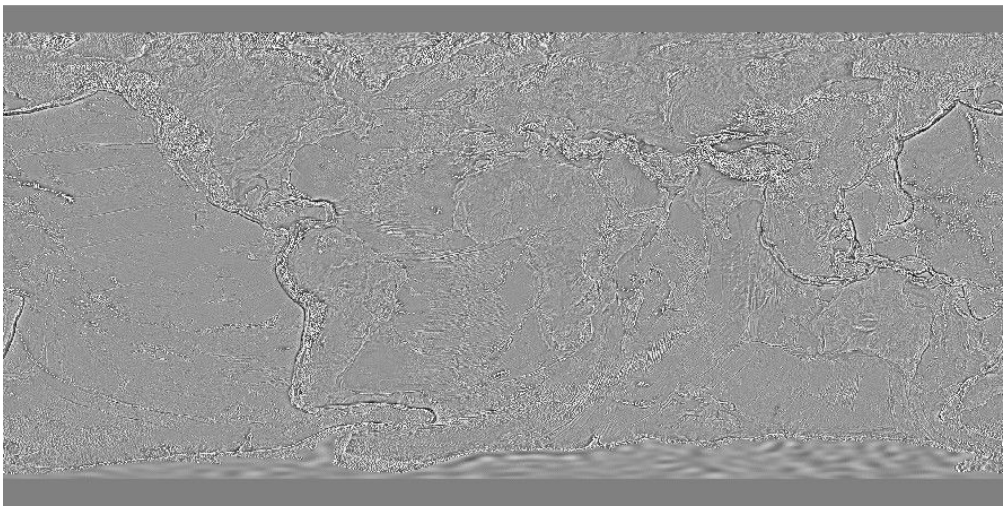


Fig. 12: IGPP Earth vertical gravity gradient dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/earth/earth_vgg/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@earth_vgg**_*res_reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@earth_vgg_01d`), the gridline-registered

grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@earth_vgg`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@earth_vgg.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@earth_vgg.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/earth-vgg.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except `"01m"` which is `"pixel"` only.

Return type

`DataArray`

Returns

grid – The Earth vertical gravity gradient grid. Coordinates are latitude and longitude in degrees. Units are in Eotvos.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_earth_vertical_gravity_gradient
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_earth_vertical_gravity_gradient()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_earth_vertical_gravity_gradient(
...     resolution="30m", registration="gridline"
... )
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_earth_vertical_gravity_gradient(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.15 pygmt.datasets.load_mars_relief

pygmt.datasets.**load_mars_relief**(*resolution='01d'*, *region=None*, *registration=None*)

Load the Mars relief dataset in various resolutions.

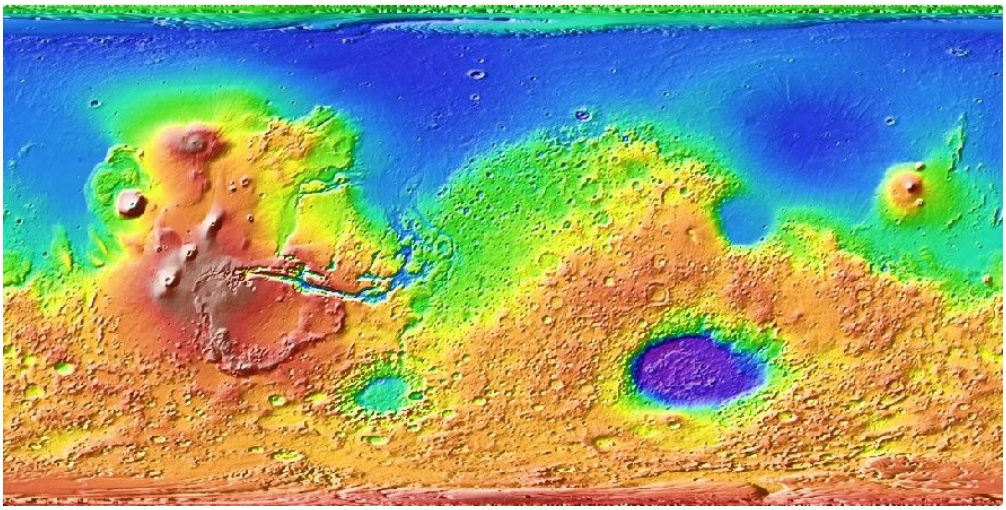


Fig. 13: Mars relief dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/mars/mars_relief/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@mars_relief**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@mars_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@mars_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@mars_relief.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@mars_relief.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/mars-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s', '15s', '12s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m` and `s` stand for arc-degrees, arc-minutes and arc-seconds. Note that `"12s"` refers to a resolution of 12.1468873601 arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except for `"12s"` which is `"pixel"` only.

Return type
DataArray

Returns
grid – The Mars relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_mars_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_mars_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_mars_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_mars_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.16 pygmt.datasets.load_mercury_relief

pygmt.datasets.**load_mercury_relief**(*resolution='01d'*, *region=None*, *registration=None*)

Load the Mercury relief dataset in various resolutions.

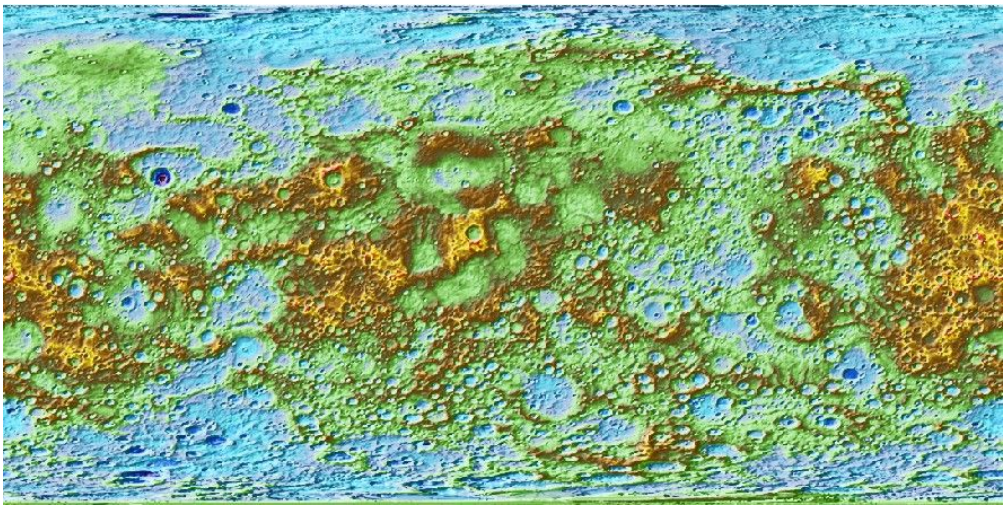


Fig. 14: Mercury relief dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/mercury/mercury_relief/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@mercury_relief**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@mercury_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@mercury_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@mercury_relief.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@mercury_relief.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/mercury-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '56s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m` and `s` stand for arc-degrees, arc-minutes and arc-seconds. Note that `"56s"` refers to a resolution of 56.25 arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except for `"56s"` which is `"pixel"` only.

Return type

`DataArray`

Returns

grid – The Mercury relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_mercury_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_mercury_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_mercury_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_mercury_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.17 pygmt.datasets.load_moon_relief

pygmt.datasets.**load_moon_relief**(*resolution='01d'*, *region=None*, *registration=None*)

Load the Moon relief dataset in various resolutions.

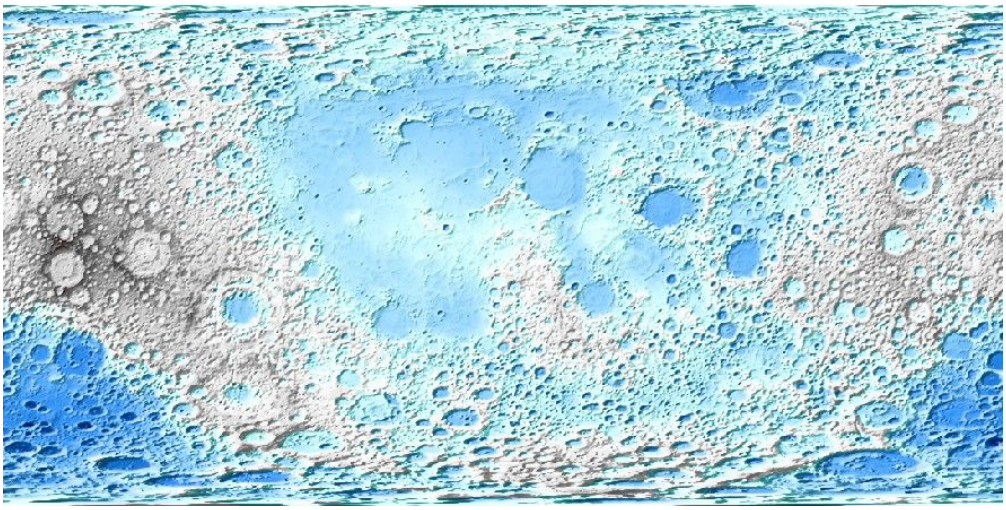


Fig. 15: Moon relief dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/moon/moon_relief/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@moon_relief**_*res_reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@moon_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@moon_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@moon_relief.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@moon_relief.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/moon-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '30s', '15s', '14s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m` and `s` stand for arc-degrees, arc-minutes and arc-seconds. Note that `"14s"` refers to a resolution of 14.0625 arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except for `"14s"` which is `"pixel"` only.

Return type
DataArray

Returns
grid – The Moon relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

> **Note**
>
> The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_moon_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_moon_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_moon_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_moon_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.18 pygmt.datasets.load_pluto_relief

pygmt.datasets.**load_pluto_relief**(*resolution='01d'*, *region=None*, *registration=None*)

Load the Pluto relief dataset in various resolutions.

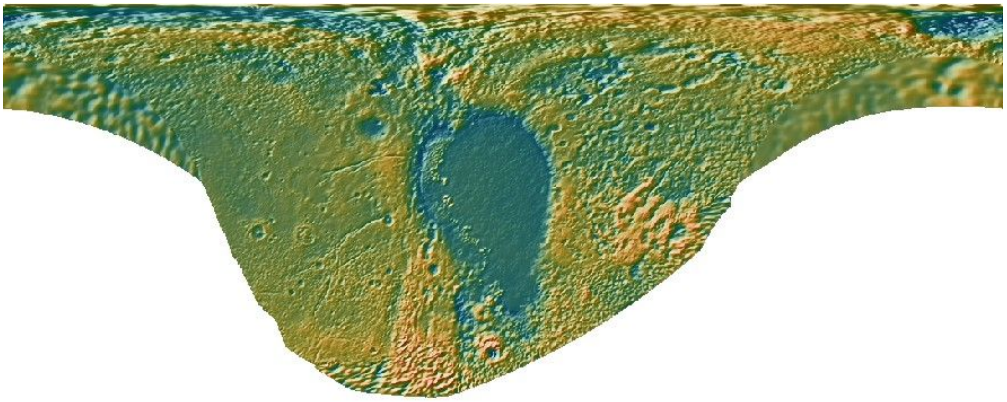


Fig. 16: Pluto relief dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/pluto/pluto_relief/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@pluto_relief**_*res*_*reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@pluto_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@pluto_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@pluto_relief.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@pluto_relief.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/pluto-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m', '52s']`, default: `'01d'`) – The grid resolution. The suffix `d`, `m` and `s` stand for arc-degrees, arc-minutes and arc-seconds. Note that `"52s"` refers to a resolution of 52.0732883317 arc-seconds.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel', None]`, default: `None`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration. Default is `None`, which means `"gridline"` for all resolutions except for `"52s"` which is `"pixel"` only.

Return type

`DataArray`

Returns

grid – The Pluto relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

Note

The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *pygmt.GMTDataArrayAccessor* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_pluto_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_pluto_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_pluto_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_pluto_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.19 pygmt.datasets.load_venus_relief

pygmt.datasets.**load_venus_relief**(*resolution='01d'*, *region=None*, *registration='gridline'*)

Load the Venus relief dataset in various resolutions.

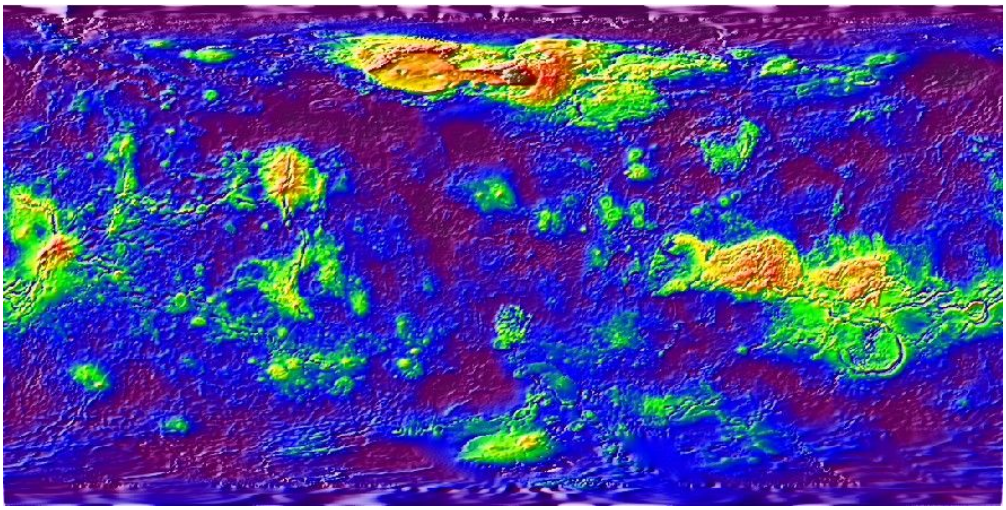


Fig. 17: Venus relief dataset.

This function downloads the dataset from the GMT data server, caches it in a user data directory (usually `~/.gmt/server/venus/venus_relief/`), and load the dataset as an `xarray.DataArray`. An internet connection is required the first time around, but subsequent calls will load the dataset from the local data directory.

The dataset can also be accessed by specifying a file name in any grid processing function or plotting method, using the following file name format: **@venus_relief**_*res_reg*. *res* is the grid resolution; *reg* is the grid registration type (**p** for pixel registration, **g** for gridline registration). If *reg* is omitted (e.g., `@venus_relief_01d`), the gridline-registered grid will be loaded for grid processing functions and the pixel-registered grid will be loaded for plotting functions. If *res* is also omitted (i.e., `@venus_relief`), GMT automatically selects a suitable resolution based on the current region and projection settings.

This dataset comes with a color palette table (CPT) file, `@venus_relief.cpt`. To use the dataset-specific CPT when plotting the dataset, explicitly set `cmap="@venus_relief.cpt"`, otherwise GMT's default CPT (*google/turbo*) will be used. If the dataset is referenced by the file name in a grid plotting method, the dataset-specific CPT file is used automatically unless another CPT is specified.

Refer to https://www.generic-mapping-tools.org/remote-datasets/venus-relief.html for more details about available datasets, including version information and references.

Parameters

- **resolution** (`Literal['01d', '30m', '20m', '15m', '10m', '06m', '05m', '04m', '03m', '02m', '01m']`, default: `'01d'`) – The grid resolution. The suffix `d` and `m` stand for arc-degrees and arc-minutes.
- **region** (`Sequence[float] | str | None`, default: `None`) – The subregion of the grid to load, in the form of a sequence [*xmin*, *xmax*, *ymin*, *ymax*] or an ISO country code. Required for grids with resolutions higher than 5 arc-minutes (i.e., `"05m"`).
- **registration** (`Literal['gridline', 'pixel']`, default: `'gridline'`) – Grid registration type. Either `"pixel"` for pixel registration or `"gridline"` for gridline registration.

Return type

`DataArray`

Returns

grid – The Venus relief grid. Coordinates are latitude and longitude in degrees. Relief is in meters.

> **Note**
>
> The registration and coordinate system type of the returned `xarray.DataArray` grid can be accessed via the *gmt* accessor. Refer to *`pygmt.GMTDataArrayAccessor`* for detailed explanations and limitations.

Examples

```
>>> from pygmt.datasets import load_venus_relief
>>> # Load the default grid (gridline-registered 1 arc-degree grid)
>>> grid = load_venus_relief()
>>> # Load the 30 arc-minutes grid with "gridline" registration
>>> grid = load_venus_relief(resolution="30m", registration="gridline")
>>> # Load high-resolution (5 arc-minutes) grid for a specific region
>>> grid = load_venus_relief(
...     resolution="05m",
...     region=[120, 160, 30, 60],
...     registration="gridline",
... )
```

11.9.20 pygmt.datasets.load_sample_data

`pygmt.datasets.`**`load_sample_data`**(*name*)

Load an example dataset from the GMT server.

The data are downloaded to a cache directory (usually `~/.gmt/cache`) the first time you invoke this function. Afterwards, it will load the data from the cache. So you'll need an internet connection the first time around.

Parameters

`name` (`Literal['bathymetry', 'earth_relief_holes', 'fractures', 'hotspots', 'japan_quakes', 'mars_shape', 'maunaloa_co2', 'notre_dame_topography', 'ocean_ridge_points', 'rock_compositions', 'usgs_quakes']`) – Name of the dataset to load.

Return type

`DataFrame` | `DataArray`

Returns

data – Sample dataset loaded as a `pandas.DataFrame` for tabular data or `xarray.DataArray` for raster data.

See also

list_sample_data
Report datasets available for tests and documentation examples.

Examples

```
>>> from pprint import pprint
>>> from pygmt.datasets import list_sample_data, load_sample_data
>>> # use list_sample_data to see the available datasets
>>> pprint(list_sample_data(), width=120)
{'bathymetry': 'Table of ship bathymetric observations off Baja California',
 'earth_relief_holes': 'Regional 20 arc-minutes Earth relief grid with holes',
 'fractures': 'Table of hypothetical fracture lengths and azimuths',
 'hotspots': 'Table of locations, names, and symbol sizes of hotpots from Müller␣
↪et al. (1993)',
 'japan_quakes': 'Table of earthquakes around Japan from the NOAA NGDC database',
 'mars_shape': 'Table of topographic signature of the hemispheric dichotomy of␣
↪Mars from Smith and Zuber (1996)',
 'maunaloa_co2': 'Table of CO2 readings from Mauna Loa',
 'notre_dame_topography': 'Table 5.11 in Davis: Statistics and Data Analysis in␣
↪Geology',
 'ocean_ridge_points': 'Table of ocean ridge points for the entire world',
 'rock_compositions': 'Table of rock sample compositions',
 'usgs_quakes': 'Table of earthquakes from the USGS'}
>>> # Load the sample bathymetry dataset
>>> data = load_sample_data("bathymetry")
```

Examples using `pygmt.datasets.load_sample_data`

In addition, there is also a special function to load XYZ tile maps via contextily to be used as base maps.

datasets.load_tile_map(region[, zoom, ...])	Load a georeferenced raster tile map from XYZ tile providers.

11.9.21 pygmt.datasets.load_tile_map

pygmt.datasets.**load_tile_map**(*region*, *zoom='auto'*, *source=None*, *lonlat=True*, *crs='EPSG:3857'*, *wait=0*, *max_retries=2*, *zoom_adjust=None*, *headers=None*)

Load a georeferenced raster tile map from XYZ tile providers.

The tiles that compose the map are merged and georeferenced into an `xarray.DataArray` image with 3 bands (RGB). Note that the returned image is in a Spherical Mercator (EPSG:3857) coordinate reference system (CRS) by default, but can be customized using the `crs` parameter.

Parameters

- **region** (`Sequence[float]`) – The bounding box of the map in the form of a list [*xmin*, *xmax*, *ymin*, *ymax*]. These coordinates should be in longitude/latitude if `lonlat=True` or Spherical Mercator (EPSG:3857) if `lonlat=False`.
- **zoom** (`int | Literal['auto']`, default: `'auto'`) – Level of detail. Higher levels (e.g. `22`) mean a zoom level closer to the Earth's surface, with more tiles covering a smaller geographical

area and thus more detail. Lower levels (e.g. `0`) mean a zoom level further from the Earth's surface, with less tiles covering a larger geographical area and thus less detail. Default is `"auto"` to automatically determine the zoom level based on the bounding box region extent.

> **Note**
>
> The maximum possible zoom level may be smaller than `22`, and depends on what is supported by the chosen web tile provider source.

- **source** (`TileProvider` | `str` | `None`, default: `None`) – The tile source: web tile provider or path to a local file. Provide either:
  - A web tile provider in the form of a `xyzservices.TileProvider` object. See Contextily providers for a list of tile providers. Default is `xyzservices.providers.OpenStreetMap.HOT`, i.e. OpenStreetMap Humanitarian web tiles.
  - A web tile provider in the form of a URL. The placeholders for the XYZ in the URL need to be {x}, {y}, {z}, respectively. E.g. `https://tile.openstreetmap.org/{z}/{x}/{y}.png`.
  - A local file path. The file is read with rasterio and all bands are loaded into the basemap. See Working with local files.

> **Important**
>
> Tiles are assumed to be in the Spherical Mercator projection (EPSG:3857).

- **lonlat** (`bool`, default: `True`) – If `False`, coordinates in `region` are assumed to be Spherical Mercator as opposed to longitude/latitude.
- **crs** (`str` | `CRS`, default: `'EPSG:3857'`) – Coordinate reference system (CRS) of the returned `xarray.DataArray` image. Default is `"EPSG:3857"` (i.e., Spherical Mercator). The CRS can be in either string or `rasterio.crs.CRS` format.
- **wait** (`int`, default: `0`) – If the tile API is rate-limited, the number of seconds to wait between a failed request and the next try.
- **max_retries** (`int`, default: `2`) – Total number of rejected requests allowed before contextily will stop trying to fetch more tiles from a rate-limited API.
- **zoom_adjust** (`int` | `None`, default: `None`) – The amount to adjust a chosen zoom level if it is chosen automatically. Values outside of -1 to 1 are not recommended as they can lead to slow execution.
- **headers** (`dict[str, str]` | `None`, default: `None`) – HTTP headers to include with requests to the tile server. This can be useful for authentication or to set a custom User-Agent. When supported by `contextily` (>=1.7.0), PyGMT sets a default `User-Agent` header like `PyGMT/X.Y.Z (+https://www.pygmt.org)`.

> **Note**
>
> Requires `contextily>=1.7.0`.

Return type

`DataArray`

Returns

raster – Georeferenced 3-D data array of RGB values.

Raises

ImportError – If `contextily` is not installed or can't be imported. Follow the install instructions for contextily, (e.g. via `python -m pip install contextily`) before using this function.

Examples

```
>>> import contextily
>>> from pygmt.datasets import load_tile_map
>>> raster = load_tile_map(
...     region=[-180.0, 180.0, -90.0, 0.0],  # West, East, South, North
...     zoom=1,  # less detailed zoom level
...     source=contextily.providers.OpenTopoMap,
...     lonlat=True,  # bounding box coordinates are longitude/latitude
... )
>>> raster.sizes
Frozen({'band': 3, 'y': 256, 'x': 512})
>>> raster.coords
Coordinates:
  * band         (band) uint8 3B 1 2 3
  * y            (y) float64 2kB -7.081e-10 -7.858e+04 ... -1.996e+07 -2.004e+07
  * x            (x) float64 4kB -2.004e+07 -1.996e+07 ... 1.996e+07 2.004e+07
    spatial_ref  int64 8B 0
>>> # CRS is set only if rioxarray is available
>>> if hasattr(raster, "rio"):
...     raster.rio.crs.to_string()
'EPSG:3857'
```

11.10 Exceptions

All custom exceptions are derived from *pygmt.exceptions.GMTError*.

exceptions.GMTError	Base class for all GMT related errors.
exceptions.GMTCLibError	Error encountered when running a function from the GMT shared library.
exceptions.GMTCLibNoSessionError	Tried to access GMT API without a currently open GMT session.
exceptions.GMTCLibNotFoundError	Could not find the GMT shared library.
exceptions.GMTInvalidInput	Raised when the input of a function/method is invalid.
exceptions.GMTOSError	Unsupported operating system.
exceptions.GMTParameterError(*[, required, ...])	Raised when parameters are missing or invalid.
exceptions.GMTTypeError(dtype, /[, reason])	Raised when an invalid type is passed to a function/method.
exceptions.GMTValueError(value, /[, ...])	Raised when an invalid value is passed to a function/method.
exceptions.GMTVersionError	Raised when an incompatible version of GMT is being used.

11.10.1 pygmt.exceptions.GMTError

exception pygmt.exceptions.**GMTError**

Base class for all GMT related errors.

11.10.2 pygmt.exceptions.GMTCLibError

exception pygmt.exceptions.**GMTCLibError**

Error encountered when running a function from the GMT shared library.

11.10.3 pygmt.exceptions.GMTCLibNoSessionError

exception pygmt.exceptions.**GMTCLibNoSessionError**

Tried to access GMT API without a currently open GMT session.

11.10.4 pygmt.exceptions.GMTCLibNotFoundError

exception pygmt.exceptions.**GMTCLibNotFoundError**

Could not find the GMT shared library.

11.10.5 pygmt.exceptions.GMTInvalidInput

exception pygmt.exceptions.**GMTInvalidInput**

Raised when the input of a function/method is invalid.

11.10.6 pygmt.exceptions.GMTOSError

exception pygmt.exceptions.**GMTOSError**

Unsupported operating system.

11.10.7 pygmt.exceptions.GMTParameterError

exception pygmt.exceptions.**GMTParameterError**(*, *required=None*, *at_least_one=None*, *at_most_one=None*, *conflicts_with=None*, *reason=None*)

Raised when parameters are missing or invalid.

Parameters

- **required** (str | Iterable[str] | None, default: None) – Name or a collection of names of required parameters.
- **at_least_one** (Iterable[str] | None, default: None) – A collection of parameter names, of which at least one must be specified.
- **at_most_one** (Iterable[str] | None, default: None) – A collection of mutually exclusive parameter names, of which at most one can be specified.
- **conflicts_with** (tuple[str, Iterable[str]] | None, default: None) – A tuple with the parameter name and a collection of conflicting parameter names, indicating which parameters cannot be used together.
- **reason** (str | None, default: None) – Detailed reason why the parameters are invalid.

11.10.8 pygmt.exceptions.GMTTypeError

exception pygmt.exceptions.**GMTTypeError**(*dtype*, */ (Positional-only parameter separator (PEP 570))*, *reason=None*)

Raised when an invalid type is passed to a function/method.

This exception is used to indicate that the type of an argument does not match the expected type.

11.10.9 pygmt.exceptions.GMTValueError

exception pygmt.exceptions.**GMTValueError**(*value*, */*, *description='value'*, *choices=None*, *reason=None*)

Raised when an invalid value is passed to a function/method.

Parameters

- **value** (Any) – The invalid value.
- **description** (str, default: 'value') – The description of the value.
- **choices** (Iterable[Any] | None, default: None) – The valid choices for the value.
- **reason** (str | None, default: None) – The detailed reason why the value is invalid.

Examples

```
>>> raise GMTValueError("invalid")
Traceback (most recent call last):
    ...
pygmt.exceptions.GMTValueError: Invalid value: 'invalid'.
>>> raise GMTValueError("invalid", description="constant name")
Traceback (most recent call last):
...
pygmt.exceptions.GMTValueError: Invalid constant name: 'invalid'.
>>> raise GMTValueError("invalid", choices=["a", "b", 1, 2])
Traceback (most recent call last):
    ...
pygmt.exceptions.GMTValueError: Invalid value: 'invalid'. Expected one of: 'a', 'b
↪', 1, 2.
>>> raise GMTValueError("invalid", choices=["a", 0, True, False, None])
Traceback (most recent call last):
    ...
pygmt.exceptions.GMTValueError: Invalid value: 'invalid'. Expected one of: 'a',␣
↪0, True, False, None.
```

```
>>> from pygmt.enums import GridType
>>> raise GMTValueError("invalid", choices=GridType)
Traceback (most recent call last):
    ...
pygmt.exceptions.GMTValueError: Invalid value: 'invalid'. Expected one of:
↪<GridType.CARTESIAN: 0>, <GridType.GEOGRAPHIC: 1>.
>>> raise GMTValueError("invalid", reason="Explain why it's invalid.")
Traceback (most recent call last):
    ...
pygmt.exceptions.GMTValueError: Invalid value: 'invalid'. Explain why it's␣
↪invalid.
```

11.10.10 pygmt.exceptions.GMTVersionError

exception pygmt.exceptions.**GMTVersionError**

Raised when an incompatible version of GMT is being used.

11.11 GMT C API

The `pygmt.clib` package is a wrapper for the GMT C API built using `ctypes`. Most calls to the C API happen through the *pygmt.clib.Session* class.

clib.Session()	A GMT API session where most operations involving the C API happen.

11.11.1 pygmt.clib.Session

class pygmt.clib.**Session**

A GMT API session where most operations involving the C API happen.

Works as a context manager (for use in a `with` block) to create a GMT C API session and destroy it in the end to clean up memory.

Functions of the shared library are exposed as methods of this class. Most methods MUST be used with an open session (inside a `with` block). If creating GMT data structures to communicate data, put that code inside the same `with` block as the API calls that will use the data.

By default, will let `ctypes` try to find the GMT shared library (`libgmt`). If the environment variable *GMT_LIBRARY_PATH* is set, will look for the shared library in the directory specified by it.

The `session_pointer` attribute holds a ctypes pointer to the currently open session.

Raises

- ***GMTCLibNotFoundError*** – If there was any problem loading the library (couldn't find it or couldn't access the functions).
- ***GMTCLibNoSessionError*** – If you try to call a method outside of a `with` block.

Examples

```
>>> from pygmt.clib import Session
>>> from pygmt.helpers.testing import load_static_earth_relief
>>> from pygmt.helpers import GMTTempFile
>>> grid = load_static_earth_relief()
>>> type(grid)
<class 'xarray.core.dataarray.DataArray'>
>>> # Create a session and destroy it automatically when exiting the "with" block.
>>> with Session() as lib:
...     # Create a virtual file and link to the memory block of the grid.
...     with lib.virtualfile_from_grid(grid) as fin:
...         # Create a temp file to use as output.
...         with GMTTempFile() as fout:
...             # Call the grdinfo module with the virtual file as input and the
...             # temp file as output.
...             lib.call_module("grdinfo", [fin, "-C", f"->{fout.name}"])
...             # Read the contents of the temp file before it's deleted.
```

(continues on next page)

(continued from previous page)

```
...             print(fout.read().strip())
-55 -47 -24 -10 190 981 1 1 8 14 1 1
```

Attributes

`Session.`**`info`**

Dictionary with the GMT version and default paths and parameters.

`Session.`**`session_pointer`**

The `ctypes.c_void_p` pointer to the current open GMT session.

Raises

GMTCLibNoSessionError – If trying to access without a currently open GMT session (i.e., outside of the context manager).

Methods Summary

Session.call_module(module, args)	Call a GMT module with the given arguments.
Session.create(name)	Create a new GMT C API session.
Session.create_data(family, geometry, mode)	Create an empty GMT data container and allocate space to hold data.
Session.destroy()	Destroy the currently open GMT API session.
Session.extract_region()	Extract the region of the currently active figure.
Session.get_common(option)	Inquire if a GMT common option has been set and return its current value if possible.
Session.get_default(name)	Get the value of a GMT configuration parameter or a GMT API parameter.
Session.get_enum(name)	Get the value of a GMT constant (C enum) from `gmt_resources.h`.
Session.get_libgmt_func(name[, argtypes, ...])	Get a ctypes function from the libgmt shared library.
Session.inquire_virtualfile(vfname)	Get the family of a virtual file.
Session.open_virtualfile(family, geometry, ...)	Open a GMT virtual file associated with a data object for reading or writing.
Session.put_matrix(dataset, matrix[, pad])	Attach a 2-D numpy array to a GMT dataset.
Session.put_strings(dataset, family, strings)	Attach a 1-D numpy array of dtype str as a column on a GMT dataset.
Session.put_vector(dataset, column, vector)	Attach a 1-D numpy array as a column on a GMT dataset.
Session.read_data(infile, kind[, family, ...])	Read a data file into a GMT data container.
Session.read_virtualfile(vfname[, kind])	Read data from a virtual file and optionally cast into a GMT data container.
Session.virtualfile_from_grid(grid)	Store a grid in a virtual file.
Session.virtualfile_from_matrix(matrix)	Store a 2-D numpy array as a matrix inside a virtual file.
Session.virtualfile_from_stringio(stringio)	Store a `io.StringIO` object in a virtual file.
Session.virtualfile_from_vectors(vectors)	Store a sequence of 1-D vectors as columns of a dataset inside a virtual file.
Session.virtualfile_in([check_kind, data, ...])	Store any data inside a virtual file.
Session.virtualfile_out([kind, fname])	Create a virtual file or an actual file for storing output data.
Session.virtualfile_to_dataset(vfname[, ...])	Output a tabular dataset stored in a virtual file to a different format.
Session.virtualfile_to_raster(vfname[, ...])	Output raster data stored in a virtual file to an `xarray.DataArray` object.

continues on next page

Table 28 – continued from previous page

Session.write_data(family, geometry, mode, ...)	Write a GMT data container to a file.

pygmt.clib.Session.call_module

Session.**call_module**(*module*, *args*)

Call a GMT module with the given arguments.

Wraps GMT_Call_Module.

The GMT_Call_Module API function supports passing module arguments in three different ways:

1. Pass a single string that contains whitespace-separated module arguments.
2. Pass a list of strings and each string contains a module argument.
3. Pass a list of GMT_OPTION data structure.

Both options 1 and 2 are implemented in this function, but option 2 is preferred because it can correctly handle special characters like whitespaces and quotation marks in module arguments.

Parameters

- **module** (str) – The GMT module name to be called ("coast", "basemap", etc).
- **args** (str | list[str]) – Module arguments that will be passed to the GMT module. It can be either a single string (e.g., "-R0/5/0/10 -JX10c -BWSen+t'My Title'") or a list of strings (e.g., ["-R0/5/0/10", "-JX10c", "-BWSEN+tMy Title"]).

Raises

- ***GMTTypeError*** – If the args argument is not a string or a list of strings.
- ***GMTCLibError*** – If the returned status code of the function is non-zero.

Return type

None

pygmt.clib.Session.create

Session.**create**(*name*)

Create a new GMT C API session.

This is required before most other methods of *pygmt.clib.Session* can be called.

Warning

Usage of *pygmt.clib.Session* as a context manager in a with block is preferred over calling *pygmt.clib.Session.create* and *pygmt.clib.Session.destroy* manually.

Calls GMT_Create_Session and generates a new GMTAPI_CTRL struct, which is a ctypes.c_void_p pointer. Sets the session_pointer attribute to this pointer.

Remember to terminate the current session using *pygmt.clib.Session.destroy* before creating a new one.

Parameters

name (str) – A name for this session. Doesn't really affect the outcome.

Return type

None

pygmt.clib.Session.create_data

Session.**create_data**(*family*, *geometry*, *mode*, *dim=None*, *ranges=None*, *inc=None*, *registration='GMT_GRID_NODE_REG'*, *pad=None*)

Create an empty GMT data container and allocate space to hold data.

Valid data families and geometries are in `FAMILIES` and `GEOMETRIES`.

There are two ways to define the dimensions needed to actually allocate memory:

1. Via `ranges`, `inc` and `registration`.
2. Via `dim` and `registration`.

   `dim` contains up to 4 values and they have different meanings for different GMT data families:

   For `GMT_DATASET`:

   - 0: number of tables
   - 1: number of segments per table
   - 2: number of rows per segment
   - 3: number of columns per row

   For `GMT_VECTOR`:

   - 0: number of columns
   - 1: number of rows [optional, can be 0 if unknown]
   - 2: data type (e.g., `GMT_DOUBLE`) [Will be overwritten by `put_vector`]

   For `GMT_GRID`/`GMT_IMAGE`/`GMT_CUBE`/`GMT_MATRIX`:

   - 0: number of columns
   - 1: number of rows
   - 2: number of bands or layers [Ignored for `GMT_GRID`]
   - 3: data type (e.g., `GMT_DOUBLE`) [For `GMT_MATRIX` only, but will be overwritten by `put_matrix`]

   In other words, `inc` is assumed to be 1.0, and `ranges` is [0, dim[0], 0, dim[1]] for pixel registration or [0, dim[0]-1.0, 0, dim[1]-1.0] for grid registration.

When creating a grid/image/cube, you can do it in one or two steps:

1. Call this function with `mode="GMT_CONTAINER_AND_DATA"`. This creates a header and allocates a grid or an image
2. Call this function twice:
   1. First with `mode="GMT_CONTAINER_ONLY"`, to create a header only and compute the dimensions based on other parameters
   2. Second with `mode="GMT_DATA_ONLY"`, to allocate the grid/image/cube array based on the dimensions already set. This time, you pass NULL for `dim`/`ranges`/`inc`/`registration`/`pad` and let `data` be the void pointer returned in the first step.

   Note: This is not implemented yet, since this function doesn't have the `data` parameter.

   Parameters

   - **family** (`str`) – A valid GMT data family name (e.g., `"GMT_IS_DATASET"`). See `FAMILIES` for valid names.

- **geometry** (str) – A valid GMT data geometry name (e.g., "GMT_IS_POINT"). See GEOMETRIES for valid names.
- **mode** (str) – A valid GMT data mode. See MODES for valid names. For GMT_IS_DATASET/GMT_IS_MATRIX/GMT_IS_VECTOR, adding GMT_WITH_STRINGS to the mode will allocate the corresponding arrays of string pointers.
- **dim** (Sequence[int] | None, default: None) – The dimensions of the dataset, as explained above. If None, will pass in the NULL pointer.
- **ranges** (Sequence[float] | None, default: None) – The data extent.
- **inc** (Sequence[float] | None, default: None) – The increments between points of the dataset.
- **registration** (Literal['GMT_GRID_NODE_REG', 'GMT_GRID_PIXEL_REG'], default: 'GMT_GRID_NODE_REG') – The node registration. Can be "GMT_GRID_PIXEL_REG" or "GMT_GRID_NODE_REG".
- **pad** (int | None, default: None) – The padding for GMT_IS_GRID/GMT_IS_IMAGE/GMT_IS_CUBE. If None, defaults to "GMT_PAD_DEFAULT".

  For GMT_IS_MATRIX, it can be:

  – 0: default row/col orientation [Default]

  – 1: row-major format (C)

  – 2: column-major format (FORTRAN)

Return type
c_void_p

Returns
data_ptr – A ctypes pointer (an integer) to the allocated GMT data container.

pygmt.clib.Session.destroy

Session.**destroy**()

Destroy the currently open GMT API session.

> **Warning**
>
> Usage of *pygmt.clib.Session* as a context manager in a with block is preferred over calling *pygmt.clib.Session.create* and *pygmt.clib.Session.destroy* manually.

Calls GMT_Destroy_Session to terminate and free the memory of a registered GMTAPI_CTRL session (the pointer for this struct is stored in the session_pointer attribute).

Always use this method after you are done using a C API session. The session needs to be destroyed before creating a new one. Otherwise, some of the configuration files might be left behind and can influence subsequent API calls.

Sets the session_pointer attribute to None.

pygmt.clib.Session.extract_region

Session.**extract_region**()

Extract the region of the currently active figure.

Retrieves the information from the PostScript file, so it works for country codes as well.

Return type
ndarray

Returns
region – A numpy 1-D array with the west, east, south, and north dimensions of the current figure.

Examples

```
>>> import pygmt
>>> fig = pygmt.Figure()
>>> fig.coast(
...     region=[0, 10, -20, -10], projection="M12c", frame=True, land="black"
... )
>>> with Session() as lib:
...     region = lib.extract_region()
>>> print(", ".join([f"{x:.2f}" for x in region]))
0.00, 10.00, -20.00, -10.00
```

Using ISO country codes for the regions (for example `"US.HI"` for Hawai'i):

```
>>> fig = pygmt.Figure()
>>> fig.coast(region="US.HI", projection="M12c", frame=True, land="black")
>>> with Session() as lib:
...     region = lib.extract_region()
>>> print(", ".join([f"{x:.2f}" for x in region]))
-164.71, -154.81, 18.91, 23.58
```

The country codes can have an extra argument that rounds the region to multiples of the argument (for example, `"US.HI+r5"` will round the region to multiples of 5):

```
>>> fig = pygmt.Figure()
>>> fig.coast(region="US.HI+r5", projection="M12c", frame=True, land="black")
>>> with Session() as lib:
...     region = lib.extract_region()
>>> print(", ".join([f"{x:.2f}" for x in region]))
-165.00, -150.00, 15.00, 25.00
```

pygmt.clib.Session.get_common

`Session.`**`get_common`**(*option*)

Inquire if a GMT common option has been set and return its current value if possible.

Parameters
option (`str`) – The GMT common option to check. Valid options are `"B"`, `"I"`, `"J"`, `"R"`, `"U"`, `"V"`, `"X"`, `"Y"`, `"a"`, `"b"`, `"f"`, `"g"`, `"h"`, `"i"`, `"n"`, `"o"`, `"p"`, `"r"`, `"s"`, `"t"`, and `":"`.

Return type
bool | int | float | ndarray

Returns
value – Whether the option was set or its value. If the option was not set, return `False`. Otherwise, the return value depends on the choice of the option.

- options `"B"`, `"J"`, `"U"`, `"g"`, `"n"`, `"p"`, and `"s"`: return `True` if set, else `False` (bool)
- `"I"`: 2-element array for the increments (float)

- `"R"`: 4-element array for the region (float)
- `"V"`: the verbose level (int)
- `"X"`: the xshift (float)
- `"Y"`: the yshift (float)
- `"a"`: geometry of the dataset (int)
- `"b"`: return 0 if `-bi` was set and 1 if `-bo` was set (int)
- `"f"`: return 0 if `-fi` was set and 1 if `-fo` was set (int)
- `"h"`: whether to delete existing header records (int)
- `"i"`: number of input columns (int)
- `"o"`: number of output columns (int)
- `"r"`: registration type (int)
- `"t"`: 2-element array for the transparency (float)
- `":"`: return 0 if `-:i` was set and 1 if `-:o` was set (int)

Examples

```
>>> with Session() as lib:
...     lib.call_module(
...         "basemap", ["-R0/10/10/15", "-JX5i/2.5i", "-Baf", "-Ve", "-fg"]
...     )
...     region = lib.get_common("R")
...     projection = lib.get_common("J")
...     timestamp = lib.get_common("U")
...     verbose = lib.get_common("V")
...     coltypes = lib.get_common("f")
...     lib.call_module("plot", ["-T", "-Xw+1i", "-Yh-1i"])
...     xshift = lib.get_common("X")  # xshift/yshift are in inches
...     yshift = lib.get_common("Y")
>>> print(region, projection, timestamp, verbose, coltypes, xshift, yshift)
[ 0. 10. 10. 15.] True False 3 0 6.0 1.5
>>> with Session() as lib:
...     lib.call_module("basemap", ["-R0/10/10/15", "-JX5i/2.5i", "-Baf"])
...     lib.get_common("A")
Traceback (most recent call last):
...
pygmt.exceptions.GMTValueError: Invalid GMT common option: 'A'. Expected ...
>>> with Session() as lib:
...     lib.get_common("BI")
Traceback (most recent call last):
...
pygmt.exceptions.GMTValueError: Invalid GMT common option: 'BI'. Expected ...
>>> with Session() as lib:
...     lib.get_common("")
Traceback (most recent call last):
...
pygmt.exceptions.GMTValueError: Invalid GMT common option: ''. Expected ...
```

pygmt.clib.Session.get_default

Session.**get_default**(*name*)

Get the value of a GMT configuration parameter or a GMT API parameter.

In addition to the long list of GMT configuration parameters, the following API parameter names are also supported:

- "API_VERSION": The GMT API version
- "API_PAD": The grid padding setting
- "API_BINDIR": The binary file directory
- "API_SHAREDIR": The share directory
- "API_DATADIR": The data directory
- "API_PLUGINDIR": The plugin directory
- "API_LIBRARY": The core library path
- "API_CORES": The number of cores
- "API_IMAGE_LAYOUT": The image/band layout
- "API_GRID_LAYOUT": The grid layout
- "API_BIN_VERSION": The GMT binary version (with git information)

Parameters
name (str) – The name of the GMT configuration parameter (e.g., "PROJ_LENGTH_UNIT") or a GMT API parameter (e.g., "API_VERSION").

Return type
str

Returns
value – The current value for the parameter.

Raises
GMTCLibError – If the parameter doesn't exist.

pygmt.clib.Session.get_enum

Session.**get_enum**(*name*)

Get the value of a GMT constant (C enum) from gmt_resources.h.

Used to set configuration values for other API calls. Wraps GMT_Get_Enum.

Parameters
name (str) – The name of the constant (e.g., "GMT_SESSION_EXTERNAL").

Return type
int

Returns
value – Integer value of the constant. Do not rely on this value because it might change.

Raises
GMTCLibError – If the constant doesn't exist.

pygmt.clib.Session.get_libgmt_func

`Session.`**`get_libgmt_func`**(*name*, *argtypes=None*, *restype=<class 'ctypes.c_int'>*)

Get a ctypes function from the libgmt shared library.

Assigns the argument and return type conversions for the function.

Use this method to access a C function from libgmt.

Parameters

- **name** (`str`) – The name of the GMT API function.
- **argtypes** (`list` | `None`, default: `None`) – List of ctypes types used to convert the Python input arguments for the API function.
- **restype** (`Any`, default: `<class 'ctypes.c_int'>`) – The ctypes type used to convert the value returned by the function into a Python type [Default is `ctypes.c_int`]. Use `None` for functions that return void.

Return type

`Callable`

Returns

function – The GMT API function.

Examples

```
>>> from ctypes import c_void_p, c_int
>>> with Session() as lib:
...     func = lib.get_libgmt_func(
...         "GMT_Destroy_Session", argtypes=[c_void_p], restype=c_int
...     )
>>> type(func)
<class 'ctypes.CDLL.__init__.<locals>._FuncPtr'>
```

pygmt.clib.Session.inquire_virtualfile

`Session.`**`inquire_virtualfile`**(*vfname*)

Get the family of a virtual file.

Parameters

vfname (`str`) – Name of the virtual file to inquire.

Return type

`int`

Returns

family – The integer value for the family of the virtual file.

Raises

GMTCLibError – If the virtual file is not found, i.e., `vfname` is not a valid virtual file name.

Examples

```
>>> from pygmt.clib import Session
>>> with Session() as lib:
...     with lib.virtualfile_out(kind="dataset") as vfile:
...         family = lib.inquire_virtualfile(vfile)
```

(continues on next page)

(continued from previous page)

```
...         assert family == lib["GMT_IS_DATASET"]
>>> with Session() as lib:
...     lib.inquire_virtualfile("not-a-virtual-file")
Traceback (most recent call last):
...
pygmt.exceptions.GMTCLibError: Failed to inquire the family of virtual file ...
```

pygmt.clib.Session.open_virtualfile

`Session.`**`open_virtualfile`**(*family*, *geometry*, *direction*, *data*)

Open a GMT virtual file associated with a data object for reading or writing.

GMT uses a virtual file scheme to pass in data or get data from API modules. Use it to pass in your GMT data structure (created using *pygmt.clib.Session.create_data*) to a module that expects an input file, or get the output from a module that writes to a file.

Use in a `with` block. Will automatically close the virtual file when leaving the `with` block. Because of this, no wrapper for `GMT_Close_VirtualFile` is provided.

Parameters

- **family** (`str`) – A valid GMT data family name (e.g., `"GMT_IS_DATASET"`). Should be the same as the one you used to create your data structure.
- **geometry** (`str`) – A valid GMT data geometry name (e.g., `"GMT_IS_POINT"`). Should be the same as the one you used to create your data structure.
- **direction** (`str`) – Either `"GMT_IN"` or `"GMT_OUT"` to indicate if passing data to GMT or getting it out of GMT, respectively. By default, GMT can modify the data you pass in. Add modifier `"GMT_IS_REFERENCE"` to tell GMT the data are read-only, or `"GMT_IS_DUPLICATE"` to tell GMT to duplicate the data.
- **data** (`c_void_p` | `None`) – The ctypes void pointer to the GMT data structure. For output (i.e., `direction="GMT_OUT"`), it can be `None` to have GMT automatically allocate the output GMT data structure.

Yields

vfname – The name of the virtual file that you can pass to a GMT module.

Examples

```
>>> from pygmt.helpers import GMTTempFile
>>> import numpy as np
>>> x = np.array([0, 1, 2, 3, 4])
>>> y = np.array([5, 6, 7, 8, 9])
>>> with Session() as lib:
...     family = "GMT_IS_DATASET|GMT_VIA_VECTOR"
...     geometry = "GMT_IS_POINT"
...     dataset = lib.create_data(
...         family=family,
...         geometry=geometry,
...         mode="GMT_CONTAINER_ONLY",
...         dim=[2, 5, lib["GMT_INT"], 0],  # ncolumns, nrows, dtype, unused
...     )
...     lib.put_vector(dataset, column=0, vector=x)
```

(continues on next page)

(continued from previous page)

```
...     lib.put_vector(dataset, column=1, vector=y)
...     # Add the dataset to a virtual file
...     vfargs = (family, geometry, "GMT_IN|GMT_IS_REFERENCE", dataset)
...     with lib.open_virtualfile(*vfargs) as vfile:
...         # Send the output to a temp file so that we can read it
...         with GMTTempFile() as ofile:
...             lib.call_module("info", [vfile, f"->{ofile.name}"])
...             print(ofile.read().strip())
<vector memory>: N = 5 <0/4> <5/9>
```

pygmt.clib.Session.put_matrix

`Session.`**`put_matrix`**(*dataset*, *matrix*, *pad=0*)

Attach a 2-D numpy array to a GMT dataset.

Use this function to attach numpy array data to a GMT dataset and pass it to GMT modules. Wraps `GMT_Put_Matrix`.

The dataset must be created by *pygmt.clib.Session.create_data* first with `family="GMT_IS_DATASET|GMT_VIA_MATRIX"`.

Not all numpy dtypes are supported, only: int8, int16, int32, int64, longlong, uint8, uint16, uint32, uint64, ulonglong, float32, and float64.

Warning

The numpy array must be C contiguous in memory. Use `numpy.ascontiguousarray` to make sure your matrix is contiguous (it won't copy if it already is).

Parameters

- **`dataset`** (`c_void_p`) – The ctypes void pointer to a `GMT_MATRIX` data container. Create it with *pygmt.clib.Session.create_data*.
- **`matrix`** (`ndarray`) – The array that will be attached to the dataset. Must be a 2-D C contiguous array.
- **`pad`** (`int`, default: `0`) – The amount of padding that should be added to the matrix. Use when creating grids for modules that require padding.

Raises

GMTCLibError – If given invalid input or `GMT_Put_Matrix` exits with a non-zero status.

Return type

`None`

pygmt.clib.Session.put_strings

`Session.`**`put_strings`**(*dataset*, *family*, *strings*)

Attach a 1-D numpy array of dtype str as a column on a GMT dataset.

Use this function to attach string type numpy array data to a GMT dataset and pass it to GMT modules. Wraps `GMT_Put_Strings`.

The dataset must be created by *pygmt.clib.Session.create_data* first.

> **Warning**
>
> The numpy array must be C contiguous in memory. If it comes from a column slice of a 2-D array, for example, you will have to make a copy. Use `numpy.ascontiguousarray` to make sure your vector is contiguous (it won't copy if it already is).

Parameters

- **dataset** (`c_void_p`) – The ctypes void pointer to a `GMT_VECTOR`/`GMT_MATRIX` data container. Create it with *pygmt.clib.Session.create_data*.
- **family** (`str`) – The family type of the dataset. Can be either `GMT_IS_VECTOR` or `GMT_IS_MATRIX`.
- **strings** (`ndarray`) – The array that will be attached to the dataset. Must be a 1-D C contiguous array.

Raises

GMTCLibError – If given invalid input or `GMT_Put_Strings` exits with a non-zero status.

Return type

`None`

pygmt.clib.Session.put_vector

`Session.`**put_vector**(*dataset*, *column*, *vector*)

Attach a 1-D numpy array as a column on a GMT dataset.

Use this function to attach numpy array data to a GMT dataset and pass it to GMT modules. Wraps `GMT_Put_Vector`.

The dataset must be created by *pygmt.clib.Session.create_data* first with `family="GMT_IS_DATASET|GMT_VIA_VECTOR"`.

Not all numpy dtypes are supported, only: int8, int16, int32, int64, longlong, uint8, uint16, uint32, uint64, ulonglong, float32, float64, str_, datetime64, and timedelta64.

> **Warning**
>
> The numpy array must be C contiguous in memory. Use `numpy.ascontiguousarray` to make sure your vector is contiguous (it won't copy if it already is).

Parameters

- **dataset** (`c_void_p`) – The ctypes void pointer to a `GMT_VECTOR` data container. Create it with *pygmt.clib.Session.create_data*.
- **column** (`int`) – The column number of this vector in the dataset (starting from 0).
- **vector** (`ndarray`) – The array that will be attached to the dataset. Must be a 1-D C contiguous array.

Raises

GMTCLibError – If given invalid input or `GMT_Put_Vector` exits with a non-zero status.

Return type

`None`

pygmt.clib.Session.read_data

Session.**read_data**(*infile*, *kind*, *family=None*, *geometry=None*, *mode='GMT_READ_NORMAL'*, *region=None*, *data=None*)

Read a data file into a GMT data container.

Wraps GMT_Read_Data but only allows reading from a file. The function definition is different from the original C API function.

Parameters

- **infile** (str) – The input file name.
- **kind** (Literal['dataset', 'grid', 'image']) – The data kind of the input file. Valid values are "dataset", "grid" and "image".
- **family** (str | None, default: None) – A valid GMT data family name (e.g., "GMT_IS_DATASET"). See the FAMILIES attribute for valid names. If None, will determine the data family from the kind parameter.
- **geometry** (str | None, default: None) – A valid GMT data geometry name (e.g., "GMT_IS_POINT"). See the GEOMETRIES attribute for valid names. If None, will determine the data geometry from the kind parameter.
- **mode** (str, default: 'GMT_READ_NORMAL') – How the data is to be read from the file. This option varies depending on the given family. See the GMT API documentation for details. Default is GMT_READ_NORMAL which corresponds to the default read mode value of 0 in the GMT_enum_read enum.
- **region** (Sequence[float] | None, default: None) – Subregion of the data, in the form of [xmin, xmax, ymin, ymax, zmin, zmax]. If None, the whole data is read.
- **data** (default: None) – None or the pointer returned by this function after a first call. It's useful when reading grids/images/cubes in two steps (get a grid/image/cube structure with a header, then read the data).

Returns

Pointer to the data container, or None if there were errors.

Raises

GMTCLibError – If the GMT API function fails to read the data.

pygmt.clib.Session.read_virtualfile

Session.**read_virtualfile**(*vfname*, *kind=None*)

Read data from a virtual file and optionally cast into a GMT data container.

Parameters

- **vfname** (str) – Name of the virtual file to read.
- **kind** (Literal['dataset', 'grid', 'image', 'cube', None], default: None) – Cast the data into a GMT data container. Valid values are "dataset", "grid", "image" and None. If None, will return a ctypes void pointer.

Returns

pointer – Pointer to the GMT data container. If kind is None, returns a ctypes void pointer instead.

Examples

```
>>> from pathlib import Path
>>> from pygmt.clib import Session
>>> from pygmt.helpers import GMTTempFile
>>>
>>> # Read dataset from a virtual file
>>> with Session() as lib:
...     with GMTTempFile(suffix=".txt") as tmpfile:
...         with Path(tmpfile.name).open(mode="w") as fp:
...             print("1.0 2.0 3.0 TEXT", file=fp)
...         with lib.virtualfile_out(kind="dataset") as vouttbl:
...             lib.call_module("read", [tmpfile.name, vouttbl, "-Td"])
...             # Read the virtual file as a void pointer
...             void_pointer = lib.read_virtualfile(vouttbl)
...             assert isinstance(void_pointer, int)  # void pointer is an int
...             # Read the virtual file as a dataset
...             data_pointer = lib.read_virtualfile(vouttbl, kind="dataset")
...             assert isinstance(data_pointer, ctp.POINTER(_GMT_DATASET))
>>>
>>> # Read grid from a virtual file
>>> with Session() as lib:
...     with lib.virtualfile_out(kind="grid") as voutgrd:
...         lib.call_module("read", ["@earth_relief_01d_g", voutgrd, "-Tg"])
...         # Read the virtual file as a void pointer
...         void_pointer = lib.read_virtualfile(voutgrd)
...         assert isinstance(void_pointer, int)  # void pointer is an int
...         data_pointer = lib.read_virtualfile(voutgrd, kind="grid")
...         assert isinstance(data_pointer, ctp.POINTER(_GMT_GRID))
```

pygmt.clib.Session.virtualfile_from_grid

Session.**virtualfile_from_grid**(*grid*)

Store a grid in a virtual file.

Use the virtual file name to pass in the data in your grid to a GMT module. Grids must be `xarray.DataArray` instances.

Context manager (use in a `with` block). Yields the virtual file name that you can pass as an argument to a GMT module call. Closes the virtual file upon exit of the `with` block.

The virtual file will contain the grid as a `GMT_MATRIX` data container with extra metadata.

Use this instead of creating a data container and virtual file by hand with *pygmt.clib.Session.create_data*, *pygmt.clib.Session.put_matrix*, and *pygmt.clib.Session.open_virtualfile*.

The grid data matrix must be C contiguous in memory. If it is not (e.g., it is a slice of a larger array), the array will be copied to make sure it is.

Parameters

grid (`DataArray`) – The grid that will be included in the virtual file.

Yields

fname – The name of virtual file. Pass this as a file name argument to a GMT module.

Examples

```
>>> from pygmt.helpers.testing import load_static_earth_relief
>>> from pygmt.helpers import GMTTempFile
>>> data = load_static_earth_relief()
>>> print(data.shape)
(14, 8)
>>> print(data.lon.values.min(), data.lon.values.max())
-54.5 -47.5
>>> print(data.lat.values.min(), data.lat.values.max())
-23.5 -10.5
>>> print(data.values.min(), data.values.max())
190.0 981.0
>>> with Session() as ses:
...     with ses.virtualfile_from_grid(data) as fin:
...         # Send the output to a file so that we can read it
...         with GMTTempFile() as fout:
...             ses.call_module(
...                 "grdinfo", [fin, "-L0", "-Cn", f"->{fout.name}"]
...             )
...             print(fout.read().strip())
-55 -47 -24 -10 190 981 1 1 8 14 1 1
>>> # The output is: w e s n z0 z1 dx dy n_columns n_rows reg gtype
```

pygmt.clib.Session.virtualfile_from_matrix

`Session.`**`virtualfile_from_matrix`**(*matrix*)

Store a 2-D numpy array as a matrix inside a virtual file.

Use the virtual file name to pass in the data in your matrix to a GMT module.

Context manager (use in a `with` block). Yields the virtual file name that you can pass as an argument to a GMT module call. Closes the virtual file upon exit of the `with` block.

The virtual file will contain the array as a `GMT_MATRIX` data container pretending to be a `GMT_DATASET` data container.

Not meant for creating ``GMT_GRID``. The grid requires more metadata than just the data matrix. Use *pygmt.clib.Session.virtualfile_from_grid* instead.

Use this instead of creating the data container and virtual file by hand with *pygmt.clib.Session.create_data*, *pygmt.clib.Session.put_matrix*, and *pygmt.clib.Session.open_virtualfile*.

The matrix must be C contiguous in memory. If it is not (e.g., it is a slice of a larger array), the array will be copied to make sure it is.

Parameters

matrix (ndarray) – The matrix that will be included in the GMT data container.

Yields

fname – The name of virtual file. Pass this as a file name argument to a GMT module.

Examples

```
>>> from pygmt.helpers import GMTTempFile
>>> import numpy as np
>>> data = np.arange(12).reshape((4, 3))
```

(continues on next page)

(continued from previous page)

```
>>> print(data)
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
>>> with Session() as ses:
...     with ses.virtualfile_from_matrix(data) as fin:
...         # Send the output to a file so that we can read it
...         with GMTTempFile() as fout:
...             ses.call_module("info", [fin, f"->{fout.name}"])
...             print(fout.read().strip())
<matrix memory>: N = 4 <0/9> <1/10> <2/11>
```

pygmt.clib.Session.virtualfile_from_stringio

Session.**virtualfile_from_stringio**(*stringio*)

Store a io.StringIO object in a virtual file.

Store the contents of a io.StringIO object in a GMT_DATASET container and create a virtual file to pass to a GMT module.

For simplicity, currently we make following assumptions in the StringIO object

- "#" indicates a comment line.
- ">" indicates a segment header.

Parameters

stringio (StringIO) – The io.StringIO object containing the data to be stored in the virtual file.

Yields

fname – The name of the virtual file.

Examples

```
>>> import io
>>> from pygmt.clib import Session
>>> # A StringIO object containing legend specifications
>>> stringio = io.StringIO(
...     "# Comment\n"
...     "H 24p Legend\n"
...     "N 2\n"
...     "S 0.1i c 0.15i p300/12 0.25p 0.3i My circle\n"
... )
>>> with Session() as lib:
...     with lib.virtualfile_from_stringio(stringio) as fin:
...         lib.virtualfile_to_dataset(vfname=fin, output_type="pandas")
                                          0
0                               H 24p Legend
1                                        N 2
2  S 0.1i c 0.15i p300/12 0.25p 0.3i My circle
```

pygmt.clib.Session.virtualfile_from_vectors

`Session.`**`virtualfile_from_vectors`**(*vectors*)

Store a sequence of 1-D vectors as columns of a dataset inside a virtual file.

Use the virtual file name to pass the dataset with your vectors to a GMT module.

Context manager (use in a `with` block). Yields the virtual file name that you can pass as an argument to a GMT module call. Closes the virtual file upon exit of the `with` block.

Use this instead of creating the data container and virtual file by hand with *pygmt.clib.Session.create_data*, *pygmt.clib.Session.put_vector*, and *pygmt.clib.Session.open_virtualfile*.

If the arrays are C contiguous blocks of memory, they will be passed without copying to GMT. If they are not (e.g., they are columns of a 2-D array), they will need to be copied to a contiguous block.

Parameters

vectors (*Sequence*) – A sequence of vectors that will be stored in the dataset. All must be of the same size.

Yields

fname – The name of virtual file. Pass this as a file name argument to a GMT module.

Examples

```
>>> from pygmt.helpers import GMTTempFile
>>> import numpy as np
>>> import pandas as pd
>>> x = [1, 2, 3]
>>> y = np.array([4, 5, 6])
>>> z = pd.Series([7, 8, 9])
>>> with Session() as ses:
...     with ses.virtualfile_from_vectors((x, y, z)) as fin:
...         # Send the output to a file so that we can read it
...         with GMTTempFile() as fout:
...             ses.call_module("info", [fin, f"->{fout.name}"])
...             print(fout.read().strip())
<vector memory>: N = 3 <1/3> <4/6> <7/9>
```

pygmt.clib.Session.virtualfile_in

`Session.`**`virtualfile_in`**(*check_kind=None*, *data=None*, *x=None*, *y=None*, *z=None*, *required=True*, *mincols=2*)

Store any data inside a virtual file.

This convenience function automatically detects the kind of data passed into it, and produces a virtualfile that can be passed into GMT later on.

Parameters

- **check_kind** (*str or None*) – Used to validate the type of data that can be passed in. Choose from 'raster', 'vector', or None. Default is None (no validation).
- **data** (default: `None`) – Any raster or vector data format. This could be a file name or path, a raster grid, a vector matrix/arrays, or other supported data input.
- **x/y/z** (*1-D arrays or None*) – x, y, and z columns as numpy arrays.
- **required** (*bool*) – Set to True when 'data' or ('x' and 'y') is required. Set to False when dealing with optional virtual files. Default is True.

- **mincols** (default: 2) – Number of minimum required columns. Default is 2 (i.e. require x and y columns).

Returns

file_context (*contextlib._GeneratorContextManager*) – The virtual file stored inside a context manager. Access the file name of this virtualfile using `with file_context as fname: ...`.

Examples

```
>>> from pygmt.helpers import GMTTempFile
>>> import xarray as xr
>>> data = xr.Dataset(
...     coords=dict(index=[0, 1, 2]),
...     data_vars=dict(
...         x=("index", [9, 8, 7]),
...         y=("index", [6, 5, 4]),
...         z=("index", [3, 2, 1]),
...     ),
... )
>>> with Session() as ses:
...     with ses.virtualfile_in(check_kind="vector", data=data) as fin:
...         # Send the output to a file so that we can read it
...         with GMTTempFile() as fout:
...             ses.call_module("info", [fin, f"->{fout.name}"])
...             print(fout.read().strip())
<vector memory>: N = 3 <7/9> <4/6> <1/3>
```

pygmt.clib.Session.virtualfile_out

Session.**virtualfile_out**(*kind='dataset'*, *fname=None*)

Create a virtual file or an actual file for storing output data.

If `fname` is not given, a virtual file will be created to store the output data into a GMT data container and the function yields the name of the virtual file. Otherwise, the output data will be written into the specified file and the function simply yields the actual file name.

Parameters

- **kind** (`Literal['dataset', 'grid', 'image']`, default: `'dataset'`) – The data kind of the virtual file to create. Valid values are `"dataset"`, `"grid"`, and `"image"`. Ignored if `fname` is specified.
- **fname** (`str | None`, default: `None`) – The name of the actual file to write the output data. No virtual file will be created.

Yields

vfile – Name of the virtual file or the actual file.

Examples

```
>>> from pathlib import Path
>>> from pygmt.clib import Session
>>> from pygmt.datatypes import _GMT_DATASET
>>> from pygmt.helpers import GMTTempFile
>>>
>>> with GMTTempFile(suffix=".txt") as tmpfile:
```

(continues on next page)

(continued from previous page)

```
...     with Path(tmpfile.name).open(mode="w") as fp:
...         print("1.0 2.0 3.0 TEXT", file=fp)
...
...     # Create a virtual file for storing the output table.
...     with Session() as lib:
...         with lib.virtualfile_out(kind="dataset") as vouttbl:
...             lib.call_module("read", [tmpfile.name, vouttbl, "-Td"])
...             ds = lib.read_virtualfile(vouttbl, kind="dataset")
...             assert isinstance(ds.contents, _GMT_DATASET)
...
...     # Write data to an actual file without creating a virtual file.
...     with Session() as lib:
...         with lib.virtualfile_out(fname=tmpfile.name) as vouttbl:
...             assert vouttbl == tmpfile.name
...             lib.call_module("read", [tmpfile.name, vouttbl, "-Td"])
...         line = Path(vouttbl).read_text()
...         assert line == "1\t2\t3\tTEXT\n"
```

pygmt.clib.Session.virtualfile_to_dataset

Session.**virtualfile_to_dataset**(*vfname*, *output_type='pandas'*, *header=None*, *column_names=None*, *dtype=None*, *index_col=None*)

Output a tabular dataset stored in a virtual file to a different format.

The format of the dataset is determined by the `output_type` parameter.

Parameters

- **vfname** (`str`) – The virtual file name that stores the result data.
- **output_type** (`Literal['pandas', 'numpy', 'file', 'strings']`, default: `'pandas'`) – Desired output type of the result data.
  - `"pandas"` will return a `pandas.DataFrame` object.
  - `"numpy"` will return a `numpy.ndarray` object.
  - `"file"` means the result was saved to a file and will return `None`.
  - `"strings"` will return the trailing text only as an array of strings.
- **header** (`int | None`, default: `None`) – Row number containing column names for the `pandas.DataFrame` output. `header=None` means not to parse the column names from table header. Ignored if the row number is larger than the number of headers in the table.
- **column_names** (`list[str] | None`, default: `None`) – The column names for the `pandas.DataFrame` output.
- **dtype** (`type | dict[str, type] | None`, default: `None`) – Data type for the columns of the `pandas.DataFrame` output. Can be a single type for all columns or a dictionary mapping column names to types.
- **index_col** (`str | int | None`, default: `None`) – Column to set as the index of the `pandas.DataFrame` output.

Return type

`DataFrame | ndarray | None`

Returns

result – The result dataset. If `output_type="file"` returns `None`.

Examples

```
>>> from pathlib import Path
>>> import numpy as np
>>> import pandas as pd
>>>
>>> from pygmt.helpers import GMTTempFile
>>> from pygmt.clib import Session
>>>
>>> with GMTTempFile(suffix=".txt") as tmpfile:
...     # prepare the sample data file
...     with Path(tmpfile.name).open(mode="w") as fp:
...         print(">", file=fp)
...         print("1.0 2.0 3.0 TEXT1 TEXT23", file=fp)
...         print("4.0 5.0 6.0 TEXT4 TEXT567", file=fp)
...         print(">", file=fp)
...         print("7.0 8.0 9.0 TEXT8 TEXT90", file=fp)
...         print("10.0 11.0 12.0 TEXT123 TEXT456789", file=fp)
...
...     # file output
...     with Session() as lib:
...         with GMTTempFile(suffix=".txt") as outtmp:
...             with lib.virtualfile_out(
...                 kind="dataset", fname=outtmp.name
...             ) as vouttbl:
...                 lib.call_module("read", [tmpfile.name, vouttbl, "-Td"])
...                 result = lib.virtualfile_to_dataset(
...                     vfname=vouttbl, output_type="file"
...                 )
...                 assert result is None
...                 assert Path(outtmp.name).stat().st_size > 0
...
...     # strings, numpy and pandas outputs
...     with Session() as lib:
...         with lib.virtualfile_out(kind="dataset") as vouttbl:
...             lib.call_module("read", [tmpfile.name, vouttbl, "-Td"])
...
...             # strings output
...             outstr = lib.virtualfile_to_dataset(
...                 vfname=vouttbl, output_type="strings"
...             )
...             assert isinstance(outstr, np.ndarray)
...             assert outstr.dtype.kind in ("S", "U")
...
...             # numpy output
...             outnp = lib.virtualfile_to_dataset(
...                 vfname=vouttbl, output_type="numpy"
...             )
...             assert isinstance(outnp, np.ndarray)
...
```

(continues on next page)

(continued from previous page)

```
...             # pandas output
...             outpd = lib.virtualfile_to_dataset(
...                 vfname=vouttbl, output_type="pandas"
...             )
...             assert isinstance(outpd, pd.DataFrame)
...
...             # pandas output with specified column names
...             outpd2 = lib.virtualfile_to_dataset(
...                 vfname=vouttbl,
...                 output_type="pandas",
...                 column_names=["col1", "col2", "col3", "coltext"],
...             )
...             assert isinstance(outpd2, pd.DataFrame)
>>> outstr
array(['TEXT1 TEXT23', 'TEXT4 TEXT567', 'TEXT8 TEXT90',
   'TEXT123 TEXT456789'], dtype='<U18')
>>> outnp
array([[1.0, 2.0, 3.0, 'TEXT1 TEXT23'],
       [4.0, 5.0, 6.0, 'TEXT4 TEXT567'],
       [7.0, 8.0, 9.0, 'TEXT8 TEXT90'],
       [10.0, 11.0, 12.0, 'TEXT123 TEXT456789']], dtype=object)
>>> outpd
      0     1     2                    3
0   1.0   2.0   3.0         TEXT1 TEXT23
1   4.0   5.0   6.0        TEXT4 TEXT567
2   7.0   8.0   9.0         TEXT8 TEXT90
3  10.0  11.0  12.0  TEXT123 TEXT456789
>>> outpd2
   col1  col2  col3             coltext
0   1.0   2.0   3.0         TEXT1 TEXT23
1   4.0   5.0   6.0        TEXT4 TEXT567
2   7.0   8.0   9.0         TEXT8 TEXT90
3  10.0  11.0  12.0  TEXT123 TEXT456789
```

pygmt.clib.Session.virtualfile_to_raster

`Session.`**`virtualfile_to_raster`**(*vfname*, *kind='grid'*, *outgrid=None*)

Output raster data stored in a virtual file to an `xarray.DataArray` object.

The raster data can be a grid, an image or a cube.

Parameters

- **vfname** (`str`) – The virtual file name that stores the result grid/image/cube.
- **kind** (`Literal['grid', 'image', 'cube', None]`, default: `'grid'`) – Type of the raster data. Valid values are `"grid"`, `"image"`, `"cube"` or `None`. If `None`, will inquire the data type from the virtual file name.
- **outgrid** (`str` | `None`, default: `None`) – Name of the output grid/image/cube. If specified, it means the raster data was already saved into an actual file and will return `None`.

Return type

`DataArray` | `None`

Returns

result – The result grid/image/cube. If `outgrid` is specified, return `None`.

Examples

```
>>> from pathlib import Path
>>> from pygmt.clib import Session
>>> from pygmt.helpers import GMTTempFile
>>> with Session() as lib:
...     # file output
...     with GMTTempFile(suffix=".nc") as tmpfile:
...         outgrid = tmpfile.name
...         with lib.virtualfile_out(kind="grid", fname=outgrid) as voutgrd:
...             lib.call_module("read", ["@earth_relief_01d_g", voutgrd, "-Tg"])
...             result = lib.virtualfile_to_raster(
...                 vfname=voutgrd, outgrid=outgrid
...             )
...             assert result == None
...             assert Path(outgrid).stat().st_size > 0
...
...     # xarray.DataArray output
...     outgrid = None
...     with lib.virtualfile_out(kind="grid", fname=outgrid) as voutgrd:
...         lib.call_module("read", ["@earth_relief_01d_g", voutgrd, "-Tg"])
...         result = lib.virtualfile_to_raster(vfname=voutgrd, outgrid=outgrid)
...         assert isinstance(result, xr.DataArray)
```

pygmt.clib.Session.write_data

`Session.`**`write_data`**(*family*, *geometry*, *mode*, *wesn*, *output*, *data*)

Write a GMT data container to a file.

The data container should be created by *pygmt.clib.Session.create_data*.

Wraps `GMT_Write_Data` but only allows writing to a file. So the `method` argument is omitted.

Parameters

- **family** (*str*) – A valid GMT data family name (e.g., `'GMT_IS_DATASET'`). See the `FAMILIES` attribute for valid names. Don't use the `GMT_VIA_VECTOR` or `GMT_VIA_MATRIX` constructs for this. Use `GMT_IS_VECTOR` and `GMT_IS_MATRIX` instead.
- **geometry** (*str*) – A valid GMT data geometry name (e.g., `'GMT_IS_POINT'`). See the `GEOMETRIES` attribute for valid names.
- **mode** (*str*) – How the data is to be written to the file. This option varies depending on the given family. See the GMT API documentation for details.
- **wesn** (*list or numpy array*) – [xmin, xmax, ymin, ymax, zmin, zmax] of the data. Must have 6 elements.
- **output** (*str*) – The output file name.
- **data** (`ctypes.c_void_p`) – Pointer to the data container created by *pygmt.clib.Session.create_data*.

Raises

GMTCLibError – For invalid input arguments or if the GMT API functions returns a non-zero status code.

Return type
None

GMT modules are executed through the *call_module* method:

clib.Session.call_module(module, args)	Call a GMT module with the given arguments.

Passing memory blocks between Python data objects (e.g. numpy.ndarray, pandas.Series, xarray.DataArray, etc) and GMT happens through *virtual files*. These methods are context managers that automate the conversion of Python objects to and from GMT virtual files:

clib.Session.virtualfile_in([check_kind, ...])	Store any data inside a virtual file.
clib.Session.virtualfile_out([kind, fname])	Create a virtual file or an actual file for storing output data.
clib.Session.virtualfile_to_dataset(vfname)	Output a tabular dataset stored in a virtual file to a different format.
clib.Session.virtualfile_to_raster(vfname[, ...])	Output raster data stored in a virtual file to an xarray.DataArray object.

Low level access (these are mostly used by the pygmt.clib package):

clib.Session.create(name)	Create a new GMT C API session.
clib.Session.destroy()	Destroy the currently open GMT API session.
clib.Session.__getitem__(name)	Get the value of a GMT constant.
clib.Session.__enter__()	Create a GMT API session.
clib.Session.__exit__(exc_type, exc_value, ...)	Destroy the currently open GMT API session.
clib.Session.get_default(name)	Get the value of a GMT configuration parameter or a GMT API parameter.
clib.Session.get_common(option)	Inquire if a GMT common option has been set and return its current value if possible.
clib.Session.create_data(family, geometry, mode)	Create an empty GMT data container and allocate space to hold data.
clib.Session.put_matrix(dataset, matrix[, pad])	Attach a 2-D numpy array to a GMT dataset.
clib.Session.put_strings(dataset, family, ...)	Attach a 1-D numpy array of dtype str as a column on a GMT dataset.
clib.Session.put_vector(dataset, column, vector)	Attach a 1-D numpy array as a column on a GMT dataset.
clib.Session.read_data(infile, kind[, ...])	Read a data file into a GMT data container.
clib.Session.write_data(family, geometry, ...)	Write a GMT data container to a file.
clib.Session.open_virtualfile(family, ...)	Open a GMT virtual file associated with a data object for reading or writing.
clib.Session.read_virtualfile(vfname[, kind])	Read data from a virtual file and optionally cast into a GMT data container.
clib.Session.extract_region()	Extract the region of the currently active figure.
clib.Session.get_libgmt_func(name[, ...])	Get a ctypes function from the libgmt shared library.
clib.Session.virtualfile_from_grid(grid)	Store a grid in a virtual file.
clib.Session.virtualfile_from_stringio(string	Store a io.StringIO object in a virtual file.
clib.Session.virtualfile_from_matrix(matrix)	Store a 2-D numpy array as a matrix inside a virtual file.
clib.Session.virtualfile_from_vectors(vectors	Store a sequence of 1-D vectors as columns of a dataset inside a virtual file.

11.11.2 pygmt.clib.Session.__getitem__

Session.**__getitem__**(*name*)

Get the value of a GMT constant.

Parameters

name (str) – The name of the constant (e.g., "GMT_SESSION_EXTERNAL").

Return type

int

Returns

value – Integer value of the constant. Do not rely on this value because it might change.

11.11.3 pygmt.clib.Session.__enter__

Session.**__enter__**()

Create a GMT API session.

Calls *pygmt.clib.Session.create*.

11.11.4 pygmt.clib.Session.__exit__

Session.**__exit__**(*exc_type*, *exc_value*, *traceback*)

Destroy the currently open GMT API session.

Calls *pygmt.clib.Session.destroy*.

CHAPTER

TWELVE

TECHNICAL REFERENCE

The Technical Reference section provides detailed information on the technical aspects of GMT and PyGMT, including supported encodings, fonts, bit and hachure patterns, and other essential components for creating high-quality visualizations. For additional details, visit the GMT Technical Reference.

12.1 Common Parameters

distcalc
Determine how spherical distances are calculated. Valid values are:

- `"g"`: Perform great circle distance calculations, with parameters such as distance increments or radii compared against calculated great circle distances [Default].
- `"e"`: Select ellipsoidal (or geodesic) mode for the highest precision but slowest calculation time.
- `"f"`: Select Flat Earth mode, which gives a more approximate but faster result.

Note: (1) All spherical distance calculations depend on the current ellipsoid (PROJ_ELLIPSOID), the definition of the mean radius (PROJ_MEAN_RADIUS), and the specification of the latitude type (PROJ_AUX_LATITUDE). Geodesic distance calculations are also controlled by the algorithm to use for geodesic calculations (PROJ_GEODESIC). (2) Coordinate transformations that can use ellipsoidal or spherical forms will first consult this parameter if given.

verbose
Select verbosity level, which modulates the messages written to stderr.

Choose among 7 levels of verbosity [Default is `"warning"`]:

- `"quiet"`: Quiet, not even fatal error messages are produced
- `"error"`: Error messages only
- `"warning"`: Warnings [Default]
- `"timing"`: Timings (report runtimes for time-intensive algorithms)
- `"info"`: Informational messages (same as `verbose=True`)
- `"compat"`: Compatibility warnings
- `"debug"`: Debugging messages

12.2 GMT Map Projections

The table below shows the projection codes for the 32 GMT map projections:

PyGMT Projection Argument	Projection Name
Alon$_0$/lat$_0$[/*horizon*]/*width*	*Lambert azimuthal equal-area projection*
Blon$_0$/lat$_0$/lat$_1$/lat$_2$/*width*	*Albers conic equal-area projection*
Clon$_0$/lat$_0$/*width*	*Cassini cylindrical projection*
Cyl_stere/[lon$_0$/[lat$_0$/]]*width*	*Cylindrical stereographic projection*
Dlon$_0$/lat$_0$/lat$_1$/lat$_2$/*width*	*Equidistant conic projection*
Elon$_0$/lat$_0$[/*horizon*]/*width*	*Azimuthal equidistant projection*
Flon$_0$/lat$_0$[/*horizon*]/*width*	*Gnomonic projection*
Glon$_0$/lat$_0$[/*horizon*]/*width*	*Orthographic projection*
Glon$_0$/lat$_0$/*width*[**+a***azimuth*][**+t***tilt*][**+v***vwidth*/*vheight*][**+v**	*Perspective projection*
H[lon$_0$/]*width*	*Hammer projection*
I[lon$_0$/]*width*	*Sinusoidal projection*
J[lon$_0$/]*width*	*Miller cylindrical projection*
Kf[lon$_0$/]*width*	*Eckert IV equal-area projection*
Ks[lon$_0$/]*width*	*Eckert VI equal-area projection*
Llon$_0$/lat$_0$/lat$_1$/lat$_2$/*width*	*Lambert conic conformal projection*
M[lon$_0$/[lat$_0$/]]*width*	*Mercator projection*
N[lon$_0$/]*width*	*Robinson projection*
Oalon$_0$/lat$_0$/*azimuth*/*width*[**+v**]	Oblique Mercator projection: *1. origin and azimuth*
Oblon$_0$/lat$_0$/lon$_1$/lat$_1$/*width*[**+v**]	Oblique Mercator projection: *2. two points*
Oclon$_0$/lat$_0$/lon$_p$/lat$_p$/*width*[**+v**]	Oblique Mercator projection: *3. origin and projection pole*
P*width*[**+a**][**+f**[**e**|**p**|*radius*]][**+r***offset*][**+t***origin*][**+z**[**p**|*radiu*	Polar *azimuthal* (θ, r) or cylindrical
Poly/[lon$_0$/[lat$_0$/]]*width*	*Polyconic projection*
Q[lon$_0$/[lat$_0$/]]*width*	*Cylindrical equidistant projection*
R[lon$_0$/]*width*	*Winkel Tripel projection*
Slon$_0$/lat$_0$[/*horizon*]/*width*	*General stereographic projection*
Tlon$_0$[/lat$_0$]/*width*	*Transverse Mercator projection*
U*zone*/*width*	*Universal Transverse Mercator projection*
V[lon$_0$/]*width*	*Van der Grinten projection*
W[lon$_0$/]*width*	*Mollweide projection*
X*width*[**l**|**p***exp*|**T**|**t**][/*height*[**l**|**p***exp*|**T**|**t**]][**d**]	Cartesian *linear*, *logarithmic*, *power*, and time
Ylon$_0$/lat$_0$/*width*	*Cylindrical equal-area projection*
+proj=spilhaus+width=*width*	*Spilhaus projection*

12.3 Dimension and Distance Units

As a geospatial analysis and mapping package, GMT needs to distinguish between two types of units: those used for plotting and those used for geographic distances. The former are called **dimension units** (or plot units) and the latter are called **distance units**.

12.3.1 Dimension Units

Dimension units are used for map dimensions, symbol sizes, pen widths, etc.

Code	Unit	Notes
`c`	Centimeter	Default for most dimensions
`i`	Inch	1 inch = 2.54 cm
`p`	Point	1 point = 1/72 inch

Defaults: If no unit is given, GMT uses PROJ_LENGTH_UNIT (default is `c`). Fonts and pen thicknesses default to `p`.

12.3.2 Distance Units

GMT supports various distance units for geographic calculations.

Code	Unit	Notes
`d`	Degree	Arc degree
`m`	Minute of arc	1/60 degree
`s`	Second of arc	1/3600 degree
`e`	Meter	Default distance unit
`k`	Kilometer	1000 meters
`f`	Foot	0.3048 meters
`u`	US Survey Foot	1200/3937 meters
`M`	Statute Mile	1.60934 km
`n`	Nautical Mile	1.852 km

12.4 Supported Fonts

PyGMT supports the 35 standard PostScript fonts. The table below lists them with their font numbers and font names. When specifying fonts in PyGMT, you can either give the font name or just the font number. For example, to use the font "Helvetica", you can use either `"Helvetica"` or `"0"`. For the special fonts "Symbol" (**12**) and "ZapfDingbats" (**34**), see the *Supported Encodings and Non-ASCII Characters* for the character set. The image below the table shows a visual sample for each font.

Font No.	Font Name	Font No.	Font Name
0	Helvetica	17	Bookman-Demi
1	Helvetica-Bold	18	Bookman-DemiItalic
2	Helvetica-Oblique	19	Bookman-Light
3	Helvetica-BoldOblique	20	Bookman-LightItalic
4	Times-Roman	21	Helvetica-Narrow
5	Times-Bold	22	Helvetica-Narrow-Bold
6	Times-Italic	23	Helvetica-Narrow-Oblique
7	Times-BoldItalic	24	Helvetica-Narrow-BoldOblique
8	Courier	25	NewCenturySchlbk-Roman
9	Courier-Bold	26	NewCenturySchlbk-Italic
10	Courier-Oblique	27	NewCenturySchlbk-Bold
11	Courier-BoldOblique	28	NewCenturySchlbk-BoldItalic
12	Symbol	29	Palatino-Roman
13	AvantGarde-Book	30	Palatino-Italic
14	AvantGarde-BookOblique	31	Palatino-Bold
15	AvantGarde-Demi	32	Palatino-BoldItalic
16	AvantGarde-DemiOblique	33	ZapfChancery-MediumItalic
		34	ZapfDingbats

#	Font Name	#	Font Name
0	Helvetica	**17**	**Bookman-Demi**
1	**Helvetica-Bold**	***18***	***Bookman-DemiItalic***
2	*Helvetica-Oblique*	19	Bookman-Light
3	***Helvetica-BoldOblique***	*20*	*Bookman-LightItalic*
4	Times-Roman	21	Helvetica-Narrow
5	**Times-Bold**	**22**	**Helvetica-Narrow-Bold**
6	*Times-Italic*	*23*	*Helvetica-Narrow-Oblique*
7	***Times-BoldItalic***	***24***	***Helvetica-Narrow-BoldOblique***
8	Courier	25	NewCenturySchlbk-Roman
9	**Courier-Bold**	*26*	*NewCenturySchlbk-Italic*
10	*Courier-Oblique*	**27**	**NewCenturySchlbk-Bold**
11	***Courier-BoldOblique***	***28***	***NewCenturySchlbk-BoldItalic***
12	Σψμβολ (Symbol)	29	Palatino-Roman
13	AvantGarde-Book	*30*	*Palatino-Italic*
14	*AvantGarde-BookOblique*	**31**	**Palatino-Bold**
15	**AvantGarde-Demi**	*32*	***Palatino-BoldItalic***
16	***AvantGarde-DemiOblique***	*33*	*ZapfChancery-MediumItalic*
		34	✺❁❒❄✤❇■✳✪❁▼▲ (ZapfDingbats)

12.5 Text Formatting

The table below shows how text added to a plot, such as text strings added via `Figure.text`, plot title, labels of Cartesian axes and colorbars, and legend entries. Changing the font as well as its color and size in this way allows to change those only for specific characters of a longer text. The supported fonts are listed at *Supported Fonts*.

Symbol	Behavior
@%*font*%*TEXT*@%%	Change the *font* of *TEXT*
@;*color*;*TEXT*@;;	Change the font *color* of *TEXT*
@:*size*:*TEXT*@::	Change the font *size* of *TEXT*
@-*TEXT*@-	Toggle subscripts on/off
@+*TEXT*@+	Toggle superscript on/off
@#*TEXT*@#	Toggle small caps on/off
@_*TEXT*@_	Toggle underline on/off
@!*CHAR1CHAR2*	Print two characters on top of each other (composite characters, e.g., overstrike)
@~*TEXT*@~	Toggle Greek letters on/off
@@	Print the @ sign
@.	Print the ° (degree) symbol
@s	Print letter ß
@i	Print letter í
@a or @A	Print letter å or Å
@c or @C	Print letter ç or Ç
@e or @E	Print letter æ or Æ
@n or @N	Print letter ñ or Ñ
@o or @O	Print letter ø or Ø
@u or @U	Print letter ü or Ü

12.6 Supported Encodings and Non-ASCII Characters

PyGMT supports a number of encodings and each encoding contains a set of ASCII and non-ASCII characters. In PyGMT, you can use any of these ASCII and non-ASCII characters in arguments and text strings. When using non-ASCII characters in PyGMT, the easiest way is to copy and paste the character from the encoding tables below.

Note: The special character (REPLACEMENT CHARACTER) is used to indicate that the character is undefined in the encoding.

12.6.1 Adobe ISOLatin1+ Encoding

Octal	0	1	2	3	4	5	6	7
\03x	◆	•	…	™	—	–	fi	ž
\04x		!	“	#	$	%	&	’
\05x	(	)	*	+	,	−	.	/
\06x	0	1	2	3	4	5	6	7
\07x	8	9	:	;	<	=	>	?
\10x	@	A	B	C	D	E	F	G
\11x	H	I	J	K	L	M	N	O
\12x	P	Q	R	S	T	U	V	W
\13x	X	Y	Z	[	\	]	^	_
\14x	‘	a	b	c	d	e	f	g
\15x	h	i	j	k	l	m	n	o
\16x	p	q	r	s	t	u	v	w
\17x	x	y	z	{	|	}	~	š
\20x	Œ	†	‡	Ł	⁄	‹	Š	›
\21x	œ	Ÿ	Ž	ł	‰	„	“	”
\22x	ı	`	´	ˆ	˜	¯	˘	˙
\23x	¨	¸	°	˛	‘	˝	˚	ˇ
\24x		¡	¢	£	¤	¥	¦	§
\25x	¨	©	ª	«	¬	-	®	¯
\26x	°	±	2	3	´	µ	¶	·
\27x	¸	1	º	»	¼	½	¾	¿
\30x	À	Á	Â	Ã	Ä	Å	Æ	Ç
\31x	È	É	Ê	Ë	Ì	Í	Î	Ï
\32x	Đ	Ñ	Ò	Ó	Ô	Õ	Ö	×
\33x	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
\34x	à	á	â	ã	ä	å	æ	ç
\35x	è	é	ê	ë	ì	í	î	ï
\36x	ð	ñ	ò	ó	ô	õ	ö	÷
\37x	ø	ù	ú	û	ü	ý	þ	ÿ

12.6.2 Adobe Symbol Encoding

Octal	0	1	2	3	4	5	6	7
\04x		!	∀	#	∃	%	&	∋
\05x	(	)	*	+	,	−	.	/
\06x	0	1	2	3	4	5	6	7
\07x	8	9	:	;	<	=	>	?
\10x	≅	Α	Β	Χ	Δ	Ε	Φ	Γ
\11x	Η	Ι	ϑ	Κ	Λ	Μ	Ν	Ο
\12x	Π	Θ	Ρ	Σ	Τ	ϒ	ς	Ω
\13x	Ξ	Ψ	Ζ	[	∴	]	⊥	_
\14x	⍰	α	β	χ	δ	ε	φ	γ
\15x	η	ι	ϕ	κ	λ	μ	ν	ο
\16x	π	θ	ρ	σ	τ	υ	ϖ	ω
\17x	ξ	ψ	ζ	{	|	}	~	
\24x	€	ϒ	′	≤	⁄	∞	ƒ	♣
\25x	♦	♥	♠	↔	←	↑	→	↓
\26x	°	±	″	≥	×	∝	∂	•
\27x	÷	≠	≡	≈	…	⍰	⍰	↵
\30x	ℵ	ℑ	ℜ	℘	⊗	⊕	∅	∩
\31x	∪	⊃	⊇	⊄	⊂	⊆	∈	∉
\32x	∠	∇	®	©	™	∏	√	·
\33x	¬	∧	∨	⇔	⇐	⇑	⇒	⇓
\34x	◊	〈	®	©	™	∑	⎛	⎪
\35x	⎝	⎡	⎪	⎣	⎧	⎨	⎩	⎪
\36x		〉	⌠	⌡	⎪	⎫	⎭	⎪
\37x	⎫	⎤	⎪	⎦	⎭	⎬	⎫	

Note: The octal code `\140` represents the RADICAL EXTENDER character, which is not available in the Unicode character set.

12.6.3 Adobe ZapfDingbats Encoding

Octal	0	1	2	3	4	5	6	7
\04x		✁	✂	✃	✄	☎	✆	✇
\05x	✈	✉	☛	☞	✌	✍	✎	✏
\06x	✐	✑	✒	✓	✔	✕	✖	✗
\07x	✘	✙	✚	✛	✜	✝	✞	✟
\10x	✠	✡	✢	✣	✤	✥	✦	✧
\11x	★	✩	✪	✫	✬	✭	✮	✯
\12x	✰	✱	✲	✳	✴	✵	✶	✷
\13x	✸	✹	✺	✻	✼	✽	✾	✿
\14x	❀	❁	❂	❃	❄	❅	❆	❇
\15x	❈	❉	❊	❋	●	❍	■	❑
\16x	❒	❑	❒	▲	▼	◆	❖	◗
\17x	❘	❙	❚	❛	❜	❝	❞	❟
\20x	❨	❩	❪	❫	❬	❭	❮	❯
\21x	❰	❱	❲	❳	❴	❵	❟	❟
\24x	❟	❡	❢	❣	❤	❥	❦	❧
\25x	♣	♦	♥	♠	①	②	③	④
\26x	⑤	⑥	⑦	⑧	⑨	⑩	❶	❷
\27x	❸	❹	❺	❻	❼	❽	❾	❿
\30x	①	②	③	④	⑤	⑥	⑦	⑧
\31x	⑨	⑩	❶	❷	❸	❹	❺	❻
\32x	❼	❽	❾	❿	➔	→	↔	↕
\33x	➘	➙	➚	➛	➜	→	→	➟
\34x	➟	➡	➢	➢	➤	➥	➦	➧
\35x	➨	➩	➪	➫	➬	➭	➮	➯
\36x	❟	➱	➲	➳	➴	➵	➶	➷
\37x	➸	➹	➺	➻	➼	➽	➾	❟

12.6.4 ISO/IEC 8859

PyGMT also supports the ISO/IEC 8859 standard for 8-bit character encodings. Refer to ISO/IEC 8859 for descriptions of the different parts of the standard.

For a list of the characters in each part of the standard, refer to the following links:

- ISO/IEC 8859-1
- ISO/IEC 8859-2
- ISO/IEC 8859-3
- ISO/IEC 8859-4
- ISO/IEC 8859-5
- ISO/IEC 8859-6
- ISO/IEC 8859-7
- ISO/IEC 8859-8
- ISO/IEC 8859-9
- ISO/IEC 8859-10
- ISO/IEC 8859-11

- ISO/IEC 8859-13
- ISO/IEC 8859-14
- ISO/IEC 8859-15
- ISO/IEC 8859-16

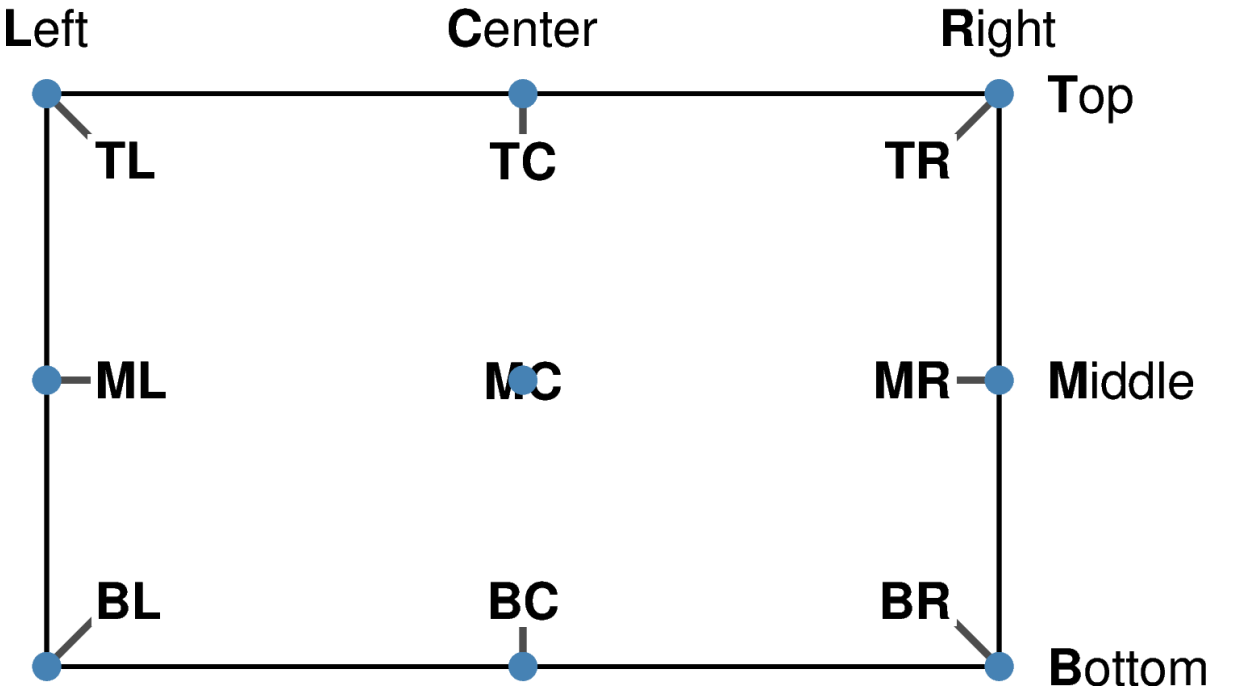
12.7 Justification Codes

To place plot embellishments, such as scalebars, directional roses, colorbars, legends, text, or images on a plot, two points have to be specified: a point somewhere on the figure (**reference point**) and a point on the feature (**anchor point**). For both, users can use a two-character code, a combination of a vertical code and a horizontal code (order-independent):

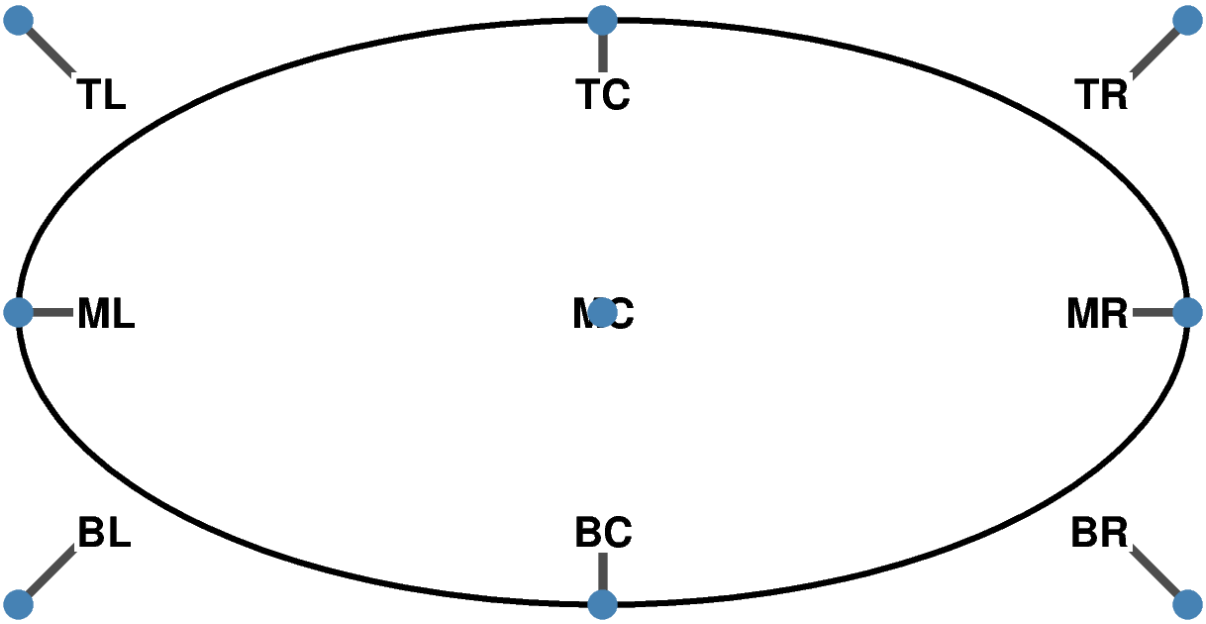
- Vertical: **T** (Top), **M** (Middle), **B** (Bottom)
- Horizontal: **L** (Left), **C** (Centre), **R** (Right)

For example, `"TL"` means **T**op **L**eft.

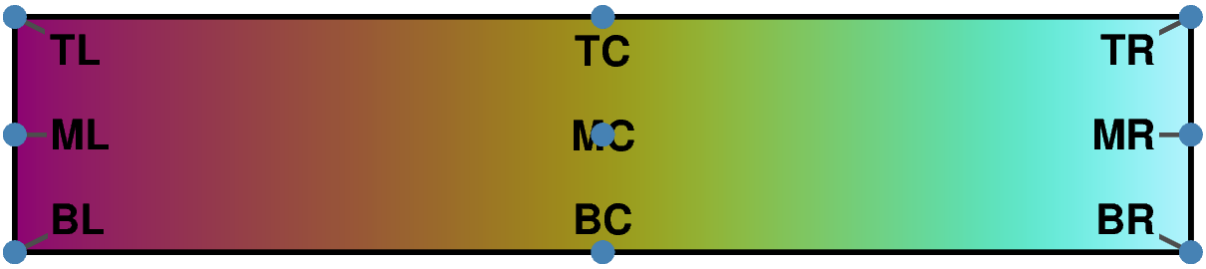
The possible nine justification codes are visualized in the sketch below:



For a non-rectangular geographic basemap, the justification codes refer to the invisible, rectangular map bounding box:



Plot embellishments can be abstracted as rectangles. Here, the justification codes are shown exemplary for a colorbar.



12.8 Environment Variables

PyGMT's behavior can be controlled through various environment variables. These variables can be set either in your shell environment or within your Python script using the `os.environ` dictionary.

Here we list the environment variables used by PyGMT which are categorized into three groups:

1. System environment variables
2. GMT/PyGMT environment variables
3. Module-specific environment variables

12.8.1 System Environment Variables

TZ

Specify the time zone for the current calendar time. It can be set to a string that defines the timezone, such as `"UTC"`, `"America/New_York"`, or `"Europe/London"`. Refer to Specifying the Time Zone with TZ for the valid format. If not set, the system's default timezone is used.

12.8.2 GMT/PyGMT Environment Variables

GMT_LIBRARY_PATH
: Specify the directory where the GMT shared library is located. This is useful when GMT is installed in a non-standard location or when you want to use a specific version of GMT. If not set, PyGMT will attempt to find the GMT library in standard system locations.

PYGMT_USE_EXTERNAL_DISPLAY
: Whether to use external viewers for displaying images. If set to `"false"`, PyGMT will not attempt to open images in external viewers. This can be useful when running tests or building the documentation to avoid popping up windows.

12.8.3 Module-Specific Environment Variables

X2SYS_HOME
: Specify the directory where x2sys databases and related settings will be stored. This environment variable is used by x2sys-related functions (e.g., *pygmt.x2sys_init*) to manage and access x2sys data. If not set, these functions will use a default directory or prompt for a location.

12.9 Supported Array Dtypes

PyGMT uses NumPy arrays as its core data structure for storing data and exchanging data with the GMT C API. This design allows PyGMT to support a wide range of array-like objects and data types (*dtypes*), as long as they can be converted to NumPy arrays. This page provides a comprehensive overview of the array dtypes supported by PyGMT.

12.9.1 Numeric Dtypes

In addition to Python's built-in numeric types (`int` and `float`), PyGMT supports most of the numeric dtypes provided by NumPy, pandas, and PyArrow.

Signed Integers

- `numpy.int8`, `numpy.int16`, `numpy.int32`, `numpy.int64`, `numpy.longlong`
- `pandas.Int8Dtype`, `pandas.Int16Dtype`, `pandas.Int32Dtype`, `pandas.Int64Dtype`
- `pyarrow.int8`, `pyarrow.int16`, `pyarrow.int32`, `pyarrow.int64`

Unsigned Integers

- `numpy.uint8`, `numpy.uint16`, `numpy.uint32`, `numpy.uint64`, `numpy.ulonglong`
- `pandas.UInt8Dtype`, `pandas.UInt16Dtype`, `pandas.UInt32Dtype`, `pandas.UInt64Dtype`
- `pyarrow.uint8`, `pyarrow.uint16`, `pyarrow.uint32`, `pyarrow.uint64`

Floating-point numbers

- `numpy.float32`, `numpy.float64`
- `pandas.Float32Dtype`, `pandas.Float64Dtype`
- `pyarrow.float32`, `pyarrow.float64`

Note

1. Signed and unsigned integer dtypes from pandas and PyArrow (e.g., `pandas.Int8Dtype`, `pyarrow.int8`) support missing values like `None` or `pandas.NA`, whereas NumPy's corresponding dtypes (e.g., `numpy.int8`) don't. Arrays of these dtypes containing missing values are automatically cast to `numpy.float64` internally.

2. For 3-D `xarray.DataArray` objects representing raster images, only 8-bit unsigned integers (i.e., `numpy.uint8`) are supported.

Note

Examples of numeric arrays supported by PyGMT:

```
# A list of integers
[1, 2, 3]

# A NumPy array with dtype int32
np.array([1, 2, 3], dtype=np.int32)

# A pandas Series with nullable Int32 dtype
pd.Series([1, 2, 3], dtype="Int32")

# A pandas Series with nullable Int32 dtype and missing values
pd.Series([1, 2, pd.NA], dtype="Int32")

# A pandas Series using a PyArrow-backed float64 dtype
pd.Series([1, 2, 3], dtype="float64[pyarrow]")

# A PyArrow array with dtype uint8
pa.array([1, 2, 3], type=pa.uint8())
```

12.9.2 String Dtypes

In addition to Python's built-in `str` type, PyGMT also supports the following string dtypes:

- NumPy: `numpy.str_` or fixed-width Unicode string dtype (e.g., `"U10"`)
- pandas: `pandas.StringDtype`, with different storage backends, including `string[python]` and `string[pyarrow]`
- PyArrow: `pyarrow.string`/`pyarrow.utf8`, `pyarrow.large_string`/`pyarrow.large_utf8`, and `pyarrow.string_view`

Note

Examples of string arrays supported by PyGMT:

```
# A list of strings
["a", "b", "c"]

# A NumPy string array
np.array(["a", "b", "c"])
np.array(["a", "b", "c"], dtype=np.str_)

# A pandas.Series string array
pd.Series(["a", "b", "c"], dtype="string")
pd.Series(["a", "b", "c"], dtype="string[python]")
pd.Series(["a", "b", "c"], dtype="string[pyarrow]")
```

```
# A PyArrow array with pyarrow.string dtype
pa.array(["a", "b", "c"], type=pa.string())
```

12.9.3 Datetime Dtypes

PyGMT supports a variety of datetime types:

- A list/tuple of elements in Python's built-in `datetime.datetime` or `datetime.date`, datetime-like strings, or mixed.
- NumPy arrays: `numpy.datetime64` with various resolutions
- pandas objects with `numpy.datetime64`, `pandas.DatetimeTZDtype`, `pyarrow.timestamp` with various resolution and timezone support, and pyarrow-backend dtypes like `date32[day][pyarrow]` and `date64[ms][pyarrow]`,
- PyArrow: `pyarrow.date32`, `pyarrow.date64` and `pyarrow.timestamp` with various resolutions and timezone support.

12.9.4 Timedelta Dtypes

PyGMT supports NumPy arrays with the `numpy.timedelta64` dtype. Timedelta values are passed to GMT as their underlying integer values, so their unit determines the numeric scale. For example, a `timedelta64[D]` array is interpreted as days, whereas a `timedelta64[s]` array is interpreted as seconds.

Timedelta values can also be used in sequence-valued parameters such as `region`. The timedelta dtype does not, by itself, configure a GMT relative-time axis; configure GMT time settings explicitly when a relative-time axis is required.

12.9.5 Unsupported Dtypes

The following dtypes are intentionally unsupported, and should be cast to an appropriate supported dtype before passing to PyGMT:

- Floating-point dtypes: `numpy.float16`, `pyarrow.float16` and `numpy.longdouble`
- Boolean dtypes: `numpy.bool`, `pandas.BooleanDtype` and `pyarrow.bool_`
- Complex dtypes: `numpy.complex64`, `numpy.complex128`
- `numpy.bytes_` and `numpy.void`

The `numpy.object_` dtype is also not supported. PyGMT may convert object arrays that can be interpreted as datetimes or text, but applications should create arrays with an explicit supported dtype instead.

CHAPTER

THIRTEEN

MINIMUM SUPPORTED VERSIONS

PyGMT has adopted SPEC 0 alongside the rest of the scientific Python ecosystem, and will therefore:

- Drop support for Python versions 3 years after their initial release.
- Drop support for core package dependencies (NumPy, pandas, xarray) 2 years after their initial release.

In addition to the above, the PyGMT team has also decided to:

- Drop support for GMT versions 3 years after their initial release, while ensuring at least two latest minor versions remain supported.
- Maintain support for *optional dependencies* for at least 1 year after their initial release. Users are encouraged to use the most up-to-date optional dependencies where possible.

Note

The SPEC 0 policy is enforced on a best-effort basis, and the PyGMT team may decide to drop support for core (and optional) package dependencies earlier than recommended for compatibility reasons.

PyGMT Version	Documentation	GMT	Python	NumPy	pandas	xarray
Dev*	Web, HTML+ZIP, PDF	>=6.5.0	>=3.12	>=2.2	>=2.3	>=2024.10
v0.19.0	Web, HTML+ZIP, PDF	>=6.5.0	>=3.12	>=2.1	>=2.3	>=2024.7
v0.18.0	Web, HTML+ZIP, PDF	>=6.5.0	>=3.12	>=2.0	>=2.2	>=2024.5
v0.17.0	Web, HTML+ZIP, PDF	>=6.5.0	>=3.11	>=1.26	>=2.2	>=2023.10
v0.16.0	Web, HTML+ZIP, PDF	>=6.4.0	>=3.11	>=1.26	>=2.1	>=2023.07
v0.15.0	Web, HTML+ZIP, PDF	>=6.4.0	>=3.11	>=1.25	>=2.0	>=2023.04
v0.14.2	Web, HTML+ZIP	>=6.4.0	>=3.11	>=1.25	>=2.0	>=2023.04
v0.14.1	Web, HTML+ZIP	>=6.4.0	>=3.11	>=1.25	>=2.0	>=2023.04
v0.14.0	Web, HTML+ZIP	>=6.4.0	>=3.11	>=1.25	>=2.0	>=2023.04
v0.13.0	Web, HTML+ZIP	>=6.3.0	>=3.10	>=1.24	>=1.5	>=2022.09
v0.12.0	Web, HTML+ZIP	>=6.3.0	>=3.10	>=1.23	>=1.5	>=2022.06
v0.11.0	Web, HTML+ZIP	>=6.3.0	>=3.9	>=1.23		
v0.10.0	Web, HTML+ZIP	>=6.3.0	>=3.9	>=1.22		
v0.9.0	Web, HTML+ZIP	>=6.3.0	>=3.8	>=1.21		
v0.8.0	Web, HTML+ZIP	>=6.3.0	>=3.8	>=1.20		
v0.7.0	Web, HTML+ZIP	>=6.3.0	>=3.8	>=1.20		
v0.6.1	Web, HTML+ZIP	>=6.3.0	>=3.8	>=1.19		
v0.6.0	Web, HTML+ZIP	>=6.3.0	>=3.8	>=1.19		
v0.5.0	Web, HTML+ZIP	>=6.2.0	>=3.7	>=1.18		
v0.4.1	Web, HTML+ZIP	>=6.2.0	>=3.7	>=1.17		
v0.4.0	Web, HTML+ZIP	>=6.2.0	>=3.7	>=1.17		
v0.3.1	Web, HTML+ZIP	>=6.1.1	>=3.7			
v0.3.0	Web, HTML+ZIP	>=6.1.1	>=3.7			
v0.2.1	Web, HTML+ZIP	>=6.1.1	>=3.6			
v0.2.0	Web, HTML+ZIP	>=6.1.1	3.6 - 3.8			
v0.1.2	Web, HTML+ZIP	>=6.0.0	3.6 - 3.8			
v0.1.1	Web, HTML+ZIP	>=6.0.0	3.6 - 3.8			
v0.1.0	Web, HTML+ZIP	>=6.0.0	3.6 - 3.8			

*Dev reflects the main branch and is for the upcoming release.

CHAPTER

FOURTEEN

CHANGELOG

14.1 Release v0.19.0 (2026/07/07)

14.1.1 Highlights

- A PyGMT paper has been accepted for publication in *Geochemistry, Geophysics, Geosystems*. The citation is as follows:

  > Tian, D., Fröhlich, Y., Leong, W. J., Grund, M., Schlitzer, W., Jones, M., Uieda, L., Luis, J. M. F. (2026). PyGMT: Bridging Python and the Generic Mapping Tools for Geospatial Visualization and Analysis. Geochemistry, Geophysics, Geosystems, 27, e2026GC013105. https://doi.org/10.1029/2026GC013105

  Feel free to cite this paper when using PyGMT in your research.
- PyGMT now has an official logo, which is available via the Figure.pygmtlogo method (#4616)
- Nineteenth minor release of PyGMT
- Six new high-level plotting methods and two new grid processing functions
- Ten new gallery examples and tutorials
- New `Frame`/`Axis` classes for setting frame and axes

14.1.2 New Features

- Add Figure.choropleth for plotting choropleth maps (#2798)
- Add Figure.directional_rose for plotting a directional rose on map (#4025, #4702)
- Add Figure.fill_between for filling between two curves (#4679, #4698, #4696, #4692)
- Add Figure.magnetic_rose for plotting a magnetic rose on map (#4051, #4702)
- Add Figure.paragraph for typesetting one or multiple paragraphs of text strings (#3709, #4605)
- Add Figure.scalebar for plotting a scale bar on maps (#4015)
- Add the Frame/Axis class for setting frame and axes (#4406, #4593, #4589, #4520, #4591)
- Add pygmt.grdmask for creating mask grid from polygons or point coverage (#4463)
- Add pygmt.grdpaste for joining two grids along their common edge (#4399)

14.1.3 Enhancements

- Add a note/warning that PS_CONVERT is not supported (#4342)
- Add filtering tags for gallery examples of projections (#4643)
- Figure.colorbar: Add parameters dpi/monochrome (#4501, #4499)
- Figure.colorbar: Add parameters label/unit/annot/tick/grid and more to set colorbar annotations/labels (#4407)
- Figure.grdview: Add parameters smooth and surftype/dpi/mesh_fill/nan_transparent/monochrome to control surface types (#4234, #4444)
- Figure.subplot: Add parameters tag/tag_box/tag_position/tag_orientation/tag_number_style/tag_font for controlling subplot tagging (#4313)
- Figure.ternary: Support Frame's xaxis/yaxis/zaxis attributes to set axis settings for a/b/c axes (#4517)
- Figure.ternary: Add the 'no_clip' parameter for plotting symbols outside the ternary diagram (#4707)
- Figure.text: Allow passing a sequence of offset to the 'offset' parameter (#4652)
- Implement the frame='none' syntax for no frames (#4404)
- Support passing np.timedelta64 to parameters that expect a sequence (e.g., 'region') (#4358)
- **BREAKING** Raise GMTTypeError exception for unsupported image dtypes (#4673)
- pygmt.grdfilter: Add parameters filter/width/highpass to set the filter and support Pythonic arguments for distance (#4401, #4405)
- pygmt.grdgradient: Add parameters normalize/norm_ambient/norm_amp/norm_sigma/norm_offset for normalization (#4365)
- pygmt.which: Support long-form arguments for the download parameter (#4429)

14.1.4 Deprecations

- Figure.set_panel: Deprecate parameter 'fixedlabel' to 'tag' (Will be removed in v0.23.0) (#4393)
- pygmt.grdclip: Remove parameter 'new' (Deprecated since v0.15.0) (#4343)
- pygmt.grdfill: Remove parameter 'mode'/'no_data' (Deprecated since v0.15.0) (#4345, #4344)
- utils.sequence_join: Remove parameter 'separator' (Deprecated since v0.17.0) (#4346)

14.1.5 Bug Fixes

- Figure.set_panel: Fix the bug when panel is not set (#4628)

14.1.6 Documentation

- Add advanced tutorial for plotting features on a 3-D surface (#4466)
- Add basic tutorial for plotting single-parameter symbols (#3598)
- Update basic tutorial for adjusting the map frame using the Axis and Frame parameter classes (#4596)
- Add gallery example for creating a 3-D bar plot (#4315)
- Add gallery example for plotting the PyGMT logo (#4715)
- Add gallery example for plotting directional roses (#4010)
- Add gallery example for plotting magnetic roses (#4381)
- Add gallery example showing how to fill the area between two curves (#3168)

- Update gallery example for creating a choropleth map using the high-level Figure.choropleth method (#2798)
- Update gallery example for plotting scalebars using the high-level Figure.scalebar method (#4382)
- Add Spilhaus projection to supported GMT projections (#4657)
- Add contributing guides for wrapping a GMT module (#1687)

14.1.7 Maintenance

- Add typos for source code spelling check (#4690)
- **BREAKING**: Figure: Raise GMTParameterError for deprecated parameters U/X/Y/timestamp/xshift/yshift. Previously raise GMTInvalidInput (#4400)
- **BREAKING**: Raise GMTParameterError exception for as_most_one parameters. Previously raise GMTInvalidInput (#4383)
- **BREAKING**: Raise GMTParameterError exception for as_most_one parameters. Previously raise GMTInvalidInput. Part 2 (#4388)
- **BREAKING**: Raise GMTParameterError exception for at_least_one parameters. Previously raise GMTInvalidInput (#4380)
- **BREAKING**: Raise GMTParameterError exception for conflicts_with parameters. Previously raise GMTInvalidInput (#4387)
- **BREAKING**: Raise GMTParameterError exception for missing required parameters. Previously raise GMTInvalidInput (#4374)
- **BREAKING**: Raise GMTValueError if input arrays don't have the same size (#4377)
- Figure.basemap: Raise a warning on deprecated parameters "map_scale", "rose", and "compass" (#4477)
- Figure: Refactor how plotting methods are attached to the Figure class (#4612)
- CI: Bump ghostscript to 10.07.0 (#4476)
- CI: Test NumPy 2.5 in the GMT Tests workflow (#4693)
- Clarify human-only authorship and exclude bot contributors (#4640)
- SPEC 0: Bump minimum supported version to NumPy 2.1, pandas 2.3 and xarray 2024.7 (#4650)
- Update .pre-commit-config.yaml hooks using dependabot (#4435)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.18.0…v0.19.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Xingchen He, Will Schlitzer, Wei Ji Leong, Michael Grund, @Bingtagui404, Max Jones

---

14.2 Release v0.18.0 (2026/01/12)

14.2.1 Highlights

- Eighteenth minor release of PyGMT
- Enhanced GMT embellishment (e.g., scale bar, GMT logo, image) placement and styling with a Pythonic `Position` class and additional controlling parameters

- Five new or updated gallery examples (#4231, #4229, #4228, #4265, #4214)

14.2.2 New Features

- Add the Position class for GMT embellishment placement (#4212)

14.2.3 Enhancements

- AliasSystem: Migrate the 'registration' parameter to the new alias system and support descriptive arguments (#4182)
- pygmt.grdfilter: Let the parameter 'nans' support descriptive arguments (#4310)
- Figure.colorbar: Add position/length/width and more parameters to specify colorbar position and properties (#4048)
- Figure.grdview: Improve parameters plane/facade_fill/facade_pen to set the plane and facade (#4235)
- Figure.histogram: Split parameter bar_width into bar_width and bar_offset (#4316)
- Figure.inset: Add parameters position/width/height to specify inset position and dimensions (#4047)
- Figure.image: Add aliases for -I (invert) and -B (frame) (#4089, #4301)
- Figure.image: Add parameters position/width/height/dpi/replicate to control image position and properties (#4045)
- Figure.legend: Add aliases for -S (scale) and -B (frame) (#4254)
- Figure.legend: Add parameters position/width/height/line_spacing to specify legend position and properties (#4046)
- Figure.logo: Add parameters position/width/height to specify logo position and dimensions (#4014)
- Figure.wiggle: Add parameters position/length/label/label_alignment for the scalebar position and properties (#4049)

14.2.4 Deprecations

- Figure: Remove the deprecated _preprocess method (Deprecated since v0.16.0) (#4146)
- pygmt.grdfill: Deprecate parameters splinefill/constantfill/neighborfill/gridfill to spline_fill/constant_fill/neighbor_fill/grid_fill (Will be removed in v0.20.0) (#4273)
- pygmt.grdlandmask: Deprecate parameters bordervalues/maskvalues to broder_values/mask_values (Will be removed in v0.20.0) (#4281)
- pygmt.grdsample: Deprecate parameter 'translate' to 'toggle' (Will be removed in v0.21.0) (#4187)
- Figure.grdview: Deprecate parameters contourpen/facadepen/meshpen to contour_pen/facade_pen/mesh_pen (Will be removed in v0.20.0) (#4260)
- Figure.grdview: Deprecate parameter drapegrid to drape_grid (Will be removed in v0.20.0) (#4276)
- Figure.histogram: Deprecate parameter barwidth to bar_width (Will be removed in v0.20.0) (#4279)
- Figure.inset: Deprecate parameter margin to clearance (Will be removed in v0.20.0) (#4296)
- Figure.meca: Deprecate parameters compressionfill/extensionfill/labelbox to compression_fill/extension_fill/label_box (Will be removed in v0.20.0) (#4269, #4274)
- pygmt.select: Deprecate parameter gridmask/mask to mask_grid/mask_values (Will be removed in v0.20.0) (#4283, #4288)
- pygmt.surface: Deprecate parameter maxradius to max_radius (Will be removed in v0.20.0) (#4300)

- Figure.velo: Deprecate parameter uncertaintyfill to uncertainty_fill (Will be removed in v0.20.0) (#4272)
- Figure.wiggle: Deprecate parameters fillnegative/fillpositive to negative_fill/positive_fill (Will be removed in v0.20.0) (#4271)
- pygmt.x2sys_cross: Deprecate parameter trackvalues to track_values (Will be removed in v0.20.0) (#4282)

14.2.5 Bug Fixes

- load_earth_relief: Clarify that 03s/01s resolutions are available for IGPP source only (#4192)
- geopandas: Fix UnicodeEncodeError on Windows for geopandas.GeoDataFrame with non-ASCII characters (#4224)
- Figure.grdview: Fix the default z-plane to the grid z minimum when plane is set to True or only facede_fill/facede_pen is set (#4287)

14.2.6 Documentation

- Add example for geopandas point geometry: Use cities in Europe from Natural Earth (#4231)
- Update example for geopandas line geometry: Use Natural Earth dataset, change to Asia (#4229)
- Update example for geopandas polygon geometry: Use the Natural Earth dataset, change to Africa (#4228)
- Advanced tutorial "Legends": Add example for adjusting line spacing (#4265)
- Tutorial "Plotting data points": Add legend for size coding (#4214)

14.2.7 Maintenance

- Add support for Python 3.14 (#4150)
- SPEC 0: Bump minimum supported version to Python 3.12, NumPy 2.0, and xarray 2024.5 (#4248, #4090)
- DOC: Update naming convention in contributors guide: Separate words in parameter names by underscores (#4284)
- fmt_docstrings: Use string template syntax for docstring placeholders and support curly braces in docstrings (#4210)
- Remove dev dependency on "geodatasets" (#4230)
- CI: Add a new workflow to cache dvc cache (#4184)
- CI: Migrate the welcome bot to GitHub Actions (#4149)
- CI: Test NumPy 2.4 in the GMT Tests workflow (#4319)
- Add an issue template for wrapping GMT modules (#4152)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.17.0…v0.18.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Xingchen He, Will Schlitzer

---

14.3 Release v0.17.0 (2025/10/03)

14.3.1 Highlights

- **Seventeenth minor release of PyGMT**
- Add support for GMT 6.6.0, while dropping support for GMT 6.4.0 (#4017)
- Implement the new alias system towards a Pythonic interface (#3986, #4000, #3993)
- GMTDataArrayAccessor: Support applying grid operations on the current xarray.DataArray object (#3854)

14.3.2 New Features

- Add the Box class for specifying the box of GMT embellishments (#3995)
- Add the Pattern class for specifying bit and hachure patterns to fill symbols and polygons (#4020)

14.3.3 Enhancements

- Figure.solar: Support terminator datetime with timezone (#4112)
- pygmt.show_versions: Report GDAL version by calling the GDAL C function (#4104)
- pygmt.binstats: Let the parameter 'statistic' support descriptive arguments (#3012)
- Figure.logo: Let the 'style' parameter support descriptive arguments (#4074)
- Let the 'verbose' parameter support descriptive arguments (#4039)
- sequence_join: Correctly join datetime-like values in pandas.DataFrame/xarray.DataArray objects (#4097)

14.3.4 Deprecations

- Drop support for GMT 6.4.0 (#4127)
- **BREAKING** Raise GMTValueError exception for invalid values. Previously raise GMTInvalidInput (#3985, #3998)
- **BREAKING** Raise GMTTypeError exception for invalid types. Previously raise GMTInvalidInput (#3999)
- **BREAKING** Figure.grdimage: Raise NotImplementedError instead of GMTInvalidInput for -A option (#4062)
- sequence_join: Deprecate parameter 'separator', use 'sep' instead. [Will be removed in v0.19.0] (#4033)

14.3.5 Bug Fixes

- pygmt.x2sys_cross: Fix IndexError to allow empty DataFrame outputs (#4060)
- is_nonstr_iter: 0-D array is now recognized as non-iterable (#4004)
- Fix the bug when passing a sequence that contains True to a parameter (#3982)

14.3.6 Documentation

- Figure.colorbar: Add note regarding automatic fontsize scaling (#4126)
- Add gallery example to show text formatting (#3987)
- Add gallery example for using EPSG codes (#3973)
- Add Technical Reference page for justification codes (#4028)
- Add Technical Reference page for text formatting (#3976)

14.3.7 Maintenance

- **BREAKING** Figure.text: Raise GMTTypeError if text is a sequence when position is given (#4065)
- Switch from pre-commit to prek (#4082)
- SPEC 0: Bump minimum supported version to pandas 2.2 and xarray 2023.10 (#4092, #4091)
- Remove the support of non-aliased aliases in use_alias/fmt_docstrings (#4042)
- CI: Bump to macOS 26 in the GMT Dev Tests workflow (#4084)
- CI: Bump to macOS 14 in the GMT Legacy Tests workflow (#3996)
- CI: Access DagsHub repository via an access token (#4078)
- CI: Bump to Ghostscript 10.06.0 (#4125)
- CI: Bump to GMT 6.6.0 in CI (#4018)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.16.0…v0.17.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund

---

14.4 Release v0.16.0 (2025/06/18)

14.4.1 Highlights

- **Sixteenth minor release of PyGMT**
- Implement gmt xarray BackendEntrypoint to load raster files via GMT (#3919, #3932)
- Remove the dependency of netCDF4 (#3643)

14.4.2 Enhancements

- pygmt.grdclip: Parameters 'between' and 'replace' accept a 2-D sequence (#3883)
- pygmt.grdlandmask: Improve the checking of parameters 'maskvalues'/'bordervalues' (#3963)
- Figure.coast/pygmt.select/pygmt.grdlandmask: Use long names ("crude"/"low"/"intermediate"/"high"/"full") for the 'resolution' parameter (#3013)
- Add a helper function sequence_join to join a sequence by a separator (#3961)
- Improve decorators @use_alias/@fmt_docstrings to list non-aliased aliases (#3965)
- Store first tile as source encoding for tiled grids (#3950)

14.4.3 Deprecations

- Figure: Deprecate the private _preprocess method (will be removed in v0.18.0) (#3948)
- Deprecate pygmt.io.load_dataarray, use xarray.load_dataarray instead (#3922)
- Session.virtualfile_in: Deprecate the parameter 'extra_arrays'. Prepare and pass a dictionary of arrays instead (will be removed in v0.20.0) (#3823)
- Session.virtualfile_in: Deprecate parameter 'required_z'. Use 'mincols' instead (will be removed in v0.20.0) (#3369)

- Session.virtualfile_in: Deprecate parameter 'required_data' to 'required' (will be removed in v0.20.0) (#3931)
- Session.virtualfile_from_vectors: Remove the deprecated '*args' parameter [deprecated since v0.14.0] (#3893)
- SPEC 0: Bump minimum supported versions to NumPy 1.26, xarray>=2023.07 and pandas>=2.1 (#3916, #3918, #3895)
- Bump minimum supported version to pyarrow>=16, packaging>=24.2, geopandas>=1.0, and contextily>=1.5 (#3917, #3933, #3908, #3912)

14.4.4 Bug Fixes

- pygmt.select: Parameter 'mask' accepts a sequence of strings (#3956)

14.4.5 Documentation

- GMTDataArrayAccessor: Improve the documentation to clarify how the accessor works (#3955)

14.4.6 Maintenance

- Update the AUTHORSHIP and Maintainers' Guides to reflect co-author contributions (#3904)
- Set registration and gtype properly as enums on gmt accessor init (#3942)
- Set license and license-files key in project metadata to follow PEP639 (#3699)
- CI: Test NumPy 2.3 in the GMT Tests workflow (#3968)
- Bump to Ruff>=0.12.0 and fix violations (#3980)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.15.0…v0.16.0

Contributors: Dongdong Tian, Wei Ji Leong, Yvonne Fröhlich, Michael Grund, Federico Esteban

---

14.5 Release v0.15.0 (2025/03/31)

14.5.1 Highlights

- **Fifteenth minor release of PyGMT**
- One new gallery example and two new tutorials
- Figure.shift_origin: Support shifting origins temporarily when used as a context manager (#2509)
- Documentation as HTML ZIP archive and in PDF format for offline reference

14.5.2 Enhancements

- **BREAKING** Support typesetting apostrophe (') and backtick (`) (#3105)
- **BREAKING** pygmt.grdcut: Refactor to store output in virtualfiles for grids (#3115)
- GMTDataArrayAccessor: Support passing values using enums GridRegistration and GridType for grid registration and type (#3696)
- pygmt.grdfill: Add new parameters 'constantfill'/'gridfill'/'neighborfill'/'splinefill' for filling holes (#3855)

- pygmt.grdfill: Add new parameter 'inquire' to inquire the bounds of holes (#3880)
- pygmt.grdfill: Add alias 'coltypes' (-f) (#3869)

14.5.3 Deprecations

- pygmt.grdfill: Deprecate parameter 'no_data' to 'hole' (remove in v0.19.0) (#3852)
- pygmt.grdfill: Deprecate parameter 'mode', use parameters 'constantfill'/'gridfill'/'neighborfill'/'splinefill' instead (remove in v0.19.0) (#3855)
- pygmt.grdclip: Deprecate parameter 'new' to 'replace' (remove in v0.19.0) (#3884)
- clib.Session: Remove deprecated open_virtual_file method, use open_virtualfile instead (Deprecated since v0.11.0) (#3738)
- clib.Session: Remove deprecated virtualfile_from_data method, use virtualfile_in instead (Deprecated since v0.13.0) (#3739)

14.5.4 Documentation

- Add an advanced tutorial for plotting focal mechanisms (beachballs) (#2550)
- Add an advanced tutorial for creating legends (#3594)
- Add a gallery example for Figure.hlines and Figure.vlines (#3755)

14.5.5 Maintenance

- Use the 'release-branch-semver' version scheme for setuptools_scm (#3828)
- Rename _GMT_DATASET.to_dataframe to .to_pandas and _GMT_GRID.to_dataarray/_GMT_IMAGE.to_dataarray to .to_xarray (#3798)
- Bump to Ruff 0.9.0, apply Ruff 2025 style, and ignore A005 (stdlib-module-shadowing) violations (#3763)
- Use well-known labels in project URLs following PEP753 (#3743)
- clib.conversion: Remove the unused array_to_datetime function (#3507)
- CI: Test on Linux arm64 runners (#3778)
- CI: Build PDF documentation using tectonic (#3765)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.14.0…v0.15.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund

---

14.6 Release v0.14.2 (2025/02/15)

14.6.1 Bug Fixes

- **Patch release fixing a critical bug introduced in PyGMT v0.14.1**
- Fix the bug for passing text strings with numeric values (#3804)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.14.1…v0.14.2

Contributors: Dongdong Tian

14.7 Release v0.14.1 (2025/02/01)

DOI 10.5281/zenodo.3782862

14.7.1 Highlights

- **Patch release fixing critical bugs in PyGMT v0.14.0**
- Fix the bug of converting Python sequence of datetime-like objects (#3760)

14.7.2 Maintenance

- CI: Separate jobs for publishing to TestPyPI and PyPI (#3742)
- clib.conversion._to_numpy: Add tests for Python sequence of datetime-like objects (#3758)
- Fix an image in README.md (broken on PyPI) and rewrap to 88 characters (#3740)
- Fix the dataset link in the RGB image gallery example (#3781)
- Update License year to 2025 (#3737)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.14.0…v0.14.1

Contributors: Dongdong Tian, Wei Ji Leong

14.8 Release v0.14.0 (2024/12/31)

DOI 10.5281/zenodo.3782862

14.8.1 Highlights

- **Fourteenth minor release of PyGMT**
- Bump minimum supported version to GMT>=6.4.0 (#3450)
- Two new plotting methods and six new functions to access more GMT remote datasets
- PyArrow as an optional dependency and improved support of PyArrow data types (#3592)

14.8.2 New Features

- Add Figure.hlines for plotting horizontal lines (#923)
- Add Figure.vlines for plotting vertical lines (#3726)
- Add load_black_marble to load “Black Marble” dataset (#3469)
- Add load_blue_marble to load “Blue Marble” dataset (#2235)
- Add load_earth_deflection to load “IGPP Earth east-west and north-south deflection” datasets (#3728)

- Add load_earth_dist to load "GSHHG Earth distance to shoreline" dataset (#3706)
- Add load_earth_mean_dynamic_topography to load "CNES Earth Mean Dynamic Topography" dataset (#3718)
- Add load_earth_mean_sea_surface to load "CNES Earth Mean Sea Surface" dataset (#3717)
- load_earth_free_air_anomaly: Add "uncertainty" parameter to load the "IGPP Earth free-air anomaly uncertainty" dataset (#3727)

14.8.3 Enhancements

- Figure.plot: Add the "symbol" parameter to support plotting data points with varying symbols (#1117)
- Figure.plot3d: Add the "symbol" parameter to support plotting data points with varying symbols (#3559)
- Figure.legend: Support passing a StringIO object as the legend specification (#3438)
- load_tile_map: Add parameter "crs" to set the CRS of the returned dataarray (#3554)
- PyArrow: Support pyarrow arrays with string/large_string/string_view types (#3619)
- Support 1-D/2-D numpy arrays with longlong and ulonglong dtype (#3566)
- GMT_IMAGE: Implement the to_dataarray method for 3-band images (#3128)
- Ensure non-ASCII characters are typeset correctly even if PS_CHAR_ENCODING is not "ISOLatin1+" (#3611)
- Add enums GridRegistration and GridType for grid registration and type (#3693)

14.8.4 Deprecations

- SPEC 0: Bump minimum supported versions to Python 3.11, NumPy 1.25, pandas>=2.0 and xarray>=2023.04 (#3460, #3606, #3697)
- clib.Session.virtualfile_from_vectors: Now takes a sequence of vectors as its single argument (Passing multiple arguments will be unsupported in v0.16.0) (#3522)
- Remove the deprecated build_arg_string function (deprecated since v0.12.0) (#3427)
- Figure.grdcontour: Remove the deprecated syntax for the 'annotation' parameter (deprecated since v0.12.0) (#3428)

14.8.5 Bug Fixes

- launch_external_viewer: Use full path when opening the file in a web browser (#3647)
- PyArrow: Map date32[day]/date64[ms] dtypes in pandas objects to np.datetime64 with correct date/time units (#3617)
- clib.session: Add the GMT_SESSION_NOGDALCLOSE flag to keep GDAL open (#3672)
- Set the "Conventions" attribute to "CF-1.7" for netCDF grids only (#3463)
- Fix the conversion error for pandas.Series with missing values in pandas<=2.1 (#3505, #3596)
- GeoPandas: Explicitly convert columns with overflow integers to avoid OverflowError with fiona 1.10 (#3455)
- Figure.plot/Figure.plot3d: Improve the check of the "style" parameter for "v" or "V" (#3603)
- Correctly reserve the grid data dtype by converting ctypes array to numpy array with np.ctypeslib.as_array (#3446)
- **Breaking**: Figure.text: Fix typesetting of integers when mixed with floating-point values (#3493)

14.8.6 Documentation

- Add basic tutorial "Plotting polygons" (#3593)
- Update the gallery example for plotting lines with LineString/MultiLineString geometry (#3711)
- Add the PyGMT ecosystem page (#3475)
- Document the support policy for optional packages (#3616)
- Document the environment variables that can affect the behavior of PyGMT (#3432)
- Document the built-in patterns in the Technical Reference section (#3466)
- Document Continuous Benchmarking in Maintainers Guides (#3631)
- Add instructions for installing optional dependencies (#3506)
- Update "PyData Ecosystem" to "Scientific Python Ecosystem" (#3447)
- Figure.savefig: Clarify that the "transparent" parameter also works for the PNG file associated with the KML format (#3579)
- Add the PyGMT talk at AGU24 to the "Overview" section (#3685)
- Add the GMT/PyGMT pre-conference workshop at AGU24 to the "External resources" section (#3689)
- Add TODO comments in the maintainers guides and update the release checklist (#3724)

14.8.7 Maintenance

- **Breaking**: data_kind: data is None and required now returns the "empty" kind (#3482)
- **Breaking**: data_kind: Now "matrix" represents a 2-D numpy array and unrecognized data types fall back to "vectors" (#3351)
- Add Support for Python 3.13 (#3490)
- Add the Session.virtualfile_from_stringio method to allow StringIO input for certain functions/methods (#3326)
- Add "geodatasets" as a dependency for docs and update the choropleth example (#3719)
- PyArrow: Check compatibility of pyarrow.array with string type (#2933)
- Rename sphinx-gallery's README.txt to GALLERY_HEADER.rst and require Sphinx-Gallery>=0.17.0 (#3348)
- clib.conversion: Remove the as_c_contiguous function and use np.ascontiguousarray instead (#3492)
- Use TODO comments to track deprecations and workarounds (#3722)
- Move Figure.psconvert into a separate file (#3553)
- Improve the data type checking for 2-D arrays passed to the GMT C API (#3563)
- Enable Ruff's TD (flake8-todos), COM (flake8-commas), TRY (tryceratops), and EM (flake8-errmsg) rules (#3723, #3531, #3665, #3661)
- CI: Install pyarrow-core instead of pyarrow from conda-forge (#3698)
- CI: Ensure no hyphens in Python file and directory names in the "Style Checks" workflow (#3703)
- Bump to Ruff>=0.8.0 and rename rule TCH to TC (#3662)
- Bump to Ghostscript 10.04.0 (#3443)
- Add enums GridFormat for GMT grid format ID (#3449)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.13.0…v0.14.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Will Schlitzer, Jiayuan Yao

14.9 Release v0.13.0 (2024/09/05)

DOI 10.5281/zenodo.3782862

14.9.1 Highlights

- **Thirteenth minor release of PyGMT**
- Add new documentation section "Technical Reference" and document the supported encodings and fonts
- Tutorial for "Draping a dataset on top of a topographic surface" (#3316)
- Tutorial for "Typesetting non-ASCII characters" (#3389)

14.9.2 New Features

- Wrap the GMT API function GMT_Read_Data to read data into GMT data containers (#3324)
- Wrap GMT's standard data type GMT_IMAGE for images (#3338)

14.9.3 Enhancements

- **Breaking**: pygmt.x2sys_cross: Refactor to use virtualfiles for output tables (#3182)
- pygmt.show_versions: Warn about incompatible Ghostscript versions (#3244)
- pygmt.show_versions: Show GDAL version (#3364, #3376)
- pygmt.datasets.load_*: Add autocompletion support for the 'resolution' parameter (#3260)
- clib.Session: Refactor the `__getitem__` special method to avoid calling API function GMT_Get_Enum repeatedly (#3261)
- clib: Refactor to avoid checking GMT version repeatedly and only check once when loading the GMT library (#3254)
- Support non-ASCII characters in ISO-8859-x charsets (#3310)
- Refactor to improve the user experience with non-ASCII characters (#3206)

14.9.4 Deprecations

- SPEC 0: Bump minimum supported version to xarray 2022.09 (#3372)
- SPEC 0: Bump minimum supported version to NumPy 1.24 (#3286)
- clib: Deprecate API function 'Session.virtualfile_from_data', use 'Session.virtualfile_in' instead (will be removed in v0.15.0) (#3225)
- Remove the unused pygmt.print_clib_info function (#3257)
- Figure.timestamp: Remove deprecated parameter 'justification', use 'justify' instead (deprecated since v0.11.0) (#3222)

14.9.5 Bug Fixes

- pygmt.set_display: Fix the bug that `method=None` doesn't reset to the default display method (#3396)
- load_tile_map: Register the rio accessor by importing rioxarray, so the returned raster has CRS (#3323)
- load_tile_map: Fix the raster band indexing, should start from 1 (#3322)
- load_tile_map: Replace deprecated rio.set_crs with rio.write_crs (#3321)
- PYGMT_USE_EXTERNAL_DISPLAY should NOT disable image display in Jupyter notebook environment (#3418)

14.9.6 Documentation

- External Resources: Add tutorial in Portuguese and using PyGMT in Google Colab (#3360)
- Remove the non-official GMT wrappers from README (#3413)
- Give recommendations about GMT-Ghostscript incompatibility and the testing example (#3249)
- Document the supported 35 standard Postscript fonts in the Technical Reference section (#3378)
- Add an offboarding access checklist for maintainers (#3411)
- Update the onboarding access checklist in Maintainers Guides (#3404)
- Add sphinx extension myst-nb to enable writing executable Markdown notebooks (#3379)

14.9.7 Maintenance

- pygmt.grd2cpt & pygmt.makecpt: Simplify the logic for dealing with CPT output (#3334)
- geopandas: Use io.StringIO to read geojson data and handle compatibility with geopandas v0.x and v1.x (#3247)
- Simplify the "Minimum Supported Versions" page using MyST customized URL schemes (#3383)
- build_arg_list: Raise an exception if an invalid output file name is given (#3336)
- sphinx-gallery: Temporarily pin to < 0.17.0 (#3350)
- Run pytest with `--color=yes` to force GitHub Actions logs to have color (#3330)
- Patch the callback print function to suppress the UnicodeDecodeError (#3367)
- Move Will from Active Maintainers to Distinguished Contributors (#3388)
- Enable Ruff's unspecified-encoding (PLW1514) rule and fix violations (#3319)
- Enable Ruff's literal-membership (PLR6201) rule and fix violations (#3317)
- Determine the minimum required versions of dependencies from package metadata for docs (#3380)
- CI: Use OIDC token for codecov uploading (#3163)
- CI: Test NumPy 2.1 in the GMT Tests workflow (#3401)
- CI: Set GMT_ENABLE_OPENMP to TRUE to enable OpenMP support on macOS (#3266)
- CI: Fix the name of the 'build' package to 'python-build' on conda-forge (#3408)
- CI: Bump to ubuntu-24.04 and mambaforge-23.11 in ReadTheDocs (#3296)
- CI: Build GMT dev source code with OpenMP enabled on Linux and GThreads enabled on Linux/macOS (#3011)
- CI: Add pytest plugins pytest-xdist and pytest-rerunfailures (#3193, #3267)

- Add pre-commit config with pre-commit-hooks and enable pre-commit.ci to update hooks quarterly (#3283, #3414)
- Add a test to make sure PyGMT works with paths that contain non-ASCII characters (#3280)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.12.0…v0.13.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Andre L. Belem

---

14.10 Release v0.12.0 (2024/05/01)

14.10.1 Highlights

- **Twelfth minor release of PyGMT**
- Almost all module wrappers (with a few exceptions) now use in-memory GMT *virtual files* instead of intermediate temporary files to improve performance (#2730)
- Almost all module wrappers (with a few exceptions) now have consistent behavior for table-like output (#1318)
- Adopt SPEC 0 policy for minimum supported versions of GMT, Python, and other core dependencies

14.10.2 Enhancements

- **Breaking**: (Unneeded) extra double quotes around text strings (containing whitespaces) are now considered as part of the text string (#3132, #3457)
- pygmt.project: Add 'output_type' parameter for output in pandas/numpy/file formats (#3110)
- pygmt.grdtrack: Add 'output_type' parameter for output in pandas/numpy/file formats (#3106)
- pygmt.blockm*: Add 'output_type' parameter for output in pandas/numpy/file formats (#3103)
- Figure.grdcontour: Adjust processing arguments passed to "annotation" and "interval" parameters (#3116)
- Figure.contour: Adjust processing arguments passed to "annotation" and "levels" parameters (#2706)
- clib: Wrap the GMT API function GMT_Read_VirtualFile (#2993)
- clib: Add virtualfile_to_dataset method for converting virtualfile to a dataset (#3083, #3140, #3157, #3117)
- clib: Add the virtualfile_out method for creating output virtualfile (#3057)
- Wrap GMT_Inquire_VirtualFile to get the family of virtualfiles (#3152)
- Wrap GMT's standard data type GMT_GRID for grids (#2398)
- Wrap GMT's standard data type GMT_DATASET for table inputs (#2729, #3131, #3174)
- Wrap GMT's data structure GMT_GRID_HEADER for grid/image/cube headers (#3127, #3134)
- Session.call_module: Support passing a list of argument strings (#3139)
- Refactor the _load_remote_dataset function to load tiled and non-tiled grids in a consistent way (#3120)
- Refactor all wrappers to pass an argument list to Session.call_module (#3132)
- Add function build_arg_list for building arguments list from keyword dictionaries (#3149)
- Support left/right single quotation marks in text and arguments (#3192)

- non_ascii_to_octal: Return the input string if it only contains printable ASCII characters (#3199)

14.10.3 Deprecations

- SPEC 0: Set minimum supported versions to Python>=3.10, pandas>=1.5 and xarray>=2022.06 (#3043, #3039, #3151)
- Figure.plot/plot3d/rose: Remove deprecated parameter "color", use "fill" instead (deprecated since v0.8.0) (#3032)
- Figure.velo: Remove deprecated parameters "color"/"uncertaintycolor", use "fill"/"uncertaintyfill" instead (deprecated since v0.8.0) (#3034)
- Figure.wiggle: Remove deprecated parameter "color", use "fillpositive"/"fillnegative" instead (deprecated since v0.8.0) (#3035)
- Figure.grdimage: Remove deprecated parameter "bit_color", use "bitcolor" instead (deprecated since v0.8.0) (#3036)
- Figure: Remove deprecated "xshift" ("X") and "yshift" ("Y") parameters, use "Figure.shift_origin" instead (deprecated since v0.8.0) (#3044)
- Figure: Remove deprecated "timestamp" ("U") parameter, use "Figure.timestamp" instead (deprecated since v0.9.0) (#3045)
- clib: Rename the "virtualfile_from_data" method to "virtualfile_in" (#3068)
- Deprecate the "build_arg_string" function, use build_arg_list instead (deprecated since v0.12.0, will be removed in v0.14.0) (#3184)
- Deprecate the "sequence_plus" converter, only used for the "annotation" parameter of Figure.grdcontour (deprecated since v0.12.0, will be removed in v0.14.0) (#3207)
- Figure.grdcontour: Deprecate parameter "interval" to "levels" (FutureWarning since v0.12.0, will be removed in v0.16.0) (#3209)

14.10.4 Documentation

- External Resources: Add repository "gmt-pygmt-plotting" (#3213)
- Gallery example "Custom symbols": Mention own custom symbols (#3186)
- Intro "04 Table inputs": Document that a list of file names, pathlib.Path objects, URLs, or remote files is supported (3214)
- Tutorial "Plotting text": Rewrite to improve structure, explain more parameters, show list input (#2760)

14.10.5 Maintenance

- pygmt.filter1d: Improve performance by storing output in virtual files (#3085)
- pygmt.grdvolume: Refactor to store output in virtual files instead of temporary files (#3102)
- pygmt.grdhisteq.compute_bins: Refactor to store output in virtual files instead of temporary files (#3109)
- pygmt.grd2xyz: Improve performance by storing output in virtual files (#3097)
- pygmt.select: Improve performance by storing output in virtual files (#3108)
- pygmt.triangulate.delaunay_triples: Improve performance by storing output in virtual files (#3107)
- pygmt.which: Refactor to get rid of temporary files (#3148)
- Use consistent names (vintbl and vingrd) for input virtual files (#3082)
- Add sequence_to_ctypes_array to convert a sequence to a ctypes array (#3136)

- Add strings_to_ctypes_array to convert a sequence of strings into a ctypes array (#3137)
- Figure.psconvert: Ignore the unrecognized "metadata" parameter added by pytest-mpl v0.17.0 (#3054)
- Remote Datasets: Adjust attributes - remove "title", use default of "name" and "long_name", introduce "description" (#3048)
- Adopt SPEC 0 policy and drop NEP 29 policy (#3037)
- Document the support policy for minimum required GMT versions (#3070)
- Bump to Ghostscript 10.03.0 (#3112)
- Bump to Ruff 0.3.0 (#3081)
- Enable Ruff's PTH (flake8-use-pathlib) rules and fix violations (#3129)
- Change the dev dependency "matplotlib" to "matplotlib-base" to reduce environment size (#3158)
- Migrate from os.path to pathlib (#3119)
- CI: Use "gh release" to upload assets to release (#3187)
- CI: Consistently use github.token instead of secrets.GITHUB_TOKEN (#3189)
- CI: Configure workflows to run on "workflow_dispatch" event (#3133)
- Switch to official GitHub action for managing app tokens (#3165)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.11.0…v0.12.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Michael Grund, Wei Ji Leong

---

14.11 Release v0.11.0 (2024/02/01)

14.11.1 Highlights

- **Eleventh minor release of PyGMT**
- Tutorial for table inputs (#2722) and gallery example for choropleth map (#2796)
- Easy access to planetary relief datasets (Mercury, Venus, Moon, Mars, Pluto) (#3028, #2906, #2674, #2847, #3027)
- Faster PyGMT by ~0.1 seconds for each module call (#2930)

14.11.2 New Features

- Support timedelta64 dtype as input (#2884)
- Figure.text: Support passing in a list of angle/font/justify values (#2720)
- Figure.savefig: Support saving figures in PPM (.ppm) format (#2771)
- Figure.savefig: Support generating GeoTIFF file (with extension '.tiff') (#2698)
- Figure.savefig: Add the 'worldfile' parameter to write a companion world file for raster images (#2766)

14.11.3 Enhancements

- geopandas: Correctly handle columns with integer values bigger than the largest 32-bit integer (#2841)
- pyarrow: Support date32[day] and date64[ms] dtypes in pandas objects (#2845)
- datasets.load_tile_map and Figure.tilemap: Add "zoom_adjust" parameter (#2934)
- grdlandmask: Add common alias "cores" for "x" (#2944)
- Figure.coast: Add alias "box" for "-F" (#2823)
- Improve the error messages for unsupported numpy dtypes (#2856)
- Set GMT_SESSION_NAME to a unique name on Windows for multiprocessing support (#2938)
- Figure.savefig: Support .jpeg as JPEG image extension (#2691)
- Figure.savefig: Support uppercase file extensions (e.g., PNG, PDF) (#2697)

14.11.4 Deprecations

- Rename API function Session.open_virtual_file to Session.open_virtualfile (remove in v0.15.0) (#2996)
- NEP29: Set minimum required version to NumPy 1.23+ (#2991)
- Figure.timestamp: Deprecate parameter 'justification' to 'justify' (remove in v0.13.0) (#3002)
- Figure.grdimage: Remove the unsupported 'img_out'/'A' parameter (#2907)

14.11.5 Bug Fixes

- pygmt.which: Fix the bug when passing multiple files (#2726)
- pygmt.filter1d: Fix the bug that the first line is read as headers (#2780)
- clib: Fix the bug when passing multiple columns of strings with variable lengths to the GMT C API (#2719)
- Let kwargs_to_strings work with default values and positional arguments (#2826)
- Figure.meca: Fix typo pricipal_axis -> principal_axis (#2940)

14.11.6 Documentation

- Add gallery example "Scale bar" (#2822)
- Add gallery example for plotting connection lines ("connection" parameter of Figure.plot) (#2999)
- Add gallery example showing how to adjust line segment ends (caps and joints) (#3015)
- Gallery example "Legend": Update regarding input data and multi-column legends (#2762)
- Add the Japanese "PyGMT-HOWTO" tutorial to "External Resources" (#2743)
- Figure.plot: Update docstring for "connection" parameter to GMT 6.5 (#2994)
- Use consistent description for the "outgrid" parameter (#2874)
- Improve the onboarding access checklist for contributors, maintainers and administrators (#2656)
- Recommend Miniforge instead of Mambaforge (#2833)

14.11.7 Maintenance

- pyarrow: Check compatibility of pyarrow-backed pandas objects with numeric dtypes (#2774)
- Switch away from Stamen basemaps (#2717)
- Add the "validate_output_table_type" function to check the "output_type" parameter (#2772)
- Create "skip_if_no" helper function to skip tests when missing a package (#2883)
- Fix "fixture_xr_image" to open "earth_day_01d" directly with rioxarray (#2963)
- Improve the way to import optional modules (#2809)
- Move variable `__gmt_version__` to pygmt.clib to avoid cyclic-import errors (#2713)
- Refactor the internal "_load_remote_dataset function" to simplify datasets' definitions (#2917)
- Benchmark grdsample, grdfilter and sph2grd with fixed cores (#2945)
- Setup Continuous Benchmarking workflow with pytest-codspeed (#2908)
- Update output shape and mean values from some x2sys_cross tests (#2986)
- Bump the GMT version in CI to 6.5.0 (#2962)
- Bump Ghostscript to 10.02.1 (#2694)
- Add support for Python 3.12 (#2711)
- NEP29: Test PyGMT on NumPy 1.26 (#2692)
- CI: Trigger the cache_data workflow in PRs if cache files are added/deleted/updated (#2939)
- CI: Test GMT dev version on Windows by building from source (#2773)
- CI: Set cache-downloads to false to speedup the "Setup Micromamba" step (#2946)
- CI: Run certain GitHub Actions workflows on official repo only (#2951)
- CI: Run benchmarks if PR is labeled with "run/benchmark" (#2958)
- CI: Run "GMT Dev Tests" if PR is labeled with "run/test-gmt-dev" (#2960)
- CI: Require at least one code block separator for example files in the Style Checks workflow (#2810)
- Add Mypy for static type checking (#2808)
- TYP: Improve the doc style for type hints (#2813)
- TYP: Add type hints support for pygmt.datasets.load_sample_data (#2859)
- TYP: Add type hints for the "registration" parameter in pygmt.datasets.load_* functions (#2867)
- TYP: Add type hints for "data_source" in load_earth_relief and load_earth_magnetic_anomaly functions (#2849)
- TYP: Add type hints for parameters of Figure.timestamp(#2890)
- TYP: Add type hints for "terminator" in Figure.solar and simplify codes (#2881)
- TYP: Add type hints for parameters of Figure.shift_origin, improve docstrings, and add inline examples (#2879)
- doc: Convert the installation guides source code from ReST to Markdown (#2992)
- doc: Convert overview source code from ReST to Markdown (#2953)
- doc: Move compatibility table from README to separate file (#2862)
- Enable the PDF format documentation in the ReadTheDocs site (#2876)
- Add the full changelog link to the release drafter template (#2838)

- Release Drafter: Automatically replace GitHub handles with names and links (#2777)
- Exclude CODE_OF_CONDUCT.md, AUTHORSHIP.md and pygmt/tests directory from distributions (#2957)
- Add Zenodo's GMT community to the maintainer's onboarding list (#2761)
- Use Ruff to lint and format codes, and remove flakeheaven/isort/black/blackdoc (#2741)
- Use codespell to check common misspellings (#2673)
- Use "# %%" as code block separators in examples (#2662)

Full Changelog: https://github.com/GenericMappingTools/pygmt/compare/v0.10.0…v0.11.0

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Max Jones

---

14.12 Release v0.10.0 (2023/09/02)

14.12.1 Highlights

- **Tenth minor release of PyGMT**
- Support non-ASCII characters in Figure.text (#2638) and other method arguments (#2584)
- Three new tutorials and seven new gallery examples

14.12.2 Enhancements

- Figure.colorbar: Add alias for "Q" (#2608)
- Figure.grdimage: Allow passing RGB xarray.DataArray images (#2590)
- Figure.image: Add alias for "G" (#2615)
- Figure.meca: Add aliases for "L", "T", and "Fr" (#2546)
- clib.Session: Wrap the GMT_Get_Common API function (#2500)
- pygmt.grdfill: Add alias for "N" (#2618)
- pygmt.select: Add aliases for "C", "F", and "L" (#2466)
- pygmt.show_versions: Show versions of IPython and rioxarray (#2492)
- Better handling of optional virtual files (e.g., shading in Figure.grdimage) (#2493)

14.12.3 Deprecations

- Remove the unused pygmt.test() function (#2652)
- Figure.grdimage: Deprecate parameter "bit_color" to "bitcolor" (remove in v0.12.0) (#2635)
- Figure.text: Remove the deprecated "incols" parameter (deprecated since v0.8.0) (#2473)
- NEP29: Set minimum required version to Python 3.9+ (#2487)
- NEP29: Set minimum required version to NumPy 1.22+ (#2586)

14.12.4 Bug Fixes

- load_earth_mask: Keep data's encoding to correctly infer data's registration and gtype information (#2632)
- Geopandas integration: Mapping int/int64 to int32 for OGR_GMT format (#2592)
- Figure.meca: Let the "scale" parameter accept int/float/str values (#2566)
- Figure.meca: Fix beachball offsetting for ndarray input (requires GMT>6.4.0) (#2576)

14.12.5 Documentation

- Document the default CPT for GMT remote datasets (#2573)
- Add tutorial to explain naming of PyGMT figure elements (#2383)
- Add tutorial to show interactive data visualization via `panel` (#2498)
- Add tutorial for cartesian histograms (#2445)
- Add gallery example to show usage of dcw parameter in Figure.coast (#2428)
- Add gallery example to show usage of tile maps (#2585)
- Add gallery example showing how to build an envelope around a curve (#2587)
- Add gallery example for plotting an RGB image from an xarray.DataArray (#2641)
- Add gallery example "Quoted lines" (`style="q"`) (#2563)
- Add gallery example "Decorated lines" (`style="~"`) (#2564)
- Add gallery example "Cross-section along a transect" (#2515)

14.12.6 Maintenance

- Use substitutions to show the minimum required Python and GMT versions dynamically in installation guides (#2488)
- Use np.asarray to convert a 1-D array to datetime type in array_to_datetime (#2481)
- Use consistent docstrings for test files (#2578)
- Use concurrency to cancel previous runs (#2589)
- Set date_format to ISO8601 to silence pandas 2.0 UserWarning on read_csv (#2569)
- Remove dummy_context and use contextlib.nullcontext instead (#2491)
- NEP29: Test PyGMT on NumPy 1.25 (#2581)
- Fix tests for Aug 2023 updated remote datasets (#2636)
- Figure.meca: Refactor the two tests for offsetting beachballs (#2572)
- Figure.meca: Refactor tests for plotting multiple focal mechanisms (#2565)
- Figure.meca: Refactor tests for plotting a single focal mechanism (#2533)
- Figure.meca: Add a test for passing event names via pandas.DataFrame (#2582)
- Exclude bots from contributors in release drafter (#2484)
- Exclude DVC files from source/binary distributions (#2634)
- CI: Use mamba-org/provision-with-micromamba to setup micromamba (#2435)
- CI: Migrate provision-with-micromamba to setup-micromamba (#2536)

- CI: Run dev tests on scientific Python nightly wheels (#2612)
- CI: Remove the deprecated cml-publish command from the dvc-diff workflow (#2559)
- CI: Fix and simplify the dvc-diff workflow (#2549)
- CI: Add the "Doctests" workflow to run doctests weekly (#2456)
- CI: Add detailed descriptions in the workflow files and update maintainer guides (#2496)
- Add private function _validate_data_input to validate input data (#2595)

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Jing-Hui Tong, Max Jones

---

14.13 Release v0.9.0 (2023/03/31)

DOI 10.5281/zenodo.3782862

14.13.1 Highlights

- **Ninth minor release of PyGMT**
- Add Figure.tilemap to plot XYZ tile maps (#2394)
- Add function to load raster tile maps using contextily (#2125)
- Eleven new/updated gallery and inline examples

14.13.2 New Features

- Add load_earth_mask function for GSHHG Global Earth Mask dataset (#2310)
- Add Figure.timestamp to plot the GMT timestamp logo (#2208, #2425)

14.13.3 Enhancements

- pygmt.surface: Add aliases for "C", "L", "M", and "T" (#2321)
- Figure.meca: Add aliases for "C", "E", "G", and "W" (#2345)
- Figure.colorbar: Add aliases for "L" and "Z" (#2357)

14.13.4 Deprecations

- NEP29: Set minimum required version to NumPy 1.21+ (#2389)
- Recommend Figure.timestamp and remove timestamp (U) alias from all plotting methods (#2135)
- Remove the deprecated load_fractures_compilation function (deprecated since v0.6.0) (#2303)
- Remove the deprecated load_hotspots function (deprecated since v0.6.0) (#2309)
- Remove the deprecated load_japan_quakes function (deprecated since v0.6.0) (#2301)
- Remove the deprecated load_mars_shape function (deprecated since v0.6.0) (#2304)
- Remove the deprecated load_ocean_ridge_points function (deprecated since v0.6.0) (#2308)
- Remove the deprecated load_sample_bathymetry function (deprecated since v0.6.0) (#2305)
- Remove the deprecated load_usgs_quakes function (deprecated since v0.6.0) (#2306)

- pygmt.grdtrack: Remove the warning about the incorrect parameter order of 'points, grid' (warned since v0.7.0) (#2312)

14.13.5 Bug Fixes

- GMTDataArrayAccessor: Fallback to default grid registration and gtype if the grid source file doesn't exist (#2009)
- Figure.subplot: Fix setting "sharex", "sharey", and "frame" in combination with Figure.basemap (#2417)
- Figure.subplot: Fix strange positioning issues after exiting subplot (#2427)
- pygmt.config: Correctly reset to default values that contain whitespaces (#2331)
- pygmt.set_display: Do nothing when the display method is set to 'none' (#2450)

14.13.6 Documentation

- GMTDataArrayAccessor: Add inline examples for setting GMT specific properties (#2370)
- Document limitations of GMT xarray accessors (#2375)
- Revise the notes about registration and gtype of remote datasets (#2384)
- Add project keywords to the pyproject.toml file (#2315)
- Add inline example for colorbar (#2373)
- Add inline example for grdview (#2381)
- Add inline example for load_earth_mask (#2355)
- Add inline example for load_earth_vertical_gravity_gradient (#2356)
- Add inline examples and improve documentation for pygmt.set_display (#2458)
- Add gallery example showing how to use patterns via the "fill" parameter (or similar parameters) (#2329)
- Add gallery example for scatter plot with histograms on sides (#2410)
- Add gallery example showing how to use advanced grdgradient via the "azimuth" & "normalize" parameters (#2354)
- Add gallery example for the Figure.timestamp method (#2391)
- Expand gallery example "Colorbar" for categorical data (#2395)
- Expand gallery example "Focal mechanisms" to use "*fill" and "pen" (#2433)
- Add working example to quickstart section of README (#2369)
- Recommend Mambaforge and mamba in the installation and contributing guides (#2385)

14.13.7 Maintenance

- Add the GMTSampleData class to simplify the load_sample_data and list_sample_data functions (#2342)
- Add a new target 'doctest' to run doctests only and simplify Makefile (#2443)
- Add a package-level variable `__gmt_version__` for development use (#2366)
- Allow printing show_versions() to in-memory buffer to enable testing (#2399)
- Accept a dict containing configurable GMT parameters in build_arg_string (#2324)
- Publish to TestPyPI and PyPI via OpenID Connect token (#2453)
- Remove –sdist –wheel flags from the build command (#2420)
- Replace ModuleNotFoundError with the more general ImportError (#2441)

Contributors: Dongdong Tian, Yvonne Fröhlich, Wei Ji Leong, Michael Grund, Will Schlitzer, Jing-Hui Tong, Max Jones

14.14 Release v0.8.0 (2022/12/30)

DOI 10.5281/zenodo.3782862

14.14.1 Highlights

- **Eighth minor release of PyGMT**
- Added support for tab auto-completion for all GMT default parameters (#2213)
- Created functions to download GMT remote datasets (#1786)
- Wrapped the ternary module (#1431)
- Added an intro tutorial for creating contour maps (#2126)

14.14.2 New Features

- Add load_earth_free_air_anomaly function for Earth free-air anomaly dataset (#2238)
- Add load_earth_geoid function for Earth Geoid dataset (#2236)
- Add load_earth_magnetic_anomaly function for Earth magnetic anomaly dataset (#2196, #2239, #2241)
- Add load_earth_vertical_gravity_gradient function for Earth vertical gravity gradient dataset (#2240)
- load_earth_relief: Add the support of data sources "gebco" , "gebcosi", and "synbath" (#1818, #2162, #2192, #2281)
- Wrap ternary (#1431)

14.14.3 Enhancements

- Set gridline (if available) as the default grid registration for remote datasets (#2266)
- Add ternary sample dataset (#2211)
- Figure.ternary: Add parameters "alabel", "blabel", and "clabel" (#2139)
- Figure.psconvert: Add a new alias "gs_path" (-G) (#2076)
- Figure.psconvert: Check if the given prefix is valid (#2170)
- Figure.savefig: Raise a FileNotFoundError if the parent directory doesn't exist (#2160)
- Figure.show: Allow keyword arguments passed to Figure.psconvert (#2078)
- pygmt.config: Support tab auto-completion for all GMT defaults (#2213)
- Rewrite the meca function to support offsetting and labeling beachballs (#1784)

14.14.4 Deprecations

- Deprecate xshift (X) and yshift (Y) aliases from all plotting modules (remove in v0.12.0) (#2071)
- Figure.plot: Deprecate parameter "color" to "fill" (remove in v0.12.0) (#2177)
- Figure.plot3d: Deprecate parameter "color" to "fill" (remove in v0.12.0) (#2178)
- Figure.rose: Deprecate parameter color to fill (remove in v0.12.0) (#2181)
- Figure.velo: Deprecate parameters "color" to "fill" and "uncertaintycolor" to "uncertaintyfill" (remove in v0.12.0) (#2206)
- Figure.wiggle: Deprecate parameter "color" (remove in v0.12.0) and add "fillpositive"/"fillnegative" (#2205)
- Figure.psconvert: Remove the deprecated parameter "icc_gray" (deprecated since v0.6.0) (#2267)
- Figure.text: Deprecate parameter "incols" to "use_word" (remove in v0.10.0) (#1964)

14.14.5 Bug Fixes

- Figure.meca: Fix line and circle of offset parameter for dict/pandas input (#2226)
- Figure.meca: Fix beachball offsetting with dict/pandas inputs (#2202)
- Figure.meca: Fix the bug when passing a dict of scalar values to the spec parameter (#2174)
- Figure.ternary: Fix the crash for pd.DataFrame input with GMT 6.3.0-6.4.0 (#2274)

14.14.6 Documentation

- Add intro tutorial section for creating contour map (#2126)
- Add gallery example for Figure.ternary method (#2138)
- Add gallery example showing the usage of vertical and horizontal bars (#1521)
- Add inline example for coast (#2142)
- Add inline example for grdcontour (#2148)
- Add inline example for grdimage (#2146)
- Add inline example for grd2cpt (#2145)
- Add inline example for solar (#2147)
- Add SciPy 2022 talk to presentations (#2053)
- Add instructions to install pygmt kernel for Jupyter users (#2153)
- Improve instructions about setting GMT_LIBRARY_PATH env variable (#2136)
- Add badges for conda package version, license, and twitter (#2081)
- Add PyOpenSci peer reviewed badge to main README (#2112)

14.14.7 Maintenance

- Add an internal function to load GMT remote datasets (#2200)
- Add support for Python 3.11 (#2172)
- NEP29: Test PyGMT on NumPy 1.24 (#2256)
- NEP29: Test PyGMT on NumPy 1.23 and 1.21 (#2057)
- Bump the GMT version in CI to 6.4.0 (#1990)

- Update baseline images for GMT 6.4.0 (#1883)
- Migrate Continuous Documentation from Vercel to Readthedocs (#1859)
- Set nested_sections to False for Sphinx-Gallery 0.11.0 regarding a correct navgation bar (#2046)
- Convert bug report, feature, and module request issue templates into yaml configured forms (#2091, #2214, #2216)
- doc: Set different html_baseurl for stable and dev versions (#2158)
- Update the instructions for checking README syntax (#2265)
- Use longname placeholders in the docstrings for common options (#1932)
- Add optional dependencies to pyproject.toml (#2069)
- Migrate project metadata from setup.py to pyproject.toml following PEP621 (#1848)
- Move blackdoc options to pyproject.toml (#2093)
- Move docformatter options from Makefile to pyproject.toml (#2072)
- Replace flake8 with flakeheaven (#1847)
- Add a workflow and Makefile target to test old GMT versions every Tuesday (#2079)
- Check if a module outputs to a temporary file using "Path().stat().st_size > 0" (#2224)
- pygmt.show_versions: Show GMT binary version and hide the Python interpreter path (#1838)
- Refactor grdview and grdimage to use virtualfile_from_data (#1988)
- Use the org-wide code of conduct (#2020)

Contributors: Dongdong Tian, Yvonne Fröhlich, Will Schlitzer, Michael Grund, Wei Ji Leong, Max Jones

---

14.15 Release v0.7.0 (2022/07/01)

14.15.1 Highlights

- **Seventh minor release of PyGMT**
- Wrapped 3 GMT modules
- Added two new PyGMT tutorials and EGU 2022 short course to external resources page (#1971 and #1935)

14.15.2 New Features

- Wrap binstats (#1652)
- Wrap filter1d (#1512)
- Wrap dimfilter (#1492)

14.15.3 Enhancements

- Support passing data in NumPy int8, int16, uint8 and uint16 dtypes to GMT (#1963)
- inset: Add region and projection aliases and fix two examples (#1931)
- basemap: Plotting frames if required parameters are not given (#1909)
- basemap: Added box alias for F (#1894)
- Add a sample dataset maunaLoa_co2 (#1961)
- Add a sample dataset notre_dame_topography (#1920)
- Add a sample dataset earth_relief_holes (#1921)

14.15.4 Deprecations

- NEP29: Set minimum required version to NumPy 1.20+ (#1985)
- Figure.wiggle: Remove parameter 'columns', use 'incols' instead. (#1977)
- Figure.histogram and pygmt.info: Remove parameter 'table', use 'data' instead (#1975)
- pygmt.surface: Remove parameter 'outfile', use 'outgrid' instead (#1976)
- blockm/contour/plot/plot3d/rose/surface/wiggle: Change the parameter order of data array and input arrays (#1978)

14.15.5 Bug Fixes

- grdtrack: Fix the bug when profile is given (#1867)
- Fix the grid accessor (grid registration and type) for 3D grids (#1913)

14.15.6 Documentation

- Add instructions to install PyGMT using mamba (#1967)
- Improve two gallery examples regarding categorical colormaps (#1934)
- Add inline example to dimfilter (#1956)
- Add inline example to surface (#1953)
- Add inline example to grdfill (#1954)
- Add inline code examples to contributing guidelines (#1924)
- Add thumbnail images to the external resources page (#1941)
- Redesign the team gallery using sphinx-design's card directive (#1937)

14.15.7 Maintenance

- Fix broken 'Improve this page' links using sphinx variable page_source_suffix (#1969)
- Split up functions for loading datasets (#1955)
- Set setuptools_scm fallback_version to follow PEP440 (#1945)
- Refactor text to use virtualfile_from_data (#1121)
- Run full tests only on Wednesday scheduled jobs (#1833)
- Run GMT Dev Tests on Monday, Wednesday and Friday only (#1922)

- Update GMT Dev Tests workflow to test on macOS-12 and ubuntu-22.04 (#1918)

Contributors: Dongdong Tian, Will Schlitzer, Wei Ji Leong, Andre L. Belem, Yvonne Fröhlich, Max Jones, Jack Beagley, Michael Grund

---

14.16 Release v0.6.1 (2022/04/11)

DOI 10.5281/zenodo.3782862

14.16.1 Highlights

- Patch release which allows passing None explicitly to pygmt functions (#1872, #1862, #1857, #1815)
- A new tutorial for grdhisteq (#1821)

14.16.2 Bug Fixes

- Fix pathlib support for plot and plot3d (#1831)

14.16.3 Documentation

- Add inline example for grdvolume (#1726)
- Format author affiliations in CITATION.cff and AUTHORS.md (#1844)

14.16.4 Maintenance

- NEP29: Run PyGMT tests and docs build on Python 3.10 (#1868)
- Let pygmt.show_versions() report geopandas version (#1846)
- Refactor build_arg_string to also deal with infile and outfile (#1837)
- Migrate build system settings to pyproject.toml following pep517 and pep518 (#1845)
- Use the build package to build sdist and wheel distributions (#1823)
- Let slash command /test-gmt-dev report job URL (#1866)

Contributors: Dongdong Tian, Max Jones, Wei Ji Leong, Michael Grund, Will Schlitzer

---

14.17 Release v0.6.0 (2022/03/14)

DOI 10.5281/zenodo.3782862

14.17.1 Highlights

- **Sixth minor release of PyGMT**
- New inline examples for 14 functions!
- Single `pygmt.datasets.load_sample_data` function for loading any sample dataset (#1685)
- Minimum required GMT version is now 6.3.0 (#1649)

14.17.2 New Features

- Wrap triangulate (#731)
- Wrap grdhisteq (#1433)

14.17.3 Enhancements

- Add alias for blockmean's -S parameter (#1601)
- Allow users to set the waiting time when displaying a preview image using an external viewer (#1618)
- Raise an exception if the given parameter is not recognized and is longer than 2 characters (#1792)

14.17.4 Deprecations

- Figure.plot/plot3d: Remove parameter "sizes", use "size" instead (#1809)
- Figure.contour/plot/plot3d/rose: Remove parameter "columns", use "incols" instead (#1806)
- Figure.psconvert: Add new aliases and deprecate parameter "icc_gray" (remove in v0.8.0) (#1673)
- NEP29: Set minimum required version to Python 3.8+ (#1676)
- NEP29: Set minimum required version to NumPy 1.19+ (#1675)

14.17.5 Bug Fixes

- Allow passing arguments containing spaces into pygmt functions (#1487)
- Fix the spacing parameter processing for many modules (#1805)
- Fix missing gcmt convention keys in pygmt.meca (#1611)
- Fix the spacing parameter and check required parameters in xyz2grd (#1804)
- Fix UnicodeDecodeError with shapefiles for plot and plot3d (#1695)

14.17.6 Documentation

- Add a shorter video introduction to the home page (#1769)
- Add Liam's 2021 ROSES video to learning resources (#1760)
- Add quick conda install instructions in main README (#1717)
- Add instructions for reporting upstream bugs to contributing.md (#1610)
- List key development dependencies to install for new contributors (#1783)
- Update Code of Conduct to v2.1 (#1754)
- Update the contributing guide about pushing changes to dvc and git (#1776)
- Update dataset links to the new remote-datasets site (#1785)
- Add more sections to the API docs (#1643)
- Add an "add a title" to starter tutorial (#1688)
- Reorganize tutorial section in the documentation sidebar (#1603)
- Update the starter tutorial introduction (#1607)
- Add gallery example to showcase blockmean (#1598)
- Add gallery example to showcase project (#1696)

- Update text symbol gallery example (#1648)
- Add inline example for blockmean (#1729)
- Add inline example for blockmedian (#1730)
- Add inline example for blockmode (#1731)
- Add inline example for grd2xyz (#1713)
- Add inline example for grdclip (#1711)
- Add inline example for grdcut (#1689)
- Add inline example for grdgradient (#1720)
- Add inline example for grdlandmask (#1721)
- Add inline example for grdproject (#1722)
- Add inline example for grdsample (#1724)
- Add inline example for grdtrack (#1725)
- Add inline example for select (#1756)
- Add inline example for sph2grd (#1718)
- Add inline example for xyz2grd (#1719)

14.17.7 Maintenance

- Add a test to make sure the incols parameter works for pandas.DataFrame (#1771)
- Add load_static_earth_relief function for internal testing (#1727)
- Migrate pylint settings from .pylintrc to pyproject.toml (#1755)
- NEP29: Test PyGMT on NumPy 1.22 (#1701)
- Replace pkg_resources with importlib.metadata (#1674)
- Update deprecated -g common option syntax (#1670)
- Update deprecated -JG syntax (#1659)
- Use pytest-doctestplus to skip some inline doctests (#1790)
- Use Python 3.10 in Continuous Integration tests (#1577)

Contributors: Will Schlitzer, Max Jones, Dongdong Tian, Michael Grund, Wei Ji Leong, Julius Busecke

---

14.18 Release v0.5.0 (2021/10/29)

DOI 10.5281/zenodo.3782862

14.18.1 Highlights

- **Fifth minor release of PyGMT**
- Wrapped 12 GMT modules
- Standardized and reorder table inputs to be 'data, x, y, z' across functions (#1479)

- Added a gallery example showing usage of line objects from a geopandas.GeoDataFrame (#1474)

14.18.2 New Features

- Wrap blockmode (#1456)
- Wrap gmtselect (#1429)
- Wrap grd2xyz (#1284)
- Wrap grdproject (#1377)
- Wrap grdsample (#1380)
- Wrap grdvolume (#1299)
- Wrap nearneighbor (#1379)
- Wrap project (#1122)
- Wrap sph2grd (#1434)
- Wrap sphdistance (#1383)
- Wrap sphinterpolate (#1418)
- Wrap xyz2grd (#636)
- Add function to import seafloor crustal age dataset (#1471)
- Add pygmt.load_dataarray function (#1439)

14.18.3 Enhancements

- Expand table-like input options for Figure.contour (#1531)
- Expand table-like input options for pygmt.surface (#1455)
- Raise GMTInvalidInput exception when required z is missing (#1478)
- Add support for passing pathlib.Path objects as filenames (#1382)
- Allow passing a list to the 'incols' parameter for blockm, grdtrack and text (#1475)
- Plot square or cube by default for OGR/GMT files with Point/MultiPoint types (#1438)
- Plot square or cube by default for geopandas Point/MultiPoint types (#1405)
- Add area_thresh to COMMON_OPTIONS (#1426)
- Add function to import Mars dataset (#1420)
- Add function to import hotspot dataset (#1386)

14.18.4 Deprecations

- pygmt.blockm*: Reorder input parameters to 'data, x, y, z' (#1565)
- pygmt.surface: Reorder input parameters to 'data, x, y, z' (#1562)
- Figure.contour: Reorder input parameters to 'data, x, y, z' (#1561)
- Figure.plot3d: Reorder input parameters to 'data, x, y, z' (#1560)
- Figure.plot: Reorder input parameters to "data, x, y" (#1547)
- Figure.rose: Reorder input parameters to 'data, length, azimuth' (#1546)

- Figure.wiggle: Reorder input parameter to 'data, x, y, z' (#1548)
- Figure.histogram: Deprecate parameter "table" to "data" (remove in v0.7.0) (#1540)
- pygmt.info: Deprecate parameter "table" to "data" (remove in v0.7.0) (#1538)
- Figure.wiggle: Deprecate parameter "columns" to "incols" (remove in v0.7.0) (#1504)
- pygmt.surface: Deprecate parameter "outfile" to "outgrid" (remove in v0.7.0) (#1458)
- NEP29: Set minimum required version to NumPy 1.18+ (#1430)

14.18.5 Bug Fixes

- Allow GMTDataArrayAccessor to work on sliced datacubes (#1581)
- Allow non-string color when input data is a matrix or a file for plot and plot3d (#1526)
- Raise RuntimeWarning instead of an exception for irregular grid spacing (#1530)
- Raise an error for zero increment grid (#1484)

14.18.6 Documentation

- Add CITATION.cff file for PyGMT (#1592)
- Update region and projection standard docstrings (#1510)
- Document gmtwhich -Ga option to download to appropriate cache folder (#1554)
- Add gallery example showing the usage of text symbols (#1522)
- Add gallery example for grdgradient (#1428)
- Add gallery example for grdlandmask (#1469)
- Add missing aliases to pygmt.grdgradient (#1515)
- Add missing aliases to pygmt.sphdistance (#1516)
- Add missing aliases to pygmt.blockmean and pygmt.blockmedian (#1500)
- Add missing aliases to pygmt.Figure.wiggle (#1498)
- Add missing aliases to pygmt.Figure.velo (#1497)
- Add missing aliases to pygmt.surface (#1501)
- Add missing aliases to pygmt.Figure.plot3d (#1503)
- Add missing aliases to pygmt.grdlandmask (#1423)
- Add missing aliases to pygmt.grdtrack (#1499)
- Add missing aliases to pygmt.Figure.plot (#1502)
- Add missing aliases to pygmt.Figure.text (#1448)
- Add missing aliases to pygmt.Figure.histogram (#1451)
- Add missing alias to pygmt.Figure.legend (#1453)
- Add missing aliases to pygmt.Figure.rose (#1452)
- Add missing alias to pygmt.Figure.grdview (#1450)
- Add missing aliases to pygmt.Figure.image.py (#1449)
- Add missing common options to contour (#1446)

- Add missing 'incols' alias to info (#1476)

14.18.7 Maintenance

- Add support for Python 3.10 (#1591)
- Make IPython partially optional on CI to increase test coverage of figure.py (#1496)
- Use mamba to install Continuous Integration dependencies (#841)
- Remove deprecated codecov dependency from CI (#1494)
- Add the use of Flake8 to check examples and fix warnings (#1477)

Contributors: Dongdong Tian, Michael Grund, Will Schlitzer, Wei Ji Leong, Max Jones, Yohai Magen, Amanda Leaman, @daroari, @obaney, @srijac, Andrés Ignacio Torres, Becky Salvage, Claudio Satriano, Jamie J Quinn, @carocamargo

---

14.19 Release v0.4.1 (2021/08/07)

DOI 10.5281/zenodo.3782862

14.19.1 Highlights

- **Patch release with multiple gallery examples**
- Change default GitHub branch name from "master" to "main" to increase inclusivity (#1360)
- Add a "PyGMT Team" page (#1308)

14.19.2 Enhancements

- Add common alias "verbose" (V) to grdlandmask and savefig (#1343)

14.19.3 Bug Fixes

- Change invalid input conditions in grdtrack (#1376)
- Fix bug so that x2sys_cross accepts dataframes with NaN values (#1369)

14.19.4 Documentation

- Combine documentation and compatibility sections in README (#1415)
- Add a gallery example for grdclip (#1396)
- Add a gallery example for different colormaps in subplots (#1394)
- Add a gallery example for the contour method (#1387)
- Add a gallery example showing individual custom symbols (#1348)
- Add common option aliases to COMMON_OPTIONS in decorators.py (#1407)
- Add return statement to grdclip and grdgradient docstring (#1390)
- Restructure contributing.md to separate docs/general info from contributing code section (#1339)

14.19.5 Maintenance

- Add tomli as a dependency in GMT Dev Tests (#1401)
- NEP29: Test PyGMT on NumPy 1.21 (#1355)

Contributors: Max Jones, Will Schlitzer, Michael Grund, Wei Ji Leong, Yohai Magen, Jiayuan Yao, Dongdong Tian, Kadatatlu Kishore, @sean0921, Soham Banerjee

---

14.20 Release v0.4.0 (2021/06/20)

14.20.1 Highlights

- **Fourth minor release of PyGMT**
- Add tutorials for datetime data (#1193) and plotting vectors (#1070)
- Support tab auto-completion in Jupyter (#1282)
- Minimum required GMT version is now 6.2.0 or newer (#1321)

14.20.2 New Features

- Wrap blockmean (#1092)
- Wrap grdclip (#1261)
- Wrap grdfill (#1276)
- Wrap grdgradient (#1269)
- Wrap grdlandmask (#1273)
- Wrap histogram (#1072)
- Wrap rose (#794)
- Wrap solar (#804)
- Wrap velo (#525)
- Wrap wiggle (#1145)
- Add new function to load fractures sample data (#1101)
- Allow load_earth_relief() to load the original land-only 01s or 03s SRTM tiles (#976)
- Handle geopandas and shapely geometries via geo_interface link (#1000)
- Support passing string type numbers, geographic coordinates and datetimes (#975)

14.20.3 Enhancements

- Allow passing an array as intensity for plot3d (#1109)
- Allow passing an array as intensity for plot (#1065)
- Allow passing xr.DataArray as shading to grdimage (#750)
- Allow x/y/z input for blockmedian and blockmean (#1319)

- Allow pygmt.which to accept a list of filenames as input (#1312)
- Refactor blockm* to use virtualfile_from_data and improve i/o (#1280)
- Refactor grdtrack to use virtualfile_from_data and improve i/o to pandas.DataFrame (#1189)
- Add parameters to histogram (#1249)
- Add alias 'aspatial' to methods blockmedian, info, plot, plot3d, surface (#1090)
- Add alias 'registration' to methods blockmean, info, grdfilter, surface (#1089)
- Add incols to COMMON_OPTIONS, blockmean, and blockmedian (#1300)
- Improve Figure.show for displaying previews in Jupyter notebooks and external viewers (#529)
- Let Figure.savefig recommend .eps or .pdf when .ps extension is used (#1307)

14.20.4 Deprecations

- Figure.contour: Deprecate parameter "columns" to "incols" (remove in v0.6.0) (#1303)
- Figure.plot: Deprecate parameter "sizes" to "size" (remove in v0.6.0) (#1254)
- Figure.plot: Deprecate parameter "columns" to "incols" (remove in v0.6.0) (#1298)
- Figure.plot3d: Deprecate parameter "sizes" to "size" (remove in v0.6.0) (#1258)
- Figure.plot3d: Deprecate parameter "columns" to "incols" (remove in v0.6.0) (#1040)
- Figure.rose: Deprecate parameter "columns" to "incols" (remove in v0.6.0) (#1306)
- NEP29: Set minimum required versions to NumPy 1.17+ and Python 3.7+ (#1074)
- Raise a warning for the use of short-form parameters when long-forms are available (#1316)

14.20.5 Bug Fixes

- Allow pandas.Series inputs to fig.histogram and pygmt.info (#1329)
- Explicitly use netcdf4 engine in xarray.open_dataarray to read grd files (#1264)
- Let Figure.savefig support filenames with spaces (#1116)
- Let Figure.show(method='external') work well in Python scripts (#1062)

14.20.6 Documentation

- Add histogram gallery example (#1272)
- Add a gallery example showing individual basic geometric symbols (#1211)
- Specify rectangle's width and height via style parameter in multi-parameter symbols example (#1325)
- Update the inset gallery example (#1287)
- Add categorical colorbars for plot, plot3d and line colors gallery examples (#1267)
- Apply NIST SI unit convention to some gallery examples (#1194)
- Use colorblind-friendly colors in the scatter plots gallery example (#1013)
- Added documentation for three oblique mercator projections (#1251)
- Add a list of external PyGMT resources (#1210)
- Complete documentation for grdtrack (#1190)

- Add projection and region to grdview docstring (#1295)
- Add common alias spacing (-I) for specifying grid increments (#1288)
- Standardize docstrings for table-like inputs (#1186)
- Clarify that the "transparency" parameter in plot/plot3d/text can be 1d array (#1265)
- Clarify that the "color" parameter in plot/plot3d can be 1d array (#1260)
- Clarify interplay of spacing and per_column in info (#1127)
- Remove the "full test" section from installation guide (#1206)
- Clarify position of deprecate_parameter decorator to be above use_alias (#1302)
- Add guidelines for managing issues to maintenance.md (#1301)
- Add alias name convention to CONTRIBUTING.md (#1256)
- Move contributing guide details to website and rename two sections (#1335)
- Update the check_figures_equal testing section in CONTRIBUTING.md (#1108)
- Revise Pull Request review process in CONTRIBUTING.md (#1119)

14.20.7 Maintenance

- Add a workflow to upload baseline images as a release asset (#1317)
- Add regression test for grdimage plotting an xarray.DataArray grid subset (#1314)
- Add download_test_data to download data files used in tests (#1310)
- Remove xfails and workarounds for datetime inputs into pygmt.info (#1236)
- Improve the DVC image diff workflow to support side-by-side comparison of modified images (#1219)
- Document the deprecation policy and add the deprecate_parameter decorator to deprecate parameters (#1160)
- Convert booleans arguments in build_arg_string, not in kwargs_to_strings (#1125)
- Create GitHub Action workflow for reporting DVC image diffs (#1104)
- Update "GMT Dev Tests" workflow to test macOS-11.0 and pre-release Python packages (#1105)
- Initialize data version control for managing test images (#1036)
- Separate workflows for running tests and building documentation (#1033)

Contributors: Dongdong Tian, Wei Ji Leong, Michael Grund, Max Jones, Will Schlitzer, Jiayuan Yao, Abhishek Anant, Claire Klima, Megan Munzek, Michael Neumann, Nathan Loria, Noor Buchi, Shivani chauhan, @alperen-kilic, Loïc Houpert, Emily McMullan, Lawrence Qupty, Matthew Tankersley, @shahid-0, Vitor Gratiere Torres

---

14.21 Release v0.3.1 (2021/03/14)

14.21.1 Highlights

- **Multiple bug fixes and an improved gallery**
- Reorganized gallery examples into new categories (#995)
- Added gallery examples for plotting vectors (#950, #890)
- Last version to support GMT 6.1.1, future PyGMT versions will require GMT 6.2.0 or newer

14.21.2 Enhancements

- Support passing a sequence to the spacing parameter of pygmt.info() (#1031)

14.21.3 Bug Fixes

- Fix issues in loading GMT's shared library (#977)
- Let pygmt.info load datetime columns into a str dtype array (#960)
- Check invalid combinations of resolution and registration in load_earth_relief() (#965)
- Open figures using the associated application on Windows (#952)
- Fix bug that stops Figure.coast from plotting with only dcw parameter (#910)

14.21.4 Documentation

- Add a gallery example showing different line front styles (#1022)
- Add a gallery example for a double y-axes graph (#1019)
- Add a gallery example of inset map showing a rectangle region (#1020)
- Add a gallery example to show coloring of points by categories (#1006)
- Add gallery example showing different polar projection use cases (#955)
- Add underscore guideline to CONTRIBUTING.md (#1034)
- Add instructions to upgrade installed PyGMT version (#1029)
- Improve the docstring of the pygmt package (#1016)
- Add common alias coltypes (-f) for specifying i/o data types (#994)
- Expand documentation linking in CONTRIBUTING.md (#802)
- Write changelog in markdown using MyST (#941)
- Update web font to Atkinson Hyperlegible (#938)
- Improve the gallery example for datetime inputs (#919)

14.21.5 Maintenance

- Refactor plot and plot3d to use virtualfile_from_data (#990)
- Explicitly exclude unnecessary files in source distributions (#999)
- Refactor grd modules to use virtualfile_from_data (#992)
- Refactor info and grdinfo to use virtualfile_from_data (#961)
- Onboarding maintainer checklist (#773)
- Add comprehensive tests for pygmt.clib.loading.clib_full_names (#872)

- Add a workflow checking links in plaintext and HTML files (#634)
- Remove nbsphinx extension (#931)
- Improve the error message for loading an old version of the GMT library (#925)
- Move requirements-dev.txt dependencies to environment.yml (#812)
- Ensure proper non-dev version string when publishing to PyPI (#900)
- Run tests in a single CI job (Ubuntu + Python 3.9) for draft PRs (#906)

Contributors: Dongdong Tian, Jiayuan Yao, Wei Ji Leong, Max Jones, Michael Grund, Will Schlitzer, Liam Toney, Kathryn Materna, Alicia Ngoc Diep Ha, Tawanda Moyo

---

14.22 Release v0.3.0 (2021/02/15)

14.22.1 Highlights

- **Third minor release of PyGMT**
- Wrap inset (#788) for making overview maps and subplot (#822) for multi-panel figures
- Apply standardized formatting conventions (#775) across most documentation pages
- Drop Python 3.6 support (#699) so PyGMT now requires Python 3.7 or newer

14.22.2 New Features

- Wrap grd2cpt (#803)
- Let Figure.text support record-by-record transparency (#716)
- Provide basic support for FreeBSD (#700, #878)

14.22.3 Enhancements

- Let load_earth_relief support the 'region' parameter for all resolutions (#873)
- Improve how PyGMT finds the GMT library (#702)
- Add common alias panel (-c) to all plotting functions (#853)
- Add aliases dcw (#765) and lakes (#781) to Figure.coast
- Add alias shading to Figure.colorbar (#752)
- Add alias annotation (A) to Figure.contour (#883)
- Wrap Figure.grdinfo aliases (#799)
- Add aliases frame and cmap to Figure.colorbar (#709)
- Add alias frame to Figure.grdview (#707)
- Improve the error message when PyGMT fails to load the GMT library (#814)
- Add GMTInvalidInput error to Figure.coast (#787)

14.22.4 Documentation

- Add authorship policy (#726)
- Update PyGMT development installation instructions (#865)
- Add a tutorial for adding a map title (#720)
- Add a tutorial for plotting Earth relief (#712)
- Add a tutorial for 3D perspective image (#743)
- Add a tutorial for contour maps (#705)
- Add a tutorial for plotting lines (#741)
- Add a tutorial for the region argument (#800)
- Add a gallery example for datetime inputs (#779)
- Add a gallery example for Figure.logo (#823)
- Add a gallery example for plotting multi-parameter symbols (#772)
- Add a gallery example for Figure.image (#777)
- Add a gallery example for setting line colors with a custom CPT (#774)
- Add more gallery examples for projections (#761, #721, #757, #723, #762, #742, #728, #727)
- Update the docstrings in the plotting modules (#881)
- Update the docstrings in the non-plotting modules (#882)
- Update Figure.coast docstrings (#798)
- Update the docstrings of common aliases (#862)
- Add sphinx-copybutton extension to easily copy codes (#838)
- Choose the best figures in tutorials for thumbnails (#826)
- Update axis label explanation in frames tutorial (#820)
- Add guidelines for types of tests to write (#796)
- Recommend using SI units in documentation (#795)
- Add a table for compatibility of PyGMT with Python and GMT (#763)
- Add description for the “columns” arguments (#766)
- Add a table of the available projections (#753)
- Add projection description for Lambert Azimuthal Equal-Area (#760)
- Change text when GMTInvalidInput error is raised for basemap (#729)

14.22.5 Bug Fixes

- Fix a bug of Figure.text when “text” is a non-string array (#724)
- Fix the error message when IPython is not available (#701)

14.22.6 Maintenance

- Add dependabot to keep GitHub Actions up to date (#861)
- Skip workflows in PRs if only non-source-code files are changed (#839)
- Add slash command '/test-gmt-dev' to test GMT dev version (#831)
- Check files for UNIX-style line breaks and 644 permission (#736)
- Rename vercel configuration file from now.json to vercel.json (#738)
- Add a CI job testing GMT master branch on Windows (#756)
- Migrate documentation deployment from Travis CI to GitHub Actions (#713)
- Move Figure.meca into a standalone module (#686)
- Move plotting functions to separate modules (#808)
- Move non-plotting modules to separate modules (#832)
- Add isort to sort imports alphabetically (#745)
- Convert relative imports to absolute imports (#754)
- Switch from versioneer to setuptools-scm (#695)
- Add docformatter to format plain text in docstrings (#642)
- Migrate pytest configurations to pyproject.toml (#725)
- Migrate coverage configurations to pyproject.toml (#667)
- Show test execution times in pytest (#835)
- Add tests for grdfilter (#809)
- Add tests for GMTInvalidInput of Figure.savefig and Figure.show (#810)
- Add args_in_kwargs function (#791)
- Add a Makefile target 'distclean' for deleting project metadata files (#744)
- Add a test for Figure.basemap map_scale (#739)
- Use args_in_kwargs for Figure.basemap error raising (#797)

Contributors: Will Schlitzer, Dongdong Tian, Wei Ji Leong, Michael Grund, Liam Toney, Max Jones

---

14.23 Release v0.2.1 (2020/11/14)

14.23.1 Highlights

- **Patch release with more tutorials and gallery examples!**
- Support Python 3.9 (#689)
- Add Liam's ROSES 2020 PyGMT talk (#643)

14.23.2 New Features

- Wrap plot3d (#471)
- Wrap grdfilter (#616)

14.23.3 Enhancements

- Allow np.object dtypes into virtualfile_from_vectors (#684)
- Let plot() accept record-by-record transparency (#626)
- Refactor info to allow datetime inputs from xarray.Dataset and pandas.DataFrame tables (#619)

14.23.4 Tutorials & Gallery

- Add tutorial for pygmt.config (#482)
- Add an example for different line styles (#604, #664)
- Add a gallery example for varying transparent points (#654)
- Add tutorial for pygmt.Figure.text (#480)
- Add an example for scatter plots with auto legends (#607)
- Improve colorbar gallery example (#596)

14.23.5 Documentation Improvements

- doc: Fix the description of grdcontour -G option (#681)
- Refresh Code of Conduct from v1.4 to v2.0 (#673)
- Add PyGMT Zenodo BibTeX entry to main README (#678)
- Complete most of documentation for makecpt (#676)
- Complete documentation for plot (#666)
- Add “no_clip” to plot, text, contour and meca (#661)
- Add common alias “verbose” (V) to all functions (#662)
- Improve documentation of Figure.logo() (#651)
- Add mini-galleries for methods and functions (#648)
- Complete documentation of grdimage (#620)
- Add common alias perspective (p) for plotting 3D illustrations (#627)
- Add common aliases xshift (X) and yshift (Y) (#624)
- Add common alias cores (x) for grdimage and other multi-threaded modules (#625)
- Enable switching different versions of documentation (#621)
- Add common alias transparency (-t) to all plotting functions (#614)

14.23.6 Bug Fixes

- Disallow passing arguments like -XNone to GMT (#639)

14.23.7 Maintenance

- Migrate PyPI release to GitHub Actions (#679)
- Upload artifacts showing diff images on test failure (#675)
- Add slash command "/format" to automatically format PRs (#646)
- Add instructions to run specific tests (#660)
- Add more tests for xarray grid shading (#650)
- Refactor xfail tests to avoid storing baseline images (#603)
- Add blackdoc to format Python codes in docstrings (#641)
- Check and lint sphinx configuration file doc/conf.py (#630)
- Improve Makefile to clean `__pycache__` directory recursively (#611)
- Update release process and checklist template (#602)

Contributors: Dongdong Tian, Wei Ji Leong, Conor Bacon, carocamargo

---

14.24 Release v0.2.0 (2020/09/12)

14.24.1 Highlights

- **Second minor release of PyGMT**
- Minimum required GMT version is now 6.1.1 or newer (#577)
- Plotting xarray grids using grdimage and grdview should not crash anymore and works for most cases (#560)
- Easier time-series plots with support for datetime-like inputs to plot (#464) and the region argument (#562)

14.24.2 New Features

- Wrap GMT_Put_Strings to pass str columns into GMT C API directly (#520)
- Wrap meca (#516)
- Wrap x2sys_init and x2sys_cross (#546)
- Let grdcut() accept xarray.DataArray as input (#541)
- Initialize a GMTDataArrayAccessor (#500)

14.24.3 Enhancements

- Allow passing in pandas dataframes to x2sys_cross (#591)
- Sensible array outputs for pygmt info (#575)
- Allow pandas.DataFrame table and 1D/2D numpy array inputs into pygmt.info (#574)
- Add auto-legend feature to grdcontour and contour (#568)
- Add common alias verbose (V) (#550)

- Let load_earth_relief() support all resolutions and optional subregion (#542)
- Allow load_earth_relief() to load pixel or gridline registered data (#509)

14.24.4 Documentation

- Link to try-gmt binder repository (#598)
- Improve docstring of data_kind() to include xarray grid (#588)
- Improve the documentation of Figure.shift_origin() (#536)
- Add shading to grdview gallery example (#506)

14.24.5 Bug Fixes

- Ensure surface and grdcut loads GMTDataArray accessor info into xarray (#539)
- Raise an error if short- and long-form arguments coexist (#537)
- Fix the grdtrack example to avoid crashes on macOS (#531)
- Properly allow for either pixel or gridline registered grids (#476)

14.24.6 Maintenance

- Add a test for xarray shading (#581)
- Remove expected failures on grdview tests (#589)
- Redesign check_figures_equal testing function to be more explicit (#590)
- Cut Windows CI build time in half to 15 min (#586)
- Add a test for Session.write_data() writing netCDF grids (#583)
- Add a test to make sure shift_origin does not crash (#580)
- Add testing.check_figures_equal to avoid storing baseline images (#555)
- Eliminate unnecessary jobs from Travis CI (#567) and Azure Pipelines (#513)
- Improve the workflow to test both GMT master (#485) and 6.1 branches (#554)
- Automatically cancel in-progress CI runs of old commits (#544)
- Remove the Stickler CI configuration file (#538), run style checks using GitHub Actions (#519)
- Cache GMT remote data as artifacts on GitHub (#530)
- Let pytest generate both HTML and XML coverage reports (#512)
- Run Continuous Integration tests on GitHub Actions (#475)

Contributors: Dongdong Tian, Wei Ji Leong, Tyler Newton, Liam Toney

---

14.25 Release v0.1.2 (2020/07/07)

14.25.1 Highlights

- Patch release in preparation for the SciPy 2020 sprint session
- Last version to support GMT 6.0, future PyGMT versions will require GMT 6.1 or newer

14.25.2 New Features

- Wrap grdcut (#492)
- Add show_versions() function for printing debugging information used in issue reports (#466)

14.25.3 Enhancements

- Change load_earth_relief()'s default resolution to 01d (#488)
- Enhance text with extra functionality and aliases (#481)

14.25.4 Documentation

- Add gallery example for grdview (#502)
- Turn all short aliases into long form (#474)
- Update the plotting example using the colormap generated by pygmt.makecpt (#472)
- Add instructions to view the test coverage reports locally (#468)
- Update the instructions for testing pygmt install (#459)

14.25.5 Bug Fixes

- Fix a bug when passing data to GMT in Session.open_virtual_file() (#490)

14.25.6 Maintenance

- Temporarily expect failures for some grdcontour and grdview tests (#503)
- Fix several failures due to updates of earth relief data (#498)
- Unpin pylint version and fix some lint warnings (#484)
- Separate tests of gmtinfo and grdinfo (#461)
- Fix the test for GMT_COMPATIBILITY=6 (#454)
- Update baseline images for updates of earth relief data (#452)
- Simplify PyGMT Release process (#446)

Contributors: Dongdong Tian, Wei Ji Leong, Liam Toney

14.26 Release v0.1.1 (2020/05/22)

14.26.1 Highlights

- Windows users rejoice, this bugfix release is for you!
- Let PyGMT work with the conda GMT package on Windows (#434)

14.26.2 Enhancements

- Handle setting special parameters without default settings for config (#411)

14.26.3 Documentation

- Update install instructions (#430)
- Add PyGMT AGU 2019 poster to website (#425)
- Redirect www.pygmt.org to latest, instead of dev (#423)

14.26.4 Bug Fixes

- Set GMT_COMPATIBILITY to 6 when pygmt session starts (#432)
- Improve how PyGMT finds the GMT library (#440)

14.26.5 Maintenance

- Finalize fixes on Windows test suite for v0.1.1 (#441)
- Cache test data on Azure Pipelines (#438)

Contributors: Dongdong Tian, Wei Ji Leong, Jason K. Moore

14.27 Release v0.1.0 (2020/05/03)

14.27.1 Highlights

- **First official release of PyGMT**
- Python 3.8 is now supported (#398)
- PyGMT now uses the stable version of GMT 6.0.0 by default (#363)
- Use sphinx-gallery to manage examples and tutorials (#268)

14.27.2 New Features

- Wrap blockmedian (#349)
- Add pygmt.config() to change gmt defaults locally and globally (#293)
- Wrap grdview (#330)
- Wrap grdtrack (#308)
- Wrap colorbar (#332)
- Wrap text (#321)

- Wrap legend (#333)
- Wrap makecpt (#329)
- Add a new method to shift plot origins (#289)

14.27.3 Enhancements

- Allow text accepting "frame" as an argument (#385)
- Allow for grids with negative lat/lon increments (#369)
- Allow passing in list to 'region' argument in surface (#378)
- Allow passing in scalar number to x and y in plot (#376)
- Implement default position/box for legend (#359)
- Add sequence_space converter in kwargs_to_string (#325)

14.27.4 Documentation

- Update PyPI install instructions and API disclaimer message (#421)
- Fix the link to GMT documentation (#419)
- Use napoleon instead of numpydoc with sphinx (#383)
- Document using a list for repeated arguments (#361)
- Add legend gallery entry (#358)
- Update instructions to set GMT_LIBRARY_PATH (#324)
- Fix the link to the GMT homepage (#331)
- Split projections gallery by projection types (#318)
- Fix the link to GMT/Matlab API in the README (#297)
- Use shinx extlinks for linking GMT docs (#294)
- Comment about country code in projection examples (#290)
- Add an overview page listing presentations (#286)

14.27.5 Bug Fixes

- Let surface return xr.DataArray instead of xr.Dataset (#408)
- Update GMT constant GMT_STR16 to GMT_VF_LEN for GMT API change in 6.1.0 (#397)
- Properly trigger pytest matplotlib image comparison (#352)
- Use uuid.uuid4 to generate unique names (#274)

14.27.6 Maintenance

- Quickfix Zeit Now miniconda installer link to anaconda.com (#413)
- Fix GitHub Pages deployment from Travis (#410)
- Update and clean TravisCI configuration (#404)
- Quickfix min elevation for new SRTM15+V2.1 earth relief grids (#401)
- Wrap docstrings to 79 chars and check with flake8 (#384)

- Update continuous integration scripts to 1.2.0 (#355)
- Use Zeit Now to deploy doc builds from PRs (#344)
- Move gmt from requirements.txt to CI scripts instead (#343)
- Change py.test to pytest (#338)
- Add Google Analytics to measure site visitors (#314)
- Register mpl_image_compare marker to remove PytestUnknownMarkWarning (#323)
- Disable Windows CI builds before PR #313 is merged (#320)
- Enable Mac and Windows CI on Azure Pipelines (#312)
- Fixes for using GMT 6.0.0rc1 (#311)
- Assign authorship to "The PyGMT Developers" (#284)

14.27.7 Deprecations

- Remove mention of gitter.im (#405)
- Remove portrait (-P) from common options (#339)
- Remove require.js since WorldWind was dropped (#278)
- Remove Web WorldWind support (#275)

Contributors: Dongdong Tian, Wei Ji Leong, Leonardo Uieda, Liam Toney, Brook Tozer, Claudio Satriano, Cody Woodson, Mark Wieczorek, Philipp Loose, Kathryn Materna

CHAPTER

FIFTEEN

CONTRIBUTORS GUIDE

This is a community driven project and everyone is welcome to contribute.

The project is hosted at the PyGMT GitHub repository.

The goal is to maintain a diverse community that's pleasant for everyone. **Please be considerate and respectful of others**. Everyone must abide by our Code of Conduct and we encourage all to read it carefully.

15.1 Ways to Contribute

15.1.1 Ways to Contribute Documentation and/or Code

- Tackle any issue that you wish! Some issues are labeled as **"good first issue"** to indicate that they are beginner friendly, meaning that they don't require extensive knowledge of the project.
- Make a tutorial or gallery example of how to do something.
- Improve the API documentation.
- Contribute code! This can be code that you already have and it doesn't need to be perfect! We will help you clean things up, test it, etc.

15.1.2 Ways to Contribute Feedback

- Provide feedback about how we can improve the project or about your particular use case. Open an issue with feature requests or bug fixes, or post general comments/questions on the forum.
- Help triage issues, or give a "thumbs up" on issues that others reported which are relevant to you.

15.1.3 Ways to Contribute to Community Building

- Participate and answer questions on the PyGMT forum Q&A.
- Participate in discussions at the quarterly PyGMT Community Meetings, which are announced on the forum governance page.
- Cite PyGMT when using the project.
- Spread the word about PyGMT or star the project!

15.2 Providing Feedback

15.2.1 Reporting a Bug

- Find the *Issues* tab on the top of the GitHub repository and click *New issue*.

- Click on *Get started* next to *Bug report*.
- **Please try to fill out the template with as much detail as you can**.
- After submitting your bug report, try to answer any follow up questions about the bug as best as you can.

Reporting Upstream Bugs

If you are aware that a bug is caused by an upstream GMT issue rather than a PyGMT-specific issue, you can optionally take the following steps to help resolve the problem:

- Add the line `pygmt.config(GMT_VERBOSE="d")` after your import statements, which will report the equivalent GMT commands as one of the debug messages.
- Either append all messages from running your script to your GitHub issue, or filter the messages to include only the GMT-equivalent commands using a command such as:

```
python <test>.py 2>&1 | awk -F': ' '$2=="GMT_Call_Command string" {print $3}'
```

  where `<test>` is the name of your test script.
- If the bug is produced when passing an in-memory data object (e.g., a pandas.DataFrame or xarray.DataArray) to a PyGMT function, try writing the data to a file (e.g., a netCDF or ASCII txt file) and passing the data file to the PyGMT function instead. In the GitHub issue, please share the results for both cases along with your code.

15.2.2 Submitting a Feature Request

- Find the *Issues* tab on the top of the GitHub repository and click *New issue*.
- Click on *Get started* next to *Feature request*.
- **Please try to fill out the template with as much detail as you can**.
- After submitting your feature request, try to answer any follow up questions as best as you can.

15.2.3 Submitting General Comments/Questions

There are several pages on the Community Forum where you can submit general comments and/or questions:

- For questions about using PyGMT, select *New Topic* from the PyGMT Q&A Page.
- For general comments, select *New Topic* from the Lounge Page.
- To share your work, select *New Topic* from the Showcase Page.

15.3 General Guidelines

15.3.1 Resources for New Contributors

Please take a look at these resources to learn about Git and pull requests (don't hesitate to *ask questions*):

- How to Contribute to Open Source.
- Git Workflow Tutorial by Aaron Meurer.
- How to Contribute to an Open Source Project on GitHub.

15.3.2 Getting Help

Discussion often happens on GitHub issues and pull requests. In addition, there is a Discourse forum for the project where you can ask questions.

15.3.3 Pull Request Workflow

We follow the git pull request workflow to make changes to our codebase. Every change made goes through a pull request, even our own, so that our continuous integration services have a chance to check that the code is up to standards and passes all our tests. This way, the *main* branch is always stable.

General Guidelines for Making a Pull Request (PR):

- What should be included in a PR
  - Have a quick look at the titles of all the existing issues first. If there is already an issue that matches your PR, leave a comment there to let us know what you plan to do. Otherwise, **open an issue** describing what you want to do.
  - Each pull request should consist of a **small** and logical collection of changes; larger changes should be broken down into smaller parts and integrated separately.
  - Bug fixes should be submitted in separate PRs.
- How to write and submit a PR
  - Use underscores, not hyphens, in the names of all files and directories, as per PEP8 for Python (*.py) files. A few exceptions are allowed (e.g., `.pre-commit-config.yaml`) and are listed in the style checks workflow.
  - Describe what your PR changes and *why* this is a good thing. Be as specific as you can. The PR description is how we keep track of the changes made to the project over time.
  - Do not commit changes to files that are irrelevant to your feature or bugfix (e.g.: `.gitignore`, IDE project files, etc).
  - Write descriptive commit messages. Chris Beams has written a guide on how to write good commit messages.
- PR review
  - Be willing to accept criticism and work on improving your code; we don't want to break other users' code, so care must be taken not to introduce bugs.
  - Be aware that the pull request review process is not immediate, and is generally proportional to the size of the pull request.

General Process for Pull Request Review:

After you've submitted a pull request, you should expect to hear at least a comment within a couple of days. We may suggest some changes, improvements or alternative implementation details.

To increase the chances of getting your pull request accepted quickly, try to:

- Submit a friendly PR
  - Write a good and detailed description of what the PR does.
  - Write some documentation for your code (docstrings) and leave comments explaining the *reason* behind non-obvious things.
  - Write tests for the code you wrote/modified if needed. Please refer to *Testing your code* or *Testing plots*.

  - Include an example of new features in the gallery or tutorials. Please refer to *Gallery plots* or *Tutorials*. If adding a new method/function/class, the gallery example or tutorial should be submitted in a separate pull request.

- Have a good coding style
  - Use readable code, as it is better than clever code (even with comments).
  - Follow the PEP8 style guide for code and the NumPy style guide for docstrings. Please refer to *Code style*.

Pull requests will automatically have tests run by GitHub Actions. This includes running both the unit tests as well as code linters. GitHub will show the status of these checks on the pull request. Try to get them all passing (green). If you have any trouble, leave a comment in the PR or *get in touch*.

15.4 Setting up your Environment

These steps for setting up your environment are necessary for *editing the documentation locally* and *contributing code*. A local PyGMT development environment is not needed for *editing the documentation on GitHub*.

We highly recommend using Miniforge and the `conda` package manager to install and manage your Python packages. It will make your life a lot easier!

The repository includes a virtual environment file `environment.yml` with the specification for all development requirements to build and test the project. In particular, these are some of the key development dependencies you will need to install to build the documentation and run the unit tests locally:

- git (for cloning the repo and tracking changes in code)
- Git LFS (for downloading baseline images used in tests)
- pytest-mpl (for checking that generated plots match the baseline)
- sphinx-gallery (for building the gallery example page)

See the `environment.yml` file for the full list of dependencies and the environment name (`pygmt`). Once you have forked and cloned the repository to your local machine, you can use this file to create an isolated environment on which you can work. Run the following on the base of the repository to create a new conda environment from the `environment.yml` file:

```
conda env create --file environment.yml
```

Before building and testing the project, you have to activate the environment (you'll need to do this every time you start a new terminal):

```
conda activate pygmt
```

We have a `Makefile` that provides commands for installing, running the tests and coverage analysis, running linters, etc. If you don't want to use `make`, open the `Makefile` and copy the commands you want to run.

To install the current source code into your testing environment, run:

```
make install  # on Linux/macOS
python -m pip install --no-deps -e .  # on Windows
```

This installs your project in *editable* mode, meaning that changes made to the source code will be available when you import the package (even if you're on a different directory).

15.5 Contributing Documentation

15.5.1 PyGMT Documentation Overview

There are four main components to PyGMT's documentation:

- Gallery examples, with source code in Python `*.py` files under the `examples/gallery/` folder.
- Tutorial examples, with source code in Python `*.py` files under the `examples/tutorials/` folder.
- API documentation, with source code in the docstrings in Python `*.py` files under the `pygmt/src/` and `pygmt/datasets/` folders.
- Getting started/developer documentation, with source text in ReST `*.rst` and markdown `*.md` files under the `doc/` folder.

The documentation is written primarily in reStructuredText and built by Sphinx. Please refer to reStructuredText Cheatsheet if you are new to reStructuredText. When contributing documentation, be sure to follow the general guidelines in the *pull request workflow* section.

There are two primary ways to edit the PyGMT documentation:

- For simple documentation changes, you can easily *edit the documentation on GitHub*. This only requires you to have a GitHub account.
- For more complicated changes, you can *edit the documentation locally*. In order to build the documentation locally, you first need to *set up your environment*.

15.5.2 Editing the Documentation on GitHub

If you're browsing the documentation and notice a typo or something that could be improved, please consider letting us know by *creating an issue* or (even better) submitting a fix.

You can submit fixes to the documentation pages completely online without having to download and install anything:

1. On each documentation page, there should be an "Improve This Page" link at the very top.
2. Click on that link to open the respective source file (usually an `.rst` file in the `doc/` folder or a `.py` file in the `examples/` folder) on GitHub for editing online (you'll need a GitHub account).
3. Make your desired changes.
4. When you're done, scroll to the bottom of the page.
5. Fill out the two fields under "Commit changes": the first is a short title describing your fixes; the second is a more detailed description of the changes. Try to be as detailed as possible and describe *why* you changed something.
6. Choose "Create a new branch for this commit and start a pull request" and click on the "Propose changes" button to open a pull request.
7. The pull request will run the GMT automated tests and make a preview deployment. You can see how your change looks in the PyGMT documentation by clicking the "Details" button of the "docs/readthedocs.org:pygmt-dev" status check, after the building has finished (usually 10-15 minutes after the pull request was created).
8. We'll review your pull request, recommend changes if necessary, and then merge them in if everything is OK.
9. Done!

Alternatively, you can make the changes offline to the files in the `doc` folder or the example scripts. See *editing the documentation locally* for instructions.

15.5.3 Editing the Documentation Locally

For more extensive changes, you can edit the documentation in your cloned repository and build the documentation to preview changes before submitting a pull request. First, follow the *setting up your environment* instructions. After making your changes, you can build the HTML files from sources using:

```
cd doc
make all
```

This will build the HTML files in `doc/_build/html`. Open `doc/_build/html/index.html` in your browser to view the pages. Follow the *pull request workflow* to submit your changes for review.

15.5.4 Adding example code

Many of the PyGMT functions have example code in their documentation. To contribute an example, add an "Example" header and put the example code below it. Have all lines begin with `>>>`. To keep this example code from being run during testing, add the code `__doctest_skip__ = ["function_name"]` to the top of the module.

Inline code example

Below the import statements at the top of the file:

```
__doctest_skip__ = ["function_name"]
```

At the end of the function's docstring:

```
Example
-------
>>> import pygmt
>>> # Comment describing what is happening
>>> Code example
```

15.5.5 Contributing Gallery Plots

The gallery and tutorials are managed by sphinx-gallery. The source files for the example gallery are `.py` scripts in `examples/gallery/` that generate one or more figures. They are executed automatically by sphinx-gallery when the *documentation is built*. The output is gathered and assembled into the gallery.

You can **add a new** plot by placing a new `.py` file in one of the folders inside the `examples/gallery` folder of the repository. See the other examples to get an idea for the format.

General guidelines for making a good gallery plot:

- Examples should highlight a single feature/command. Good: *how to add a label to a colorbar*. Bad: *how to add a label to the colorbar and use two different CPTs and use subplots*.
- Try to make the example as simple as possible. Good: *use only commands that are required to show the feature you want to highlight*. Bad: *use advanced/complex Python features to make the code smaller*.
- Use a sample dataset from `pygmt.datasets` if you need to plot data. If a suitable dataset isn't available, open an issue requesting one and we'll work together to add it.
- Add comments to explain things that aren't obvious from reading the code. Good: *Use a Mercator projection and make the plot 15 centimeters wide*. Bad: *Draw coastlines and plot the data*.
- Describe the feature that you're showcasing and link to other relevant parts of the documentation.
- SI units should be used in the example code for gallery plots.

15.5.6 Contributing Tutorials

The tutorials (the User Guide in the docs) are also built by sphinx-gallery from the `.py` files in the `examples/tutorials` folder of the repository. To add a new tutorial:

- Create a `.py` file in the `examples/tutorials/advanced` folder.
- Write the tutorial in "notebook" style with code mixed with paragraphs explaining what is being done. See the other tutorials for the format.
- Choose the most representative figure as the thumbnail figure by adding the comment line `# sphinx_gallery_thumbnail_number = <fig_number>` at the end of the tutorial. The *fig_number* starts from 1.

Guidelines for a good tutorial:

- Each tutorial should focus on a particular set of tasks that a user might want to accomplish: plotting grids, interpolation, configuring the frame, projections, etc.
- The tutorial code should be as simple as possible. Avoid using advanced/complex Python features or abbreviations.
- Explain the options and features in as much detail as possible. The gallery has concise examples while the tutorials are detailed and full of text.
- SI units should be used in the example code for tutorial plots.

Note that the pygmt.Figure.show method needs to be called for a plot to be inserted into the documentation.

15.5.7 Editing the API Documentation

The API documentation is built from the docstrings in the Python `*.py` files under the `pygmt/src/` and `pygmt/datasets/` folders. **All docstrings** should follow the NumPy style guide. All functions/classes/methods should have docstrings with a full description of all arguments and return values.

While the maximum line length for code is automatically set by Ruff, docstrings must be formatted manually. To play nicely with Jupyter and IPython, **keep docstrings limited to 88 characters** per line.

15.5.8 Standards for Example Code

When editing documentation, use the following standards to demonstrate the example code:

1. Python arguments, such as import statements, Boolean expressions, and function arguments should be wrapped as `code` by using `` on both sides of the code. Examples: ``import pygmt`` results in `import pygmt`, ``True`` results in `True`, ``style="v"`` results in `style="v"`.
2. Literal GMT arguments should be **bold** by wrapping the arguments with ** (two asterisks) on both sides. The argument description should be in *italicized* with * (single asterisk) on both sides. Examples: `**+l**\ *label*` results in **+l***label*, `**05m**` results in **05m**.
3. Optional arguments are wrapped with [ ] (square brackets).
4. Arguments that are mutually exclusive are separated with a | (bar) to denote "or".
5. Default arguments for parameters and configuration settings are wrapped with [ ] (square brackets) with the prefix "Default is". Example: [Default is **p**].

15.5.9 Cross-referencing with Sphinx

The API reference is manually assembled in `doc/api/index.rst`. The *autodoc* sphinx extension will automatically create pages for each function/class/module/method listed there.

You can reference functions, classes, modules, and methods from anywhere (including docstrings) using:

- :func:`package.module.function`
- :class:`package.module.class`
- :meth:`package.module.method`
- :mod:`package.module`

An example would be to use :meth:`pygmt.Figure.grdview` to link to https://www.pygmt.org/latest/api/generated/pygmt.Figure.grdview.html. PyGMT documentation that is not a class, method, or module can be linked with :doc:`Any Link Text </path/to/the/file>`. For example, :doc:`Install instructions </install>` links to https://www.pygmt.org/latest/install.html.

Linking to the GMT documentation and GMT configuration parameters can be done using:

- :gmt-docs:`page_name.html`
- :gmt-term:`GMT_PARAMETER`

An example would be using :gmt-docs:`makecpt.html` to link to https://docs.generic-mapping-tools.org/6.7/makecpt.html. For GMT configuration parameters, an example is :gmt-term:`COLOR_FOREGROUND` to link to COLOR_FOREGROUND.

Sphinx will create a link to the automatically generated page for that function/class/module/method.

15.6 Contributing Code

15.6.1 PyGMT Code Overview

The source code for PyGMT is located in the `pygmt/` directory. When contributing code, please open an issue first to discuss the feature and its implementation and be sure to follow the general guidelines in the *pull request workflow* section.

15.6.2 Code Style

We use the Ruff tool to format the code, so we don't have to think about it. It loosely follows the PEP8 guide but with a few differences. Regardless, you won't have to worry about formatting the code yourself. Before committing, run it to automatically format your code:

```
make format
```

For consistency, we also use `pre-commit` hooks (via `prek`) to enforce UNIX-style line endings (`\n`) and file permission 644 (`-rw-r--r--`) throughout the whole project. Don't worry if you forget to do it. Our continuous integration systems will warn us and you can make a new commit with the formatted code. Even better, you can just write `/format` in the first line of any comment in a pull request to lint the code automatically.

When introducing a new parameter name, use an underscore to separate words. This improves readability and aligns with the PEP 8 style guide. For common shortcuts no underscore is needed, e.g., `surftype`, `outgrid`, or `timefmt`. This convention is not applied by the code checking tools, but the PyGMT maintainers will comment on any pull requests as needed.

When working on a tutorial or a gallery plot, it is good practice to use code block separators to split a long script into multiple blocks. The separators also make it possible to run the script like a Jupyter notebook in some modern text editors or IDEs. We consistently use `# %%` as code block separators (please refer to issue #2660 for the discussions) and require at least one separator in all example files.

We also use Ruff to check the quality of the code and quickly catch common errors.

The `Makefile` contains rules for running the linter checks:

```
make check   # Runs Ruff in check mode
```

15.6.3 Wrapping a GMT Module

Wrapping a GMT module in PyGMT is usually a big task, but it can progress more smoothly and efficiently when divided into **small, manageable chunks**. This section gives an overview of the main tasks involved.

1. Open "wrapper request" issue - create a feature request for wrapping a module and discuss what features should be available in the wrapper [optional, usually done by users].
2. Open a "wrapper tracking" issue - use the "Wrapper for a GMT module" issue template, to track the progress of wrapping the module. Link it to the Project board, and close the "wrapper request issue" with a comment such as [usually done by maintainers]:

   > Thank you for opening the feature request. The progress of wrapping the module will be tracked in issue #XXX and the Project board.

3. Open one PR for the initial implementation, focusing on required and essential parameters [done by maintainers or contributors].
4. Close the "wrapper tracking issue" once the initial implementation is merged. Leave a comment such as [done by maintainers]:

   > The initial implementation of wrapping the XXX module was completed in PR #XXX. Not all functionalities are implemented yet. Further progress will be tracked in the Project board. This is necessary to avoid having too many long-term open issues.

5. Open one or more PRs to implement the remaining features and missing aliases.
6. Open one PR to add a gallery example or a tutorial.

These PRs can be split among multiple contributors. There is no obligation for a single contributor to complete all steps. Please comment on the "wrapper tracking issue" if you would like to open a PR for any of these tasks to avoid redundant efforts.

Initial Feature Implementation

First, comment on the "Wrapper tracking issue" that you will be working on the initial implementation. This first pull request should be as minimal as possible - only adding the required functionality (i.e., wrapping the required GMT parameters and supporting the primary input/output types).

The following steps are common to all initial implementation pull requests that wrap a GMT module:

- Create a new module `<module-name>.py` in `pygmt/src`. The module docstring should include the module name and a short description of the functionality (e.g., `grdfill - Fill blank areas from a grid.`).
- Add a function `<module-name>` to the module. When writing the new function, it is generally easiest to reference the source code for other functions that input/output similar object types.
- Write a detailed docstring following the numpy style guide.
- Add the function to the import statements in `pygmt/src/__init__.py` and `pygmt/__init__.py`.
- Add the function to appropriate section of the API documentation in `doc/api/index.rst`.
- Add a testing module `test_<module-name>.py` in `pygmt/tests`, following the guidelines in the *testing your code* section.

Add Missing Aliases

After the initial implementation, missing aliases can be added in separate PRs:

- Select a suitable alias for each GMT option, following the guidelines in the *code style* section. Before creating a new alias, check:
  - whether the parameter is listed in the `COMMON_PARAMETERS` dictionary in `pygmt/helpers/decorators.py`
  - whether other wrapped GMT modules have a similar parameter
  - whether GMT.jl has defined an alias
- Add the alias to the `AliasSystem` class and the function signature.
- Add the alias and description to the parameters section of the docstring, using the `fmt_docstring` decorator to add descriptions for parameters included in the `COMMON_PARAMETERS` dictionary.

15.6.4 Testing your Code

Automated testing helps ensure that our code is as free of bugs as it can be. It also lets us know immediately if a change we make breaks any other part of the code.

All of our test code and data are stored in the `tests` subpackage. We use the pytest framework to run the test suite.

Please write tests for your code so that we can be sure that it won't break any of the existing functionality. Tests also help us be confident that we won't break your code in the future.

When writing tests, don't test everything that the GMT function already tests, such as every unique combination of arguments. An exception to this would be the most popular methods, such as pygmt.Figure.plot and pygmt.Figure.basemap. The highest priority for tests should be the Python-specific code, such as NumPy, pandas, and xarray objects and the virtual file mechanism.

If you're **new to testing**, see existing test files for examples of things to do. **Don't let the tests keep you from submitting your contribution!** If you're not sure how to do this or are having trouble, submit your pull request anyway. We will help you create the tests and sort out any kind of problem during code review.

Pull the latest changes (including baseline images), run the tests, and calculate test coverage using:

```
git pull
make test
```

The coverage report will let you know which lines of code are touched by the tests. If all the tests pass, you can view the coverage reports by opening `htmlcov/index.html` in your browser. **Strive to get 100% coverage for the lines you changed.** It's OK if you can't or don't know how to test something. Leave a comment in the PR and we'll help you out.

You can also run tests in just one test script using:

```
pytest pygmt/tests/NAME_OF_TEST_FILE.py
```

or run tests which contain names that match a specific keyword expression:

```
pytest -k KEYWORD pygmt/tests
```

15.6.5 Testing Plots

Writing an image-based test is only slightly more difficult than a simple test. The main consideration is that you must specify the "baseline" or reference image, and compare it with a "generated" or test image. This is handled using the *decorator* functions `@pytest.mark.mpl_image_compare` and `@check_figures_equal` whose usage are further described below.

Using mpl_image_compare

This is the preferred way to test plots whenever possible.

This method uses the pytest-mpl plug-in to test plot generating code. Every time the tests are run, `pytest-mpl` compares the generated plots with known correct ones stored in `pygmt/tests/baseline`. If your test created a `pygmt.Figure` object, you can test it by adding a *decorator* and returning the `pygmt.Figure` object:

```
@pytest.mark.mpl_image_compare
def test_my_plotting_case():
    """
    Test that my plotting method works.
    """
    fig = Figure()
    fig.basemap(region=[0, 360, -90, 90], projection="W15c", frame=True)
    return fig
```

Your test function **must** return the `pygmt.Figure` object and you can only test one figure per function.

Before you can run your test, you'll need to generate a *baseline* (a correct version) of your plot. Run the following from the repository root:

```
pytest --mpl-generate-path=baseline pygmt/tests/NAME_OF_TEST_FILE.py
```

This will create a `baseline` folder with all the plots generated in your test file. Visually inspect the one corresponding to your test function. If it's correct, copy it (and only it) to `pygmt/tests/baseline`. When you run `make test` the next time, your test should be executed and passing.

Don't forget to commit the baseline image as well! Baseline images are tracked with Git LFS, as explained in the next section.

Using Git Large File Storage (LFS) to Manage Test Images

As the baseline images are quite large blob files that can change often (e.g., with new GMT versions), they are managed with Git LFS rather than regular Git objects.

You need to run the following command once to set up Git LFS on your machine:

```
git lfs install
```

After that, `git clone` and `git pull` download the baseline images automatically. If Git LFS downloads were deliberately skipped, run `git lfs pull` to download the missing images. The images in `pygmt/tests/baseline` can then be used by the image-comparison tests as usual.

The entire workflow for generating or modifying baseline test images can be summarized as follows:

```
# Sync Git; Git LFS objects are downloaded automatically
git pull

# Generate new baseline images
pytest --mpl-generate-path=baseline pygmt/tests/test_logo.py
mv baseline/*.png pygmt/tests/baseline/

# Stage the Git LFS-tracked baseline image
git add pygmt/tests/baseline/test_logo.png

# Commit and push; Git LFS uploads the image automatically
```

(continues on next page)

(continued from previous page)

```
git commit -m "Add test_logo.png baseline image"
git push
```

Using `check_figures_equal`

This approach draws the same figure using two different methods (the reference method and the tested method), and checks that both of them are the same. It takes two `pygmt.Figure` objects (`fig_ref` and `fig_test`), generates a png image, and checks for the Root Mean Square (RMS) error between the two. Here's an example:

```
@check_figures_equal()
def test_my_plotting_case():
    """
    Test that my plotting method works.
    """
    fig_ref, fig_test = Figure(), Figure()
    fig_ref.grdimage("@earth_relief_01d_g", projection="W120/15c", cmap="gmt/geo")
    fig_test.grdimage(grid, projection="W120/15c", cmap="gmt/geo")
    return fig_ref, fig_test
```

CHAPTER

SIXTEEN

MAINTAINERS GUIDE

This page contains instructions for project maintainers about how our setup works, making releases, creating packages, etc.

If you want to make a contribution to the project, see the *Contributors Guide* instead.

16.1 Onboarding/Offboarding Access Checklist

Note that anyone can contribute to PyGMT, even without being added to the GenericMappingTools team. The onboarding items below are for people who would like to make regular contributions, and could benefit from extra permissions to the developer and communication tools we use.

16.1.1 As a Contributor

- Add to the pygmt-contributors team (gives 'write' permission to the repository)
- Add as a member on HackMD (for draft announcements) [optional]

16.1.2 As a Maintainer

- Add to the pygmt-maintainers team (gives 'maintain' permission to the repository)
- Add to "Active Maintainers" on the *Team Gallery page*
- Add as a moderator on the GMT forum (to see mod-only discussions) [optional]
- Add as a maintainer on ReadtheDocs [optional]
- Add as a curator to the GMT community on Zenodo (for making releases) [optional]

16.1.3 As an Administrator

- Add to the pygmt-admin team (gives 'admin' permission to the repository)
- Add as a maintainer on PyPI and Test PyPI [optional]

Note: When a maintainer is no longer active (no activity in one year), we will mirror the onboarding access checklist:

- Move from the pygmt-maintainers team to the pygmt-contributors team
- Move from "Active Maintainers" to "Distinguished Contributors" on the *Team Gallery page*
- Remove 'maintain' permission from GMT forum, ReadTheDocs, Zenodo

16.2 Branches

- *main*: Always tested and ready to become a new version. Don't push directly to this branch. Make a new branch and submit a pull request instead.
- *gh-pages*: Holds the HTML documentation and is served by GitHub. Pages for the main branch are in the `dev` folder. Pages for each release are in their own folders. **Automatically updated by GitHub Actions** so you shouldn't have to make commits here.

16.3 Managing GitHub Issues

A few guidelines for managing GitHub issues:

- Assign labels and the expected milestone to issues as appropriate.
- When people request to work on an open issue, either assign the issue to that person and post a comment about the assignment or explain why you are not assigning the issue to them and, if possible, recommend other issues for them to work on.
- People with write access should self-assign issues and/or comment on the issues that they will address.
- For upstream bugs, close the issue after an upstream release fixes the bug. If possible, post a comment when an upstream PR is merged that fixes the problem, and consider adding a regression test for serious bugs.

16.4 Reviewing and Merging Pull Requests

A few guidelines for reviewing:

- Always **be polite** and give constructive feedback.
- Welcome new users and thank them for their time, even if we don't plan on merging the PR.
- Don't be harsh with code style or performance. If the code is bad, either (1) merge the pull request and open a new one fixing the code and pinging the original submitter or (2) comment on the PR detailing how the code could be improved. Both ways are focused on showing the contributor **how to write good code**, not shaming them.

Pull requests should be **squash merged**. This means that all commits will be collapsed into one. The main advantages of this are:

- Eliminates experimental commits or commits to undo previous changes.
- Makes sure every commit on the main branch passes the tests and has a defined purpose.
- The maintainer writes the final commit message, so we can make sure it's good and descriptive.

16.5 Continuous Integration

We use GitHub Actions continuous integration (CI) services to build, test and manage the project on Linux, macOS and Windows. The GitHub Actions CI are controlled by workflow files located in `.github/workflows`. Here we briefly summarize the functions of the workflows. Please refer to the comments in the workflow files for more details.

- `benchmarks.yml`: Benchmark the execution speed of tests to track performance of PyGMT functions
- `cache_data.yaml`: Cache GMT remote data files and upload as artifacts
- `check-links.yml`: Check links in the repository and documentation
- `ci_docs.yml`: Build documentation on Linux/macOS/Windows and deploy to GitHub
- `ci_doctest.yaml`: Run all doctests on Linux/macOS/Windows

- `ci_tests.yaml`: Run regular PyGMT tests on Linux/macOS/Windows
- `ci_tests_dev.yaml`: Run regular PyGMT tests with GMT dev version on Linux/macOS/Windows
- `ci_tests_legacy.yaml`: Run regular PyGMT tests with GMT legacy versions on Linux/macOS/Windows
- `format-command.yml`: Format the codes using slash command
- `publish-to-pypi.yml`: Publish archives to PyPI and TestPyPI
- `release-baseline-images.yml`: Upload the ZIP archive of baseline images as a release asset
- `release-drafter.yml`: Draft the next release notes
- `slash-command-dispatch.yml`: Support slash commands in pull requests
- `style_checks.yaml`: Code lint and style checks
- `type_checks.yml`: Static type checks

16.6 Continuous Documentation

We use the ReadtheDocs service to preview changes made to our documentation website every time we make a commit in a pull request. The service has a configuration file `.readthedocs.yaml`, with a list of options to change the default behaviour at https://docs.readthedocs.io/en/stable/config-file/index.html.

16.7 Continuous Benchmarking

We use the CodSpeed service to continuously track PyGMT's performance. The `pytest-codspeed` plugin collects benchmark data and uploads it to the CodSpeed server, where results are available at https://codspeed.io/GenericMappingTools/pygmt.

Benchmarking is handled through the `benchmarks.yml` GitHub Actions workflow. It's automatically executed when a pull request is merged into the main branch. To trigger benchmarking in a pull request, add the `run/benchmark` label to the pull request.

To include a new test in the benchmark suite, apply the `@pytest.mark.benchmark` decorator to a test function.

16.8 Dependencies Policy

PyGMT has adopted SPEC 0 alongside the rest of the scientific Python ecosystem, and made a few extensions based on the needs of the project. Please see *Minimum Supported Versions* for the detailed policy and the minimum supported versions of GMT, Python and core package dependencies.

In `pyproject.toml`, the `requires-python` key should be set to the minimum supported version of Python. Minimum supported versions of GMT, Python and core package dependencies should be adjusted upward on every major and minor release, but never on a patch release.

16.9 Backwards Compatibility and Deprecation Policy

PyGMT is still undergoing rapid development. All of the API is subject to change until the 1.0.0 release. Versioning in PyGMT is based on the semantic versioning specification (i.e., *MAJOR.MINOR.PATCH*). Basic policy for backwards compatibility:

- Any incompatible changes should go through the deprecation process below.
- Incompatible changes are only allowed in major and minor releases, not in patch releases.

- Incompatible changes should be documented in the release notes.

When making incompatible changes, we should follow the process:

- Discuss whether the incompatible changes are necessary on GitHub.
- Make the changes in a backwards compatible way, and raise a `FutureWarning` warning for the old usage. At least one test using the old usage should be added.
- The warning message should clearly explain the changes and include the versions in which the old usage is deprecated and is expected to be removed.
- The `FutureWarning` warning should appear for 2-4 minor versions, depending on the impact of the changes. It means the deprecation period usually lasts 3-12 months.
- Remove the old usage and warning when reaching the declared version.

16.9.1 Deprecating a function parameter

To rename a function parameter, add the `@deprecate_parameter` decorator near the top after the `@fmt_docstring` decorator but before the `@use_alias` decorator (if those two exist). A `TODO` comment should also be added to indicate the deprecation period (see below). Here is an example:

```
# TODO(PyGMT>=0.6.0): Remove the deprecated "columns" parameter.
@fmt_docstring
@deprecate_parameter("columns", "incols", "0.4.0", remove_version="0.6.0")
@use_alias(J="projection", R="region", V="verbose", i="incols")
@kwargs_to_strings(R="sequence", i="sequence_comma")
def plot(self, x=None, y=None, data=None, size=None, direction=None, **kwargs):
    pass
```

In this case, the old parameter name `columns` is deprecated since 0.4.0, and will be fully removed in 0.6.0. The new parameter name is `incols`.

16.9.2 TODO comments

Occasionally, we need to implement temporary code that should be removed in the future. This can occur in situations such as:

- When a parameter, function, or method is deprecated and scheduled for removal.
- When workarounds are necessary to address issues in older or upcoming versions of GMT or other dependencies.

To track these temporary codes or workarounds, we use TODO comments. These comments should adhere to the following format:

```
# TODO(package>=X.Y.Z): A brief description of the TODO item.
# Additional details if necessary.
```

The TODO comment indicates that we should address the item when *package* version *X.Y.Z* or later is required.

It's important not to overuse TODO comments for tracking unimplemented features. Instead, open issues to monitor these features.

16.10 Making a Release

We try to automate the release process as much as possible. GitHub Actions workflow handles publishing new releases to PyPI and updating the documentation. The version number is set automatically using setuptools_scm based information obtained from git. There are a few steps that still must be done manually, though.

16.10.1 Updating the Changelog

The Release Drafter GitHub Action will automatically keep a draft changelog at https://github.com/GenericMappingTools/pygmt/releases, adding a new entry every time a pull request (with a proper label) is merged into the main branch. This release drafter tool has two configuration files, one for the GitHub Action at `.github/workflows/release-drafter.yml`, and one for the changelog template at `.github/release-drafter.yml`. Configuration settings can be found at https://github.com/release-drafter/release-drafter.

The drafted release notes are not perfect, so we will need to tidy it prior to publishing the actual release notes at *Changelog*.

1. Go to https://github.com/GenericMappingTools/pygmt/releases and click on the 'Edit' button next to the current draft release note. Copy the text of the automatically drafted release notes under the 'Write' tab to `doc/changes.md`. Add a section separator `---` between the new and old changelog sections.
2. Update the DOI badge in the changelog. Remember to replace the DOI number inside the badge url.

   ```
   [![Digital Object Identifier for PyGMT␣
   ↪vX.Y.Z](https://img.shields.io/badge/DOI-10.5281/zenodo.<INSERT-DOI-HERE>-blue)](https://doi.c
   ```

3. Open a new pull request using the title 'Changelog entry for vX.Y.Z' with the updated release notes, so that other people can help to review and collaborate on the changelog curation process described next.
4. Edit the change list to remove any trivial changes (updates to the README, typo fixes, CI configuration, test updates due to GMT releases, etc).
5. Sort the items within each section (i.e., New Features, Enhancements, etc.) such that similar items are located near each other (e.g., new wrapped modules and methods, gallery examples, API docs changes) and entries within each group are alphabetical.
6. Move a few important items from the main sections to the Highlights section.
7. Edit the list of people who contributed to the release, linking to their GitHub accounts. Sort their names by the number of contributions (including commits and PR reviews) made since the last release (e.g., use `git shortlog vX.Y.Z..HEAD -sne --group=author --group=trailer:co-authored-by`). Keep the contributors as a comma-separated list (one name per line with a trailing comma), rather than a bulleted list, to save vertical space.
8. Update `doc/minversions.md` with new information on the new release version, including a vX.Y.Z documentation link, and minimum required versions of GMT, Python and core package dependencies (NumPy, pandas, xarray). Follow SPEC 0 for updates.
9. Refresh citation information. Specifically, the BibTeX in `README.md` and `CITATION.cff` needs to be updated with any metadata changes, including the DOI, release date, and version information. Please also follow guidelines in `AUTHORSHIP.md` for updating the author list in the BibTeX. More information about the `CITATION.cff` specification can be found at https://github.com/citation-file-format/citation-file-format/blob/main/schema-guide.md.

16.10.2 Pushing to PyPI and Updating the Documentation

After the changelog is updated, making a release can be done by going to https://github.com/GenericMappingTools/pygmt/releases, editing the draft release, and clicking on publish. A git tag will also be created, make sure that this tag is a proper version number (following Semantic Versioning) with a leading `v` (e.g., `v0.2.1`).

Once the release/tag is created, this should trigger GitHub Actions to do all the work for us. A new source distribution will be uploaded to PyPI, a new folder with the documentation HTML will be pushed to *gh-pages*, and the `latest` link will be updated to point to this new folder.

16.10.3 Archiving on Zenodo

Grab both the source code and baseline images ZIP files from the GitHub release page and upload them to Zenodo using the previously reserved DOI.

16.10.4 Updating the Conda Package

When a new version is released on PyPI, conda-forge's bot automatically creates version updates for the feedstock. In most cases, the maintainers can simply merge that PR.

If changes need to be done manually, you can:

1. Fork the pygmt feedstock repository if you haven't already. If you have a fork, update it.
2. Update the version number and sha256 hash on `recipe/meta.yaml`. You can get the hash from the PyPI "Download files" section.
3. Add or remove any new dependencies (most are probably only `run` dependencies).
4. Make sure the minimum support versions of all dependencies are correctly pinned.
5. Make a new branch, commit, and push the changes **to your personal fork**.
6. Create a PR against the original feedstock main.
7. Once the CI tests pass, merge the PR or ask a maintainer to do so.

CHAPTER

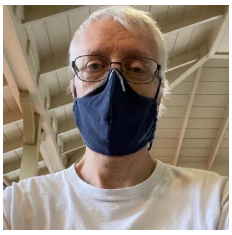
SEVENTEEN

PYGMT TEAM

We are an international team dedicated to building a Pythonic API for the Generic Mapping Tools (GMT). PyGMT is a community developed project. All are welcome! For more information about how to get involved, see the *Contributors Guide*. A more complete list of contributors and a definition of the term "PyGMT Developers" is available in the `AUTHORS.md` file in the source repository.

17.1 Founders

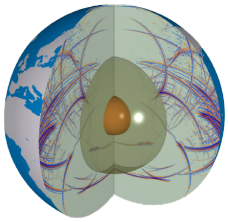
Leonardo Uieda
@leouieda



Paul Wessel
Paul Wessel

17.2 Active Maintainers

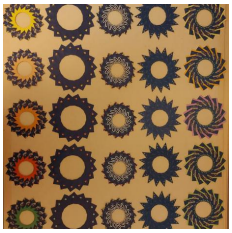
Active maintainers are recognized for their responsibilities in maintaining the project, as detailed in the Maintainers Guide. *New active maintainers are selected and voted by the currently active maintainers before each release.*



Dongdong Tian
@seisman

Wei Ji Leong
@weiji14

Michael Grund
@michaelgrund

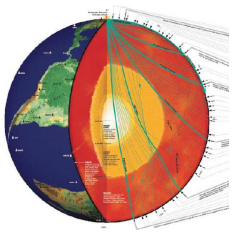


Yvonne Fröhlich
@yvonnefroehlich

17.3 Distinguished Contributors

Distinguished contributors are recognized for their substantial contributions to PyGMT, which may include code, documentation, pull request review, triaging, forum responses, community building and engagement, outreach, and inclusion and diversity. New distinguished contributors are selected and voted by the currently active maintainers before each release. Maintainers that are inactive for more than one year will also be moved to distinguished contributors.

Max Jones
@maxrjones



Jiayuan Yao
@core-man

Liam Toney
@liamtoney

Will Schlitzer
@willschlitzer